

Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет
информатики и радиоэлектроники»

Факультет информационных технологий и управления

Кафедра интеллектуальных информационных технологий

**МАТЕМАТИЧЕСКИЕ ОСНОВЫ ИНТЕЛЛЕКТУАЛЬНЫХ
СИСТЕМ**

Учебное пособие

(Черновик от 04.05.2026)

Минск БГУИР 2026

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ.....	2
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ.....	20
ВВЕДЕНИЕ.....	21
ТЕОРЕТИЧЕСКАЯ ЧАСТЬ.....	23
1 ОСНОВЫ ТЕОРИИ МНОЖЕСТВ В ИНТЕЛЛЕКТУАЛЬНЫХ СИСТЕМАХ.....	24
1.1 Понятие множества.....	24
1.1.1 Понятие элемента множества.....	25
1.1.2 Понятие мощности множества.....	25
1.1.3 Классификация множеств.....	26
1.1.3.1 Понятие конечного множества.....	26
1.1.3.2 Понятие бесконечного множества.....	26
1.1.3.3 Понятие пустого множества.....	26
1.1.4 Понятие подмножества множества.....	26
1.1.4.1 Понятие собственного подмножества множества.....	27
1.1.4.2 Понятие несобственного подмножества множества.....	27
1.1.5 Понятие надмножества множества.....	27
1.1.6 Операции над множествами.....	28
1.1.6.1 Понятие объединения множеств.....	28
1.1.6.2 Понятие пересечения множеств.....	28
1.1.6.3 Понятие разности двух множеств.....	29
1.1.6.4 Понятие симметрической разности двух множеств.....	30
1.1.6.5 Понятие булеана множества.....	30
1.1.6.6 Понятие разбиения множества.....	30
1.1.6.7 Понятие декомпозиции множества.....	31
1.2 Понятие связки.....	31
1.2.1 Понятие синглтона.....	31
1.2.2 Понятие пары.....	31
1.2.3 Понятие тройки.....	31
1.2.4 Понятие небинарной связки.....	31
1.2.5 Ориентированные и неориентированные связки.....	31
1.2.5.1 Понятие ориентированной связки.....	32
1.2.5.2 Понятие неориентированной связки.....	32
1.2.6 Понятие декартова произведения двух множеств.....	32
1.3 Понятие класса.....	33

1.3.1 Понятие понятия.....	33
1.3.1.1 Понятие абсолютного понятия.....	34
1.3.1.1 Понятие относительного понятия.....	34
1.4 Понятие отношения.....	34
1.4.1 Классификация отношений по типу ориентированности связок.....	35
1.4.1.1 Понятие ориентированного отношения.....	35
1.4.1.2 Понятие неориентированного отношения.....	35
1.4.2 Понятие арности отношения.....	35
1.4.3 Понятие области определения отношения.....	35
1.4.4 Понятие домена отношения.....	35
1.4.5 Классификация отношений по типу связей.....	37
1.4.5.1 Понятие ролевого отношения.....	37
1.4.5.2 Понятие неролевого отношения.....	37
1.4.6 Классификация отношений по признаку арности.....	37
1.4.6.1 Понятие унарного отношения.....	38
1.4.6.2 Понятие бинарного отношения.....	38
1.4.6.3 Понятие квазибинарного отношения.....	38
1.4.6.4 Понятие тернарного отношения.....	39
1.4.7 Классификация бинарных отношений.....	39
1.4.7.1 Понятие рефлексивного бинарного отношения.....	40
1.4.7.2 Понятие антирефлексивного бинарного отношения.....	40
1.4.7.3 Понятие симметричного бинарного отношения.....	40
1.4.7.4 Понятие асимметричного бинарного отношения.....	41
1.4.7.5 Понятие антисимметричного бинарного отношения.....	41
1.4.7.6 Понятие транзитивного бинарного отношения.....	41
1.4.7.7 Понятие антитранзитивного бинарного отношения.....	42
1.4.7.8 Понятие линейного бинарного отношения.....	42
1.4.7.9 Понятие отношения толерантности.....	43
1.4.7.10 Понятие отношения эквивалентности.....	43
1.4.7.11 Понятие транзитивного замыкания бинарного отношения.....	43
1.4.7.12 Понятие обратного отношения.....	44
1.4.8 Базовые отношения.....	44
1.4.8.1 Понятие отношения принадлежности.....	44
1.4.8.2 Понятие отношения включения.....	44
1.4.9 Понятие метаотношения.....	45
1.5 Понятие параметра.....	45

1.5.1 Понятие измеряемого параметра.....	45
1.5.2 Понятие неизмеряемого параметра.....	46
1.5.3 Понятие величины.....	47
1.5.3.1 Понятие точной величины.....	47
1.5.3.2 Понятие неточной величины.....	48
1.5.3.3 Понятие интервальной величины.....	48
1.6 Понятие структуры.....	48
1.6.1 Понятие тривиальной структуры.....	49
1.6.2 Понятие связной структуры.....	49
1.7 Понятие соответствия.....	49
1.7.1 Понятие области отправления соответствия.....	49
1.7.2 Понятие области прибытия соответствия.....	49
1.7.3 Понятие прообраза.....	49
1.7.4 Понятие прообраза.....	49
1.7.5 Классификация соответствий.....	50
1.7.5.1 Понятие однозначного соответствия.....	50
1.7.5.1.1 Понятие инъективного соответствия.....	50
1.7.5.1.2 Понятие обратимого соответствия.....	51
1.7.5.2 Понятие неоднозначного соответствия.....	51
1.7.5.3 Понятие сюръективного соответствия.....	51
1.7.5.4 Понятие несюръективного соответствия.....	51
1.7.5.5 Понятие всюду определённого соответствия.....	51
1.7.5.6 Понятие частично определённого соответствия.....	52
1.7.5.7 Понятие взаимно однозначного соответствия.....	52
1.7.5.8 Понятие обратного соответствия.....	52
1.7.5.9 Понятие функции.....	52
1.7.6 Специализированные виды соответствий.....	53
1.7.6.2 Понятие гомоморфности структур.....	53
1.7.6.3 Понятие изоморфности структур.....	53
1.7.6.4 Понятие автоморфности структур.....	54
1.7.6.5 Понятие полиморфности структур.....	54
2 ОСНОВЫ ОБЩЕЙ АЛГЕБРЫ В ИНТЕЛЛЕКТУАЛЬНЫХ СИСТЕМАХ.....	55
2.1 Понятие алгебры.....	55
2.1.1 Понятие носителя алгебры.....	55
2.1.2 Понятие сигнатуры алгебры.....	56
2.2 Понятие алгебраической операции.....	56

2.2.1 Понятие аргумента алгебраической операции.....	56
2.2.1.1 Понятие типа алгебры.....	56
2.2.2 Понятие аргумента алгебраической операции.....	56
2.2.3 Классификация алгебраических операций.....	57
2.2.3.1 Понятие унарной алгебраической операции.....	57
2.2.3.2 Понятие бинарной алгебраической операции.....	57
2.2.3.2.1 Классификация бинарных алгебраических операций.....	57
2.2.3.2.1.1 Понятие ассоциативной бинарной алгебраической операции.....	57
2.2.3.2.1.2 Понятие коммутативной бинарной алгебраической операции.....	58
2.2.3.2.1.3 Понятие дистрибутивной бинарной алгебраической операции по отношению к заданной бинарной операции.....	58
2.2.3.2.1.4 Понятие идемпотентной бинарной алгебраической операции.....	58
2.2.3.3 Понятие небинарной алгебраической операции.....	58
2.3 Понятие группоида.....	59
2.4 Понятие полгруппы.....	59
2.5 Понятие моноида.....	59
2.6 Понятие группы.....	60
2.7 Понятие кольца.....	60
2.8 Понятие поля.....	61
2.9 Понятие алгебраической системы.....	61
2.9.1 Понятие носителя алгебраической системы.....	61
2.9.2 Понятие сигнатуры алгебраической системы.....	62
2.10 Частично упорядоченные множества.....	62
2.11 Понятие решётки.....	62
2.12 Гомоморфизм и изоморфизм алгебр.....	63
2.12.1 Понятие гомоморфизма алгебр.....	63
2.12.2 Понятие изоморфизма алгебр.....	63
2.13 Реляционная алгебра.....	64
3 ОСНОВЫ ТЕОРИИ ФОРМАЛЬНЫХ ЯЗЫКОВ В ИНТЕЛЛЕКТУАЛЬНЫХ СИСТЕМАХ.....	64
3.1 Понятие языка.....	65
3.1.1 Понятие искусственного языка.....	65
3.2 Понятие знака.....	65
3.2.1 Понятие денотата знака.....	65
3.2.2 Понятие сигнификата знака.....	66
3.3 Понятие знаковой конструкции.....	66

3.3.1.1 Понятие синтактики.....	66
3.3.1.2 Понятие семантики.....	66
3.3.1.3 Понятие прагматики.....	66
3.4 Понятие формального языка.....	66
3.4.1 Понятие алфавита формального языка.....	67
3.4.2 Понятие синтаксиса формального языка.....	67
3.4.3 Понятие грамматики формального языка.....	67
3.4.3.1 Понятие формы Бэкуса-Наура.....	67
3.4.4 Понятие семантики формального языка.....	67
3.4.5 Понятие денотационной семантики формального языка.....	67
3.4.6 Понятие операционной семантики формального языка.....	68
4 ОСНОВЫ МАТЕМАТИЧЕСКОЙ ЛОГИКИ В ИНТЕЛЛЕКТУАЛЬНЫХ СИСТЕМАХ.....	69
4.1 Логика высказываний.....	69
4.1.1 Язык логики высказываний.....	70
4.1.2 Понятие высказывания.....	70
4.1.3 Алфавит языка логики высказываний.....	70
4.1.4 Грамматика языка логики высказываний.....	71
4.1.5 Классификация логических формул в языке логики высказываний.....	72
4.1.5.1 Понятие атомарной логической формулы.....	72
4.1.5.2 Понятие неатомарной логической формулы.....	72
4.1.6 Понятие логической связки.....	72
4.1.7 Понятие подформулы логической формулы.....	73
4.2 Алгебра высказываний.....	73
4.2.1 Понятие интерпретации логической формулы.....	73
4.2.2 Понятие истинностной функции.....	74
4.2.3 Понятие унарной логической связки и понятие бинарной логической связки.....	74
4.2.3.1 Понятие унарной логической связки.....	74
4.2.3.1.1 Понятие отрицания.....	75
4.2.3.2 Понятие бинарной логической связки.....	75
4.2.3.2.1 Понятие дизъюнкции.....	75
4.2.3.2.2 Понятие конъюнкции.....	75
4.2.3.2.3 Понятие импликации.....	76
4.2.3.2.4 Понятие эквиваленции.....	76
4.2.4 Классификация логических формул.....	76

4.2.4.1 Понятие общезначимой логической формулы.....	76
4.2.4.2 Понятие необщезначимой логической формулы.....	77
4.2.4.2.1 Понятие невыполнимой логической формулы.....	77
4.2.4.2.2 Понятие нейтральной логической формулы.....	77
4.2.4.3 Понятие выполнимой логической формулы.....	77
4.2.4.4 Примеры построения таблиц истинности.....	77
4.2.5 Понятие следования логической формулы из другой логической формулы.....	80
4.2.6 Понятие равносильности двух логических формул.....	81
4.2.6.1 Примеры отношений равносильности логических формул.....	81
4.2.6.1.1 Законы ассоциативности.....	81
4.2.6.1.2 Законы коммутативности.....	82
4.2.6.1.3 Законы дистрибутивности.....	82
4.2.6.1.4 Законы идемпотентности.....	82
4.2.6.1.5 Закон двойного отрицания.....	82
4.2.6.1.6 Законы де Моргана.....	82
4.2.6.1.7 Законы логических констант.....	83
4.2.6.1.8 Закон тождества.....	83
4.2.6.1.9 Закон противоречия.....	83
4.2.6.1.10 Закон исключённого третьего.....	83
4.2.6.1.11 Закон выражения связки импликации.....	83
4.2.6.1.12 Закон выражения связки эквиваленции.....	84
4.2.6.1.13 Законы поглощения.....	84
4.2.6.1.14 Законы склеивания.....	84
4.2.6.1.15 Закон экспортации-импортации.....	84
4.2.6.1.15 Закон распределения импликации относительно дизъюнкции.....	84
4.2.6.1.16 Закон распределения импликации относительно конъюнкции.....	85
4.2.6.1.17 Закон контрапозиции.....	85
4.2.6.1.18 Законы материальной импликации.....	85
4.2.6.1.19 Закон транзитивности импликации.....	85
4.2.6.1.20 Закон обобщённой контрапозиции.....	86
4.2.6.1.21 Закон доминирования конъюнкции.....	86
4.2.6.1.22 Закон доминирования дизъюнкции.....	86
4.2.6.2 Понятие равносильного преобразования логической формулы.....	86
4.2.6.2.1 Упрощение сложных формул.....	87
4.2.6.2.2 Доказательство тавтологий.....	87

4.2.6.2.3	Понятие нормальной формы.....	88
4.2.6.2.3.1	Понятие конъюнктивной нормальной формы логической формулы	88
4.2.6.2.3.2	Понятие дизъюнктивной нормальной формы логической формулы.	89
4.2.6.2.3.3	Понятие совершенной конъюнктивной нормальной формы.....	90
4.2.6.2.3.4	Понятие совершенной дизъюнктивной нормальной формы.....	92
4.3	Исчисление высказываний.....	94
4.3.1	Понятие аксиомы языка логики высказываний.....	94
4.3.1.1	Аксиомные схемы исчисления высказываний.....	95
4.3.1.1.1	Аксиомная схема введения импликации.....	95
4.3.1.1.2	Аксиомная схема удаления импликации.....	95
4.3.1.1.3	Аксиомная схема введения конъюнкции.....	96
4.3.1.1.4	Аксиомная схема удаления конъюнкции.....	96
4.3.1.1.5	Аксиомная схема введения дизъюнкции.....	96
4.3.1.1.6	Аксиомная схема удаления дизъюнкции.....	96
4.3.1.1.7	Аксиомная схема введения отрицания.....	96
4.3.1.1.8	Аксиомная схема удаления отрицания.....	96
4.3.1.1.8	Аксиомная схема введения эквиваленции.....	97
4.3.1.1.9	Аксиомные схемы удаления эквиваленции.....	97
4.3.2	Понятие формального логического вывода.....	97
4.3.2.1	Понятие теоремы языка логики высказываний.....	97
4.3.2.2	Понятие отношения выводимости.....	98
4.3.2.3	Правила вывода.....	98
4.3.2.3.1	Правило прямого заключения (Modus Ponens).....	98
4.3.2.4	Метатеоремы исчисления высказываний.....	99
4.3.2.4.1	Метатеорема дедукции.....	99
4.3.2.4.2	Метатеорема, обратная метатеореме дедукции.....	100
4.3.2.4.2.1	Понятие квазивывода.....	100
4.3.2.4.2.2	Правило перевода квазивывода в формальный логический вывод...	101
4.3.2.4.2.2.1	Теорема об оценке длины формального вывода.....	102
4.3.2.4.3	Метатеорема о полноте.....	104
4.3.2.4.4	Метатеорема, обратная метатеореме о полноте (метатеорема об адекватности).....	104
4.3.2.4.5	Метатеорема о компактности.....	104
4.3.2.5	Правило резолюций.....	105

4.3.3.5.1 Понятие резольвенты.....	105
4.4 Логика предикатов.....	106
4.4.1 Язык логики предикатов.....	106
4.4.2 Алфавит языка логики предикатов.....	106
4.4.3 Грамматика языка логики предикатов.....	108
4.4.4 Классификация логических формул языка логики предикатов.....	108
4.4.4.1 Понятие атомарной логической формулы.....	109
4.4.4.2 Понятие атомарной предикатной формулы.....	109
4.4.4.3 Понятие неатомарной логической формулы.....	109
4.4.5 Классификация предметных переменных.....	109
4.4.5.1 Понятие связанной предметной переменной логической формулы..	109
4.4.5.2 Понятие свободной предметной переменной логической формулы.	110
4.4.5.3 Понятие замкнутой логической формулы.....	110
4.4.5.4 Понятие открытой логической формулы.....	110
4.5 Алгебра предикатов.....	110
4.5.1 Понятие предиката.....	110
4.5.2 Понятие модели в алгебре предикатов.....	111
4.5.3 Семантика логических связей.....	111
4.5.4 Семантика кванторов.....	112
4.5.5 Понятие равносильности двух логических формул.....	112
4.5.5.1 Примеры отношений равносильности логических формул.....	112
4.5.5.1.1 Законы коммутативности кванторов.....	112
4.5.5.1.2 Законы дистрибутивности кванторов.....	112
4.5.5.1.3 Законы выноса констант.....	113
4.5.5.1.4 Законы двойственности кванторов.....	113
4.5.5.2 Понятие равносильного преобразования логической формулы.....	113
4.5.5.2.1 Понятие предварённой нормальной формы логической формулы.	113
4.5.5.2.2 Понятие сколемовской стандартной формы логической формулы.	114
4.6 Исчисление предикатов.....	114
4.6.1 Понятие аксиомы языка логики предикатов.....	115
4.6.2 Аксиомные схемы исчисления предикатов.....	115
4.6.2.1 Пропозициональные аксиомные схемы.....	115
4.6.2.2 Аксиомные схемы кванторов.....	115
4.6.2.2.1 Аксиомная схема удаления квантора всеобщности.....	115
4.6.2.2.2 Аксиомная схема введения квантора всеобщности.....	116
4.6.3 Понятие формального логического вывода в исчислении предикатов.	116

4.6.4 Понятие теоремы языка логики предикатов.....	116
4.6.5 Понятие отношения выводимости в логике предикатов.....	116
4.6.6 Правила вывода в исчислении предикатов.....	117
4.6.6.1 Правило Modus Ponens.....	117
4.6.6.2 Правило введения квантора всеобщности.....	117
4.6.6.3 Правило удаления квантора существования.....	117
4.6.7 Метатеоремы исчисления предикатов.....	118
4.6.8 Правило резолюции.....	118
4.6.9 Понятие унификации.....	118
4.7 Формализация высказываний на языке логики предикатов.....	118
4.7.1 Основные компоненты формализации.....	118
4.7.2 Алгоритм формализации.....	119
4.7.3 Формализация атомарных высказываний.....	119
4.7.4 Формализация составных высказываний.....	120
4.7.4.1 Формализация отрицания.....	120
4.7.4.2 Формализация конъюнкции.....	120
4.7.4.3 Формализация дизъюнкции.....	120
4.7.4.4 Формализация импликации.....	121
4.7.4.5 Формализация эквиваленции.....	121
4.7.5 Формализация высказываний с кванторами.....	121
4.7.5.1 Формализация квантора всеобщности.....	121
4.7.5.2 Формализация квантора существования.....	122
4.7.5.3 Формализация вложенных кванторов.....	122
4.7.6 Формализация специальных конструкций естественного языка.....	123
4.7.6.1 «Единственный» (существует ровно один).....	123
4.7.6.2 «Не менее n», «не более n», «ровно n» объектов.....	123
4.7.6.3 «Большинство», «почти все» — пределы формализации.....	124
4.7.7 Типичные ошибки формализации.....	124
4.7.8 Примеры формализации.....	124
4.7.8.1 Пример. Формализация аксиом теории графов.....	124
4.7.8.2 Пример. Формализация условий доступа в системе безопасности..	125
4.8 Логическое программирование.....	125
5 ОСНОВЫ ОБЩЕЙ ТЕОРИИ ИНТЕЛЛЕКТУАЛЬНЫХ СИСТЕМ.....	126
5.1 Понятие интеллектуальной системы.....	126
5.2 Понятие интеллектуальной задачи.....	126
5.3 Понятие знания.....	126

5.3.1 Классификация знаний.....	127
5.3.1.1 Понятие декларативного знания.....	127
5.3.1.2 Понятие процедурного знания.....	127
5.3.2 Понятие метазнания.....	128
5.4 Понятие представления знаний.....	128
5.4.2 Классификация моделей представления знаний.....	129
5.4.3 Понятие формализации знаний.....	129
5.4.3.1 Понятие понимания.....	129
5.4.3.2 Понятие смысла знания.....	129
5.4.3.3 Понятие смысла.....	130
5.4.4 Понятие языка представления знаний.....	130
5.4.4.1 Понятие внутреннего языка компьютерной системы.....	130
5.4.5 Понятие базы знаний интеллектуальной системы.....	130
5.4.5.1 Понятие семантической окрестности сущности.....	131
5.4.5.2 Понятие предметной области.....	131
5.4.5.3 Понятие исследуемого понятия предметной области.....	131
5.4.5.3.1 Понятие максимального класса объектов исследования предметной области.....	132
5.4.5.3.2 Понятие немаксимального класса объектов исследования предметной области.....	132
5.4.5.3.3 Понятие исследуемого отношения предметной области.....	132
5.4.5.3.4 Понятие класса исследуемых структур предметной области.....	132
5.4.5.3.5 Понятие онтологии предметной области.....	132
5.4.5.3.5.1 Классификация онтологий.....	132
5.4.5.3.5.1.1 Понятие терминологической онтологии предметной области..	133
5.4.5.3.5.1.2 Понятие теоретико-множественной онтологии предметной области.....	133
5.4.5.3.5.1.3 Понятие логической онтологии предметной области.....	133
5.4.5.3.5.1.4 Понятие структурной спецификации (онтологии) предметной области.....	134
5.4.5.3.5.1.5 Понятие интегрированной онтологии предметной области.....	134
5.4.5.3.6 Понятие частной предметной области.....	134
5.7 Понятие обработки знаний.....	134
5.7.1 Понятие процесса.....	134
5.7.2 Понятие ситуации.....	135
5.7.2.1 Понятие начальной ситуации процесса.....	135
5.7.2.2 Понятие конечной ситуации процесса.....	135

5.7.3 Понятие воздействия.....	135
5.7.3.1 Понятие субъекта воздействия.....	135
5.7.3.2 Понятие объекта воздействия.....	135
5.7.4 Понятие действия.....	135
5.7.4.1 Понятие целевой ситуации действия.....	136
5.7.4.2 Понятие атомарного действия и понятие неатомарного действия....	136
5.7.4.2.1 Понятие атомарного действия.....	136
5.7.4.2.2 Понятие неатомарного действия.....	136
5.7.5 Понятие задачи.....	136
5.7.5.1 Понятие декларативной формулировки задачи.....	136
5.7.5.2 Понятие процедурной формулировки задачи.....	136
5.7.6 Понятие решения задачи.....	137
5.7.7 Понятие класса задач.....	137
5.7.8 Понятие класса действий.....	137
5.7.9 Понятие метода.....	137
5.7.9.1 Понятие класса методов.....	137
5.7.10 Понятие языка представления методов.....	138
5.7.10.1 Понятие денотационной семантики языка представления методов	138
5.7.10.2 Понятие операционной семантики языка представления методов..	138
5.7.11 Понятие модели решения задач.....	139
5.7.12 Понятие навыка.....	139
5.7.13 Понятие решателя задач интеллектуальной системы.....	139
5.7.1 Понятие языка обмена сообщениями компьютерной системы.....	140
5.7.2 Понятие внешнего языка компьютерной системы.....	140
6 ОСНОВЫ ТЕХНОЛОГИИ OSTIS.....	141
6.1 Понятие Технологии OSTIS.....	141
6.2 Понятие ostis-системы.....	141
6.3 Понятие SC-кода.....	141
6.3.1 Понятие sc-элемента.....	142
6.3.1.1 Структурная классификация sc-элементов (Алфавит SC-кода).....	142
6.3.1.1.1 Понятие sc-константы и понятие sc-переменной.....	144
6.3.1.1.2 Понятие sc-множества.....	144
6.3.1.1.2.1 Понятие sc-связки.....	144
6.3.1.1.2.2 Понятие sc-пары принадлежности.....	145
6.3.1.1.2.3 Понятие sc-класса.....	146
6.3.1.1.2.4 Понятие sc-структуры.....	146

6.3.1.1.2.5 Понятие внешней сущности.....	147
6.3.2 Синтаксис SC-кода.....	147
6.3.2.1 Синтаксическая классификация sc-элементов.....	148
6.3.3 Понятие sc-идентификатора.....	151
6.3.3.1 Понятие простого sc-идентификатора.....	151
6.3.3.2 Понятие sc-выражения.....	152
6.3.3.3 Понятие основного sc-идентификатора.....	152
6.3.3.4 Понятие строкового sc-идентификатора.....	152
6.3.3.5 Понятие нестрокового sc-идентификатора.....	152
6.3.3.6 Основные правила построения простых строковых sc-идентификаторов некоторых классов сущностей на SC-коде.....	152
6.3.3.7 Понятие системного sc-идентификатора.....	154
6.3.3.8 Основные правила построения системных sc-идентификаторов некоторых классов сущностей на SC-коде.....	154
6.3.4 Формы внешнего представления SC-кода: SCg-код и SCs-код.....	155
6.3.4.1 Понятие SCg-кода.....	156
6.3.4.1.1 Понятие sc.g-узла.....	156
6.3.4.1.2 Понятие sc.g-коннектора.....	158
6.3.4.2 Понятие SCs-кода.....	159
6.3.4.2.1 Понятие sc-идентификатора в SCs-коде.....	160
6.3.4.2.1.1 Понятие нетранслируемого системного sc-идентификатора.....	160
6.3.4.2.1.1.1 Понятие локального системного sc-идентификатора.....	160
6.3.4.2.1.1.2 Понятие глобального системного sc-идентификатора.....	161
6.3.4.2.2 Понятие sc.s-разделителя.....	161
6.3.4.2.2.1 Понятие sc.s-коннектора.....	161
6.3.4.2.3 Понятие sc.s-ограничителя.....	163
6.3.4.2.4 Понятие sc.s-предложения.....	163
6.3.4.2.4.1 Понятие простого sc.s-предложения.....	165
6.3.4.2.4.2 Понятие сложного sc.s-предложения.....	166
6.3.4.2.4.3 Понятие присоединённого sc.s-предложения.....	166
6.3.4.2.5 Системные sc-идентификаторы некоторых классов sc-элементов на SCs-коде.....	167
6.3.4.3 Синтаксические и семантические характеристики sc-элементов.....	167
6.3.4.4 Примеры формализации знаний на SCg-коде и SCs-коде.....	168
6.3.4.5 Правила перевода формализации знаний из SCg-кода на SCs-код....	170
6.3.4.6 Примеры формализации различных видов знаний на SC-коде.....	173
6.3.4.7 Принципы структуризации знаний в ostis-системах.....	184

6.3.4.7.1 Понятие семантической sc-окрестности сущности.....	184
6.3.4.7.1.2 Структура базовой семантической sc-окрестности sc-класса.....	184
6.3.4.7.1.3 Структура базовой семантической sc-окрестности sc-отношения....	186
6.3.4.7.4 Структура полной семантической sc-окрестности sc-класса.....	189
6.3.4.7.5 Структура полной семантической sc-окрестности sc-отношения..	189
6.3.4.7.2 Понятие предметной sc-области.....	190
6.5.2.1 Понятия, используемые для описания предметных sc-областей.....	190
6.5.3 Формализация иерархии предметных областей.....	191
6.5.5 Пример формализации предметной области, её структурной спецификации и теоретико-множественной онтологии.....	192
6.6 Принципы обработки знаний при помощи Технологии OSTIS.....	201
6.6.1 Понятие операции над sc-конструкцией.....	201
6.6.2 Виды операций над sc-конструкциями.....	201
6.6.3 Виды sc-конструкций.....	202
6.6.3.1 Понятие sc-конструкции стандартного вида.....	202
6.6.3.1.1 Понятие трёхэлементной sc-конструкция.....	203
6.6.3.1.2 Понятие пятиэлементной sc-конструкция.....	203
6.6.3.2 Понятие sc-конструкции нестандартного вида.....	204
6.6.4 Понятие операции поиска sc-конструкции нестандартного вида.....	205
6.6.5 Понятие обобщённой sc-структуры.....	206
6.6.5.1 Понятие sc-переменной.....	206
6.6.5.2 Понятие sc-шаблона.....	206
6.6.5.2.1 Примеры sc-шаблонов и sc-конструкций, изоморфных им, представленных на SCg-коде.....	208
6.6.5.2.2 Недостатки sc-шаблонов и способы устранения этих недостатков	212
6.6.5.3 Понятие sc-шаблона с фиксированной стратегией поиска.....	213
6.6.5.4 Принципы выбора стратегии поиска по sc-шаблону и её спецификации в рамках этого sc-шаблона.....	213
6.6.6 Понятие Языка SCP.....	219
6.6.6.1 Понятие scp-программы.....	219
6.6.6.2 Понятие scp-процесса.....	219
6.6.6.3 Пример выполнения scp-программы.....	220
6.6.6.4 Понятие scp-оператора.....	227
6.6.6.5 Понятие scp-операнда.....	227
6.6.6.5.1 Понятие scp-константы и понятие scp-переменной.....	227
6.6.6.5.1.1 Понятие scp-константы.....	228

6.6.6.5.1.2 Понятие scr-переменной.....	228
6.6.6.5.2 Понятие scr-операнда с заданным значением и понятие scr-операнда со свободным значением.....	228
6.6.6.5.2.1 Понятие scr-операнда с заданным значением.....	228
6.6.6.5.2.2 Понятие scr-операнда со свободным значением'.....	229
6.6.6.5.3 Понятие типа sc-элемента.....	230
6.6.6.5.3.1 Классификация типов sc-элементов.....	230
6.6.6.6 Понятие начального scr-оператора.....	232
6.6.6.6.1 Классификация scr-операторов.....	232
6.6.6.6.1.1 scr-операторы генерации sc-конструкций.....	232
6.6.6.6.1.1.1 scr-оператор генерации одноэлементной sc-конструкции.....	233
6.6.6.6.1.1.2 scr-оператор генерации трехэлементной sc-конструкции.....	234
6.6.6.6.1.1.3 scr-оператор генерации пятиэлементной sc-конструкции.....	236
6.6.6.6.1.1.4 scr-оператор генерации sc-конструкции по произвольному образцу.....	241
6.6.6.6.1.2 scr-операторы поиска sc-конструкций.....	247
6.6.6.6.1.2.1 scr-оператор поиска трехэлементной sc-конструкции.....	248
6.6.6.6.1.2.2 scr-оператор поиска пятиэлементной sc-конструкции.....	250
6.6.6.6.1.2.3 scr-оператор поиска трехэлементной sc-конструкции с формированием множества.....	254
6.6.6.6.1.2.4 scr-оператор поиска пятиэлементной sc-конструкции с формированием множества.....	256
6.6.6.6.1.2.5 scr-оператор поиска sc-конструкции по произвольному образцу.. 257	
6.6.6.6.1.3 scr-операторы удаления sc-конструкций.....	262
6.6.6.6.1.3.1 scr-оператор удаления одноэлементной sc-конструкции.....	262
6.6.6.6.1.3.2 scr-оператор удаления трёхэлементной sc-конструкции.....	265
6.6.6.6.1.3.3 scr-оператор удаления пятиэлементной sc-конструкции.....	269
6.6.6.6.1.3.4 scr-оператор удаления множества sc-элементов трёхэлементной sc-конструкции.....	271
6.6.6.6.1.4 scr-операторы проверки условий.....	276
6.6.6.6.1.4.1 scr-оператор проверки типа sc-элемента.....	276
6.6.6.6.1.4.2 scr-оператор проверки наличия значения у sc-переменной.....	279
6.6.6.6.1.4.3 scr-оператор проверки наличия содержимого у файла.....	281
6.6.6.6.1.4.4 scr-оператор проверки совпадения значений scr-операндов....	284
6.6.6.6.1.4.5 scr-оператор проверки равенства числовых содержимых файлов. 286	

6.6.6.6.1.4.6 scp-оператор сравнения числовых содержимых файлов.....	290
6.6.6.6.1.5 scp-операторы обработки содержимых файлов.....	292
6.6.6.6.1.5.1 scp-оператор сложения числовых содержимых файлов.....	294
6.6.6.6.1.5.2 scp-оператор вычитания числовых содержимых файлов.....	295
6.6.6.6.1.5.3 scp-оператор умножения числовых содержимых файлов.....	297
6.6.6.6.1.5.4 scp-оператор деления числовых содержимых файлов.....	299
6.6.6.6.1.5.5 scp-оператор возведения числового содержимого файла.....	300
6.6.6.6.1.5.6 scp-оператор вычисления логарифма числового содержимого файла.....	302
6.6.6.6.1.5.7 scp-оператор вычисления синуса числового содержимого файла..	304
6.6.6.6.1.5.8 scp-оператор вычисления косинуса числового содержимого файла.....	305
6.6.6.6.1.5.9 scp-оператор вычисления тангенса числового содержимого файла.....	306
6.6.6.6.1.5.10 scp-оператор вычисления арксинуса числового содержимого файла.....	308
6.6.6.6.1.5.11 scp-оператор вычисления арккосинуса числового содержимого файла.....	309
6.6.6.6.1.5.12 scp-оператор вычисления арктангенса числового содержимого файла.....	311
6.6.6.6.1.5.13 scp-оператор копирования содержимого файла.....	312
6.6.6.6.1.5.14 scp-оператор удаления содержимого файла.....	314
6.6.6.6.1.5.15 scp-оператор перевода в нижний регистр строкового содержимого файла.....	316
6.6.6.6.1.5.16 scp-оператор перевода в верхний регистр строкового содержимого файла.....	318
6.6.6.6.1.5.17 scp-оператор замены определённой части строкового содержимого файла на содержимое указанного файла.....	319
6.6.6.6.1.5.18 scp-оператор проверки совпадения конца строкового содержимого файла со строковым содержимым другого файла.....	321
6.6.6.6.1.5.19 scp-оператор проверки совпадения начала строкового содержимого файла со строковым содержимым другого файла.....	323
6.6.6.6.1.5.20. scp-оператор получения части строкового содержимого файла по индексам.....	324
6.6.6.6.1.5.21 scp-оператор поиска строкового содержимого файла по индексам в строковом содержимом другого файла.....	326
6.6.6.6.1.5.22 scp-оператор вычисления длины строкового содержимого	

файла.....	328
6.6.6.6.1.5.23 scr-оператор разбиения строки на подстроки.....	330
6.6.6.6.1.5.24 scr-оператор лексикографического сравнения строковых содержимых файлов.....	331
6.6.6.6.1.5.25 scr-оператор проверки равенства строковых содержимых файлов.....	333
6.6.6.6.1.6 scr-операторы управления событиями.....	335
6.6.6.6.1.6.1 scr-оператор ожидания события.....	335
6.6.6.6.1.7 scr-операторы управления scr-процессами.....	336
6.6.6.6.1.7.1 scr-оператор асинхронного вызова подпрограммы.....	336
6.6.6.6.1.7.2 scr-оператор ожидания выполнения scr-программы.....	338
6.6.6.6.1.7.3 scr-оператор ожидания выполнения множества scr-процессов.....	339
6.6.6.6.1.7.4 scr-оператор завершения выполнения scr-программы.....	341
6.6.6.6.1.8 scr-операторы управления значениями scr-операндов.....	342
6.6.6.6.1.8.1 scr-оператор присваивания значения scr-переменной.....	342
6.6.6.6.1.8.2 scr-оператор удаления значения scr-переменной.....	344
6.6.6.6.2 Системные sc-идентификаторы классов scr-операторов.....	347
6.7 Принципы визуализации знаний при помощи Технологии OSTIS.....	352
6.7.1 Принципы навигации по базе знаний ostis-системы.....	352
6.7.1.1 Принципы навигации по базе знаний ostis-системы при помощи SCn-кода.....	352
6.7.1.1.1 Навигация по базе знаний ostis-системы на SCn-коде посредством щелчка левой кнопки компьютерной мыши по гиперссылкам sc-идентификаторов сущностей.....	352
6.7.1.1.2 Навигация по базе знаний ostis-системы на SCn-коде посредством щелчка левой кнопки компьютерной мыши по элементам истории навигации по базе знаний ostis-системы.....	353
6.7.1.1.3 Навигация по базе знаний ostis-системы на SCn-коде посредством задания вопросов к сущностям, обозначаемым гиперссылками соответствующих им sc-идентификаторов.....	354
6.7.1.2 Принципы навигации по базе знаний ostis-системы при помощи SCg-кода.....	356
6.7.1.2.1 Навигация по базе знаний ostis-системы на SCg-коде посредством двойного щелчка левой кнопки компьютерной мыши по sc.g-элементам....	356
6.7.1.2.2 Навигация по базе знаний ostis-системы на SCg-коде посредством задания вопросов к сущностям, обозначаемым sc.g-элементами.....	358
6.7.1.3 Навигация по базе знаний ostis-системы на SCg-коде или SCn-коде	

посредством поиска сущности по её sc-идентификатору.....	359
6.8 Принципы решения задач в ostis-системах.....	362
6.8.1 Агентно-ориентированная архитектура ostis-систем.....	362
6.8.2 Понятие агента.....	362
6.8.2.1 Понятие платформенно-независимого агента и понятие платформенно-зависимого агента.....	362
6.8.3 Понятие события в sc-памяти.....	363
6.8.4 Понятие абстрактного sc-агента.....	363
6.8.4.1 Понятие неатомарного абстрактного sc-агента и понятие атомарного абстрактного sc-агента.....	364
6.8.5 Понятие спецификации абстрактного sc-агента.....	364
6.8.5.1 Понятие первичного условия инициирования sc-агента.....	367
6.8.5.2 Понятие класса действия sc-агента.....	367
6.8.5.3 Понятие условия инициирования и результата sc-агента.....	367
6.8.5.4 Понятие ключевых sc-элементов sc-агента.....	368
6.8.5.5 Понятие программы sc-агента.....	368
6.8.5.6 Понятие агентной scr-программы.....	368
6.8.6 Понятие машины обработки знаний ostis-системы.....	368
6.9 Принципы разработки ostis-систем.....	369
6.9.1 Проект примера ostis-системы.....	369
6.9.1.1 Компоненты проекта примера ostis-системы.....	369
6.9.2 Принципы разработки базы знаний ostis-системы.....	369
6.9.2.1 Понятие фрагмента базы знаний.....	369
6.9.2.2 Понятие исходного файла базы знаний.....	370
6.9.2.3 Принципы работы с инструментами разработки исходных текстов баз знаний.....	370
6.9.2.3.1 Принципы работы с редактором КВЕ.....	370
6.9.2.3.1.1 Создание sc.g-узла заданного типа.....	370
6.9.2.3.1.2 Создание sc.g-коннектора заданного типа.....	376
6.9.2.3.1.3 Создание sc.g-шины.....	381
6.9.2.3.1.4 Создание sc.g-контура.....	384
6.9.2.3.1.5 Выравнивание sc.g-конструкций.....	388
6.9.2.3.1.6 Выделение множества sc.g-элементов.....	392
6.9.2.3.1.7 Экспорт изображения sc.g-графа.....	394
6.9.2.3.1.8 Регулирование масштаба холста.....	397
6.9.2.4 Пример формализации фрагментов базы знаний в примере ostis-системы.....	397

6.9.2.5 Принципы тестирования базы знаний ostis-системы.....	402
6.9.2.5.1 Сборка и пересборка базы знаний ostis-системы.....	403
6.9.2.5.1.1 Понятие бинарного файла базы знаний.....	403
6.9.2.5.1.2 Понятие sc-сборщика базы знаний ostis-системы.....	403
6.9.2.5.1.3 Понятие Программной платформы ostis-систем.....	403
6.9.2.5.1.4 Принципы использования sc-сборщика базы знаний ostis-системы.	404
6.9.2.5.2 Запуск ostis-системы.....	406
6.9.2.5.2.1 Понятие sc-машины.....	407
6.9.2.5.2.2 Понятие sc-веба.....	407
6.9.2.5.2.3 Понятие localhost:8000.....	407
6.9.2.5.3 Примеры навигации по формализованным фрагментам базы знаний в примере ostis-системы.....	408
6.9.2.5.4 Типичные ошибки, выявляемые в процессе навигации по фрагментам базы знаний ostis-системы.....	410
6.9.2.5.5 Принципы тестирования sc-шаблонов с фиксированной стратегией поиска.....	410
6.9.3 Принципы разработки решателя задач ostis-системы.....	421
6.9.3.1 Принципы разработки scr-программ для заданной предметной области.....	421
6.9.3.1.1 Пример scr-программы для заданной предметной области.....	422
6.9.3.1.2 Пример тестовой scr-программы для scr-программы заданной предметной области.....	428
6.9.3.2.3 Принципы тестирования scr-программ для заданной предметной области.....	429
6.9.3.2 Принципы разработки sc-агентов.....	437
6.9.3.2.1 Пример агентной scr-программы.....	437
6.9.3.2.2 Пример спецификации абстрактного sc-агента.....	440
6.9.3.2.3 Пример компонента пользовательского интерфейса для вызова sc-агента.....	441
6.9.3.2.4 Добавление компонента пользовательского интерфейса в меню пользовательского интерфейса.....	442
6.9.3.2.4 Принципы тестирования sc-агентов в ostis-системах.....	442
ЗАКЛЮЧЕНИЕ.....	446
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	447

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

В разработке...

ВВЕДЕНИЕ

В начале всего стоит Слово — знак, в котором человеческий опыт обретает форму, устойчивость и возможность быть переданным другому. Через слово мы различаем становление и устойчивость, бытие и небытие, задавая первичное разделение хаоса на упорядоченный мир сущего и сферу возможного. В классической метафизической традиции это связывали с идеей высшего упорядочивающего начала, которое задаёт меру, закон и логику мира, но в современном научном языке эти функции берёт на себя понятие системы.

Система — это множество элементов и отношений между ними, рассматриваемое как целое; состояние системы фиксирует конкретную конфигурацию этих элементов в данный момент времени. Именно через смену состояний мы описываем динамику, обучаемость и поведение интеллектуальных систем, будь то формальные вычислительные модели или сложные кибер-физические комплексы. Там, где раньше говорили о «порядке творения», сегодня мы говорим о структурах, инвариантах и законах эволюции состояний.

Любая интеллектуальная система неразрывно связана с языком, понимаемым как знаковую (семиотическую) систему — организованное множество знаков, служащих для передачи смысла между агентами. Знак связывает чувственно воспринимаемую форму и смысл, а язык задаёт правила их комбинирования; тем самым он превращает разрозненные впечатления в знание и делает возможным общение между людьми и машинами. Для интеллектуальных систем важно не только хранить и обрабатывать данные, но и оперировать знаками и смыслами, соотносить представления с реальностью и задачами, которые перед ними ставит человек.

Переход от человеческого опыта к машинной обработке требует ещё одного шага — кодирования: «дать данные» означает представить наблюдения, смыслы и задачи в виде структур, пригодных для математического анализа и алгоритмической обработки. Данные становятся тем материалом, из которого строится модель мира внутри интеллектуальной системы; постановка задачи определяет, какие аспекты мира считать существенными, какие — отбросить, и какие критерии успеха заложить в алгоритм. Так, исходя из семантических требований пользователя, мы приходим к строго формулируемым оптимизационным, логическим или вероятностным задачам.

Исторически путь к математическим основам интеллектуальных систем проходит через две большие культурные традиции: грамматику, логику и риторику слова (тривиум) и арифметику, геометрию, музыку и астрономию

числа (квадривиум). В средневековой образовательной модели тривиум формировал умение мыслить и выражать мысль, тогда как квадривиум вводил в мир абстрактных структур и числовых закономерностей; вместе они образовывали семь свободных искусств как фундамент работы разума. Современная теория искусственного интеллекта наследует обе линии: от первой — внимание к структуре языка, смысла и аргументации, от второй — строгую математизацию знаний и вычислительных процессов.

Цель данного учебного пособия «Математические основы интеллектуальных систем» — провести читателя по аналогичному пути «входа в математику», необходимую для понимания и разработки интеллектуальных технологий. Мы будем рассматривать ключевые математические конструкции, на которых опираются современные интеллектуальные системы — элементы дискретной математики. Тем самым от философских вопросов о слове, бытии и знаке мы перейдём к формальным аппаратам, позволяющим строить, анализировать и развивать интеллектуальные системы как строгие математические объекты.

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1 ОСНОВЫ ТЕОРИИ МНОЖЕСТВ В ИНТЕЛЛЕКТУАЛЬНЫХ СИСТЕМАХ

Любая интеллектуальная система решает задачи.

Чтобы интеллектуальная система могла решить задачу, необходимо представить (записать) эту задачу на языке, понятном этой системе. Такими языками являются формальные языки.

Любой формальный язык задаётся при помощи языка математики. Основу математики составляет теория множеств.

1.1 Понятие множества

Понятие множества является фундаментальным неопределяемым понятием математики.

Содержательно, *множество*¹ — это абстрактная математическая сущность, элементами которой являются другие сущности.

Понятие сущности также не определяется.

Можно сказать, что *сущность* — это нечто материальное или абстрактное; то, что существует; или, другими словами, познаваемая часть реального мира.

Будем считать, что сущности бывают сущностями, которые являются множествами, и сущностями, которые не являются множествами.

Множества зачастую обозначают прописными буквами латинского алфавита: А, В, С и так далее. Также можно применять буквенные и цифровые индексы.

Примерами множеств являются:

- алфавит русского языка — {А, Б, В, ..., Я};
- группа студентов — {Иванов, Петров, Сидоров};
- натуральные числа — {1, 2, 3, 4, ...}.

¹ Понятие множества было введено Георгом Кантором — немецким математиком, который создал теорию множеств в период между 1874 и 1897 годами.

1.1.1 Понятие элемента множества

Элемент множества — это сущность, которая принадлежит этому множеству.

Принадлежность элемента a множеству M записывают так:

$$a \in M,$$

где $\in$ — знак принадлежности. Этот знак представляет собой видоизменённую букву греческого алфавита ϵ , с которой начинается слово $\epsilon\sigma\tau\iota$, по-русски обозначающее «есть».

Читается запись следующим образом: «сущность a есть элемент множества M », либо «сущность a является элементом множества M », либо «сущность a принадлежит множеству M ».

Понятие принадлежности является отношением (относительным понятием), поэтому вводится и рассматривается ниже.

Стоит отметить, что элементом множества может быть другое множество, в том числе само это множество.

Пример. Пусть $A = \{a, b, c\}$. Тогда элементами множества A являются сущности a , b и c , то есть $a \in M$, $b \in M$, $c \in M$.

1.1.2 Понятие мощности множества

Мощность множества, или *кардинальность множества* — это число элементов этого множества.

Для обозначения числа элементов множества часто используют две вертикальные черты, между которыми записывается само множество или его обозначение.

Для конечных множеств мощность — это натуральное число или ноль (для пустого множества). Для бесконечных множеств вводятся специальные кардинальные числа: $\aleph_0$ (счётное бесконечное) и $2^{\aleph_0}$ (несчётное).

Пример 1. Пусть $A = \{a, b, c\}$. Тогда $|A| = 3$.

Пример 2. Пусть $A = \varnothing$. Тогда $|A| = 0$.

Пример 3. Пусть $\mathbb{N}$ — это множество натуральных чисел. Тогда $|\mathbb{N}| = \aleph_0$.

1.1.3 Классификация множеств

Множества бывают конечными и бесконечными.

1.1.3.1 Понятие конечного множества

Конечное множество — это множество, для которого существует неотрицательное целое число, равное мощности этого множества.

Пример конечного множества — множество дней недели $A = \{\text{понедельник, вторник, ..., воскресенье}\}$.

1.1.3.2 Понятие бесконечного множества

Бесконечное множество — это множество, для которого не существует неотрицательного целого числа, равного мощности этого множества.

Пример бесконечного множества — множество натуральных чисел $A = \{1, 2, 3, 4, \dots\}$.

1.1.3.3 Понятие пустого множества

Пустое множество — это множество, мощность которого равна нулю.

Пустое множество обозначается как $\emptyset$ или $\{\}$.

Пример. Пусть $A = \emptyset$. Тогда мощность множества A равна $|A| = 0$.

1.1.4 Понятие подмножества множества

Подмножество множества A — это множество B , элементами которого являются только и только те сущности, которые являются элементами множества A .

Пример. Пусть $A = \{1, 2, 3, 4, 5\}$. Тогда $B = \{2, 4\}$ является подмножеством множества A , так как все элементы множества B являются элементами множества A .

Другими примерами подмножеств являются:

- множество гласных букв $\{А, Е, И, О, У, Ы, Э, Ю, Я\}$, которое является подмножеством множества всех букв русского алфавита;
- множество кошек, которое является подмножеством множества млекопитающих.

1.1.4.1 Понятие собственного подмножества множества

Собственное подмножество множества A — это подмножество B множества A , которое не является пустым множеством и которое не равно множеству A .

Принято обозначать собственное подмножество B множества A как:

$$B \subsetneq A,$$

где $\subsetneq$ — знак нестрогого включения. Указанная запись $B \subsetneq A$ читается как «множество B не строго включено во множество A ».

1.1.4.2 Понятие несобственного подмножества множества

Несобственное подмножество множества A — это подмножество B множества A , которое является либо пустым множеством, либо которое равно множеству A .

Таким образом, пустое множество ($\emptyset$) является несобственным подмножеством любого другого множества.

Принято обозначать несобственное подмножество B множества A как:

$$B \subseteq A,$$

где $\subseteq$ — знак строгого включения. Указанная запись $B \subseteq A$ читается как «множество B строго включено во множество A ».

1.1.5 Понятие надмножества множества

Надмножество множества A — это множество B , элементами которого являются элементы множества A .

Пример. Пусть $A = \{2, 4\}$. Тогда $B = \{1, 2, 3, 4, 5\}$ является надмножеством множества A , так как все элементы множества A являются элементами множества B .

Надмножество B множества A обозначается как $B \supseteq A$.

Другими примерами надмножеств являются:

- множество всех млекопитающих, которое является надмножеством множества кошек;
- множество всех чисел, которое является надмножеством множества натуральных чисел.

1.1.6 Операции над множествами

Множества могут быть получены в результате применения операции (операций) над заданными (известными) множествами.

1.1.6.1 Понятие объединения множеств

Объединение множеств $A_1, A_2, A_3, \dots, A_n$, или сумма множеств $A_1, A_2, A_3, \dots, A_n$ — это множество B , элементы которого являются элементами хотя бы одного из заданных множеств $A_1, A_2, A_3, \dots, A_n$:

$$B = A_1 \cup A_2 \cup A_3 \cup \dots \cup A_n,$$

где $\cup$ — знак операции объединения множеств.

Пример. Пусть $A = \{1, 2, 3\}$, $B = \{3, 4, 5\}$. Тогда $C = A \cup B = \{1, 2, 3, 4, 5\}$.

Операция объединения множеств обладает свойствами:

1) коммутативности:

$$A \cup B = B \cup A;$$

$$A \cup B \cup C = B \cup A \cup C = C \cup A \cup B;$$

2) ассоциативности:

$$(A \cup B) \cup C = (A \cup C) \cup B = (B \cup C) \cup A = A \cup B \cup C;$$

3) если $B \subseteq A$ или $B \subset A$, то $A \cup B = A$.

Из свойства 3) следует, что:

$$A \cup A = A;$$

$$A \cup \emptyset = A.$$

1.1.6.2 Понятие пересечения множеств

Пересечение множеств $A_1, A_2, A_3, \dots, A_n$, или произведение множеств $A_1, A_2, A_3, \dots, A_n$ — это множество B , элементы которого являются элементами каждого из заданных множеств $A_1, A_2, A_3, \dots, A_n$:

$$B = A_1 \cap A_2 \cap A_3 \cap \dots \cap A_n,$$

где $\cap$ — знак операции пересечения множеств.

Пример. Пусть $A = \{1, 2, 3\}$, $B = \{3, 4, 5\}$. Тогда $C = A \cap B = \{3\}$.

Операция пересечения множеств обладает свойствами:

1) коммутативности:

$$A \cap B = B \cap A;$$

$$A \cap B \cap C = B \cap A \cap C = C \cap A \cap B;$$

2) ассоциативности:

$$(A \cap B) \cap C = (A \cap C) \cap B = (B \cap C) \cap A = A \cap B \cap C;$$

3) если $A \subseteq B$ или $A \subset B$, то $A \cap B = A$.

Из свойства 3) следует, что:

$$A \cap A = A;$$

$$A \cap \emptyset = \emptyset.$$

Стоит отметить другие свойства операций объединения и пересечения множеств.

Операция пересечения множеств дистрибутивна относительно операции объединения множеств:

$$A \cap (B \cup C) = (A \cap B) \cup (A \cap C).$$

Операция объединения множеств дистрибутивна относительно операции пересечения множеств:

$$A \cup (B \cap C) = (A \cup B) \cap (A \cup C);$$

1.1.6.3 Понятие разности двух множеств

Разность множеств A и B — это множество C , элементы которого являются элементами множества A и не являются элементами множества B :

$$C = A \setminus B,$$

где $\setminus$ — знак операции разности множеств.

Пример. Пусть $A = \{1, 2, 3\}$, $B = \{3, 4, 5\}$. Тогда $C = A \setminus B = \{1, 2\}$, а $D = B \setminus A = \{4, 5\}$.

В отличие от операции объединения двух множеств и операции пересечения двух множеств, операция разности множеств не является коммутативной и ассоциативной операцией.

1.1.6.4 Понятие симметрической разности двух множеств

Симметрическая разность множеств A и B — это множество C , которое является объединением результата разности множества A и множества B и результата разности множества B и множества A :

$$C = A \Delta B,$$

где Δ — знак операции разности множеств.

Пример. Пусть $A = \{1, 2, 3\}$, $B = \{3, 4, 5\}$. Тогда $C = A \Delta B = (A \setminus B) \cup (B \setminus A) = \{1, 2, 4, 5\}$.

Операция симметрической разности двух множеств обладает свойствами:

1) коммутативности:

$$A \Delta B = B \Delta A;$$

2) ассоциативности:

$$(A \Delta B) \Delta C = (A \Delta C) \Delta B = (B \Delta C) \Delta A = A \Delta B \Delta C;$$

1.1.6.5 Понятие булеана множества

Булеан множества A — это множество B , элементами которого являются все подмножества множества A .

Пример. Пусть $A = \{1, 2\}$. Тогда булеан множества A : $B = P(A) = \{\emptyset, \{1\}, \{2\}, \{1, 2\}\}$.

1.1.6.6 Понятие разбиение множества

Разбиение множества — это *семейство* попарно непересекающихся непустых множеств, объединение которых является заданным множеством.

Пример. Пусть дано множество $A = \{1, 2, 3, 4\}$. Разбиение этого множества является множество $P = \{\{1, 2\}, \{3\}, \{4\}\}$.

Другими словами, отношение разбиение задаёт связки между множествами и семействами их подмножеств, которые:

- не пустые;
- попарно не пересекаются между собой;
- и являются покрытиями первых множеств.

1.1.6.7 Понятие декомпозиции множества

Декомпозиция множества — это семейство множеств, объединение которых является заданным множеством.

В отличие от операции разбиения множества операция декомпозиция множества допускает пересечение подмножеств для заданного множества.

1.2 Понятие связки

Связка (связь) — это конечное множество, элементами которого являются одна или более сущностей.

1.2.1 Понятие синглтона

Синглетон — это связка, элементом которой является только и только одна сущность.

Пример. Множество стран, столицей которых является город Минск.

1.2.2 Понятие пары

Пара — это связка, элементами которой являются только и только две сущности. Также она называется *бинарной связкой*.

Пример. Множество из двух друзей (дружба между двумя людьми).

1.2.3 Понятие тройки

Тройка — это связка, элементами которой являются только и только три сущности.

Пример. Множество из трёх студентов, участвующих в олимпиаде (участие трёх студентов в олимпиаде).

1.2.4 Понятие небинарной связки

Небинарная связка — это связка, которая не является парой.

Все синглтоны и все тройки являются небинарными связками.

1.2.5 Ориентированные и неориентированные связки

Связки бывают *ориентированными* и *неориентированными*.

1.2.5.1 Понятие ориентированной связки

Ориентированная связка (кортеж) — это связка, в которой элементы имеют различные роли.

Пример. Пусть a — это человек, а b — это ребёнок человека a . Тогда ориентированной связкой между отцом и сыном является $A = \langle a, b \rangle$. Эта связка является ориентированной, поскольку её элементы имеют различные роли: “отец” и “сын”.

1.2.5.2 Понятие неориентированной связки

Неориентированная связка — это связка, в которой элементы имеют одинаковые роли или не имеют их вовсе.

Пример. Пусть a — это человек, а b — это брат человека a . Тогда неориентированной связкой между братьями является $A = \{a, b\}$. Эта связка является неориентированной, поскольку её элементы имеют одинаковые роли: “брат” и “брат”.

Понятие *роли* (или, по-другому, *ролевого отношения*) рассматривается ниже.

Таким образом, *ориентированные связки* обозначаются парой скобок $\langle \rangle$, а *неориентированные связки* — парой скобок $\{ \}$.

1.2.6 Понятие декартова произведения двух множеств

Декартово² произведение множеств A и B , или прямое произведение множеств A и B — это множество ориентированных пар C , первыми элементами которых являются элементы множества A и вторыми элементами которых являются элементы множества B :

$$C = A \times B,$$

где $\times$ — знак декартова произведения множеств.

² Декарт Рене — французский философ и математик, один из первых создателей формального языка математики — жил в XVII веке (1596–1659). Теория множеств появилась через 200 лет, поэтому Р. Декарт о ней никогда не слышал и заниматься ею не мог. Название операции «декартово произведение» появилось в связи с тем, что в теории множеств нашёл применение метод координат, разработанный Р. Декартом.

Пример. Пусть $A = \{1, 2\}$, $B = \{a, b\}$. Тогда $C = A \times B = \{<1, a>, <1, b>, <2, a>, <2, b>\}$.

Операция декартова произведения множеств не коммутативна:

$$B \times A \neq A \times B.$$

Операция декартова произведения множеств ассоциативна:

$$(A \times B) \times C = A \times (B \times C) = A \times B \times C.$$

1.3 Понятие класса

Оперирование исключительно конкретными множествами и связками недостаточно для построения интеллектуальных систем, требующих способности к обобщению. Для построения интеллектуальных систем нужно уметь оперировать не только единичными сущностями, но и целыми категориями (классами) этих сущностей, объединённых общими свойствами (характеристиками). Формализацией такого объединения сущностей по признаку наличия общих свойств является понятие класса.

Класс — это множество сущностей (в том числе связок), обладающих явно указанными общими свойствами.

Классы обычно бесконечны.

1.3.1 Понятие понятия

Понятие понятия можно определить через понятие класса.

Понятие — это класс, являющийся исследуемой сущностью хотя бы для одной предметной области.

Каждому понятию ставится в соответствие:

- *объём (экстенционал) понятия* — это множество сущностей, которые обозначаются этим понятием;
- *содержание (интенционал) понятия* — это множество признаков, характеризующих это понятие;
- *термин (имя) понятия* — это строка символов, представляющая (изображающая) это понятие.

Исследуя классы сущностей, можно заметить фундаментальное различие между ними. Одни понятия описывают самостоятельные сущности,

существующие «сами по себе» (например, «стол» или «студент»), в то время как другие существуют только как связь между сущностями (например, «дружба» или «превосходство»). Это наблюдение приводит к делению понятий на абсолютные и относительные.

Таким образом, понятия бывают *относительными* и *абсолютными*.

1.3.1.1 Понятие абсолютного понятия

Абсолютное понятие — это понятие, которое не является относительным; или класс сущностей, обладающих явно указанными общими свойствами и не являющихся связками.

Понятие *относительного понятия* рассматривается ниже.

Примерами абсолютных понятий являются:

- *стул* (как множество всевозможных стульев, класс мебели);
- *кот* (как множество всех котов, класс котов);
- *студент* (как множество всех студентов, класс студентов);
- *книга* (как множество всех книг, класс книг).

1.3.1.1 Понятие относительного понятия

Относительное понятие — это понятие, представляющее собой класс связок, которые не могут существовать без указания на их элементы; или класс связок, обладающих явно указанными общими свойствами.

Примерами относительных понятий являются:

- “больше чем” (отношение сравнения);
- “принадлежность” (отношение принадлежности элемента множеству);
- “является родителем” (семейное отношение);
- “следует за” (временное отношение).

1.4 Понятие отношения

Частным случаем относительного понятия является отношение.

Отношение — это множество (класс) связок.

Говорят, что отношение, то есть множество связок между сущностями, задаётся на некотором множестве (классе) этих сущностей.

То есть, с формальной точки зрения, *отношение*, заданное на некотором множестве A — это множество, являющееся подмножеством декартова произведения множества A самого на себя некоторое количество раз; или множество, являющееся подмножеством декартовой степени множества A .

1.4.1 Классификация отношений по типу ориентированности связок

Отношения бывают *ориентированными* и *неориентированными*.

1.4.1.1 Понятие ориентированного отношения

Ориентированное отношение — это отношение, элементами которого являются ориентированные связки.

Следует отличать ориентированное отношение от упорядоченного отношения (отношения, для связок которого задано отношение порядка).

1.4.1.2 Понятие неориентированного отношения

Неориентированное отношение — это отношение, элементами которого являются неориентированные связки.

1.4.2 Понятие арности отношения

Арность отношения — это мощность связок данного отношения.

Арность отношения принадлежности равна 2.

Арность отношения включения равна 2.

1.4.3 Понятие области определения отношения

Область определения отношения — это результат теоретико-множественного объединения всех связок этого отношения; или, другими словами, результат теоретико-множественного объединения всех множеств, являющихся доменами данного отношения.

1.4.4 Понятие домена отношения

Первый домен отношения — это множество всех элементов, являющихся первыми элементами связок данного отношения.

Второй домен отношения — это множество всех элементов, являющихся вторыми элементами связок данного отношения.

Если отношение имеет большую арность (больше чем 2), то для него можно задать больше доменов (если арность 3, то 3 домена, и так далее).

Пример. Пусть есть отношение “быть отцом” на множестве людей – $R = \{ \langle \text{Иван, Пётр} \rangle, \langle \text{Сергей, Михаил} \rangle, \dots \}$, тогда:

- арность отношения равна 2;
- областью определения отношения R является множество $\{ \text{Иван, Пётр, Сергей, Михаил, } \dots \}$ — множество всех сущностей, являющихся элементами связок заданного отношения R ;
- первым доменом отношения R является множество $\{ \text{Иван, Сергей, } \dots \}$ — множество всех отцов;
- вторым доменом отношения R является множество $\{ \text{Пётр, Михаил, } \dots \}$ — множество всех детей.

Пример. Пусть есть отношение “человек дарит вещь другому человеку” — $R = \{ \langle \text{Анна, книга, Вера} \rangle, \langle \text{Олег, цветы, Катя} \rangle, \dots \}$, тогда:

- областью определения отношения R является множество $\{ \text{Анна, Олег, книга, цветы, Вера, Катя, } \dots \}$ — множество всех сущностей, являющихся элементами связок заданного отношения R ;
- первым доменом отношения R является множество $\{ \text{Анна, Олег, } \dots \}$ — множество всех дарителей;
- вторым доменом отношения R является множество $\{ \text{книга, цветы, } \dots \}$ — множество всех вещей, которые можно подарить;
- третьим доменом отношения R является множество $\{ \text{Вера, Катя, } \dots \}$ — множество всех получателей подарков.

Если элементы связок отношения имеют роли, то есть для принадлежностей этих элементов связкам заданного отношения указаны ролевые отношения, то для отношения можно задать это ролевое отношение в качестве атрибута. Тогда можно указать домен заданного отношения относительно этого атрибута.

Пример. Пусть есть отношение “работник занимает должность в отделе” — $R = \{ \langle \text{Алексей, инженер, продажи} \rangle, \langle \text{Мария, менеджер, маркетинг} \rangle, \dots \}$, тогда:

- атрибутами отношения R являются работник, должность, отдел;

- доменом по атрибуту "работник" отношения R является множество {Алексей, Мария, ...};
- доменом по атрибуту "должность" отношения R является множество {инженер, менеджер, ...};
- доменом по атрибуту "отдел" отношения R является множество {продажи, маркетинг, ...}.

1.4.5 Классификация отношений по типу связей

По типу связей в отношениях выделяются следующие классы отношений:

- *ролевое отношение*;
- *и неролевое отношение*.

1.4.5.1 Понятие ролевого отношения

Ролевое отношение, или *атрибут* — это отношение, являющееся подмножеством отношения принадлежности.

Ролевое отношение задаёт *роль* элемента в рамках некоторого множества.

Пример. В отношении “учитель — ученик” по отношению передачи знаний роли элементов в связках — передающий и получающий.

1.4.5.2 Понятие неролевого отношения

Неролевое отношение — это отношение, не являющееся подмножеством отношения принадлежности.

1.4.6 Классификация отношений по признаку арности

По признаку арности выделяются следующие классы отношений:

- *унарное отношение*;
- *бинарное отношение*;
- *квазибинарное отношение*;
- *тернарное отношение*;
- и так далее.

1.4.6.1 Понятие унарного отношения

Унарное отношение — это множество (класс) синглетонов; или отношение, элементами которого являются синглетоны.

В отличие от класса, элементами унарного отношения являются синглетоны сущностей.

Примерами унарных отношений являются:

- быть целым числом — $\{<1>, <2>, \dots\}$;
- быть человеком — $\{<\text{Маша}>, <\text{Саша}>, \dots\}$;
- быть студентом — $\{<\text{Маша}>, <\text{Саша}>, \dots\}$.

1.4.6.2 Понятие бинарного отношения

Бинарное отношение (по латыни «бис» означает «дважды») — это множество (класс) пар; или отношение, элементами которого являются пары.

С формальной точки зрения, *бинарное отношение* — это множество, являющееся подмножеством декартового произведения множеств A и B , то есть множество пар, первые элементы которых являются элементами множества A , а вторые элементы которых являются элементами множества B .

Примерами бинарных отношений являются:

- отношение равенства ($=$) на множестве чисел — $\{\{1, 1\}, \{2, 2\}, \dots\}$;
- отношение порядка ($<$) на множестве чисел — $\{<1, 2>, <2, 3>, \dots\}$;
- отношение принадлежности элемента множеству;
- отношение включения одного множества в другое.

1.4.6.3 Понятие квазибинарного отношения

Квазибинарное отношение — это бинарное отношение, первые или вторые компоненты пар которого являются связками.

Примерами квазибинарных отношений являются:

- иметь друзей — $\{<\text{Вася}, \{\text{Саша}, \text{Слава}, \text{Петя}\}>, <\text{Даша}, \{\text{Катя}, \text{Вика}, \text{Настя}\}>, \dots\}$;

- авторы книги — $\{ \langle \text{Война и Мир}, \{ \text{Толстой} \} \rangle, \langle \text{Технологии ИИ}, \{ \text{Сидоров, Петров} \} \rangle \}$.

1.4.6.4 Понятие тернарного отношения

Тернарное отношение — это множество (класс) троек; или отношение, элементами которого являются тройки.

Примерами тернарных отношений являются:

- студент сдал лабораторную работу по предмету — $\{ \langle \text{Максим, ЛР 1, МОИС} \rangle, \langle \text{Даша, ЛР 1, МОИС} \rangle, \dots \}$;
- человек имеет профессию и работает в организации — $\{ \langle \text{Петров, инженер, БЕЛАЗ} \rangle, \langle \text{Иванов, преподаватель, БГУИР} \rangle, \dots \}$.

1.4.7 Классификация бинарных отношений

Существуют важные свойства бинарных отношений, такие как:

- рефлексивность (aRa),
- антирефлексивность ($\text{не } aRa$),
- симметричность (если aRb , то bRa),
- асимметричность (если aRb , то $\text{не } bRa$),
- транзитивность (если aRb и bRc , то aRc),
- антитранзитивность (если aRb и bRc , то $\text{не } aRc$),
- антисимметричность (если aRb и bRa , то $a = b$),
- линейность (тотальность) (если $a \neq b$, то aRb и bRa);

и производные:

- толерантность (рефлексивность + симметричность);
- эквивалентность (рефлексивность + симметричность + транзитивность);
- отношение порядка (рефлексивность + антисимметричность + транзитивность + линейность);
- отношение строгого порядка;
- отношение нестрогого порядка;
- и другие.

1.4.7.1 Понятие рефлексивного бинарного отношения

Рефлексивное бинарное отношение — это бинарное отношение, которое содержит все пары, в которых первый и второй элементы совпадают.

Примерами рефлексивных бинарных отношений являются:

- отношение равенства ($=$) на множестве чисел — $\{\{1, 1\}, \{2, 2\}, \dots\}$;
- отношение делимости ($\%$) на множестве натуральных чисел — $\{<4, 2>, <6, 2>, \dots\}$;
- отношение включения ($\supseteq$) на множестве всех подмножеств данного множества — $\{<\{a, b, c\}, \{a, b\}>, <\{a, b, c\}, \{a\}>, \dots\}$.

1.4.7.2 Понятие антирефлексивного бинарного отношения

Антирефлексивное бинарное отношение — это бинарное отношение, которое не содержит ни одной пары, в которой первый и второй элементы совпадают.

Примерами антирефлексивных бинарных отношений являются:

- отношение строгого порядка ($<$) на множестве чисел — $\{<1, 2>, <2, 3>, \dots\}$;
- отношение "быть родителем" на множестве людей;
- отношение перпендикулярности прямых в евклидовой геометрии.

1.4.7.3 Понятие симметричного бинарного отношения

Симметричное бинарное отношение — это бинарное отношение, в котором для каждой пары, первым элементом которой является сущность a , а вторым элементом которой — сущность b , существует обратная пара, первым элементом которой является сущность b , а вторым элементом которой — сущность a .

Примерами симметричных бинарных отношений являются:

- отношение равенства ($=$) — “если $a = b$, то $b = a$ ”;
- отношение перпендикулярности прямых — “если $a \perp b$, то $b \perp a$ ”;
- отношение "быть соседом по парте" на множестве учеников — “если ученик A сосед ученика B , то ученик B сосед ученика A ”.

1.4.7.4 Понятие асимметричного бинарного отношения

Асимметричное бинарное отношение — это бинарное отношение, в котором для каждой пары, первым элементом которой является сущность a , а вторым элементом которой — сущность b , не существует обратной пары, первым элементом которой является сущность b , а вторым элементом которой — сущность a .

Асимметричное отношение всегда антирефлексивно.

Примерами асимметричных бинарных отношений являются:

- отношение строгого порядка ($<$) — “если $a < b$, то b не $< a$ ”;
- отношение "быть родителем" — “если A — родитель B , то B не является родителем A ”;
- отношение "быть старше на 5 лет" — “если человек A старше человека B на 5 лет, то человек B не старше человека A на 5 лет”.

1.4.7.5 Понятие антисимметричного бинарного отношения

Антисимметричное бинарное отношение — это бинарное отношение, в котором если для каждой пары, первым элементом которой является сущность a , а вторым элементом которой — сущность b , существует обратная пара, первым элементом которой является сущность b , а вторым элементом которой — сущность a , то сущность a совпадает с сущностью b .

Примерами антисимметричных бинарных отношений являются:

- отношение нестрогого порядка ($\leq$) — “если $a \leq b$ и $b \leq a$, то $a = b$ ”;
- отношение делимости на натуральных числах — “если $a \mid b$ и $b \mid a$, то $a = b$ ”;
- отношение включения $\subseteq$ — “если $A \subseteq B$ и $B \subseteq A$, то $A = B$ ”.

1.4.7.6 Понятие транзитивного бинарного отношения

Транзитивное бинарное отношение — это бинарное отношение, в котором для каждой пары, первым элементом которой является сущность a , а вторым элементом которой — сущность b , существует пара, первым элементом которой является сущность b , а вторым элементом которой — сущность c , при условии, что в этом же отношении существует пара, первым элементом которой является сущность a , а вторым элементом которой — сущность c .

Примерами транзитивных бинарных отношений являются:

- отношение равенства ($=$) — “если $a = b$ и $b = c$, то $a = c$ ”;
- отношение порядка ($<$) — “если $a < b$ и $b < c$, то $a < c$ ”;
- отношение параллельности прямых — “если $a \parallel b$ и $b \parallel c$, то $a \parallel c$ ”.

1.4.7.7 Понятие антитранзитивного бинарного отношения

Антитранзитивное бинарное отношение — это бинарное отношение, в котором для каждой пары, первым элементом которой является сущность a , а вторым элементом которой — сущность b , существует пара, первым элементом которой является сущность b , а вторым элементом которой — сущность c , при условии, что в этом же отношении не существует пары, первым элементом которой является сущность a , а вторым элементом которой — сущность c .

Примерами антитранзитивных бинарных отношений являются:

- отношение "быть родителем" — “если A — родитель B , и B — родитель C , то A не является родителем C (A является дедушкой/бабушкой C)”;
- отношение "быть соседом по парте" — “если A сосед B , и B сосед C , то A не является соседом C (при условии, что за одной партой сидят только два ученика)”.

1.4.7.8 Понятие линейного бинарного отношения

Линейное (тотальное) бинарное отношение — это бинарное отношение, заданное на множестве A , в котором для любых двух несовпадающих сущностей a и b из этого множества существует либо пара, где первый элемент является сущностью a , а второй — сущностью b , либо пара, где первый элемент является сущностью b , а второй — сущностью a (то есть любые два элемента связаны парой в одну или другую сторону).

Линейное отношение обеспечивает сравнимость всех элементов.

Примерами линейных отношения являются:

- отношение $\leq$ на множестве вещественных чисел — “для любых $a, b \in \mathbb{R}$ либо $a \leq b$, либо $b \leq a$ ”;
- отношение "не старше" на множестве людей — “для любых двух людей либо первый не старше второго, либо второй не старше первого”;
- отношение лексикографического порядка на множестве слов — “любые два различных слова можно сравнить по алфавитному порядку”.

1.4.7.9 Понятие отношения толерантности

Отношение толерантности — это рефлексивное и симметричное бинарное отношение.

Примерами отношения толерантности являются:

- отношение "иметь одинаковую длину" на множестве строк — каждая строка имеет одинаковую длину с собой (рефлексивность), и если строка А имеет такую же длину, как строка В, то и строка В имеет такую же длину, как строка А (симметричность);
- отношение "быть знакомыми" на множестве людей — каждый человек знаком сам с собой (рефлексивность), и если А знаком с В, то В знаком с А (симметричность);
- отношение "отличаться не более чем на 1" на множестве целых чисел — для каждого $n \in \mathbb{Z}$ ($|n - n| \leq 1$) и если $|a - b| \leq 1$, то $|b - a| \leq 1$.

1.4.7.10 Понятие отношения эквивалентности

Отношение эквивалентности — это рефлексивное, симметричное и транзитивное бинарное отношение.

Примерами отношения эквивалентности являются:

- отношение равенства по модулю 3 на множестве целых чисел — $a \equiv b \pmod{3} \Leftrightarrow 3 \mid (a - b)$;
- отношение "иметь одинаковую фамилию" на множестве людей;
- отношение "быть параллельными" на множестве прямых в плоскости.

1.4.7.11 Понятие транзитивного замыкания бинарного отношения

Транзитивное замыкание бинарного отношения — это наименьшее (по включению) транзитивное бинарное отношение, которое содержит заданное отношение.

Транзитивное замыкание бинарного отношения R обозначается как R^+ .

Примеры транзитивных замыканий бинарных отношений:

- если $R = \{<1,2>, <2,3>\}$, то $R^+ = \{<1,2>, <2,3>, <1,3>\}$;
- отношение "быть предком" является транзитивным замыканием отношения "быть родителем";
- отношение "быть меньше или равно" $\leq$ является транзитивным замыканием отношения "быть строго меньше" $<$.

1.4.7.12 Понятие обратного отношения

Обратное отношение для заданного ориентированного бинарного отношения — это ориентированное бинарное отношение, состоящее из пар, полученных путём перестановки первых и вторых элементов каждой пары заданного отношения.

Примеры обратных отношений для заданных ориентированных бинарных отношений:

- если заданное отношение — «быть родителем» (пары вида <Родитель, Ребенок>), то обратное — «быть ребенком» (пары вида <Ребенок, Родитель>);
- если заданное отношение — «больше» ($>$), то обратное — «меньше» ($<$).

1.4.8 Базовые отношения

Базовыми отношениями являются:

- отношение принадлежности;
- и отношение включения.

1.4.8.1 Понятие отношения принадлежности

Отношение принадлежности — это ориентированное бинарное отношение, первыми элементами пар которого являются множества, а вторыми элементами пар которого являются сущности, которые являются элементами множеств, являющихся первыми элементами пар этого отношения; или, другими словами, это ориентированное бинарное отношение, элементами которого являются пары принадлежности.

Пример пары *отношения принадлежности* — $2 \in \{1,2,3\}$. То есть $R_{\text{принадлежность}} (\in) = \{ \langle \{1,2,3\}, 2 \rangle, \dots \}$.

1.4.8.2 Понятие отношения включения

Отношение включения — это ориентированное бинарное отношение, первыми элементами пар которого являются множества, а вторыми элементами пар которого являются множества, которые являются подмножествами множеств, которые являются первыми элементами пар этого отношения (или для которых множества, которые являются первыми элементами пар этого отношения, являются надмножествами).

Пример пары *отношения включения* — $\{1,2\} \subseteq \{1,2,3\}$. То есть $R_{\text{включение}} (\subseteq) = \{<\{1,2,3\}, \{1,2\}>, \dots\}$.

Другими словами, *отношение принадлежности* задаёт связки между множествами и их элементами, а *отношение включения* задаёт связки между множествами и их подмножествами.

1.4.9 Понятие метаотношения

Метаотношение — это отношение, элементами связок которого являются отношения.

1.5 Понятие параметра

Параметр — это класс, являющийся семейством всевозможных классов эквивалентности или толерантности, задаваемых либо отношением эквивалентности, либо отношением толерантности.

Примерами параметров как отношений эквивалентности являются:

- равновеликость геометрических фигур (по длине, площади, объему — в зависимости от размерности этих фигур);
- иметь одинаковый цвет (быть эквивалентными по цвету);
- эквивалентность, по вкусу, запаху, твердости и так далее.

Пример. Пусть $X = \{0, 1, 2, 3, 4, 5\}$ и на X задано отношение эквивалентности — множество пар, состоящих из чисел из X , равных по модулю 3.

Тогда классы эквивалентности таковы: $\{0, 3\}$, $\{1, 4\}$, $\{2, 5\}$, а семейство всех классов эквивалентности, то есть параметр “равенство по модулю 3” есть $P = \{\{0, 3\}, \{1, 4\}, \{2, 5\}\}$.

1.5.1 Понятие измеряемого параметра

Изменяемый параметр — это параметр, значение (элемент, экземпляр) которого трактуется как величина, которой можно поставить в соответствие ее числовое значение на основании выбранной единицы измерения и точки отсчета (нулевой отметки выбранной шкалы).

Примерами измеряемого параметра являются:

- возраст,

- площадь,
- вес.

Пример. Пусть $L = \{\text{Анна, Борис, Виктор, Галина}\}$ — множество людей и на L задано отношение эквивалентности R — множество пар, состоящих из людей одинакового возраста.

Тогда:

- классы эквивалентности таковы:

$20_лет = \{\text{все люди возраста 20 лет}\};$

$25_лет = \{\text{все люди возраста 25 лет}\};$

$30_лет = \{\text{все люди возраста 30 лет}\};$

- единицей измерения является год;
- точкой отсчёта является момент рождения (0 лет);
- числовым эквивалентом — положительное действительное число;
- а семейством всех классов эквивалентности, то есть параметром “быть одинакового возраста” является $P = \{\{\text{все люди возраста 20 лет}\}, \{\text{все люди возраста 25 лет}\}, \{\text{все люди возраста 30 лет}\}\}$.

1.5.2 Понятие неизмеряемого параметра

Неизмеряемый параметр — это параметр, для которого нельзя подобрать числовой эквивалент и единицу измерения.

Примерами неизмеряемого параметра являются:

- цвет,
- вкус.

Пример. Пусть $O = \{\text{яблоко}_1, \text{помидор}_1, \text{лимон}_1, \text{яблоко}_2, \text{банан}_1, \text{помидор}_2\}$ — множество объектов.

Пусть отношение эквивалентности — множество ориентированных пар, состоящих из объектов с эквивалентным цветом.

Тогда:

- классы эквивалентности таковы:

Красный = {яблоко₁, помидор₁, помидор₂};

Жёлтый = {лимон₁, банан₁};

Зелёный = {яблоко₂};

- а параметр “цвет” = {Красный, Жёлтый, Зелёный, Синий, Белый, Чёрный, ...} является семейством всех возможных цветов.

1.5.3 Понятие величины

Величина — это класс сущностей, имеющих одинаковое значение соответствующего параметра.

Каждая величина представляет собой однозначный и независимый от шкалы измерения результат измерения некоторой характеристики у некоторой сущности.

Каждой величине можно поставить в соответствие её числовое значение на основании выбранной единицы измерения и точки отсчета (нулевой отметки выбранной шкалы).

Пример. Пусть $R = \{\text{комната}_1, \text{комната}_2, \text{комната}_3, \text{комната}_4\}$ — множество помещений с площадями:

- комната₁: 20 м²;
- комната₂: 20 м²;
- комната₃: 15 м²;
- комната₄: 20 м².

Тогда величина “площадь 20 м²” есть $V_{20} = \{\text{комната}_1, \text{комната}_2, \text{комната}_4\}$ — класс всех комнат, имеющих площадь 20 м².

1.5.3.1 Понятие точной величины

Точная величина — это величина, которой соответствует измеряемый числовой эквивалент.

Каждая точная величина имеет одно фиксированное значение в некоторой единице измерения или по какой-либо шкале. При этом считается, что все элементы такого класса имеют одинаковое значение данного параметра и отклонениями можно пренебречь.

Пример. Пусть количество студентов в группе $V = 25$ человек.

Тогда классом эквивалентности может быть множество всех групп, в которых ровно 25 студентов.

Здесь числовым значением параметра является число 25, единицей измерения — человек, точность — абсолютная (дискретная величина).

1.5.3.2 Понятие неточной величины

Неточная величина — это величина, полученная в ходе эксперимента, не обладающая достоверностью и характеризуется некоторой точностью измерения, являющаяся точной величиной.

Каждой неточной величине ставится в соответствие её значение в некоторой единице измерения или по какой-либо шкале, а также дополнительно указывается *точность*, т.е. возможное отклонение от данного значения.

Пример. Измерение температуры тела термометром показало, что $T = 36.6^{\circ}\text{C}$.

Нормальная температура тела находится в диапазоне от 36.5°C до 36.7°C .

Тогда числовым значением данного параметра является 36.6°C , абсолютной погрешность — $\Delta = \pm 0.1^{\circ}\text{C}$.

1.5.3.3 Понятие интервальной величины

Интервальная величина — это величина, задающая некоторый промежуток числовых эквивалентов, которому соответствует некоторое множество объектов.

Каждая интервальная величина представляет собой класс сущностей, находящихся в рамках точно заданного интервала, минимальная и максимальная точка которого являются точными величинами. Результатом *измерения* такой величины является ориентированная пара, первым компонентом которой является левая (меньшая) граница интервала, вторым компонентом — правая (большая) граница интервала.

1.6 Понятие структуры

Структура — это множество, элементами которого являются связи между элементами этого множества.

Примерами структур являются:

- аудитория университета;

- магазин;
- солнечная система.

Пример. Аудитория университета $V = \{\text{Стол}_1, \text{Стол}_2, \text{Стол}_3, \text{Стол}_4, \text{Стол}_5, \text{Стул}_1, \text{Стул}_2, \dots, \text{Доска}, \text{Проектор}, (\text{Стол}_1, \text{Стул}_1), (\text{Стол}_1, \text{Стул}_2), (\text{Стол}_2, \text{Стул}_3), (\text{Доска}, \text{Проектор}), (\text{Студент}_1, \text{Стол}_1), (\text{Преподаватель}, \text{Доска})\}$ является структурой.

1.6.1 Понятие тривиальной структуры

Тривиальная структура — это структура, элементами которой не являются связи между элементами этой структуры.

1.6.2 Понятие связной структуры

Связная структура — это структура, удаление элемента из которой приводит к нарушению целостности этой структуры.

1.7 Понятие соответствия

Соответствие — это ориентированное бинарное отношение, заданное на некоторых множествах и содержащее связи, первые и вторые элементы которых являются элементами этих множеств соответственно.

1.7.1 Понятие области отправления соответствия

Область отправления соответствия — это множество, которое является первым компонентом пары соответствия.

1.7.2 Понятие области прибытия соответствия

Область прибытия соответствия — множество, которое является вторым компонентом пары соответствия.

1.7.3 Понятие прообраза

Прообраз — это первый элемент пары в рамках множества пар, которое является вторым компонентом отношения соответствия.

1.7.4 Понятие образа

Образ — это второй элемент пары в рамках множества пар, которое является вторым компонентом отношения соответствия.

Пример. Пусть A — множество людей, а B — то же самое множество людей. Зададим соответствие $R \subseteq A \times B$ так: $\langle a, b \rangle \in R$, если a является родителем b . Тогда:

- область отправления соответствия R — множество A (все люди);
- область прибытия соответствия R — множество B (все люди);
- пара $\langle a, b \rangle$ в R означает, что a — прообраз, b — образ.

1.7.5 Классификация соответствий

Соответствия бывают:

- *однозначными и неоднозначными;*
- *сюръективными и несюръективными;*
- *всюду определёнными и частично определёнными.*

1.7.5.1 Понятие однозначного соответствия

Однозначное соответствие — это соответствие, при котором каждому элементу из области отправления соответствия ставится не более чем один элемент из области прибытия соответствия.

Пример. Пусть A — множество сотрудников компании, а B — множество их персональных идентификаторов (ИД). Связываем каждого сотрудника с его ИД. Если каждому сотруднику соответствует не более одного ИД, то это однозначное соответствие. Если каждому сотруднику сопоставлен ровно один ИД и разным сотрудникам соответствуют разные ИД, мы получаем инъективное однозначное соответствие.

Однозначные соответствия делятся на *инъективные* и *обратимые соответствия*.

1.7.5.1.1 Понятие инъективного соответствия

Инъективное соответствие — это однозначное соответствие, при котором разным элементам из области отправления соответствия всегда соответствуют разные элементы из области прибытия соответствия и наоборот.

Пример. Пусть $A = \{1, 2, 3\}$, $B = \{2, 4, 6\}$, и задано соответствие $f: A \rightarrow B$ по правилу $f(1) = 2$, $f(2) = 4$, $f(3) = 6$. Тогда разным элементам A соответствуют разные элементы B . Это инъективное соответствие.

1.7.5.1.2 Понятие обратимого соответствия

Обратимое соответствие — это однозначное соответствие, для которого обратное соответствие является однозначным соответствием.

Пример. Рассмотрим предыдущее соответствие $f: A \rightarrow B$, $A = \{1, 2, 3\}$, $B = \{2, 4, 6\}$. Можно задать обратное соответствие $g: B \rightarrow A$: $2 \mapsto 1$, $4 \mapsto 2$, $6 \mapsto 3$. Поскольку f и g однозначны, исходное соответствие является обратимым.

1.7.5.2 Понятие неоднозначного соответствия

Неоднозначное соответствие — это соответствие, при котором хотя бы одному элементу из области отправления соответствия ставится более чем один элемент из области прибытия соответствия.

Пример. В соответствии "преподаватель — курсы, которые он ведёт" одному преподавателю могут соответствовать несколько курсов. Тогда для некоторого элемента области отправления (конкретного преподавателя) существует два и более образа (разные курсы). Это делает соответствие неоднозначным.

1.7.5.3 Понятие сюръективного соответствия

Сюръективное соответствие — это соответствие, при котором существует прообраз для каждого элемента области прибытия соответствия.

Пример. Пусть A — множество студентов, а B — множество тем дипломных работ. Если каждой теме назначен хотя бы один студент (для каждой темы существует прообраз), то соответствие "студент — выбранная тема" будет сюръективным.

1.7.5.4 Понятие несюръективного соответствия

Несюръективное соответствие — это соответствие, при котором не для каждого элемента области прибытия соответствия существует прообраз.

1.7.5.5 Понятие всюду определённого соответствия

Всюду определённое соответствие — это соответствие, при котором существует образ для каждого элемента области отправления соответствия.

Пример. Соответствие "студент — номер зачётной книжки" обычно всюду определено: у каждого студента есть хотя бы один номер зачётной книжки.

1.7.5.6 Понятие частично определённого соответствия

Частично определенное соответствие — это соответствие, при котором существует образ для некоторых, но не всех элементов области отправления соответствия.

Пример. Соответствие "студент — оценка за текущий экзамен" на момент экзамена может быть определено только для тех студентов, которые уже успели сдать. Для остальных образ ещё не задан. Это частично определённое соответствие.

1.7.5.7 Понятие взаимно однозначного соответствия

Взаимно однозначное соответствие — это всюду определённое инъективное сюръективное соответствие.

Пример. Нумерация мест в аудитории: A — множество физических мест, B — множество их номеров. Если каждому месту поставлен в соответствие ровно один номер и каждому номеру соответствует ровно одно место, то соответствие $A \leftrightarrow B$ является взаимно однозначным (биекцией).

1.7.5.8 Понятие обратного соответствия

Обратное соответствие для заданного соответствия — это соответствие, область отправления которого является областью прибытия заданного соответствия, а область прибытия является областью отправления заданного соответствия.

Пример. Для соответствия "сотрудник — его кабинет" обратное соответствие будет "кабинет — сотрудник, который в нём работает". Область отправления и прибытия меняются местами.

1.7.5.9 Понятие функции

Функция — это однозначное соответствие.

1.7.6 Специализированные виды соответствий

Среди всех соответствий можно выделить в отдельные категории такие виды соответствий, как:

- *аналогичность структур*;
- *гомоморфность структур*;
- *полиморфность структур*.

1.7.6.1 Понятие аналогичности структур

Аналогичность структур — соответствие, задаваемое на структурах, и фиксирующее факт наличия некоторой аналогии на подструктурах (подмножествах) указанных структур.

Каждой ориентированной паре, принадлежащей аналогичности структур может быть поставлено в соответствие множество пар, задающих сходства некоторых подструктур и различия некоторых подструктур исходных структур.

1.7.6.2 Понятие гомоморфности структур

Гомоморфность структур, или *гомоморфное соответствие* — это соответствие, заданное на структурах, при котором каждому элементу из области отправления соответствия (первой структуры) ставится в соответствие только один элемент из области прибытия соответствия (второй структуры).

Частным случаем *гомоморфности структур* является *изоморфность структур*.

1.7.6.3 Понятие изоморфности структур

Изоморфность структур, или *изоморфное соответствие* — это гомоморфность структур, при которой для каждого элемента из области прибытия соответствия существует ровно один соответствующий элемент из области отправления соответствия.

Частным случаем *изоморфности структур* является *автоморфность структур*.

1.7.6.4 Понятие автоморфности структур

Аutomорфность структур, или автоморфное соответствие — это изоморфность структур, у которой область отправления соответствия и область прибытия соответствия совпадают.

1.7.6.5 Понятие полиморфности структур

Полиморфность структур, или полиморфное соответствие — это соответствие, заданное на структурах, при котором каждому элементу из области отправления соответствия (первой структуры) ставится в соответствие один или более элемент из области прибытия соответствия (второй структуры), при этом существует хотя бы один элемент области отправления соответствия, которому соответствуют два или более элемента из области прибытия соответствия.

2 ОСНОВЫ ОБЩЕЙ АЛГЕБРЫ В ИНТЕЛЛЕКТУАЛЬНЫХ СИСТЕМАХ

Общая алгебра — это раздел математики, предметом изучения которой являются множества сущностей с заданными на них операциями.

Для интеллектуальных систем общая алгебра важна тем, что она обеспечивает формальные модели для представления и обработки знаний.

В контексте современных информационных технологий именно общая алгебра даёт язык для описания манипуляций с нечисловыми структурами данных: деревьями разбора, графами, онтологиями, логическими формулами, наборами правил и ограничений. Алгебраические методы лежат в основе многих формальных моделей данных (реляционная модель, алгебра графов, алгебра событий), а также методов спецификации и верификации программ и протоколов. Переход от «чистых вычислений» к обработке знаний и сложных структур неизбежно приводит к алгебраическим абстракциям, поскольку именно они позволяют описывать операции над такими объектами единообразно, независимо от их конкретной реализации в памяти компьютера.

2.1 Понятие алгебры

Алгебра — это ориентированная пара A , первым элементом которой является некоторое непустое множество M , а вторым элементом которой является множество алгебраических операций $\Omega = \{\varphi_1, \varphi_2, \dots, \varphi_m\}$, заданных на множестве M .

Пример. Булева алгебра $A = \langle \{0, 1\}, \{\wedge, \vee, \neg\} \rangle$, где $M = \{0, 1\}$ — носитель этой алгебры, $\Omega = \{\wedge, \vee, \neg\}$ — сигнатура этой алгебры.

2.1.1 Понятие носителя алгебры

Носитель алгебры (или *основное (несущее) множество алгебры*) — это множество, которое является первым элементом заданной алгебры.

Пример. В алгебре строк M — множество всех строк над алфавитом $\{a, b\}$, например, $M = \{\varepsilon, a, b, aa, ab, \dots\}$, где ε — пустая строка.

2.1.2 Понятие сигнатуры алгебры

Сигнатура алгебры — это множество, которое является вторым элементом заданной алгебры; то есть множество алгебраических операций заданной алгебры.

Пример. Сигнатура $\Omega = \{+, \cdot\}$, где $+$ — бинарная операция сложения, $\cdot$ — бинарная операция умножения.

2.2 Понятие алгебраической операции

Алгебраическая операция, заданная на множестве M — это операция, областью отправления (определения) которой является декартова степень некоторого заданного множества A , а областью прибытия (значений) которой является множество A :

$$\varphi : M^n \rightarrow M.$$

2.2.1 Понятие арности алгебраической операции

Арность алгебраической операции — это мощность области отправления этой алгебраической операции.

2.2.1.1 Понятие типа алгебры

Тип алгебры — это ориентированная связка арностей алгебраических операций сигнатуры заданной алгебры:

$$\tau = \langle n_{\varphi_1}, \dots, n_{\varphi_m} \rangle.$$

где n_{φ_i} — арность операции φ_i .

Тип алгебры фиксирует «формат» операций, но не их конкретное поведение; две алгебры с одинаковым типом могут иметь разные носители и разные по сути операции, но совпадает количество аргументов у каждой операции сигнатуры.

Пример. Алгебра $\langle \mathbb{R}, \{+, \cdot\} \rangle$ действительных чисел с операциями сложения и умножения имеет тип $\langle 2, 2 \rangle$, поскольку обе операции бинарные.

2.2.2 Понятие аргумента алгебраической операции

Аргумент алгебраической операции — это элемент связки области отправления этой алгебраической операции, то есть конкретный элемент кортежа $\langle x_1, \dots, x_n \rangle \in M^n$, к которому применяется операция φ .

Для бинарной операции $\varphi : M^2 \rightarrow M$ аргументами являются первый и второй операнды; для унарной операции $\varphi : M \rightarrow M$ — единственный операнд.

2.2.3 Классификация алгебраических операций

Алгебраические операции классифицируют по ариности: различают унарные (одноместные), бинарные (двуместные) и в общем случае n -арные операции; операции ариности, отличной от 2, удобно объединять термином «небинарные». Кроме того, для бинарных операций выделяют важные алгебраические свойства (ассоциативность, коммутативность, дистрибутивность, идемпотентность и др.), которые определяют тип алгебраической структуры и поведение её операций при комбинировании элементов.

2.2.3.1 Понятие унарной алгебраической операции

Унарная алгебраическая операция — это алгебраическая операция, ариность которой равна 1, то есть отображение вида $\varphi : M \rightarrow M$.

2.2.3.2 Понятие бинарной алгебраической операции

Бинарная алгебраическая операция — это алгебраическая операция, ариность которой равна 2, то есть отображение вида $\varphi : M \times M \rightarrow M$.

2.2.3.2.1 Классификация бинарных алгебраических операций

Бинарные алгебраические операции могут обладать различными алгебраическими свойствами. Наиболее важными являются ассоциативность, коммутативность, дистрибутивность относительно другой операции и идемпотентность. Эти свойства лежат в основе определения алгебраических структур (полугрупп, моноидов, колец, решёток, булевых алгебр и др.) и определяют, какие тождества выполняются для всех элементов носителя.

2.2.3.2.1.1 Понятие ассоциативной бинарной алгебраической операции

Ассоциативная бинарная алгебраическая операция — это бинарная алгебраическая операция φ , для которой для всех a, b, c , являющихся элементами носителя этой операции, выполняется тождество $(a \varphi b) \varphi c = a \varphi (b \varphi c)$.

Это означает, что при последовательном применении операции к трём элементам порядок группировки не влияет на результат.

2.2.3.2.1.2 Понятие коммутативной бинарной алгебраической операции

Коммутативная бинарная алгебраическая операция — это бинарная алгебраическая операция φ , для которой для всех a, b , являющихся элементами носителя этой операции, выполняется тождество $a \varphi b = b \varphi a$.

Это означает, что перестановка аргументов не изменяет результат операции.

2.2.3.2.1.3 Понятие дистрибутивной бинарной алгебраической операции по отношению к заданной бинарной операции

Дистрибутивная бинарная алгебраическая операция φ по отношению к заданной бинарной операции ψ — это бинарная алгебраическая операция, для которой для всех a, b, c , являющихся элементами носителя этой операции, выполняется тождество $a \varphi (b \psi c) = (a \varphi b) \psi (a \varphi c)$.

В этом случае говорят, что φ дистрибутивна относительно ψ слева; аналогично можно определить дистрибутивность справа. Дистрибутивность описывает согласованность двух операций и играет ключевую роль, например, в определении кольца (умножение дистрибутивно относительно сложения) и булевой алгебры (конъюнкция дистрибутивна относительно дизъюнкции и наоборот).

2.2.3.2.1.4 Понятие идемпотентной бинарной алгебраической операции

Идемпотентная бинарная алгебраическая операция — это бинарная алгебраическая операция φ , для которой для каждого a , являющегося элементом носителя этой операции, выполняется тождество $a \varphi a = a$.

Интуитивно это означает, что «повторное применение» операции к одному и тому же элементу не меняет его: результат совпадает с исходным аргументом. Примеры: операции объединения и пересечения множеств, операции $\vee$ и $\wedge$ в булевой алгебре. В отличие от этого, сложение и умножение чисел не являются идемпотентными (обычно $a + a \neq a$ и $a \cdot a \neq a$).

2.2.3.3 Понятие небинарной алгебраической операции

Небинарная алгебраическая операция — это алгебраическая операция, арность которой равна 1 или больше чем 2.

К небинарным обычно также относят нулевые (0-арные) операции, то есть выделенные элементы носителя, интерпретируемые как константы алгебры. В универсальной алгебре унарные и бинарные операции рассматриваются особенно часто, но при моделировании сложных структур (например, операций над отношениями или деревьями) естественно возникают и операции более высоких арностей.

2.3 Понятие группоида

Группоид — это алгебра $A = \langle M, \{\cdot\} \rangle$, сигнатура которой состоит из одной бинарной алгебраической операции.

Иными словами, на непустом множестве M задана одна операция $\cdot : M \times M \rightarrow M$.

Пример 1. $A = \langle \mathbb{N}, \{-\} \rangle$, где $-$ — вычитание натуральных чисел.

Пример 2. $A = \langle \mathbb{R} \setminus \{0\}, \{\div\} \rangle$, где $\div$ — деление.

2.4 Понятие полгруппы

Полугруппа — это группоид $A = \langle M, \{\cdot\} \rangle$, чья единственная бинарная операция ассоциативна.

То есть для всех $a, b, c \in M$ выполняется тождество $(a \cdot b) \cdot c = a \cdot (b \cdot c)$. Ассоциативность позволяет однозначно записывать произведения нескольких элементов без скобок ($a \cdot b \cdot c \cdot d$), что делает полугруппы естественной моделью последовательного «умножения» или композиции действий.

Пример 1. $\langle \Sigma^+, ++ \rangle$ — множество непустых строк над алфавитом Σ с конкатенацией является полугруппой, так как конкатенация ассоциативна.

Пример 2. $\langle \mathbb{N} \setminus \{0\}, \cdot \rangle$ — натуральные числа без нуля с умножением является полугруппой, так как умножение ассоциативно.

2.5 Понятие моноида

Моноид — это полугруппа $A = \langle M, \{\cdot\} \rangle$, в носителе которой существует нейтральный элемент e относительно операции $\cdot$.

Это означает, что для всех $a \in M$ выполняются тождества $e \cdot a = a$ и $a \cdot e = a$. Наличие ассоциативной операции и нейтрального элемента позволяет рассматривать моноиды как абстрактную модель «умножения» с единицей.

Пример 1. $\langle \Sigma^*, ++ \rangle$ — множество всех строк над заданным алфавитом Σ с операцией конкатенации $++$ и нейтральным элементом ε (" abc " $++$ $\varepsilon = \varepsilon ++$ " abc " = " abc ").

Пример 2. $\langle \mathbb{N}, + \rangle$ — множество натуральных чисел с операцией сложения $+$ и нейтральным элементом 0 .

Пример 3. $\langle \mathbb{Z}, \cdot \rangle$ — множество целых чисел с операцией умножения $\cdot$ и нейтральным элементом 1 .

2.6 Понятие группы

Группа — это моноид $A = \langle M, \{\cdot\} \rangle$, в котором каждый элемент $a \in M$ имеет обратный элемент a^{-1} относительно операции $\cdot$.

Это значит, что для каждого a существует такой элемент $a^{-1} \in M$, что $a \cdot a^{-1} = e$ и $a^{-1} \cdot a = e$, где e — нейтральный элемент группы.

Таким образом, группа характеризуется тремя свойствами: ассоциативностью операции, существованием нейтрального элемента и существованием обратных элементов.

Пример 1. $\langle \mathbb{Z}, + \rangle$ — множество всех целых чисел с операцией сложения $+$ и нейтральным элементом 0 , где обратным элементом к n является $-n$ ($n + (-n) = 0$).

Пример 2. $\langle \mathbb{R} \setminus \{0\}, \cdot \rangle$ — множество всех рациональных чисел с операцией умножения $\cdot$ и нейтральным элементом 1 , где обратным элементом к x является $1/x$.

2.7 Понятие кольца

Кольцо — это алгебраическая структура $A = \langle R, \{+, \cdot\} \rangle$, в которой:

- $\langle R, + \rangle$ является абелевой группой (операция сложения ассоциативна, коммутативна, имеет нейтральный элемент 0 и для каждого элемента существует аддитивно обратный);
- $\langle R, \cdot \rangle$ является полугруппой (операция умножения ассоциативна);

- умножение дистрибутивно относительно сложения: для всех $a, b, c \in R$ выполняются тождества $a \cdot (b + c) = (a \cdot b) + (a \cdot c)$ и $(b + c) \cdot a = (b \cdot a) + (c \cdot a)$.

Пример 1. $\langle \mathbb{Z}, \{+, \cdot\} \rangle$ — коммутативное кольцо с единицей (1).

Пример 2. $\langle M_n(\mathbb{R}), \{+, \cdot\} \rangle$ — кольцо квадратных матриц $n \times n$.

Пример 3. $\langle \mathbb{Z}/n\mathbb{Z}, \{+, \cdot\} \rangle$ — кольцо вычетов по модулю n .

2.8 Понятие поля

Поле — это коммутативное кольцо $A = \langle K, \{+, \cdot\} \rangle$ с единицей, в котором всякий ненулевой элемент обратим относительно умножения, то есть в котором:

- $\langle K, + \rangle$ — абелева группа с нейтральным элементом 0;
- $\langle K \setminus \{0\}, \cdot \rangle$ — абелева группа с нейтральным элементом 1;
- умножение дистрибутивно относительно сложения.

Пример 1. $\langle \mathbb{Q}, \{+, \cdot\} \rangle, \langle \mathbb{R}, \{+, \cdot\} \rangle, \langle \mathbb{C}, \{+, \cdot\} \rangle$ — числовые поля.

Пример 2. $\langle \{0, 1\}, \{\oplus, \wedge\} \rangle$ — поле Галуа из двух элементов, лежащее в основе двоичной арифметики и помехоустойчивого кодирования.

2.9 Понятие алгебраической системы

Алгебраическая система — это система, представляющая собой ориентированную тройку, первым элементом которой является некоторое непустое множество M , вторым элементом которой является множество отношений $P = \{R_1, R_2, \dots, R_m\}$, заданное на множестве M , а третьим элементом является множество алгебраических операций $\Omega = \{\varphi_1, \varphi_2, \dots, \varphi_m\}$, заданных на множестве M .

Пример 1. Упорядоченная группа $\langle \mathbb{Z}, \{\leq\}, \{+\} \rangle$: носитель — $\mathbb{Z}$, отношение — $\leq$ (линейный порядок), операция — $+$ (сложение).

Пример 2. Реляционная база данных: носитель — множество кортежей, отношения — ограничения целостности (первичный ключ, внешний ключ), операции — проекция, выборка, соединение.

2.9.1 Понятие носителя алгебраической системы

Носитель алгебраической системы — это множество, которое является первым элементом заданной алгебраической системы.

Пример 1. В алгебраической системе $\langle \mathbb{R}, \{<\}, \{+, \cdot\} \rangle$ носителем является $\mathbb{R}$ — множество действительных чисел.

Пример 2. В системе $\langle V(G), \{\sim\}, \{\cup\} \rangle$ (граф G) носителем является $V(G)$ — множество вершин графа G .

2.9.2 Понятие сигнатуры алгебраической системы

Сигнатура алгебраической системы — это множества, которые являются вторым и третьим элементами заданной алгебраической системы.

2.10 Частично упорядоченные множества

Частично упорядоченное множество (ЧУМ) — это алгебраическая система $\langle M, \{\leq\}, \Omega \rangle$, где $\leq$ — бинарное отношение на M , удовлетворяющее трём аксиомам:

- рефлексивность: $a \leq a$ для всех $a \in M$;
- антисимметричность: если $a \leq b$ и $b \leq a$, то $a = b$;
- транзитивность: если $a \leq b$ и $b \leq c$, то $a \leq c$.

ЧУМ вводится как формализация понятия «иерархии» или «порядка»: в интеллектуальных системах понятийные иерархии (онтологии), цепочки зависимостей задач и отношения включения классов естественно описываются частичным порядком.

Пример 1. $\langle 2^U, \subseteq \rangle$ — множество всех подмножеств U , упорядоченное по включению: $\{a\} \subseteq \{a, b\} \subseteq \{a, b, c\}$.

Пример 2. В онтологии OWL иерархия классов образует ЧУМ: $\text{Animal} \leq \text{LivingBeing}$, $\text{Mammal} \leq \text{Animal}$, $\text{Dog} \leq \text{Mammal}$.

2.11 Понятие решётки

Решётка — это алгебра $\langle L, \{\vee, \wedge\} \rangle$, в которой определены две бинарные ассоциативные, коммутативные и идемпотентные операции $\vee$ (sup, объединение) и $\wedge$ (inf, пересечение), связанные законами поглощения:

$$a \vee (a \wedge b) = a \text{ и } a \wedge (a \vee b) = a.$$

Решётка вводится как алгебраическое обобщение ЧУМ, в котором для любых двух элементов существуют точные верхняя (sup) и нижняя (inf) грани. Это позволяет формально описывать иерархии понятий в описательных логиках

(Formal Concept Analysis (FCA) — анализ формальных понятий) и семантику типов в языках программирования (решётка типов с $\top = \text{Any}$ и $\perp = \text{Nothing}$).

Пример 1. $\langle 2^U, \{\cup, \cap\} \rangle$ — булева решётка: $\vee = \cup$, $\wedge = \cap$, нижняя грань = $\emptyset$, верхняя = U .

Пример 2. В системе типов языка Scala: $\text{String} \wedge \text{Int} = \text{Nothing}$ (нижний элемент), $\text{String} \vee \text{Int} = \text{Any}$ (верхний элемент).

Пример 3. В FCA: решётка формальных понятий описывает все возможные обобщения признаков объектов предметной области.

2.12 Гомоморфизм и изоморфизм алгебр

Гомоморфизм позволяет сравнивать и отображать одну алгебраическую структуру в другую, сохраняя действие операций. Если известно, что некоторая алгебра A обладает определённым свойством, и существует гомоморфизм $A \rightarrow B$, то ряд свойств автоматически наследуется алгеброй B .

2.12.1 Понятие гомоморфизма алгебр

Пусть $A = \langle M, \Omega \rangle$ и $A' = \langle M', \Omega' \rangle$ — две алгебры одного типа τ . Отображение $h : M \rightarrow M'$ называется гомоморфизмом из A в A' , если для каждой n -арной операции $\phi \in \Omega$ и соответствующей $\phi' \in \Omega'$ и для всех $a_1, \dots, a_n \in M$ выполняется:

$$h(\phi(a_1, \dots, a_n)) = \phi'(h(a_1), \dots, h(a_n)).$$

Иными словами, h «переставляется» с операциями: сначала применить операцию, затем отобразить — то же самое, что сначала отобразить аргументы, затем применить операцию в образе. Гомоморфизмы позволяют строить «упрощённые» модели: образ гомоморфизма сохраняет операционную структуру исходной алгебры, что используется в компиляторах (отображение абстрактного синтаксического дерева в промежуточное представление).

Пример 1. Функция $h : \langle \mathbb{Z}, + \rangle \rightarrow \langle \mathbb{Z}/n\mathbb{Z}, + \rangle$, $h(a) = a \bmod n$ — гомоморфизм групп: $h(a + b) = (a + b) \bmod n = (a \bmod n) + (b \bmod n) = h(a) + h(b)$.

Пример 2. Функция $\text{length} : \langle \Sigma^*, ++ \rangle \rightarrow \langle \mathbb{N}, + \rangle$ — гомоморфизм моноидов: $\text{length}(s ++ t) = \text{length}(s) + \text{length}(t)$.

2.12.2 Понятие изоморфизма алгебр

Изоморфизм алгебр — это биективный гомоморфизм $h : M \rightarrow M'$, обратное отображение h^{-1} которого также является гомоморфизмом.

Две алгебры, между которыми существует изоморфизм, называются *изоморфными* ($A \cong A'$) и считаются «одинаковыми» с алгебраической точки зрения.

Пример 1. $\langle \mathbb{Z}, + \rangle \cong \langle 2\mathbb{Z}, + \rangle$ (чётные целые числа), изоморфизм $h(n) = 2n$.

2.13 Реляционная алгебра

Реляционная алгебра — это алгебра $A = \langle \text{Rel}, \Omega \rangle$, носителем которой является множество Rel всех отношений (таблиц) над некоторой схемой базы данных, а сигнатура включает операции:

$\sigma(R, \varphi)$ — операция выборки, которая возвращает кортежи отношения R, удовлетворяющие условию φ ;

$\pi(R, A)$ — операция проекции, которая возвращает отношение, содержащее только столбцы из A;

$R \Join S$ — операция соединения;

$R \cup S$ — операция объединения;

$R - S$ — операция разности;

$R \times S$ — операция декартова произведения.

Пример. Запрос "найти имена сотрудников отдела 10": $\pi_{\{\text{Имя}\}}(\sigma_{\{\text{Отдел}=10\}}(\text{Сотрудники}))$. Запрос "найти имена сотрудников с их проектами": $\pi_{\{\text{Имя}, \text{Проект}\}}(\text{Сотрудники} \Join \text{Назначения})$.

3 ОСНОВЫ ТЕОРИИ ФОРМАЛЬНЫХ ЯЗЫКОВ В ИНТЕЛЛЕКТУАЛЬНЫХ СИСТЕМАХ

Теория формальных языков — это раздел математики, предметом изучения которого являются математические модели формальных языков.

Теория формальных языков лежит в основе современных методов представления и обработки знаний в интеллектуальных системах.

Понятие формального языка является фундаментальным понятием теории формальных языков.

Любой формальный язык является искусственно созданным языком.

3.1 Понятие языка

Язык — это знаковая система; или система, состоящая из знаков.

3.1.1 Понятие искусственного языка

Искусственный язык — это язык, созданный человеком.

Многие искусственные языки создаются для компьютерных систем.

3.2 Понятие знака

Знак (имя, символ, изображение), или обозначающее («обладать смыслом, иметь значение, быть знаком») — это атомарная единица *знаковой конструкции* некоторого языка, которая обозначает (описывает) некоторую (материальную или абстрактную) сущность.

Говорят, что знак является обозначением некоторой материальной или абстрактной сущности.

3.2.1 Понятие денотата знака

Денотат знака (от лат. denotatum — «обозначенное»), или *референт знака* (от лат. referens — «относящийся, сопоставляющий»), или *обозначаемое* («сделать имеющим значение, пометить знаком, указать, определить») — это сущность, обозначаемая этим знаком.

3.2.2 Понятие сигнификата знака

Сигнификат знака (от лат. significātum — «значимое»), или *десигнат знака* (от лат. designatum — «означаемое»), или *означаемое* («наделять знаком, помечать знаком, иметь определённый смысл») — это множество признаков, характеризующих денотат этого знака.

3.3 Понятие знаковой конструкции

Знаковая конструкция (строка) — это конструкция (структура), состоящая из знаков.

3.3.1 Понятие семиотики

Семиотика, или *семиология* (от греч. σημειωτική — «знак; признак») — это раздел лингвистики, исследующий свойства знаков и знаковых систем.

3.3.1.1 Понятие синтактики

Синтактика (греч. σύνταξις — «построение, порядок») — это раздел семиотики, исследующий связи между знаками.

3.3.1.2 Понятие семантики

Семантика (от др.-греч. σημαντικός «обозначающий») — это раздел лингвистики, исследующий связи между знаками и их значениями.

3.3.1.3 Понятие прагматики

Прагматика (от др.-греч. πράγμα — «дело, действие») — это раздел лингвистики, исследующий связи между знаками и субъектами, которые их используют.

3.4 Понятие формального языка

Формальный язык — это *искусственный язык*, представляющий собой множество последовательностей знаков (символов) (*знаковых конструкций (строк)*), построенных из знаков (символов) определённого алфавита, объединённых *синтаксическими правилами*, которые однозначно задаются *синтаксисом* этого языка.

Формальный язык является знаковой системой, полностью определённой синтаксически, без обращения к семантике.

3.4.1 Понятие алфавита формального языка

Алфавит формального языка — это конечное множество различных знаков (символов), используемых для построения знаковых конструкций (строк) в этом языке.

3.4.2 Понятие синтаксиса формального языка

Синтаксис (синтактика) формального языка — это множество синтаксических отношений, определяющих, какие последовательности знаков (символов) алфавита этого формального языка (знаковые конструкции (строки)) считаются правильными (принадлежат этому формальному языку).

Синтаксические отношения задаются при помощи грамматики формального языка.

3.4.3 Понятие грамматики формального языка

Грамматика формального языка — это множество синтаксических правил, определяющих, как используются синтаксические отношения между знаками (символами) алфавита этого языка.

3.4.3.1 Понятие формы Бэкуса-Наура

Форма Бэкуса–Наура — это метаязык для описания контекстно-свободных грамматик.

Она широко используется в информатике для точного определения синтаксиса формальных языков.

3.4.4 Понятие семантики формального языка

Семантика формального языка — это множество семантических отношений (интерпретаций), придающих смысл построенным конструкциям формального языка, то есть связывающих знаковые конструкции формального языка с их значениями.

3.4.5 Понятие денотационной семантики формального языка

Денотационная семантика формального языка — это семантика, которая задаёт отображение синтаксической структуры знаковых конструкций формального языка в их семантически эквивалентные смысловые представления.

3.4.6 Понятие операционной семантики формального языка

Операционная семантика формального языка — это семантика, которая задаёт множество правил трансформации знаковых конструкций формального языка.

4 ОСНОВЫ МАТЕМАТИЧЕСКОЙ ЛОГИКИ В ИНТЕЛЛЕКТУАЛЬНЫХ СИСТЕМАХ

Логика — это наука, предметом изучения которой являются общие схемы правильного человеческого мышления.

Математическая логика — это раздел математики, использующий математические модели и методы для изучения предмета логики. Математическая логика изучает логические формы мышления и такие понятия, как истина и ложь, с помощью логических констант, логических переменных, логических связок, логических функций и операций.

В современной математической логике принято выделять, по крайней мере, два крупных направления. Классическая логика опирается на двухзначную (булеву) семантику и включает, в частности, логику предикатов и как её частный случай — логику высказываний. В рамках классической логики принимаются законы непротиворечия и исключённого третьего, а также стандартное понимание отрицания, конъюнкции, дизъюнкции, импликации и эквиваленции. Наряду с этим развиваются различные неклассические логики (многозначные, модальные, интуиционистские, релевантные и др.), в которых те или иные классические законы ослабляются или модифицируются для адекватного моделирования неопределённости, неполноты информации, временных и модальных аспектов рассуждений.

В интеллектуальных системах математическая логика применяется как базовый инструмент представления знаний и вывода новых знаний.

4.1 Логика высказываний

Логика высказываний, или *пропозициональная логика* — это раздел математической логики, предметом изучения которого являются высказывания и их свойства.

Логика высказываний является наиболее простым и фундаментальным разделом математической логики: она оперирует атомарными высказываниями и правилами их комбинирования посредством логических связок. Любое более сложное логическое исчисление — логика предикатов, модальные логики и др. — строится на основе логики высказываний.

4.1.1 Язык логики высказываний

Язык логики высказываний — это формальный язык, конструкциями (строками) которого являются высказывания.

Язык логики высказываний является подмножеством операции замыкания Клини Σ^* над его алфавитом Σ без пустых строк. Операция замыкания Клини порождает все конечные строки символов этого алфавита.

4.1.2 Понятие высказывания

В естественном языке высказывание — это повествовательное предложение, относительно которого имеет смысл говорить, что оно истинно или ложно (например: « $2 + 2 = 4$ », «Сейчас идёт дождь»). Вопросы, приказания и восклицания высказываниями не являются.

В формальной логике удобно отделять содержательный уровень от синтаксического (формы).

Формально, *высказывание* — это структура, в состав которой входят константы из предметной области заданной формальной теории, или связка, элементами которой являются другие высказывания. Такая структура задаётся *логической формулой языка логики высказываний*. *Логические формулы языка логики высказываний* служат символической записью высказываний.

Содержательно, *высказывание* — это утверждение, которое истинно или ложно.

4.1.3 Алфавит языка логики высказываний

Алфавит языка логики высказываний можно задать при помощи формы Бэкуса-Наура:

<логическая константа> ::= T | $\perp$

<ненулевая цифра> ::= 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

<символ> ::= A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z

<цифра> ::= 0 | <ненулевая цифра>

<цифры> ::= <цифра> | <цифра> <цифры>

<натуральное число> ::= <ненулевая цифра> | <ненулевая цифра> <цифры>

<логическая связка> ::= <унарная логическая связка> | <бинарная логическая связка>

<унарная логическая связка> ::= <отрицание>

<отрицание> ::= $\neg$

<бинарная логическая связка> ::= <дизъюнкция> | <конъюнкция> | <импликация>
| <эквиваленция>

<дизъюнкция> ::= $\vee$
 <конъюнкция> ::= $\wedge$
 <импликация> ::= $\rightarrow$
 <эквиваленция> ::= $\sim$
 <открывающая полукруглая скобка> ::= (
 <закрывающая полукруглая скобка> ::=)

В алфавит языка логики высказываний входят: логические константы ($\top$ — истина, $\perp$ — ложь), пропозициональные переменные ($A, B, \dots, Z$ с натуральными индексами), унарная логическая связка отрицания ($\neg$), бинарные логические связки ($\vee, \wedge, \rightarrow, \sim$), вспомогательные символы — полукруглые скобки (и).

Таким образом, в алфавите языка логики высказываний используется 45 различных символов, образующих знаки этого алфавита.

4.1.4 Грамматика языка логики высказываний

Грамматика языка логики высказываний определяет правила построения правильно сформированных *логических формул* из знаков алфавита языка логики высказываний.

Грамматику языка логики высказываний также можно задать при помощи формы Бэкуса-Наура:

<логическая формула> ::= <логическая константа>
 | <атомарная логическая формула>
 | <неатомарная логическая формула>
 <атомарная логическая формула> ::= <символ> | <символ> <натуральное>
 <неатомарная логическая формула> ::= <унарная неатомарная логическая формула>
 | <бинарная неатомарная логическая формула>
 <унарная неатомарная логическая формула> ::= <открывающая круглая скобка>
 <унарная логическая связка>
 <логическая формула>
 <закрывающая круглая скобка>
 <бинарная неатомарная логическая формула> ::= <открывающая круглая скобка>
 <логическая формула>
 <бинарная логическая связка>
 <логическая формула>
 <закрывающая круглая скобка>

Примеры логических формул *языка логики высказываний*:

- P ,
- Q_1 ,
- $(\neg P)$,
- $((P \vee Q) \rightarrow R)$.

4.1.5 Классификация логических формул в языке логики высказываний

В языке логики высказываний *логические формулы* бывают *атомарными* и *неатомарными*.

4.1.5.1 Понятие атомарной логической формулы

Атомарная логическая формула — это логическая формула, которая не является неатомарной логической формулой.

Часто *атомарную логическую формулу* в языке логики высказываний называют *пропозициональной переменной*.

Пример. Атомарными логическими формулами в языке логики высказываний являются:

- P ,
- Q ,
- Q_1 ,
- R_{10} .

4.1.5.2 Понятие неатомарной логической формулы

Неатомарная логическая формула — это логическая формула, элементами которой являются другие логические формулы.

Неатомарные логические формулы образуются при помощи логических связок.

Пример. Неатомарными логическими формулами являются:

- $(\neg P)$,
- $(Q \vee Q)$,
- $(P \rightarrow (Q_1 \rightarrow Q))$.

4.1.6 Понятие логической связки

Логическая связка — это связка, которая обозначает функцию, определяющую как истинностное значение неатомарного высказывания зависит от истинностных значений его подформул.

4.1.7 Понятие подформулы логической формулы

Подформула логической формулы — это логическая формула, которая является заданной логической формулой или которая является элементом заданной логической формулы или которая является подформулой логической формулы, являющейся элементом заданной логической формулы.

Другими словами, всякая формула F является подформулой самой себя; если формула имеет вид $(\neg G)$, то подформулами считаются сама $(\neg G)$ и все подформулы формулы G ; если формула имеет вид $(G * H)$, где $*$ — одна из бинарных логических связок, то подформулами являются сама $(G * H)$, формулы G и H и все их подформулы.

Пример. Для логической формулы $((P \vee Q) \rightarrow P)$ подформулами являются:

- P ,
- Q ,
- $(P \vee Q)$,
- $((P \vee Q) \rightarrow P)$.

4.2 Алгебра высказываний

Алгебра высказываний рассматривает логические формулы как обозначения булевых функций, то есть функций, аргументы и значения которых принимают только два значения: истина ($\top$) и ложь ($\perp$).

Формально, *алгебра высказываний* — это алгебра $\langle \{\top, \perp\}, \{\neg, \vee, \wedge, \rightarrow, \sim\} \rangle$, где $\{\top, \perp\}$ — носитель этой алгебры (множество истинностных значений), а $\{\neg, \vee, \wedge, \rightarrow, \sim\}$ — сигнатура этой алгебры (набор операций, соответствующих логическим связкам).

4.2.1 Понятие интерпретации логической формулы

Интерпретация логической формулы — это функция, отображающая множество всех атомарных подформул заданной логической формулы на множество $\{\top, \perp\}$ (множество истинностных значений).

Другими словами, интерпретация фиксирует, какие атомарные логические формулы относительно заданной логической формулы в рамках заданной ситуации считаются истинными, а какие ложными.

Пример. Для логической формулы $((P \wedge Q) \rightarrow P)$ возможны 4 интерпретации:

$$\begin{aligned}
I_1 &= \{\langle P, \top \rangle, \langle Q, \top \rangle\}, \\
I_2 &= \{\langle P, \top \rangle, \langle Q, \perp \rangle\}, \\
I_3 &= \{\langle P, \perp \rangle, \langle Q, \top \rangle\}, \\
I_4 &= \{\langle P, \perp \rangle, \langle Q, \perp \rangle\}.
\end{aligned}$$

Их можно представить в виде следующей таблицы:

Интерпретация	P	Q
I_1	$\top$	$\top$
I_2	$\top$	$\perp$
I_3	$\perp$	$\top$
I_4	$\perp$	$\perp$

4.2.2 Понятие истинностной функции

Истинностная функция — это функция, которая является элементом класса $(\{\top, \perp\}^m)^{(\{\top, \perp\}^n)}$, где m и n — натуральные числа.

Каждой логической связке можно поставить в соответствие истинностную функцию.

Пример. Для конъюнкции существует истинностная функция $f: \{\top, \perp\}^2 \rightarrow \{\top, \perp\}$, возвращающая $\top$ только при обоих аргументах $\top$.

4.2.3 Понятие унарной логической связки и понятие бинарной логической связки

Логические связки бывают унарными и бинарными.

Унарная логическая связка применяется к одной логической формуле, *бинарная логическая связка* — соединяет две логические формулы.

4.2.3.1 Понятие унарной логической связки

Унарная логическая связка — это логическая связка, которая является синглетоном и обозначает функцию $f: \{\top, \perp\} \rightarrow \{\top, \perp\}$.

В классической логике высказываний единственной унарной связкой является *отрицание*.

4.2.3.1.1 Понятие отрицания

Отрицание — это неориентированная унарная логическая связка, которая обозначается знаком $\neg$ и задаётся таблицей истинности:

A	($\neg A$)
т	⊥
⊥	т

Логическая формула $(\neg A)$ истинна тогда и только тогда, когда логическая формула A ложна.

4.2.3.2 Понятие бинарной логической связки

Бинарная логическая связка — это логическая связка, которая является парой и обозначает функцию $f: \{\text{т}, \perp\} \times \{\text{т}, \perp\} \rightarrow \{\text{т}, \perp\}$.

4.2.3.2.1 Понятие дизъюнкции

Дизъюнкция, или *логическое ИЛИ* — это неориентированная бинарная логическая связка, которая обозначается знаком $\vee$ и задаётся таблицей истинности:

A	B	($A \vee B$)
т	т	т
т	⊥	т
⊥	т	т
⊥	⊥	⊥

Логическая формула $(A \vee B)$ истинна тогда и только тогда, когда хотя бы одна из её атомарных подформул A , B истинна.

4.2.3.2.2 Понятие конъюнкции

Конъюнкция, или *логическое И* — это неориентированная бинарная логическая связка, которая обозначается знаком $\wedge$ и задаётся таблицей истинности:

A	B	($A \wedge B$)
т	т	т
т	⊥	⊥
⊥	т	⊥
⊥	⊥	⊥

Логическая формула $(A \wedge B)$ истинна тогда и только тогда, когда обе её атомарные подформулы A и B истинны.

4.2.3.2.3 Понятие импликации

Импликация — это ориентированная бинарная логическая связка, которая обозначается знаком $\rightarrow$ и задаётся таблицей истинности:

A	B	$(A \rightarrow B)$
Т	Т	Т
Т	⊥	⊥
⊥	Т	Т
⊥	⊥	Т

Логическая формула $(A \rightarrow B)$ ложна тогда и только тогда, когда логическая формула A истинна, а логическая формула B ложна.

4.2.3.2.4 Понятие эквиваленции

Эквиваленция — это неориентированная бинарная логическая связка, которая обозначается знаком $\sim$ и задаётся таблицей истинности:

A	B	$(A \sim B)$
Т	Т	Т
Т	⊥	⊥
⊥	Т	⊥
⊥	⊥	Т

Логическая формула $(A \sim B)$ истинна тогда и только тогда, когда логические формулы A и B имеют одно и то же истинностное значение.

4.2.4 Классификация логических формул

С точки зрения алгебры высказываний всякая логическая формула задаёт некоторую булеву функцию от своих пропозициональных переменных. В зависимости от того, какие значения принимает эта функция при всех возможных интерпретациях, различают общезначимые, невыполнимые и нейтральные формулы.

4.2.4.1 Понятие общезначимой логической формулы

Общезначимая логическая формула, или *тавтология* — это логическая формула, которая обозначает функцию, равнозначную логической константе Т.

Пример. $(P \vee (\neg P)), (P \rightarrow P), (A \rightarrow (B \rightarrow A))$.

4.2.4.2 Понятие необщезначимой логической формулы

Необщезначимая логическая формула — это логическая формула, которая не является общезначимой логической формулой.

Пример. Логическая формула P является необщезначимой, поскольку при интерпретации $I(P) = \perp$ она ложна.

Необщезначимые логические формулы бывают невыполнимыми или нейтральными.

4.2.4.2.1 Понятие невыполнимой логической формулы

Невыполнимая логическая формула — это логическая формула, которая обозначает функцию, равнозначную логической константе $\perp$.

Пример. $(P \wedge (\neg P)), (P \sim (\neg P))$.

4.2.4.2.2 Понятие нейтральной логической формулы

Нейтральная логическая формула — это логическая формула, которая не является ни общезначимой логической формулой, ни невыполнимой логической формулой.

Пример. $(P \rightarrow Q), (\neg P)$.

4.2.4.3 Понятие выполнимой логической формулы

Выполнимая логическая формула — это логическая формула, которая является либо общезначимой логической формулой, либо нейтральной логической формулой.

4.2.4.4 Примеры построения таблиц истинности

Таблица истинности строится следующим образом: определяется множество всех атомарных подформул (пропозициональных переменных); для n переменных перечисляются все 2^n наборов истинностных значений; для каждого набора последовательно вычисляется значение каждой подформулы, начиная с атомарных и заканчивая всей формулой.

Пример 1. Построить таблицу истинности формулы $(\neg P \vee Q)$.

Формула содержит 2 переменные (P, Q), поэтому таблица имеет $2^2 = 4$ строки. Подформулы в порядке вычисления: P, Q, $(\neg P)$, $((\neg P) \vee Q)$.

Таблица истинности $(\neg P \vee Q)$:

P	Q	$(\neg P)$	$((\neg P) \vee Q)$
т	т	$\perp$	т
т	$\perp$	$\perp$	$\perp$
$\perp$	т	т	т
$\perp$	$\perp$	т	т

Вывод: формула ложна только при $P = \text{т}, Q = \perp$ — является нейтральной (выполнимой, но не тавтологией).

Пример 2. Построить таблицу истинности формулы $(P \vee (\neg P))$.

Формула содержит одну переменную P, таблица имеет $2^1 = 2$ строки.

Таблица истинности $(P \vee (\neg P))$:

P	$(\neg P)$	$(P \vee (\neg P))$
т	$\perp$	т
$\perp$	т	т

Вывод: формула истинна при всех интерпретациях — тавтология.

Пример 3. Построить таблицу истинности формулы $(P \wedge (\neg P))$.

Таблица истинности $(P \wedge (\neg P))$:

P	$(\neg P)$	$(P \wedge (\neg P))$
т	$\perp$	$\perp$
$\perp$	т	$\perp$

Вывод: формула ложна при всех интерпретациях — невыполнимая (закон противоречия).

Пример 4. Построить таблицу истинности формулы $((P \rightarrow Q) \wedge (Q \rightarrow P))$.

Подформулы: P , Q , $(P \rightarrow Q)$, $(Q \rightarrow P)$, $((P \rightarrow Q) \wedge (Q \rightarrow P))$.

Таблица истинности $((P \rightarrow Q) \wedge (Q \rightarrow P))$

P	Q	$(P \rightarrow Q)$	$(Q \rightarrow P)$	$((P \rightarrow Q) \wedge (Q \rightarrow P))$
$\top$	$\top$	$\top$	$\top$	$\top$
$\top$	$\perp$	$\perp$	$\top$	$\perp$
$\perp$	$\top$	$\top$	$\perp$	$\perp$
$\perp$	$\perp$	$\top$	$\top$	$\top$

Вывод: формула является нейтральной.

Пример 5. Построить таблицу истинности формулы $((P \rightarrow Q) \rightarrow P) \rightarrow P$.

Подформулы: P , Q , $(P \rightarrow Q)$, $((P \rightarrow Q) \rightarrow P)$, $((P \rightarrow Q) \rightarrow P) \rightarrow P$.

Таблица истинности $((P \rightarrow Q) \rightarrow P) \rightarrow P$:

P	Q	$(P \rightarrow Q)$	$((P \rightarrow Q) \rightarrow P)$	$((P \rightarrow Q) \rightarrow P) \rightarrow P$
$\top$	$\top$	$\top$	$\top$	$\top$
$\top$	$\perp$	$\perp$	$\top$	$\top$
$\perp$	$\top$	$\top$	$\perp$	$\top$
$\perp$	$\perp$	$\top$	$\perp$	$\top$

Вывод: все строки дают $\top$ — тавтология.

Пример 6. Построить таблицу истинности формулы $((P \vee Q) \wedge (\neg P))$.

Таблица истинности $((P \vee Q) \wedge (\neg P))$:

P	Q	$(\neg P)$	$(P \vee Q)$	$((P \vee Q) \wedge (\neg P))$
$\top$	$\top$	$\perp$	$\top$	$\perp$
$\top$	$\perp$	$\perp$	$\top$	$\perp$
$\perp$	$\top$	$\top$	$\top$	$\top$

$\perp$	$\perp$	$\top$	$\perp$	$\perp$
---------	---------	--------	---------	---------

Вывод: формула является нейтральной.

Пример 7. Построить таблицу истинности формулы $((P \rightarrow Q) \wedge (Q \rightarrow R)) \rightarrow (P \rightarrow R)$.

Формула содержит 3 переменные — таблица имеет $2^3 = 8$ строк.

Таблица истинности:

P	Q	R	$(P \rightarrow Q)$	$(Q \rightarrow R)$	$(P \rightarrow R)$	$((P \rightarrow Q) \wedge (Q \rightarrow R))$	Результат
$\top$	$\top$	$\top$	$\top$	$\top$	$\top$	$\top$	$\top$
$\top$	$\top$	$\perp$	$\top$	$\perp$	$\perp$	$\perp$	$\top$
$\top$	$\perp$	$\top$	$\perp$	$\top$	$\top$	$\perp$	$\top$
$\top$	$\perp$	$\perp$	$\perp$	$\top$	$\perp$	$\perp$	$\top$
$\perp$	$\top$	$\top$	$\top$	$\top$	$\top$	$\top$	$\top$
$\perp$	$\top$	$\perp$	$\top$	$\perp$	$\top$	$\perp$	$\top$
$\perp$	$\perp$	$\top$	$\top$	$\top$	$\top$	$\top$	$\top$
$\perp$	$\perp$	$\perp$	$\top$	$\top$	$\top$	$\top$	$\top$

Вывод: все 8 строк дают $\top$ — тавтология.

4.2.5 Понятие следования логической формулы из другой логической формулы

Логическая формула α следует из логической формулы β (обозначается $\beta \models \alpha$), если во всех интерпретациях, где логическая формула β истинна, логическая формула α также истинна.

Отношение следования является семантическим отношением — $\beta \models \alpha$ означает, что невозможно ситуация, в которой β истинна, а α ложна.

Пример. $P \models (P \vee Q)$, поскольку в каждой интерпретации, где логическая формула P истинна, логическая формула $(P \vee Q)$ также истинна (так как дизъюнкция истинна при истинности хотя бы одного дизъюнкта).

$\models \alpha$ означает, что логическая формула α является тавтологией (общезначимой формулой).

4.2.6 Понятие равносильности двух логических формул

Две логические формулы A и B равносильны (обозначается $A \Leftrightarrow B$, или $A \equiv B$), если они истинны при одних и тех же интерпретациях; иными словами, $A \Leftrightarrow B$ тогда и только тогда, когда $A \models B$ и $B \models A$ одновременно.

Пример. $(P \rightarrow Q) \Leftrightarrow ((\neg P) \vee Q)$.

Отношение равносильности является отношением эквивалентности на множестве логических формул: оно рефлексивно ($A \Leftrightarrow A$), симметрично ($A \Leftrightarrow B \Rightarrow B \Leftrightarrow A$) и транзитивно.

4.2.6.1 Примеры отношений равносильности логических формул

Перечисленные в подпунктах 4.2.5.1.* законы представляют собой конкретные случаи равносильности логических формул, то есть примеры того, как можно заменить одну формулу другой, не изменяя её истинностной функции. Каждая запись вида $\varphi \Leftrightarrow \psi$ означает, что формулы φ и ψ принимают одинаковые истинностные значения при любых интерпретациях входящих в них переменных; эквивалентно, формула $(\varphi \sim \psi)$ является тавтологией.

Семантически равносильность можно обосновать, сравнивая таблицы истинности: если две формулы имеют совпадающие столбцы значений для всех наборов аргументов, они равносильны. Синтаксически тот же факт может быть установлен с помощью цепочки равносильных преобразований (замен подформул по уже известным законам) или как теорема исчисления высказываний.

4.2.6.1.1 Законы ассоциативности

$$((\alpha \wedge \beta) \wedge \gamma) \Leftrightarrow (\alpha \wedge (\beta \wedge \gamma))$$

$$((\alpha \vee \beta) \vee \gamma) \Leftrightarrow (\alpha \vee (\beta \vee \gamma))$$

$$((\alpha \sim \beta) \sim \gamma) \Leftrightarrow (\alpha \sim (\beta \sim \gamma))$$

Ассоциативность показывает, что при повторном применении одной и той же связки порядок объединения слагаемых (расстановка скобок) не влияет на результат.

4.2.6.1.2 Законы коммутативности

$$(\alpha \wedge \beta) \Leftrightarrow (\beta \wedge \alpha)$$

$$(\alpha \vee \beta) \Leftrightarrow (\beta \vee \alpha)$$

$$(\alpha \sim \beta) \Leftrightarrow (\beta \sim \alpha)$$

Коммутативность фиксирует симметрию аргументов бинарной логической связки: перестановка местами α и β не влияет на истинностное значение логической формулы.

4.2.6.1.3 Законы дистрибутивности

$$(\alpha \wedge (\beta \vee \gamma)) \Leftrightarrow ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma))$$

$$(\alpha \vee (\beta \wedge \gamma)) \Leftrightarrow ((\alpha \vee \beta) \wedge (\alpha \vee \gamma))$$

Дистрибутивность описывает, как одна связка «распределяется» относительно другой: конъюнкция дистрибутивна относительно дизъюнкции, и дизъюнкция — относительно конъюнкции.

4.2.6.1.4 Законы идемпотентности

$$(\alpha \wedge \alpha) \Leftrightarrow \alpha$$

$$(\alpha \vee \alpha) \Leftrightarrow \alpha$$

Идемпотентность показывает, что повторение одного и того же высказывания под одной логической связкой не меняет его логического содержания.

4.2.6.1.5 Закон двойного отрицания

$$(\neg(\neg\alpha)) \Leftrightarrow \alpha$$

Закон двойного отрицания утверждает, что отрицание отрицания возвращает исходное истинностное значение.

4.2.6.1.6 Законы де Моргана

$$(\neg(\alpha \wedge \beta)) \Leftrightarrow ((\neg\alpha) \vee (\neg\beta))$$

$$(\neg(\alpha \vee \beta)) \Leftrightarrow ((\neg\alpha) \wedge (\neg\beta))$$

Данные законы устанавливают связь между отрицанием и связками $\wedge$, $\vee$: отрицание конъюнкции превращается в дизъюнкцию отрицаний, а отрицание дизъюнкции — в конъюнкцию отрицаний.

4.2.6.1.7 Законы логических констант

$$(\neg \top) \Leftrightarrow \perp$$

$$(\neg \perp) \Leftrightarrow \top$$

$$(\alpha \wedge \perp) \Leftrightarrow \perp$$

$$(\alpha \wedge \top) \Leftrightarrow \alpha$$

$$(\alpha \vee \perp) \Leftrightarrow \alpha$$

$$(\alpha \vee \top) \Leftrightarrow \top$$

Данные законы описывают взаимодействие логических констант истины ($\top$) и лжи ($\perp$) с отрицанием, конъюнкцией и дизъюнкцией.

4.2.6.1.8 Закон тождества

$$\alpha \Leftrightarrow \alpha$$

“Мысль о предмете в пределах заданного рассуждения должна сохранять один и тот же смысл.”

4.2.6.1.9 Закон противоречия

$$(\alpha \wedge (\neg \alpha)) \Leftrightarrow \perp$$

“Неверно, что одно и то же высказывание одновременно истинно и ложно.”

4.2.6.1.10 Закон исключённого третьего

$$(\alpha \vee (\neg \alpha)) \Leftrightarrow \top$$

“Из двух противоречащих суждений одно обязательно истинно, другое ложно, третьего не дано.”

4.2.6.1.11 Закон выражения связки импликации

$$(\alpha \rightarrow \beta) \Leftrightarrow ((\neg \alpha) \vee \beta)$$

4.2.6.1.12 Закон выражения связки эквиваленции

$$(\alpha \sim \beta) \Leftrightarrow ((\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha))$$

4.2.6.1.13 Законы поглощения

$$(\alpha \wedge (\alpha \vee \beta)) \Leftrightarrow \alpha$$

$$(\alpha \vee (\alpha \wedge \beta)) \Leftrightarrow \alpha$$

Данные законы описывают ситуации, когда более сложное высказывание не добавляет новой информации по сравнению с α : в первом случае, если α уже истинно, добавление « $\vee \beta$ » под конъюнкцией ничего не меняет; во втором случае дизъюнкция с « α и β » также не расширяет множество удовлетворяющих интерпретаций.

4.2.6.1.14 Законы склеивания

$$((\alpha \vee \beta) \wedge (\alpha \vee (\neg\beta))) \Leftrightarrow \alpha$$

$$((\alpha \wedge \beta) \vee (\alpha \wedge (\neg\beta))) \Leftrightarrow \alpha$$

Данные законы отражают, что при объединении двух случаев, различающихся только значением β , зависимость от β исчезает.

4.2.6.1.15 Закон экспортации-импортации

$$(\alpha \rightarrow (\beta \rightarrow \gamma)) \Leftrightarrow ((\alpha \wedge \beta) \rightarrow \gamma)$$

Данный закон показывает, что вложенная импликация «если α , то (если β , то γ)» эквивалентна одной импликации от конъюнкции: «если одновременно α и β , то γ ».

4.2.6.1.15 Закон распределения импликации относительно дизъюнкции

$$((\alpha \rightarrow \gamma) \wedge (\beta \rightarrow \gamma)) \Leftrightarrow ((\alpha \vee \beta) \rightarrow \gamma)$$

Данный закон выражает принцип доказательства «по случаям»: если из α следует γ и из β следует γ , то из дизъюнкции $(\alpha \vee \beta)$ также следует γ , и наоборот.

4.2.6.1.16 Закон распределения импликации относительно конъюнкции

$$((\alpha \rightarrow \gamma) \vee (\beta \rightarrow \gamma)) \Leftrightarrow ((\alpha \wedge \beta) \rightarrow \gamma)$$

Данный закон фиксирует, что утверждение «если α и β , то γ » эквивалентно утверждению, что по крайней мере одна из импликаций $(\alpha \rightarrow \gamma)$ или $(\beta \rightarrow \gamma)$ выполняется во всех интерпретациях, где истинны и α , и β .

4.2.6.1.17 Закон контрапозиции

$$(\alpha \rightarrow \beta) \Leftrightarrow ((\neg\beta) \rightarrow (\neg\alpha))$$

Данный закон выражает принцип контрапозиции: импликация логически эквивалентна импликации между отрицаниями в противоположном направлении.

4.2.6.1.18 Законы материальной импликации

$$(\alpha \rightarrow (\beta \rightarrow \alpha)) \Leftrightarrow \top$$

$$((\neg\alpha) \rightarrow (\alpha \rightarrow \beta)) \Leftrightarrow \top$$

Данные законы показывают, что в классической семантике материальной импликации некоторые, с интуитивной точки зрения «подозрительные», высказывания являются тавтологиями. Первый парадокс отражает, что «если α , то (если β , то α)» истинно независимо от значений α и β ; второй — что из ложности α следует любое условное высказывание «если α , то β », что связано с тем, что импликация с ложной посылкой в классической логике всегда истинна.

4.2.6.1.19 Закон транзитивности импликации

$$((\alpha \rightarrow \beta) \wedge (\beta \rightarrow \gamma)) \Leftrightarrow (\alpha \rightarrow \gamma)$$

Данный закон формализует транзитивность следования: если из α следует β и из β следует γ , то из α следует γ .

«Если из того, что идёт дождь (α), следует, что мокро (β), и из того, что мокро (β), следует, что дорога скользкая (γ), то из того, что идёт дождь (α), следует, что дорога скользкая (γ)».

4.2.6.1.20 Закон обобщённой контрапозиции

$$((\alpha \wedge \beta) \rightarrow (\gamma \wedge \delta)) \Leftrightarrow ((\alpha \wedge (\neg\beta)) \rightarrow (\neg\gamma))$$

Этот закон можно понимать как обобщение контрапозиции для составных посылок и заключений.

Семантически обе формулы исключают одни и те же комбинации значений: если существует оценка, где $(\alpha \wedge \beta)$ истинно, а $(\gamma \wedge \delta)$ ложно, то это означает, что при тех же α и β истина γ невозможна; наоборот, невозможность γ при $(\alpha \wedge (\neg\beta))$ эквивалентна требованию перехода от $(\alpha \wedge \beta)$ к $(\gamma \wedge \delta)$ во всех допустимых интерпретациях.

4.2.6.1.21 Закон доминирования конъюнкции

$$((\alpha \wedge \beta) \wedge (\alpha \vee \beta)) \Leftrightarrow (\alpha \wedge \beta)$$

Здесь конъюнкция $(\alpha \wedge \beta)$ «доминирует» (в смысле охвата интерпретаций) над более слабой формулой $(\alpha \vee \beta)$. Выражение $(\alpha \wedge \beta)$ уже означает, что обе формулы α и β истинны; тогда $(\alpha \vee \beta)$ автоматически истинно и ничего не добавляет к информации.

4.2.6.1.22 Закон доминирования дизъюнкции

$$((\alpha \vee \beta) \vee (\alpha \wedge \beta)) \Leftrightarrow (\alpha \vee \beta)$$

Здесь дизъюнкция $(\alpha \vee \beta)$ «сильнее», чем конъюнкция $(\alpha \wedge \beta)$. Формула $(\alpha \wedge \beta)$ истинна только в части случаев, в которых истинно $(\alpha \vee \beta)$; поэтому добавление $(\alpha \wedge \beta)$ под $\vee$ не расширяет множество случаев истинности.

4.2.6.2 Понятие равносильного преобразования логической формулы

Равносильные преобразования логической формулы — это последовательность замен подформул заданной логической формулы равносильными им логическими формулами. Каждый шаг преобразования обоснован одним из перечисленных выше законов равносильности. Корректность преобразований гарантируется тем, что замена подформулы равносильной ей формулой не меняет истинностного значения всей формулы ни при какой интерпретации.

Метод равносильных преобразований используется для:

- упрощения формул (уменьшения числа переменных и операций);
- доказательства равносильностей;
- приведения формул к заданной нормальной форме (КНФ, ДНФ, СКНФ, СДНФ).

4.2.6.2.1 Упрощение сложных формул

Пример 1. Упростить формулу $(\neg(\neg P \wedge \neg Q))$.

Решение:

$$(\neg((\neg P) \wedge (\neg Q)))$$

$$\Leftrightarrow ((\neg(\neg P)) \vee (\neg(\neg Q))) \text{ [закон де Моргана]}$$

$$\Leftrightarrow (P \vee Q) \text{ [закон двойного отрицания]}$$

$$\text{Ответ: } (\neg((\neg P) \wedge (\neg Q))) \Leftrightarrow (P \vee Q).$$

Пример 2. Упростить формулу $((P \rightarrow Q) \wedge (P \rightarrow (\neg Q)))$.

Решение:

$$((P \rightarrow Q) \wedge (P \rightarrow (\neg Q)))$$

$$\Leftrightarrow (((\neg P) \vee Q) \wedge ((\neg P) \vee (\neg Q))) \text{ [выражение импликаций]}$$

$$\Leftrightarrow ((\neg P) \vee (Q \wedge (\neg Q))) \text{ [дистрибутивность } \vee \text{]}$$

$$\Leftrightarrow ((\neg P) \vee \perp) \text{ [закон противоречия]}$$

$$\Leftrightarrow (\neg P) \text{ [закон логической константы]}$$

$$\text{Ответ: } ((P \rightarrow Q) \wedge (P \rightarrow (\neg Q))) \Leftrightarrow (\neg P).$$

Пример 3. Упростить формулу $((P \vee Q) \wedge ((\neg P) \vee Q))$.

Решение:

$$((P \vee Q) \wedge ((\neg P) \vee Q))$$

$$\Leftrightarrow ((P \wedge (\neg P)) \vee Q) \text{ [дистрибутивность } \vee \text{ относительно } \wedge \text{]}$$

$$\Leftrightarrow (\perp \vee Q) \text{ [закон противоречия]}$$

$$\Leftrightarrow Q \text{ [закон логической константы]}$$

$$\text{Ответ: } ((P \vee Q) \wedge ((\neg P) \vee Q)) \Leftrightarrow Q.$$

4.2.6.2.2 Доказательство тавтологий

Пример 1. Доказать, что $(P \rightarrow (Q \rightarrow P))$ является тавтологией.

Решение:

$$(P \rightarrow (Q \rightarrow P))$$

$$\Leftrightarrow ((\neg P) \vee (Q \rightarrow P)) \text{ [выражение внешней импликации]}$$

$$\Leftrightarrow ((\neg P) \vee ((\neg Q) \vee P)) \text{ [выражение внутренней импликации]}$$

$$\Leftrightarrow (((\neg P) \vee P) \vee (\neg Q)) \text{ [ассоциативность и коммутативность } \vee \text{]}$$

$$\Leftrightarrow (\top \vee (\neg Q)) \text{ [закон исключённого третьего]}$$

$\Leftrightarrow \top$ [закон логической константы]

Ответ: логическая формула $(P \rightarrow (Q \rightarrow P))$ является тавтологией.

Пример 2. Доказать тавтологию $(P \rightarrow P)$.

Решение:

$(P \rightarrow P)$

$\Leftrightarrow ((\neg P) \vee P)$ [выражение импликации]

$\Leftrightarrow \top$ [закон исключённого третьего]

Ответ: логическая формула $(P \rightarrow P)$ является тавтологией.

Пример 3. Доказать, что $((P \wedge (P \rightarrow Q)) \rightarrow Q)$ является тавтологией.

Решение:

$((P \wedge (P \rightarrow Q)) \rightarrow Q)$

$\Leftrightarrow ((\neg(P \wedge (P \rightarrow Q))) \vee Q)$ [выражение импликации]

$\Leftrightarrow (((\neg P) \vee \neg(P \rightarrow Q)) \vee Q)$ [закон де Моргана]

$\Leftrightarrow (((\neg P) \vee (P \wedge (\neg Q))) \vee Q)$ [выражение импликации]

$\Leftrightarrow (((\neg P) \vee P) \wedge ((\neg P) \vee (\neg Q))) \vee Q$ [дистрибутивность]

$\Leftrightarrow ((\top \wedge ((\neg P) \vee (\neg Q))) \vee Q)$ [закон исключённого третьего]

$\Leftrightarrow (((\neg P) \vee (\neg Q)) \vee Q)$ [закон логической константы]

$\Leftrightarrow ((\neg P) \vee ((\neg Q) \vee Q))$ [ассоциативность]

$\Leftrightarrow ((\neg P) \vee \top)$ [закон исключённого третьего]

$\Leftrightarrow \top$ [закон логической константы]

Ответ: логическая формула $((P \wedge (P \rightarrow Q)) \rightarrow Q)$ является тавтологией.

4.2.6.2.3 Понятие нормальной формы

4.2.6.2.3.1 Понятие конъюнктивной нормальной формы логической формулы

Конъюнктивная нормальная форма (КНФ) логической формулы — это нормальная форма, в которой заданная логическая формула представляет собой конъюнкцию элементарных дизъюнкций атомарных подформул заданной логической формулы и/или их отрицаний.

Элементарная дизъюнкция — это дизъюнкция конечного непустого множества литералов, в которой ни одна переменная не встречается более одного раза (ни в положительной, ни в отрицательной форме).

Ниже представлена грамматика конъюнктивной нормальной формы (КНФ) логической формулы:

```

<КНФ> ::= <дизъюнкт>
        | <открывающая круглая скобка>
          <дизъюнкт>
          <конъюнкция>
          <КНФ>
          <закрывающая круглая скобка>
<дизъюнкт> ::= <литерал>
        | <открывающая круглая скобка>
          <литерал>
          <дизъюнкция>
          <дизъюнкт>
          <закрывающая круглая скобка>
<литерал> ::= <атомарная логическая формула>
        | <открывающая круглая скобка>
          <отрицание>
          <атомарная логическая формула>
          <закрывающая круглая скобка>

```

Пример. Используя метод равносильных преобразований, привести логическую формулу $(\neg(P \rightarrow Q))$ к конъюнктивной нормальной форме.

Решение:

$$\begin{aligned}
 &(\neg(P \rightarrow Q)) \\
 &\Leftrightarrow (\neg((\neg P) \vee Q)) \quad [\text{выражение импликации: } (P \rightarrow Q) \Leftrightarrow ((\neg P) \vee Q)] \\
 &\Leftrightarrow ((\neg(\neg P)) \wedge (\neg Q)) \quad [\text{закон де Моргана: } (\neg(\alpha \vee \beta)) \Leftrightarrow ((\neg\alpha) \wedge (\neg\beta))] \\
 &\Leftrightarrow (P \wedge (\neg Q)) \quad [\text{закон двойного отрицания: } (\neg(\neg P)) \Leftrightarrow P]
 \end{aligned}$$

Проверка:

Логическая формула $(P \rightarrow Q)$ ложна тогда и только тогда, когда $P = \top$ и $Q = \perp$; следовательно, логическая формула $(\neg(P \rightarrow Q))$ истинна ровно при $P = \top$ и $Q = \perp$. В свою очередь, формула $(P \wedge \neg Q)$ истинна ровно при $P = \top$ и $Q = \perp$. Следовательно, $(\neg(P \rightarrow Q)) \Leftrightarrow (P \wedge \neg Q)$.

Ответ: логическая формула $(P \wedge (\neg Q))$ является конъюнктивной нормальной формой для логической формулы $(\neg(P \rightarrow Q))$.

4.2.6.2.3.2 Понятие дизъюнктивной нормальной формы логической формулы

Дизъюнктивная нормальная форма (ДНФ) логической формулы — это нормальная форма, в которой заданная логическая формула представляет собой дизъюнкцию элементарных конъюнкций атомарных подформул заданной логической формулы и/или их отрицаний.

Элементарная конъюнкция — это конъюнкция конечного непустого множества литералов, в которой ни одна переменная не встречается более одного раза (ни в положительной, ни в отрицательной форме).

Ниже представлена грамматика дизъюнктивной нормальной формы (ДНФ) логической формулы:

```

<ДНФ> ::= <конъюнкт>
        | <открывающая круглая скобка>
          <конъюнкт>
          <дизъюнкция>
          <ДНФ>
          <закрывающая круглая скобка>
<конъюнкт> ::= <литерал>
              | <открывающая круглая скобка>
                <литерал>
                <конъюнкция>
                <конъюнкт>
                <закрывающая круглая скобка>
<литерал> ::= <атомарная логическая формула>
              | <открывающая круглая скобка>
                <отрицание>
                <атомарная логическая формула>
                <закрывающая круглая скобка>

```

Пример. Используя метод равносильных преобразований, привести логическую формулу $(P \rightarrow (Q \rightarrow R))$ к дизъюнктивной нормальной форме.

Решение:

$$\begin{aligned}
 & (P \rightarrow (Q \rightarrow R)) \\
 \Leftrightarrow & ((\neg P) \vee (Q \rightarrow R)) && [\text{выражение импликации для } (P \rightarrow (Q \rightarrow R))] \\
 \Leftrightarrow & ((\neg P) \vee ((\neg Q) \vee R)) && [\text{выражение импликации для } (Q \rightarrow R)] \\
 \Leftrightarrow & (((\neg P) \vee (\neg Q)) \vee R) && [\text{ассоциативность } \vee]
 \end{aligned}$$

Ответ: логическая формула $((\neg P) \vee (\neg Q)) \vee R$ является дизъюнктивной нормальной формой для логической формулы $(P \rightarrow (Q \rightarrow R))$.

4.2.6.2.3.3 Понятие совершенной конъюнктивной нормальной формы

Совершенная конъюнктивная нормальная форма (СКНФ) логической формулы — это конъюнктивная нормальная форма заданной логической формулы, в которой каждый дизъюнкт содержит все пропозициональные переменные заданной логической формулы ровно по одному разу (в виде литерала или его отрицания).

СКНФ строится по строкам таблицы истинности, в которых логическая формула принимает значение $\perp$. Для каждой такой строки выписывается дизъюнкт: переменная берётся без отрицания, если её значение $\perp$, и с отрицанием, если её значение $\top$.

Пример 1. Построить СКНФ логической формулы $(P \rightarrow Q)$.

Решение:

1. Построим таблицу истинности логической формулы $(P \rightarrow Q)$.

Таблица истинности логической формулы $(P \rightarrow Q)$:

P	Q	$(P \rightarrow Q)$
$\top$	$\top$	$\top$
$\top$	$\perp$	$\perp$
$\perp$	$\top$	$\top$
$\perp$	$\perp$	$\top$

2. Построим СКНФ заданной логической формулы.

СКНФ строится по строке 2 (результат $\perp$):

В строке 2 $P = \top$, $Q = \perp$, следовательно, дизъюнкт — $((\neg P) \vee Q)$.

Таким образом, логическая формула $((\neg P) \vee Q)$ является СКНФ для логической формулы $(P \rightarrow Q)$.

Ответ: $((\neg P) \vee Q)$ — СКНФ.

Пример 2. Построить СКНФ логической формулы $((P \rightarrow Q) \wedge (Q \rightarrow R))$.

Решение:

1. Построим таблицу истинности логической формулы $((P \rightarrow Q) \wedge (Q \rightarrow R))$.

Таблица истинности логической формулы $((P \rightarrow Q) \wedge (Q \rightarrow R))$:

P	Q	R	$(P \rightarrow Q)$	$(Q \rightarrow R)$	Результат
$\top$	$\top$	$\top$	$\top$	$\top$	$\top$
$\top$	$\top$	$\perp$	$\top$	$\perp$	$\perp$
$\top$	$\perp$	$\top$	$\perp$	$\top$	$\perp$
$\top$	$\perp$	$\perp$	$\perp$	$\top$	$\perp$
$\perp$	$\top$	$\top$	$\top$	$\top$	$\top$
$\perp$	$\top$	$\perp$	$\top$	$\perp$	$\perp$
$\perp$	$\perp$	$\top$	$\top$	$\top$	$\top$
$\perp$	$\perp$	$\perp$	$\top$	$\top$	$\top$

2. Построим СКНФ заданной логической формулы.

СКНФ строится по строкам 2, 3, 4, 6 (результат $\perp$).

В строке 2 $P = \top$, $Q = \top$, $R = \perp$, следовательно, конъюнкт — $((\neg P) \vee (\neg Q)) \vee R$.

В строке 3 $P = \top$, $Q = \perp$, $R = \top$, следовательно, конъюнкт — $((\neg P) \vee Q) \vee (\neg R)$.

В строке 4 $P = \top$, $Q = \perp$, $R = \perp$, следовательно, конъюнкт — $((\neg P) \vee Q) \vee R$.

В строке 6 $P = \perp$, $Q = \top$, $R = \perp$, следовательно, конъюнкт — $(P \vee (\neg Q)) \vee R$.

Таким образом, логическая формула $(((((\neg P) \vee (\neg Q)) \vee R) \wedge ((\neg P) \vee Q) \vee (\neg R))) \wedge (((\neg P) \vee Q) \vee R)) \wedge ((P \vee (\neg Q)) \vee R)$ является СКНФ для логической формулы $(P \rightarrow Q)$.

Ответ: $(((((\neg P) \vee (\neg Q)) \vee R) \wedge ((\neg P) \vee Q) \vee (\neg R))) \wedge (((\neg P) \vee Q) \vee R)) \wedge ((P \vee (\neg Q)) \vee R)$ — СДНФ.

4.2.6.2.3.4 Понятие совершенной дизъюнктивной нормальной формы

Совершенная дизъюнктивная нормальная форма (СДНФ) логической формулы — это дизъюнктивная нормальная форма заданной логической формулы, в которой каждый конъюнкт содержит все пропозициональные переменные заданной логической формулы ровно по одному разу (в виде литерала или его отрицания).

СДНФ строится по строкам таблицы истинности, в которых логическая формула принимает значение $\top$. Для каждой такой строки выписывается конъюнкт: переменная берётся без отрицания, если её значение $\top$, и с отрицанием, если её значение $\perp$.

Пример 1. Построить СДНФ логической формулы $(P \rightarrow Q)$.

Решение:

1. Построим таблицу истинности логической формулы $(P \rightarrow Q)$.

Таблица истинности логической формулы $(P \rightarrow Q)$:

P	Q	$(P \rightarrow Q)$
$\top$	$\top$	$\top$
$\top$	$\perp$	$\perp$
$\perp$	$\top$	$\top$
$\perp$	$\perp$	$\top$

2. Построим СДНФ заданной логической формулы.

СДНФ строится по строкам 1, 3, 4 (результат $\top$).

В строке 1 $P = \top$, $Q = \top$, следовательно, конъюнкт — $(P \wedge Q)$.

В строке 3 $P = \perp$, $Q = \top$, следовательно, конъюнкт — $((\neg P) \wedge Q)$.

В строке 4 $P = \perp$, $Q = \perp$, следовательно, конъюнкт — $((\neg P) \wedge (\neg Q))$.

Таким образом, логическая формула $((P \wedge Q) \vee ((\neg P) \wedge Q) \vee ((\neg P) \wedge (\neg Q)))$ является СДНФ для логической формулы $(P \rightarrow Q)$.

Ответ: $((P \wedge Q) \vee ((\neg P) \wedge Q) \vee ((\neg P) \wedge (\neg Q)))$ — СДНФ.

Пример 2. Построить СДНФ формулы $((P \rightarrow Q) \wedge (Q \rightarrow R))$.

Решение:

3. Построим таблицу истинности логической формулы $((P \rightarrow Q) \wedge (Q \rightarrow R))$.

Таблица истинности логической формулы $((P \rightarrow Q) \wedge (Q \rightarrow R))$:

P	Q	R	$(P \rightarrow Q)$	$(Q \rightarrow R)$	Результат
$\top$	$\top$	$\top$	$\top$	$\top$	$\top$
$\top$	$\top$	$\perp$	$\top$	$\perp$	$\perp$
$\top$	$\perp$	$\top$	$\perp$	$\top$	$\perp$
$\top$	$\perp$	$\perp$	$\perp$	$\top$	$\perp$
$\perp$	$\top$	$\top$	$\top$	$\top$	$\top$
$\perp$	$\top$	$\perp$	$\top$	$\perp$	$\perp$
$\perp$	$\perp$	$\top$	$\top$	$\top$	$\top$
$\perp$	$\perp$	$\perp$	$\top$	$\top$	$\top$

4. Построим СДНФ заданной логической формулы.

СДНФ строится по строкам 1, 5, 7, 8 (результат $\top$).

В строке 1 $P = \top$, $Q = \top$, $R = \top$, следовательно, конъюнкт — $((P \wedge Q) \wedge R)$.

В строке 5 $P = \perp$, $Q = \top$, $R = \top$, следовательно, конъюнкт — $((\neg P) \wedge Q) \wedge R$.

В строке 7 $P = \perp$, $Q = \perp$, $R = \top$, следовательно, конъюнкт — $((\neg P) \wedge (\neg Q)) \wedge R$.

В строке 8 $P = \perp$, $Q = \perp$, $R = \perp$, следовательно, конъюнкт — $((\neg P) \wedge (\neg Q)) \wedge (\neg R)$.

Таким образом, логическая формула $(((((P \wedge Q) \wedge R) \vee ((\neg P) \wedge Q) \wedge R) \vee ((\neg P) \wedge (\neg Q)) \wedge R) \vee ((\neg P) \wedge (\neg Q)) \wedge (\neg R)))$ является СДНФ для логической формулы $(P \rightarrow Q)$.

Ответ: $(((((P \wedge Q) \wedge R) \vee (((\neg P) \wedge Q) \wedge R)) \vee (((\neg P) \wedge (\neg Q)) \wedge R)) \vee (((\neg P) \wedge (\neg Q)) \wedge (\neg R))))$ — СДНФ.

4.3 Исчисление высказываний

Исчисление высказываний (пропозициональное исчисление) возникло в рамках программы формализации математики на рубеже XIX–XX веков и связано с именами Г. Фреге, Д. Гильберта и их последователей. Его основная цель состоит в том, чтобы выделить чисто формальную сторону рассуждений, абстрагируясь от конкретного содержания высказываний и оперируя лишь с их логической формой. В этом смысле исчисление высказываний представляет собой строго определённую формальную систему, в которой понятия доказательства, теоремы и следствия задаются синтаксически, через правила построения формул и правила вывода.

Содержательно, *исчисление высказываний*, или *пропозициональное исчисление* — это множество доказуемых логических формул языка логики высказываний.

В самом общем виде под исчислением высказываний понимают формальную систему, заданную:

- (1) алфавитом языка логики высказываний,
- (2) грамматикой языка логики высказываний, определяющей множество всех правильных логических формул этого языка,
- (3) системой аксиомных схем,
- (4) системой правил вывода.

В алгебре высказываний каждый атомарная логическая формула языка логики высказываний интерпретируется как некоторое простое высказывание (утверждение, имеющее значение «истина» или «ложь»), а логические связки задают стандартные логические операции (соответствуют логическим функциям). В рамках исчисления высказываний основное внимание уделяется не семантике, а синтаксису — тому, какие формулы можно получить из других формул по строго заданным правилам.

4.3.1 Понятие аксиомы языка логики высказываний

Внутри исчисления высказываний аксиомы играют роль исходных, бездоказательных формул, от которых начинается всякое формальное доказательство.

Аксиома языка логики высказываний — это логическая формула, объявленная истинной без доказательства в рамках заданного исчисления, получаемая в результате подстановки логических формул в некоторую *аксиомную схему* и используемая для построения формального логического вывода *теорем языка логики высказываний*.

4.3.1.1 Аксиомные схемы исчисления высказываний

В исчислении высказываний используются не конкретные аксиомы, а *аксиомные схемы*.

Аксиомная схема — это логическая формула метаязыка для языка логики высказываний, каждая из которых порождает бесконечное семейство аксиом путём подстановки произвольных формул вместо метапеременных (обычно обозначаемыми греческими буквами α , β , γ и т.п.).

Другими словами, разделение на аксиомные схемы и конкретные аксиомы позволяет бесконечно порождать аксиомы путём подстановки произвольных формул вместо метапеременных.

Ниже приведена стандартная система аксиомных схем, используемая в *исчислении высказываний*.

4.3.1.1.1 Аксиомная схема введения импликации

$$(\alpha \rightarrow (\beta \rightarrow \alpha))$$

Данная аксиомная схема выражает принцип, что если некое высказывание α истинно, то истинно и высказывание « β влечёт α » для любого β ; подставляя вместо α и β любые формулы, получаем конкретные аксиомы, например: $(A \rightarrow (B \rightarrow A))$, $((P \wedge Q) \rightarrow (\neg R \rightarrow (P \wedge Q)))$ и т.п.

4.3.1.1.2 Аксиомная схема удаления импликации

$$((\gamma \rightarrow \alpha) \rightarrow ((\gamma \rightarrow (\alpha \rightarrow \beta)) \rightarrow (\gamma \rightarrow \beta)))$$

Данная аксиомная схема выражает своего рода транзитивность следования: если из γ следует α , а из γ следует также $(\alpha \rightarrow \beta)$, то из γ следует β ; она позволяет в чисто синтаксических терминах реализовать переход от двух посылок к заключению, соответствующему их «сочетанию».

4.3.1.1.3 Аксиомная схема введения конъюнкции

$$(\alpha \rightarrow (\beta \rightarrow (\alpha \wedge \beta)))$$

Данная аксиомная схема кодирует правило, согласно которому при наличии посылок α и β вправе заключить $(\alpha \wedge \beta)$.

4.3.1.1.4 Аксиомная схема удаления конъюнкции

$$((\alpha \wedge \beta) \rightarrow \alpha)$$

$$((\alpha \wedge \beta) \rightarrow \beta)$$

Обе аксиомные схемы реализуют правила удаления конъюнкции: из истинности сложного высказывания $(\alpha \wedge \beta)$ следует истинность каждого из его конъюнктов по отдельности.

4.3.1.1.5 Аксиомная схема введения дизъюнкции

$$(\alpha \rightarrow (\alpha \vee \beta))$$

$$(\alpha \rightarrow (\beta \vee \alpha))$$

Обе аксиомные схемы выражают тот факт, что если α истинно, то истинны и высказывания $(\alpha \vee \beta)$ и $(\beta \vee \alpha)$ соответственно; то есть истинная формула может быть «ослаблена» до дизъюнкции с произвольной формулой.

4.3.1.1.6 Аксиомная схема удаления дизъюнкции

$$((\alpha \rightarrow \gamma) \rightarrow ((\beta \rightarrow \gamma) \rightarrow ((\alpha \vee \beta) \rightarrow \gamma)))$$

Данная аксиомная схема является формализацией принципа доказательства «по случаям»: если из α следует γ и из β следует γ , то из дизъюнкции $(\alpha \vee \beta)$ также следует γ .

4.3.1.1.7 Аксиомная схема введения отрицания

$$((\alpha \rightarrow \beta) \rightarrow ((\alpha \rightarrow (\neg\beta)) \rightarrow (\neg\alpha)))$$

Данная аксиомная схема отражает классический принцип доказательства от противного: если из гипотезы α выводятся противоречащие друг другу формулы β и $(\neg\beta)$, то α должна быть отвергнута, то есть истинно $(\neg\alpha)$.

4.3.1.1.8 Аксиомная схема удаления отрицания

$$((\neg(\neg\alpha)) \rightarrow \alpha)$$

Данная аксиомная схема фиксирует закон двойного отрицания в классической логике: невозможность ложности ($\neg\alpha$) означает истинность α .

4.3.1.1.8 Аксиомная схема введения эквиваленции

$$((\alpha \rightarrow \beta) \rightarrow ((\beta \rightarrow \alpha) \rightarrow (\alpha \sim \beta)))$$

Данная аксиомная схема устанавливает, что эквиваленция ($\alpha \sim \beta$) вводится как сочетание двух направленных импликаций.

4.3.1.1.9 Аксиомные схемы удаления эквиваленции

$$((\alpha \sim \beta) \rightarrow (\alpha \rightarrow \beta))$$

$$((\alpha \sim \beta) \rightarrow (\beta \rightarrow \alpha))$$

Обе аксиомные схемы выражают правило удаления эквиваленции: из эквиваленции следуют обе соответствующие импликации.

4.3.2 Понятие формального логического вывода

Формальный логический вывод из множества логических формул G — это конечная последовательность $\langle \varphi_1, \varphi_2, \dots, \varphi_n \rangle$, где для каждого i выполняется одно из условий: $\varphi_i \in G$ (гипотеза), φ_i — аксиома, или φ_i получена по правилу вывода из предыдущих элементов последовательности.

Формальный логический вывод позволяет доказывать теоремы для произвольных формул без перебора всех интерпретаций.

4.3.2.1 Понятие теоремы языка логики высказываний

Теорема языка логики высказываний — это логическая формула, для которой существует формальный логический вывод в данном исчислении высказываний, то есть конечная последовательность формул, начинающаяся с аксиом и, возможно, гипотез, и завершающаяся данной формулой, причём каждый шаг вывода осуществляется по фиксированным правилам вывода.

В обозначениях это записывается как $\vdash \{\varphi\}$ и означает, что формула φ выводима без дополнительных предположений и потому считается теоремой рассматриваемой аксиоматической системы. Если вывод ведётся из множества гипотез G и в конце получается формула φ , то говорят, что φ выводима из G , и пишут $G \vdash \{\varphi\}$; при $G = \emptyset$ запись упрощают до $\vdash \{\varphi\}$.

4.3.2.2 Понятие отношения выводимости

Отношение выводимости ($\vdash$) — это синтаксическое отношение между множеством логических формул G (гипотезами) и множеством выводимых логических формул F .

$G \vdash F$ означает, что множество логических формул F выводимо из G в исчислении высказываний, то есть существует конечная последовательность формул $\langle \varphi_1, \varphi_2, \dots, \varphi_n \rangle$, каждая из которых является либо аксиомой, либо элементом G , либо получена из предыдущих формул по правилам вывода.

Если $G = \emptyset$, то пишут просто $\vdash F$ (« F — множество теорем»).

Ключевое различие в современной логике проводится между отношением логического следования и отношением выводимости. Отношение следования, обозначаемое символом $\models$, является семантическим: запись $\alpha \models \varphi$ означает, что для любой интерпретации, в которой формула α истинна, формула φ также истинна. Это отношение определено через модели (интерпретации) и не зависит от выбранной формальной системы доказательств. Напротив, отношение выводимости, обозначаемое символом $\vdash$, носит сугубо синтаксический характер: $G \vdash \{\varphi\}$ означает, что существует конечный формальный вывод формулы φ из множества гипотез G в рамках данного исчисления.

4.3.2.3 Правила вывода

Правило вывода — это логическая формула, которая задаёт функцию перехода от одной или нескольких формул-посылок к формуле-заключению.

В отличие от аксиом, которые могут быть включены в вывод без обоснования, применение правила вывода всегда опирается на уже полученные формулы: если формулы, соответствующие посылкам правила, присутствуют в предыдущих шагах вывода, то по этому правилу допускается добавить в вывод заключение. Типичным примером является правило прямого заключения (правило Modus Ponens).

4.3.2.3.1 Правило прямого заключения (Modus Ponens)

$$(G \cup \{\alpha, (\alpha \rightarrow \beta)\}) \vdash (G \cup \{\beta\})$$

Данное правило гласит: если в выводе уже получены формула α и импликация $(\alpha \rightarrow \beta)$, то разрешается заключить формулу β , сохраняя все остальные гипотезы G . Иными словами, из « α » и «если α , то β » следует « β ». Здесь G — произвольное множество формул, которое присутствует в выводе как

фон, а правило касается только добавления β при наличии двух указанных посылок.

Другое название правила прямого заключения — правило Modus Ponens (MP).

Пример 1. Доказать: $\vdash \{(A \rightarrow A)\}$.

Решение: Ответ:

1. $(A \rightarrow (A \rightarrow A))$ [схема введения импликации, $\alpha := A, \beta := A$]
2. $((A \rightarrow (A \rightarrow A)) \rightarrow ((A \rightarrow ((A \rightarrow A) \rightarrow A)) \rightarrow (A \rightarrow A)))$
[схема удаления импликации]
3. $((A \rightarrow ((A \rightarrow A) \rightarrow A)) \rightarrow (A \rightarrow A))$ [MP из 1 и 2]
4. $(A \rightarrow ((A \rightarrow A) \rightarrow A))$ [схема введения импликации, $\alpha := A, \beta := (A \rightarrow A)$]
5. $(A \rightarrow A)$ [MP из 4 и 3] $\boxtimes$

Пример 2. Доказать: $\vdash \{(A \rightarrow (B \rightarrow B))\}$

Решение:

1. $(B \rightarrow (B \rightarrow B))$ [схема введения импликации, $\alpha := B, \beta := B$]
2. $((B \rightarrow (B \rightarrow B)) \rightarrow ((B \rightarrow ((B \rightarrow B) \rightarrow B)) \rightarrow (B \rightarrow B)))$
[схема удаления импликации]
3. $((B \rightarrow ((B \rightarrow B) \rightarrow B)) \rightarrow (B \rightarrow B))$ [MP из 1 и 2]
4. $(B \rightarrow ((B \rightarrow B) \rightarrow B))$ [схема введения импликации, $\alpha := B, \beta := (B \rightarrow B)$]
5. $(B \rightarrow B)$ [MP из 4 и 3]
6. $((B \rightarrow B) \rightarrow (A \rightarrow (B \rightarrow B)))$ [схема введения импликации, $\alpha := (B \rightarrow B), \beta := A$]
7. $(A \rightarrow (B \rightarrow B))$ [MP из 1 и 2] $\boxtimes$

4.3.2.4 Метатеоремы исчисления высказываний

Метатеорема исчисления высказываний — это правило о самой аксиоматической системе, формулируемое и доказываемое на метаязыке.

К числу базовых метатеорем исчисления высказываний относятся: метатеорема дедукции, обратная метатеорема дедукции, теорема о полноте, теорема об адекватности и теорема о компактности.

4.3.2.4.1 Метатеорема дедукции

$$(G \cup \{\alpha\} \vdash \{\beta\}) \Rightarrow (G \vdash \{(\alpha \rightarrow \beta)\})$$

Теорема означает: если из множества гипотез G вместе с дополнительной гипотезой α можно вывести формулу β , то существует вывод формулы $(\alpha \rightarrow \beta)$ из одних только формул множества G . Интуитивно это соответствует переходу от рассуждения «пусть α , тогда β » к утверждению «если α , то β » без явного использования α как гипотезы.

4.3.2.4.2 Метатеорема, обратная метатеореме дедукции

$$(G \vdash \{(\alpha \rightarrow \beta)\}) \Rightarrow (G \cup \{\alpha\} \vdash \{\beta\})$$

Теорема утверждает, что если в исчислении высказываний удалось вывести импликацию $(\alpha \rightarrow \beta)$ из гипотез G , то при добавлении α к этим гипотезам можно вывести β . Иначе говоря, из доказанного утверждения «если α , то β » разрешается, считая α дополнительной предпосылкой, делать вывод β . В паре с метатеоремой дедукции это устанавливает двухстороннюю связь между выводом с гипотезой α и выводом импликации $(\alpha \rightarrow \beta)$ без этой гипотезы.

4.3.2.4.2.1 Понятие квазивывода

Квазивывод, или *формальный вывод с гипотезами* — конечная последовательность $\langle \varphi_1, \varphi_2, \dots, \varphi_n \rangle$, в которой каждая формула φ_i является аксиомой, гипотезой из $G \cup \{\alpha\}$, либо получена из предыдущих по правилу Modus Ponens.

Квазивывод обозначается $G \cup \{\alpha\} \vdash \{\varphi_n\}$ и используется как вспомогательный инструмент при доказательстве теорем с применением метатеоремы дедукции.

Пример 1. Доказать: $\vdash \{(A \rightarrow ((\neg A) \rightarrow B))\}$.

Решение:

По метатеореме дедукции докажем, что $\{A, (\neg A)\} \vdash \{B\}$ (мета-переход).

1. A [гипотеза]
2. $(\neg A)$ [гипотеза]
3. $(A \rightarrow ((\neg B) \rightarrow A))$ [схема введения импликации, $\alpha := A$, $\beta := (\neg B)$]
4. $((\neg B) \rightarrow A)$ [MP из 1 и 3]
5. $((\neg A) \rightarrow ((\neg B) \rightarrow (\neg A)))$ [схема введения импликации, $\alpha := (\neg A)$, $\beta := (\neg B)$]
6. $((\neg B) \rightarrow (\neg A))$ [MP из 2 и 5]

7. $((\neg B) \rightarrow A) \rightarrow (((\neg B) \rightarrow (\neg A)) \rightarrow (\neg(\neg B)))$ [схема введения отрицания, $\alpha := (\neg B)$, $\beta := A$]
8. $((\neg B) \rightarrow (\neg A)) \rightarrow (\neg(\neg B))$ [MP из 4 и 7]
9. $(\neg(\neg B))$ [MP из 6 и 8]
10. $(\neg(\neg B) \rightarrow B)$ [схема удаления отрицания, $\alpha := B$]
11. B [MP из 9 и 10] $\boxtimes$

4.3.2.4.2.2 Правило перевода квазивывода в формальный логический вывод

Пусть дан квазивывод $\langle \varphi_1, \varphi_2, \dots, \varphi_i \rangle$ формулы β из $G \cup \{\alpha\}$. Тогда формальный логический вывод формулы $(\alpha \rightarrow \beta)$ из G строится индуктивно: для каждого шага φ_i конструируется сегмент S_i , оканчивающийся формулой $(\alpha \rightarrow \varphi_i)$, в соответствии со следующими тремя случаями.

Случай 1. φ_i является аксиомой или $\varphi_i \in G$ (гипотеза, не совпадающая с α). Добавляются 3 шага:

- (1) φ_i [аксиома или гипотеза G]
- (2) $(\varphi_i \rightarrow (\alpha \rightarrow \varphi_i))$ [схема введения импликации: $\alpha := \varphi_i$, $\beta := \alpha$]
- (3) $(\alpha \rightarrow \varphi_i)$ [MP из (1) и (2)]

Случай 2. $\varphi_i = \alpha$ (сама выделенная гипотеза). Добавляются 5 шагов — стандартный вывод $(\alpha \rightarrow \alpha)$:

- (1) $(\alpha \rightarrow (\alpha \rightarrow \alpha))$ [схема введения импликации: $\alpha := \alpha$, $\beta := \alpha$]
- (2) $((\alpha \rightarrow (\alpha \rightarrow \alpha)) \rightarrow ((\alpha \rightarrow ((\alpha \rightarrow \alpha) \rightarrow \alpha)) \rightarrow (\alpha \rightarrow \alpha)))$ [схема удаления импликации: $\gamma := \alpha$, $\alpha := (\alpha \rightarrow \alpha)$, $\beta := \alpha$]
- (3) $((\alpha \rightarrow ((\alpha \rightarrow \alpha) \rightarrow \alpha)) \rightarrow (\alpha \rightarrow \alpha))$ [MP из (1) и (2)]
- (4) $(\alpha \rightarrow ((\alpha \rightarrow \alpha) \rightarrow \alpha))$ [схема введения импликации: $\alpha := \alpha$, $\beta := (\alpha \rightarrow \alpha)$]
- (5) $(\alpha \rightarrow \alpha)$ [MP из (4) и (3)]

Случай 3. φ_i получена по MP из φ_j и $\varphi_k = (\varphi_j \rightarrow \varphi_i)$, где $j, k < i$. К этому моменту сегменты S_j и S_k уже построены и содержат $(\alpha \rightarrow \varphi_j)$ и $(\alpha \rightarrow (\varphi_j \rightarrow \varphi_i))$ соответственно. Добавляются 3 шага:

- (1) $((\alpha \rightarrow \varphi_j) \rightarrow ((\alpha \rightarrow (\varphi_j \rightarrow \varphi_i)) \rightarrow (\alpha \rightarrow \varphi_i)))$ [схема удаления импликации: $\gamma := \alpha$, $\alpha := \varphi_j$, $\beta := \varphi_i$]
- (2) $((\alpha \rightarrow (\varphi_j \rightarrow \varphi_i)) \rightarrow (\alpha \rightarrow \varphi_i))$ [MP из $(\alpha \rightarrow \varphi_j)$ и (1)]
- (3) $(\alpha \rightarrow \varphi_i)$ [MP из $(\alpha \rightarrow (\varphi_j \rightarrow \varphi_i))$ и (2)]

Полный формальный вывод есть конкатенация всех сегментов: $\Pi' = S_1 \oplus S_2 \oplus \dots \oplus S_i$.

4.3.2.4.2.2.1 Теорема об оценке длины формального вывода

Теорема (оценка $3n + 2$). Если квазивывод $\langle \varphi_1, \varphi_2, \dots, \varphi_n \rangle$ формулы β из $(G \cup \{\alpha\})$ содержит n шагов, то полученный по правилу перевода формальный вывод $(\alpha \rightarrow \beta)$ из G содержит не более $3n + 2$ шагов.

Доказательство. Каждый шаг квазивывода порождает сегмент длиной:

3 шага — если φ_i является аксиомой или гипотезой из G (Случай 1),

5 шагов — если $\varphi_i = \alpha$ (Случай 2),

3 шага — если φ_i получена по МР (Случай 3).

Случай 2 встречается не более одного раза (α входит в квазивывод как отдельный шаг не более одного раза). Таким образом, суммарная длина равна не более $5 + 3(n - 1) = 3n + 2$. Равенство достигается, когда $\varphi_i = \alpha$ встречается ровно один раз, а все остальные $n - 1$ шагов принадлежат случаям 1 или 3.

$$|\Pi'| \leq 3n + 2.$$

Пример 1. Доказать: $\vdash \{((A \wedge B) \rightarrow (B \wedge A))\}$

Решение:

По метатеореме дедукции докажем, что $\{(A \wedge B)\} \vdash \{(B \wedge A)\}$ (мета-переход).

1. $(A \wedge B)$ [гипотеза]
2. $((A \wedge B) \rightarrow A)$ [схема удаления конъюнкции]
3. $((A \wedge B) \rightarrow B)$ [схема удаления конъюнкции]
4. A [МР из 1 и 2]
5. B [МР из 1 и 3]
6. $(B \rightarrow (A \rightarrow (B \wedge A)))$ [схема введения конъюнкции]
7. $(A \rightarrow (B \wedge A))$ [МР из 5 и 6]
8. $(B \wedge A)$ [МР из 4 и 7] $\square$

Полученный квазивывод из $\{\alpha\}$, где $\alpha = (A \wedge B)$, содержит $n = 8$ шагов. По оценке $3n + 2 = 26$: ожидаемая длина формального вывода — не более 26 шагов.

Квазивывод $\langle \varphi_1, \dots, \varphi_8 \rangle$: $\varphi_1 = (A \wedge B)$, $\varphi_2 = ((A \wedge B) \rightarrow A)$, $\varphi_3 = ((A \wedge B) \rightarrow B)$, $\varphi_4 = A$ [МР из φ_1 и φ_2], $\varphi_5 = B$ [МР из φ_1 и φ_3], $\varphi_6 = (B \rightarrow (A \rightarrow (B \wedge A)))$, $\varphi_7 = (A \rightarrow (B \wedge A))$ [МР из φ_5 и φ_6], $\varphi_8 = (B \wedge A)$ [МР из φ_4 и φ_7].

Обозначим $\alpha = (A \wedge B)$ для компактности записи. Построение сегментов:

Ответ:

Сегмент S_1 для $\varphi_1 = \alpha$ (Случай 2 — 5 шагов):

1. $(\alpha \rightarrow (\alpha \rightarrow \alpha))$ [схема введения импликации: $\alpha := (A \wedge B)$, $\beta := (A \wedge B)$]
2. $((\alpha \rightarrow (\alpha \rightarrow \alpha)) \rightarrow ((\alpha \rightarrow ((\alpha \rightarrow \alpha) \rightarrow \alpha)) \rightarrow (\alpha \rightarrow \alpha)))$ [схема удаления импликации: $\gamma := \alpha$, $\alpha := (\alpha \rightarrow \alpha)$, $\beta := \alpha$]
3. $((\alpha \rightarrow ((\alpha \rightarrow \alpha) \rightarrow \alpha)) \rightarrow (\alpha \rightarrow \alpha))$ [МР из 1, 2]
4. $(\alpha \rightarrow ((\alpha \rightarrow \alpha) \rightarrow \alpha))$ [схема введения импликации: $\alpha := (A \wedge B)$, $\beta := ((A \wedge B) \rightarrow (A \wedge B))$]
5. $(\alpha \rightarrow \alpha)$, т.е. $((A \wedge B) \rightarrow (A \wedge B))$ [МР из 4 и 3]

Сегмент S_2 для $\varphi_2 = ((A \wedge B) \rightarrow A)$ (Случай 1 — 3 шага):

6. $((A \wedge B) \rightarrow A)$ [схема удаления конъюнкции, $\alpha := A$, $\beta := B$]
7. $((((A \wedge B) \rightarrow A) \rightarrow (\alpha \rightarrow ((A \wedge B) \rightarrow A)))$ [схема введения импликации]
8. $(\alpha \rightarrow ((A \wedge B) \rightarrow A))$, т.е. $((A \wedge B) \rightarrow ((A \wedge B) \rightarrow A))$ [МР из 6 и 7]

Сегмент S_3 для $\varphi_3 = ((A \wedge B) \rightarrow B)$ (Случай 1 — 3 шага):

9. $((A \wedge B) \rightarrow B)$ [Ах удал. конъюнкции, $\alpha := A$, $\beta := B$]
10. $((((A \wedge B) \rightarrow B) \rightarrow (\alpha \rightarrow ((A \wedge B) \rightarrow B)))$ [схема введения импликации]
11. $(\alpha \rightarrow ((A \wedge B) \rightarrow B))$, т.е. $((A \wedge B) \rightarrow ((A \wedge B) \rightarrow B))$ [МР из 9 и 10]

Сегмент S_4 для $\varphi_4 = A$ [МР из $\varphi_1 = (A \wedge B)$ и $\varphi_2 = ((A \wedge B) \rightarrow A)$] (Случай 3 — 3 шага):

12. $((\alpha \rightarrow (A \wedge B)) \rightarrow ((\alpha \rightarrow ((A \wedge B) \rightarrow A)) \rightarrow (\alpha \rightarrow A)))$ [схема удаления импликации: $\gamma := \alpha$, $\alpha := (A \wedge B)$, $\beta := A$]
13. $((\alpha \rightarrow ((A \wedge B) \rightarrow A)) \rightarrow (\alpha \rightarrow A))$ [МР из 5 и 12]
14. $(\alpha \rightarrow A)$, т.е. $((A \wedge B) \rightarrow A)$ [МР из 8 и 13]

Сегмент S_5 для $\varphi_5 = B$ [МР из $\varphi_1 = (A \wedge B)$ и $\varphi_3 = ((A \wedge B) \rightarrow B)$] (Случай 3 — 3 шага):

15. $((\alpha \rightarrow (A \wedge B)) \rightarrow ((\alpha \rightarrow ((A \wedge B) \rightarrow B)) \rightarrow (\alpha \rightarrow B)))$ [схема удаления импликации: $\gamma := \alpha$, $\alpha := (A \wedge B)$, $\beta := B$]
16. $((\alpha \rightarrow ((A \wedge B) \rightarrow B)) \rightarrow (\alpha \rightarrow B))$ [МР из 5 и 15]
17. $(\alpha \rightarrow B)$, т.е. $((A \wedge B) \rightarrow B)$ [МР из 11 и 16]

Сегмент S_6 для $\varphi_6 = (B \rightarrow (A \rightarrow (B \wedge A)))$ (Случай 1 — 3 шага):

18. $(B \rightarrow (A \rightarrow (B \wedge A)))$ [схема введения конъюнкции, $\alpha := B$, $\beta := A$]
19. $((B \rightarrow (A \rightarrow (B \wedge A))) \rightarrow (\alpha \rightarrow (B \rightarrow (A \rightarrow (B \wedge A)))))$ [схема введения импликации]
20. $(\alpha \rightarrow (B \rightarrow (A \rightarrow (B \wedge A))))$ [МР из 18 и 19]

Сегмент S_7 для $\varphi_7 = (A \rightarrow (B \wedge A))$ [МР из $\varphi_5 = B$ и $\varphi_6 = (B \rightarrow (A \rightarrow (B \wedge A)))$] (Случай 3 — 3 шага):

21. $((\alpha \rightarrow B) \rightarrow ((\alpha \rightarrow (B \rightarrow (A \rightarrow (B \wedge A)))) \rightarrow (\alpha \rightarrow (A \rightarrow (B \wedge A))))$

[схема удаления импликации: $\gamma := \alpha$, $\alpha := B$, $\beta := (A \rightarrow (B \wedge A))$]

22. $((\alpha \rightarrow (B \rightarrow (A \rightarrow (B \wedge A)))) \rightarrow (\alpha \rightarrow (A \rightarrow (B \wedge A))))$ [МР из 17 и 21]

23. $(\alpha \rightarrow (A \rightarrow (B \wedge A)))$, т.е. $((A \wedge B) \rightarrow (A \rightarrow (B \wedge A)))$ [МР из 20 и 22]

Сегмент S_8 для $\varphi_8 = (B \wedge A)$ [МР из $\varphi_4 = A$ и $\varphi_7 = (A \rightarrow (B \wedge A))$] (Случай 3 — 3 шага):

24. $((\alpha \rightarrow A) \rightarrow ((\alpha \rightarrow (A \rightarrow (B \wedge A))) \rightarrow (\alpha \rightarrow (B \wedge A))))$ [схема удаления импликации: $\gamma := \alpha$, $\alpha := A$, $\beta := (B \wedge A)$]

25. $((\alpha \rightarrow (A \rightarrow (B \wedge A))) \rightarrow (\alpha \rightarrow (B \wedge A)))$ [МР из 14 и 24]

26. $(\alpha \rightarrow (B \wedge A))$, т.е. $((A \wedge B) \rightarrow (B \wedge A))$ [МР из 23 и 25] $\boxtimes$

4.3.2.4.3 Метатеорема о полноте

$$(\models \alpha) \Rightarrow (\vdash \{\alpha\})$$

То есть, если формула α является логическим следствием в семантическом смысле (является общезначимой: истинна во всех интерпретациях), то она выводима как теорема в данном исчислении. Полнота означает, что нет тавтологии, которая была бы недоказуема.

4.3.2.4.4 Метатеорема, обратная метатеореме о полноте (метатеорема об адекватности)

$$(\vdash \{\alpha\}) \Rightarrow (\models \alpha)$$

Если формула α выводима в исчислении (то есть является теоремой), то она общезначима, то есть истинна в каждой интерпретации. Это означает, что система не доказывает «лишнего»: всё, что выводится по её аксиомам и правилам, действительно является логически верным в семантическом смысле. В сочетании с полнотой адекватность даёт равенство множества теорем исчисления и множества тавтологий логики высказываний.

4.3.2.4.5 Метатеорема о компактности

Если формула α логически выводится из Γ ($\Gamma \vdash \alpha$), то существует конечное подмножество $\Gamma_0 \subseteq \Gamma$, из которого α выводится: $\Gamma_0 \vdash \alpha$.

Теорема отражает тот факт, что все формальные доказательства конечны, и потому любое логическое следствие можно обосновать, опираясь лишь на конечное число посылок из Γ .

4.3.2.5 Правило резолюций

Правило резолюций — это правило вывода в исчислении высказываний, которое позволяет из двух дизъюнктов, содержащих контрарную пару литералов γ и $(\neg\gamma)$, получать новый дизъюнкт — резольвенту.

Правило резолюций имеет следующий вид:

$$(G \cup \{(\alpha \vee \gamma), (\beta \vee (\neg\gamma))\}) \vdash (G \cup \{(\alpha \vee \beta)\}),$$

где:

G — произвольное (возможно пустое) множество дизъюнктов;

γ — литерал (атомарная логическая формула или её отрицание);

$(\neg\gamma)$ — литерал, контрарный γ ;

α, β — (возможно пустые) дизъюнкции литералов;

$(\alpha \vee \gamma)$ и $(\beta \vee (\neg\gamma))$ — родительские дизъюнкты;

$(\alpha \vee \beta)$ — резольвента родительских дизъюнктов по литералу γ .

4.3.3.5.1 Понятие резольвенты

Резольвента — это логическая формула вида $(\alpha \vee \beta)$, получаемая путём применения правила резолюций для двух логических формул вида $(\alpha \vee \gamma)$, $(\beta \vee (\neg\gamma))$.

Пример. Доказать невыполнимость $\{(P \rightarrow Q), (Q \rightarrow R), P, (\neg R)\}$.

Решение:

Пронумеруем логические формулы.

1. $(P \rightarrow Q)$

2. $(Q \rightarrow R)$

3. P

4. $(\neg R)$

Преобразуем заданные логические формулы к логическим формулам вида $(\alpha \vee \gamma)$ и/или $(\beta \vee (\neg\gamma))$.

1. $(P \rightarrow Q) \Leftrightarrow ((\neg P) \vee Q)$ [закон выражения импликации]

2. $(Q \rightarrow R) \Leftrightarrow ((\neg Q) \vee R)$ [закон выражения импликации]

Применяем правило резолюций для контрарных пар логических формул.

1. Q [правило резолюции для контрарной пары $((\neg P) \vee Q)$ и P]

2. R [правило резолюции для контрарной пары $((\neg Q) \vee R)$ и Q]

3. $\perp$ [правило резолюции для контрарной пары R и ($\neg$ R)]

Ответ: Получен $\perp$ — множество $\{(P \rightarrow Q), (Q \rightarrow R), P, (\neg R)\}$ невыполнимо. $\boxtimes$

4.4 Логика предикатов

Логика предикатов, или *логика первого порядка* — это раздел математической логики, в котором, в отличие от логики высказываний, рассматриваются не только целые высказывания, но и их внутренняя структура.

Логика предикатов позволяет формализовать высказывания вида «для всех объектов...», «существует объект, такой что...», описывать отношения между объектами и их свойства. Любая теория, задаваемая на языке классической логики предикатов, строится поверх логики высказываний.

В отличие от логики высказываний логика предикатов является неразрешимой (Чёрч, 1936): не существует алгоритма, проверяющего общезначимость произвольной формулы первого порядка. Это обуславливает использование полурешимых процедур (метод резолюции, метод построения формального логического вывода).

4.4.1 Язык логики предикатов

Язык логики предикатов — это формальный язык, конструкциями которого являются формулы, описывающие свойства объектов и отношения между ними.

Формально язык логики предикатов первого порядка задаётся алфавитом, грамматикой, определяющей множество всех правильно сформированных термов и формул, и сигнатурой рассматриваемой теории (набором предикатных и функциональных символов с их арностями).

Как и в логике высказываний, язык логики предикатов можно рассматривать как подмножество операции замыкания Клини Σ^* над его алфавитом Σ без пустых строк, ограниченное грамматикой языка логики предикатов.

4.4.2 Алфавит языка логики предикатов

Алфавит языка логики предикатов можно задать при помощи формы Бэкуса-Наура:

<логическая константа> ::= T | $\perp$

<символ> ::= A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z

$\langle \text{ненулевая цифра} \rangle ::= 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$
 $\langle \text{цифра} \rangle ::= 0 \mid \langle \text{ненулевая цифра} \rangle$
 $\langle \text{цифры} \rangle ::= \langle \text{цифра} \rangle \mid \langle \text{цифра} \rangle \langle \text{цифры} \rangle$
 $\langle \text{натуральное число} \rangle ::= \langle \text{ненулевая цифра} \rangle \mid \langle \text{ненулевая цифра} \rangle \langle \text{цифры} \rangle$
 $\langle \text{предикатное имя} \rangle ::= \langle \text{символ} \rangle \mid \langle \text{символ} \rangle \langle \text{натуральное число} \rangle$
 $\langle \text{конечный строчный символ} \rangle ::= z \mid y \mid x \mid w \mid v \mid u \mid t \mid s$
 $\langle \text{предметная переменная} \rangle ::= \langle \text{конечный строчный символ} \rangle$
 $\quad \mid \langle \text{конечный строчный символ} \rangle \langle \text{натуральное число} \rangle$
 $\langle \text{начальный строчный символ} \rangle ::= a \mid b \mid c \mid d \mid e \mid f \mid g \mid h$
 $\langle \text{константное имя} \rangle ::= \langle \text{начальный строчный символ} \rangle$
 $\quad \mid \langle \text{начальный строчный символ} \rangle \langle \text{натуральное} \rangle$
 $\langle \text{запятая} \rangle ::= ,$
 $\langle \text{открывающая полукруглая скобка} \rangle ::= ($
 $\langle \text{закрывающая полукруглая скобка} \rangle ::=)$
 $\langle \text{терм} \rangle ::= \langle \text{предметная переменная} \rangle \mid \langle \text{начальный строчный символ} \rangle$
 $\quad \mid \langle \text{начальный строчный символ} \rangle \langle \text{список термов} \rangle$
 $\langle \text{список термов} \rangle ::= \langle \text{открывающая полукруглая скобка} \rangle \langle \text{термы} \rangle$
 $\quad \langle \text{закрывающая полукруглая скобка} \rangle$
 $\langle \text{термы} \rangle ::= \langle \text{терм} \rangle \mid \langle \text{терм} \rangle \langle \text{запятая} \rangle \langle \text{термы} \rangle$
 $\langle \text{квантор} \rangle ::= \langle \text{всеобщность} \rangle \mid \langle \text{существование} \rangle$
 $\langle \text{всеобщность} \rangle ::= \forall$
 $\langle \text{существование} \rangle ::= \exists$
 $\langle \text{логическая связка} \rangle ::= \langle \text{унарная логическая связка} \rangle \mid \langle \text{бинарная логическая связка} \rangle$
 $\langle \text{унарная логическая связка} \rangle ::= \langle \text{отрицание} \rangle$
 $\langle \text{отрицание} \rangle ::= \neg$
 $\langle \text{бинарная логическая связка} \rangle ::= \langle \text{дизъюнкция} \rangle \mid \langle \text{конъюнкция} \rangle \mid \langle \text{импликация} \rangle$
 $\quad \mid \langle \text{эквиваленция} \rangle$
 $\langle \text{дизъюнкция} \rangle ::= \vee$
 $\langle \text{конъюнкция} \rangle ::= \wedge$
 $\langle \text{импликация} \rangle ::= \rightarrow$
 $\langle \text{эквиваленция} \rangle ::= \sim$

В алфавит языка логики предикатов входят: логические константы ($\top$ — истина, $\perp$ — ложь), предикатные имена (символы A–Z с необязательными натуральными индексами), предметные переменные (строчные символы z, y, x, w, v, u, t, s с необязательными натуральными индексами), константные имена (строчные символы a–h с необязательными натуральными индексами), кванторы всеобщности ($\forall$) и существования ($\exists$), унарная логическая связка отрицания ($\neg$), бинарные логические связки дизъюнкции ($\vee$), конъюнкции ($\wedge$), импликации ($\rightarrow$) и эквиваленции ($\sim$), а также вспомогательные символы — полукруглые скобки (и) и запятая (,).

Таким образом, в алфавите языка логики предикатов используется 64 различных символов, образующих знаки этого алфавита.

4.4.3 Грамматика языка логики предикатов

Грамматика языка логики предикатов определяет правила построения правильно сформированных термов и формул из символов алфавита.

Граматику языка логики предикатов можно задать при помощи формы Бэкуса-Наура:

```
<логическая формула> ::= <логическая константа>
                        | <атомарная логическая формула>
                        | <неатомарная логическая формула>
<атомарная логическая формула> ::= <атомарная предикатная формула>
<атомарная предикатная формула> ::= <предикатное имя>
                                     | <предикатное имя> <список термов>
<неатомарная логическая формула> ::= <унарная неатомарная логическая формула>
                                     | <бинарная неатомарная логическая формула>
                                     | <кванторная формула>
<унарная неатомарная логическая формула> ::= <открывающая круглая скобка>
                                                <унарная логическая связка>
                                                <логическая формула>
                                                <закрывающая круглая скобка>
<бинарная неатомарная логическая формула> ::= <открывающая круглая скобка>
                                                <логическая формула>
                                                <бинарная логическая связка>
                                                <логическая формула>
                                                <закрывающая круглая скобка>
<кванторная формула> ::= <открывающая круглая скобка>
                           <квантор>
                           <предметная переменная>
                           <логическая формула>
                           <закрывающая круглая скобка>
```

Примеры логических формул языка логики предикатов:

- $P(x)$,
- $R(f(x), y)$,
- $(\neg P(x))$,
- $((P(x) \vee Q(x,y)) \rightarrow R(y))$,
- $(\forall x P(x))$,
- $(\exists y (P(y) \wedge Q(f(y))))$.

4.4.4 Классификация логических формул языка логики предикатов

В языке логики предикатов *логические формулы* бывают *атомарными* и *неатомарными*.

4.4.4.1 Понятие атомарной логической формулы

Атомарная логическая формула — это логическая формула, которая не является неатомарной логической формулой.

Часто *атомарную логическую формулу* в языке логики предикатов называют *атомарной предикатной формулой*.

4.4.4.2 Понятие атомарной предикатной формулы

Атомарная предикатная формула — это логическая формула вида $P(t_1, \dots, t_n)$, которая не содержит логических связок и кванторов.

Пример. Атомарными предикатными формулами в языке логики предикатов являются:

- $P(x)$,
- $Q(f(x), y)$,
- $R(a, f(b))$.

4.4.4.3 Понятие неатомарной логической формулы

Неатомарная логическая формула — это логическая формула, элементами которой являются другие формулы.

Пример. Неатомарными логическими формулами в языке логики предикатов являются:

- $(\neg P(x))$,
- $(P(x) \wedge Q(x, y))$,
- $(\forall x P(x))$,
- $(\exists y (P(y) \rightarrow Q(f(y))))$,
- $((\forall x (\exists y (R(x, y))) \rightarrow (\neg S(y))))$.

4.4.5 Классификация предметных переменных

В логике предикатов важно различать *свободные* и *связанные* вхождения переменных.

4.4.5.1 Понятие связанной предметной переменной логической формулы

Связанная предметная переменная логической формулы — это предметная переменная в заданной логической формуле, вхождение которой находится в области действия квантора ($\forall$ или $\exists$), связывающего эту переменную.

4.4.5.2 Понятие свободной предметной переменной логической формулы

Свободная предметная переменная логической формулы — это предметная переменная в заданной логической формуле, которая не является связанной.

Пример. В формуле $(\forall x P(x, y))$ переменная x связана квантором всеобщности, а переменная y свободна. В формуле $(\exists x (\forall y P(x, y, u)))$ переменные x и y — связанные, а переменная u — свободная. Одна и та же переменная может быть одновременно свободной и связанной в одной формуле, например, в формуле $((\forall x P(x, y)) \wedge (\forall y Q(y)))$ переменная y является и свободной, и связанной.

4.4.5.3 Понятие замкнутой логической формулы

Замкнутая логическая формула — это логическая формула, в которой все переменные связаны; или это логическая формула, в которой нет свободных переменных.

4.4.5.4 Понятие открытой логической формулы

Открытая логическая формула — это логическая формула, которая не является замкнутой.

Пример. Формула $(\forall x P(x))$ является замкнутой, так как переменная x связана квантором всеобщности. Формула $(\forall x (\exists y (Q(x, y))))$ также замкнута, поскольку обе переменные связаны. Напротив, формула $P(x, y)$ является открытой, так как обе переменные свободны.

4.5 Алгебра предикатов

Алгебра предикатов изучает семантику формул логики предикатов через понятия предиката и модели.

Интерпретация формул логики предикатов задаётся на основе модели.

4.5.1 Понятие предиката

Предикат — это характеристическая функция, областью определения которой является множество объектов предметной области, а множеством значений — множество истинностных значений.

Предикат является обобщением пропозициональной переменной: при $n = 0$ предикат совпадает с логической константой.

Унарный предикат ($n = 1$) характеризует свойство объекта предметной области; бинарный предикат ($n = 2$) — отношение между двумя объектами предметной области; нулевой предикат ($n = 0$) — константное истинностное значение.

Пример. Предикат $P(x, y)$ характеризует отношение “больше” ($x > y$).

4.5.2 Понятие модели в алгебре предикатов

Модель (структура, или интерпретация) для языка логики предикатов — это пара $M = \langle D, I \rangle$, где:

- D — это носитель, непустое множество объектов предметной области;
- I — это функция интерпретации, которая:
 - каждому n -местному предикату P ставит в соответствие n -местное отношение $I(P) \subseteq D^n$;
 - каждому n -местному функциональному символу f ставит в соответствие функцию $I(f) \in D^{D^n}$;
 - каждому константному имени c ставит в соответствие объект предметной области $I(c) \in D$.

Пример.

$D = \{1, 2, 3\}$,

$I(P) = \{\langle 1, 2 \rangle, \langle 2, 3 \rangle\}$ — бинарный предикат «меньше» на $\{1, 2, 3\}$,

$I(f)(x) = x + 1 \bmod 3$ — функциональный символ,

$I(a) = 1$ — константа предметной области.

4.5.3 Семантика логических связок

Семантика логических связок $\neg$, $\vee$, $\wedge$, $\rightarrow$, $\sim$ в логике предикатов совпадает с логикой высказываний: истинностное значение неатомарных формул определяется по истинностным таблицам от значений их подформул для фиксированной модели M и оценки переменных σ .

$M, \sigma \models \varphi$ означает, что формула φ истинна в модели M при оценке переменных σ .

Примеры:

$M, \sigma \models (\neg \varphi) \Leftrightarrow \text{не } M, \sigma \models \varphi$

$M, \sigma \models (\varphi \wedge \psi) \Leftrightarrow M, \sigma \models \varphi \text{ и } M, \sigma \models \psi$

$$M, \sigma \models (\varphi \vee \psi) \Leftrightarrow M, \sigma \models \varphi \text{ или } M, \sigma \models \psi$$

$$M, \sigma \models (\varphi \rightarrow \psi) \Leftrightarrow \text{если } M, \sigma \models \varphi, \text{ то } M, \sigma \models \psi$$

4.5.4 Семантика кванторов

Кванторы определяют истинностное значение формулы путём проверки всех возможных значений переменной в предметной области.

$M, \sigma \models (\forall x \varphi(x))$ тогда и только тогда, когда для любого элемента $d \in D$ выполняется $M, \sigma[x \mapsto d] \models \varphi(x)$.

$M, \sigma \models (\exists x \varphi(x))$ тогда и только тогда, когда существует элемент $d \in D$ такой, что $M, \sigma[x \mapsto d] \models \varphi(x)$.

$\sigma[x \mapsto d]$ — оценка переменных, совпадающая с оценкой переменных σ на всех переменных кроме x , которому присвоено значение d .

Пример. $M = \langle \{1, 2, 3\}, I(P) = \langle \langle x \text{ чётное} \rangle \rangle$

Тогда:

$$(\exists x P(x)): d = 2 \rightarrow P(2) = \top.$$

$$(\forall x P(x)): d = 1 \rightarrow P(1) = \perp.$$

4.5.5 Понятие равносильности двух логических формул

Две формулы логики предикатов A и B равносильны ($A \Leftrightarrow B$), если для любой модели M и любой оценки σ : $M, \sigma \models A$ тогда и только тогда, когда $M, \sigma \models B$.

4.5.5.1 Примеры отношений равносильности логических формул

4.5.5.1.1 Законы коммутативности кванторов

$$(\forall \chi (\forall \lambda \alpha(\chi, \lambda))) \Leftrightarrow (\forall \lambda (\forall \chi \alpha(\chi, \lambda)))$$

$$(\exists \chi (\exists \lambda \alpha(\chi, \lambda))) \Leftrightarrow (\exists \lambda (\exists \chi \alpha(\chi, \lambda)))$$

Одноимённые кванторы коммутируют; разноимённые — не коммутируют в общем случае.

4.5.5.1.2 Законы дистрибутивности кванторов

$$(\forall \chi (\alpha(\chi) \wedge \beta(\chi))) \Leftrightarrow ((\forall \chi \alpha(\chi)) \wedge (\forall \chi \beta(\chi)))$$

$$(\exists \chi (\alpha(\chi) \vee \beta(\chi))) \Leftrightarrow ((\exists \chi \alpha(\chi)) \vee (\exists \chi \beta(\chi)))$$

Кванторы «распределяются» только над своими «совместимыми» связками.

4.5.5.1.3 Законы выноса констант

$$(\forall x (\alpha(x) \wedge \gamma)) \Leftrightarrow ((\forall x \alpha(x)) \wedge \gamma)$$

$$(\forall x (\alpha(x) \vee \gamma)) \Leftrightarrow ((\forall x \alpha(x)) \vee \gamma)$$

$$(\exists x (\alpha(x) \wedge \gamma)) \Leftrightarrow ((\exists x \alpha(x)) \wedge \gamma)$$

$$(\exists x (\alpha(x) \vee \gamma)) \Leftrightarrow ((\exists x \alpha(x)) \vee \gamma)$$

4.5.5.1.4 Законы двойственности кванторов

$$(\neg(\forall x \alpha(x))) \Leftrightarrow (\exists x (\neg\alpha(x)))$$

$$(\neg(\exists x \alpha(x))) \Leftrightarrow (\forall x (\neg\alpha(x)))$$

Отрицание квантора одного типа даёт квантор другого типа.

4.5.5.2 Понятие равносильного преобразования логической формулы

Равносильные преобразования логической формулы в логике предикатов — это последовательность замен подформул законами равносильности, включающими как пропозициональные законы (из алгебры высказываний), так и законы кванторов.

4.5.5.2.1 Понятие предварённой нормальной формы логической формулы

Предварённая нормальная форма логической формулы, или пренексная нормальная форма логической формулы — это форма, в которой все кванторы вынесены в префикс (начало) логической формулы, а за ними стоит бескванторная подформула, содержащая только логические связки и записанная в нормальной форме:

$$(Q_1 x_1 (Q_2 x_2 \dots (Q_n x_n \varphi(x_1, \dots, x_n))))),$$

где каждый $Q_i \in \{\forall, \exists\}$, а φ — формула без кванторов.

Пример. Привести $((\neg(\forall x P(x))) \rightarrow (\exists y Q(y)))$ к ПНФ.

Решение:

$$\begin{aligned}
& ((\neg(\forall x P(x))) \rightarrow (\exists y Q(y))) \\
& \Leftrightarrow ((\neg(\neg(\forall x P(x)))) \vee (\exists y Q(y))) \quad [\text{выражение импликации}] \\
& \Leftrightarrow ((\forall x P(x)) \vee (\exists y Q(y))) \quad [\text{замена двойного отрицания}] \\
& \Leftrightarrow (\forall x (P(x) \vee (\exists y Q(y)))) \quad [\text{вынос } \forall x \text{ за } \vee, x \text{ не входит в } Q(y)] \\
& \Leftrightarrow (\forall x (\exists y (P(x) \vee Q(y)))) \quad [\text{вынос } \exists y \text{ за } \vee] \\
& \text{Ответ: } (\forall x (\exists y (P(x) \vee Q(y)))) \text{ — ПНФ.}
\end{aligned}$$

Любую формулу логики предикатов можно равносильно преобразовать к предварённой нормальной форме при помощи последовательности корректных равносильных преобразований (устранение импликаций, вынесение отрицаний, переименование связанных переменных и т.п.).

4.5.5.2 Понятие сколемовской стандартной формы логической формулы

Сколемовская стандартная форма логической формулы — это форма заданной логической формулы, получаемая из логической формулы в предварённой нормальной форме путём устранения существующих кванторов с помощью введения новых функциональных термов (сколемовских функций).

Процедура сколемизации заключается в следующем. Если $\exists x$ встречается в области действия $\forall y_1 \dots \forall y_n$, то x заменяется сколемовской функцией $f(y_1, \dots, y_n)$. Если $\exists x$ не находится в области ни одного $\forall$, то x заменяется сколемовской константой c . В конце все кванторы всеобщности опускаются.

При сколемизации сохраняется выполнимость, но в общем случае не сохраняется полная логическая эквивалентность.

Пример 1. Для логической формулы в ПНФ — $(\forall x (\exists y P(x, y)))$ — ССФ является $P(x, f(x))$.

Пример 2. Для логической формулы в ПНФ — $(\exists x (\forall y (\forall z (\exists w P(x, y, z, w))))$ — ССФ является $P(a, y, z, f(y, z))$.

4.6 Исчисление предикатов

Исчисление предикатов — это множество доказуемых логических формул языка логики предикатов.

В самом общем виде под исчислением предикатов понимают формальную систему, заданную:

- (1) алфавитом языка логики предикатов,
- (2) грамматикой языка логики предикатов, определяющей множество всех правильных формул этого языка,
- (3) системой аксиомных схем,
- (4) системой правил вывода.

4.6.1 Понятие аксиомы языка логики предикатов

Внутри исчисления предикатов аксиомы играют роль исходных, бездоказательных формул, от которых начинаются формальные доказательства. Система аксиом включает пропозициональные схемы (по сути, те же, что и в исчислении высказываний, но на формулы первого порядка), а также схемы, описывающие поведение кванторов.

Аксиома языка логики предикатов — это логическая формула логики предикатов, объявленная истинной без доказательства в рамках данного исчисления и получаемая в результате подстановки формул логики предикатов в некоторую аксиомную схему.

4.6.2 Аксиомные схемы исчисления предикатов

Система аксиомных схем исчисления предикатов включает:

- пропозициональные аксиомные схемы — все аксиомные схемы исчисления высказываний, в которых метаварiable α , β , γ пробегают формулы логики предикатов;
- аксиомы кванторов — аксиомные схемы, связывающие формулы с их универсальными и экзистенциальными обобщениями.

4.6.2.1 Пропозициональные аксиомные схемы

Все аксиомные схемы исчисления высказываний из пункта 4.3.1.1 (введение и удаление импликации, конъюнкции, дизъюнкции, отрицания и эквиваленции) сохраняются в исчислении предикатов, если рассматривать α , β , γ как произвольные формулы логики предикатов.

4.6.2.2 Аксиомные схемы кванторов

4.6.2.2.1 Аксиомная схема удаления квантора всеобщности

$((\forall \chi \alpha(\chi)) \rightarrow \alpha(\tau))$, где τ свободно подставим вместо χ в α .

Схема выражает, что если некоторое свойство $\alpha(\chi)$ выполняется для всех элементов предметной области, то оно выполняется и для любого конкретного терма τ .

4.6.2.2 Аксиомная схема введения квантора всеобщности

$(\alpha(\tau) \rightarrow (\exists \chi \alpha(\chi)))$, где τ свободно подставим вместо χ в α .

Схема отражает, что если формула $\alpha(\tau)$ истинна для некоторого конкретного терма τ , то существует элемент, для которого формула $\alpha(\chi)$ истинна.

4.6.3 Понятие формального логического вывода в исчислении предикатов

Формальный логический вывод в исчислении предикатов — это конечная последовательность $\langle \varphi_1, \varphi_2, \dots, \varphi_n \rangle$ формул логики предикатов, где каждая φ_i является либо аксиомой, либо элементом множества гипотез G , либо результатом применения правил вывода к предыдущим формулам последовательности.

4.6.4 Понятие теоремы языка логики предикатов

Теорема языка логики предикатов — это формула, для которой существует формальный логический вывод в исчислении предикатов из пустого множества гипотез.

Обозначение $\vdash \{\varphi\}$ означает, что формула φ выводима без дополнительных предположений и считается теоремой рассматриваемой аксиоматической системы.

Если вывод формулы φ осуществляется из множества гипотез G , то пишут $G \vdash \{\varphi\}$; при $G = \emptyset$ запись сокращают до $\vdash \{\varphi\}$.

4.6.5 Понятие отношения выводимости в логике предикатов

Отношение выводимости ($\vdash$) в логике предикатов — это синтаксическое отношение между множеством формул G и формулами, для которых существует формальный вывод из G в исчислении предикатов.

Запись $G \vdash F$ означает, что множество формул F выводимо из G . Как и в случае логики высказываний, отношение выводимости не зависит от конкретной семантической интерпретации, а определяется только аксиомами и правилами вывода формальной системы.

4.6.6 Правила вывода в исчислении предикатов

Правило вывода в исчислении предикатов задаёт допустимый переход от одной или нескольких формул-посылок к формуле-заключению. В отличие от аксиом, правила вывода не входят в список формул, но определяют корректные шаги построения вывода.

4.6.6.1 Правило Modus Ponens

$$(G \cup \{\alpha, (\alpha \rightarrow \beta)\}) \vdash (G \cup \{\beta\})$$

Правило Modus Ponens в исчислении предикатов формулируется так же, как в исчислении высказываний: если в выводе уже получены формула α и импликация $(\alpha \rightarrow \beta)$, то разрешается заключить формулу β .

Пример. Доказать: $\vdash \{((\forall x P(x)) \rightarrow (\exists x P(x)))\}$.

Решение: Ответ:

1. $((\forall x P(x)) \rightarrow P(x))$ [схема удаления квантора всеобщности]
2. $(P(x) \rightarrow (\exists x P(x)))$ [схема введения квантора существования]
3. $((\forall x P(x)) \rightarrow P(x)) \rightarrow (((\forall x P(x)) \rightarrow (P(x) \rightarrow (\exists x P(x)))) \rightarrow ((\forall x P(x) \rightarrow (\exists x P(x))))$ [схема введения импликации]
4. $((\forall x P(x)) \rightarrow (P(x) \rightarrow (\exists x P(x)))) \rightarrow ((\forall x P(x) \rightarrow (\exists x P(x))))$ [MP из 1 и 3]
5. $((P(x) \rightarrow (\exists x P(x))) \rightarrow ((\forall x P(x)) \rightarrow (P(x) \rightarrow (\exists x P(x))))$ [схема введения импликации]
6. $((\forall x P(x)) \rightarrow (P(x) \rightarrow (\exists x P(x))))$ [MP из 2 и 5]
7. $((\forall x P(x)) \rightarrow (\exists x P(x)))$ [MP из 6 и 4] $\square$

4.6.6.2 Правило введения квантора всеобщности

$$(G \cup \{(\gamma \rightarrow \alpha(\chi))\}) \vdash (G \cup \{(\gamma \rightarrow (\forall \chi \alpha(\chi)))\})$$

Пример. Доказать: $\vdash \{((\forall x P(x)) \rightarrow (\forall y P(y)))\}$.

Решение: Ответ:

1. $((\forall x P(x)) \rightarrow P(y))$ [схема удаления квантора всеобщности]
2. $((\forall x P(x)) \rightarrow (\forall y P(y)))$ [правило введения квантора всеобщности] $\square$

4.6.6.3 Правило удаления квантора существования

$$(G \cup \{(\alpha(\chi) \rightarrow \gamma)\}) \vdash (G \cup \{((\exists \chi \alpha(\chi)) \rightarrow \gamma)\})$$

Пример. Доказать: $\vdash \{((\exists x P(x)) \rightarrow (\exists y P(y)))\}$.

Решение: Ответ:

1. $(P(x) \rightarrow (\exists y P(y)))$ [схема введения квантора существования]
2. $((\exists x P(x)) \rightarrow (\exists y P(y)))$ [правило удаления квантора существования] $\boxtimes$

4.6.7 Метатеоремы исчисления предикатов

Метатеорема исчисления предикатов — это утверждение о самой аксиоматической системе логики предикатов, формулируемое и доказываемое на метаязыке. К метатеоремам относятся варианты метатеоремы дедукции, теорема о полноте, теорема о корректности (адекватности) и теорема о компактности.

4.6.8 Правило резолюции

4.6.9 Понятие унификации

4.7 Формализация высказываний на языке логики предикатов

Формализация высказываний на языке логики предикатов — это процесс перевода высказываний, записанных на естественном языке (или на языке предметной области), в строгие логические формулы языка логики предикатов. Результатом формализации является формула, точно передающая смысл исходного высказывания и допускающая формальную обработку: проверку выполнимости, построение вывода, верификацию.

Необходимость формализации обусловлена фундаментальным противоречием между естественным языком и формальными системами: естественный язык допускает многозначность, эллипсис, контекстную зависимость и прагматические импликатуры. Язык логики предикатов, напротив, однозначен синтаксически и семантически. Переход между ними требует явного выбора предметной области, сигнатуры предикатов и функций, а также корректного расставления кванторов и связок.

Формализация высказываний применяется: в интеллектуальных системах — для построения баз знаний и правил вывода; в автоматическом доказательстве теорем — как входные данные для резолюционных процедур; в программировании на основе логики (Prolog) — как описание фактов и запросов; в верификации программ — как предусловия, постусловия и инварианты (логика Хоара).

4.7.1 Основные компоненты формализации

Процесс формализации всегда предполагает явное определение четырёх компонент: предметной области, сигнатуры и самих формул.

Предметная область D представляет собой непустое множество объектов, о которых ведётся рассуждение. Точное задание D необходимо, поскольку истинность одной и той же формулы зависит от D . Формула, истинная в одной предметной области, может быть ложной в другой.

Сигнатура представляет собой конечный набор предикатных символов и функциональных термов с указанием их ариности (числа аргументов).

Пример сигнатуры для предметной области «сотрудники предприятия»:

Терм	Тип терма	Ариность	Значение
$E(x)$	Предикат	1	x является сотрудником
$M(x)$	Предикат	1	x является менеджером
$S(x, y)$	Предикат	2	x непосредственно подчинён y
$D(x, d)$	Предикат	2	x работает в отделе d
$S(x, s)$	Предикат	2	зарплата сотрудника x равна s
director	Константа	0	конкретный директор предприятия

4.7.2 Алгоритм формализации

Формализация естественно-языкового высказывания выполняется в следующей последовательности шагов.

Шаг 1. Определить предметную область D .

Шаг 2. Выбрать сигнатуру: именовать предикаты, функциональные термы, константы.

Шаг 4. Определить область действия кванторов.

Шаг 5. Записать атомарные подформулы.

Шаг 6. Соединить атомарные подформулы логическими связками.

Шаг 7. Проверить: каждая переменная либо связана квантором, либо явно свободна.

Шаг 8. Убедиться, что формула истинна/ложна ровно на тех объектах, на которых должно быть истинно/ложно исходное высказывание.

4.7.3 Формализация атомарных высказываний

Атомарное высказывание описывает одно свойство или одно отношение. Оно формализуется атомарной предикатной формулой $P(t_1, \dots, t_n)$, где P —

предикатный символ, $t_1, \dots, t_n$ — термы (переменные, константы или функциональные термы).

Примеры:

Высказывание	Логическая формула
«Иван — сотрудник»	$E(a)$
«Иван подчинён Марии»	$S(a, b)$
«Зарплата Ивана равна 50 000»	$M(a, e)$
«Олег является отцом Ивана»	$F(a, d)$
« $f(x)$ является простым числом»	$P(f(x))$

4.7.4 Формализация составных высказываний

Составные высказывания строятся из атомарных с помощью логических связок. При формализации важно корректно определить, какая связка соответствует союзу естественного языка.

4.7.4.1 Формализация отрицания

Отрицание формализуется связкой $\neg$. Частица «не», «неверно, что», «никакой» (в определённых контекстах) — типичные индикаторы отрицания.

Высказывание	Логическая формула
«Иван не является менеджером»	$(\neg M(a))$
«Неверно, что x подчинён y »	$(\neg S(x, y))$
« x не является ни менеджером, ни директором»	$((\neg M(x)) \wedge (\neg D(x)))$

4.7.4.2 Формализация конъюнкции

Союзы «и», «а также», «при этом», «но» в большинстве случаев формализуются конъюнкцией $\wedge$.

Высказывание	Логическая формула
« x — сотрудник и менеджер»	$(E(x) \wedge M(x))$
« x работает в отделе z и подчинён y »	$(D(x, z) \wedge S(x, y))$
« x — сотрудник, зарплата которого равна s , и он не менеджер»	$(E(x) \wedge S(x, s) \wedge (\neg M(x)))$

4.7.4.3 Формализация дизъюнкции

Союз «или» чаще всего формализуется дизъюнкцией $\vee$. Исключающее «или» формализуется через: $((A \vee B) \wedge (\neg(A \wedge B)))$, то есть ровно одно из двух истинно.

Высказывание	Логическая формула
«x — менеджер или директор»	$(M(x) \vee D(x))$
«x получает зарплату s_1 или s_2 »	$(S(x, s_1) \vee S(x, s_2))$
«x выполняет функцию A исключительно или функцию B»	$((A(x) \vee B(x)) \wedge (\neg(A(x) \wedge B(x))))$

4.7.4.4 Формализация импликации

Условные конструкции «если ..., то ...», «при условии ...», «всякий ... является ...» формализуются импликацией $\rightarrow$.

Стоит отметить, что импликация используется именно для выражения причинно-следственных и условных связей, а не хронологических.

Высказывание	Логическая формула
«Если x — менеджер, то x — сотрудник»	$(M(x) \rightarrow E(x))$
«Если x подчинён y, то зарплата x не превышает зарплату y»	$(S(x, y) \rightarrow L(x, y))$
«Только сотрудники получают зарплату»	$(S(x, s) \rightarrow E(x))$

Фраза «только если» формализуется как $(A \rightarrow B)$, а не как $(B \rightarrow A)$. Фраза «тогда и только тогда, когда» формализуется эквиваленцией $(A \sim B)$.

4.7.4.5 Формализация эквиваленции

Конструкции «тогда и только тогда, когда», «эквивалентно», «необходимо и достаточно» формализуются эквиваленцией $\sim$.

Высказывание	Логическая формула
«x является директором тогда и только тогда, когда x управляет всеми»	$(D(x) \sim (\forall y (S(y, x))))$
«x — менеджер тогда и только тогда, когда у x есть хотя бы один подчинённый»	$(M(x) \sim (\exists y S(y, x)))$

4.7.5 Формализация высказываний с кванторами

Кванторы выражают обобщения: квантор всеобщности $\forall$ — «для всех объектов», квантор существования $\exists$ — «существует хотя бы один объект». Корректное расставление кванторов — наиболее сложная часть формализации, поскольку порядок кванторов существенно влияет на смысл формулы.

4.7.5.1 Формализация квантора всеобщности

Фразы «все», «каждый», «любой», «для всякого» формализуются через $\forall$.

Для высказывания «каждый объект, обладающий свойством А, обладает свойством В» результатом формализации является $(\forall x (A(x) \rightarrow B(x)))$. Использование $\wedge$ вместо $\rightarrow$ при $\forall$ является типичной ошибкой: $(\forall x (A(x) \wedge B(x)))$ утверждает, что все объекты обладают сразу обоими свойствами.

Выражение	Логическая формула	Схема
«Все сотрудники имеют зарплату»	$(\forall x (E(x) \rightarrow (\exists s S(x, s))))$	$\forall \rightarrow \exists$
«Каждый менеджер является сотрудником»	$(\forall x (M(x) \rightarrow E(x)))$	$\forall \rightarrow$
«Любой подчинённый директора является менеджером»	$(\forall x (S(x, a) \rightarrow M(x)))$	$\forall \rightarrow$
«Никакой менеджер не подчинён рядовому сотруднику»	$(\forall x (M(x) \rightarrow (\forall y ((\neg M(y)) \rightarrow (\neg S(x, y)))))$	$\forall \rightarrow \forall \rightarrow \neg$

4.7.5.2 Формализация квантора существования

Фразы «существует», «некоторый», «есть хотя бы один», «найдётся» формализуются через $\exists$.

Для высказывания «существует объект с свойствами А и В» результатом формализации является $(\exists x (A(x) \wedge B(x)))$. Использование $\rightarrow$ вместо $\wedge$ при $\exists$ является типичной ошибкой: $(\exists x (A(x) \rightarrow B(x)))$ истинна, когда есть хотя бы один объект, для которого А ложно.

Выражение	Логическая формула	Схема
«Существует сотрудник с зарплатой 50 000»	$(\exists x (E(x) \wedge S(x, a)))$	$\exists \wedge$
«Некоторый менеджер не имеет подчинённых»	$(\exists x (M(x) \wedge (\neg(\exists y S(y, x)))))$	$\exists \wedge \neg \exists$
«У каждого сотрудника есть хотя бы один коллега»	$(\forall x (E(x) \rightarrow (\exists y (E(y) \wedge (\neg(C(x, y)))))$	$\forall \rightarrow \exists \wedge \neg$

4.7.5.3 Формализация вложенных кванторов

Вложение кванторов задаёт зависимость между переменными. Принципиально важен порядок. $(\forall x (\exists y \varphi(x, y)))$ и $(\exists y (\forall x \varphi(x, y)))$ —

разные формулы с различными истинностными значениями в большинстве моделей.

Выражение	Логическая формула	Примечание
«Каждый сотрудник имеет некоторого менеджера»	$(\forall x (E(x) \rightarrow (\exists y (M(y) \wedge S(x, y)))))$	для каждого x — свой y
«Существует менеджер, которому подчинены все сотрудники»	$(\exists y (M(y) \wedge (\forall x (E(x) \rightarrow S(x, y)))))$	один y для всех x
«Для любых двух сотрудников один из них подчинён другому»	$(\forall x (\forall y (((E(x) \wedge E(y)) \wedge (\neg R(x, y))) \rightarrow (S(x, y) \vee S(y, x)))))$	линейный порядок
«Никакой сотрудник не является своим собственным подчинённым»	$(\forall x (S(x, x)))$	нерефлексивность

4.7.6 Формализация специальных конструкций естественного языка

Ряд конструкций естественного языка не имеет прямого синтаксического соответствия в логике предикатов и требует особой техники формализации.

4.7.6.1 «Единственный» (существует ровно один)

Конструкция «существует единственный x такой, что $\varphi(x)$ » обозначается $\exists !x \varphi(x)$ и разворачивается как:

$$(\exists !x \varphi(x)) \Leftrightarrow (\exists x (\varphi(x) \wedge (\forall y (\varphi(y) \rightarrow R(y, x)))))$$

Читается: «существует x , для которого φ , и любой y , удовлетворяющий φ , совпадает с x ».

Высказывание	Логическая формула
«Существует единственный директор»	$(\exists x (D(x) \wedge (\forall y (D(y) \rightarrow R(y, x)))))$
«У каждого сотрудника ровно один непосредственный руководитель»	$(\forall x (E(x) \rightarrow (\exists y ((M(y) \wedge S(x, y)) \wedge (\forall z ((M(z) \wedge S(x, z)) \rightarrow R(z, y)))))$

4.7.6.2 «Не менее n », «не более n », «ровно n » объектов

Утверждения о количестве объектов формализуются через вложенные кванторы и предикат равенства.

«Существует не менее двух менеджеров»:

$$(\exists x (\exists y ((M(x) \wedge M(y)) \wedge (\neg R(x, y)))))$$

«Существует не более одного директора»:

$$(\forall x (\forall y ((D(x) \wedge D(y)) \rightarrow R(x, y)))$$

«Существует ровно два менеджера»:

$$(\exists x (\exists y ((M(x) \wedge M(y)) \wedge (\neg R(x, y)) \wedge (\forall z (M(z) \rightarrow (R(z, x) \vee R(z, y)))))))$$

4.7.6.3 «Большинство», «почти все» — пределы формализации

Конструкции «большинство», «почти все», «обычно» не выражаются средствами стандартной логики предикатов первого порядка. Они требуют квантификации второго порядка (логика второго порядка). Это принципиальное ограничение логики первого порядка.

Не формализуется в логике предикатов первого порядка:

«Большинство менеджеров имеют высокую зарплату».

4.7.7 Типичные ошибки формализации

При формализации систематически встречаются следующие классы ошибок. Знание этих ошибок позволяет проверять формулы до их использования в логическом выводе.

Ошибка	Неверная формула	Верная формула
$\wedge$ вместо $\rightarrow$ при $\forall$	$(\forall x (M(x) \wedge E(x)))$ [«все объекты — одновременно менеджеры и сотрудники»]	$(\forall x (M(x) \rightarrow E(x)))$ [«каждый менеджер — сотрудник»]
$\rightarrow$ вместо $\wedge$ при $\exists$	$(\exists x (M(x) \rightarrow H(x)))$ [истинна тривиально при не-менеджере]	$(\exists x (M(x) \wedge H(x)))$ [«есть менеджер с высокой зарплатой»]
Неверный порядок кванторов	$(\exists y (\forall x S(x, y)))$ [«один начальник над всеми»]	$(\forall x (\exists y S(x, y)))$ [«у каждого есть начальник»]

4.7.8 Примеры формализации

4.7.8.1 Пример. Формализация аксиом теории графов

Предметная область: вершины графа.

Сигнатура:

$V(x)$ — x является вершиной.

$E(x, y)$ — существует ребро от x к y .

$P(x, y)$ — существует путь от x до y .

Формализация:

1. Рефлексивность пути:

$$(\forall x (V(x) \rightarrow P(x, x)))$$

2. Если есть ребро, то есть путь:

$$(\forall x (\forall y (E(x, y) \rightarrow P(x, y))))$$

3. Транзитивность пути:

$$(\forall x (\forall y (\forall z ((P(x, y) \wedge P(y, z)) \rightarrow P(x, z))))))$$

4. Граф связан (из каждой вершины достижима каждая):

$$(\forall x (\forall y ((V(x) \wedge V(y)) \rightarrow P(x, y))))$$

4.7.8.2 Пример. Формализация условий доступа в системе безопасности

Предметная область: пользователи системы и ресурсы.

Сигнатура:

$U(u)$ — u является пользователем.

$R(r)$ — r является ресурсом.

$T(u, t)$ — пользователь u имеет роль t .

$A(u, r)$ — пользователь u имеет доступ к ресурсу r .

a, b — константы (роли).

c — константа (секретный ресурс).

Политика доступа (формализация требований):

1. Только зарегистрированные пользователи имеют доступ к ресурсам:

$$(\forall u (\forall r (A(u, r) \rightarrow U(u)))).$$

2. Администраторы имеют доступ ко всем ресурсам:

$$(\forall u (\forall r (((U(u) \wedge T(u, a)) \wedge R(r)) \rightarrow A(u, r)))).$$

3. Гости не имеют доступа к секретным ресурсам:

$$(\forall u ((U(u) \wedge T(u, b)) \rightarrow (\neg A(u, c)))).$$

4. Каждый пользователь имеет хотя бы одну роль:

$$(\forall u (U(u) \rightarrow (\exists t T(u, t)))).$$

5. Никакой пользователь не является одновременно администратором и гостем:

$$(\forall u (U(u) \rightarrow (\neg(T(u, a) \wedge T(u, b)))))$$

4.8 Логическое программирование

5 ОСНОВЫ ОБЩЕЙ ТЕОРИИ ИНТЕЛЛЕКТУАЛЬНЫХ СИСТЕМ

5.1 Понятие интеллектуальной системы

Интеллектуальная система — это компьютерная система, способная решать интеллектуальные задачи.

Необходимость создания интеллектуальных систем обусловлена тем, что широкий класс реальных задач (диагностика, проектирование, управление в условиях неполноты данных) не допускает полной алгоритмизации и требует использования накопленных знаний предметной области.

Принято считать, что интеллектуальная система состоит из:

- базы знаний;
- решателя задач;
- интерфейса.

5.2 Понятие интеллектуальной задачи

Интеллектуальная задача — это задача, для решения которых требуется совместное использование различных видов знаний и различных моделей решения задач.

Отличительные признаки интеллектуальной задачи:

- слабая структурированность или полное отсутствие формального алгоритма решения;
- необходимость сочетать декларативные и процедурные знания;
- наличие неопределённости в условии или целевой ситуации;
- допустимость множества равнозначных решений.

Например, интеллектуальными задачами являются:

- постановка медицинского диагноза по неполному набору симптомов;
- разработка оптимального расписания с ограничениями;
- интерпретация естественно-языкового запроса;
- доказательство математической теоремы.

5.3 Понятие знания

Знание — это знаковая конструкция, которая обладает свойствами:

- *связности* (знание интегрируется с другими знаниями, то есть связано с другими знаниями и может расширяться);
- *сложной структуры* (знание состоит из элементов, структурировано и способно наращиваться, может быть включено в другие знания (метазнания));
- *интерпретируемости* (элементы знания — это знаки, которые имеют семантику, в том числе рефлексивную семантику относительно других элементов знания);
- *активности* (знание имеет операционную семантику и механизмы её реализации, реализующие переходы от текущего состояния знания к следующему состоянию знания);
- *семантической метрики* (наличие метрики измерения смысловой близости между знаниями и их знаками).

Примерами знания являются:

- правило нахождения площади треугольника;
- $2 + 2 = 4$.

5.3.1 Классификация знаний

Знания бывают декларативными и процедурными.

5.3.1.1 Понятие декларативного знания

Декларативное знание — это знание, которое отвечает на вопрос “что”.

Оно описывает факты, состояния, свойства и отношения объектов предметной области, не указывая, как с ними работать. Декларативные знания типично представляются в форме фактов, аксиом, ограничений, онтологий.

Примерами декларативных знаний являются:

- «Земля является планетой Солнечной системы»;
- «Для прямоугольного треугольника выполняется соотношение $a^2 + b^2 = c^2$ »;
- «Температура тела здорового человека составляет $36,6\text{ }^{\circ}\text{C}$ ».

5.3.1.2 Понятие процедурного знания

Процедурное знание, или императивное знание — это знание, которое отвечает на вопрос “как”.

Оно задаёт последовательность действий, необходимых для достижения некоторой цели. Процедурные знания типично представляются в форме алгоритмов, правил вывода, планов, программных процедур. Введение данного класса обусловлено необходимостью хранить не только факты, но и способы оперирования ими.

Примерами процедурных знаний являются:

- алгоритм вычисления НОД методом Евклида;
- правило продукции «ЕСЛИ пациент кашляет И температура $> 38,5$ ТО подозревать грипп»;
- процедура сортировки массива слиянием.

5.3.2 Понятие метазнания

Метазнание — это знание о знании.

Необходимость метазнаний обусловлена тем, что сложная база знаний должна не только хранить знания предметной области, но и «знать» о своей собственной структуре, полноте, достоверности и способах навигации по ней.

Метазнания являются одним из механизмов, обеспечивающих рефлекссию интеллектуальной системы.

Примерами метазнаний являются:

- знание о том, как организовано учебное пособие;
- системы классификаций знаний.

5.4 Понятие представления знаний

Чтобы знание можно было использовать в компьютерной системе, его необходимо представить: зафиксировать в форме, допускающей машинную обработку.

Представление знаний — это кодирование (запись) знаний на определённом языке.

Задача представления знаний возникла вместе с появлением первых систем искусственного интеллекта (1950-е–60-е гг.) как центральная инженерная проблема: накопленный человеческий опыт необходимо было «вложить» в память ЭВМ в виде, пригодном для автоматической обработки.

5.4.1 Модели представления знаний

Модель представления знаний — это форма (схема, шаблон), с помощью которой представляются знания.

Модель представления знаний выполняет двоякую роль: с одной стороны, задаёт синтаксис записи знаний, с другой — определяет механизм их обработки (интерпретации). Именно поэтому выбор модели влечёт выбор метода обработки знаний.

5.4.2 Классификация моделей представления знаний

К основным моделям представления знаний относятся следующие пять классов:

- семантические сети;
- фреймовые модели представления знаний;
- продукционные модели представления знаний;
- логические модели представления знаний;
- искусственные нейронные сети.

5.4.3 Понятие формализации знаний

Формализация знаний — это *представление знаний на формальном языке*.

Формализация необходима потому, что компьютерная система оперирует только символами и отношениями между ними: неформализованное знание не может быть обработано машиной. Формализация неизбежно связана с потерей части смысла, поэтому её качество определяется степенью сохранения семантической эквивалентности исходного и формального представлений.

5.4.3.1 Понятие понимания

Понимание — это *трансляция (перевод) знания в семантически эквивалентное смысловое представление этого знания*.

5.4.3.2 Понятие смысла знания

В то время, как *язык* — это форма представления изображения различных видов знаний, то есть “одежда”, в которую эти знания “одеваются”, *смысл* представляемого знания — это инвариант многообразия форм его представления (выражения), многообразия “одежд”, в которые это знания может быть “одето”.

Смысл (смысловое представление) знания — это абстрактное представление этого знания, которое является инвариантом всего многообразия семантически эквивалентных форм (вариантов) представления этого знания в различных языках.

5.4.3.3 Понятие смысла

Смысл — это абстрактная знаковая конструкция, принадлежащая внутреннему языку компьютерной системы, удовлетворяющая следующим требованиям:

- *универсальности* (возможность представления любой информации (декларативной и процедурной));
- *отсутствия синонимии знаков* (многократного вхождения знаков с одинаковыми денотатами);
- *отсутствия дублирования информации в виде семантически эквивалентных текстов* (не путать с логической эквивалентностью);
- *отсутствия омонимичных знаков* (в том числе местоимений);
- *отсутствия внутренней структуры у знаков* (атомарный характер знаков);
- *отсутствия склонений, спряжений* (как следствие отсутствия у знаков внутренней структуры);
- *отсутствия фрагментов знаковой конструкции, не являющихся знаками* (разделителей, ограничителей, и так далее);
- *выделения знаков связей*, элементами которых могут быть любые знаки, с которыми знаки связей связываются синтаксически задаваемыми отношениями инцидентности.

5.4.4 Понятие языка представления знаний

Язык представления знаний — это формальный язык, используемый для кодирования (записи) знаний.

5.4.4.1 Понятие внутреннего языка компьютерной системы

Внутренний язык компьютерной системы — это язык представления знаний в памяти компьютерной системы.

5.4.5 Понятие базы знаний интеллектуальной системы

База знаний интеллектуальной системы — это система знаний, хранимая в памяти этой интеллектуальной системы и достаточная для обеспечения

целенаправленного функционирования этой интеллектуальной системы как в своей внешней среде, так и в своей внутренней среде.

5.4.5.1 Понятие семантической окрестности сущности

Семантическая окрестность сущности — это метазнание, являющееся спецификацией (описанием) некоторой сущности, которая является ключевым элементом этого описания.

5.4.5.2 Понятие предметной области

Предметная область — это метазнание, которое является спецификацией (описанием) некоторых сущностей, связей между ними, и классов, которым принадлежат эти сущности и связи.

Предметная область — это максимальное фактографическое высказывание некоторой формальной теории.

Понятие *предметной области* является важнейшим методологическим приёмом, позволяющим выделить из всего многообразия исследуемого Мира только определенный класс исследуемых сущностей и только определенное семейство отношений, заданных на указанном классе. То есть осуществляется локализация, фокусирование внимания только на этом, абстрагируясь от всего остального исследуемого Мира.

Для указания предметной области используются следующие понятия:

- *исследуемое понятие предметной области;*
- *максимальный класс объектов исследования предметной области;*
- *немаксимальный класс объектов исследования предметной области;*
- *исследуемое отношение предметной области;*
- *класс исследуемых структур предметной области.*

5.4.5.3 Понятие исследуемого понятия предметной области

Исследуемое понятие предметной области — это понятие, которое является элементом заданной предметной области, а также свойства которого и связи которого с другими понятиями описываются в рамках заданной предметной области.

5.4.5.3.1 Понятие максимального класса объектов исследования предметной области

Максимальный класс объектов исследования предметной области — это исследуемое понятие заданной предметной области, для которого в рамках данной предметной области не существует другого исследуемого понятия, которое бы являлось надмножеством для заданного понятия.

5.4.5.3.2 Понятие немаксимального класса объектов исследования предметной области

Немаксимальный класс объектов исследования предметной области — это исследуемое понятие заданной предметной области, для которого в рамках данной предметной области существует исследуемое понятие, которое является надмножеством для заданного понятия.

5.4.5.3.3 Понятие исследуемого отношения предметной области

Исследуемое отношение предметной области — это относительное понятие, исследуемое в заданной предметной области, все связки которого являются элементами этой предметной области.

5.4.5.3.4 Понятие класса исследуемых структур предметной области

Класс исследуемых структур предметной области — это класс структур, каждая из которых принадлежит заданной предметной области.

5.4.5.3.5 Понятие онтологии предметной области

Каждой предметной области можно поставить в соответствие семейство соответствующих ей *онтологий* разного вида.

Онтология предметной области — это метазнание, которое является спецификацией (описанием) заданной предметной области, включающей описание свойств и связей между понятиями, входящих в состав заданной предметной области.

5.4.5.3.5.1 Классификация онтологий

Онтологию предметной области можно трактовать, с одной стороны, как семантическую окрестность соответствующей предметной области, с другой стороны, как объединение определенного вида семантических окрестностей всех понятий, используемых в рамках указанной предметной области, а также,

возможно, ключевых экземпляров указанных понятий, если таковые экземпляры имеются.

Онтологии предметной области бывают *структурными, терминологическими, теоретико-множественными, логическими* и другими.

5.4.5.3.5.1.1 Понятие терминологической онтологии предметной области

Терминологическая онтология предметной области — это онтология, описывающая основные и неосновные термины (имена, внешние обозначения), соответствующие понятиям заданной предметной области.

Другими словами, данная онтология содержит информацию о связках между понятиями заданной предметной области и их внешними идентификаторами.

Простыми словами, терминологическая онтология предметной области представляет собой словарь понятий и их терминов в этой предметной области.

5.4.5.3.5.1.2 Понятие теоретико-множественной онтологии предметной области

Теоретико-множественная онтология предметной области — это онтология, описывающая теоретико-множественные связки между понятиями заданной предметной области (связки отношений: включения, разбиения, объединения, пересечения, разность, область определения, домен).

Другими словами, данная онтология содержит информацию о связках между понятиями предметной области, заданных теоретико-множественными отношениями.

Простейшим теоретико-множественным отношением является *отношение включения*.

5.4.5.3.5.1.3 Понятие логической онтологии предметной области

Логическая онтология предметной области — это онтология, описывающая логические высказывания заданной предметной области.

5.4.5.3.5.1.4 Понятие структурной спецификации (онтологии) предметной области

Структурная спецификация (онтология) предметной области — это онтология, которая является описанием ролей понятий этой предметной области и её связей с другими предметными областями.

Для задания *структурной спецификации предметной области* используются следующие понятия:

- *максимальный класс объектов исследования предметной области;*
- *немаксимальный класс объектов исследования предметной области;*
- *исследуемое отношение предметной области;*
- *частная предметная область;*
- и другие.

5.4.5.3.5.1.5 Понятие интегрированной онтологии предметной области

Также выделяют *интегрированную онтологию предметной области*.

Интегрированная онтология предметной области — это онтология, объединяющая все онтологии различного вида заданной предметной области.

5.4.5.3.6 Понятие частной предметной области

Основным понятием используемым для задания иерархии предметных областей является понятие *частной предметной области*.

Частная предметная область заданной предметной области — это предметная область, максимальным классом объектов исследования которой является немаксимальный класс объект исследования заданной предметной области.

5.7 Понятие обработки знаний

Обработка знаний — это процесс преобразования знаний (выполнения операций над знаниями) в базе знаний интеллектуальной системы.

5.7.1 Понятие процесса

Процесс — это структура, описывающая последовательность ситуаций некоторой одной и той же сущности, среди которых можно выделить начальную ситуацию и конечную ситуацию.

5.7.2 Понятие ситуации

Ситуация — это структура, которая содержит по крайней мере один элемент, который является временной (темпоральной) сущностью.

5.7.2.1 Понятие начальной ситуации процесса

Начальная ситуация процесса — это ситуация, которая была актуальна для текущего момента времени до начала выполнения заданного процесса.

5.7.2.2 Понятие конечной ситуации процесса

Конечная ситуация процесса — это ситуация, которая станет актуальной для текущего момента времени после конца выполнения заданного процесса.

5.7.3 Понятие воздействия

Воздействие — это процесс, в рамках которого некоторая сущность является причиной изменения другой сущности.

5.7.3.1 Понятие субъекта воздействия

Субъект воздействия — это сущность, которая является причиной изменения другой сущности.

5.7.3.2 Понятие объекта воздействия

Объект воздействия — это сущность, которая является в рамках заданного воздействия исходным условием, необходимым для выполнения этого воздействия.

Примеры воздействий:

- камень падает на землю и оставляет вмятину в почве;
- излучение солнца воздействует на растения, запуская фотосинтез;
- ветер гнёт деревья.

5.7.4 Понятие действия

Действие — это воздействие, которое осуществляется некоторым субъектом целенаправленно в соответствии с некоторой целевой ситуацией (целью).

5.7.4.1 Понятие целевой ситуации действия

Целевая ситуация действия, или *цель действия* — это спецификация (описание) того, что требуется получить в результате выполнения заданного действия.

Отличие между действием и воздействием заключается в том, что действие является целенаправленным процессом, а воздействие — нет.

Примеры действий:

- человек пишет письмо, чтобы передать информацию;
- учитель объясняет тему, чтобы обучить учеников;
- зверь строит укрытие, чтобы защититься от холода.

5.7.4.2 Понятие атомарного действия и понятие неатомарного действия

Действия бывают *атомарными* и *неатомарными*.

5.7.4.2.1 Понятие атомарного действия

Атомарное действие — это действие, которое не является не атомарным.

5.7.4.2.2 Понятие неатомарного действия

Неатомарное действие — это действие, которое декомпозируется на более частные действия.

5.7.5 Понятие задачи

Задача — это спецификация некоторого действия, обладающая достаточной полнотой для выполнения этого действия.

Задачу можно описать в декларативном и процедурном стилях.

5.7.5.1 Понятие декларативной формулировки задачи

Задача с декларативной формулировкой — это задача, в формулировке которой явно описывается *целевая ситуация*, т.е. то, что является результатом выполнения (решения) данной задачи.

5.7.5.2 Понятие процедурной формулировки задачи

Задача с процедурной формулировкой — это задача, в которой указывается:

- субъекты, выполняющие это действие;
- объекты, над которыми выполняется действие (аргументы действия);
- инструменты, с помощью которых выполняется действие;
- момент и дополнительные условия начала и завершения выполнения действия;
- декомпозиция действия на более частные и т.д.

5.7.6 Понятие решения задачи

Решение задачи — это процесс, задающийся множеством путей от начальной ситуации данной задачи к каждой из её целевых ситуаций.

5.7.7 Понятие класса задач

Класс задач — это множество задач, для которого можно построить обобщенную формулировку задач, соответствующую всему этому множеству задач.

5.7.8 Понятие класса действий

Класс действий — это максимальное множество аналогичных действий, для которых существует (но не обязательно известных в текущий момент) по крайней мере один *метод*, обеспечивающий выполнение любого действия из указанного множества действий.

5.7.9 Понятие метода

Метод, или *программа* — это обобщенная спецификация решения задач некоторого класса.

5.7.9.1 Понятие класса методов

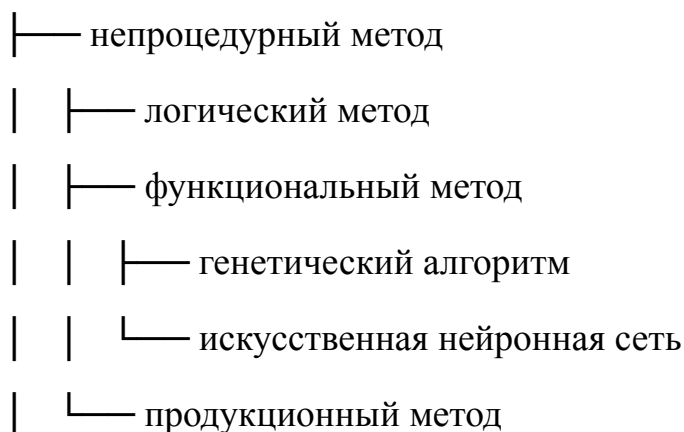
Класс методов — это множество всевозможных методов решения задач, имеющих общий язык *представления* этих методов.

Ниже представлена классификация классов методов:

класс методов

| — процедурный метод

| └— алгоритмический метод



5.7.10 Понятие языка представления методов

Язык представления методов, или язык программирования — это формальный язык, знаковыми конструкциями которого являются соответствующие методы, для которых существуют общие правила построения и общие правила соотнесения с теми сущностями и связями между ними, которые описываются этими методами.

Метод принадлежит языку представления методов, если он является:

- синтаксически корректным,
- синтаксически целостным,
- семантически корректным
- и семантически целостным методом заданного языка представления методов.

Спецификация каждого класса методов сводится к спецификации соответствующего языка представления методов, т.е. к описанию его синтаксиса, денотационной и операционной семантики.

5.7.10.1 Понятие денотационной семантики языка представления методов

Денотационная семантика языка представления методов — это онтология соответствующего класса методов этого языка представления методов, или обобщённая формулировка классов задач, решаемых при помощи заданного языка представления методов.

5.7.10.2 Понятие операционной семантики языка представления методов

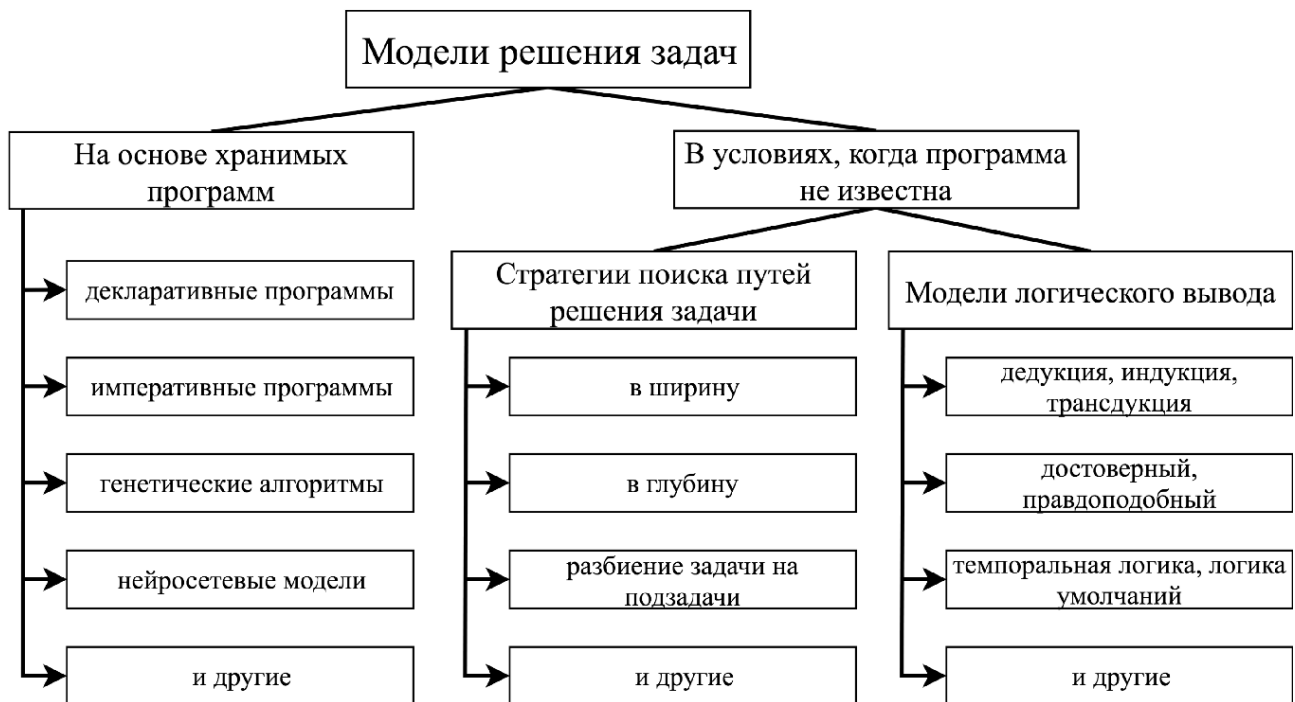
Операционная семантика языка представления методов — это метаметод

интерпретации соответствующего класса методов этого языка представления методов, или формальное описание интерпретатора заданного языка представления методов.

5.7.11 Понятие модели решения задач

Модель решения задач — это метаметод интерпретации соответствующего класса методов.

Многообразие моделей решения задач представлено на рисунке ниже:



5.7.12 Понятие навыка

Навык — это метод решения некоторой задачи и метаметод его интерпретации, то есть модель решения класса таких задач.

5.7.13 Понятие решателя задач интеллектуальной системы

Решатель задач интеллектуальной системы — это множество навыков заданной интеллектуальной системы.

5.7 Понятие передачи знаний

Передача знаний — это процессе обмена знаниями между двумя интеллектуальными системами.

5.7.1 Понятие языка обмена сообщениями компьютерной системы

Язык обмена сообщениями компьютерной системы — это формальный язык, используемый для кодирования (записи) знаний и их передачи между компьютерными системами.

5.7.2 Понятие внешнего языка компьютерной системы

Внешний язык компьютерной системы — это язык обмена сообщениями для компьютерной системы.

5.7.3 Понятие интерфейса интеллектуальной системы

Интерфейс интеллектуальной системы — это компонент интеллектуальной системы, обеспечивающий передачу знаний.

6 ОСНОВЫ ТЕХНОЛОГИИ OSTIS

Существуют различные технологии и инструменты для представления и обработки знаний в современных интеллектуальных системах.

Одним из примеров таких технологий является Технология OSTIS.

6.1 Понятие Технологии OSTIS

Технология OSTIS — это открытая технология проектирования семантически совместимых интеллектуальных систем — ostis-систем.

6.2 Понятие ostis-системы

ostis-система — это интеллектуальная система, разработанная по принципам *Технологии OSTIS*.

Языком представления знаний в ostis-системах является *SC-код*.

6.3 Понятие SC-кода

*SC-код (Semantic Computer Code)*³ — это универсальный формальный язык унифицированного смыслового представления знаний в памяти ostis-систем.

В основе SC-кода лежат следующие основные понятия:

- *sc-элемент* — это обозначение сущности; элементарный (атомарный) фрагмент информационной конструкции, принадлежащей SC-коду;
- *sc-конструкция* — это знаковая конструкция, принадлежащая SC-коду;
- *sc-множество* — это множество sc-элементов;
- *sc-структура* — это sc-множество, содержащее связки между элементами этого множества;
- *sc-текст* — это связная sc-структура, являющаяся семантически корректной в рамках SC-кода;
- *sc-знание* — это sc-текст, обладающий свойствами знания;

³ Одним из создателей первой версии SC-кода является Владимир Васильевич Голенков — белорусский и российский учёный в области искусственного интеллекта, доктор технических наук, профессор, профессор кафедры интеллектуальных информационных технологий Белорусского государственного университета информатики и радиоэлектроники (БГУИР).

а также:

- *файл* — информационная конструкция, которая не является sc-конструкцией и которая может храниться в файловой памяти ostis-системы;
- *sc-идентификатор* — файл, являющийся внешним идентификатором (в частности, именем) соответствующего sc-элемента, хранимого в памяти ostis-системы;
- *основной sc-идентификатор* — sc-идентификатор, который взаимно однозначно соответствует идентифицируемому sc-элементу.

SC-код является способом абстрактного внутреннего представления баз знаний в ostis-системах.

6.3.1 Понятие sc-элемента

sc-элемент — это атомарная (неделимая) единица, с помощью которой записывается информация на SC-коде.

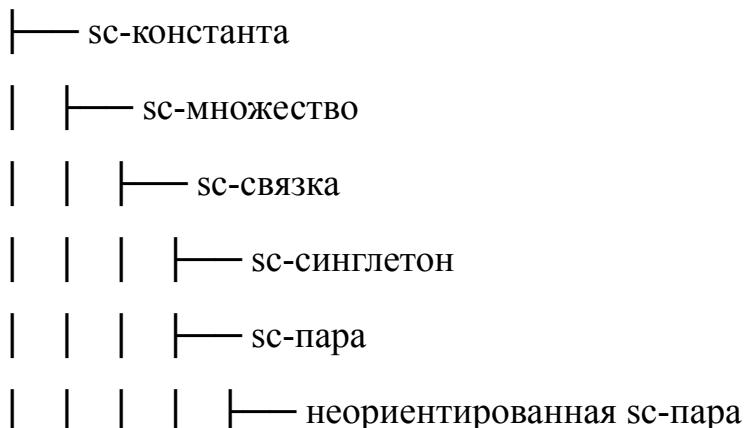
Принято считать, что *sc-элемент* — это sc-обозначение множества, представимого в SC-коде, то есть sc-множества. То есть каждый sc-элемент является обозначением соответствующего множества.

Существуют различные типы sc-элементов.

6.3.1.1 Структурная классификация sc-элементов (Алфавит SC-кода)

Структурная классификация sc-элементов определяет классификацию sc-элементов по структурному признаку. Такая классификация выглядит следующим образом:

sc-элемент



- | | | | └─ ориентированная sc-пара
- | | | | └─ sc-пара принадлежности
- | | | | | └─ sc-пара позитивной принадлежности
- | | | | | └─ sc-пара негативной принадлежности
- | | | | | └─ sc-пара нечёткой принадлежности
- | | | | └─ ориентированная sc-пара, не являющаяся sc-парой
- | | | | принадлежнoсти
- | | | └─ sc-связка, не являющаяся ни sc-синглетоном, ни sc-парой
- | | └─ sc-класс
- | | | └─ sc-класс sc-связок
- | | | | └─ sc-отношение
- | | | | └─ sc-класс sc-классов
- | | | | └─ sc-параметр
- | | | | └─ sc-класс sc-структур
- | | | | └─ sc-класс внешних сущностей
- | | | └─ sc-класс sc-констант разного типа
- | | └─ sc-структура
- | └─ внешняя сущность
- | └─ файл
- | └─ информационная конструкция, которая не является ни
- | | sc-множеством, ни файлом
- | └─ внешняя сущность, которая не является информационной
- | конструкцией
- └─ sc-переменная

Данная структурная классификация sc-элементов содержит только структурную классификацию sc-констант. Структурная классификация sc-переменных аналогична структурной классификации sc-констант.

6.3.1.1.1 Понятие sc-константы и понятие sc-переменной

Тип	Определение	Логико-семантическое значение	Примеры
sc-константа	sc-элемент, значением которого является он сам	Сам элемент	человек, красный_цвет, число_6
sc-переменная	sc-элемент, областью возможных значений которого является множество sc-констант	Множество sc-констант	?x, ?person, ?color

6.3.1.1.2 Понятие sc-множества

sc-множество — это множество sc-элементов.

sc-множества делятся на *sc-связки*, *sc-классы* и *sc-структуры*.

Тип sc-множества	Определение	Пример
sc-связка	sc-множество, которое связывает конечное число (1 и более) элементов	{Иванов, Петя}
sc-класс	sc-множество sc-элементов, имеющих явно указанные общие свойства.	класс_студентов = {Иванов, Петя}
sc-структура	sc-множество, содержащее sc-связки между элементами этого sc-множества.	{студенты, преподаватели, предметы, связи между ними}

6.3.1.1.2.1 Понятие sc-связки

sc-связка — это конечное sc-множество, состоящее из одного или более sc-элементов.

sc-связки делятся на sc-синглетоны, sc-пары и sc-связки, не являющиеся ни sc-синглетоном, ни sc-парами. sc-пары делятся на неориентированные sc-пары и ориентированные sc-пары.

Тип sc-связки	Определение	Пример
sc-синглетон	sc-связка с одним элементом	{Иванов}
sc-пара	sc-связка с двумя элементами	{Иванов, Петров}
неориентированная sc-пара	sc-пара без направления	{Иванов, Петров} $\in$ друзья
ориентированная sc-пара	sc-пара с направлением	<Иванов, МОИС> $\in$ изучает
sc-связка, не являющаяся ни sc-синглетоном, ни sc-парой	sc-связка более двух элементов	{Иванов, МОИС, 7,6} $\in$ успеваемость_студента

Ориентированные sc-пары делятся на sc-пары принадлежности и ориентированные sc-пары, не являющиеся sc-парой принадлежности.

6.3.1.1.2.2 Понятие sc-пары принадлежности

sc-пара принадлежности — это ориентированная sc-пара, связывающая sc-множество и sc-элемент, который принадлежит этому sc-множеству или не принадлежит.

Тип sc-пары принадлежности	Пояснение	Пример
sc-пара позитивной принадлежности	Элемент точно принадлежит множеству	Иванов $\in$ студенты
sc-пара негативной принадлежности	Элемент точно <u>НЕ</u> принадлежит множеству	Иванов $\notin$ преподаватели
sc-пара нечёткой принадлежности	Элемент принадлежит множеству с некоторой степенью	Иванов $\in \sim (0.7)$ отличники погода $\in \sim (0.6)$ ясная

6.3.1.1.2.3 Понятие sc-класса

sc-класс — это *sc-множество* *sc-элементов* (*sc-связок*), имеющих явно указанные общие свойства.

В отличие от *sc-связки* принципом формирования *sc-класса* является наличие общего свойства, присущего всем элементам этого *sc-класса* и только им, (или присущего всем сущностям, которые обозначаются указанными *sc-элементами*).

sc-классы делятся на *sc-классы sc-связок*, *sc-классы sc-классов*, *sc-классы sc-структур*, *sc-классы внешних сущностей* и *sc-классы sc-констант* разного структурного типа.

sc-классы sc-связок включают *sc-отношения*.

sc-отношения делятся на *бинарные неориентированные sc-отношения* и *бинарные ориентированные sc-отношения*.

Тип <i>sc-класса</i>	Принцип формирования	Пример
sc-классы sc-связок	Общие свойства связок (отношения)	отношение_"больше", отношение_"изучает"
sc-классы sc-классов	Метаклассы (параметры)	класс_математических_объектов
sc-классы sc-структур	Типы структур	граф, дерево, список (как классы)
sc-классы внешних сущностей	Типы внешних сущностей	файл_изображения, файл_текста (как классы)

6.3.1.1.2.4 Понятие sc-структуры

sc-структура — это *sc-множество*, содержащее *sc-связки* между элементами этого *sc-множества*.

Компонент структуры	Описание	Пример
Элементы	<i>sc-элементы</i> , входящие в структуру	узлы графа
Связки	Отношения между элементами	рёбра графа
Классы	Характеристики элементов и связок	веса рёбер, ориентированность

Примерами sc-структур являются:

- семантическая сеть университета — {студенты, преподаватели, предметы, связи между ними};
- граф социальных связей — {пользователи, отношения дружбы/подписки};
- структура документа — {разделы, параграфы, связи порядка и вложенности}.

6.3.1.1.2.5 Понятие внешней сущности

внешняя сущность — это sc-элемент, являющийся знаком внешней сущности.

Внешние сущности делятся на *файлы*, *информационные конструкции*, не являющиеся ни *sc-множествами*, ни *файлами*, и *внешние сущности*, не являющиеся *информационными конструкциями*.

Тип внешней сущности	Описание	Примеры
файл	внешняя информационная конструкция по отношению к SC-коду, записываемая на SC-коде	document.pdf, image.jpg, video.mp4
информационная конструкция, не являющаяся sc-множеством, ни файлом	информационная конструкция вне SC-кода	XML-документ, JSON-объект
внешняя сущность, не являющаяся информационной конструкцией	реальный объект или абстракция	конкретный человек, конкретный стол, конкретная идея

6.3.2 Синтаксис SC-кода

Синтаксис SC-кода, то есть синтаксическая структура линейных информационных конструкций (строк символов) на SC-коде, задаётся:

- *Алфавитом Ядра SC-кода* (или алфавитом используемых символов — элементарных, атомарных фрагментов информационных конструкций), то есть семейством таких попарно непересекающихся классов

синтаксически эквивалентных символов, для которых существует простая процедура, позволяющая для любого символа по его синтаксическим особенностям установить факт его принадлежности одному из указанных классов;

- 2мя бинарными ориентированными отношениями инцидентности sc-элементов, заданными на множестве всех sc-элементов:
 - *Отношением инцидентности обозначений sc-пар с их элементами;*
 - *Отношением инцидентности обозначений ориентированных sc-пар с их вторыми элементами, которое является подмножеством первого отношения инцидентности.*

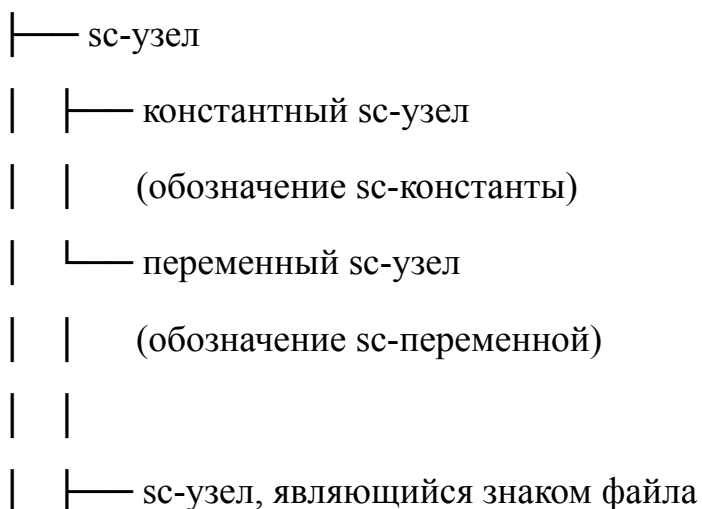
Алфавит Ядра SC-кода — это семейство классов синтаксически эквивалентных sc-элементов, в каждый из которых входят sc-элементы с одинаковыми синтаксическими характеристиками или, условно говоря, с одинаковыми наборами синтаксических меток, а именно классов:

- *sc-узлов, не являющихся знаками файлов;*
- *sc-узлов, являющихся знаками файлов;*
- *sc-рёбер;*
- *sc-дуг общего вида;*
- *sc-дуг основного вида (принадлежности).*

6.3.2.1 Синтаксическая классификация sc-элементов

Синтаксическая классификация sc-элементов, то есть классификация sc-элементов по синтаксическому признаку, представлена ниже.

sc-элемент



| | | — константный sc-узел, являющийся знаком файла

| | | (обозначение константного файла)

| | | — переменный sc-узел, являющийся знаком файла

| | | (обозначение переменного файла)

| | — sc-узел, не являющийся знаком файла

| |

| | — константный sc-узел

| | (обозначение sc-константы)

| | — переменный sc-узел

| — sc-коннектор

| (обозначение sc-пары)

| — sc-ребро (неориентированный sc-коннектор)

| | (обозначение неориентированной sc-пары)

| | — константное sc-ребро

| | (обозначение константной неориентированной sc-пары)

| | — переменное sc-ребро

| | (обозначение переменной неориентированной sc-пары)

| — sc-дуга (ориентированный sc-коннектор)

| (обозначение ориентированной sc-пары)

| — sc-дуга общего вида

| | (обозначение ориентированной sc-пары, не являющейся sc-парой

| | принадлежности)

| | — константная sc-дуга общего вида

| | — переменная sc-дуга общего вида

- └─ sc-дуга основного вида (sc-дуга принадлежности)
 - | (обозначение sc-пары принадлежности)
- └─ константная sc-дуга принадлежности
 - | (обозначение константной sc-пары принадлежности)
- └─ переменная sc-дуга принадлежности
 - | (обозначение переменной sc-пары принадлежности)
 - |
- └─ sc-дуга постоянной принадлежности
 - | (обозначение sc-пары постоянной принадлежности)
- └─ sc-дуга временной (темпоральной) принадлежности
 - | (обозначение sc-пары временной принадлежности)
- └─ └─ sc-дуга актуальной временной принадлежности
 - | | (обозначение sc-пары актуальной временной принадлежности)
- └─ └─ sc-дуга неактуальной временной принадлежности
 - | (обозначение sc-пары неактуальной временной принадлежности)
 - |
- └─ sc-дуга позитивной принадлежности
 - | (обозначение sc-пары позитивной принадлежности)
- └─ sc-дуга негативной принадлежности
 - | (обозначение sc-пары негативной принадлежности)
- └─ sc-дуга нечёткой принадлежности
 - (обозначение sc-пары нечёткой принадлежности)

6.3.3 Понятие sc-идентификатора

sc-идентификатор — это внешний идентификатор (имя) sc-элемента; строка символов или пиктограмма, взаимно однозначно представляющая соответствующий sc-элемент, хранимый в памяти ostis-систем.

sc-идентификаторы необходимы ostis-системе исключительно для того, чтобы осуществлять обмен информацией с другими системами и со своими пользователями (например, чтобы пояснять смысл sc-конструкций).

Для того чтобы представить свою базу знаний, решать самые различные задачи, связанные с анализом текущего состояния и эволюцией своей базы знаний, задачи, связанные с анализом текущего состояния (текущих ситуаций) окружающей среды, принятием соответствующих решений (целей) и организацией соответствующих действий, направленных на выполнение принятых решений (на достижение поставленных целей), ostis-системе не нужны никакие внешние идентификаторы, соответствующие sc-элементам.

Все sc-идентификаторы представляются в виде файлов на SC-коде.

Каждому sc-элементу в памяти ostis-системы можно дать один или несколько sc-идентификаторов. Но далеко не все sc-элементы должны иметь sc-идентификаторы.

sc-идентификаторы делятся на:

- *простые sc-идентификаторы*
- *и sc-выражения (сложные sc-идентификаторы).*

sc-идентификаторы также делятся на:

- *основные sc-идентификаторы*
- *и неосновные sc-идентификаторы.*

sc-идентификаторы также делятся на:

- *строковые sc-идентификаторы*
- *и нестроковые sc-идентификаторы.*

6.3.3.1 Понятие простого sc-идентификатора

простой sc-идентификатор — это sc-идентификатор sc-элемента, в состав которого sc-идентификаторы других sc-элементов не входят и который не содержит транслируемую в SC-код информацию об обозначаемой им сущности.

6.3.3.2 Понятие sc-выражения

sc-выражение — это sc-идентификатор, который не только обозначает соответствующую сущность, но также содержит информацию, представляющую собой по возможности однозначную спецификацию указанной сущности.

6.3.3.3 Понятие основного sc-идентификатора

основной sc-идентификатор — это sc-идентификатор, который является уникальным (не имеет синонимов и омонимов) в рамках соответствующего естественного языка.

Каждому sc-элементу может соответствовать целое семейство внешних sc-идентификаторов этого sc-элемента, которые обычно являются терминами, именующими обозначаемую сущность. Среди этих внешних sc-идентификаторов для каждого идентифицируемого sc-элемента выделяется один как *основной sc-идентификатор*. Неосновные термины (имена), синонимы, соответствующие этим sc-элементам (в том числе и классам), то есть *неосновные sc-идентификаторы*, поясняют семантику указанного sc-элемента.

6.3.3.4 Понятие строкового sc-идентификатора

строковый sc-идентификатор — это sc-идентификатор, представленный строкой символов, которая является именем обозначаемой сущности.

6.3.3.5 Понятие нестрокового sc-идентификатора

нестроковый sc-идентификатор — это sc-идентификатор, представленный в виде пиктограммы, условного обозначения или аудиофрагмента.

простой строковый sc-идентификатор — это простой sc-идентификатор, представляющий собой строку (цепочку) символов, которая является именем (названием) той же сущности, что и идентифицируемый sc-элемент.

6.3.3.6 Основные правила построения простых строковых sc-идентификаторов некоторых классов сущностей на SC-коде

Ниже представлены основные правила построения простых строковых sc-идентификаторов некоторых классов сущностей на SC-коде.

Тип sc-элемента	Правило	Правильные примеры	Неправильные примеры
-----------------	---------	--------------------	----------------------

sc-класс	Простой строковый sc-идентификатор, идентифицирующий <i>sc-узел</i> , обозначающий <i>sc-класс</i> , представляется в единственном числе.	человек, студент, университет	люди, студенты, университеты
sc-класс	Все символы простого строкового sc-идентификатора, идентифицирующего <i>sc-узел</i> , обозначающий <i>sc-класс</i> , являются строчными буквами.	человек, студент, университет,	Человек, Студент, Университет
sc-синглетон, sc-структура, sc-узел, обозначающий не sc-класс	Первым символом простого строкового sc-идентификатора, идентифицирующего <i>sc-узел</i> , обозначающий <i>sc-синглетон</i> , или <i>sc-узел</i> , обозначающий <i>sc-структуру</i> , или <i>sc-узел</i> , обозначающий не <i>sc-класс</i> , является заглавная буква.	Текущее время, Расписание БГУИР, Группа 421702, Студент Петя	текущее время, расписание БГУИР, группа 421702, студент Петя
неролевое sc-отношение	Последним символом простого строкового sc-идентификатора, идентифицирующего <i>sc-узел</i> , обозначающий <i>неролевое sc-отношение</i> , является надстрочная звездочка (*)	включение*, разбиение*,	включение, Включение, разбиение, Разбиение
ролевое sc-отношение	Последним символом простого строкового sc-идентификатора, идентифицирующего <i>sc-узел</i> , обозначающий <i>ролевое sc-отношение</i> , является штрих (прямой апостроф) (')	студент группы', завтра', слагаемое'	студент группы, Студент группы, завтра, Завтра, слагаемое, Слагаемое

sc-класс sc-классов (sc-параметр)	Последним символом простого строкового sc-идентификатора, идентифицирующего <i>sc-узел</i> , обозначающий <i>sc-класс</i> <i>sc-классов</i> , является циркумфлекс (^)	длина^, рост^, мощность^	длина, Длина, рост, Рост, мощность, Мощность
--	--	--------------------------------	---

6.3.3.7 Понятие системного sc-идентификатора

системный sc-идентификатор — это строковый sc-идентификатор, являющийся уникальным в рамках всей базы знаний Экосистемы OSTIS.

Данный sc-идентификатор, как правило, используется в исходных текстах базы знаний, при обмене сообщениями между ostis-системами, а также для взаимодействия ostis-системы с элементами, реализованными с использованием средств, внешних с точки зрения Технологии OSTIS, например, программ, написанных на традиционных языках программирования.

Алфавит системных sc-идентификаторов максимально упрощен для того, чтобы обеспечить удобство автоматической обработки таких sc-идентификаторов с использованием современных технических средств, в частности, запрещены пробелы и различные специальные символы.

Символами, использующимися в системном sc-идентификаторе, могут быть:

- буквы латинского алфавита (A-Z),
- цифры (0-9),
- знак нижнего подчеркивания (_).

Для обеспечения интернационализации рекомендуется формировать системные sc-идентификаторы на основании основных англоязычных sc-идентификаторов.

6.3.3.8 Основные правила построения системных sc-идентификаторов некоторых классов сущностей на SC-коде

Ниже представлены основные правила построения системных sc-идентификаторов некоторых классов сущностей на SC-коде.

Тип sc-элемента	Правило	Простой строковый	Системный sc-идентифика-
-----------------	---------	-------------------	--------------------------

		sc-идентификатор	тор
sc-класс	Системный sc-идентификатор, идентифицирующий <i>sc-узел</i> , обозначающий <i>sc-класс</i> , должен иметь приставку “concept_”.	человек, студент, университет	concept_human, concept_student, concept_university
неролевое sc-отношение	Системный sc-идентификатор, идентифицирующий <i>sc-узел</i> , обозначающий <i>неролевое sc-отношение</i> , должен иметь приставку “nrel_”.	включение*, разбиение*	nrel_inclusion, nrel_subdividing
ролевое sc-отношение	Системный sc-идентификатор, идентифицирующий <i>sc-узел</i> , обозначающий <i>ролевое sc-отношение</i> , должен иметь приставку “rrel_”.	студент группы’, завтра’, слагаемое’	rrel_group_student, rrel_tomorrow, rrel_addendum
sc-переменная	В начале системного sc-идентификатора, идентифицирующего <i>переменный sc-узел</i> указывается нижнее подчеркивание “_”.	какой-то студент	_some_student

6.3.4 Формы внешнего представления SC-кода: SCg-код и SCs-код

Для эксплуатации ostis-систем кроме способа абстрактного внутреннего представления баз знаний требуются способы внешнего изображения абстрактных текстов, удобных для пользователей и используемых при оформлении исходных текстов баз знаний указанных интеллектуальных компьютерных систем и исходных текстов фрагментов этих баз знаний, а также используемых для отображения пользователям различных фрагментов баз знаний по пользовательским запросам.

Существуют три формы внешнего представления текстов на SC-коде:

- *SCg-код (Semantic Code graphical)* — язык внешнего графического представления конструкций SC-кода;
- *SCs-код (Semantic Code string)* — язык внешнего линейного (строкового) представления конструкций SC-кода;
- *SCn-код (Semantic Code natural)* — язык внешнего гипертекстового представления конструкций SC-кода.

SCg-код и *SCs-код* являются средствами разработки исходников баз знаний *ostis-систем*.

6.3.4.1 Понятие SCg-кода

SCg-код представляет собой способ визуализации *sc*-текстов (информационных конструкций SC-кода) в виде рисунков этих абстрактных конструкций.

Основная цель SCg-кода — иметь четкие синтаксические графические признаки изображения *sc.g-элементов* (графических обозначений *sc*-элементов (обозначений *sc*-элементов на SCg-коде)), позволяющие легко выделить и различать такие классы *sc.g-элементов*, как:

- *sc.g-константы* (знаки константных сущностей) и *sc.g-переменные* (изображения переменных, значениями которых являются соответствующие *sc*-элементы);
- *sc.g-переменные*, значениями которых являются *sc*-константы, и *sc.g-переменные*, значениями которых являются *sc*-переменные;
- знаки постоянных сущностей и знаки временных сущностей;
- *sc.g-коннекторы* (знаки бинарных связей) и *sc.g-элементы*, не являющиеся *sc.g-коннекторами*;
- неориентированные *sc.g-коннекторы* (*sc.g-ребра*) и ориентированные (*sc.g-дуги*);
- *sc.g-дуги* принадлежности и *sc.g-дуги*, не являющиеся таковыми;
- *sc.g-дуги* позитивной принадлежности, негативной принадлежности и нечёткой принадлежности.

6.3.4.1.1 Понятие *sc.g*-узла

Ниже представлены обозначения некоторых (часто используемых) классов *sc*-узлов на SCg-коде, то есть *sc.g*-узлов.

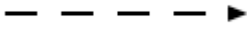
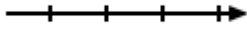
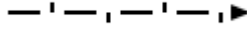
Класс sc-узлов	sc.g-узел, являющийся обозначением соответствующего класса sc-узлов
sc-узел	
константный sc-узел	
константный sc-узел, который является знаком sc-связки	
константный sc-узел, который является знаком sc-структуры	
константный sc-узел, который является знаком ролевого sc-отношения	
константный sc-узел, который является знаком неролевого sc-отношения	
константный sc-узел, который является знаком sc-класса	
константный sc-узел, который является знаком sc-класса sc-классов	
константный sc-узел, который является знаком файла	
переменный sc-узел	
переменный sc-узел, который является знаком sc-связки	
переменный sc-узел, который является знаком sc-структуры	
переменный sc-узел, который является знаком ролевого sc-отношения	

переменный sc-узел, который является знаком неролевого sc-отношения	
переменный sc-узел, который является знаком sc-класса	
переменный sc-узел, который является знаком sc-класса sc-классов	
переменный sc-узел, который является знаком файла	

6.3.4.1.2 Понятие sc.g-коннектора

Ниже представлены обозначения некоторых (часто используемых) классов sc-коннекторов на SCg-коде, то sc.g-коннекторов.

Класс sc-коннекторов	sc.g-коннектор, являющийся обозначением соответствующего класса sc-коннекторов
константное sc-ребро	
переменное sc-ребро	
константная sc-дуга общего вида	
переменная sc-дуга общего вида	
константная sc-дуга постоянной позитивной принадлежности	
константная sc-дуга временной позитивной принадлежности	
константная sc-дуга актуальной временной позитивной принадлежности	дуга с волнистой линией
константная sc-дуга неактуальной временной позитивной принадлежности	дуга с линией в виде цепочки

переменная sc-дуга постоянной позитивной принадлежности	
переменная sc-дуга временной позитивной принадлежности	
переменная sc-дуга актуальной временной позитивной принадлежности	дуга с прерывистой волнистой линией
переменная sc-дуга неактуальной временной позитивной принадлежности	дуга с прерывистой линией в виде цепочки
константная sc-дуга постоянной негативной принадлежности	
константная sc-дуга временной негативной принадлежности	
константная sc-дуга актуальной временной негативной принадлежности	дуга с волнистой линией с вертикальными чётками
константная sc-дуга неактуальной временной негативной принадлежности	дуга с линией в виде цепочки с вертикальными чётками
переменная sc-дуга постоянной негативной принадлежности	
переменная sc-дуга временной негативной принадлежности	
переменная sc-дуга актуальной временной негативной принадлежности	дуга с прерывистой волнистой линией с вертикальными чётками
переменная sc-дуга неактуальной временной негативной принадлежности	дуга с прерывистой линией в виде цепочки с вертикальными чётками
константная sc-дуга нечёткой принадлежности	
переменная sc-дуга нечёткой принадлежности	

6.3.4.2 Понятие SCs-кода

SCs-код представляет собой множество линейных текстов (*sc.s-текстов*), каждый из которых состоит из предложений (*sc.s-предложений*), разделенных друг от друга двойной точкой с запятой (*разделителем sc.s-предложений*). При

этом *sc.s-предложение* представляет собой последовательность *sc-идентификаторов*, являющихся именами описываемых сущностей и разделяемых между собой различными *sc.s-разделителями* и *sc.s-ограничителями*.

Таким образом *Алфавит SCs-кода* разбивается на:

- *Алфавит символов, используемых в sc-идентификаторах*, который разбивается на:
 - *Алфавит символов, используемых в простых строковых sc-идентификаторах*;
 - *Алфавит символов, используемых в sc-выражениях*;
- *Алфавит символов, используемых в sc.s-разделителях*;
- *Алфавит символов, используемых в sc.s-ограничителях*.

6.3.4.2.1 Понятие *sc-идентификатора* в SCs-коде

На SCs-коде все *sc-узлы* обозначаются при помощи соответствующих системных *sc-идентификаторов*.

При этом *системные sc-идентификаторы* могут быть *транслируемыми* и *нетранслируемыми*.

6.3.4.2.1.1 Понятие *нетранслируемого системного sc-идентификатора*

нетранслируемый системный sc-идентификатор — это такой *системный sc-идентификатор*, которые не транслируются в память *ostis-системы* после сборки базы знаний из исходников.

нетранслируемые системные sc-идентификаторы делятся на *локальные системные sc-идентификаторы* и *глобальные системные sc-идентификаторы*.

6.3.4.2.1.1.1 Понятие *локального системного sc-идентификатора*

локальный системный sc-идентификатор — это *нетранслируемый системный sc-идентификатор*, в начале которого указывается две точки “..”, и который склеивается с таким же идентификатором только в рамках одного и того же источника базы знаний.

Пример локального системного *sc-идентификатора*: `..set_1`.

6.3.4.2.1.2 Понятие глобального системного sc-идентификатора

глобальный системный sc-идентификатор — *нетранслируемый системный sc-идентификатор*, в начале которого указывается одна точка “.”, и который склеивается с таким же идентификатором в рамках всех исходников базы знаний.

Пример глобального системного sc-идентификатора: `.set_1`.

Также в SCs-коде существуют обозначения константного sc-узла и переменного sc-узла, для которых не задаётся *системный sc-идентификатор*:

- ... — обозначение (безымянного) константного sc-узла на SCs-коде, для которого не задаётся системный sc-идентификатор;
- _... — обозначение (безымянного) переменного sc-узла на SCs-коде, для которого не задаётся системный sc-идентификатор.

Обозначения константного sc-узла и переменного sc-узла, для которых не задаётся системный sc-идентификатор, не склеиваются с такими же обозначениями.

6.3.4.2.2 Понятие sc.s-разделителя

sc.s-разделитель — это символ на SCs-коде, который разделяет (выделяет, отделяет) один sc-идентификатор от другого.

Основными (часто используемыми) элементами *Алфавита символов, используемых в sc.s-разделителях* являются:

- sc.s-разделитель двойная точка с запятой — “..”;
- sc.s-разделитель простая точка с запятой — “.”;
- sc.s-коннектор;
- модификатор sc.s-коннектора:
 - sc.s-разделитель двойное двоеточие — “::”;
 - sc.s-разделитель простое двоеточие — “:”.

6.3.4.2.2.1 Понятие sc.s-коннектора

sc.s-коннектор — это обозначение sc-коннектора на SCs-коде.

Ниже представлены основные (часто используемые) sc.s-коннекторы, являющиеся обозначениями соответствующих sc-коннекторов.

sc-коннектор	sc.s-коннектор, являющийся обозначением соответствующего sc-коннектора
константное sc-ребро	$\langle = \rangle$
переменное sc-ребро	$_ \langle = \rangle$
константная sc-дуга общего вида	$\Rightarrow$ или $\langle =$
переменная sc-дуга общего вида	$_ \Rightarrow$ или $\langle = _$
константная sc-дуга постоянной позитивной принадлежности	$\rightarrow$ или $\langle -$
константная sc-дуга временной позитивной принадлежности	$\rightarrow$ или $\langle ..$
константная sc-дуга актуальной временной позитивной принадлежности	$\sim \rightarrow$ или $\langle \sim$
константная sc-дуга неактуальной временной позитивной принадлежности	$\% \rightarrow$ или $\langle \%$
переменная sc-дуга постоянной позитивной принадлежности	$_ \rightarrow$ или $\langle - _$
переменная sc-дуга временной позитивной принадлежности	$_ \rightarrow$ или $\langle .. _$
переменная sc-дуга актуальной временной позитивной принадлежности	$_ \sim \rightarrow$ или $\langle \sim _$
переменная sc-дуга неактуальной временной позитивной принадлежности	$_ \% \rightarrow$ или $\langle \% _$
константная sc-дуга постоянной негативной принадлежности	$\rightarrow$ или $\langle -$
константная sc-дуга временной негативной принадлежности	$\rightarrow$ или $\langle ..$
константная sc-дуга актуальной временной негативной принадлежности	$\sim \rightarrow$ или $\langle \sim$
константная sc-дуга неактуальной временной негативной принадлежности	$\% \rightarrow$ или $\langle \%$
переменная sc-дуга постоянной негативной принадлежности	$_ \rightarrow$ или $\langle - _$
переменная sc-дуга временной негативной принадлежности	$_ \rightarrow$ или $\langle .. _$

переменная sc-дуга актуальной временной негативной принадлежности	$_ \sim >$ или $< \sim _$
переменная sc-дуга неактуальной временной негативной принадлежности	$_ \% >$ или $< \% _$
константная sc-дуга нечёткой принадлежности	$/ >$ или $< /$
переменная sc-дуга нечёткой принадлежности	$_ / >$ или $< / _$

6.3.4.2.3 Понятие sc.s-ограничителя

sc.s-ограничитель — это специальный символ на SCs-коде, который ограничивает встроенное sc.s-предложение от основного sc.s-предложения, часть которого является это встроенное sc.s-предложение.

Основным элементом *Алфавита символов, используемых в sc.s-ограничителях*:

- sc.s-ограничители встроенных предложений — “(*)” и “(*)”;

6.3.4.2.4 Понятие sc.s-предложения

Информация на SCs-коде записывается в виде *sc.s-текста*. *sc.s-текст* представляет собой множество sc.s-предложений.

Каждое sc.s-предложение представляет собой последовательность (1) sc-идентификаторов, (2) sc.s-коннекторов, (3) точек с запятыми, (4) ограничителей присоединенных sc.s-предложений, завершаемых двойной точкой с запятой. При этом непосредственно соседствовать друг с другом не могут ни sc-идентификаторы, ни sc.s-коннекторы, ни, очевидно, точки с запятыми и ограничители присоединенных sc.s-предложений.

Между sc-идентификаторами в рамках sc.s-предложения может находиться либо точка с запятой, либо sc.s-коннектор. Слева и справа от sc.s-коннектора должны находиться sc-идентификаторы.

Признаком завершения любого sc.s-предложения, то есть последними его символами является двойная точка с запятой, которую, следовательно, можно считать разделителем sc.s-предложений.

sc.s-предложение является минимальным sc.s-текстом. Смысл sc.s-текста (а также sc.s-текста, включенного в структуру не зависит от порядка sc.s-предложений в этих sc-текстах. Т.е. перестановка sc.s-предложений в

рамках таких sc.s-текстов смысла этих sc.s-текстов не меняет (то есть приводит к семантически эквивалентным sc.s-текстам), но сильно влияет на трудоемкость человеческого восприятия (на "читабельность") этих текстов.

Ниже показаны примеры sc.s-предложений.

Пример	sc.s-предложение
Человек Иван (он же Иванов Иван Иванович) является студентом группы 421702 и имеет друзей Васю и Игоря.	<pre> Ivan <- concept_human; => nrel_fio: [Иванов Иван Иванович]; <- rrel_group_student: 421702; => nrel_friend: Vasya; => nrel_friend: Igor;; </pre>

Примеры записи sc-пар некоторых видов sc-отношений:

sc-отношение	Пример	sc.s-предложение
неролевое ориентированное бинарное sc-отношение "друг*"	Петя имеет друга Васю и Петя имеет друга Игоря.	<pre> Petya => nrel_friend: Vasya; => nrel_friend: Igor;; </pre> или <pre> Petya => nrel_friend: Vasya; Igor;; </pre>
неролевое ориентированное квазибинарное sc-отношение "друзья*"	Петя имеет друзей Васю и Игоря. То есть есть sc-множество — неориентированная sc-пара (sc-связка мощности 2), состоящая из sc-узла,	<pre> Petya => nrel_friends: { Vasya; Igor };; </pre>

	который обозначает Васю, и sc-узла, который обозначает Игоря, и sc-пара (sc-дуга общего вида (ориентированная sc-пара)), состоящая из предыдущей sc-пары и sc-узла, который обозначает Петю, принадлежит неролевому sc-отношению “друзья*”.	
неролевое ориентированное квазибинарное sc-отношение “декомпозиция*”	Книга состоит из трёх разделов.	<pre> some_book <- concept_book; => nrel_decomposition: < some_book_part_1; some_book_part_2; some_book_part_3 >; </pre>

sc.s-предложения делятся на *простые sc.s-предложения* и *сложные sc.s-предложения*.

6.3.4.2.4.1 Понятие простого sc.s-предложения

простое sc.s-предложение — это фрагмент sc.s-текста, (1) состоящий из двух системных sc-идентификаторов, соединенных между собой sc.s-коннектором, и (2) завершающийся двойной точкой с запятой.

Ниже показаны примеры простых sc.s-предложений.

Пример	простое sc.s-предложение
Человек Петя	<pre> concept_human -> Petya;; или Petya <- concept_human;; </pre>

Любое sc.s-предложение можно представить как множество простых sc.s-предложений.

6.3.4.2.4.2 Понятие сложного sc.s-предложения

сложное sc.s-предложение — это sc.s-предложение, в котором есть присоединённое sc.s-предложение.

6.5.4.2.4.3 Понятие присоединённого sc.s-предложения

присоединённое sc.s-предложение — это sc.s-предложение, которое описывает sc-элемент, каким-то образом связанный с sc-элементом, который описывается основным sc.s-предложением. Оно начинается с “(*)” и заканчивается “*)”, при этом все “тройки” в нём заканчиваются двойной точкой с запятой.

Ниже показаны примеры сложных sc.s-предложений.

Пример	сложное sc.s-предложение
Человек Петя является студентом группы 421702 и имеет друзей Васю и Игоря. Человек Вася студент группы 421701.	<pre>Petya <- concept_human; <- rrel_group_student: 421702; => nrel_friend: Vasya (*) <- concept_human;; <- rrel_group_student: 421701;; => nrel_friend: Petya;; *); => nrel_friend: Igor;;</pre>

Над sc.s-предложениями можно проводить операции:

- конверсии (операции разворота компонентов sc.s-предложения, после которой меняется направление sc.s-коннекторов);
- присоединения (операции, в результате которой второе sc.s-предложение становится встроенным в первое sc.s-предложение);
- слияния (операции присоединения простого sc.s-предложения другому sc.s-предложению, если эти предложения описывают один и тот же sc-элемент);
- разложения (операции, в результате которой sc.s-предложение разбивается на множество простых sc.s-предложений).

6.3.4.2.5 Системные sc-идентификаторы некоторых классов sc-элементов на SCs-коде

Для того, чтобы задать тип sc-узла на SCs-коде (например, что sc-узел является знаком sc-класса), необходимо его указать явно, то есть через sc.s-коннектор, который обозначает константную sc.дугу постоянной позитивной принадлежности.

Пример:

```
concept_human <- sc_node_class;;
```

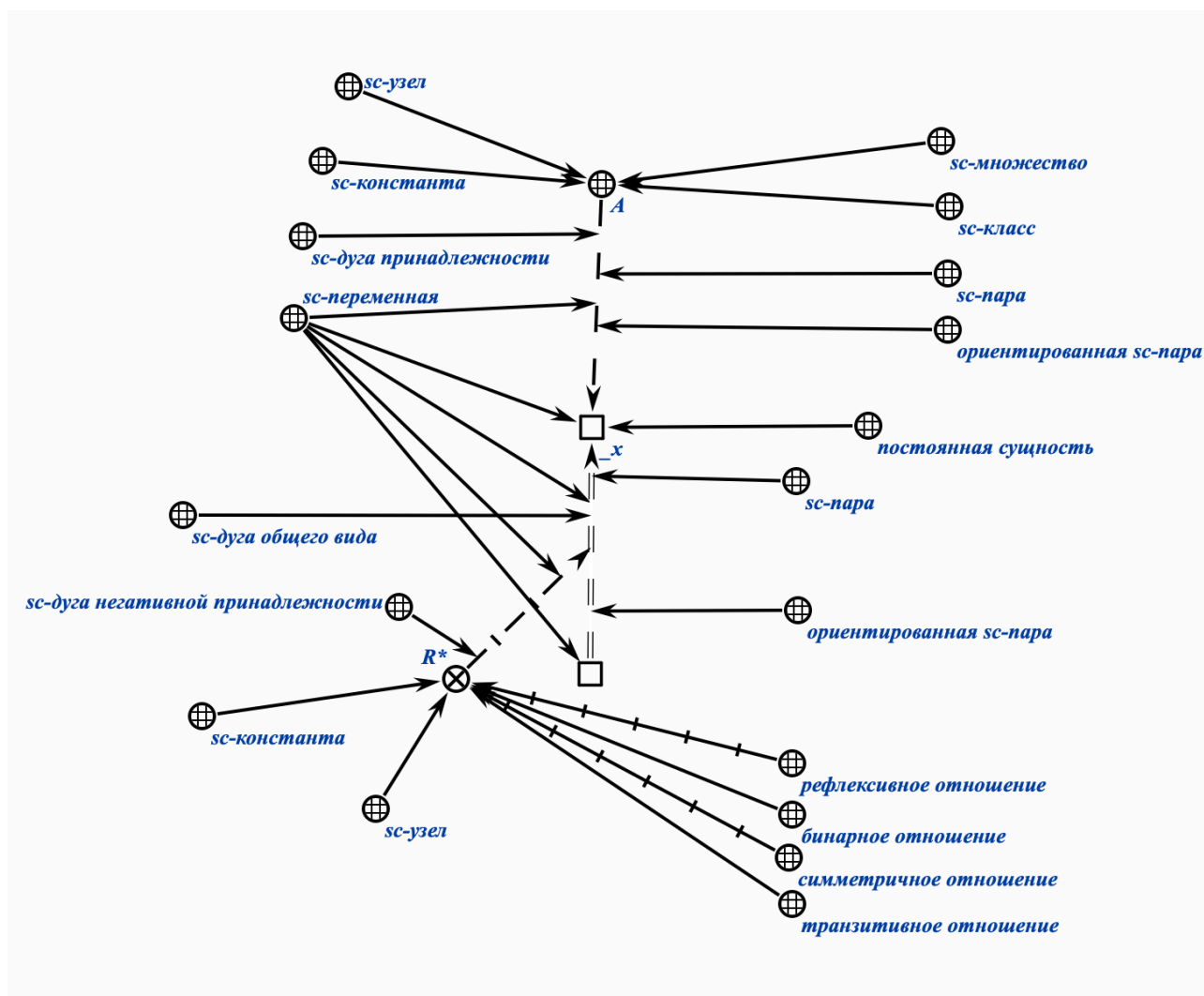
Ниже показаны sc-идентификаторы некоторых (часто используемых) классов sc-элементов на SCs-коде.

Класс sc-элементов	Системный sc-идентификатор
sc-узел	sc_node
sc-узел, который является знаком файла	sc_link
sc-узел, который является знаком sc-связки	sc_node_tuple
sc-узел, который является знаком sc-структуры	sc_node_structure
sc-узел, который является знаком ролевого sc-отношения	sc_node_role_relation
sc-узел, который является знаком неролевого sc-отношения	sc_node_non_role_relation
sc-узел, который является знаком sc-класса	sc_node_class
sc-узел, который является знаком sc-класса sc-классов	sc_node_superclass

Указание принадлежности sc-элементов данным sc-классам непосредственно влияет на визуальное отображение этих sc-элементов в процессе навигации по базе знаний.

6.3.4.3 Синтаксические и семантические характеристики sc-элементов

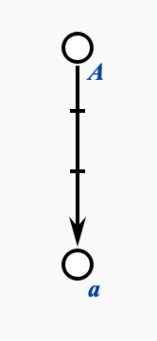
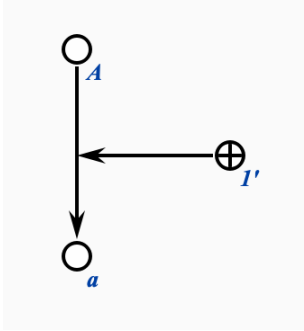
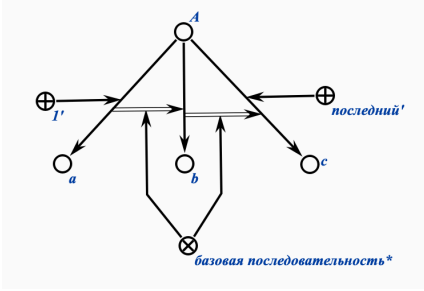
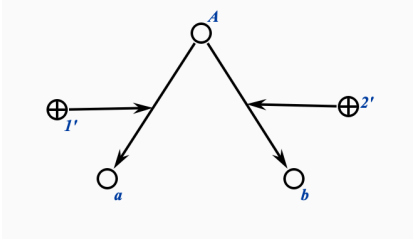
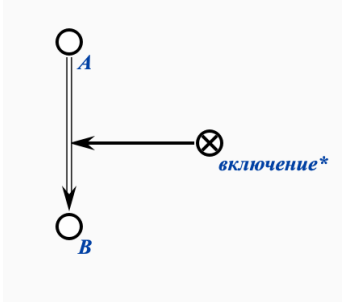
На рисунке ниже изображена sc-конструкция на SCg-коде, для sc-элементов которой показаны их синтаксические (слева от sc-конструкции) и семантические характеристики (справа от sc-конструкции).

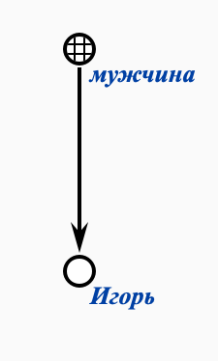
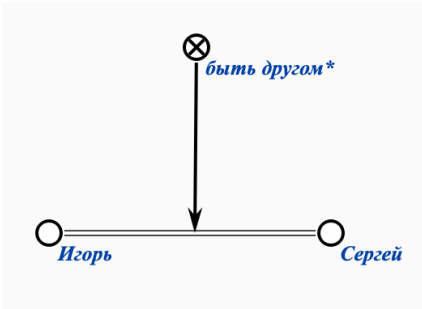
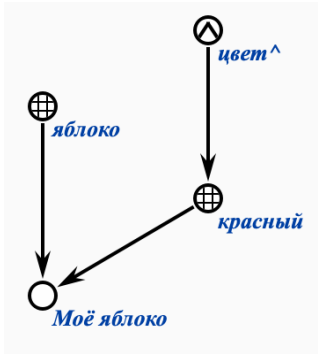


6.3.4.4 Примеры формализации знаний на SCg-коде и SCs-коде

Ниже представлены примеры формализации простейших знаний на Языке множеств, SCg-коде и SCs-коде соответственно.

Пример на Языке множеств	Пример на SCg-коде	Пример на SCs-коде
$a \in A$ или $A \ni a$		$A \rightarrow a ; ;$ или $a \leftarrow A ; ;$

$a \notin A$ или $A \nexists a$		$A \rightarrow a;$ или $a \leftarrow A;$
$A = \{ \langle a, 1 \rangle, \dots \}$ (множеству A принадлежит элемент a с ролью 1)		$A \rightarrow \text{rrel_1: } a;$ или $a \leftarrow \text{rrel_1: } A;$
$A = \langle a, b, c \rangle$ (связный список из трёх элементов)		$\langle a, b, c \rangle$ $\Rightarrow \text{nrel_system_identifier:}$ $[A];$
$A = \{ \langle a, 1 \rangle, \langle b, 2 \rangle \}$ (массив из двух элементов)		A $\rightarrow \text{rrel_1: } a;$ $\rightarrow \text{rrel_2: } b;$
$A \supset B$ или $B \subset A$		A $\Rightarrow \text{nrel_inclusion:}$ $B;$

мужчины = { Игорь, ...}		<pre> Igor => nrel_main_idtf: [Игорь]; <- concept_man;; concept_man => nrel_main_idtf: [мужчина]; <- sc_node_class;; </pre>
друзья = {<Игорь, Сергей>, ...}		<pre> Igor => nrel_main_idtf: [Игорь]; <=> nrel_friends: Sergey;; Sergey => nrel_main_idtf: [Сергей];; nrel_friends => nrel_main_idtf: [быть другом*]; <- sc_node_non_role_relation; ; </pre>
цвет = {красный, ...} красный = {Моё яблоко, ...} яблоко = {		<pre> apple => nrel_main_idtf: [Моё яблоко]; <- concept_apple; <- concept_red;; concept_apple => nrel_main_idtf: [яблоко];; concept_red => nrel_main_idtf: [красный]; <- concept_color;; concept_color => nrel_main_idtf: [цвет^];; </pre>

6.3.4.5 Правила перевода формализации знаний из SCg-кода на SCs-код

Правило	Трансляция из SCg-кода в SCs-код
При переводе sc-конструкций из SCg-кода на SCs-код все русскоязычные идентификаторы, указываемые рядом с sc.g-элементами (sc.g-узлами), то есть sc-элементами на SCg-коде, необходимо переводить в явные sc-конструкции на SCs-коде, в которых обозначаемые sc-элементы именуются соответствующими системными sc-идентификаторами, являющимися переводами на английский язык основных идентификаторов этих sc-элементов.	SCg-код:  SCs-код: <pre>Igor => nrel_main_idtf: [Игорь];;</pre>
При переводе sc-конструкций из SCg-кода на SCs-код все русскоязычные идентификаторы, указываемые рядом с sc.g-узлами, обозначающими sc-классы, необходимо переводить в явные sc-конструкции на SCs-коде, в которых обозначаемые sc-классы именуются соответствующими системными sc-идентификаторами, являющимися переводами на английский язык основных идентификаторов этих sc-классов. При этом для системных sc-идентификаторов sc-классов дополнительно указывается приставка “concept_”, а также указывается принадлежность этого sc-класса специальному sc-классу с системным sc-идентификатором “sc_node_class”.	SCg-код:  SCs-код: <pre>concept_man => nrel_main_idtf: [мужчина]; <- sc_node_class;;</pre>
При переводе sc-конструкций из SCg-кода на SCs-код все русскоязычные идентификаторы, указываемые рядом с sc.g-узлами, обозначающими ролевые	SCg-код: 

sc-отношения, необходимо переводить в явные sc-конструкции на SCs-коде, в которых обозначаемые ролевые sc-отношения именуются соответствующими системными sc-идентификаторами, являющимися переводами на английский язык основных идентификаторов этих ролевых sc-отношений. При этом для системных sc-идентификаторов ролевых sc-отношений дополнительно указывается приставка “nrel_”, а также указывается принадлежность этого ролевого sc-отношения специальному sc-классу с системным sc-идентификатором “sc_node_role_relation”.	SCs-код: <pre>rrel_1 => nrel_main_idtf: [1']; <- sc_node_role_relation;;</pre>
При переводе sc-конструкций из SCg-кода на SCs-код все русскоязычные идентификаторы, указываемые рядом с sc.g-узлами, обозначающими неролевые sc-отношения, необходимо переводить в явные sc-конструкции на SCs-коде, в которых обозначаемые неролевые sc-отношения именуются соответствующими системными sc-идентификаторами, являющимися переводами на английский язык основных идентификаторов этих неролевых sc-отношений. При этом для системных sc-идентификаторов неролевых sc-отношений дополнительно указывается приставка “nrel_”, а также указывается принадлежность этого неролевого sc-отношения специальному sc-классу с системным sc-идентификатором “sc_node_non_role_relation”.	SCg-код: SCs-код: <pre>nrel_friends => nrel_main_idtf: [быть другом*]; <- sc_node_non_role_relation;;</pre>

Также считается, что все русскоязычные идентификаторы, указываемые рядом с sc.g-элементами, по-умолчанию, являются основными русскоязычными sc-идентификаторами соответствующих sc-элементов.

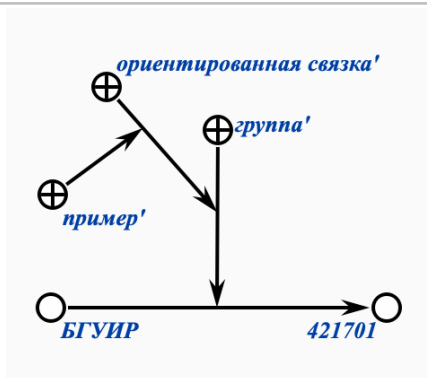
Кроме этого, также считается, что все англоязычные идентификаторы, указываемые рядом с sc.g-элементами и удовлетворяющие правилам построения системных sc-идентификаторов, по-умолчанию, являются системными sc-идентификаторами соответствующих sc-элементов. Иначе, если англоязычные идентификаторы, указываемые рядом с sc.g-элементами, не удовлетворяют правилам построения системных sc-идентификаторов, то они считаются основными англоязычными sc-идентификаторами соответствующих sc-элементов.

6.3.4.6 Примеры формализации различных видов знаний на SC-коде

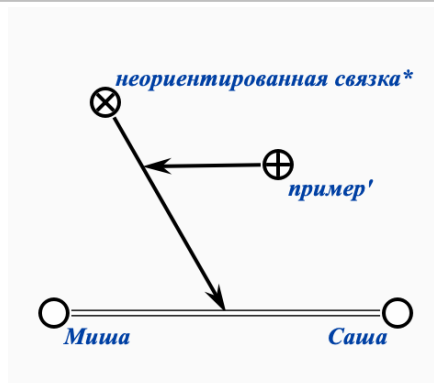
Ниже представлены примеры формализации различных видов знаний на SCg-коде.

Сущность	Пример сущности на SCg-коде
МНОЖЕСТВО	
СВЯЗКА	

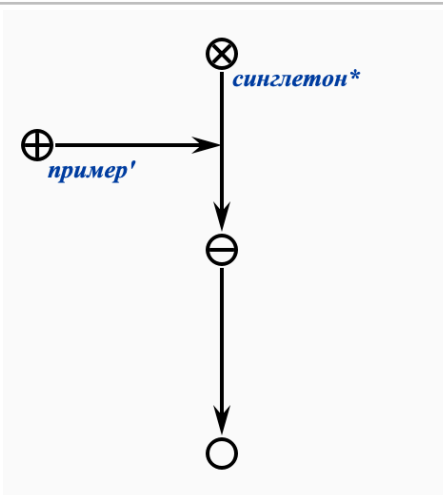
ориентированная
связка



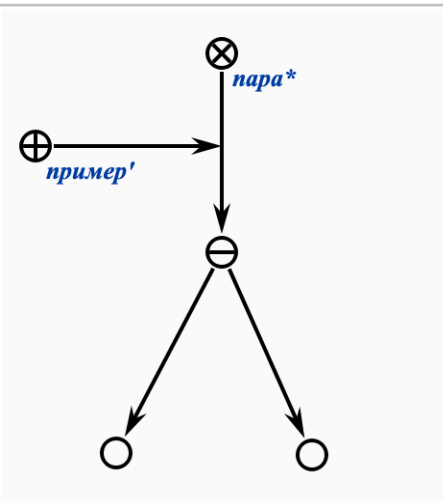
неориентированная
связка

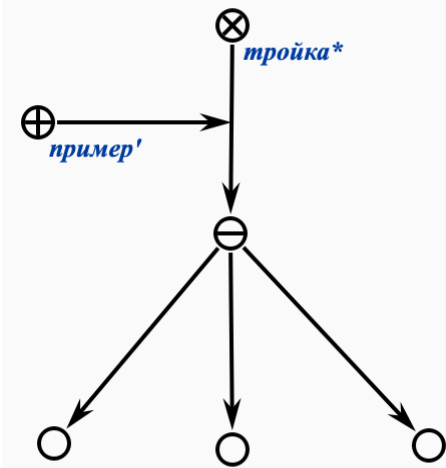
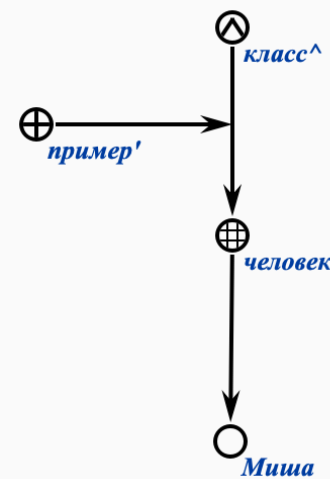
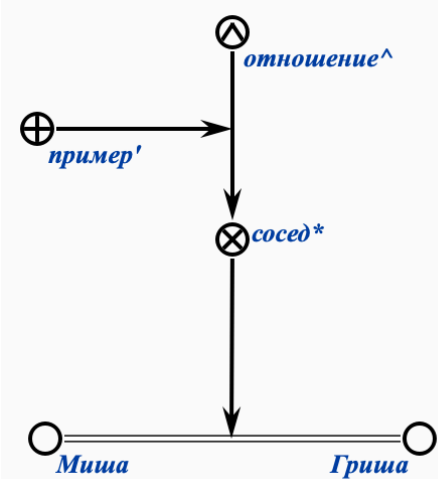


синглетон

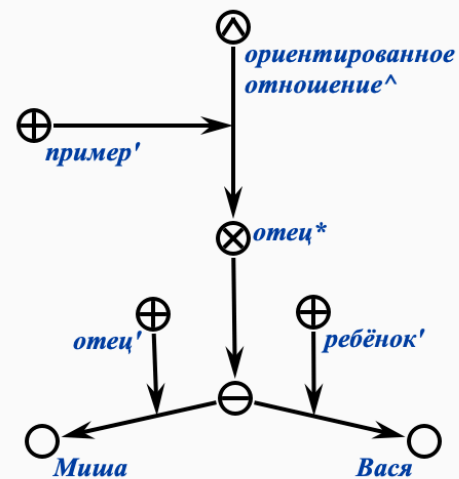
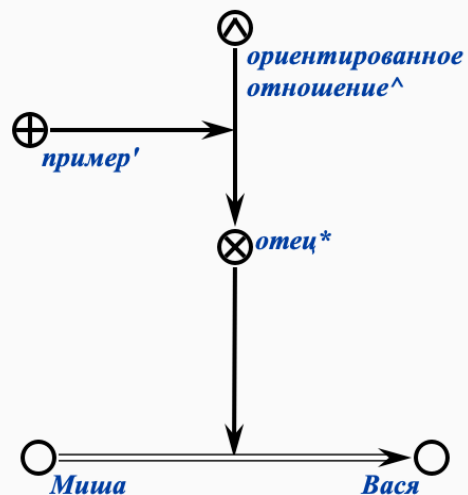


пара

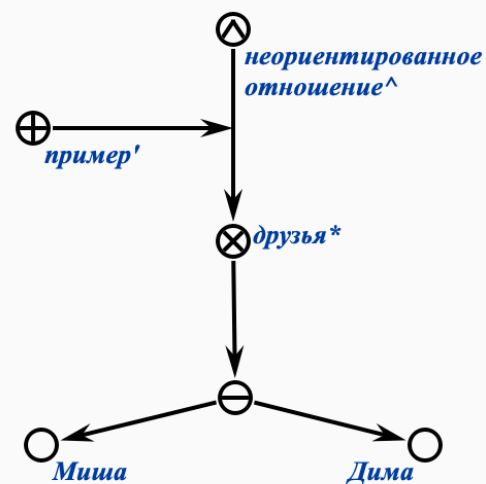


тройка	
класс	
отношение	

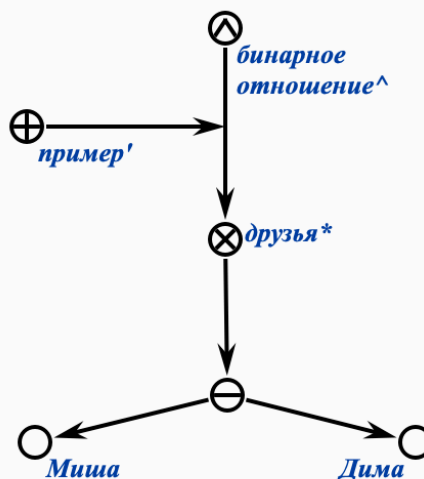
ориентированное
отношение



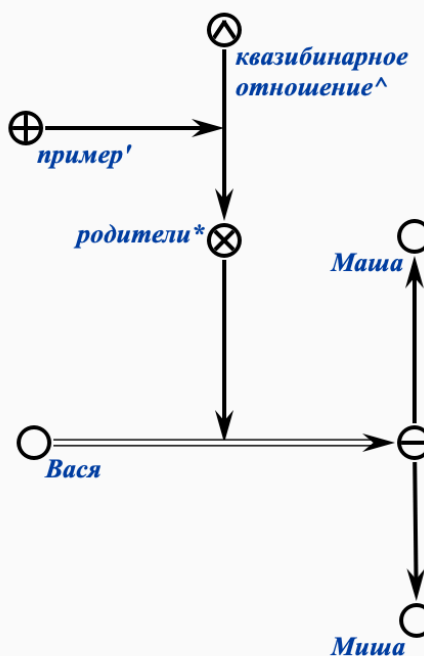
неориентированное
отношение



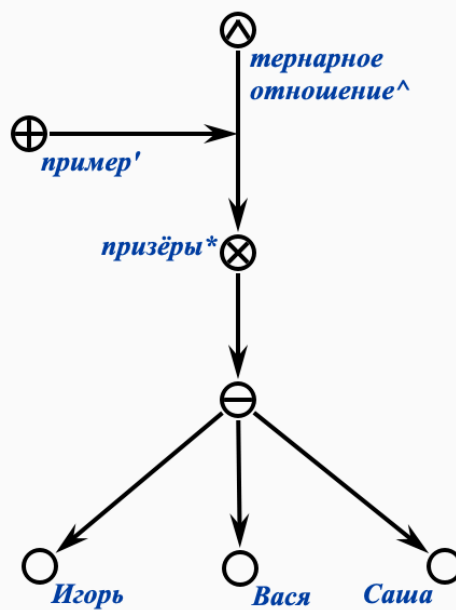
бинарное отношение



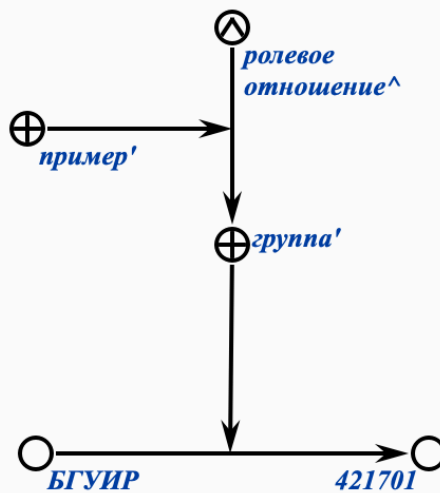
квазибинарное отношение



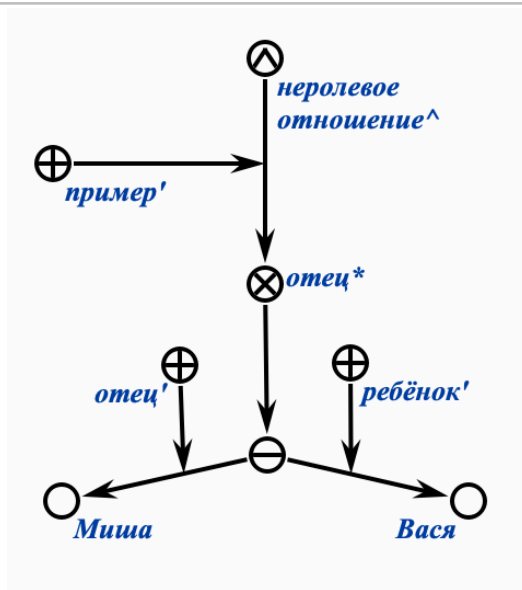
тернарное
отношение



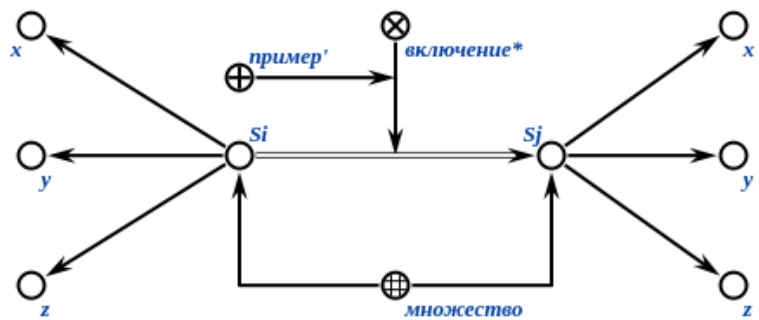
ролевое отношение



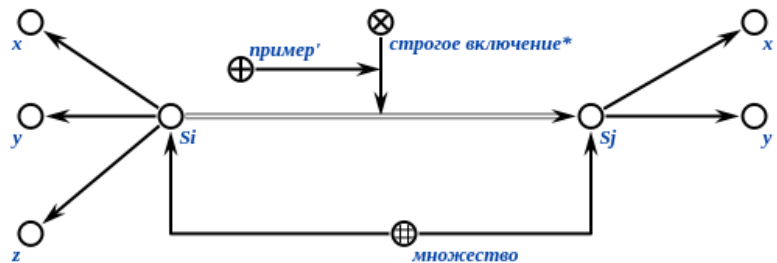
неролевое
отношение



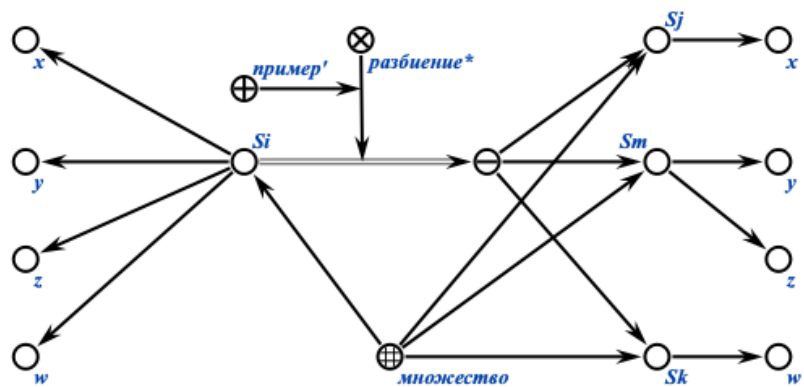
отношение
включения

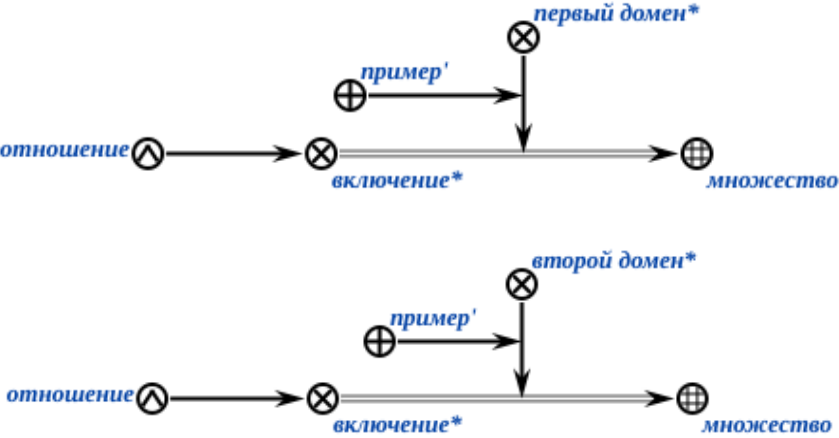
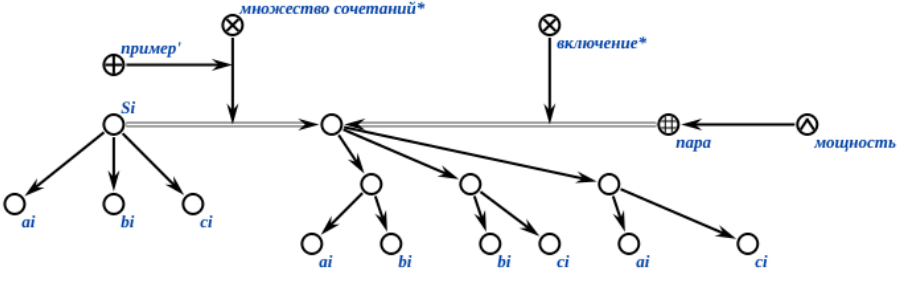
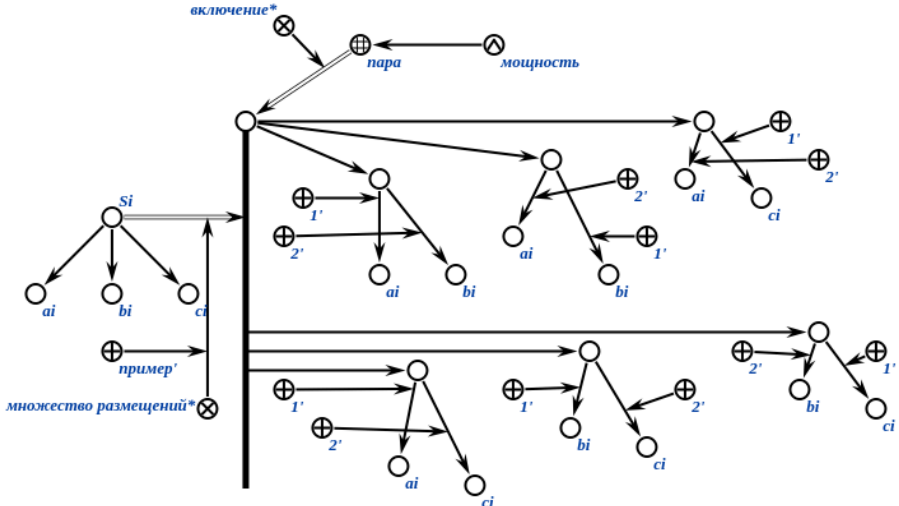


отношение строгого
включения

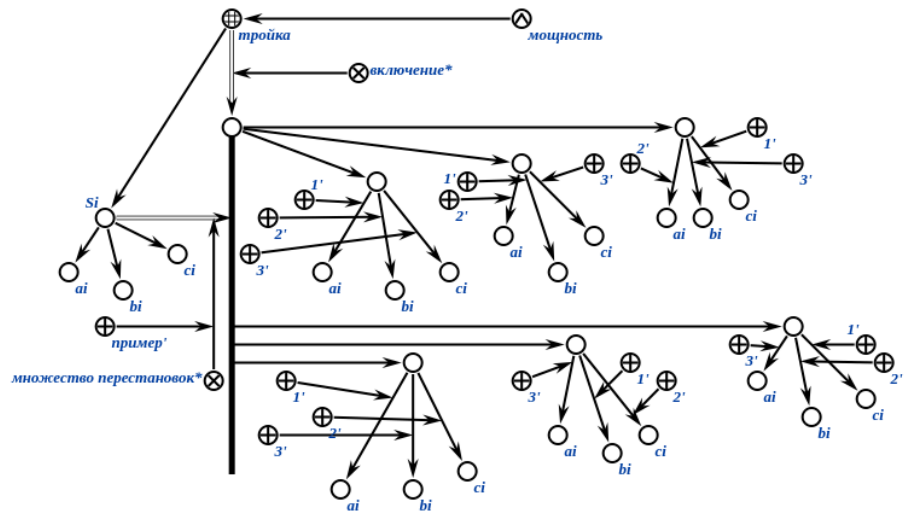


отношение
разбиения

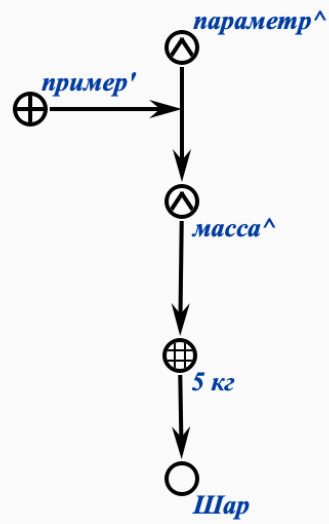


область определения отношения	
домен отношения	
множество сочетаний	
множество размещений	

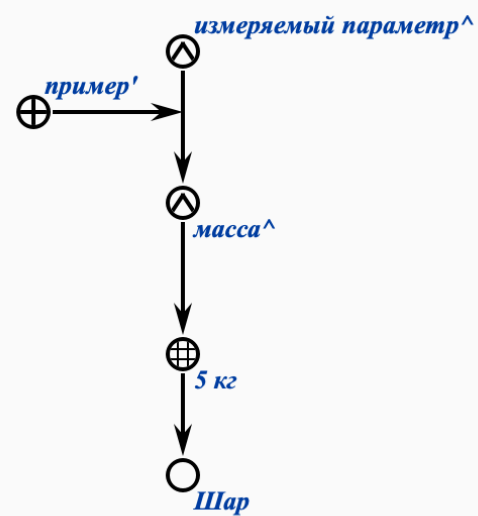
множество
перестановок



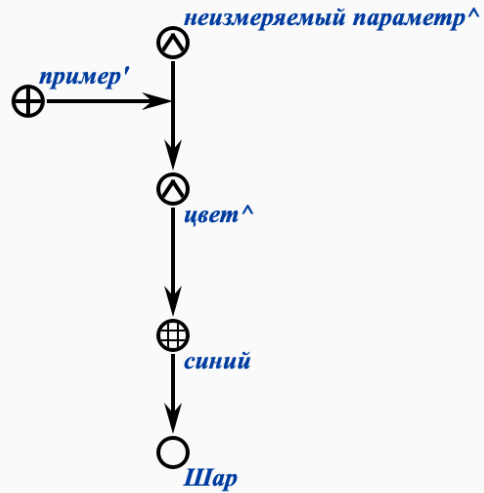
параметр



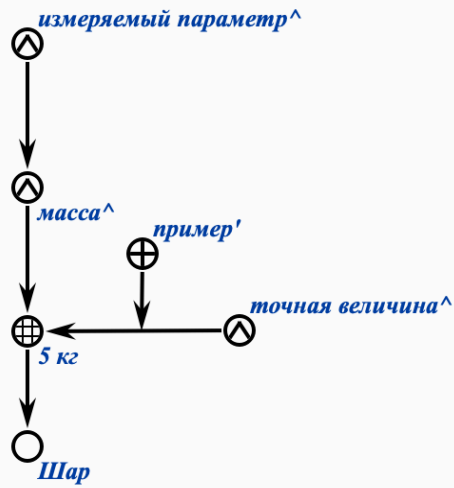
измеряемый
параметр



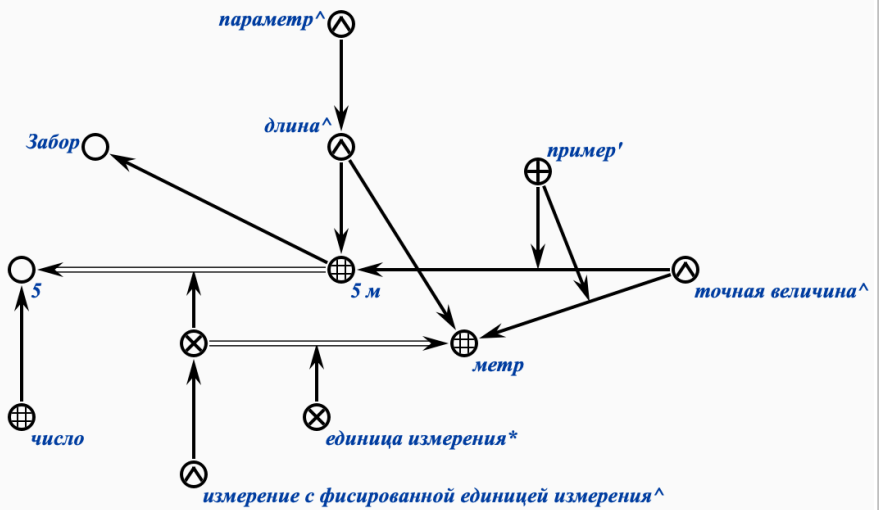
неизмеряемый
параметр



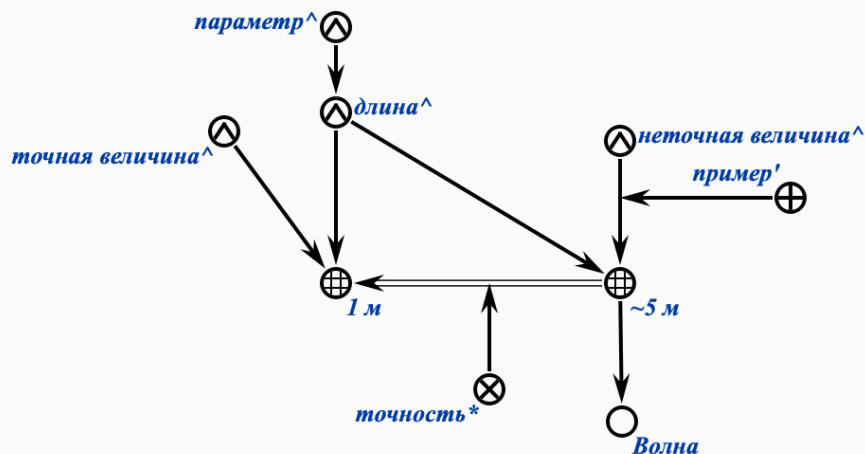
точная величина



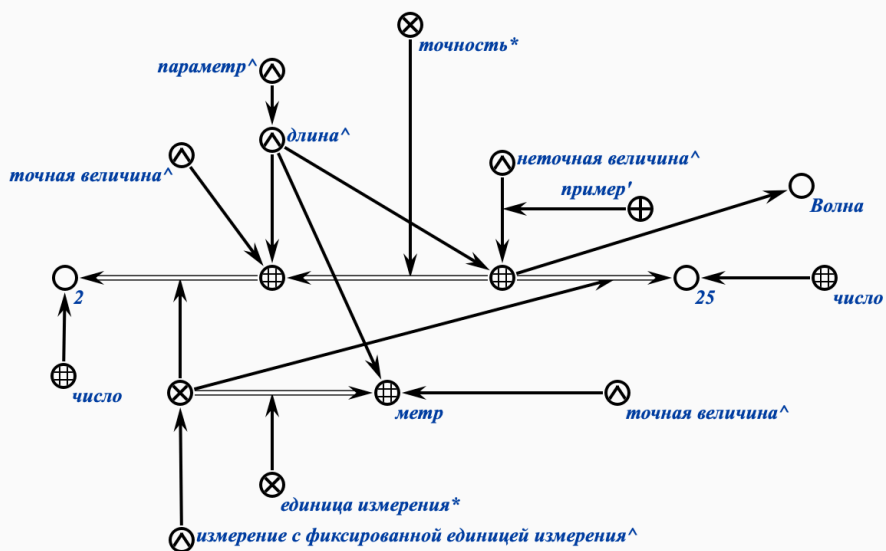
точная величина с
фиксированной
единицей измерения



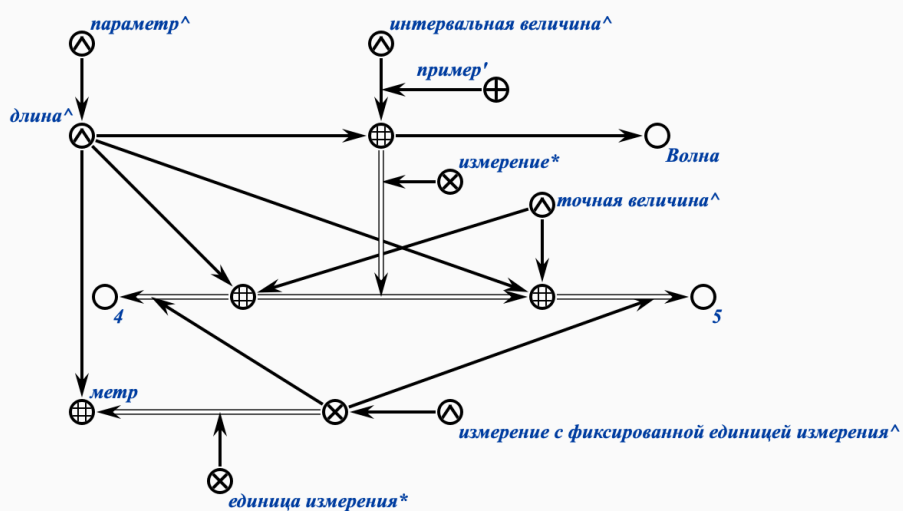
неточная величина с
заданной точностью

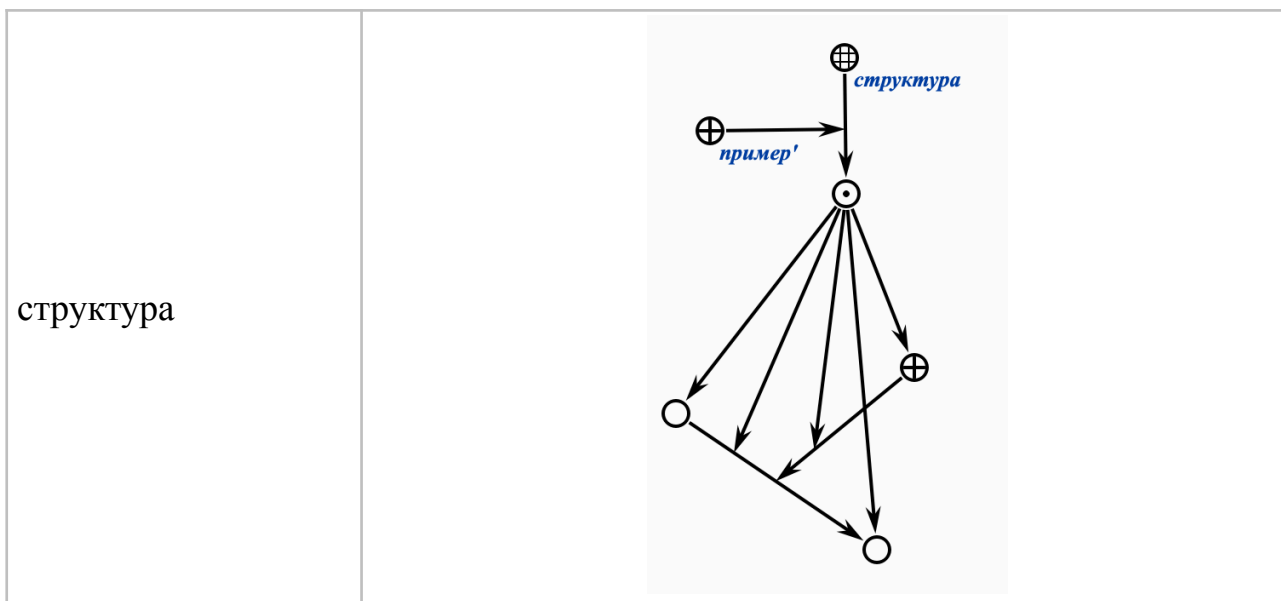


неточная величина с
фиксированной
единицей измерения
и точностью



интервальная
величина





6.3.4.7 Принципы структуризации знаний в ostis-системах

Для структуризации знаний в ostis-системах используются такие понятия как:

- sc-структура;
- семантическая sc-окрестность сущности;
- предметная sc-область;
- sc-онтология.

6.3.4.7.1 Понятие семантической sc-окрестности сущности

семантическая sc-окрестность сущности — это sc-метазнание, являющееся спецификацией (описанием) связей заданной сущности с другими сущностями.

В контексте Технологии OSTIS различают полную, базовую и специализированную семантические sc-окрестности, каждая из которых является некоторым видом описания конкретной сущности.

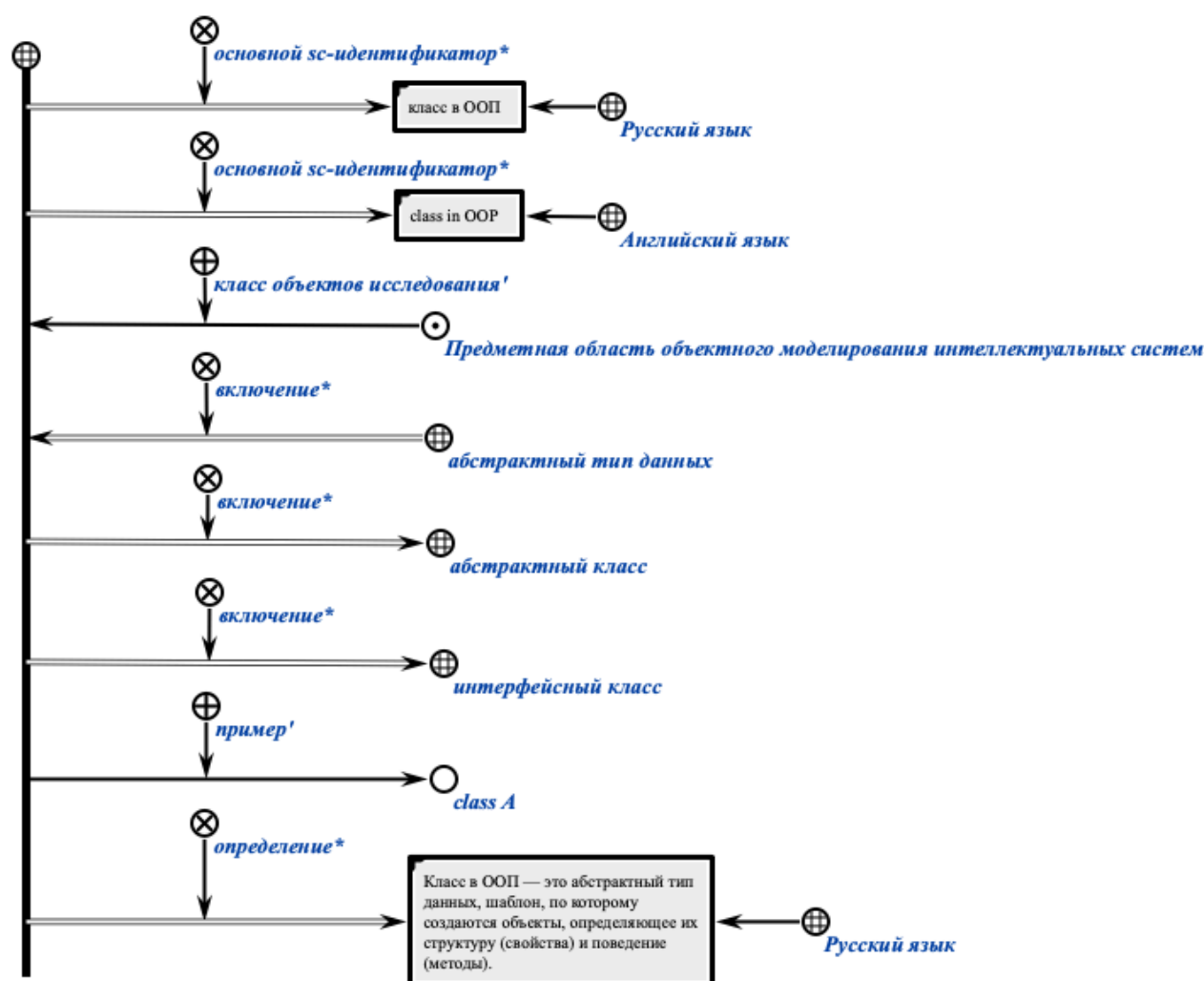
6.3.4.7.1.2 Структура базовой семантической sc-окрестности sc-класса

В базовую семантическую sc-окрестность sc-класса необходимо включать следующую информацию (при наличии):

- варианты идентификации на различных внешних языках (sc-идентификаторы) (основной sc-идентификатор*, sc-идентификатор* и т.п.);

- принадлежность некоторой предметной области с указанием роли, выполняемой в рамках этой предметной области;
- теоретико-множественные связи заданного понятия с другими сущностями (включение* и т.п.);
- пример экземпляра данного класса;
- определение или пояснение (определение*, пояснение*).

На рисунке ниже показан пример базовой семантической sc-окрестности абсолютного понятия “класс в ООП” на SCg-коде.



На рисунке ниже показан пример базовой семантической sc-окрестности sc-класса “класс в ООП” на SCs-коде.

```
concept_oop_class
=> nrel_main_idtf:
    [класс в ООП]
    (
        *
```

```

        <- lang_ru;;

    *);

=> nrel_main_idtf:

    [class in OOP]

    (*

        <- lang_en;;

    *);

<- rrel_explored_concept:

    subject_domain_of_object_modeling_intelligent_systems;

<= nrel_inclusion:

    concept_abstract_data_type;

=> nrel_inclusion:

    concept_abstract_class;

    concept_interface_class;

-> rrel_example:

    class_A;

=> nrel_definition:

```

[Класс в ООП — это абстрактный тип данных, шаблон, по которому создаются объекты, определяющий их структуру (свойства) и поведение (методы).]

```

    (*

        <- lang_ru;;

    *);;

```

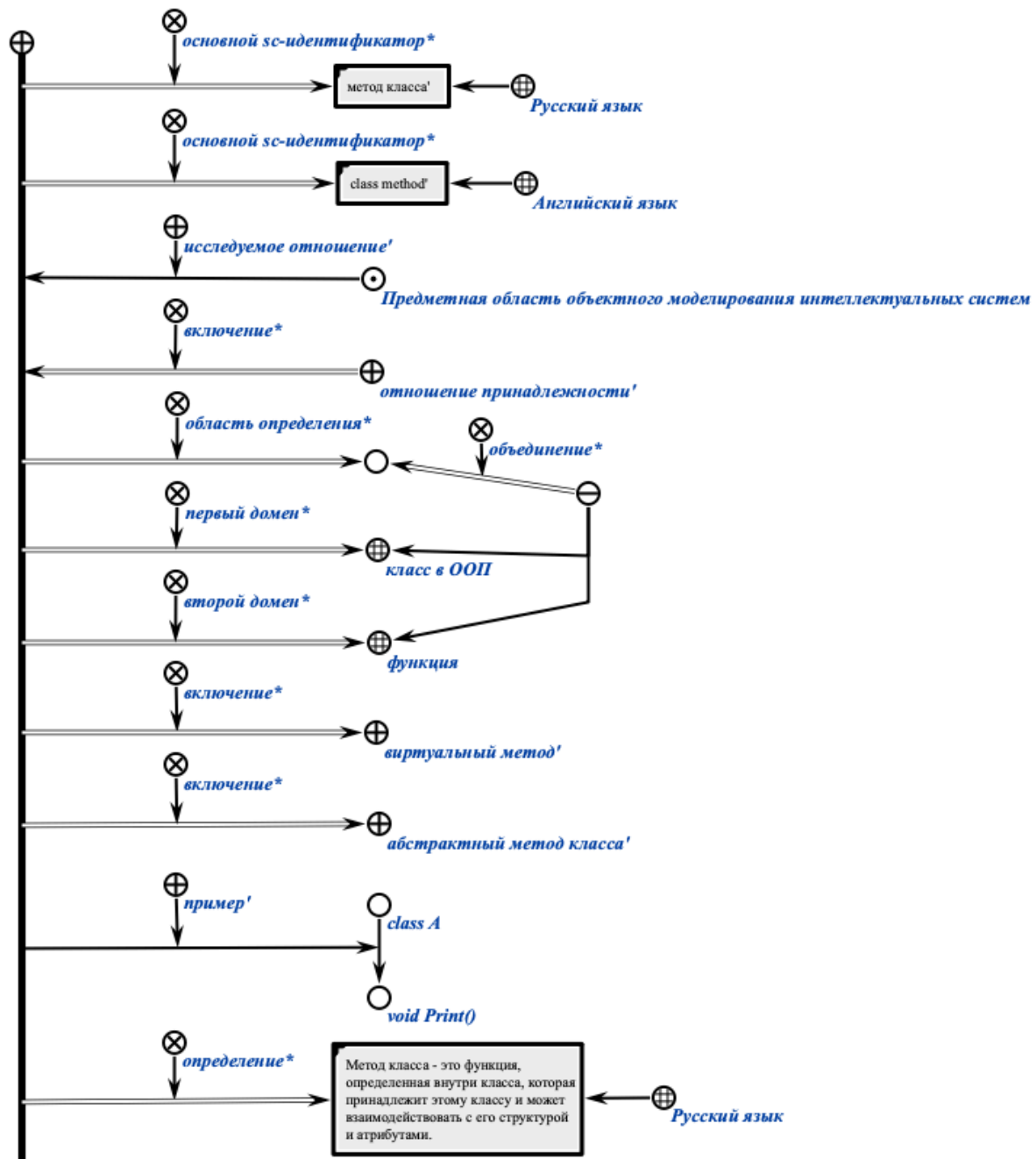
6.3.4.7.1.3 Структура базовой семантической sc-окрестности sc-отношения

В базовой семантической sc-окрестности sc-отношения дополнительно указываются:

- домены отношения (первый домен*, второй домен*);
- область определения отношения (область определения*);
- пример связки указанного отношения;

- классы отношения.

На рисунке ниже показан пример базовой семантической sc-окрестности ролевого sc-отношения “метод класса’ ” на SCg-коде.



На рисунке ниже показан пример базовой семантической sc-окрестности ролевого sc-отношения “ метод класса’ ” на SCs-коде.

```
rrel_class_method
=> nrel_main_idtf:
    [метод класса']
```



```

    (*
        <- lang_ru;;
    *);

=> nrel_main_idtf:
    [class method']
    (*
        <- lang_en;;
    *);

<- rrel_explored_relation:
    subject_domain_of_object_modeling_intelligent_systems;

<= nrel_inclusion:
    rrel_membership;

=> nrel_definitional_domain:
    ...
    (*
        <= nrel_set_union: {
            concept_oop_class;
            concept_function
        };
    *);

=> nrel_first_domain:
    concept_oop_class;

=> nrel_second_domain:
    concept_function;

=> nrel_inclusion:
    rrel_virtual_method;
    rrel_abstract_method;

-> rrel_example:
    (class_A -> function_Print);

```

```
=> nrel_definition:
```

[Метод класса – это функция, определенная внутри класса, которая принадлежит этому классу и может взаимодействовать с его структурой и атрибутами.]

```
(*  
    <- lang_ru;;  
*);;
```

6.3.4.7.4 Структура полной семантической sc-окрестности sc-класса

В полную семантическую sc-окрестность sc-класса необходимо включать при наличии следующую информацию:

- варианты идентификации на различных внешних языках (sc-идентификаторы);
- принадлежность некоторой предметной области с указанием роли, выполняемой в рамках этой предметной области;
- теоретико-множественные связи заданного понятия с другими сущностями (включение* и т.п.);
- определение или пояснение (определение*, пояснение*);
- высказывания, описывающие свойства указанного понятия;
- задачи и их классы, в которых данное понятие является ключевым;
- описание типичного примера использования указанного понятия;
- экземпляры описываемого понятия;
- авторы и библиографические источники указанного понятия;
- и другие.

6.3.4.7.5 Структура полной семантической sc-окрестности sc-отношения

В полной семантической sc-окрестности sc-отношения дополнительно указываются:

- домены (первый домен*, второй домен*);
- область определения (область определения*);
- описание типичного примера связки указанного отношения (спецификация типичного экземпляра);
- классы отношения.

6.3.4.7.2 Понятие предметной sc-области

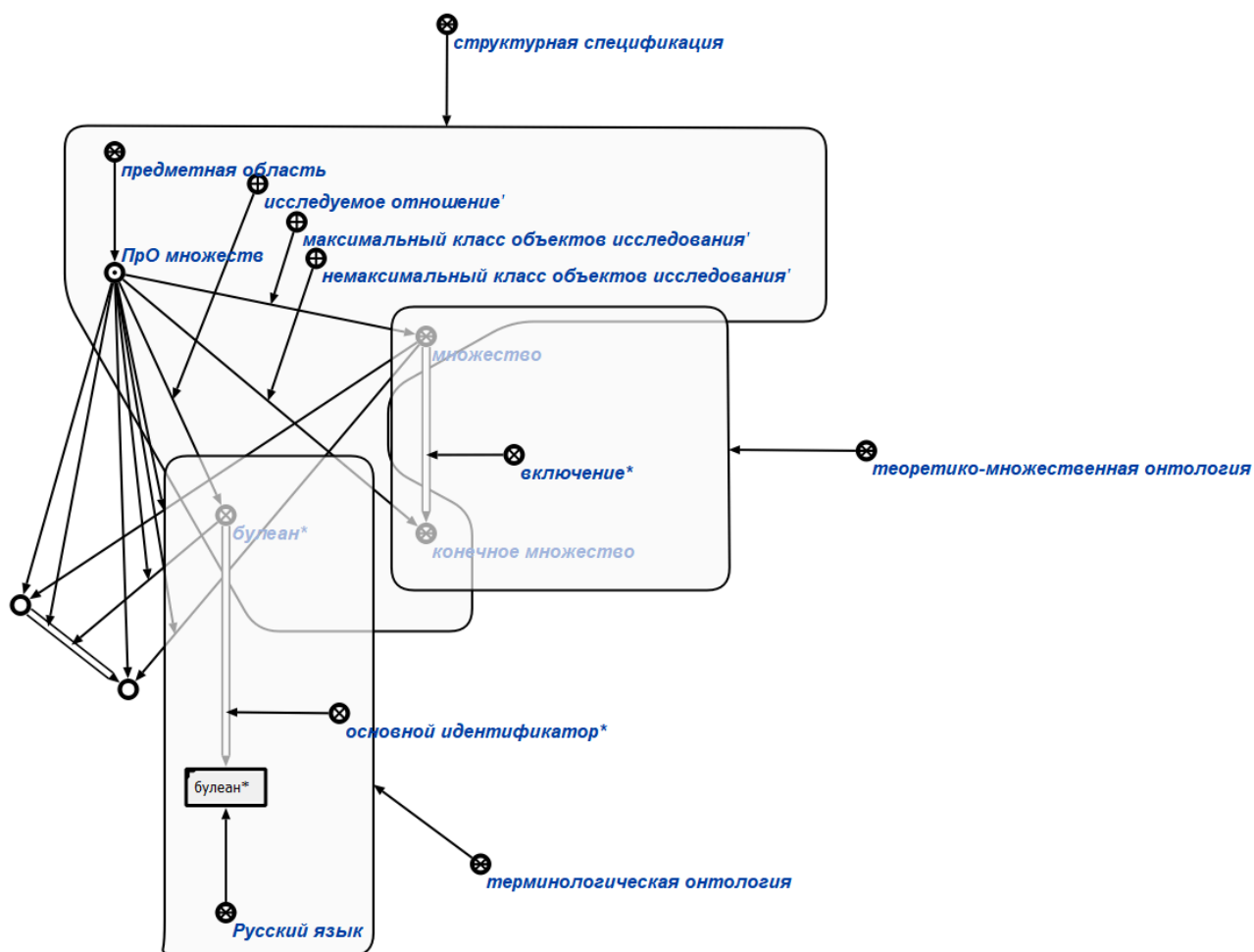
предметная sc-область — это sc-метазнание, которое является спецификацией (описанием) некоторых сущностей, связей между ними, и классов, которым принадлежат эти сущности и связи.

6.5.2.1 Понятия, используемые для описания предметных sc-областей

В *структурной спецификации предметной sc-области* используются следующие понятия:

- *максимальный класс объектов исследования*';
- *немаксимальный класс объектов исследования*';
- *исследуемое отношение*';
- *частная предметная область**.

Ниже представлен пример описания различных типов онтологий предметной области на SCg-коде.



6.5.3 Формализация иерархии предметных областей

Для любой предметной области можно найти место среди других предметных областей. Указание этого места — есть не что иное, как формализация теоретико-множественных связей с другими предметными областями. Результатом этого процесса является иерархия предметных областей.

Основным понятием используемым для задания иерархии предметных областей является отношение *частная предметная область**.

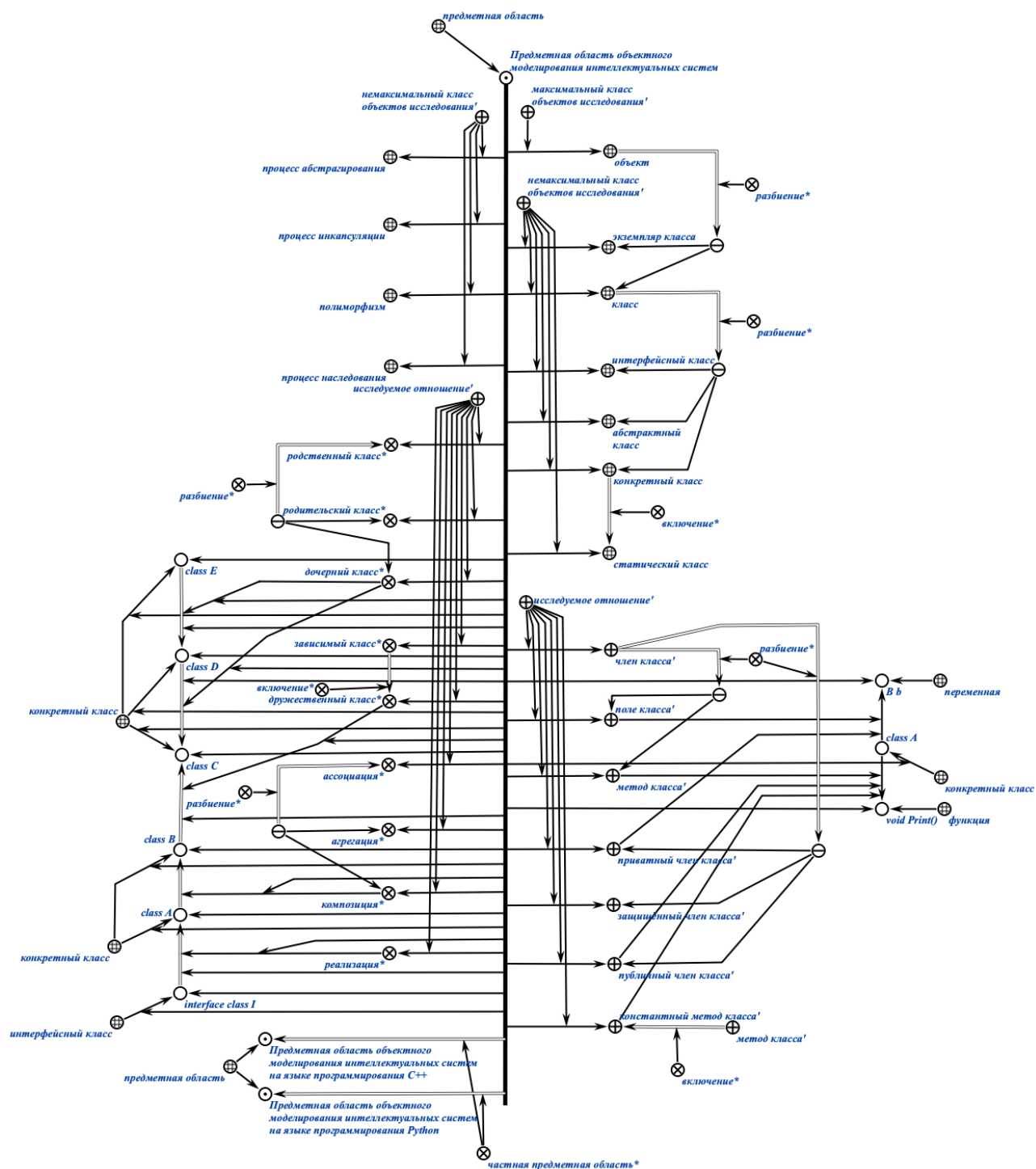
*частная предметная область** — это ориентированное бинарное отношение, с помощью которого задаётся иерархия предметных областей путём перехода от менее детального к более детальному рассмотрению соответствующих исследуемых понятий.

Ниже представлен пример иерархии предметных областей и соответствующих им онтологий на SCg-коде.

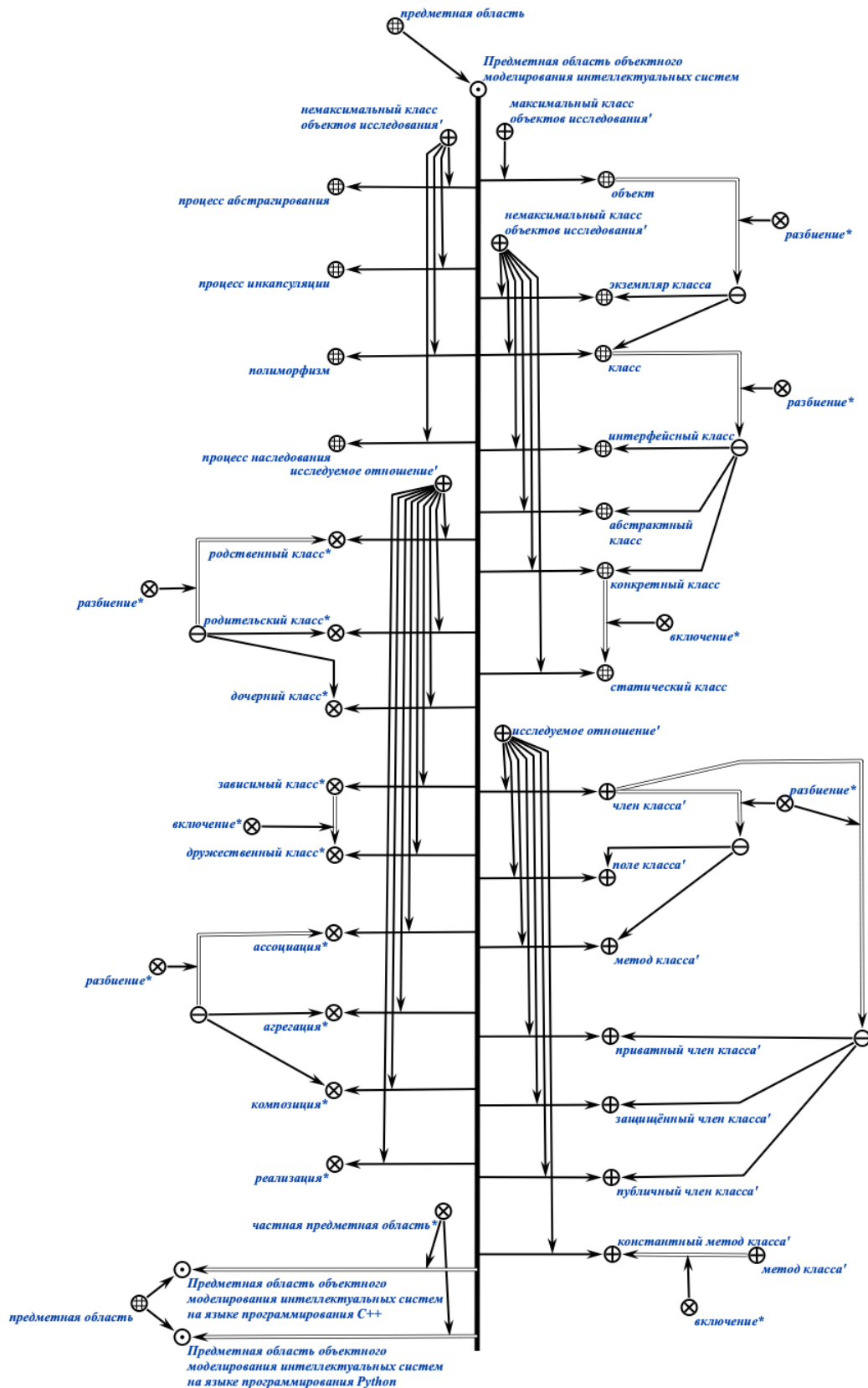


6.5.5 Пример формализации предметной области, её структурной спецификации и теоретико-множественной онтологии

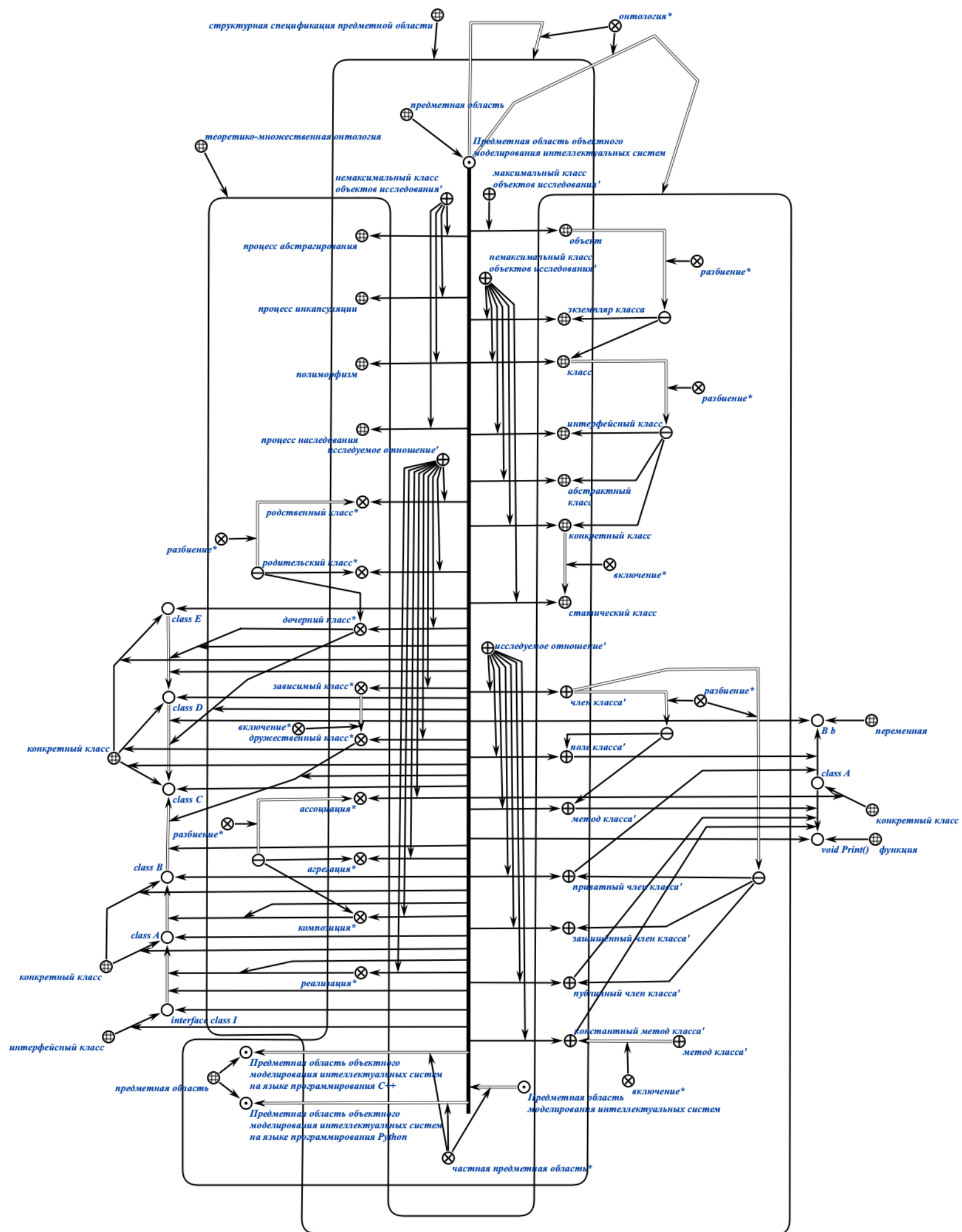
На рисунке ниже представлен пример полной формализации предметной области объектного моделирования интеллектуальных систем на SCg-коде.



На рисунке ниже представлен пример формализации предметной области объектного моделирования без указания экземпляров понятий на SCg-коде.



На рисунке ниже представлен пример формализации предметной области объектного моделирования интеллектуальных систем, её структурной спецификации и теоретико-множественной онтологии на SCg-коде.



Стоит отметить, что sc.g-коннектор принадлежит sc.g-контуре, если его начало и конец принадлежат этому sc.g-контуре.

Ниже представлен пример формализации предметной области объектного моделирования интеллектуальных систем, её структурной спецификации и теоретико-множественной онтологии на SCs-коде.

```
subject_domain_of_object_modeling_intelligent_systems
=> nrel_main_idtf:

  [Предметная область объектного моделирования интеллектуальных
   систем]

  (*
    <- lang_ru;;
  *);

<- concept_subject_domain;

=> nrel_ontology:

  structural_specification_of_subject_domain_of_object_modeling
  _intelligent_systems;

  set_theoretic_ontology_of_subject_domain_of_object_modeling
  _intelligent_systems;;

  structural_specification_of_subject_domain_of_object_modeling_intel
  ligent_systems = [*

<- concept_structural_specification_of_subject_domain;;

subject_domain_of_object_modeling_intelligent_systems = [*

-> rrel_maximum_studied_object_class:

  concept_object;

-> rrel_not_maximum_studied_object_class:

  concept_class_example;

  concept_class;
```

```

concept_interface_class;
concept_abstract_class;
concept_concrete_class;
concept_static_class;
concept_abstraction_process;
concept_encapsulation_process;
concept_polymorphism;
concept_inheritance_process;
-> rrel_explored_relation:
    rrel_class_member;
    rrel_class_field;
    rrel_class_method;
    rrel_private_class_member;
    rrel_protected_class_member;
    rrel_public_class_member;
    rrel_constant_class_member;
    nrel_related_class;
    nrel_parent_class;
    nrel_child_class;
    nrel_dependent_class;
    nrel_friend_class;
    nrel_association;
    nrel_aggregation;
    nrel_composition;
    nrel_implementation;;
*];
<= nrel_child_subject_domain:
    subject_domain_of_modeling_intelligent_systems;
=> nrel_child_subject_domain:

```

```
subject_domain_of_object_modeling_intelligent_systems_in_CPP;  
  
subject_domain_of_object_modeling_intelligent_systems_in_Python;;  
  
*];;
```

```
subject_domain_of_object_modeling_intelligent_systems = [  
  interface_class_I  
  <- concept_interface_class;  
  => nrel_implementation:  
    class_A;;
```

```
class_A  
  <- concept_concrete_class;  
  => nrel_composition:  
    class_B;  
  -> rrel_class_field: rrel_private_class_member:  
    B_b;  
  -> rrel_class_method: rrel_public_class_member:  
    void_Print;;
```

```
class_B  
  <- concept_concrete_class;  
  => nrel_friend_class:  
    class_C;;
```

```
class_C  
  <- concept_concrete_class;  
  <= nrel_child_class:  
    class_D;;
```

```
class_D
```

```
<- concept_concrete_class;
```

```
<= nrel_child_class:
```

```
    class_E;;
```

```
*];;
```

```
set_theoretic_ontology_of_subject_domain_of_object_modeling_intelli  
gent_systems = [*
```

```
<- concept_set_theoretic_ontology_of_subject_domain;;
```

```
concept_object
```

```
=> nrel_subdividing: {
```

```
    concept_class_example;
```

```
    concept_class
```

```
};;
```

```
concept_class
```

```
=> nrel_subdividing: {
```

```
    concept_interface_class;
```

```
    concept_abstract_class;
```

```
    concept_concrete_class
```

```
};;
```

```
concept_concrete_class
```

```
=> nrel_inclusion:
```

```
    concept_static_class;;
```

```
rrel_class_member
```

```
=> nrel_subdividing: {  
    rrel_class_field;  
    rrel_class_method  
};  
  
=> nrel_subdividing: {  
    rrel_private_class_member;  
    rrel_protected_class_member;  
    rrel_public_class_member  
};;
```

```
rrel_class_method  
=> nrel_inclusion:  
    rrel_constant_class_method;;
```

```
nrel_related_class  
=> nrel_subdividing: {  
    nrel_parent_class;  
    nrel_child_class  
};;
```

```
nrel_dependent_class  
=> nrel_inclusion:  
    nrel_friend_class;;
```

```
nrel_association  
=> nrel_inclusion:  
    nrel_aggregation;  
    nrel_composition;;  
*];;
```

6.6 Принципы обработки знаний при помощи Технологии OSTIS

База знаний ostis-системы представляет собой *sc-конструкцию*, из которой можно выделить множество других *sc-конструкций*, которые являются *sc-знаниями*.

Поскольку *sc-конструкции* являются знаковыми конструкциями, а точнее конструкциями, принадлежащими SC-коду, который, в свою очередь, является формальным языком, то над базой знаний, то есть над её *sc-конструкциями*, можно осуществлять различные операции.

Понятие операции сводится к понятию действия. В свою очередь понятие действия сводится к понятию воздействия. А понятие воздействия сводится к понятию процесса.

6.6.1 Понятие операции над *sc-конструкцией*

операция над sc-конструкцией — это операция, которая осуществляет переход из одного состояния этой *sc-конструкции* в другое состояние либо путём изменения конфигурации связей между *sc-элементами* этой *sc-конструкции*, либо путём установления факта наличия (отсутствия) этой *sc-конструкции* в базе знаний.

6.6.2 Виды операций над *sc-конструкциями*

Исходя из синтаксиса SC-кода и задач, которые решаются в базах знаний *ostis-систем*, операции над *sc-конструкциями* разбиваются на:

- операции создания *sc-элементов*:
 - операцию создания *sc-узла* заданного типа,
 - операцию создания *sc-коннектора* заданного типа между двумя заданными *sc-элементами*;
- операцию установки содержимого для заданного *sc-узла*, который является знаком файла;
- операцию уточнения типа заданного *sc-элемента*;
- операцию удаления *sc-элемента*;
- операции поиска *sc-конструкций*:
 - операции поиска *sc-конструкций* стандартного вида:
 - операцию поиска трёхэлементной *sc-конструкции*;
 - операцию поиска пятиэлементной *sc-конструкции*;

- операцию поиска *sc*-конструкции нестандартного вида.

Каждый вид операции реализуется как соответствующим методом (программой (функцией)), являющимся элементом Программного интерфейса Программной платформы *ostis*-систем, так и соответствующим *scr*-оператором языка процедурного программирования в *ostis*-системах — Языка *SCP*.

Метод поиска *sc*-конструкций является ключевым методом в задачах навигации и просмотра базы знаний *ostis*-системы.

6.6.3 Виды *sc*-конструкций

sc-конструкция — это знаковая конструкция, принадлежащая *SC*-коду. *sc*-конструкция является частным случаем *sc*-структуры.

sc-конструкции состоят из *sc*-элементов. Каждый *sc*-элемент *sc*-конструкции имеет свою строго фиксированную позицию.

sc-конструкции разбиваются на:

- *sc*-конструкции стандартного вида;
- *sc*-конструкции нестандартного вида.

6.6.3.1 Понятие *sc*-конструкции стандартного вида

sc-конструкция стандартного вида — это *sc*-конструкция, элементами которой являются:

- *sc*-коннектор и инцидентные ему *sc*-элементы,
- или *sc*-коннектор, а также инцидентные ему *sc*-элемент и *sc*-коннектор с инцидентными ему другими *sc*-элементами.

Каждый *sc*-элемент *sc*-конструкции стандартного вида имеет свою условную строго фиксированную позицию в рамках этой *sc*-конструкции (первый *sc*-элемент, второй *sc*-элемент, третий *sc*-элемент и т. д.).

В свою очередь *sc*-конструкции стандартного вида разбиваются на:

- *трёхэлементные sc*-конструкции;
- *пятиэлементные sc*-конструкции.

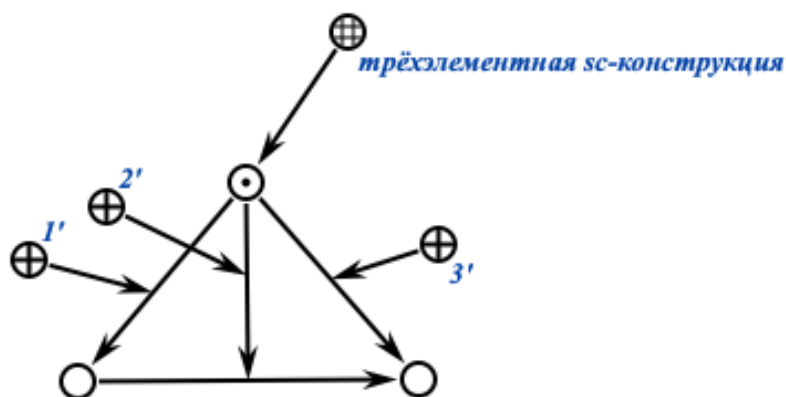
6.6.3.1.1 Понятие трёхэлементной sc-конструкция

трёхэлементная sc-конструкция — это *sc-конструкция стандартного вида*, элементами которой являются *sc-коннектор* и инцидентные ему *sc-элементы*.

Каждая *трёхэлементная sc-конструкция* состоит из трёх *sc-элементов*. Второй *sc-элемент* всегда является *sc-коннектором*, остальные элементы могут быть любого типа.

Другими словами, каждая *трёхэлементная sc-конструкция* представляет собой тройку вида: $\langle \text{sc-элемент } 1 \rangle \langle \text{sc-элемент } 2, \text{ являющийся sc-коннектором} \rangle \langle \text{sc-элемент } 3 \rangle$.

Пример *трёхэлементной sc-конструкции* на SCg-коде:



Поиск трёхэлементной *sc-конструкций* реализуется при помощи операции поиска трёхэлементной *sc-конструкции*.

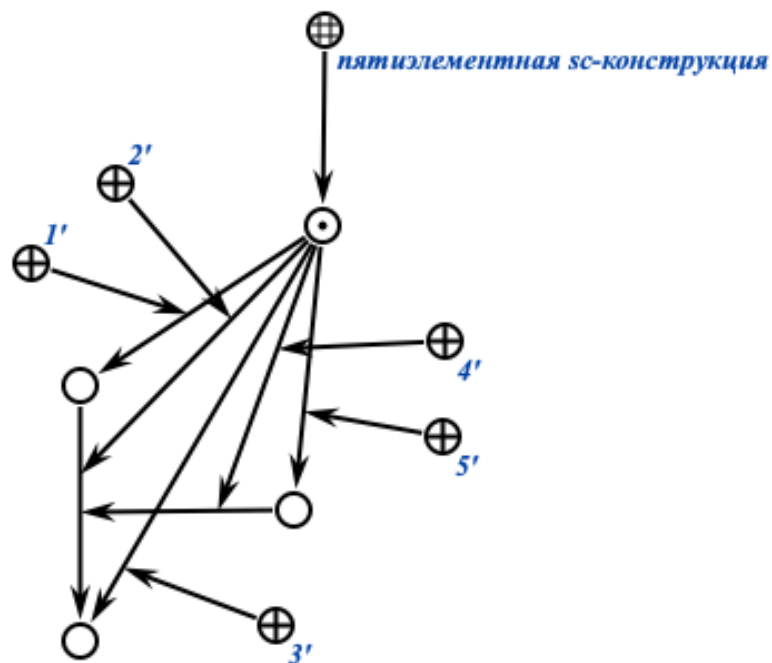
6.6.3.1.2 Понятие пятиэлементной sc-конструкция

пятиэлементная sc-конструкция — это *sc-конструкция стандартного вида*, элементами которой являются *sc-коннектор*, а также инцидентные ему *sc-элемент* и *sc-коннектор* с инцидентными ему другими *sc-элементами*.

Каждая *пятиэлементная sc-конструкция* состоит из пяти *sc-элементов*. Второй и четвертый *sc-элементы* всегда являются *sc-коннекторами* остальные элементы могут быть любого типа.

Другими словами, каждая *пятиэлементная sc-конструкция* представляет собой пятёрку вида: $\langle \text{sc-элемент } 1 \rangle \langle \text{sc-элемент } 2, \text{ являющийся sc-коннектором} \rangle \langle \text{sc-элемент } 3 \rangle \langle \text{sc-элемент } 4, \text{ являющийся sc-коннектором, входящим в sc-элемент } 2 \rangle \langle \text{sc-элемент } 5 \rangle$.

Пример *пятиэлементной sc-конструкции* на SCg-коде:



Поиск пятиэлементной *sc*-конструкций реализуется при помощи операции поиска пятиэлементной *sc*-конструкции.

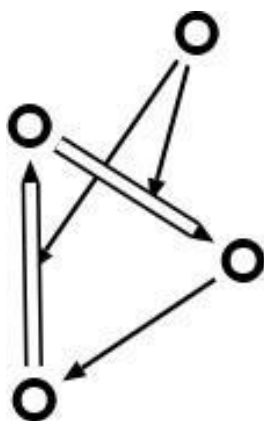
Операция поиска трёхэлементной *sc*-конструкции и операция поиска пятиэлементной *sc*-конструкции являются тривиальными.

6.6.3.2 Понятие *sc*-конструкции нестандартного вида

sc-конструкция нестандартного вида — это *sc*-конструкция, которая не является *sc*-конструкцией стандартного вида.

Каждая *sc*-конструкция нестандартного вида состоит из произвольного количества *sc*-элементов произвольного типа.

Пример *sc*-конструкции нестандартного вида на SCg-коде:



Поиск sc-конструкций нестандартного вида реализуется при помощи операции поиска sc-конструкции нестандартного вида.

Операция поиска sc-конструкции нестандартного вида является нетривиальной.

6.6.4 Понятие операции поиска sc-конструкции нестандартного вида

Поиск sc-конструкций нестандартного вида, как и sc-конструкций стандартного вида, можно свести к изоморфному поиску sc-конструкций, то есть поиску sc-конструкций по некоторым заданным обобщённым sc-конструкциям (графам-шаблонам (графам-образцам)).

Изоморфный поиск является одним из способов решения задачи поиска подграфа в графе. Эта задача заключается в том, чтобы найти все вхождения заданного графа-образца в исходном графе, заданном в базе знаний. Данная задача является алгоритмически и вычислительно сложной.

С вычислительной точки зрения задача поиска подграфа принадлежит классу NP-полных задач. Это означает, что для неё не известен полиномиальный алгоритм решения, и время работы существующих точных алгоритмов экспоненциально растёт с увеличением размера входных данных. Доказательство NP-полноты данной задачи основано на полиномиальной сводимости к ней задачи о клике — классической NP-полной задачи, в которой требуется определить, содержит ли граф полный подграф заданного размера.

Сложность задачи обусловлена необходимостью установления взаимно-однозначного соответствия (биективной функции) $f: V_0 \rightarrow V_p$ между вершинами подграфа и вершинами образца, при котором рёбра $\{v_1, v_2\} \in E_0$

существуют тогда и только тогда, когда существуют соответствующие рёбра $\{f(v1), f(v2)\} \in E_p$.

Количество возможных вариантов соответствия растёт факториально с увеличением числа вершин, что приводит к комбинаторному взрыву вычислений.

6.6.5 Понятие обобщённой sc-структуры

обобщённая sc-структура, или *граф-образец* — это sc-структура, которая содержит хотя бы одну переменную sc-связку, обозначаемой sc-коннектором.

6.6.5.1 Понятие sc-переменной

sc-переменная — это sc-элемент, областью возможных значений которого является множество sc-констант.

переменная sc-связка — это sc-связка, областью возможных значений которой является множество sc-связок.

6.6.5.2 Понятие sc-шаблона

Частным видом обобщённой sc-структуры является sc-шаблон.

sc-шаблон — это обобщённая sc-структура, в которой для каждой sc-переменной в каждой трёхэлементной sc-конструкции можно поставить в изоморфное соответствие sc-константу соответствующего типа так, чтобы:

- *сохранялась структурная согласованность sc-элементов* — каждому переменному sc-элементу ставился в соответствие константный sc-элемент того же структурного типа;
- *сохранялась топологическая структура sc-конструкций* — отношения инцидентности между sc-элементами в шаблоне сохранялись после подстановки (если в шаблоне переменная sc-дуга выходит из одного переменного sc-узла и входит в другой, то константная sc-дуга должна выходить из соответствующего константного sc-узла и входить в соответствующий константный sc-узел);
- *результат подстановки образовывал синтаксически и семантически корректную sc-структуру* — после замены всех переменных sc-элементов на соответствующие константные sc-элементы каждая трёхэлементная sc-конструкция шаблона превращалась в синтаксически и

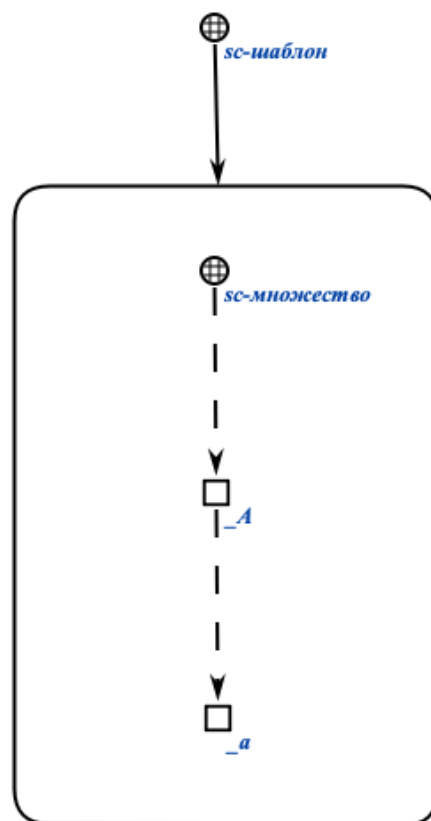
семантически корректную трёхэлементную sc-конструкцию, удовлетворяющую правилам SC-кода;

- *изоморфизм sc-структур был корректным* — отображение между переменными и константными sc-элементами оставалось взаимно однозначным в рамках одного экземпляра подстановки (одной и той же переменной всегда соответствует один и тот же константный элемент в рамках данной подстановки, но разным переменным могут соответствовать как разные, так и одинаковые константные элементы в зависимости от ограничений шаблона).

Проще говоря, sc-шаблон можно интерпретировать как параметрическую схему сопоставления, которую можно корректно "наложить" на любую sc-структуру путём замены всех sc-переменных на соответствующие sc-константы, так, чтобы результат был синтаксически и семантически корректной sc-структурой.

Частным случаем задачи изоморфного поиска в базе знаний ostis-системы является задача поиска по sc-шаблону.

Пример sc-шаблона на SCg-коде:



Данный sc-шаблон является описанием задачи поиска всех sc-множеств и их элементов в базе знаний ostis-системы.

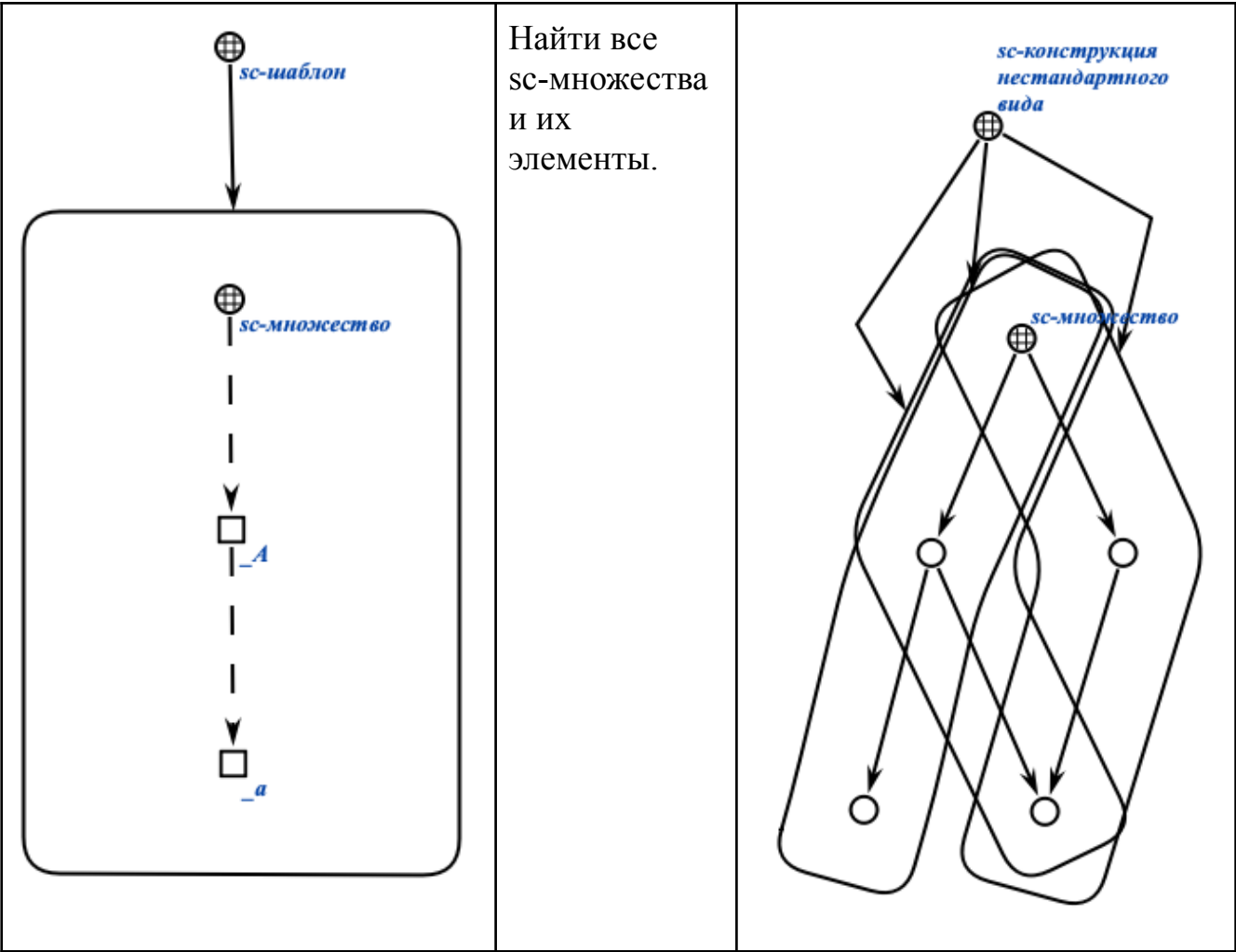
Зачастую sc-шаблоны используются не только для поиска sc-конструкций в базе знаний, изоморфных данному sc-шаблону, но и для создания таких sc-конструкций в базе знаний ostis-системы.

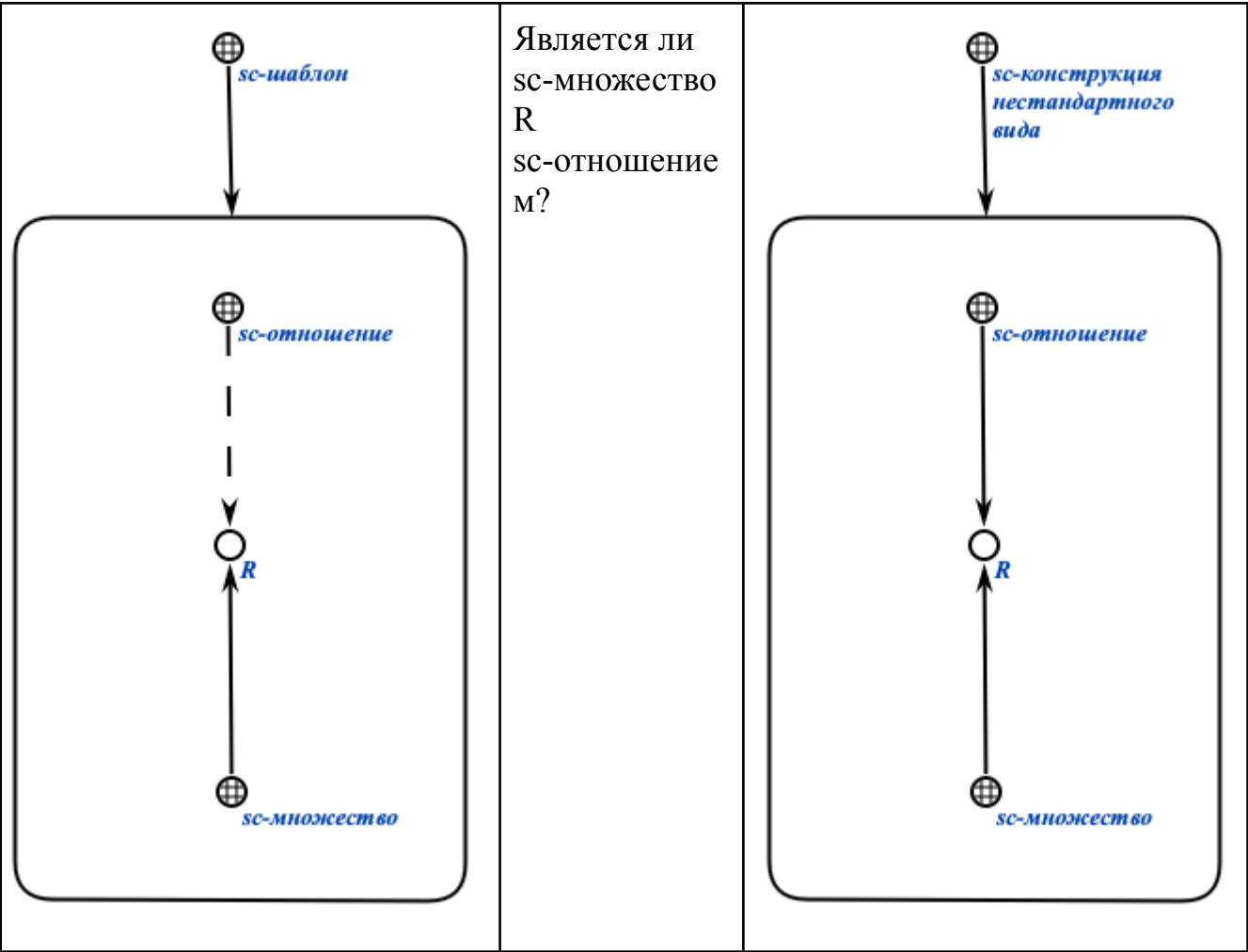
Задача удаления sc-конструкций по sc-шаблону сводится к задаче поиска таких sc-конструкций и удаления их или отдельных sc-элементов, принадлежащих этим sc-конструкциям.

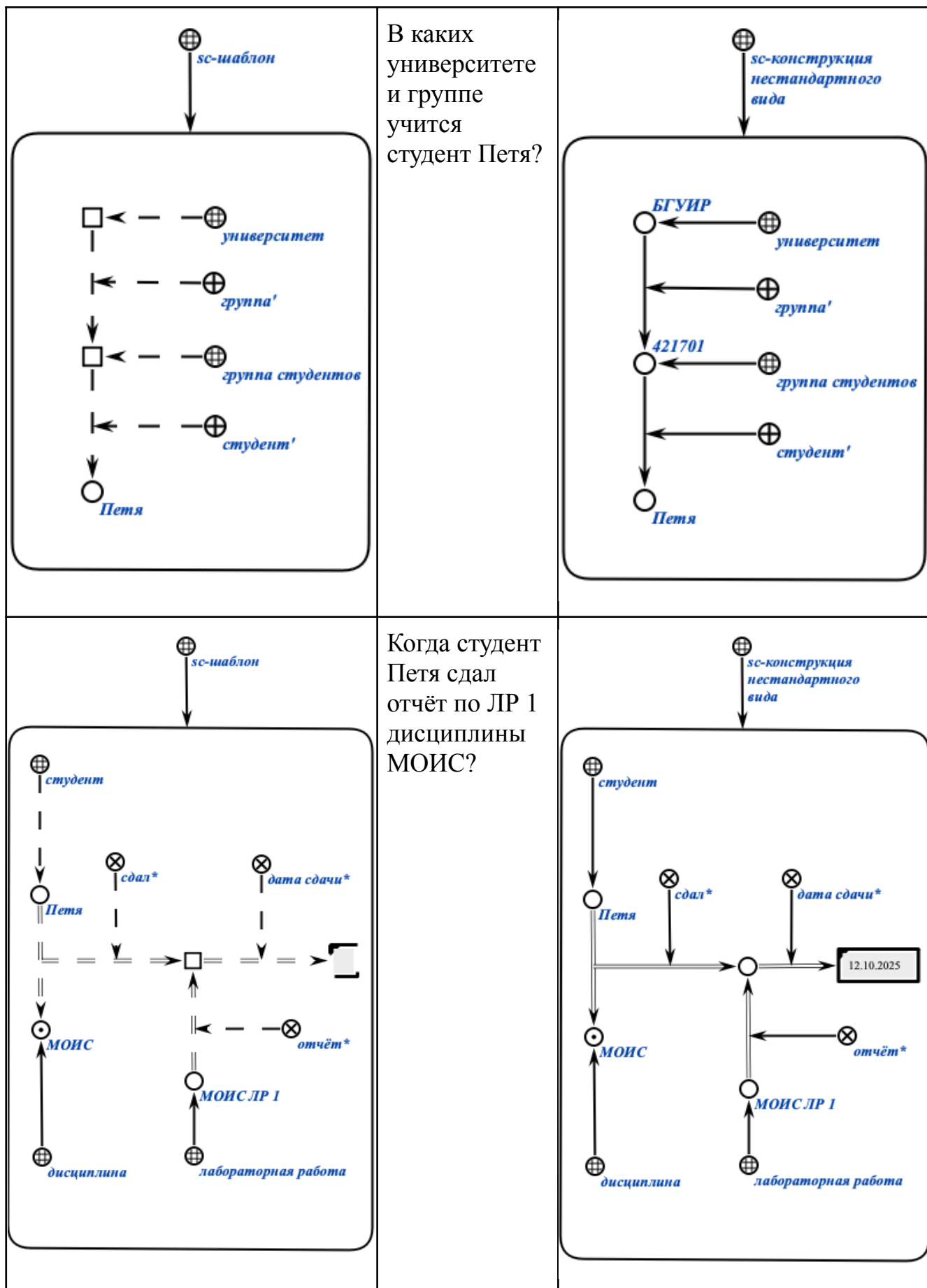
6.6.5.2.1 Примеры sc-шаблонов и sc-конструкций, изоморфных им, представленных на SCg-коде

В таблице ниже на SCg-коде представлены sc-шаблоны (слева) и sc-конструкции, изоморфные им (справа). Для удобства связки изоморфного соответствия между sc-шаблоном и sc-конструкциями, а также связки второго компонента пары отношения изоморфизма между элементами sc-шаблона и sc-конструкциями опущены.

sc-шаблон	Описание задачи	sc-конструкции, изоморфные заданному sc-шаблону
------------------	------------------------	--







6.6.5.2.2 Недостатки sc-шаблонов и способы устранения этих недостатков

sc-шаблоны представляют собой удобный инструмент для поиска информации в базе знаний ostis-системы. Множество однотипных изоморфных друг другу sc-конструкций любого размера можно обобщить в виде sc-шаблона. Однако использование sc-шаблонов сопряжено с рядом существенных недостатков, в первую очередь обусловленных вычислительной сложностью алгоритма изоморфного поиска, реализующего поиск по таким sc-шаблонам:

1. Задача изоморфного поиска в графе представляет собой NP-полную задачу, что означает экспоненциальный рост вычислительной сложности с увеличением размера входных данных. Некоторые алгоритмы изоморфного поиска имеют сложность $O(n!)$, что делает их непрактичными для работы с крупными графами. Даже современные алгоритмы, способные решать задачу за квазиполиномиальное время, остаются практически недостаточно эффективными для крупных баз знаний.
2. При работе с графами значительного размера время, затрачиваемое на перебор всех возможных изоморфизмов, может быть слишком высоким, что существенно снижает эффективность поиска и ограничивает применимость изоморфного поиска в реальных системах. Сложность поиска возрастает с увеличением количества циклов в исходном графе, поскольку это приводит к большему числу итераций.
3. Эффективность изоморфного поиска сильно зависит от состояния базы знаний. Чем более разнообразны фрагменты базы знаний, тем хуже производительность алгоритма изоморфного поиска.

В [Программной платформе ostis-систем](#) реализован специализированный алгоритм изоморфного поиска, который превосходит по эффективности большинство существующих решений, однако не является универсальным. С помощью данного алгоритма можно за малое количество времени находить sc-конструкции, изоморфные достаточно сложным sc-шаблонам, но не всем возможным. Реализация данного алгоритма сложна и поэтому не рассматривается в данной лабораторной работе.

Наиболее правильным вариантом решения данных проблем являются реализация и использование алгоритма поиска по sc-шаблону с возможностью выбора и корректирования стратегии поиска по такому sc-шаблону.

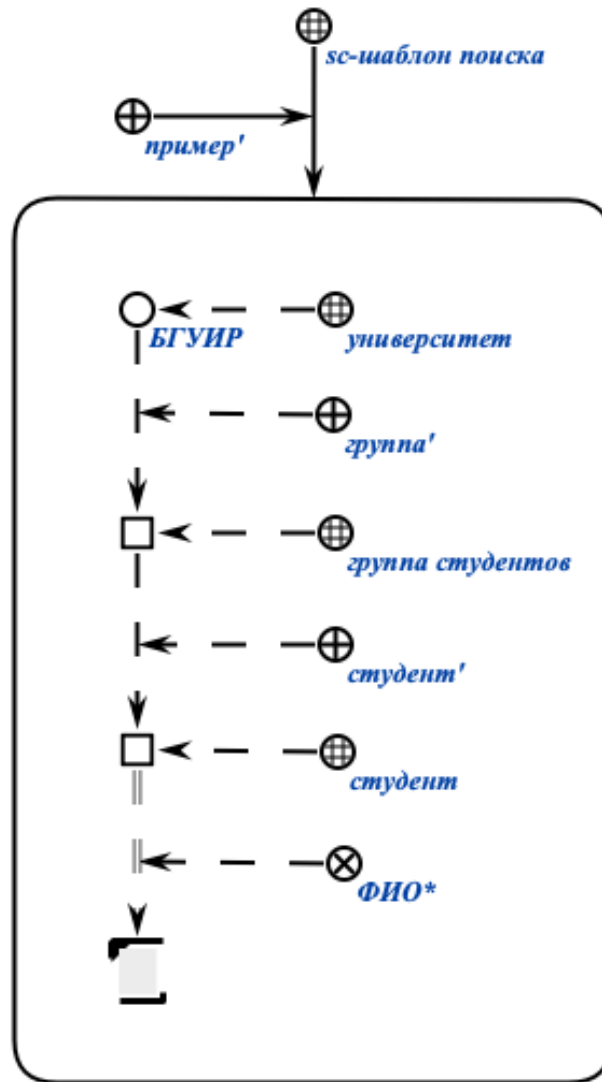
6.6.5.3 Понятие sc-шаблона с фиксированной стратегией поиска

sc-шаблон с фиксированной стратегией поиска — это множество, представляющее собой декомпозицию некоторого исходного sc-шаблона на частные sc-шаблоны с указанием порядка применения и поиска по этим sc-шаблонам.

Любой sc-шаблон можно и следует приводить к sc-шаблону с фиксированной стратегией поиска.

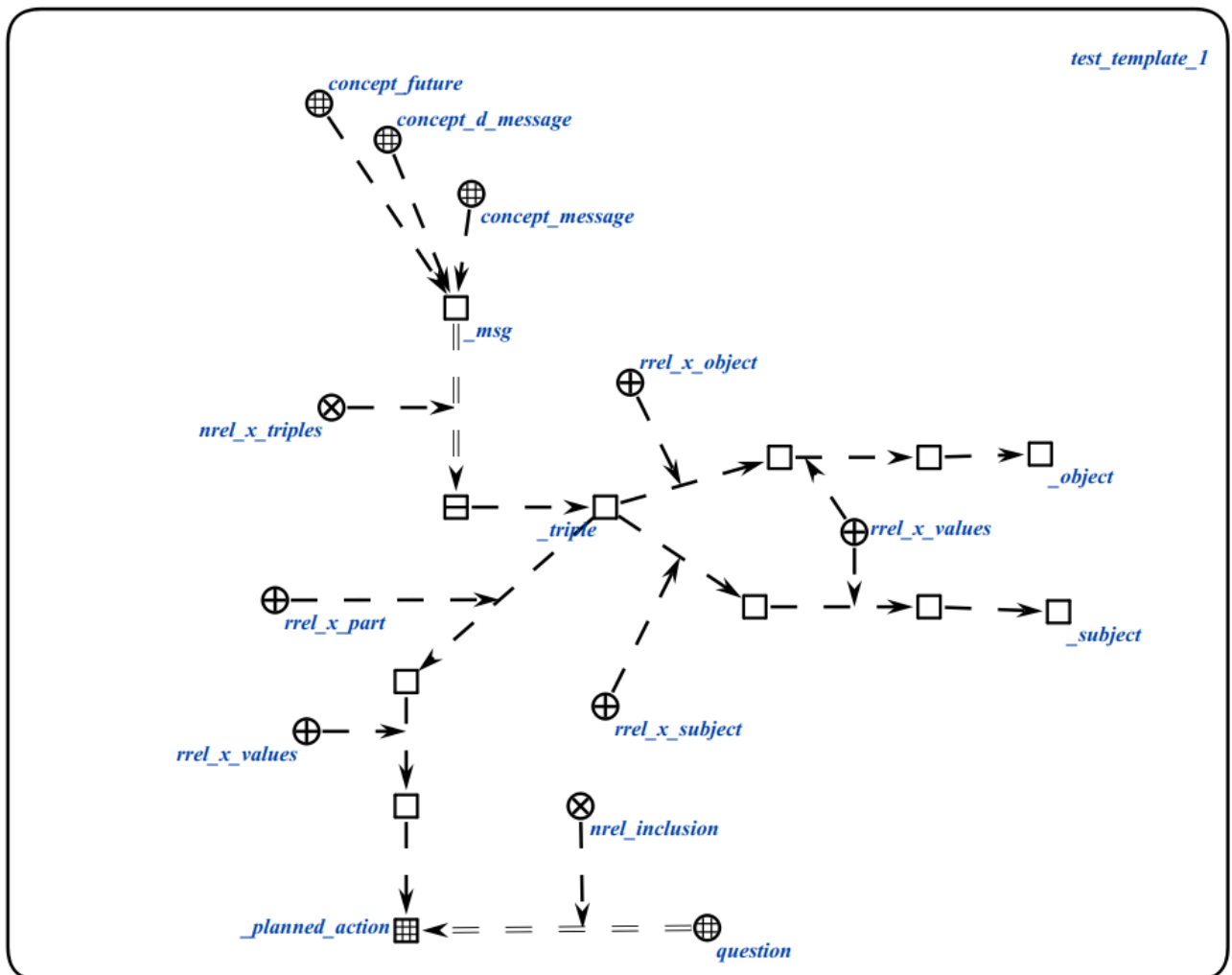
6.6.5.4 Принципы выбора стратегии поиска по sc-шаблону и её спецификации в рамках этого sc-шаблона

Рассмотрим проблему выбора стратегии поиска по заданному sc-шаблону. На рисунке ниже на SCg-коде представлен sc-шаблон без фиксированной стратегии поиска:



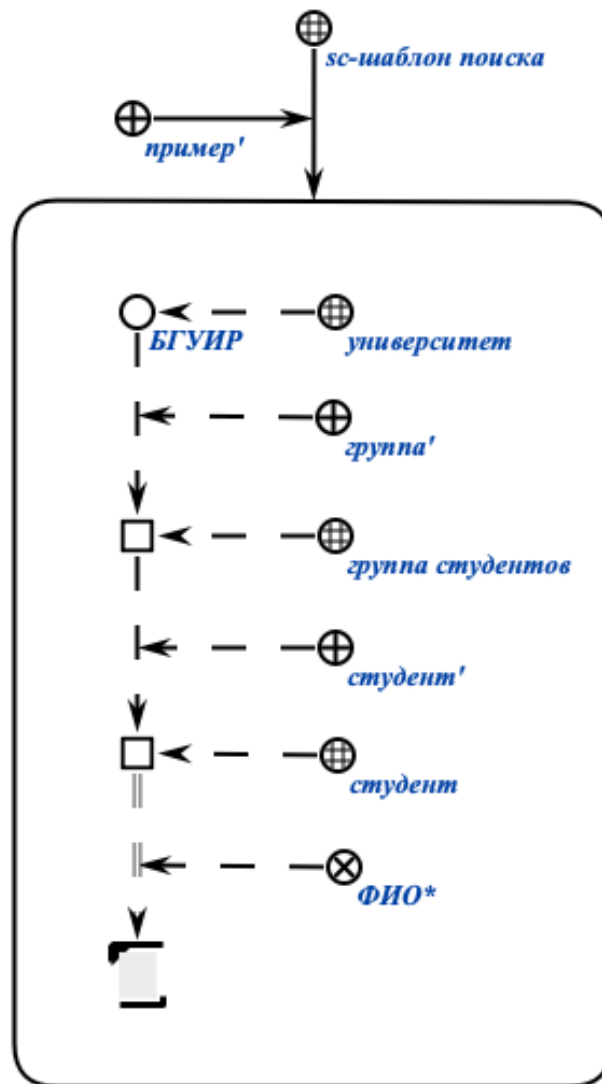
Основной задачей данного sc-шаблона является задача поиска ФИО всех студентов всех групп университета БГУИР. Очевидно, что самый простой способ начать искать sc-конструкции по такому sc-шаблону — это начать поиск от sc-константы, обозначающей класс университетов, или от sc-константы, обозначающего университет БГУИР. Однако, программа, реализующая алгоритм поиска по некоторому sc-шаблону (необязательно заданному) не знает этого и не может узнать об этом, поскольку ей это никак не указывается. Программа может начать поиск с любой другой sc-константы, например, ФИО*, тем самым затратив намного больше времени на обход всех sc-конструкций, которые изоморфны данному sc-шаблону, и всех sc-конструкций, которые не изоморфны данному sc-шаблону, однако после дополнения которых, могли бы стать изоморфными данному sc-шаблону.

На практике при реализации прикладных ostis-систем может появляться необходимость в создании более сложных sc-шаблонов, например, таких, как ЭТОТ:

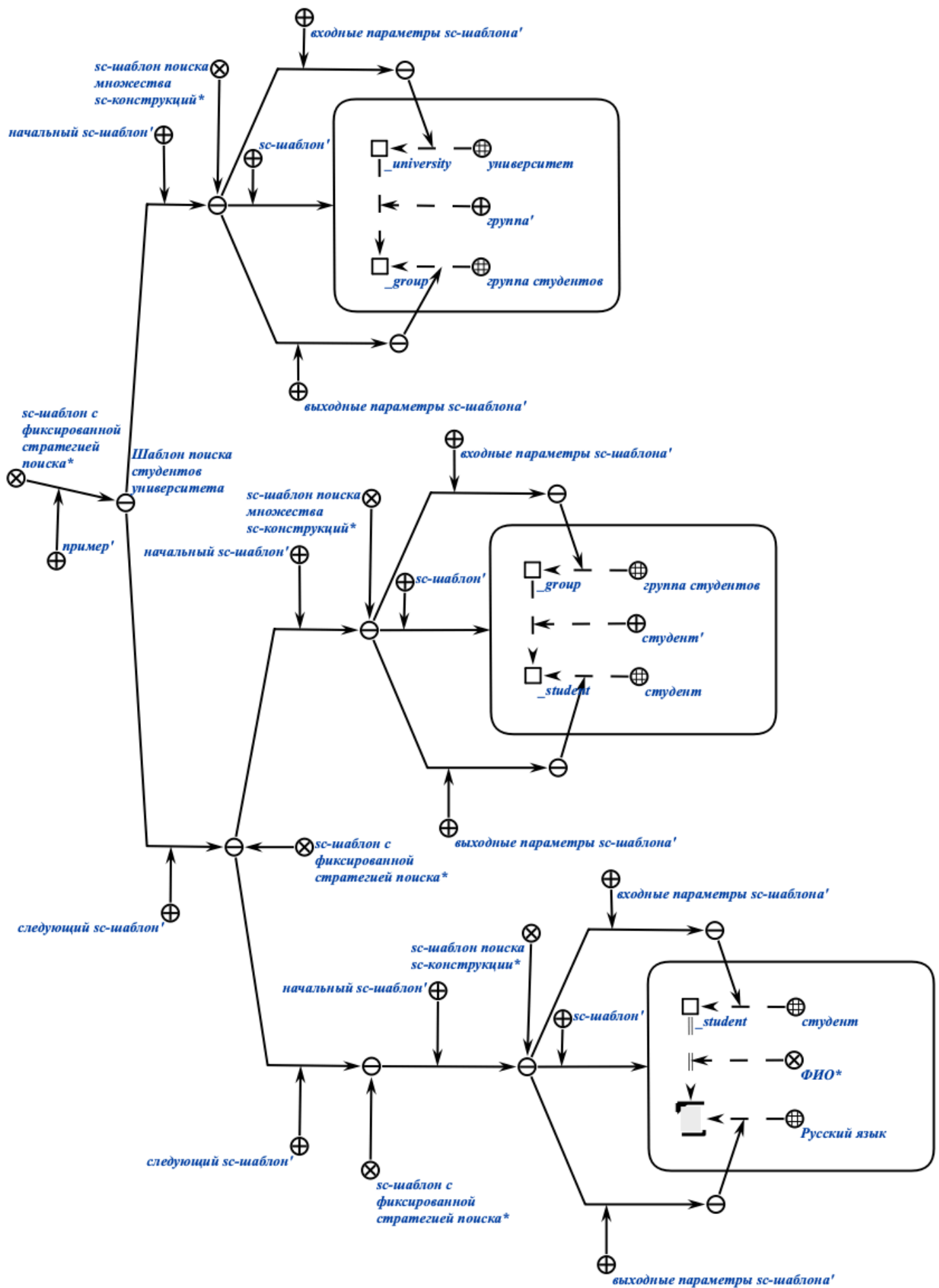


Неправильный выбор стратегии поиска по такому sc-шаблону может существенно повлиять на работу программы, реализующей алгоритм поиска по sc-шаблонам, и соответственно, на работу всей системы в целом.

Рассмотрим процесс преобразования sc-шаблона в sc-шаблон с фиксированной стратегией поиска. На рисунке ниже на SCg-коде представлен рассмотренный ранее sc-шаблон без фиксированной стратегии поиска:



sc-шаблон с фиксированной стратегией поиска, соответствующий заданному sc-шаблону, можно представить на SCg-коде следующим образом:



Для описания sc-шаблона с фиксированной стратегией поиска используются следующие понятия:

- *начальный sc-шаблон* — это ролевое отношение, указывающее в рамках sc-шаблона с фиксированной стратегией поиска на частный sc-шаблон (sc-шаблон с заданными входными и выходными параметрами или sc-шаблон с фиксированной стратегией поиска), с которого следует начинать операцию поиска по заданному sc-шаблону.
- *следующий sc-шаблон* — это ролевое отношение, указывающее в рамках sc-шаблона с фиксированной стратегией поиска на частный sc-шаблон (sc-шаблон с заданными входными и выходными параметрами или sc-шаблон с фиксированной стратегией поиска), с которого следует продолжать операцию поиска по заданному sc-шаблону после успешного поиска по начальному sc-шаблону.
- *sc-шаблон с заданными входными и выходными параметрами* — это sc-связка, у которой элементом с ролью sc-шаблон' является sc-шаблон, элементом с ролью входные параметры sc-шаблона' является множество переменных sc-коннекторов, являющихся элементами указанного sc-шаблона и являющихся входными параметрами указанного sc-шаблона, а элементом с ролью выходные параметры sc-шаблона' является множество переменных sc-коннекторов, являющихся элементами указанного sc-шаблона и являющихся выходными (результатирующими) параметрами указанного sc-шаблона.
- *sc-шаблон* — это ролевое отношение, указывающее в рамках sc-шаблона с заданными входными и выходными параметрами на sc-шаблон.
- *входные параметры sc-шаблона* — это ролевое отношение, указывающее в рамках sc-шаблона с заданными входными и выходными параметрами на множество входных параметров этого sc-шаблона, то есть на множество переменных sc-коннекторов, являющихся элементами этого sc-шаблона, на место которых необходимо подставить соответствующие константные sc-коннекторы, чтобы можно было начать поиск по заданному sc-шаблону.
- *выходные параметры sc-шаблона* — это ролевое отношение, указывающее в рамках sc-шаблона с заданными входными и выходными параметрами на множество выходных параметров этого sc-шаблона, то есть на множество переменных sc-коннекторов, являющихся элементами этого sc-шаблона, для которых необходимо найти константные sc-коннекторы и использовать их далее.

- *sc-шаблон поиска множества sc-конструкций* — это *sc-шаблон поиска*, по которому необходимо найти множество изоморфных ему *sc-конструкций*.
- *sc-шаблон поиска sc-конструкции* — это *sc-шаблон поиска*, по которому необходимо найти хотя бы одну изоморфную ему *sc-конструкцию*.

sc-шаблоны с фиксированной стратегией поиска можно использовать как аргументы поисковых *scp-операторов* Языка SCP.

6.6.6 Понятие Языка SCP

В качестве базового языка для описания программ обработки текстов SC-кода используется Язык SCP.

Язык SCP — это графовый язык параллельного асинхронного процедурного программирования, предназначенный для эффективной обработки *sc-текстов*.

Язык SCP является ассемблером для *ostis-систем*.

6.6.6.1 Понятие *scp-программы*

scp-программа — это обобщенная *sc-структура*, описывающая один из вариантов декомпозиции действий некоторого класса, выполняемых в *sc-памяти*.

Каждая *scp-программа* описывает в обобщенном виде декомпозицию некоторого *scp-процесса* на взаимосвязанные *scp-операторы*, с указанием, при их наличии, аргументов для данного *scp-процесса*.

6.6.6.2 Понятие *scp-процесса*

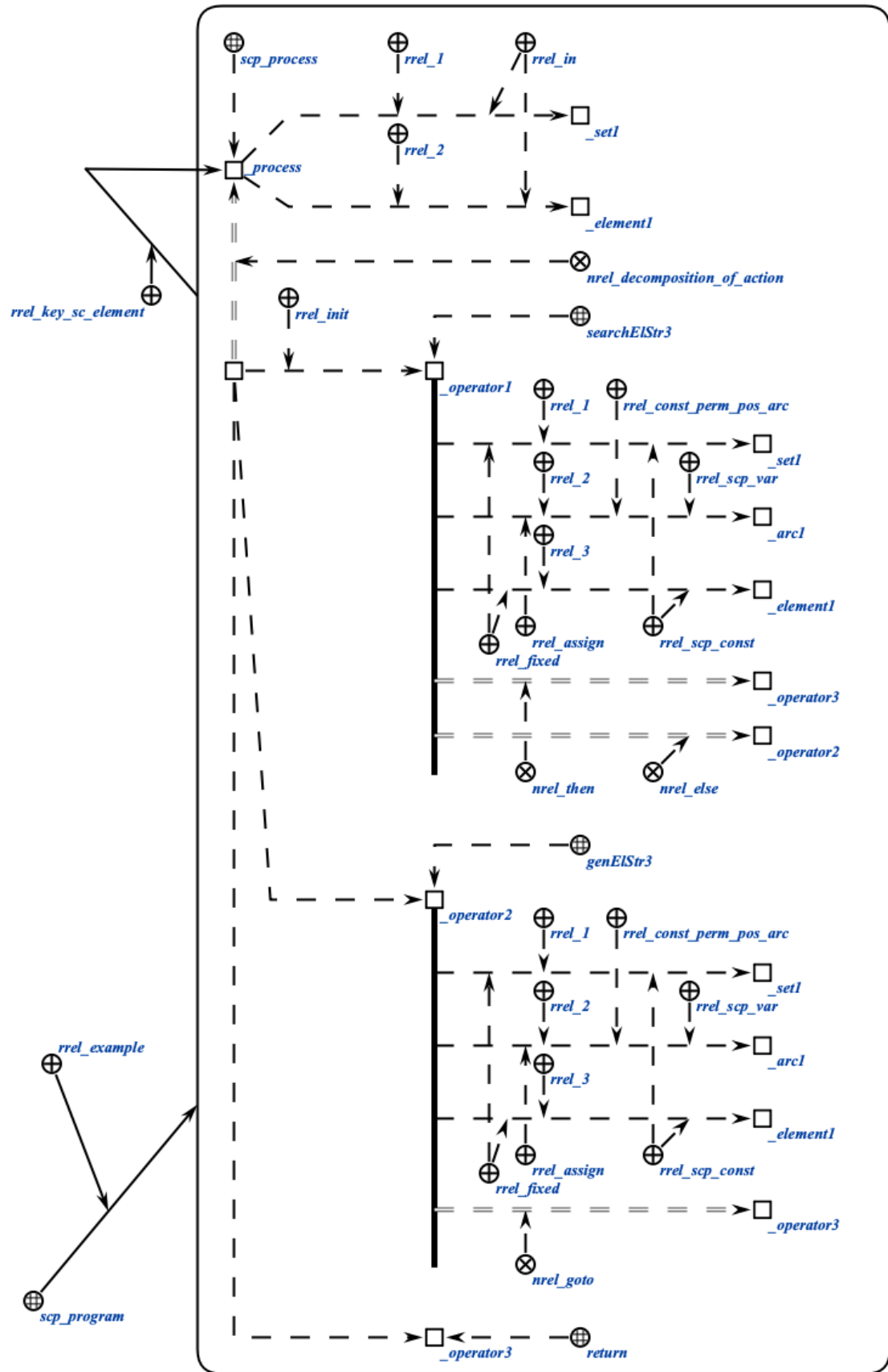
scp-процесс — это действие в *sc-памяти*, однозначно описывающее конкретный акт выполнения некоторой *scp-программы* для заданных исходных данных.

Если *scp-программа* описывает алгоритм решения какой-либо задачи в общем виде, то *scp-процесс* обозначает конкретное действие, реализующее данный алгоритм для заданных входных параметров.

Принадлежность некоторого действия множеству *scp-процессов* гарантирует тот факт, что в декомпозиции данного действия будут присутствовать только знаки элементарных действий (*scp-операторов*).

6.6.6.3 Пример выполнения scr-программы

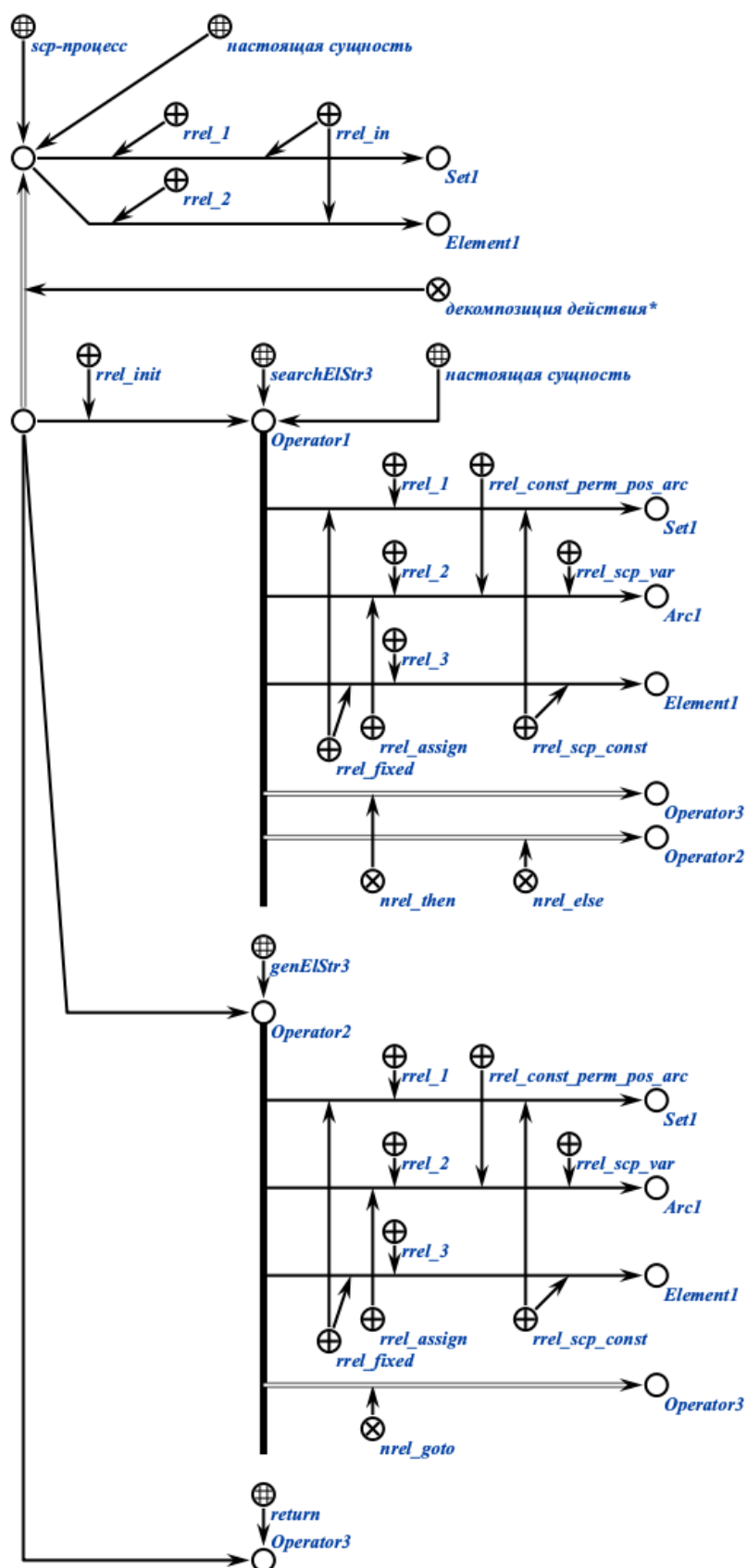
На рисунке ниже представлен пример scr-программы на SCg-коде:



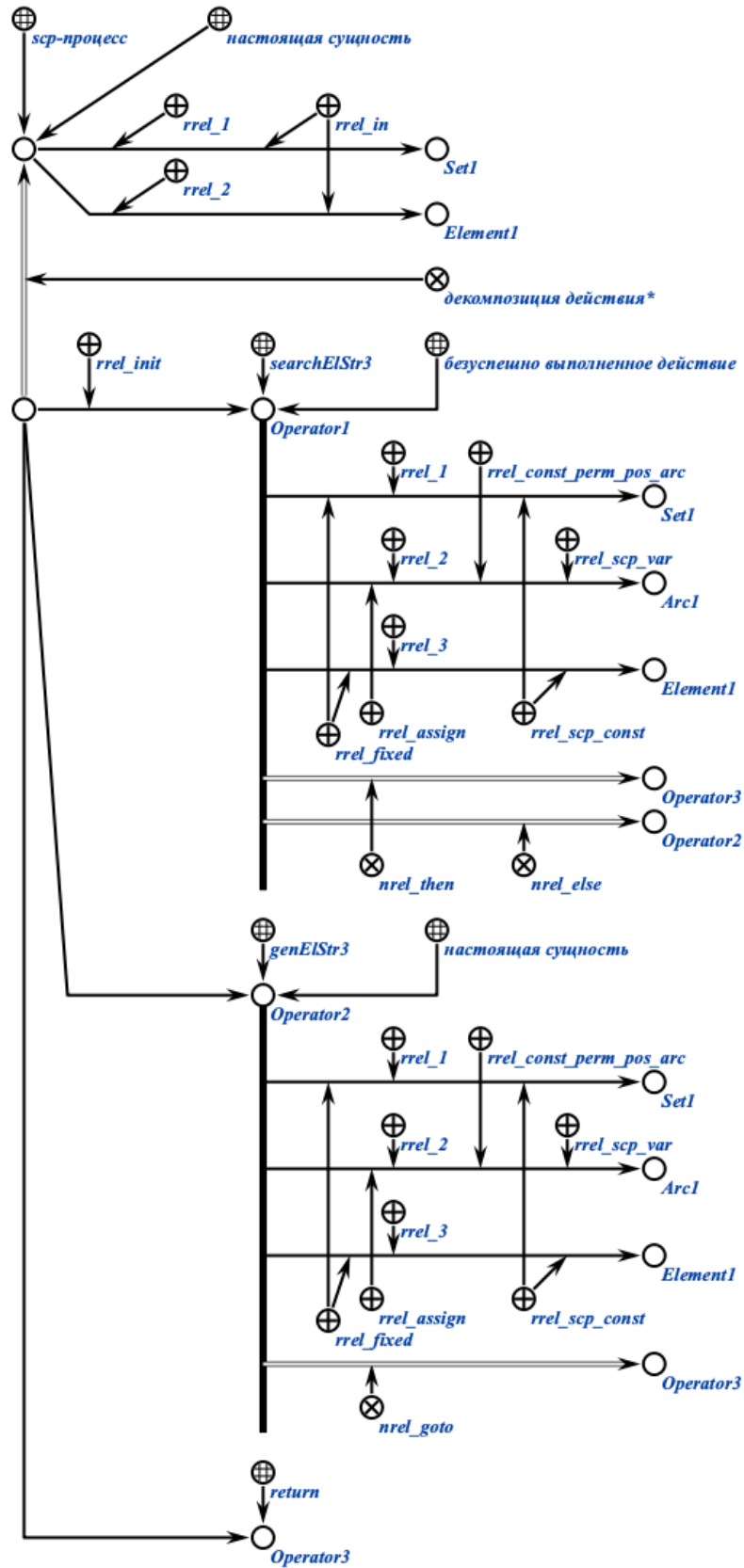
В приведенном примере показана *scp-программа*, описывающая *scp-процесс*, состоящий из трёх *scp-операторов*. Данная программа проверяет, содержится ли в заданном *sc-множестве* (первый параметр) заданный *sc-элемент* (второй параметр), и, если нет, то добавляет его в это множество.

Выполнение программы происходит следующим образом.

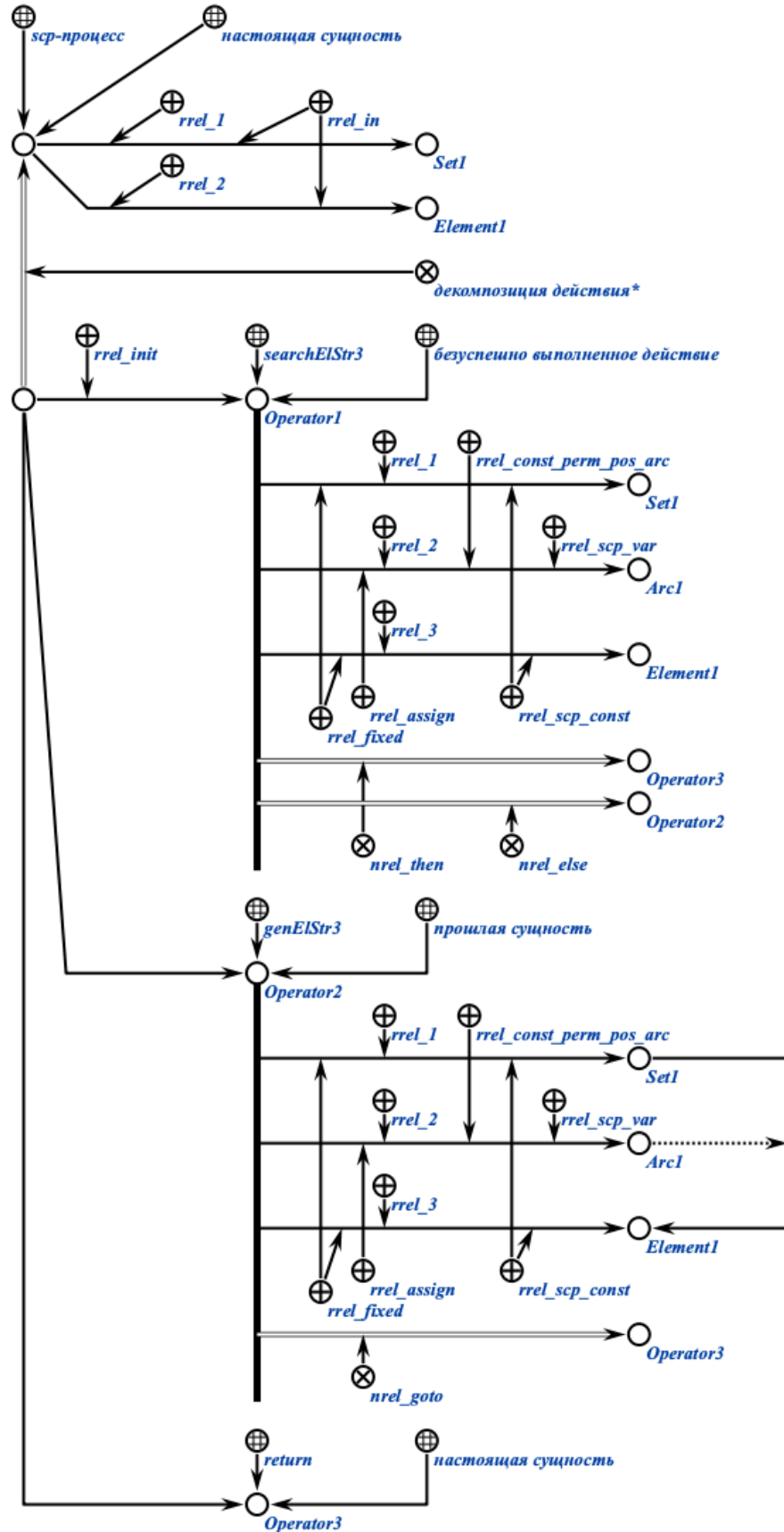
Осуществляется вызов заданной *scp-программы*. По ней создаётся соответствующий *scp-процесс*. Происходит инициирование начального *scp-оператора* *Operator1*.



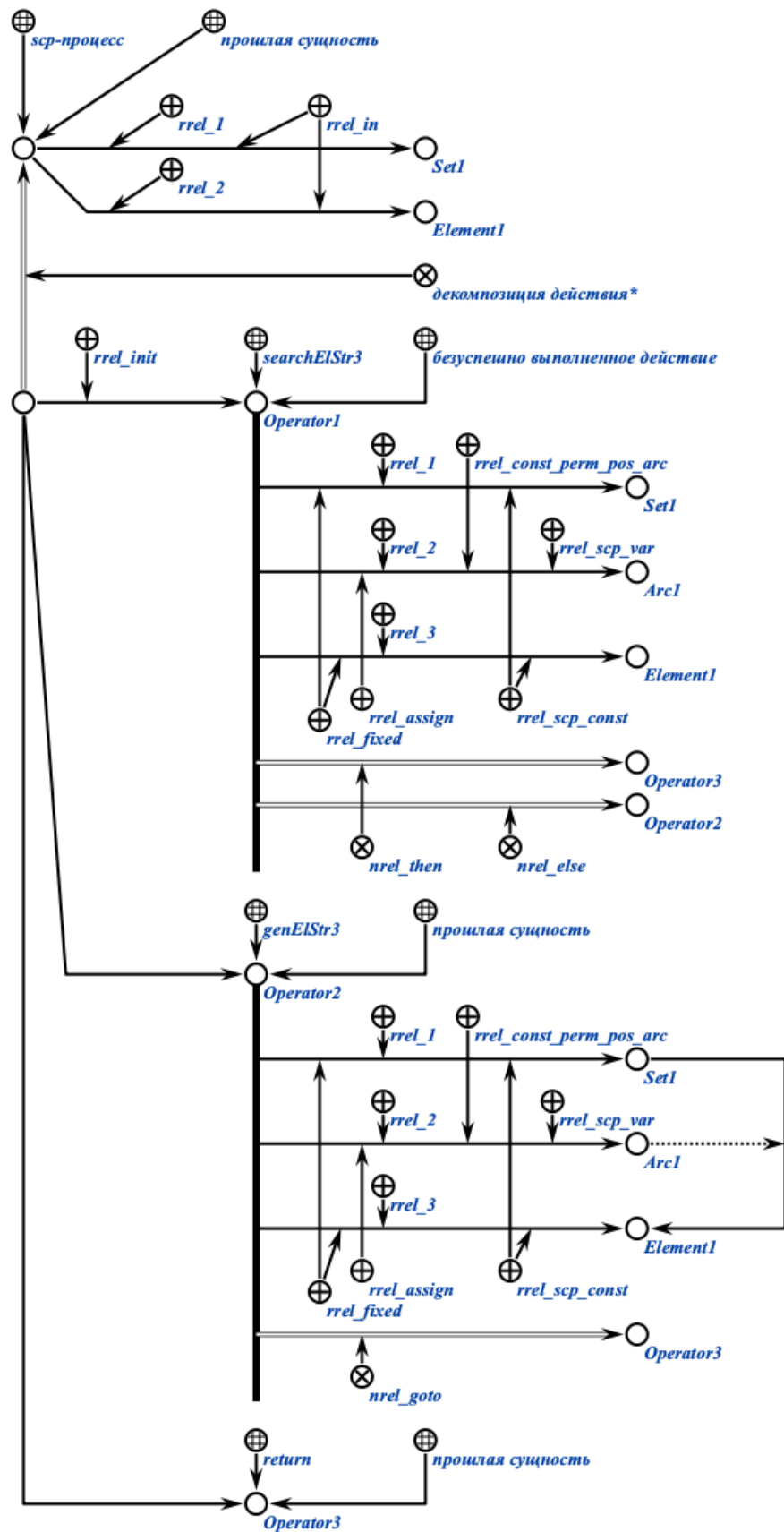
Допустим, *scp-оператор* *Operator1* оказался безуспешно выполненным. Тогда производится инициирование *scp-оператора* генерации трёхэлементной конструкции *Operator2*.



После выполнения *scp-оператора* *Operator2* производится инициирование *scp-оператора* завершения выполнения программы *Operator3*.



Когда scp-оператор *Operator3* выполнен, выполнение всего *scp-процесса* завершается.



В приведенном примере последовательно показаны состояния *scp-процесса*, соответствующего *scp-программе*, добавляющей заданный элемент в заданное множество, если он там ранее не содержался. В примере предполагается, что рассматриваемый элемент (*Element1*) изначально не содержится во множестве (*Set1*).

6.6.6.4 Понятие *scp-оператора*

scp-оператор — это элементарное действие в *sc-памяти*.

Каждый *scp-оператор* должен иметь один и более *scp-операндов*, а также указание того *scp-оператора* (или нескольких), который должен быть выполнен следующим. Исключение из данного правила составляет *scp-оператор завершения выполнения программы*, который не содержит ни одного *scp-операнда* и после выполнения которого никакие *scp-операторы* в рамках данной программы выполняться не могут.

6.6.6.5 Понятие *scp-операнда*

scp-операнд — это аргумент *scp-оператора*.

Порядок *scp-операндов* указывается при помощи соответствующих ролевых отношений (*1'*, *2'*, *3'* и так далее).

первый scp-операнд' — это *scp-операнд* с ролью *1'*.

второй scp-операнд' — это *scp-операнд* с ролью *2'*.

В общем случае *scp-операндом* может быть любой *sc-элемент*, в том числе, знак какой-либо *scp-программы*, в том числе самой программы, содержащей данный *scp-оператор*.

Тип и смысл каждого *scp-операнда* также уточняется при помощи различных подклассов ролевого отношения *scp-операнд'*.

scp-операнд' — это ролевое отношение, которое указывает на принадлежность аргументов заданному *scp-оператору*.

6.6.6.5.1 Понятие *scp-константы* и понятие *scp-переменной*

scp-операнды' делятся на *scp-константы'* и *scp-переменные'*.

6.6.6.5.1.1 Понятие *spr*-константы

spr-константами' в рамках *spr*-программы могут быть *sc*-элементы любого типа, как *sc*-константы, так и *sc*-переменные. *spr*-константы' остаются неизменными в течение всего срока интерпретации *spr*-программы.

spr-константа' — это ролевое отношение, указывающее в рамках *spr*-оператора на *spr*-операнд, значение которого совпадает с самим *spr*-операндом в каждый момент времени и никогда не изменяется.

Таким образом, далее будем считать, что *spr*-константа' и ее значение это одно и то же.

Каждый *входной параметр*' при интерпретации каждой конкретной копии *spr*-программы становится *spr*-константой' в рамках всех её *spr*-операторов, хотя в исходном теле данной *spr*-программы в каждом из этих *spr*-операторов он является *spr*-переменной'.

6.6.6.5.1.2 Понятие *spr*-переменной

spr-переменная' — это ролевое отношение, указывающее в рамках *spr*-оператора на *spr*-операнд, который в каждый момент времени имеет только одно значение, то есть представляет собой ситуативный *синглтон*, элементом которого является текущее значение этой *sc*-переменной.

Значение каждой *spr*-переменной' может меняться в ходе интерпретации *spr*-программы.

При этом интерпретатор при обработке *spr*-оператора работает непосредственно со значениями *spr*-переменных', а не с самими *spr*-переменными' (которые также являются узлами той же семантической сети).

6.6.6.5.2 Понятие *spr*-операнда с заданным значением и понятие *spr*-операнда со свободным значением

spr-операнды' делятся на *spr*-операнды с заданным значением' и *spr*-операнды со свободным значением'.

6.6.6.5.2.1 Понятие *spr*-операнда с заданным значением

spr-операнд с заданным значением' — это ролевое отношение, указывающее в рамках *spr*-оператора на *sc*-операнд, который имеет значение в рамках этого *spr*-оператора.

Значение *scr-операнда с заданным значением* учитывается при выполнении *scr-оператора* и остается неизменным после окончания выполнения *scr-оператора*.

Каждая *scr-константа* по-умолчанию рассматривается как *scr-операнд с заданным значением*, в связи с чем явное использование данного ролевого отношения в таком случае является избыточным. В таком случае в качестве значения рассматривается непосредственно сам *scr-операнд*.

В случае если отношением *scr-операнд с заданным значением* помечена *scr-переменная*, то осуществляется попытка поиска значения для данной *scr-переменной* (её элемента). Если попытка оказалась безуспешной, то возникает ошибка времени выполнения, которая должна быть обработана соответствующим образом.

Любой *scr-операнд с заданным значением* независимо от конкретного типа *scr-оператора* может быть *scr-переменной*.

6.6.6.5.2.2 Понятие *scr-операнда со свободным значением*

scr-операнд со свободным значением — это ролевое отношение, указывающее в рамках *scr-оператора* на *scr-операнд*, который не имеет значение в рамках этого *scr-оператора*.

В начале выполнения *scr-оператора* связь между *scr-переменной*, помеченной данным ролевым отношением, и её элементом (значением) всегда удаляется.

В результате выполнения данного *scr-оператора* может быть либо создано новое значение *scr-переменной*, либо не создано, тогда *scr-переменная* будет считаться не имеющей значения.

Ни одна *scr-константа* не может быть помечена как *scr-операнд со свободным значением*, поскольку *scr-константа* не может изменять свое значение в ходе интерпретации *scr-программы*.

Таблица ниже поясняет возможные комбинации рассмотренных ролевых отношений.

	<i>scr-операнд с заданным значением</i>	<i>scr-операнд со свободным значением</i>
--	---	---

<i>scr-константа'</i>	Разрешено, может быть опущено	Запрещено
<i>scr-переменная'</i>	Разрешено, значение останется неизменным	Разрешено, значение переменной будет изменено либо потеряно

6.6.6.5.3 Понятие типа *sc*-элемента

тип sc-элемента' — это ролевое отношение, указывающее в рамках *scr*-оператора на *sc*-операнд, для значения которого уточняется тип.

тип sc-элемента' имеет смысл указывать только для *scr*-операндов, помеченных как *scr-операнд со свободным значением'*, тогда данное уточнение типа *sc-элемента* будет использовано для сужения области поиска либо уточнения параметров генерации каких-либо конструкций.

Значением *scr-операндов с заданным значением'* является конкретный, известный на момент начала выполнения *scr-оператора sc-элемент* с конкретным типом, не зависящим от указания *типа sc-элемента'*, в связи с чем использование ролевого отношения *тип sc-элемента'* в данном случае является некорректным.

Допускается использование комбинаций семантически не противоречащих друг другу подмножеств указанного отношения. Например, допускается комбинация *константный sc-элемент'* и *sc-дуга общего вида'*, но не допускается комбинация *sc-узел'* и *sc-дуга'*.

6.6.6.5.3.1 Классификация типов *sc*-элементов

Данное ролевое отношение разбивается на следующие подмножества:

- *sc-узел'*;
- *sc-дуга'*;
- *sc-ребро'*;
- *файл'*.

Ролевое отношение *sc-узел'* разбивается на следующие подмножества:

- *sc-структура'*;
- *sc-отношение'*;

- *sc-класс*'.

Ролевое отношение *sc-дуга*' разбивается на следующие подмножества:

- *sc-дуга общего вида*';
- *sc-дуга принадлежности*'.

Ролевое отношение *sc-дуга принадлежности*' разбивается на следующие подмножества:

- *sc-дуга позитивной принадлежности*';
- *sc-дуга негативной принадлежности*';
- *sc-дуга нечёткой принадлежности*';
- *sc-дуга временной принадлежности*';
- *sc-дуга постоянной принадлежности*'.

Для удобства программирования выделяется также ролевое отношение *sc-дуга основного вида*' являющееся объединением отношений *константный sc-элемент*', *sc-дуга позитивной принадлежности*' и *sc-дуга постоянной принадлежности*'.

Ролевое отношение *формируемое множество*' используется для указания того факта, что в результате выполнения *scr-оператора* должно быть сформировано либо дополнено некоторое *sc-множество sc-элементов*, являющееся значением одного из *scr-операндов* данного *scr-оператора*. При этом если данный *scr-операнд* помечен как *scr-операнд со свободным значением*', то *sc-множество* будет сформировано с нуля (сгенерирован новый *sc-элемент*, обозначающий данное *sc-множество*), в противном случае уже существующее *sc-множество* может быть дополнено. Использование данного ролевого отношения предполагает, что при его отсутствии *sc-множество* бы не формировалось, а значением указанного *scr-операнда* стал бы произвольный *sc-элемент* из данного *sc-множества*.

Ролевое отношение *формируемое множество*' без уточнения порядкового номера используется только в *scr-операторах обработки произвольных конструкций*. Для явного указания номера *scr-операнда*, которому соответствует *формируемое множество*', используются подмножества данного ролевого отношения, аналогичные ролевым отношениям, задающим порядок элементов в кортеже (*1'*, *2'*, *3'* и т. д.), например, *формируемое множество 1'*, *формируемое множество 2'* и т. д. Указанные ролевые отношения

используются только в *scr-операторах* поиска конструкций с формированием множеств.

Ролевое отношение *удаляемый sc-элемент* используется для указания тех *scr-операндов*, значение которых должно быть удалено в процессе выполнения *scr-операторов* удаления. Данным ролевым отношением может быть помечен как *scr-операнд с заданным значением*, так и *scr-операнд со свободным значением*. При этом удаляемым *sc-элементом* может быть как *scr-константа*, так и *scr-переменная* (в случае *scr-переменной* удаляется не только связка принадлежности между этой *scr-переменной* и её значением, но и непосредственно сам *sc-элемент*, являющийся значением).

6.6.6.6 Понятие начального scr-оператора

начальный scr-оператор — это ролевое отношение, которое указывает в рамках декомпозиции соответствующего *scr-программе scr-процесса* те *scr-операторы*, которые должны быть выполнены в первую очередь, то есть те, с которых собственно начинается выполнение *scr-процесса*.

6.6.6.6.1 Классификация scr-операторов

В соответствии с типом описываемых операций *scr-операторы* разбиваются на следующие классы:

- *scr-оператор генерации sc-конструкций*;
- *scr-оператор поиска sc-конструкций*;
- *scr-оператор удаления sc-конструкций*;
- *scr-оператор проверки условий*;
- *scr-оператор обработки содержимых файлов*;
- *scr-оператор управления событиями*;
- *scr-оператор управления scr-процессами*;
- *scr-оператор управления значениями scr-операндов*;
- *scr-оператор управления блокировками*.

6.6.6.6.1.1 scr-операторы генерации sc-конструкций

Данный класс *scr-операторов* разбивается на следующие подклассы:

- *scr-оператор генерации одноэлементной sc-конструкции*;
- *scr-оператор генерации трехэлементной sc-конструкции*;
- *scr-оператор генерации пятиэлементной sc-конструкции*;
- *scr-оператор генерации sc-конструкции по произвольному образцу*.

scp-операторы класса *scp-оператор генерации конструкций* описывают действия генерации каких-либо *sc-конструкций*. В результате выполнения scp-операторов данного класса в памяти системы появляются новые сгенерированные *sc-элементы*. scp-операторы класса не предполагают ветвления программы в зависимости от выполнения дополнительных условий, поэтому указание следующих scp-операторов осуществляется только при помощи отношения *следующий scp-оператор**, без подмножеств.

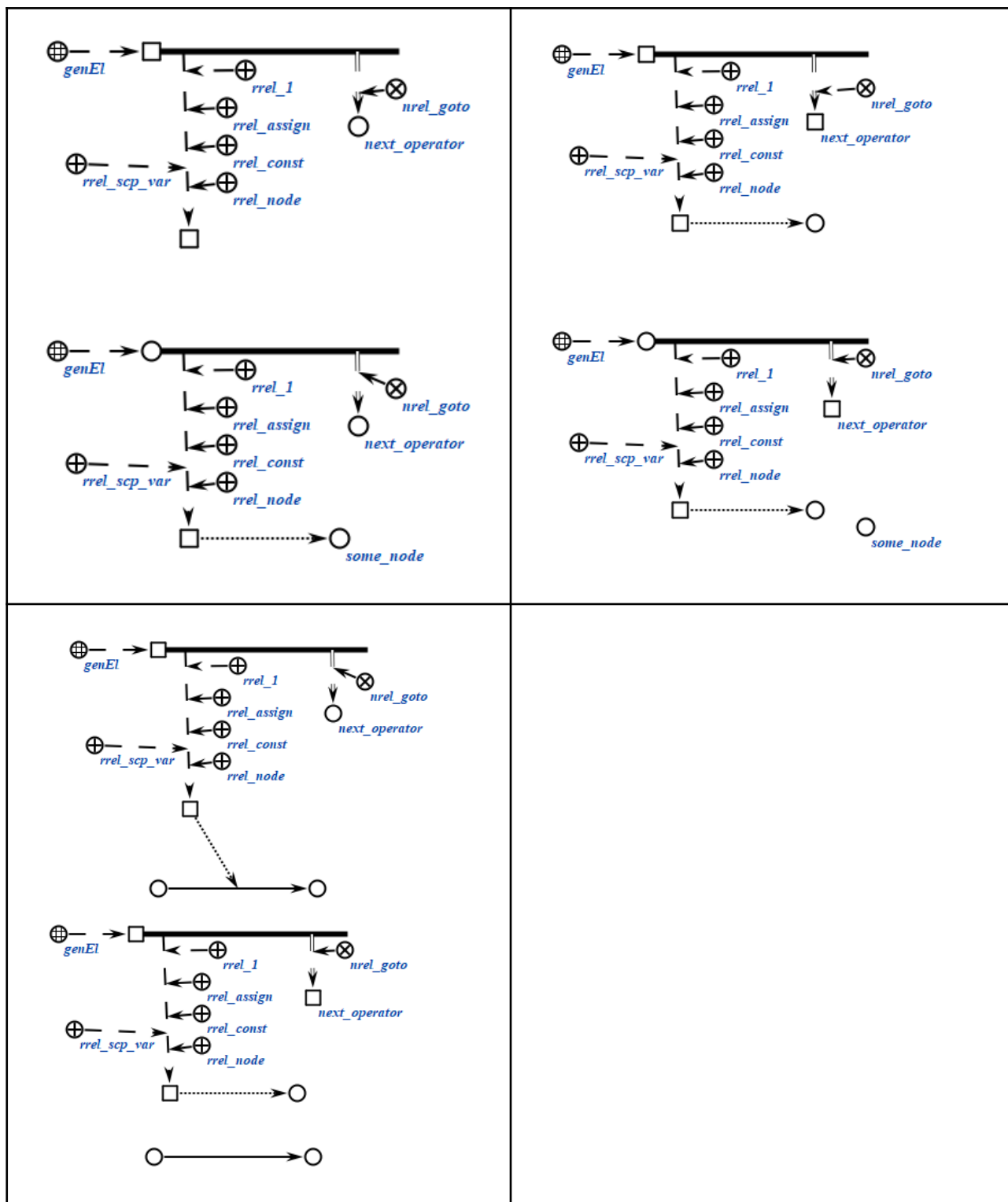
6.6.6.1.1.1 scp-оператор генерации одноэлементной sc-конструкции

scp-операторы класса *scp-оператор генерации одноэлементной sc-конструкции* описывают действие генерации одного *sc-элемента* указанного типа. Каждый scp-оператор данного класса содержит один scp-операнд, обязательно помеченный как *scp-операнд со свободным значением*'. Также может быть уточнен тип генерируемого sc-элемента при помощи подмножеств ролевого отношения тип *sc-элемента*'. В результате выполнения scp-оператора будет сгенерирован новый *sc-элемент* указанного типа.

Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_genEl (*
    <-_ genEl;;
    _-> rrel_1:: rrel_assign:: rrel_const:: rrel_node:: rrel_scp_var:: _elem1;;
    _=> nrel_goto:: scp_operator_example_goto;;
*);;
```

Примеры scp-операторов данного класса на SCg-коде представлены ниже:



6.6.6.6.1.1.2 scp-оператор генерации трехэлементной sc-конструкции

scp-операторы класса *scp-оператор генерации трехэлементной sc-конструкции* описывают действие генерации всех либо некоторых *sc-элементов* указанного типа, составляющих *трехэлементную*

sc-конструкцию. Каждый *scp-оператор* данного класса содержит три *scp-операнда*.

Первый *scp-операнд* может иметь произвольный *тип значения scp-операнда'*, и соответствует первому элементу *трехэлементной sc-конструкции*.

Второй *scp-операнд* может иметь произвольный *тип значения scp-операнда'*, и соответствует второму элементу *трехэлементной sc-конструкции*.

В случае, если данный *scp-операнд* помечен как *scp-операнд со свободным значением'*, то генерируется *sc-коннектор* указанного типа. В случае, если данный *scp-операнд* помечен как *scp-операнд с заданным значением'*, то *sc-коннектор* не генерируется, а будут изменены оба или один из инцидентных ему *sc-элемента*. То есть можно сказать, что будут «переставлены» начало и/или конец данного *sc-коннектора*. При этом физический адрес *sc-коннектора* может измениться, но конфигурация окружающей его семантической сети (входящие и выходящие *sc-коннекторы*, если такие были) будет полностью соответствовать той, что была до выполнения *scp-оператора*.

Третий *scp-операнд* может иметь произвольный *тип значения scp-операнда'*, и соответствует третьему элементу *трехэлементной sc-конструкции*.

В результате выполнения *scp-оператора* для всех *scp-операндов*, помеченных как *scp-операнд со свободным значением'*, будет сгенерировано значение, т.е. новый *sc-элемент* соответствующего типа, с учетом уточнений типа. Тип генерируемого *sc-элемента* может быть уточнен при помощи подмножеств ролевого отношения *тип sc-элемента'*.

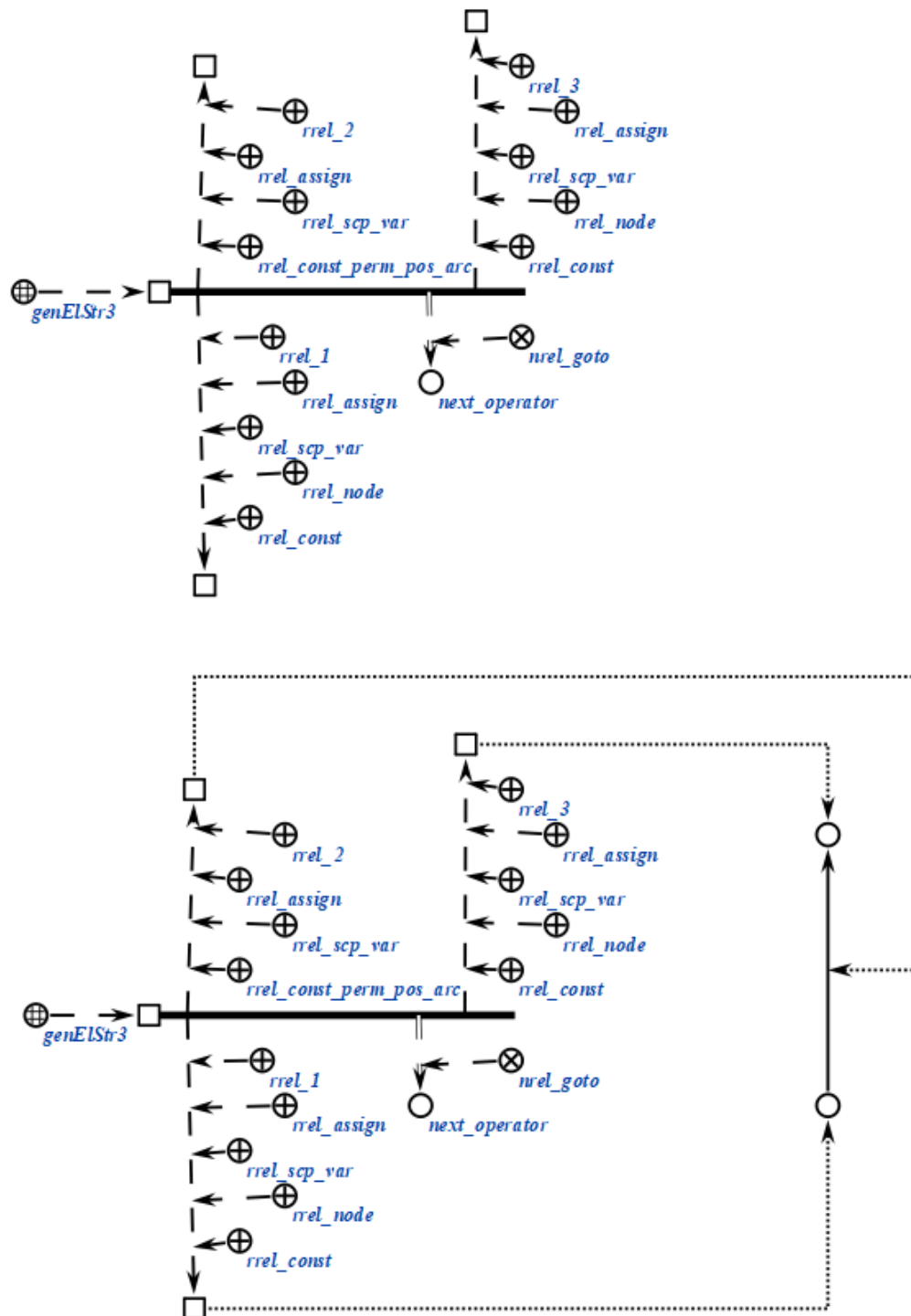
Пример *scp-оператора* данного класса на SCs-коде представлен ниже:

```
scp_operator_example_genElStr3 (*
  <-_ genElStr3;;
  _-> rrel_1:: rrel_fixed:: rrel_scp_var:: _set;;
  _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc::
_arcl;;
  _-> rrel_3:: rrel_assign:: rrel_scp_var:: _element;;

  _=> nrel_goto:: scp_operator_example_goto;;
```


*) ; ;

Пример scp-оператора данного класса на SCg-коде представлен ниже:



6.6.6.6.1.1.3 scp-оператор генерации пятиэлементной sc-конструкции

scp-операторы класса *scp-оператор генерации пятиэлементной sc-конструкции* описывают действие генерации всех либо некоторых

sc-элементов указанного типа, составляющих *пятиэлементную sc-конструкцию*. Каждый *scr-оператор* данного класса содержит пять *scr-операндов*.

Первый *scr-операнд* может иметь произвольный *тип значения scr-операнда'*, и соответствует первому элементу *пятиэлементной sc-конструкции*.

Второй *scr-операнд* может иметь произвольный *тип значения scr-операнда'*, и соответствует второму элементу *пятиэлементной sc-конструкции*.

В случае, если данный *scr-операнд* помечен как *scr-операнд со свободным значением'*, то генерируется *sc-коннектор* указанного типа. В случае, если данный *scr-операнд* помечен как *scr-операнд с заданным значением'*, то *sc-коннектор* не генерируется, а будут изменены оба или один из инцидентных ему *sc-элемента*. То есть можно сказать, что будут «переставлены» начало и/или конец данного *sc-коннектора*. При этом физический адрес *sc-коннектора* может измениться, но конфигурация окружающей его семантической сети (входящие и выходящие *sc-коннекторы*, если такие были) будет полностью соответствовать той, что была до выполнения *scr-оператора*.

Третий *scr-операнд* может иметь произвольный *тип значения scr-операнда'*, и соответствует третьему элементу *пятиэлементной конструкции*.

Четвертый *scr-операнд* может иметь произвольный *тип значения scr-операнда'*, и соответствует четвертому элементу *пятиэлементной sc-конструкции*. Результаты выполнения *scr-оператора* в зависимости от *типа значения scr-операнда'* определяются так же, как в случае со вторым *scr-операндом*.

Пятый *scr-операнд* может иметь произвольный *тип значения scr-операнда'*, и соответствует пятому элементу *пятиэлементной sc-конструкции*.

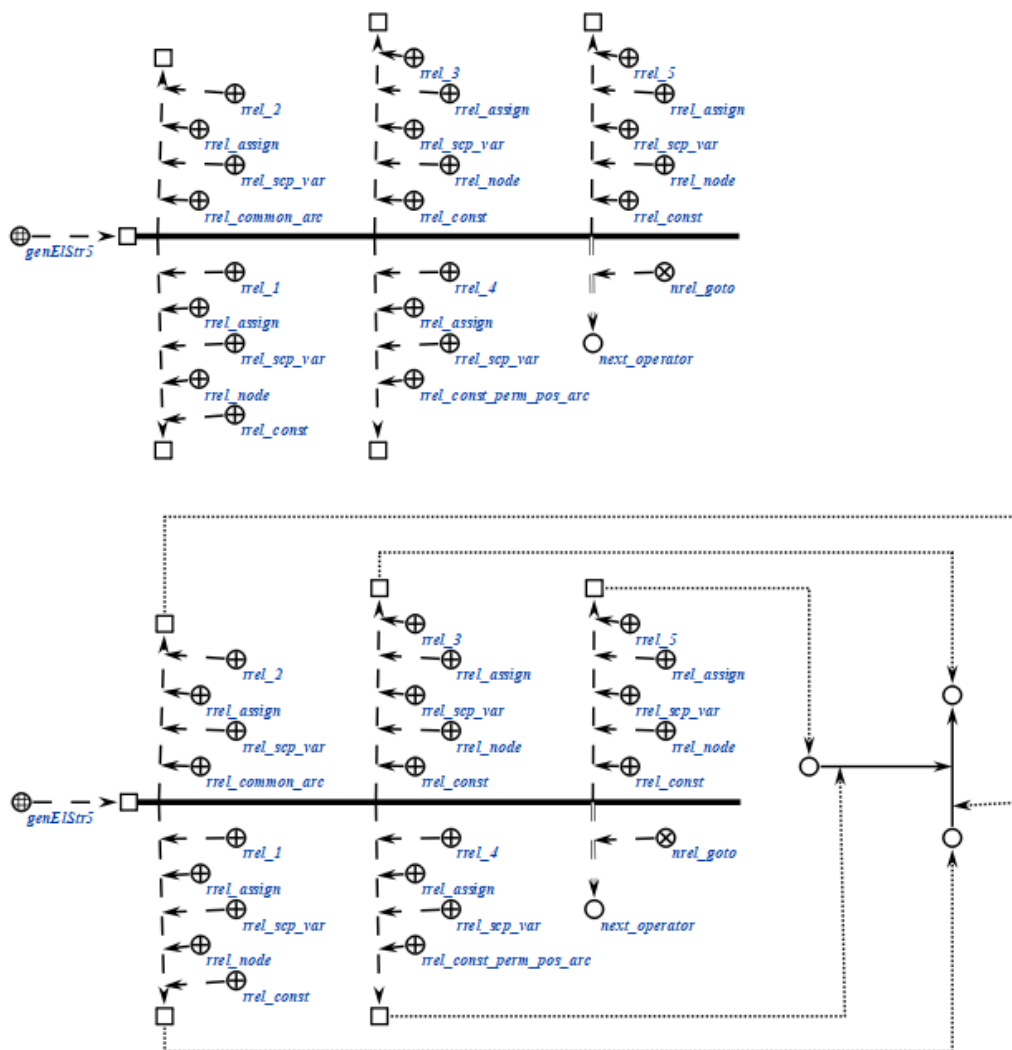
В результате выполнения *scr-оператора* для всех *scr-операндов*, помеченных как *scr-операнд со свободным значением'*, будет сгенерировано значение, т.е. новый *sc-элемент* соответствующего типа, с учетом уточнений типа. Тип генерируемого *sc-элемента* может быть уточнен при помощи подмножеств ролевого отношения *тип sc-элемента'*.

Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_genElStr5 (*
    <_ genElStr5;;
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: rrel_node:: rrel_const::
_node1;;
    _-> rrel_2:: rrel_assign:: rrel_const:: rrel_common_arc:: rrel_scp_var::
_arc1;;
    _-> rrel_3:: rrel_assign:: rrel_scp_var:: rrel_node:: rrel_const::
_node2;;
    _-> rrel_4:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc::
_arc2;;
    _-> rrel_5:: rrel_assign:: rrel_scp_var:: rrel_node:: rrel_const::
_node3;;

    _=> nrel_goto:: scp_operator_example_goto;;
*);;
```

Пример scp-оператора данного класса на SCg-коде представлен ниже:

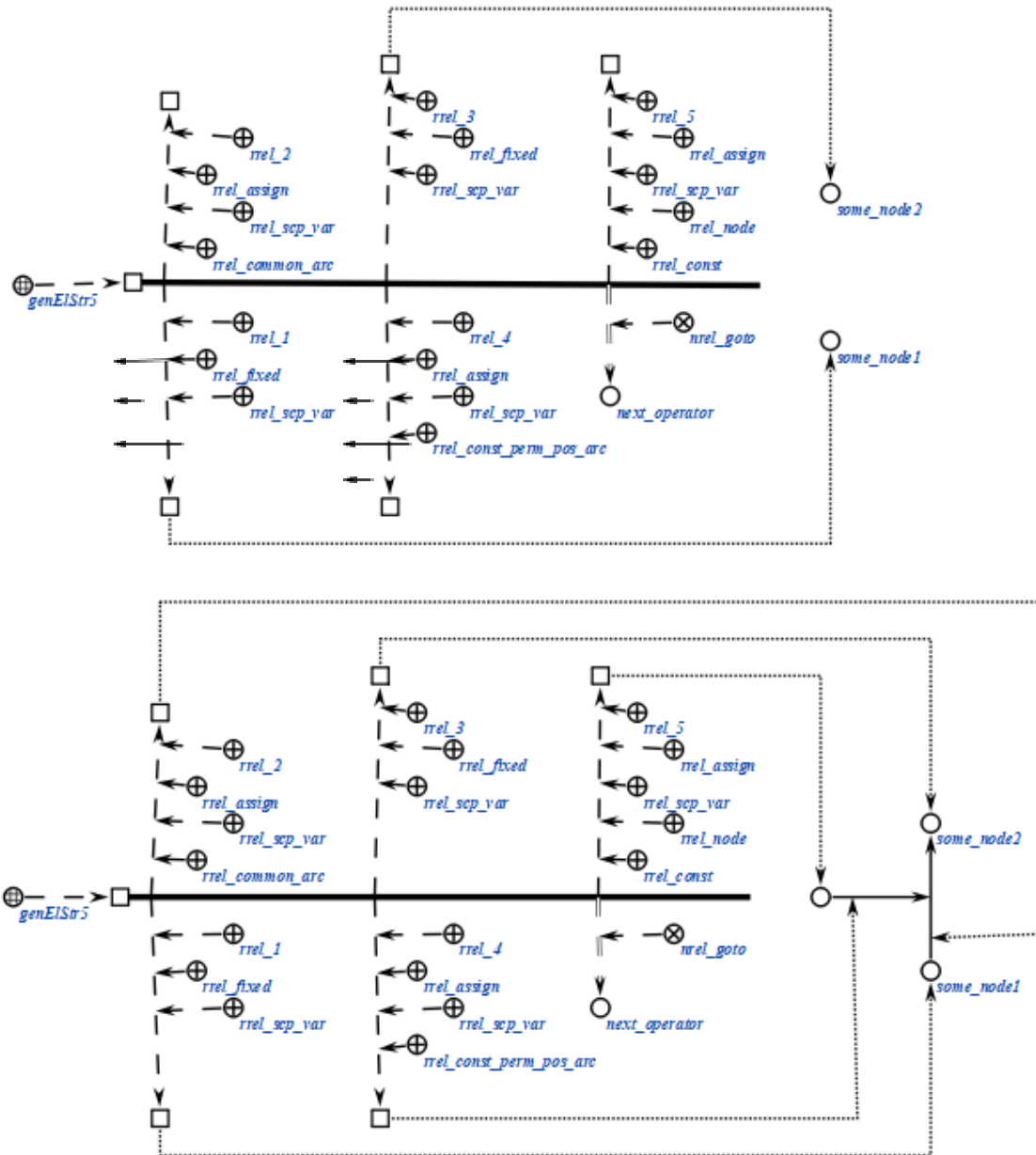


Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_genElStr5 (*
    <_ genElStr5;;
    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: _node1;;
    _-> rrel_2:: rrel_assign:: rrel_const:: rrel_common_arc:: rrel_scp_var::
    _arc1;;
    _-> rrel_3:: rrel_fixed:: rrel_scp_var:: _node2;;
    _-> rrel_4:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc::
    _arc2;;
    _-> rrel_5:: rrel_assign:: rrel_scp_var:: rrel_node:: rrel_const::
    _node3;;

    _=> nrel_goto:: scp_operator_example_goto;;
*);;
```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_genElStr5 (*)

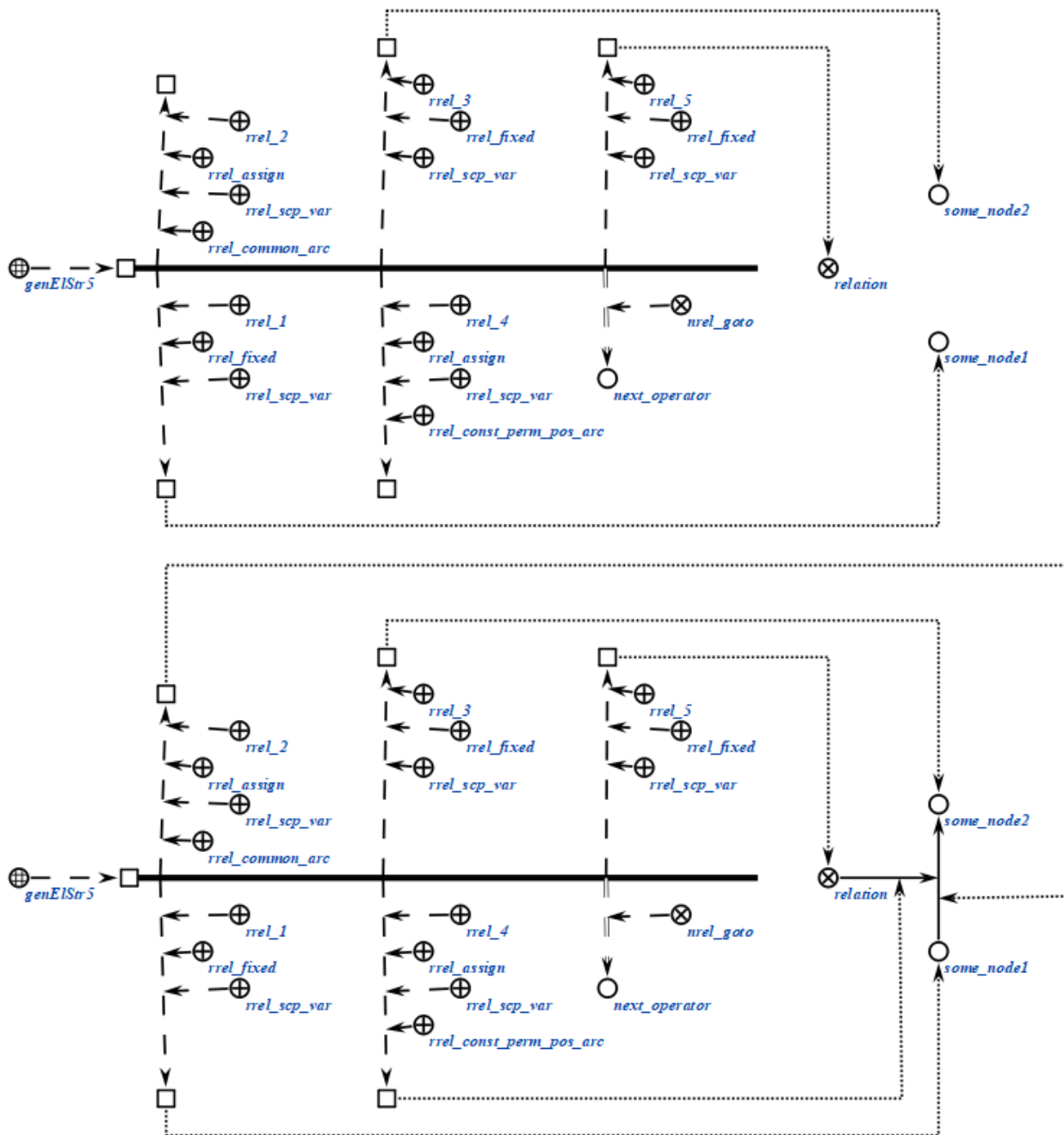
<_ genElStr5;;
-> rrel_1:: rrel_fixed:: rrel_sep_var:: _node1;;
-> rrel_2:: rrel_assign:: rrel_common_arc:: rrel_sep_var:: _arc1;;
-> rrel_3:: rrel_fixed:: rrel_sep_var:: _node2;;
-> rrel_4:: rrel_assign:: rrel_sep_var:: rrel_const_perm_pos_arc::
_arc2;;
-> rrel_5:: rrel_fixed:: rrel_sep_var:: _node3;;
```

```

=> nrel_goto:: scp_operator_example_goto;;
*);;

```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



6.6.6.6.1.1.4 scp-оператор генерации sc-конструкции по произвольному образцу

scp-операторы класса *scp-оператор генерации sc-конструкции по произвольному образцу* описывают действие генерации *sc-элементов* указанного типа, составляющих *sc-конструкцию* произвольного вида. Данный scp-оператор

может применяться и для генерации стандартных *sc-конструкций*, однако его использование в таком случае нецелесообразно, поскольку для этого имеются специальные *scr-операторы*. Каждый *scr-оператор* данного класса содержит четыре *scr-операнда*.

Первый *scr-операнд* должен быть помечен как *scr-операнд с заданным значением'*. Значением данного *scr-операнда* является *sc-узел*, обозначающий шаблон генерации, содержащий хотя бы один *переменный sc-элемент*. Константные *sc-элементы* не могут быть *sc-коннекторами*, поскольку в таком случае нарушается семантика самого понятия *sc-коннектора*, гласящая, что *sc-коннектор* имеет в один момент времени ровно два инцидентных ему *sc-элемента*.

Второй *scr-операнд* может иметь произвольный *тип значения scr-операнда'*. Значением данного *scr-оператора* является знак *sc-множества*, представляющего собой результат генерации, то есть *sc-множество* ориентированных *sc-пар* соответствия всех *sc-переменных* из *sc-шаблона* генерации их сгенерированным значениям (если значения не были заданы в третьем *scr-операнде*). В случае, если данный *scr-операнд* помечен как *scr-операнд с заданным значением'*, то необходимо явно указать пары соответствия, в которых под атрибутом *1'* указывается *переменный sc-элемент* из шаблона генерации (помеченный, как *scr-константа'*), под атрибутом *2'* - *scr-переменные'*, в значения которых будут записаны сгенерированные *sc-элементы*, соответствующие указанным переменным *sc-элементам* из шаблона генерации (помеченные, как *scr-переменная'* и *scr-операнд со свободным значением'*). При этом *sc-пары* соответствия для остальных *sc-переменных* из *sc-шаблона* не будут явно формироваться в *sc-памяти*. Указанное *sc-множество* может быть пустым, тогда *sc-пары* соответствия не будут формироваться вообще, но *sc-конструкция*, соответствующая *sc-шаблону* генерации будет сгенерирована в любом случае. В случае, если данный *scr-операнд* помечен как *scr-операнд со свободным значением'*, то будут сформированы *sc-пары* соответствия сгенерированным значениям для всех *sc-переменных* из *sc-шаблона* генерации. Значением *scr-операнда* станет знак *sc-множества* указанных *sc-пар*.

Третий *scr-операнд* должен быть помечен как *scr-операнд с заданным значением'*. Значением данного *scr-оператора* является знак *sc-множества* параметров генерации, то есть ориентированных *sc-пар* соответствия некоторых *sc-переменных* из *sc-шаблона* генерации их значениям, в случае, если значения

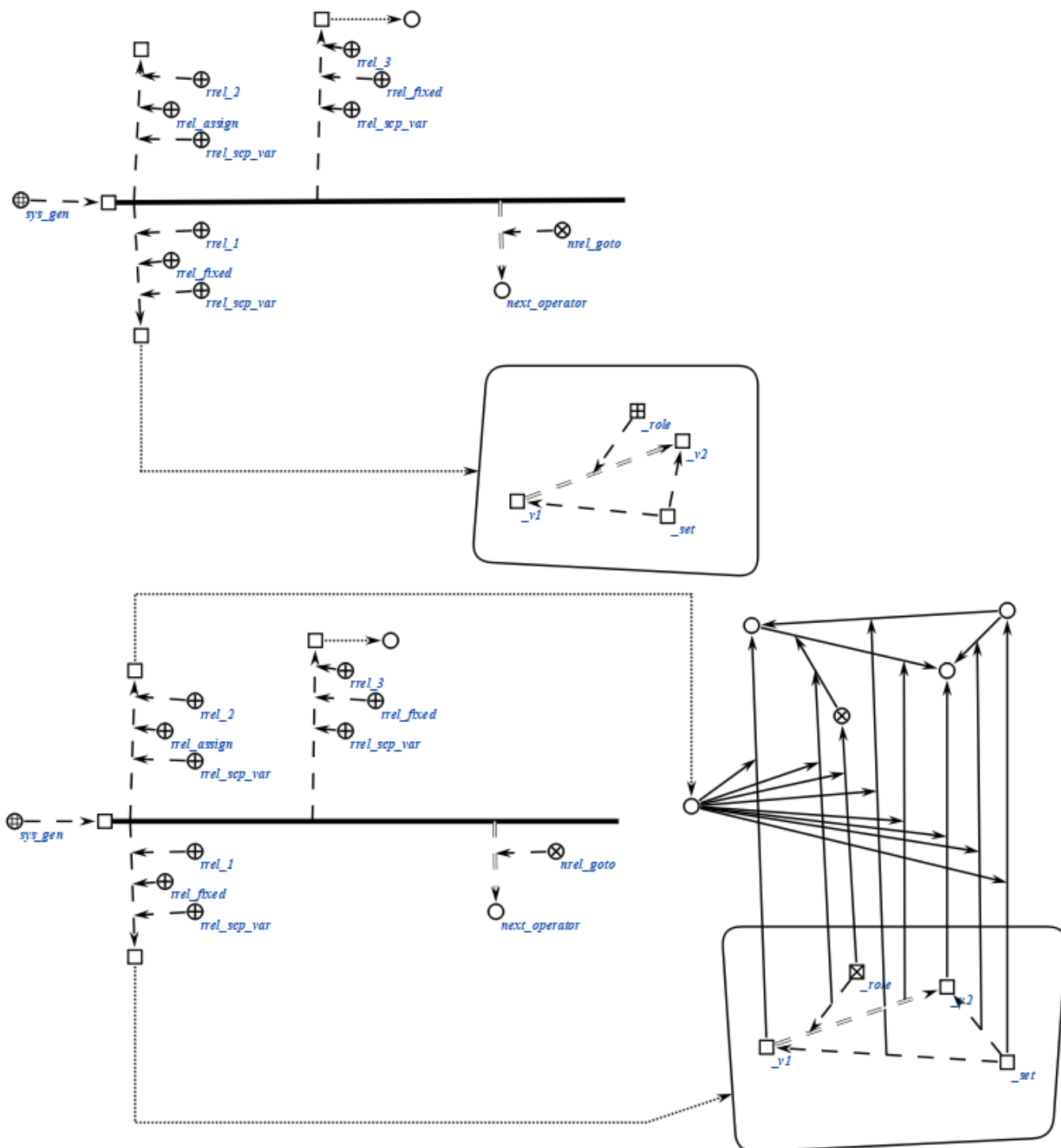
заранее известны. При этом каждый из элементов sc-пар должен быть помечен, как *scp-операнд с заданным значением*', так же при необходимости должен быть указан *тип scp-операнда*'. Указанное sc-множество может быть пустым.

Четвертый scp-операнд может иметь произвольный *тип значения scp-операнда*'. Данный scp-операнд является необязательным и может быть опущен. Значением данного scp-операнда является знак sc-множества всех *sc-элементов*, сгенерированных в результате выполнения данного *scp-оператора*. В случае, если scp-операнд помечен, как *scp-операнд с заданным значением*', то будет дополнено множество, являющееся значением данного scp-операнда, если как *scp-операнд со свободным значением*', то множество будет сгенерировано и сформировано с нуля.

Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_sys_gen (*  
  
    <_ sys_gen;;  
    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: _el1;;  
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: _el2;;  
    _-> rrel_3:: rrel_fixed:: rrel_scp_var:: _el3;;  
    _=> nrel_goto:: next_operator;;  
*);;
```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_sys_gen (*)

<_ sys_gen;;
-> rrel_1:: rrel_fixed:: rrel_scp_var:: _el1;;
-> rrel_2:: rrel_fixed:: rrel_scp_var:: _el2;;
-> rrel_3:: rrel_fixed:: rrel_scp_var:: _el3;;
```

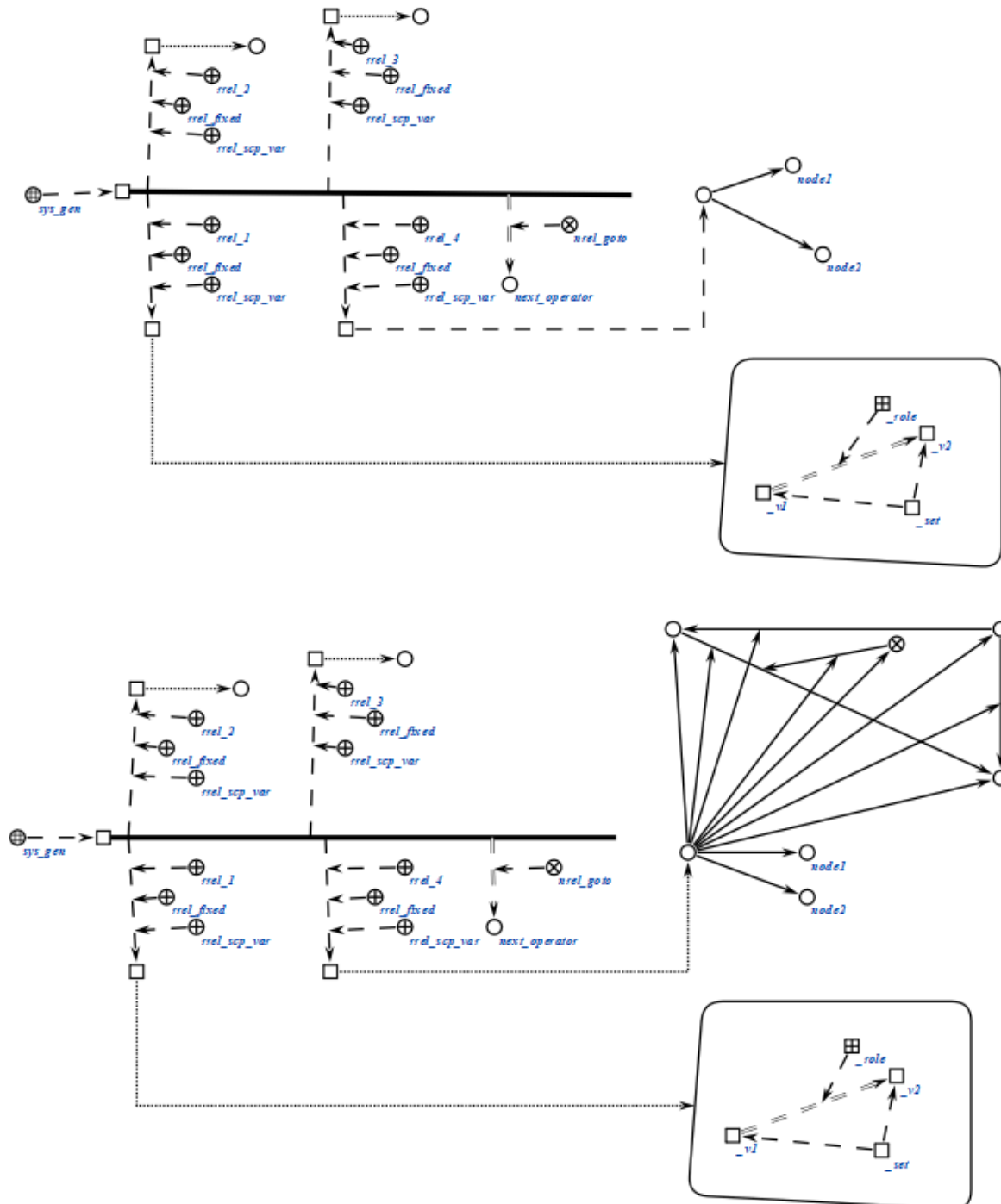
```

-> rrel_4:: rrel_fixed:: rrel_scp_var:: _el4;;
=> nrel_goto:: next_operator;;

*);;

```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



Пример scp-оператора данного класса на SCs-коде представлен ниже:

```

scp_operator_example_sys_gen (*
  <-_ sys_gen;;

```

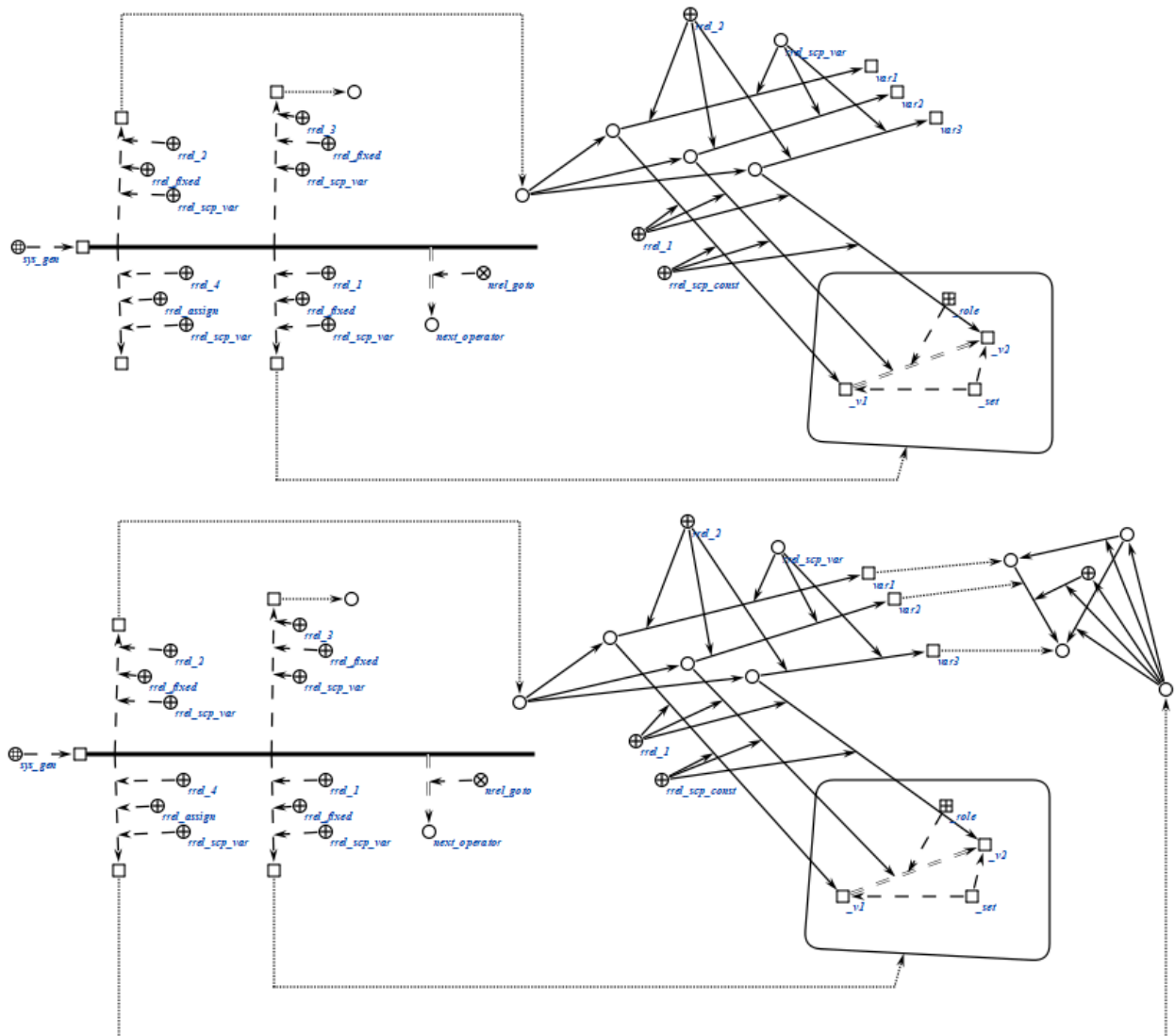
```

-> rrel_1:: rrel_fixed:: rrel_scp_var:: _el1;;
-> rrel_2:: rrel_fixed:: rrel_scp_var:: _el2;;
-> rrel_3:: rrel_fixed:: rrel_scp_var:: _el3;;
-> rrel_4:: rrel_assign:: rrel_scp_var:: _el4;;
=> nrel_goto:: next_operator;;

```

*);;

Пример scp-оператора данного класса на SCg-коде представлен ниже:



Пример scp-оператора данного класса на SCs-коде представлен ниже:

```

scp_operator_example_sys_gen (*)

<-_ sys_gen;;

-> rrel_1:: rrel_fixed:: rrel_scp_var:: _el1;;
-> rrel_2:: rrel_assign:: rrel_scp_var:: _el2;;

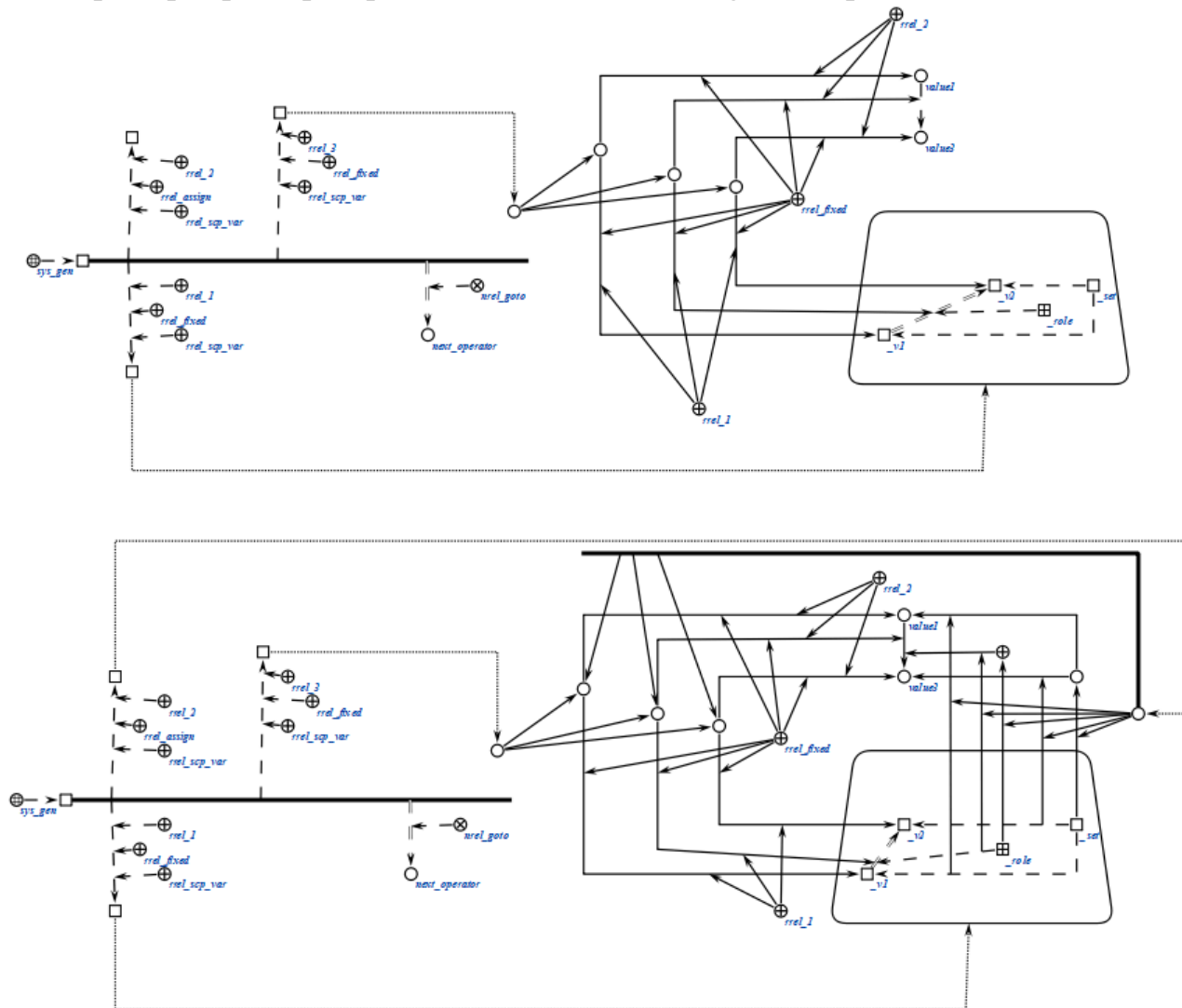
```

```

-> rrel_3:: rrel_fixed:: rrel_scp_var:: _el3;;
=> nrel_goto:: next_operator;;
*);;

```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



6.6.6.6.1.2 scp-операторы поиска sc-конструкций

scp-операторы класса *scp-оператор поиска sc-конструкций* описывают действие поиска *sc-элементов*, удовлетворяющих некоторому заданному образцу.

Образец может представлять собой как одну из *sc-конструкций стандартного вида*, так и *sc-конструкцию нестандартного вида*. scp-операторы класса предполагают поиск только по знаниям, явно представленным в базе знаний *ostis-системы*, и не предполагают дополнительных действий, таких как, например, логический вывод,

позволяющий получить дополнительные сведения на основе имеющихся знаний.

scr-операторы класса предполагают ветвление программы в зависимости от того, оказалась ли успешной попытка поиска. Указание следующих scr-операторов может осуществляться при помощи подмножеств отношения *следующий scr-оператор**. В случае, если программист заранее уверен в успешности поиска, может быть использовано само по себе отношение *следующий scr-оператор**.

Данный класс *scr-операторов* разбивается на следующие подклассы:

- *scr-оператор поиска трехэлементной sc-конструкции;*
- *scr-оператор поиска пятиэлементной sc-конструкции;*
- *scr-оператор поиска трехэлементной sc-конструкции с формированием множеств;*
- *scr-оператор поиска пятиэлементной sc-конструкции с формированием множеств;*
- *scr-оператор поиска sc-конструкции по произвольному образцу.*

6.6.6.6.1.2.1 scr-оператор поиска трехэлементной sc-конструкции

scr-операторы класса *scr-оператор поиска трехэлементной sc-конструкции* описывают действие поиска некоторых из *sc-элементов*, составляющих *трехэлементную sc-конструкцию*, а также проверки инцидентности заданных *sc-элементов*. Каждый scr-оператор данного класса содержит три scr-операнда.

Первый, второй и третий scr-операнды соответствуют первому, второму и третьему *sc-элементам трехэлементной sc-конструкции*.

Каждый из scr-операндов может иметь произвольный *тип значения scr-операнда'*. При выполнении scr-оператора осуществляется попытка поиска *sc-элементов*, соответствующих scr-операндам, помеченным, как *scr-операнд со свободным значением'*. Однако запрещена ситуация, когда все scr-операнды помечены, как *scr-операнд со свободным значением'*.

В случае, если scr-операнд, соответствующий *sc-коннектору*, помечен как *scr-операнд с заданным значением'*, а scr-операнды, соответствующие инцидентным *sc-коннектору* элементам *трехэлементной sc-конструкции* помечены как *scr-операнд со свободным значением'*, то значениями данных

scp-операндов станут соответствующие *sc-элементы*, инцидентные заданному *sc-коннектору*. Таким образом можно, например, найти начальный и конечный элементы *sc-дуги*.

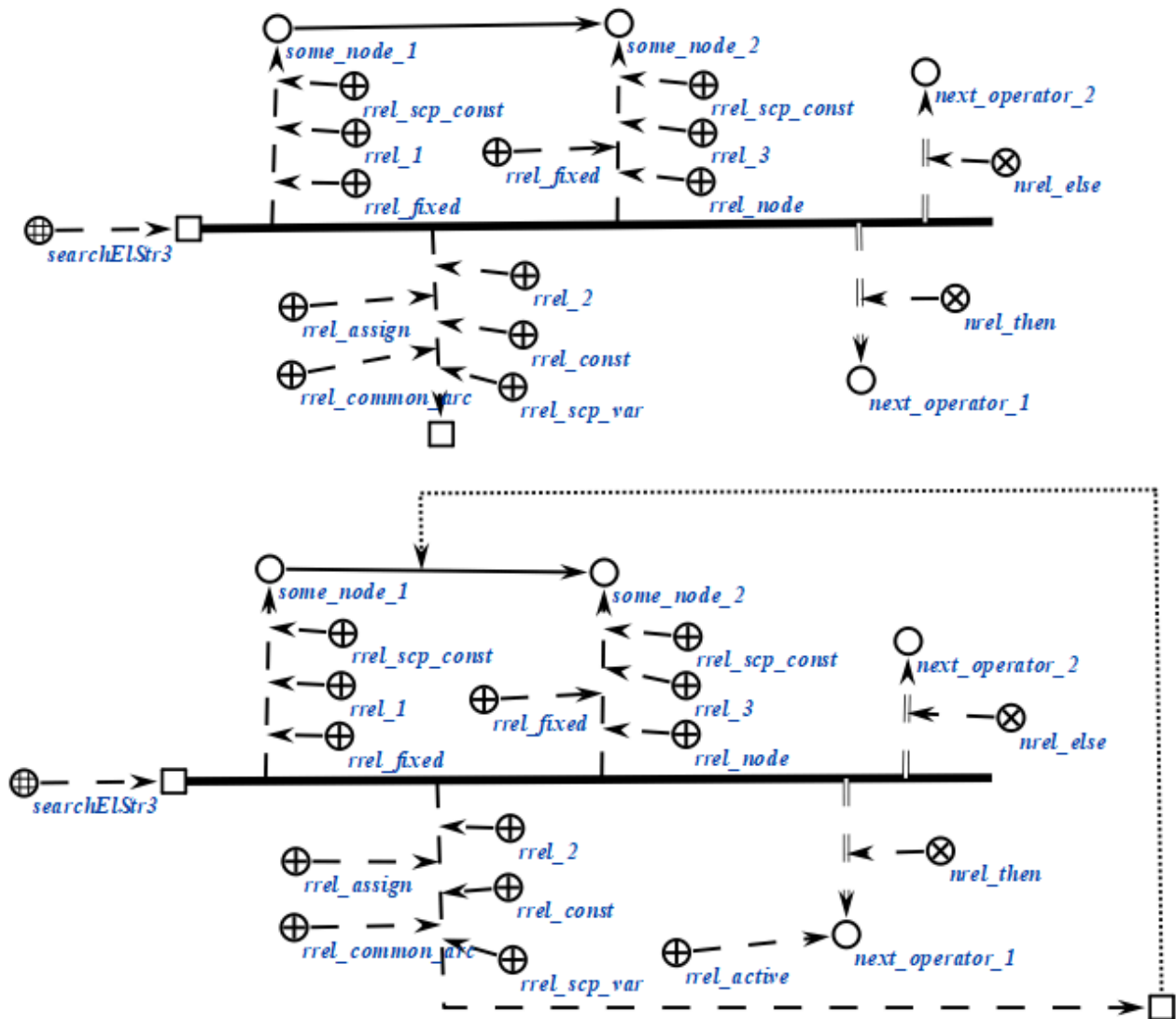
В случае, если scp-операнд, соответствующий *sc-коннектору*, помечен как *scp-операнд с заданным значением'*, и scp-операнды, соответствующие инцидентным *sc-коннектору* элементам *трехэлементной sc-конструкции* также помечены как *scp-операнд с заданным значением'*, то будет осуществлена проверка, действительно ли известный *sc-коннектор* и *sc-элемент (sc-элементы)* инцидентны указанным образом. Значения соответствующих scp-операндов изменены не будут, а переход к следующему scp-оператору будет осуществлен по одному из отношений *следующий scp-оператор при успешном выполнении** или *следующий scp-оператор при безуспешном выполнении** в зависимости от успешности указанной проверки. Таким образом можно, например, проверить, являются ли найденные ранее *sc-элементы* началом и концом заданной *sc-дуги*.

Важно заметить, что в случае, если заданному критерию поиска соответствуют несколько возможных результатов, то будет выбран один произвольный вариант.

Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_searchElStr3 (*  
  
    <_ searchElStr3;;  
    _-> rrel_1:: rrel_fixed:: rrel_scp_const:: some_node_1;;  
    _-> rrel_2:: rrel_assign:: rrel_common_arc:: rrel_const:: rrel_scp_var::  
_arc;;  
    _-> rrel_3:: rrel_fixed:: rrel_scp_const:: rrel_node:: some_node_2;;  
    _=> nrel_then:: next_operator_1;;  
    _=> nrel_else:: next_operator_2;;  
*);;
```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



6.6.6.6.1.2.2 scp-оператор поиска пятиэлементной sc-конструкции

scp-операторы класса *scp-оператор поиска пятиэлементной sc-конструкции* описывают действие поиска некоторых из *sc-элементов*, составляющих *пятиэлементную sc-конструкцию*, а также проверки инцидентности заданных *sc-элементов*. Каждый scp-оператор данного класса содержит пять scp-операндов.

Первый, второй, третий, четвертый и пятый scp-операнды соответствуют первому, второму, третьему, четвертому и пятому *sc-элементам пятиэлементной sc-конструкции*.

Каждый из scp-операндов может иметь произвольный *тип значения scp-операнда*'. При выполнении scp-оператора осуществляется попытка поиска *sc-элементов*, соответствующих scp-операндам, помеченным, как *scp-операнд*

со свободным значением'. Однако запрещена ситуация, когда все scp-операнды помечены, как *scp-операнд со свободным значением'*.

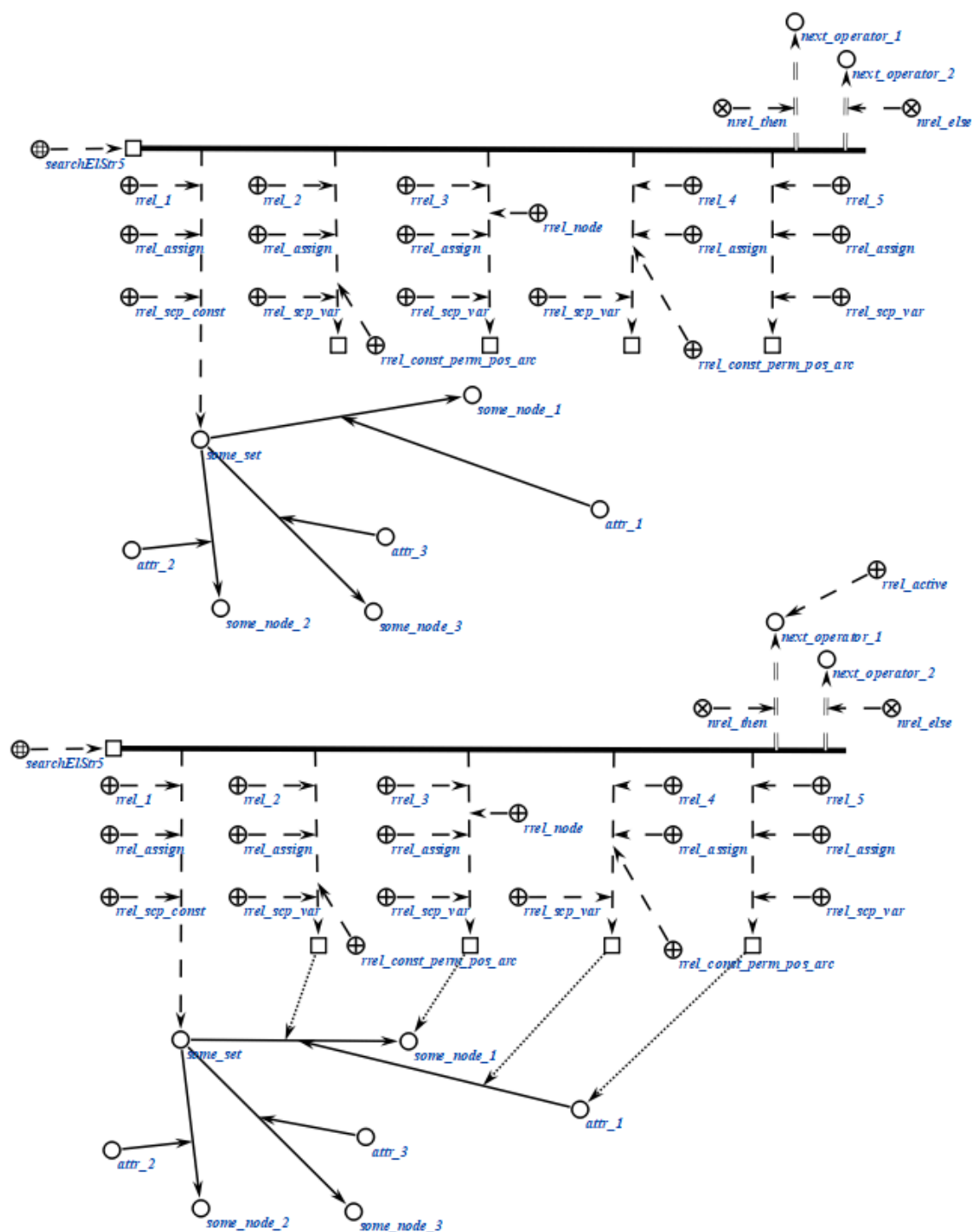
Различные варианты выполнения scp-операторов данного класса в зависимости от комбинации *типов значений scp-операндов'* аналогичны вариантам выполнения *scp-оператор поиска трехэлементной sc-конструкции*.

Важно заметить, что в случае, если заданному критерию поиска соответствуют несколько возможных результатов, то будет выбран один произвольный вариант.

Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_searchElStr5 (*  
  
  <_ searchElStr5;;  
  _-> rrel_1:: rrel_assign:: rrel_scp_const:: some_set;;  
  _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: _arc;;  
  _-> rrel_3:: rrel_assign:: rrel_scp_var:: _el3;;  
  _-> rrel_4:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc::  
_arc2;;  
  _-> rrel_5:: rrel_assign:: rrel_scp_var:: _el5;;  
  _=> nrel_then:: next_operator_1;;  
  _=> nrel_else:: next_operator_2;;  
*);;
```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_searchElStr5 (*
  <-_ searchElStr5;;
  _-> rrel_1:: rrel_assign:: rrel_scp_var:: _el1;;
  _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: _arc;;
  _-> rrel_3:: rrel_fixed:: rrel_scp_var:: _el3;;
```

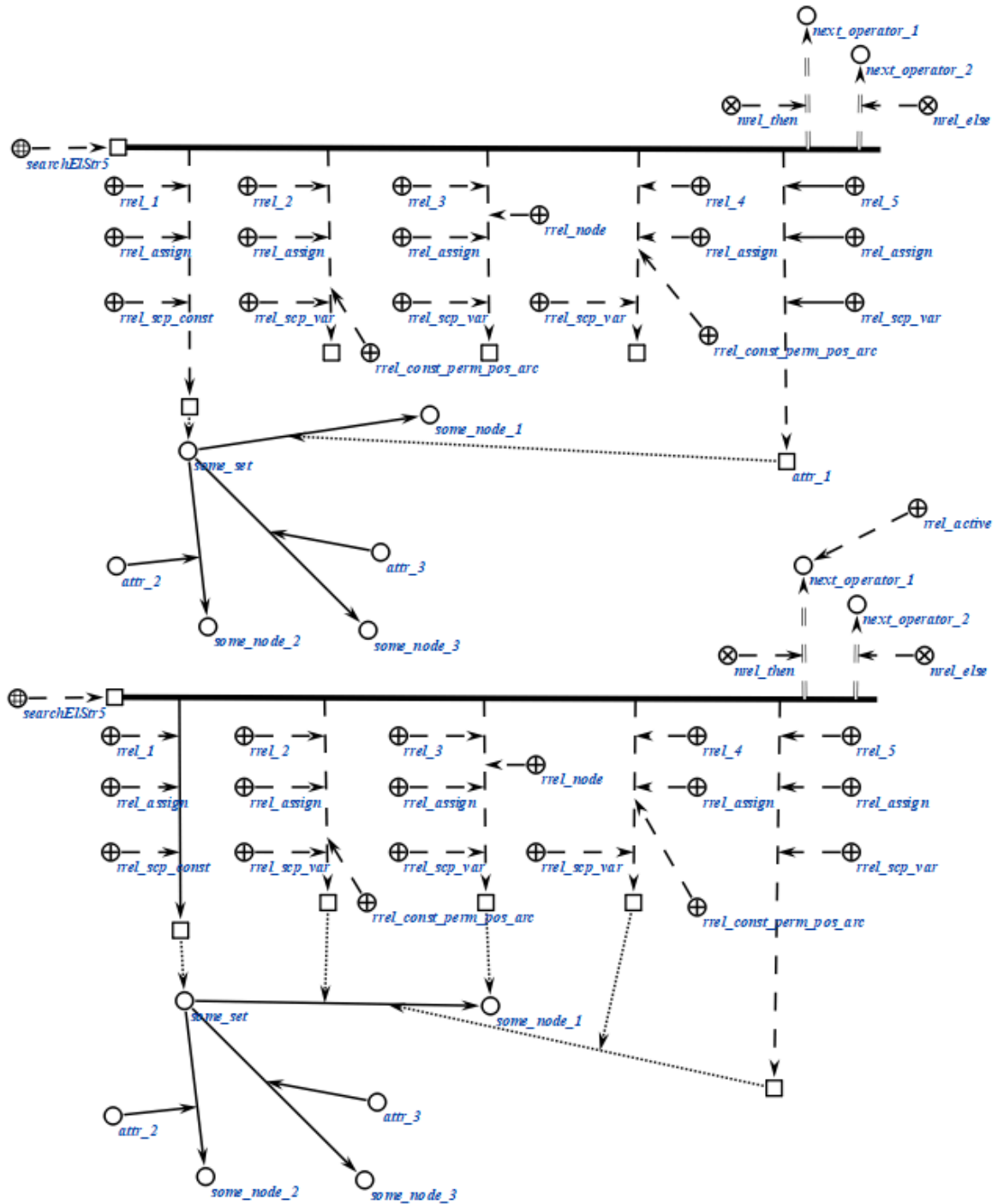
```

-> rrel_4:: rrel_fixed:: rrel_scp_var:: _arc2;;
-> rrel_5:: rrel_fixed:: rrel_scp_const:: attr_1;;
=> nrel_then:: next_operator_1;;
=> nrel_else:: next_operator_2;;

```

*) ;;

Пример scp-оператора данного класса на SCg-коде представлен ниже:



6.6.6.6.1.2.3 scp-оператор поиска трехэлементной sc-конструкции с формированием множества

scp-операторы класса *scp-оператор поиска трехэлементной sc-конструкции с формированием множества* описывают действие формирования множества *sc-элементов*, соответствующих указанным элементам *трехэлементной sc-конструкции*. Каждый scp-оператор данного класса содержит четыре или пять scp-операндов.

Первый, второй и третий scp-операнды соответствуют первому, второму и третьему *sc-элементам трехэлементной sc-конструкции*. Указанные sco-операнды в данном случае можно назвать основными.

Первый и третий scp-операнд могут иметь произвольный *тип значения scp-операнда'*, но как минимум один из них должен быть помечен как *scp-операнд с заданным значением'*. Второй scp-операнд должен быть помечен как *scp-операнд со свободным значением'*.

Помимо трех основных scp-операндов каждый scp-оператор данного класса содержит один или два дополнительных scp-операнда, значениями которых являются *sc-множества*, содержащие все *sc-элементы*, которые могли бы стать значениями соответствующего основного scp-операнда при заданных условиях поиска. При этом множество возможных значений scp-операнда, помеченного ролевым отношением *I'*, соответствует *sc-множеству*, которое является значением scp-операнда, помеченного ролевым отношением *формируемое sc-множество I'* и т.д. Если scp-операнд, соответствующий такому *sc-множеству*, помечен как *scp-операнд со свободным значением'*, то *sc-множество* будет сформировано заново (сгенерирован новый *sc-элемент*), в противном случае будут добавлены *sc-элементы* в *sc-множество*, являющееся значением данного *sc-операнда*.

В качестве примера применения данного класса *scp-операторов* можно рассматривать задачу копирования множества (т.е. создание *sc-множества*, содержащего те же *sc-элементы*). Также scp-операторы класса применяются для организации циклических переборов каких-либо элементов *sc-множества*.

Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_searchSetStr3 (*  
  
    <-_ searchSetStr3;;
```

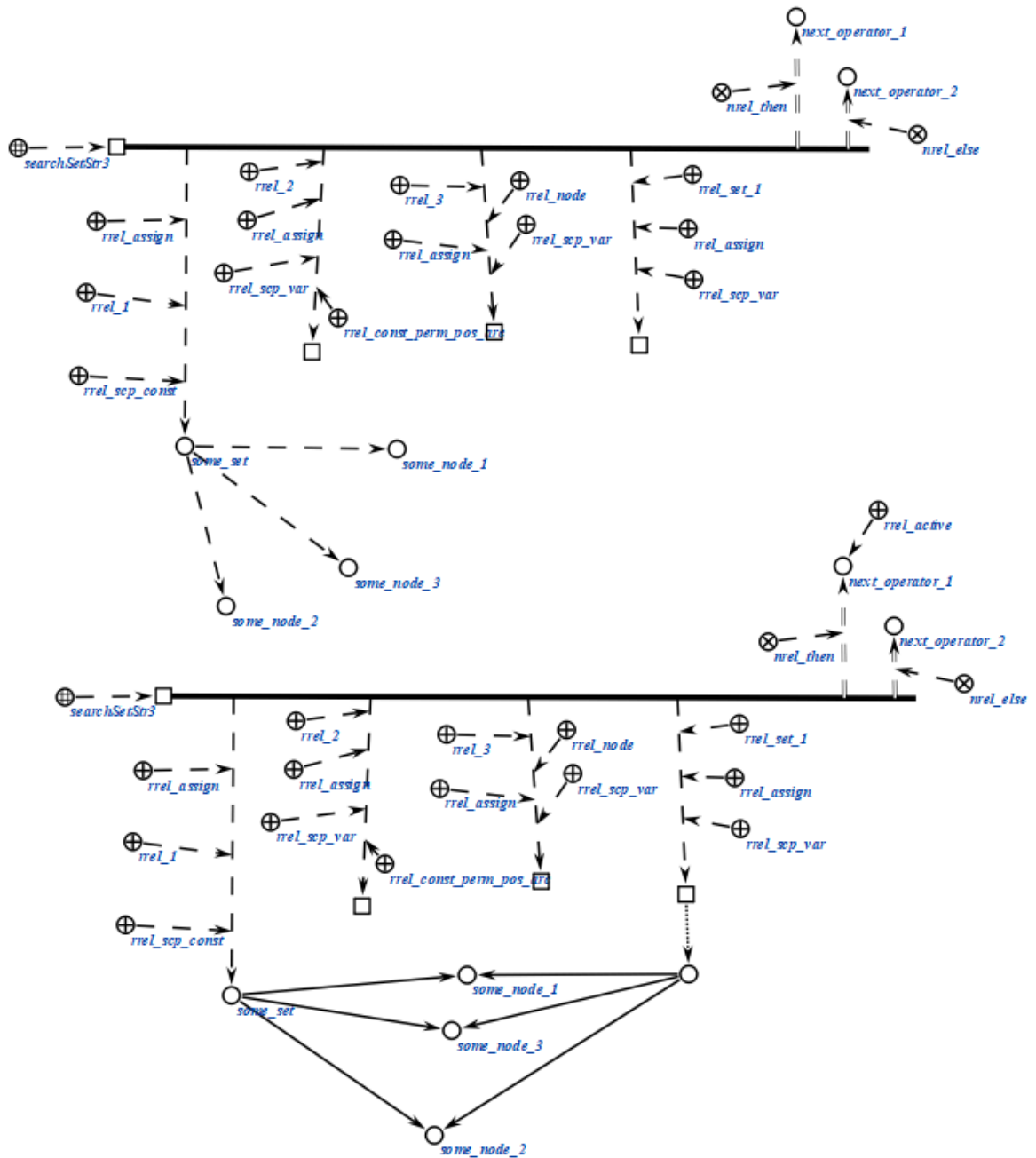
```

-> rrel_1:: rrel_fixed:: rrel_scp_const:: some_set;;
-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: _arc;;
-> rrel_3:: rrel_assign:: rrel_node:: rrel_scp_var:: _el3;;
-> rrel_set_3:: rrel_assign:: rrel_scp_var:: _set3;;
=> nrel_then:: next_operator_1;;
=> nrel_else:: next_operator_2;;

*);;

```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



6.6.6.6.1.2.4 scp-оператор поиска пятиэлементной sc-конструкции с формированием множества

scp-операторы класса *scp-оператор поиска пятиэлементной sc-конструкции с формированием множества* описывают действие формирования sc-множества *sc-элементов*, соответствующих указанным элементам *пятиэлементной sc-конструкции*. Каждый scp-оператор данного класса содержит от шести до десяти scp-операндов.

Первый, второй, третий, четвертый и пятый scp-операнды соответствуют первому, второму, третьему, четвертому и пятому *sc-элементам пятиэлементной sc-конструкции*. Указанные scp-операнды в данном случае можно назвать основными.

Первый, третий и пятый scp-операнд могут иметь произвольный *тип значения scp-операнда'*, но как минимум один из них должен быть помечен как *scp-операнд с заданным значением'*. Второй и четвертый scp-операнд должны быть помечены как *scp-операнд со свободным значением'*.

Помимо пяти основных scp-операндов каждый scp-оператор данного класса содержит от одного до четырех дополнительных scp-операндов, значениями которых являются множества, содержащие все *sc-элементы*, которые могли бы стать значениями соответствующего основного scp-операнда при заданных условиях поиска. При этом множество возможных значений scp-операнда, помеченного ролевым отношением *I'*, соответствует *sc-множеству*, которое является значением scp-операнда, помеченного ролевым отношением *формируемое sc-множество I'* и т.д. Если scp-операнд, соответствующий такому множеству, помечен как *scp-операнд со свободным значением'*, то *sc-множество* будет сформировано заново (сгенерирован новый *sc-элемент*), в противном случае будут добавлены *sc-элементы* в *sc-множество*, являющееся значением данного scp-операнда.

scp-операторы класса применяются в тех же случаях, что и *scp-операторы поиска трехэлементной sc-конструкции с формированием множеств*.

Пример scp-оператора данного класса на SCs-коде представлен ниже:

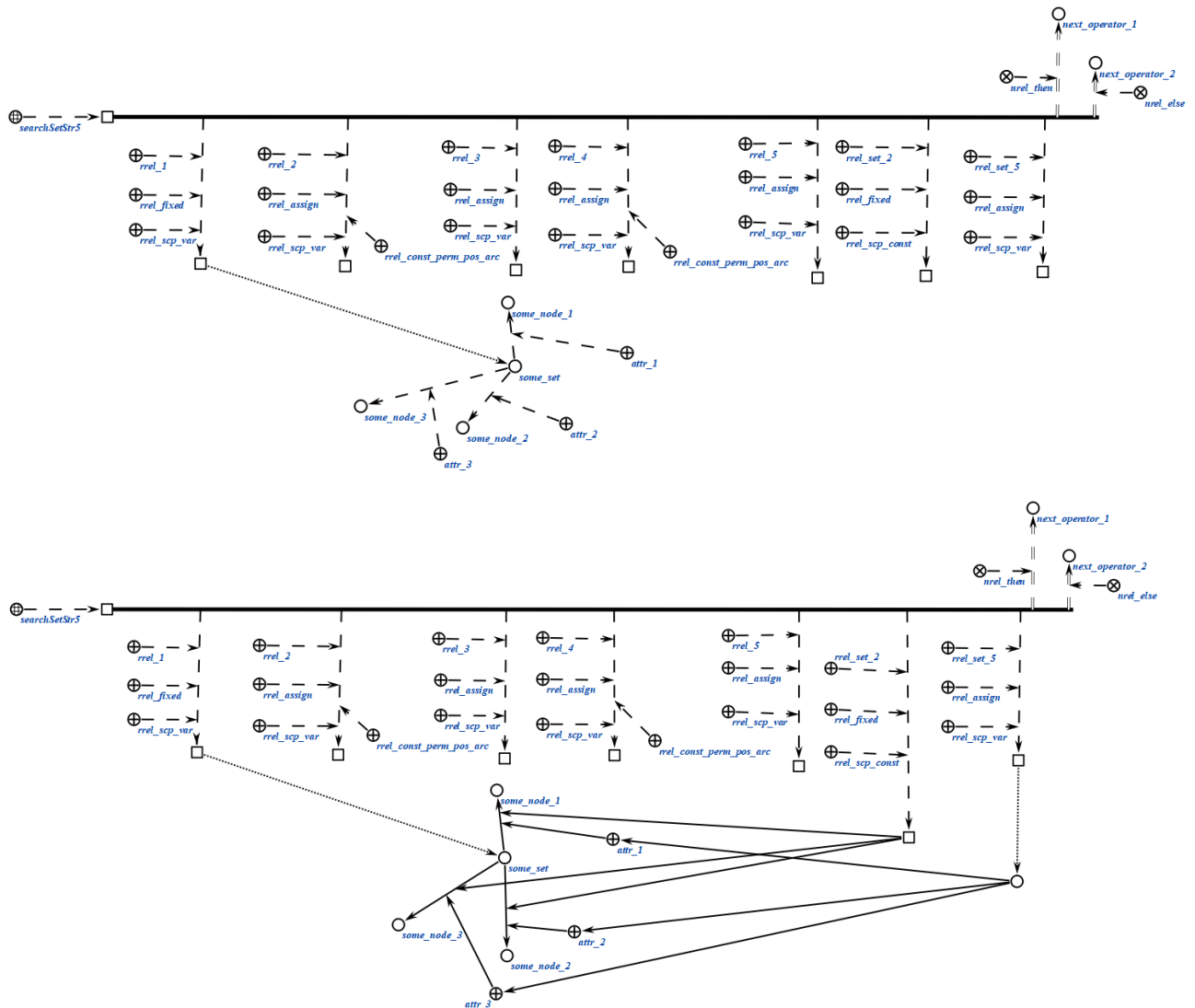
```
scp_operator_example_searchSetStr5 (*  
  <-_ searchSetStr5;;  
  _-> rrel_1:: rrel_fixed:: rrel_scp_var:: _el1;;  
  _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: _arc;;  
  _-> rrel_3:: rrel_assign:: rrel_scp_var:: _el3;;
```

```

-> rrel_4:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc::
_arc2;;
-> rrel_5:: rrel_assign:: rrel_scp_var:: _el5;;
-> rrel_set_2:: rrel_fixed:: rrel_scp_const:: some_node;;
-> rrel_set_5:: rrel_assign:: rrel_scp_var:: _set5;;
=> nrel_then:: next_operator_1;;
=> nrel_else:: next_operator_2;;
*);;

```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



6.6.6.6.1.2.5 scp-оператор поиска sc-конструкции по произвольному образцу

scp-операторы класса *scp-оператор поиска sc-конструкции по произвольному образцу* описывают действие поиска всех либо некоторых sc-элементов указанного типа, составляющих sc-конструкцию произвольного

вида. Данный *spr*-оператор может применяться для поиска на основе стандартных *sc*-конструкций, однако его использование в таком случае нецелесообразно, поскольку для этого имеются специальные *spr*-операторы. Каждый *spr*-оператор данного класса содержит четыре *spr*-операнда.

Первый *spr*-операнд должен быть помечен как *spr*-операнд с заданным значением'. Значением данного *spr*-операнда является *sc*-узел, обозначающий шаблон поиска, содержащий хотя бы один *переменный sc-элемент* и один *константный sc-элемент*. *Константные sc-элементы* не могут быть *sc*-коннекторами, поскольку в таком случае нарушается семантика самого понятия *sc*-коннектора, гласящая, что *sc*-коннектор имеет в один момент времени ровно два инцидентных ему *sc*-элемента. *Константные sc-элементы* могут отсутствовать, если в третьем *spr*-операнде указано значение хотя бы для одного *переменного sc-элемента* из *sc*-шаблона поиска.

Второй *spr*-операнд может иметь произвольный *тип значения spr-операнда*'. Значением данного *spr*-оператора является знак *sc*-множества, содержащего все возможные результаты поиска, каждый из которых является *sc*-множеством ориентированных *sc*-пар соответствия всех *sc*-переменных из *sc*-шаблона поиска их найденным значениям (если значения не были заданы в третьем *spr*-операнде). В случае, если данный *spr*-операнд помечен как *spr*-операнд с заданным значением', то необходимо явно указать *sc*-пары соответствия, в которых под атрибутом *1'* указывается *переменный sc-элемент* из *sc*-шаблона поиска (помеченный, как *spr*-константа'), под атрибутом *2'* - *spr*-переменные', в значения которых будут записаны найденные *sc*-элементы, соответствующие указанным *sc*-переменным *sc*-элементам из шаблона поиска (помеченные, как *spr*-переменная' и *spr*-операнд со свободным значением'). При этом *sc*-пары соответствия для остальных *sc*-переменных из *sc*-шаблона не будут явно формироваться в *sc*-памяти. Элемент пары под атрибутом *2'* может быть также помечен как *формируемое sc-множество*', тогда значением соответствующей *spr*-переменной' станет *sc*-множество, содержащее все возможные *sc*-элементы, соответствующие указанному переменному *sc*-элементу из *sc*-шаблона с учетом критериев поиска. В противном случае значением указанной *spr*-переменной' станет один из *sc*-элементов, удовлетворяющих условиям поиска.

Указанное *sc*-множество может быть пустым, тогда *sc*-пары соответствия не будут формироваться вообще, а *spr*-оператор может быть использован для проверки факта, имеется ли в *sc*-памяти конструкция изоморфная заданному

шаблону поиска с учетом параметров. В случае, если данный *scp-операнд* помечен как *scp-операнд со свободным значением*', то будут сформированы *sc-пары* соответствия сгенерированным значениям для всех *sc-переменных* из *sc-шаблона* генерации. Значением *scp-операнда* станет знак *sc-множества* результатов поиска, каждый из которых представляет собой возможную комбинацию указанных *sc-пар* соответствия переменных и их значений, удовлетворяющую критериям поиска. Количество возможных комбинаций зависит от конкретного вида *sc-шаблона* поиска.

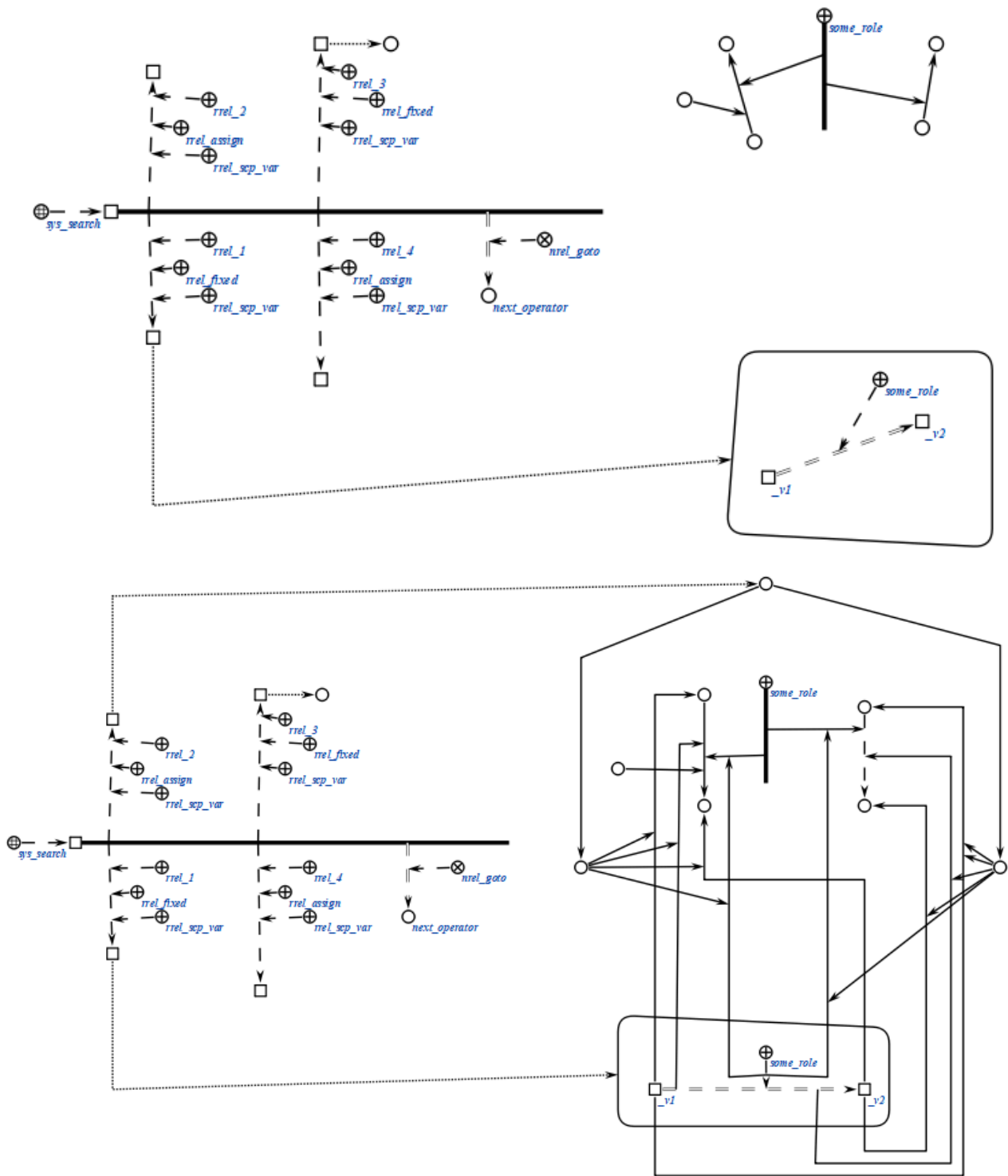
Третий *scp-операнд* должен быть помечен как *scp-операнд с заданным значением*'. Значением данного *scp-оператора* является знак *sc-множества* параметров поиска, то есть ориентированных *sc-пар* соответствия некоторых *sc-переменных* из *sc-шаблона* поиска их значениям, в случае, если значения заранее известны. При этом каждый из элементов *sc-пар* должен быть помечен, как *scp-операнд с заданным значением*', так же при необходимости должен быть указан *тип scp-операнда*'. Указанное *sc-множество* может быть пустым.

Четвертый *scp-операнд* может иметь произвольный *тип значения scp-операнда*'. Данный *scp-операнд* является необязательным и может быть опущен. Значением данного *scp-операнда* является знак *sc-множества* всех *sc-элементов*, найденных в результате выполнения данного *scp-оператора*. В случае, если *scp-операнд* помечен, как *scp-операнд с заданным значением*', то будет дополнено множество, являющееся значением данного *scp-операнда*, если как *scp-операнд со свободным значением*', то *sc-множество* будет сгенерировано и сформировано с нуля. Указанное *sc-множество* не содержит кратных элементов.

Пример *scp-оператора* данного класса на SCs-коде представлен ниже:

```
scp_operator_example_sys_search (*  
  
    <-_ sys_search;;  
    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: _el1;;  
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: _el2;;  
    _-> rrel_3:: rrel_fixed:: rrel_scp_var:: _el3;;  
    _-> rrel_4:: rrel_assign:: rrel_scp_var:: _el4;;  
    _=> nrel_goto:: next_operator;;  
*);;
```

Пример *scp-оператора* данного класса на SCg-коде представлен ниже:



Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_sys_search (*
  <_ sys_search;;
  _-> rrel_1:: rrel_fixed:: rrel_scp_var:: _el1;;
  _-> rrel_2:: rrel_fixed:: rrel_scp_var:: _el2;;
```

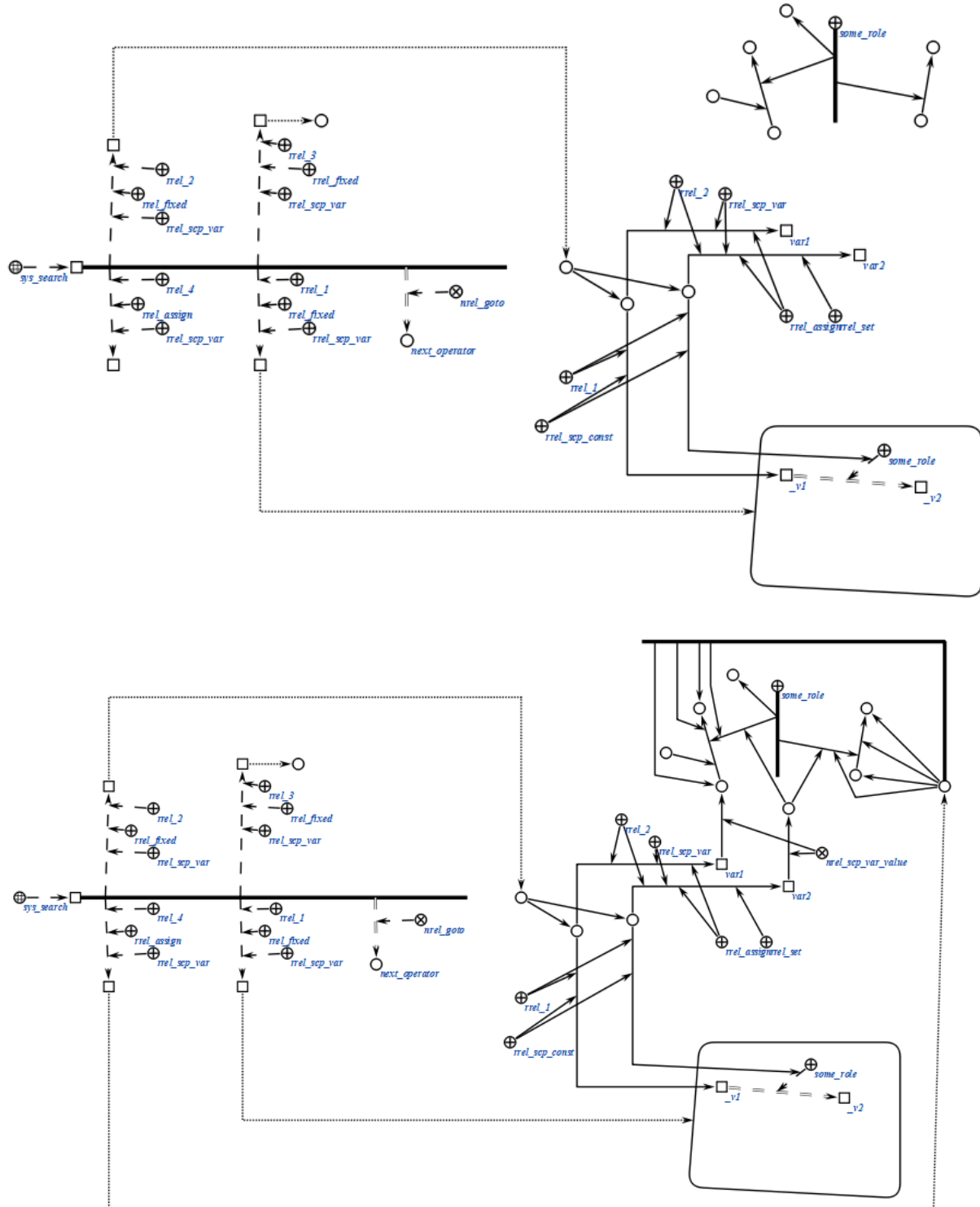
```

-> rrel_3:: rrel_fixed:: rrel_scp_var:: _el3;;
-> rrel_4:: rrel_assign:: rrel_scp_var:: _el4;;
=> nrel_goto:: next_operator;;

```

*);;

Пример scp-оператора данного класса на SCg-коде представлен ниже:



6.6.6.6.1.3 scp-операторы удаления sc-конструкций

scp-операторы класса *scp-оператор удаления sc-конструкций* описывают действие удаления множества *sc-элементов*, удовлетворяющих некоторому условию. При этом следует заметить, что при указании того факта, что значение scp-операнда должно быть удалено (при помощи ролевого отношения *удаляемый sc-элемент*'), удален будет физически именно тот элемент, который является значением scp-операнда, а не только связка отношения *значение**, связывающая scp-операнд с его значением (если такая есть). Таким образом, удалена может быть и *scp-константа*'. Следует помнить, что при удалении *sc-элемента* автоматически удаляются все инцидентные ему *sc-коннекторы*, а также *sc-коннекторы*, инцидентные указанным *sc-коннекторам* и так далее рекурсивно.

scp-операторы класса не предполагают ветвления программы в зависимости от выполнения дополнительных условий, поэтому указание следующих scp-операторов осуществляется только при помощи отношения *следующий scp-оператор**, без подмножеств.

Данный класс *scp-операторов* разбивается на следующие подклассы::

- *scp-оператор удаления одноэлементной конструкции;*
- *scp-оператор удаления трехэлементной конструкции;*
- *scp-оператор удаления пятиэлементной конструкции;*
- *scp-оператор удаления множества элементов трехэлементной sc-конструкции;*

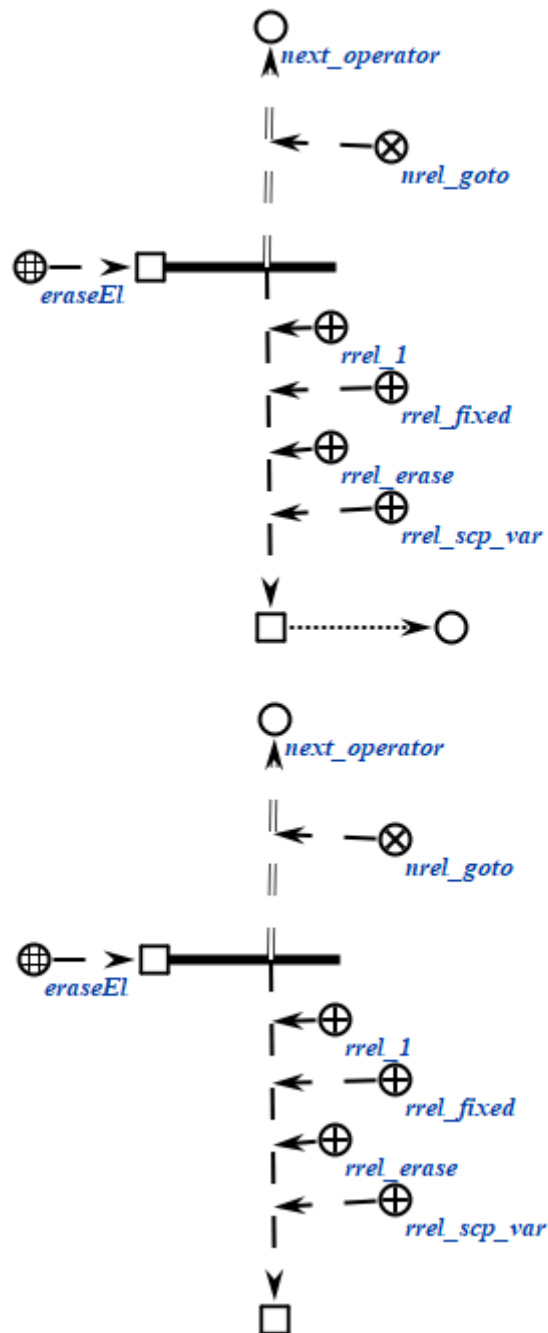
6.6.6.6.1.3.1 scp-оператор удаления одноэлементной sc-конструкции

scp-операторы класса *scp-оператор удаления одноэлементной sc-конструкции* описывают действие удаления одного заданного *sc-элемента*. Каждый scp-оператор данного класса содержит один scp-операнд, обязательно помеченный как *scp-операнд с заданным значением*'.

Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_eraseEl (*
    <_ eraseEl;;
    _-> rrel_1:: rrel_fixed:: rrel_erase:: rrel_scp_var:: _el1;;
    _=> nrel_goto:: next_operator;;
*) ;;
```

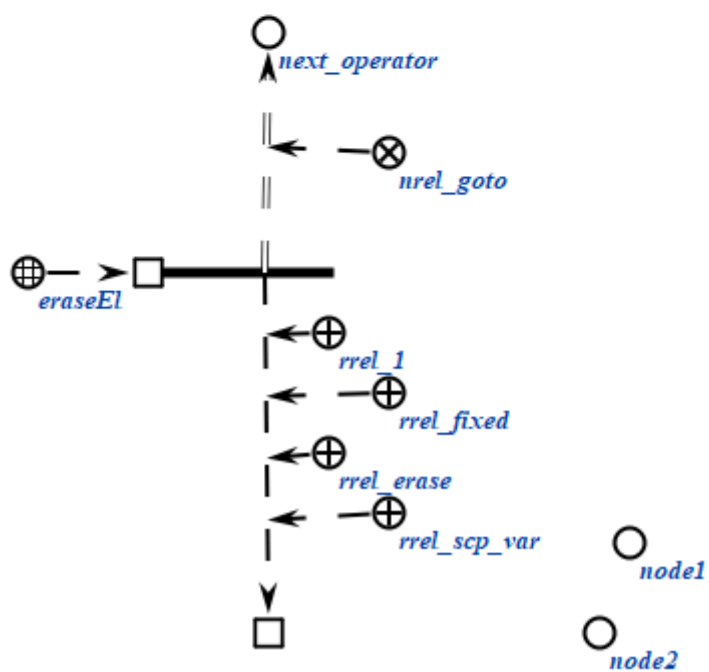
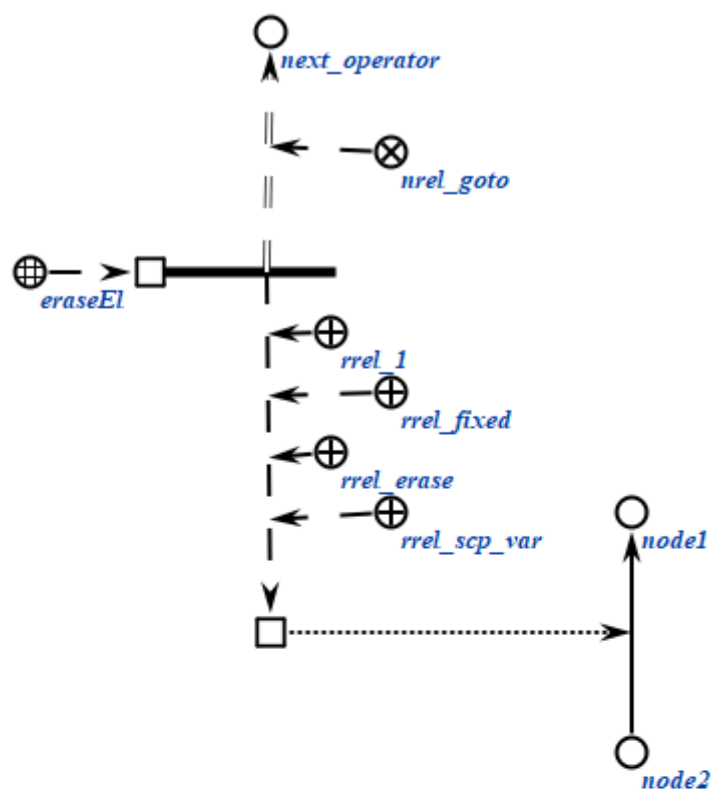
Пример scp-оператора данного класса на SCg-коде представлен ниже:



Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_eraseEl (*
    <-_ eraseEl;;
    _-> rrel_1:: rrel_fixed:: rrel_erase:: rrel_scp_var:: _el1;;
    _=> nrel_goto:: next_operator;;
*);;
```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



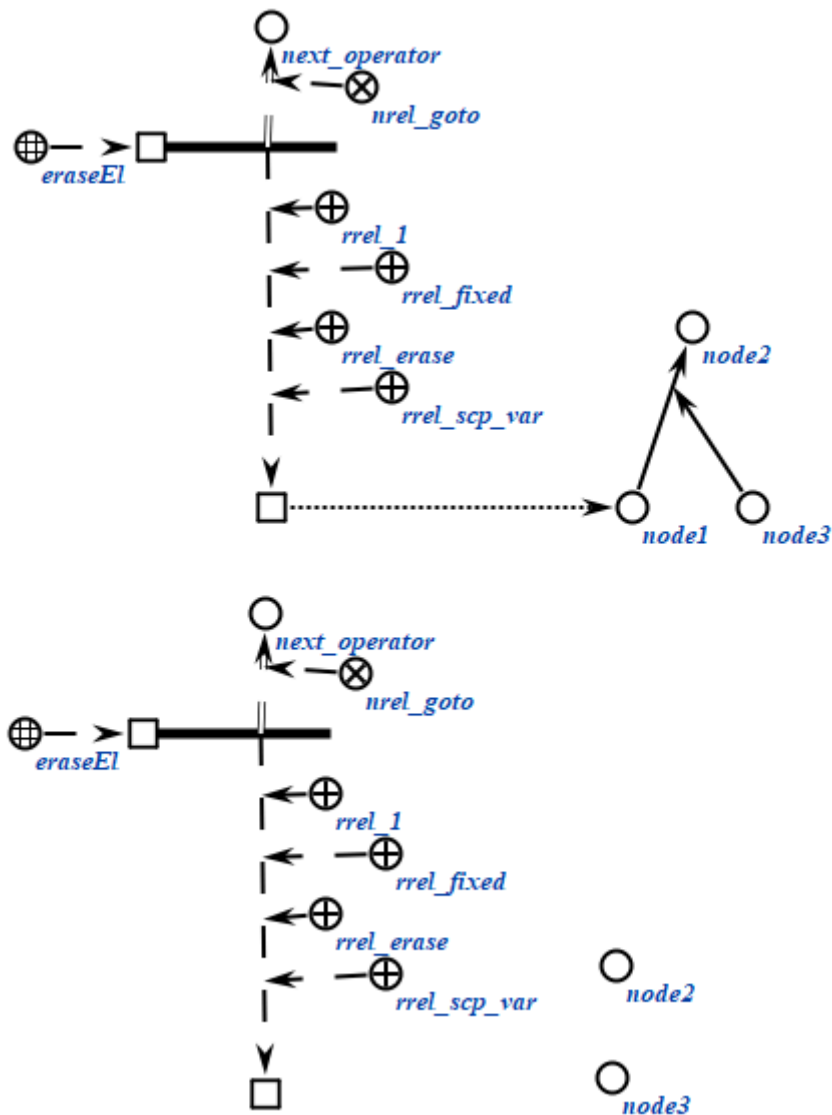
Пример scp-оператора данного класса на SCs-коде представлен ниже:

```

scp_operator_example_eraseEl (*
    <_ eraseEl;;
    _-> rrel_1:: rrel_fixed:: rrel_erase:: rrel_scp_var:: _el1;;
    _=> nrel_goto:: next_operator;;
*);;

```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



6.6.6.6.1.3.2 scp-оператор удаления трёхэлементной sc-конструкции

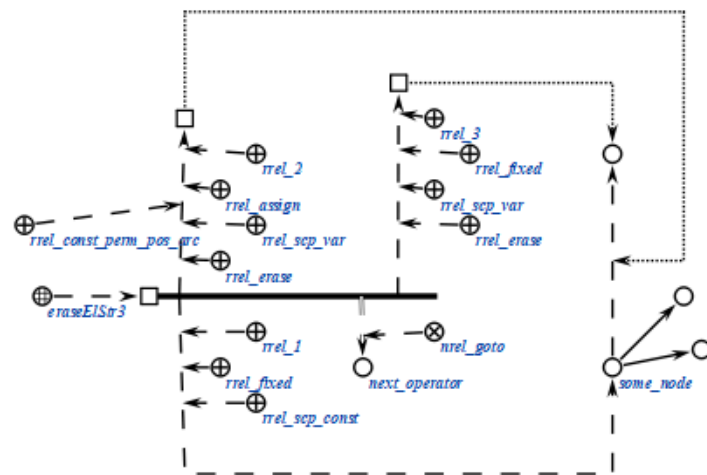
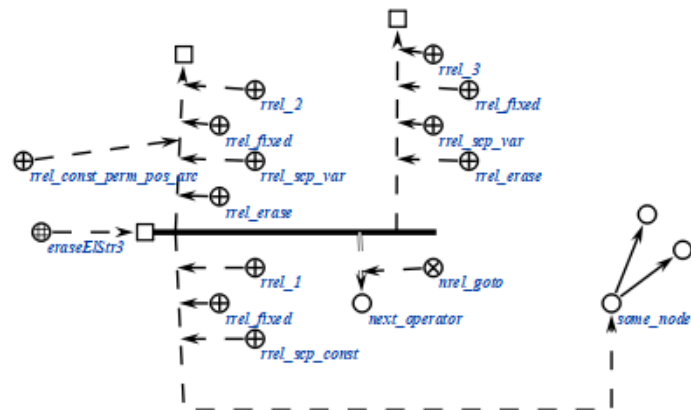
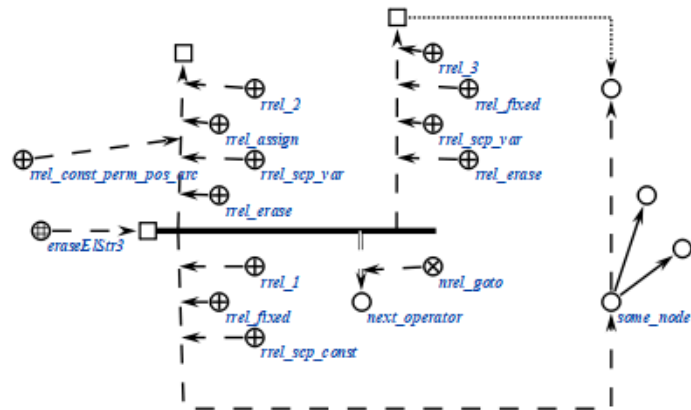
scp-операторы класса *scp-оператор* удаления трёхэлементной sc-конструкции описывают действие удаления одного или нескольких sc-элементов, составляющих трёхэлементную sc-конструкцию. Каждый scp-оператор данного класса содержит три scp-операнда.

При выполнении scp-операторов данного класса выполняется поиск *sc-элементов*, полностью аналогичный поиску в *scp-операторах поиска трёхэлементных sc-конструкций*, после чего выполняется удаление значений scp-операндов, помеченных как *удаляемый sc-элемент*'. Если условиям поиска удовлетворяет несколько имеющихся в памяти *sc-конструкций*, то будут удалены sc-элементы только одной из них.

Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_eraseElStr3 (*  
    <_ eraseElStr3;;  
    _-> rrel_1:: rrel_fixed:: rrel_scp_const:: some_node;;  
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc::  
rrel_erase:: _arc;;  
    _-> rrel_3:: rrel_fixed:: rrel_scp_var:: rrel_erase:: _el3;;  
    _=> nrel_goto:: next_operator;;  
*);;
```

Пример scp-оператора данного класса на SCg-коде представлен ниже:

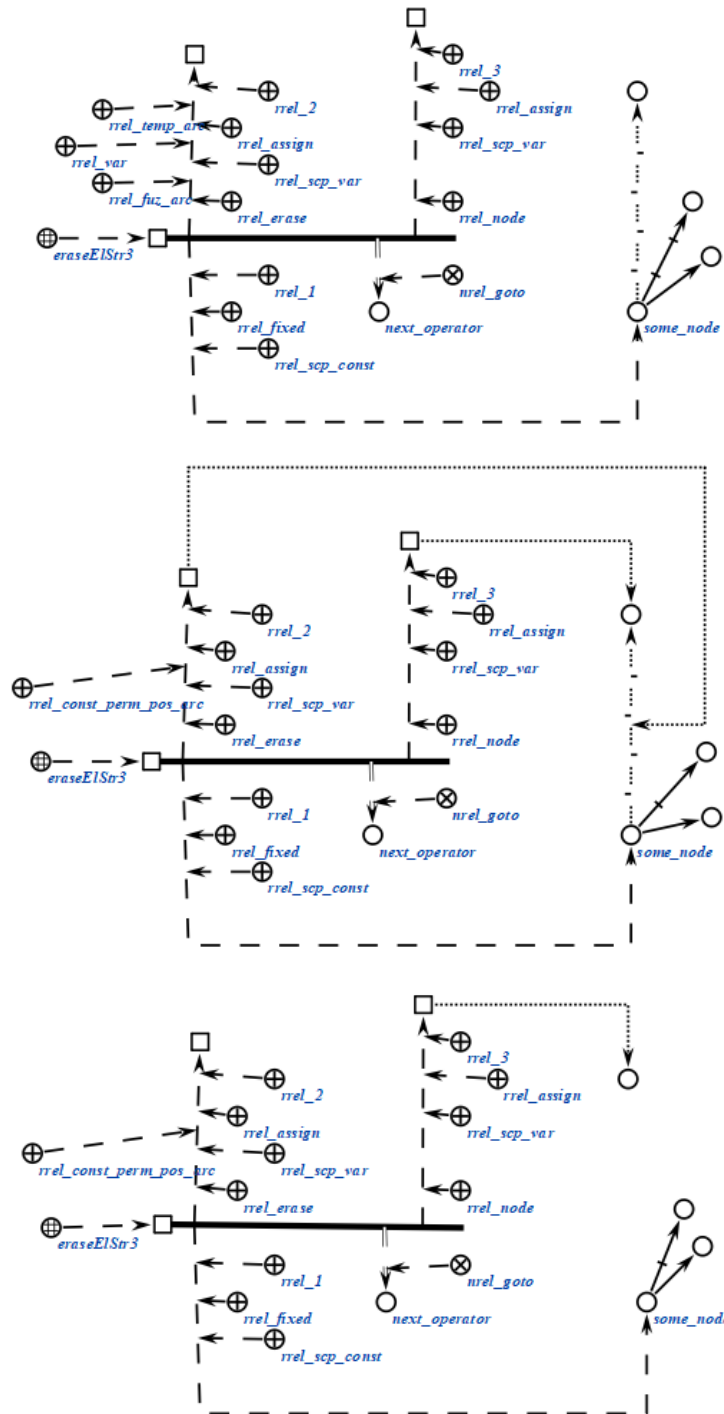



```

scp_operator_example_eraseElStr3 (*
    <_ eraseElStr3;;
    _-> rrel_1:: rrel_fixed:: rrel_scp_const:: some_node;;
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_erase:: rrel_temp::
rrel_fuz:: rrel_var:: _arc;;
    _-> rrel_3:: rrel_assign:: rrel_scp_var:: rrel_node:: _el3;;
    _=> nrel_goto:: next_operator;;
*);;

```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



6.6.6.1.3.3 scp-оператор удаления пятиэлементной sc-конструкции

scp-операторы класса *scp-оператор* удаления пятиэлементной *sc-конструкции* описывают действие удаления одного или нескольких *sc-элементов*, составляющих *пятиэлементную sc-конструкцию*. Каждый scp-оператор данного класса содержит пять scp-операнда.

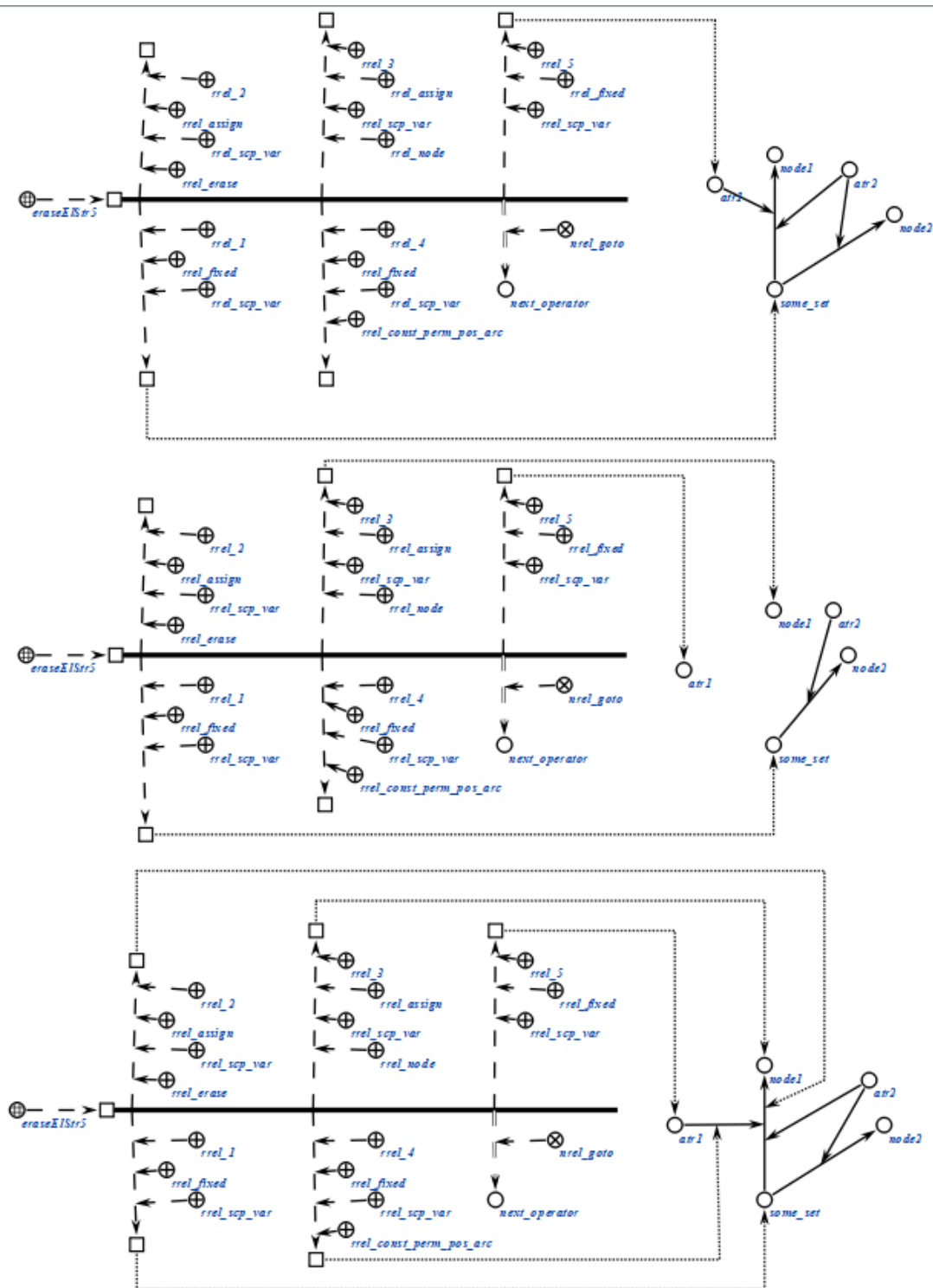
При выполнении scp-операторов данного класса выполняется поиск *sc-элементов*, полностью аналогичный поиску в *scp-операторах поиска*

пятиэлементных *sc*-конструкций, после чего выполняется удаление значений *scp*-операндов, помеченных как *удаляемый sc-элемент*'. Если условиям поиска удовлетворяет несколько имеющихся в памяти *sc*-конструкций, то будут удалены *sc*-элементы только одной из них.

Пример *scp*-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_eraseElStr5 (*  
    <_ eraseElStr5;;  
    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: _el1;;  
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_erase:: _arc;;  
    _-> rrel_3:: rrel_assign:: rrel_scp_var:: rrel_node:: _el3;;  
    _-> rrel_4:: rrel_fixed:: rrel_scp_var:: rrel_const_perm_pos_arc:: _arc2;;  
    _-> rrel_5:: rrel_fixed:: rrel_scp_var:: _el5;;  
    _=> nrel_goto:: next_operator;;  
*);;
```

Пример *scp*-оператора данного класса на SCg-коде представлен ниже:



6.6.6.6.1.3.4 scp-оператор удаления множества sc-элементов трёхэлементной sc-конструкции

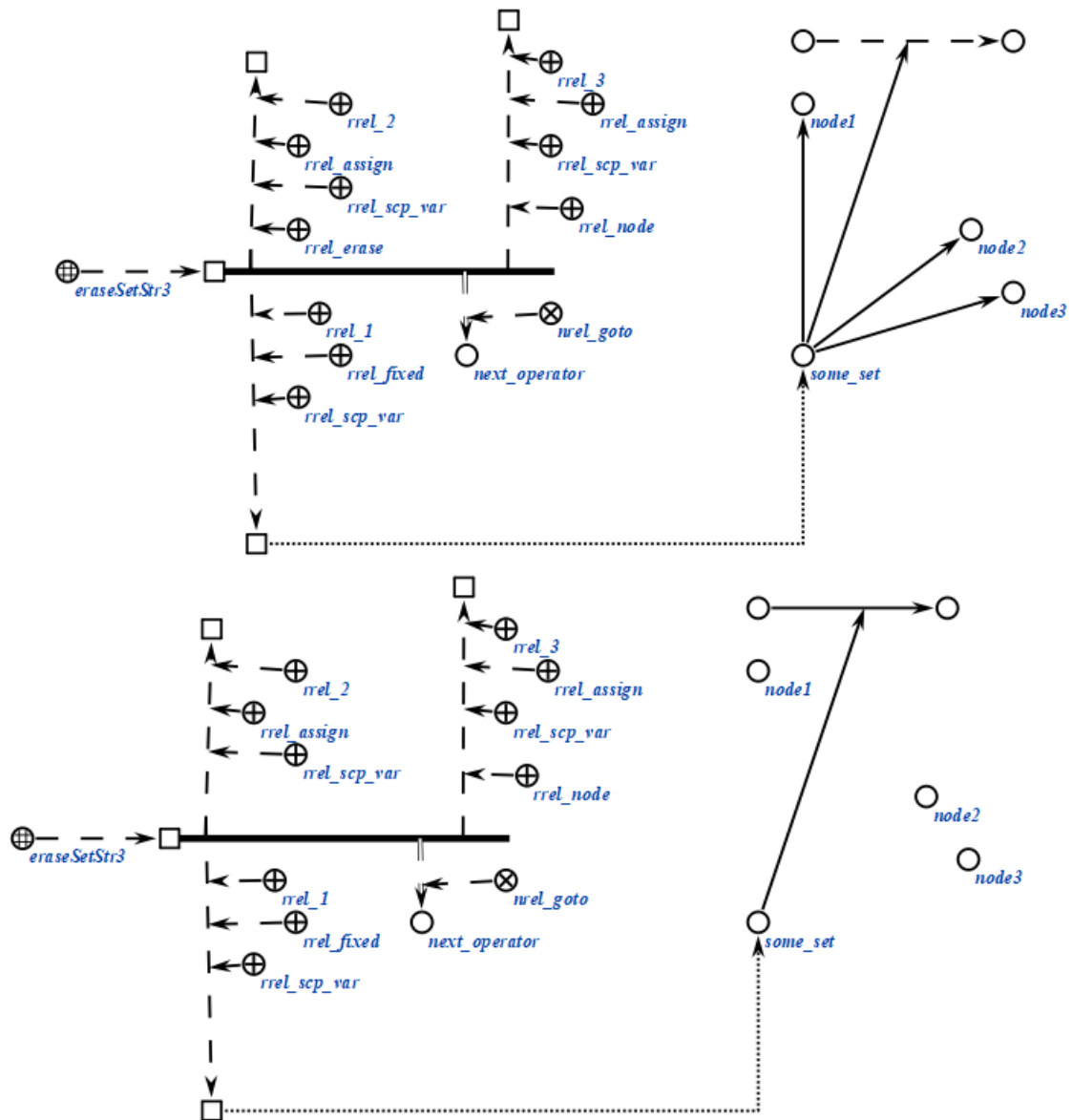
scp-операторы класса *scp-оператор* удаления множества *sc-элементов* трёхэлементной *sc-конструкции* описывают действие, полностью аналогичное действию, выполняемому *scp-операторами* удаления трёхэлементной *sc-конструкции*, однако удалены будут всевозможные *sc-элементы*,

удовлетворяющие заданному условию поиска и соответствующие scp-операнду, помеченному как *удаляемый sc-элемент*'.

Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_eraseSetStr3 (*  
    <_ eraseSetStr3;;  
    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: _el1;;  
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_erase:: _el2;;  
    _-> rrel_3:: rrel_assign:: rrel_scp_var:: rrel_node:: _el3;;  
    _=> nrel_goto:: next_operator;;  
*);;
```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_eraseSetStr3 (*
  <_ eraseSetStr3;;
  _-> rrel_1:: rrel_fixed:: rrel_scp_var:: _el1;;
  _-> rrel_2:: rrel_assign:: rrel_scp_var:: _el2;;
  _-> rrel_3:: rrel_assign:: rrel_scp_var:: rrel_node:: rrel_erase:: _el3;;
  _=> nrel_goto:: next_operator;;
*);;
```

Пример scp-оператора данного класса на SCg-коде представлен ниже:

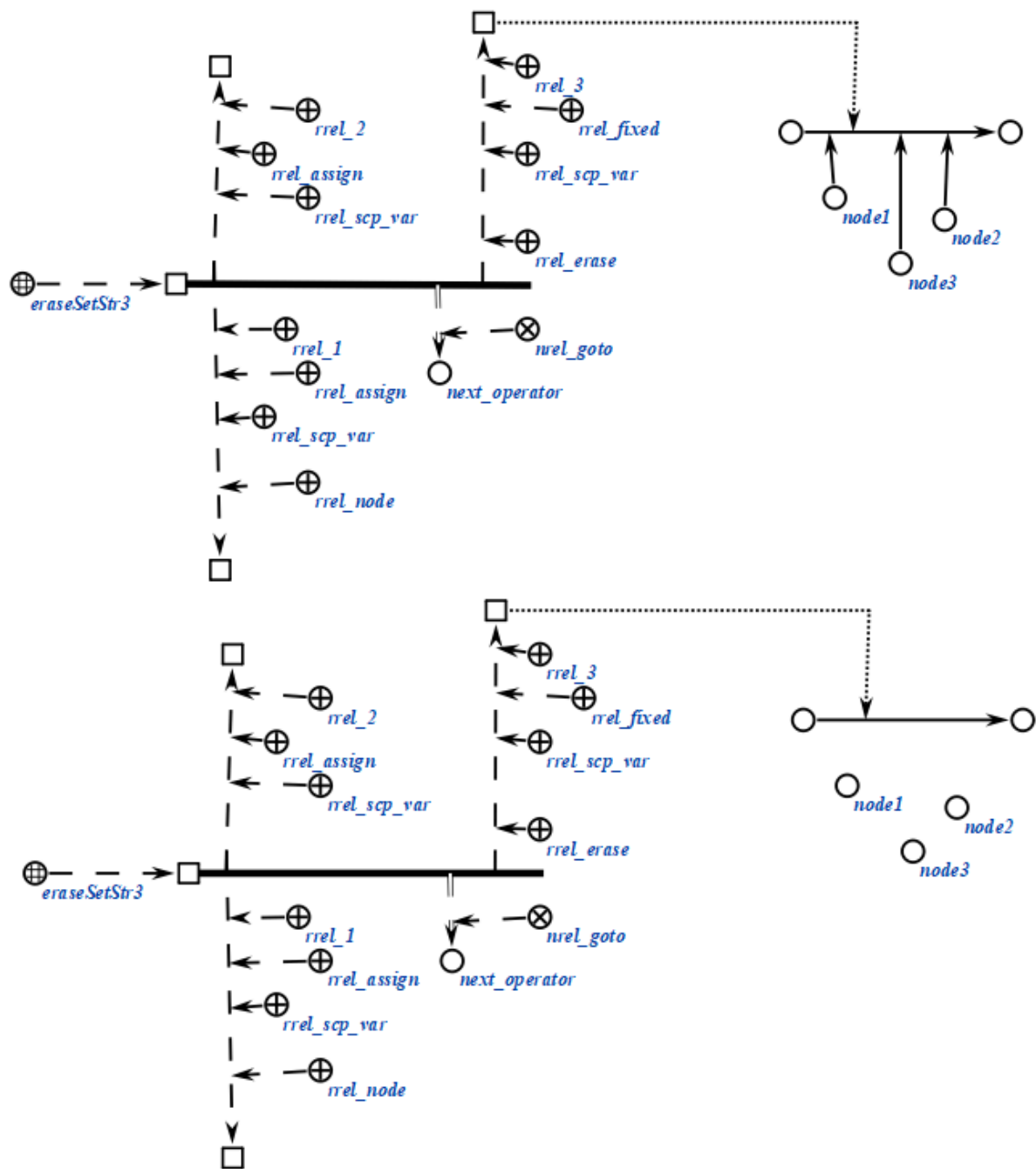


```

scp_operator_example_eraseSetStr3 (*
    <-_ eraseSetStr3;;
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: rrel_node:: _el1;;
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: _el2;;
    _-> rrel_3:: rrel_fixed:: rrel_scp_var:: rrel_erase:: _el3;;
    _=> nrel_goto:: next_operator;;
*);;

```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



6.6.6.6.1.4 *scp*-операторы проверки условий

scp-операторы класса *scp-оператор проверки условий* описывают действие проверки некоторого условия с целью ветвления *scp-программы*. При этом состояние памяти системы не изменяется, а после выполнения *scp*-оператора осуществляется переход к следующему *scp*-оператору по связке отношения *следующий scp-оператор при успешном выполнении** или *следующий scp-оператор при безуспешном выполнении**, в зависимости от истинности некоторого условия.

Данный класс *scp-операторов* разбивается на следующие подклассы:

- *scp-оператор проверки типа sc-элемента*;
- *scp-оператор проверки наличия значения у переменной*;
- *scp-оператор проверки наличия содержимого у файла*;
- *scp-оператор проверки совпадения значений scp-операндов*;
- *scp-оператор проверки равенства числовых содержимых файлов*;
- *scp-оператор сравнения числовых содержимых файлов*.

6.6.6.6.1.4.1 *scp*-оператор проверки типа *sc*-элемента

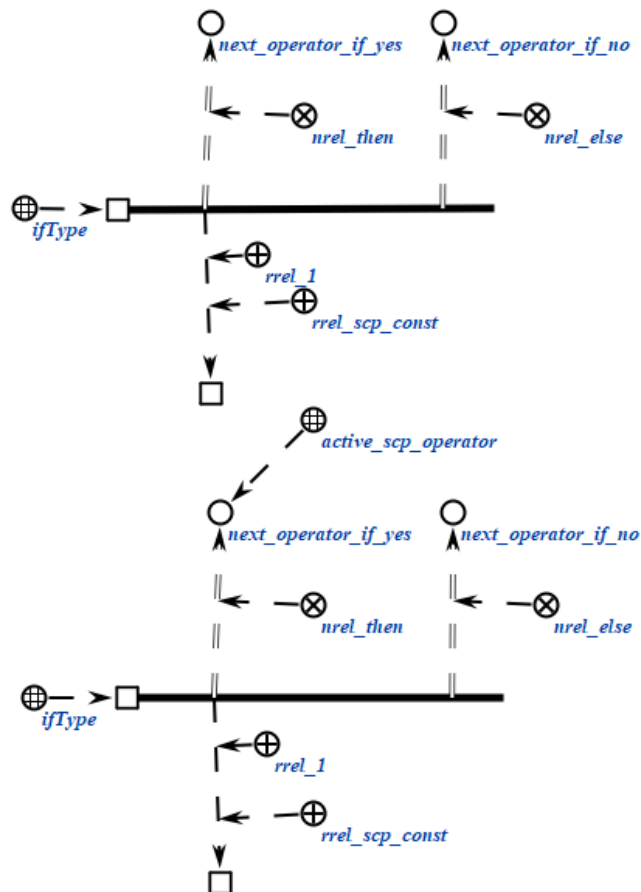
scp-операторы класса *scp-оператор проверки типа sc-элемента* описывают действие проверки того, соответствует ли *sc-элемент*, являющийся значением единственного *scp*-операнда, указанному типу. Успешным выполнение считается, если структурный тип значения *scp*-операнда соответствует заданному.

Каждый *scp*-оператор данного класса содержит один *scp*-операнд, который должен быть помечен как *scp-операнд с заданным значением'*. Тип *sc-элемента*, на соответствие которому будет осуществляться проверка, указывается при помощи подмножеств ролевого отношения *тип sc-элемента'*.

Пример *scp*-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_ifType (*
    <-_ ifType;;
    _-> rrel_1:: rrel_scp_const:: some_node;;
    _=> nrel_then:: next_operator_if_yes;;
    _=> nrel_else:: next_operator_if_no;;
*);;
```

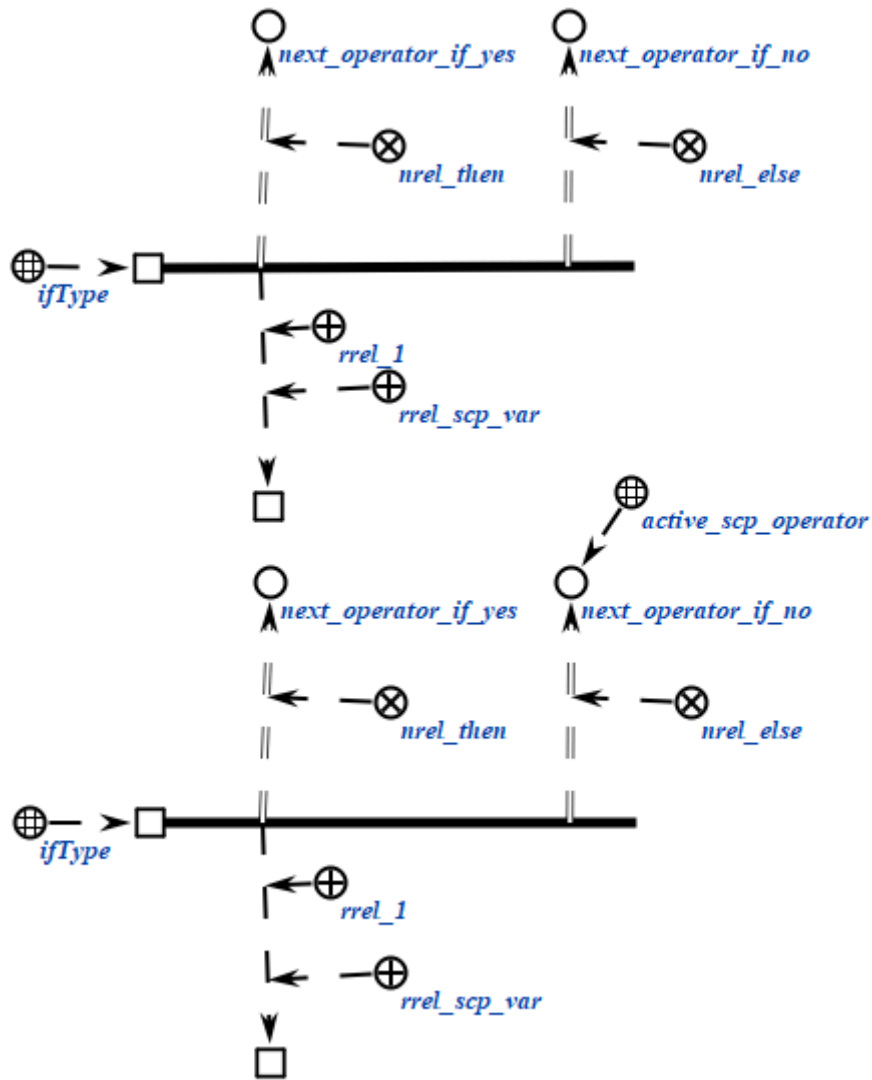
Пример *scp*-оператора данного класса на SCg-коде представлен ниже:



Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_ifType (*
    <_ ifType;;
    _-> rrel_1:: rrel_scp_var:: some_node;;
    _=> nrel_then:: next_operator_if_yes;;
    _=> nrel_else:: next_operator_if_no;;
*);;
```

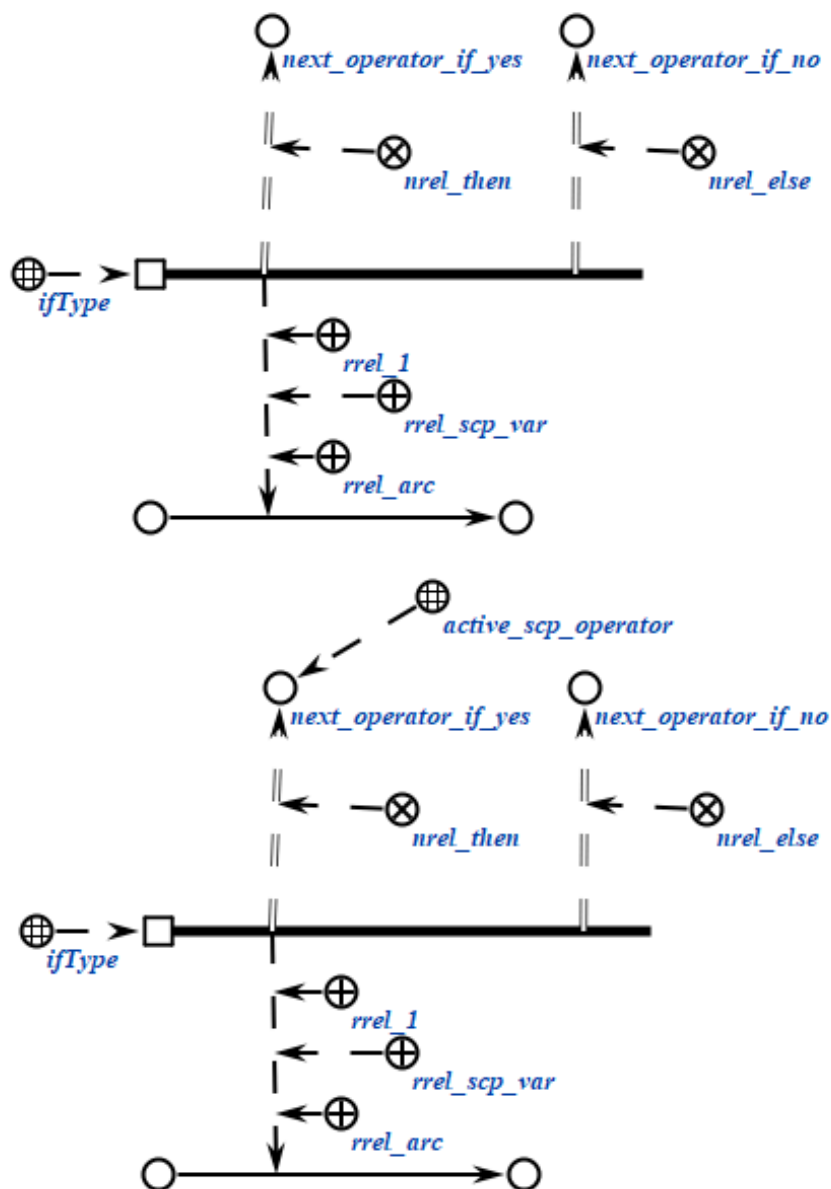
Пример scp-оператора данного класса на SCg-коде представлен ниже:



Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_ifType (*
  <_ ifType;;
  _-> rrel_1:: rrel_scp_var:: rrel_arc:: some_node;;
  _=> nrel_then:: next_operator_if_yes;;
  _=> nrel_else:: next_operator_if_no;;
*) ;;
```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



6.6.6.6.1.4.2 scp-оператор проверки наличия значения у sc-переменной

scp-операторы класса *scp-оператор проверки наличия значения у sc-переменной* описывают действие проверки того, имеет ли единственный scp-операнд значение. Успешным выполнение считается, если значение найдено.

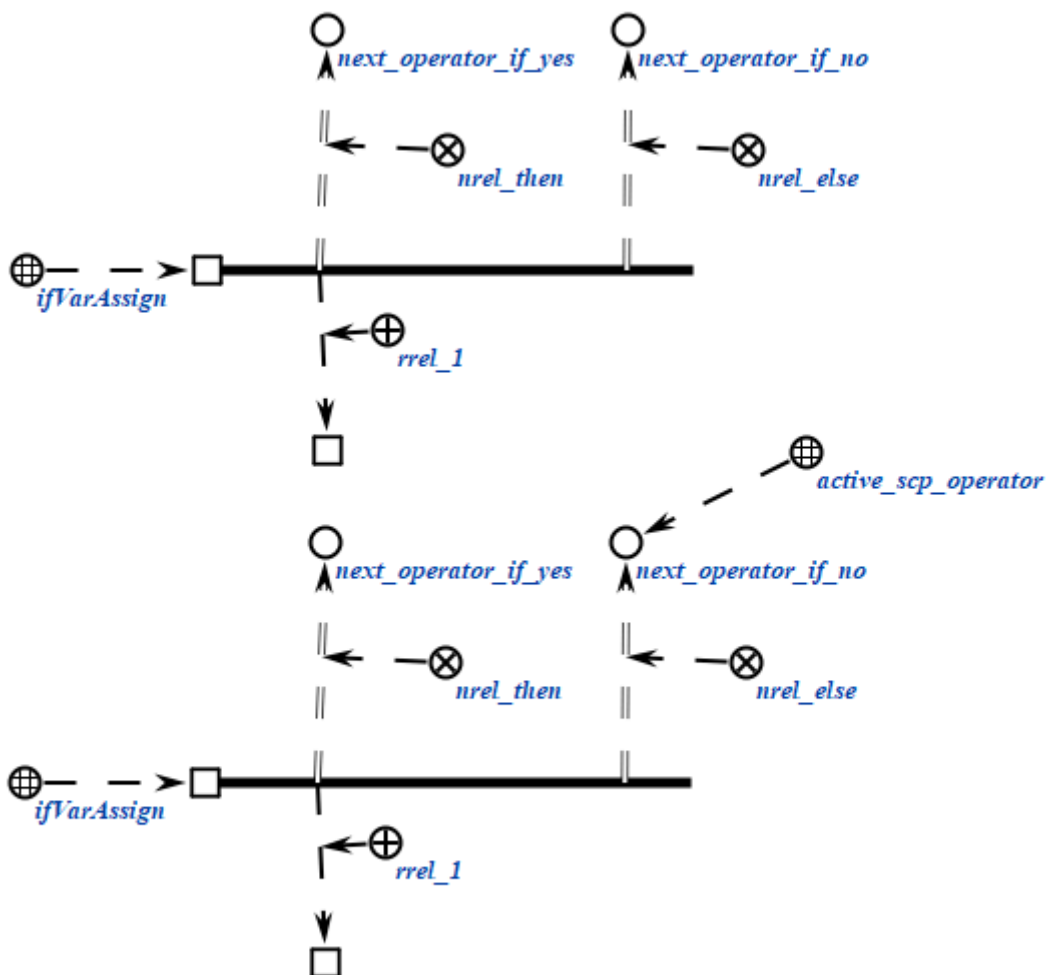
Каждый scp-оператор данного класса содержит один scp-операнд, который должен быть помечен как *scp-операнд с заданным значением*'. Если указанный scp-операнд является *scp-переменной*', то осуществляется попытка поиска sc-связки отношения *значение** для данного scp-операнда. Использование

данного класса *scp-операторов* с *scp-константами*' нецелесообразно, так как *scp-константа*' всегда имеет значение.

Пример *scp-оператора* данного класса на SCs-коде представлен ниже:

```
scp_operator_example_ifVarAssign (*
    <-_ ifVarAssign;;
    _-> rrel_1:: _node;;
    _=> nrel_then:: next_operator_if_yes;;
    _=> nrel_else:: next_operator_if_no;;
*);;
```

Пример *scp-оператора* данного класса на SCg-коде представлен ниже:



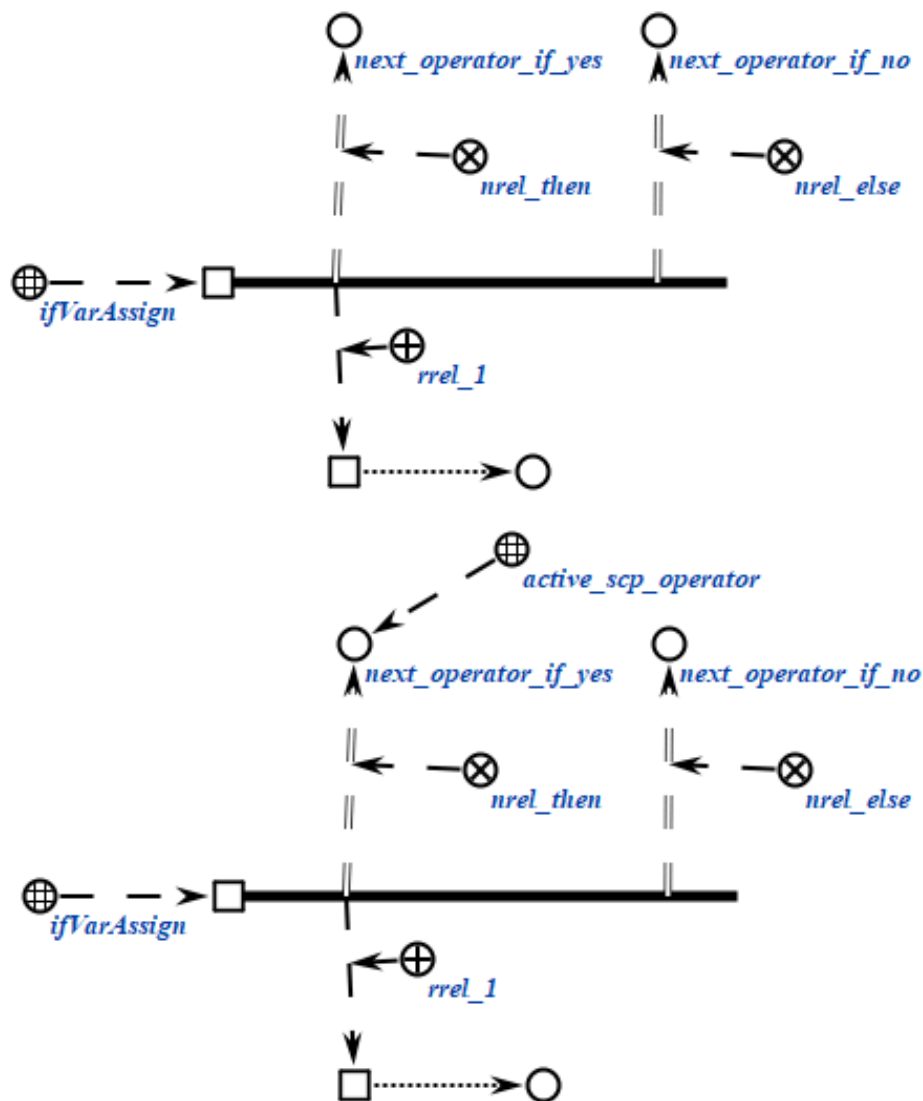
Пример *scp-оператора* данного класса на SCs-коде представлен ниже:

```

scp_operator_example_ifVarAssign (*
  <-_ ifVarAssign;;
  _-> rrel_1:: _node;;
  _=> nrel_then:: next_operator_if_yes;;
  _=> nrel_else:: next_operator_if_no;;
*);;

```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



6.6.6.6.1.4.3 scp-оператор проверки наличия содержимого у файла

scp-операторы класса *scp-оператор проверки наличия содержимого у файла* описывают действие проверки того, имеет ли заданный файл

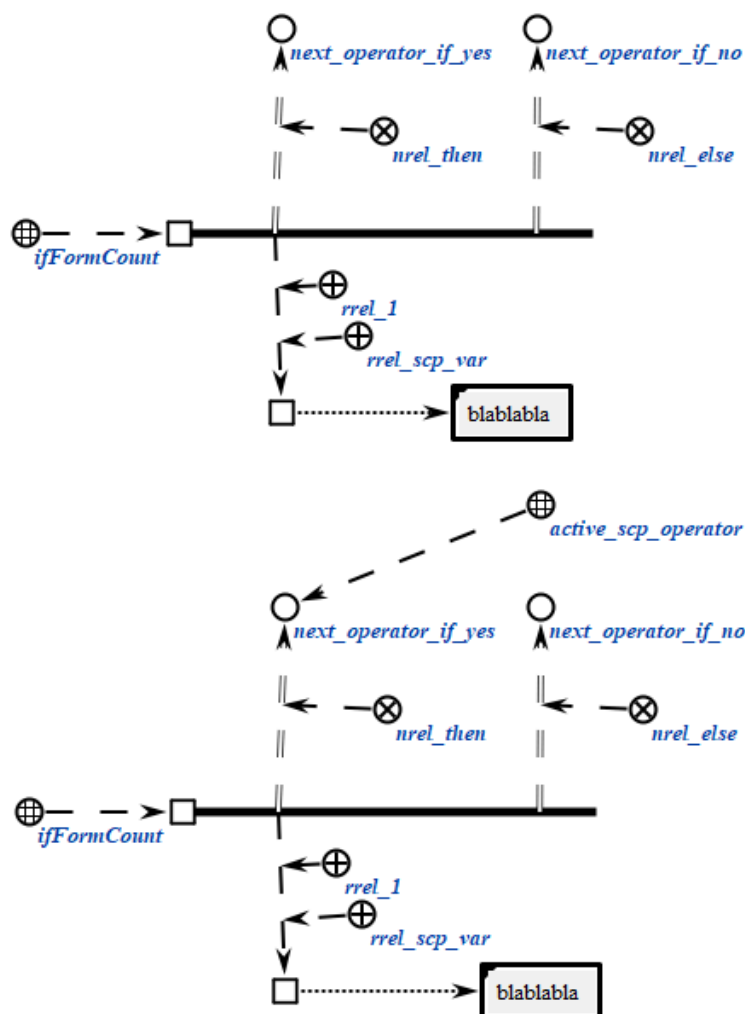
сформированное содержимое. Успешным выполнение считается, если содержимое сформировано.

Каждый scp-оператор данного класса содержит один scp-операнд, который должен быть помечен как *scp-операнд с заданным значением*. *sc-элемент*, являющийся значением scp-операнда должен быть *файлом*.

Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_ifFormCount (*
    <_ ifFormCount;;
    _-> rrel_1:: rrel_scp_var:: _node;;
    _=> nrel_then:: next_operator_if_yes;;
    _=> nrel_else:: next_operator_if_no;;
*);;
```

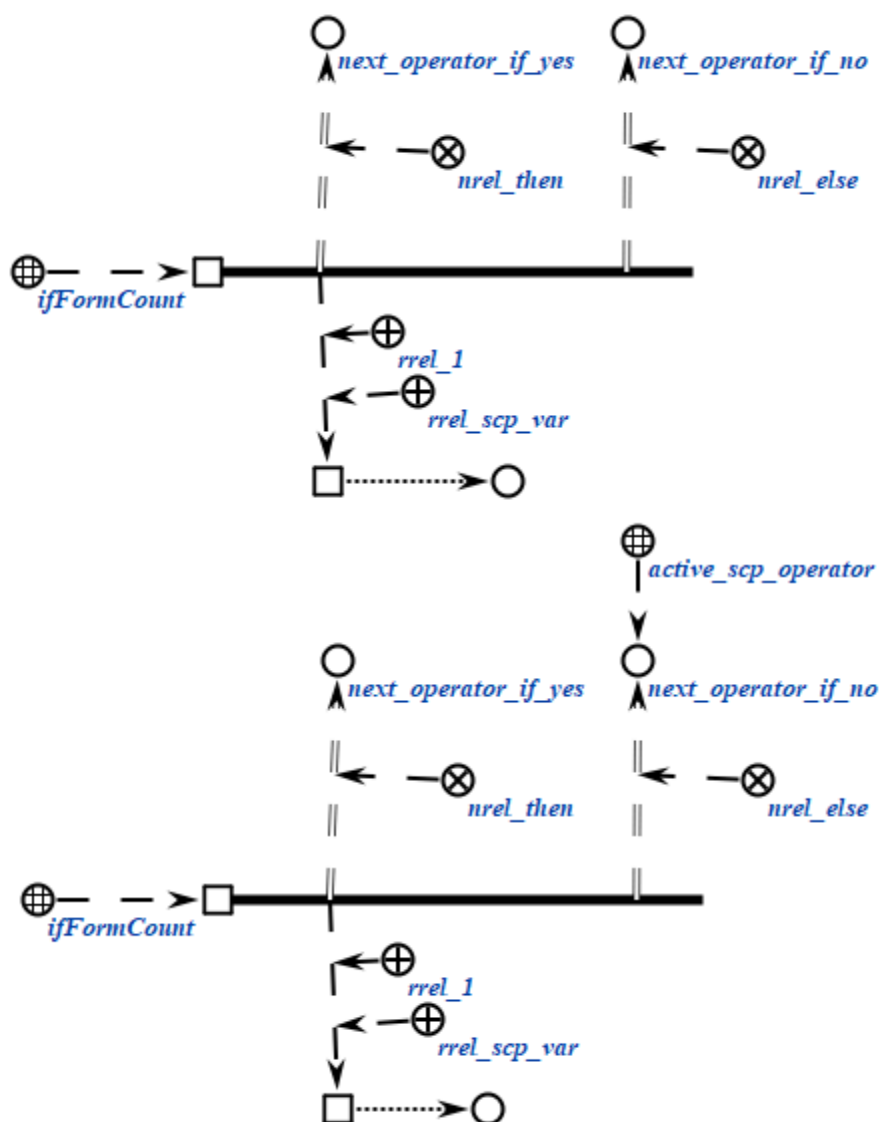
Пример scp-оператора данного класса на SCg-коде представлен ниже:



Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_ifFormCount (*
  <_ ifFormCount;;
  _-> rrel_1:: rrel_scp_var:: _node;;
  _=> nrel_then:: next_operator_if_yes;;
  _=> nrel_else:: next_operator_if_no;;
*);;
```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



6.6.6.6.1.4.4 *scp*-оператор проверки совпадения значений *scp*-операндов

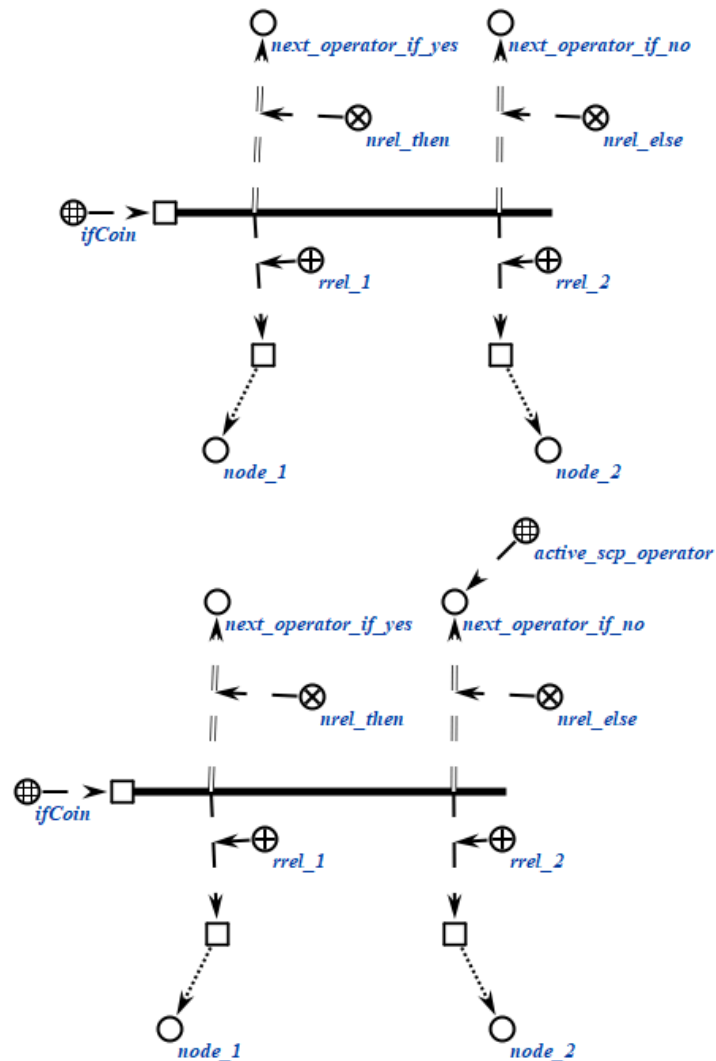
scp-операторы класса *scp-оператор проверки совпадения значений scp-операндов* описывают действие проверки того, совпадают ли значения *scp*-операндов. Успешным выполнение считается, если значения совпадают. При этом учитывается только явное совпадение двух *sc*-элементов, дополнительные проверки на семантическую или логическую эквивалентность не осуществляются.

Каждый *scp*-оператор данного класса содержит два *scp*-операнда, которые должны быть помечены как *scp-операнд с заданным значением*'. Каждый из *scp*-операндов может быть как *scp-константой*', так и *scp-переменной*', однако сравнение значений двух *scp-констант*' не является целесообразным.

Пример *scp*-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_ifCoin (*
    <-_ ifCoin;;
    _-> rrel_1:: _node1;;
    _-> rrel_2:: _node2;;
    _=> nrel_then:: next_operator_if_yes;;
    _=> nrel_else:: next_operator_if_no;;
*);;
```

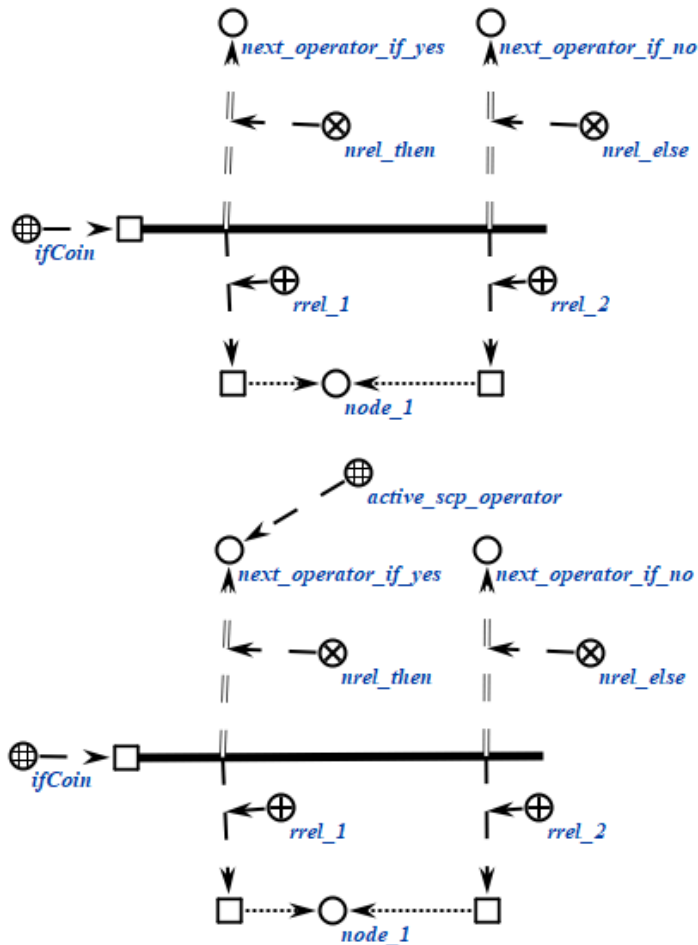
Пример *scp*-оператора данного класса на SCg-коде представлен ниже:



Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_ifCoin (*
    <-_ ifCoin;;
    _-> rrel_1:: _node1;;
    _-> rrel_2:: _node2;;
    _=> nrel_then:: next_operator_if_yes;;
    _=> nrel_else:: next_operator_if_no;;
*);;
```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



6.6.6.6.1.4.5 scp-оператор проверки равенства числовых содержимых файлов

scp-операторы класса *scp-оператор проверки равенства числовых содержимых файлов* описывают действие проверки того, равны ли между собой числовые содержимые двух *файлов*, являющихся значениями scp-операндов. Под числовым содержимым понимаются те же виды файлов, что и в случае с *scp-операторами обработки содержимых файлов*. Успешным выполнение считается, если численные содержимые равны с точки зрения теории чисел. Например, числа «1e3» и «600» считаются равными.

Каждый scp-оператор данного класса содержит два scp-операнда, которые должны быть помечены как *scp-операнд с заданным значением*'. Каждый из scp-операндов может быть как *scp-константой*', так и *scp-переменной*'.

Пример scp-оператора данного класса на SCs-коде представлен ниже:

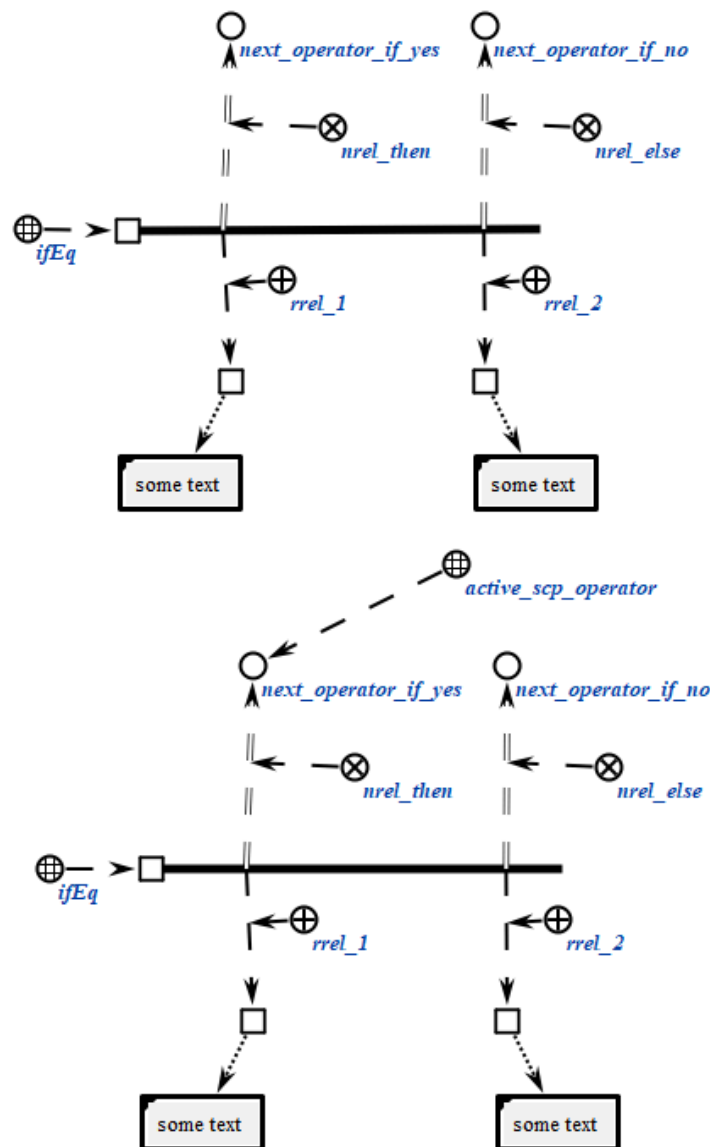
```
scp_operator_example_ifEq (*)
```

```

<-_ ifEq;;
-> rrel_1:: _node1;;
-> rrel_2:: _node2;;
=> nrel_then:: next_operator_if_yes;;
=> nrel_else:: next_operator_if_no;;
*);;

```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



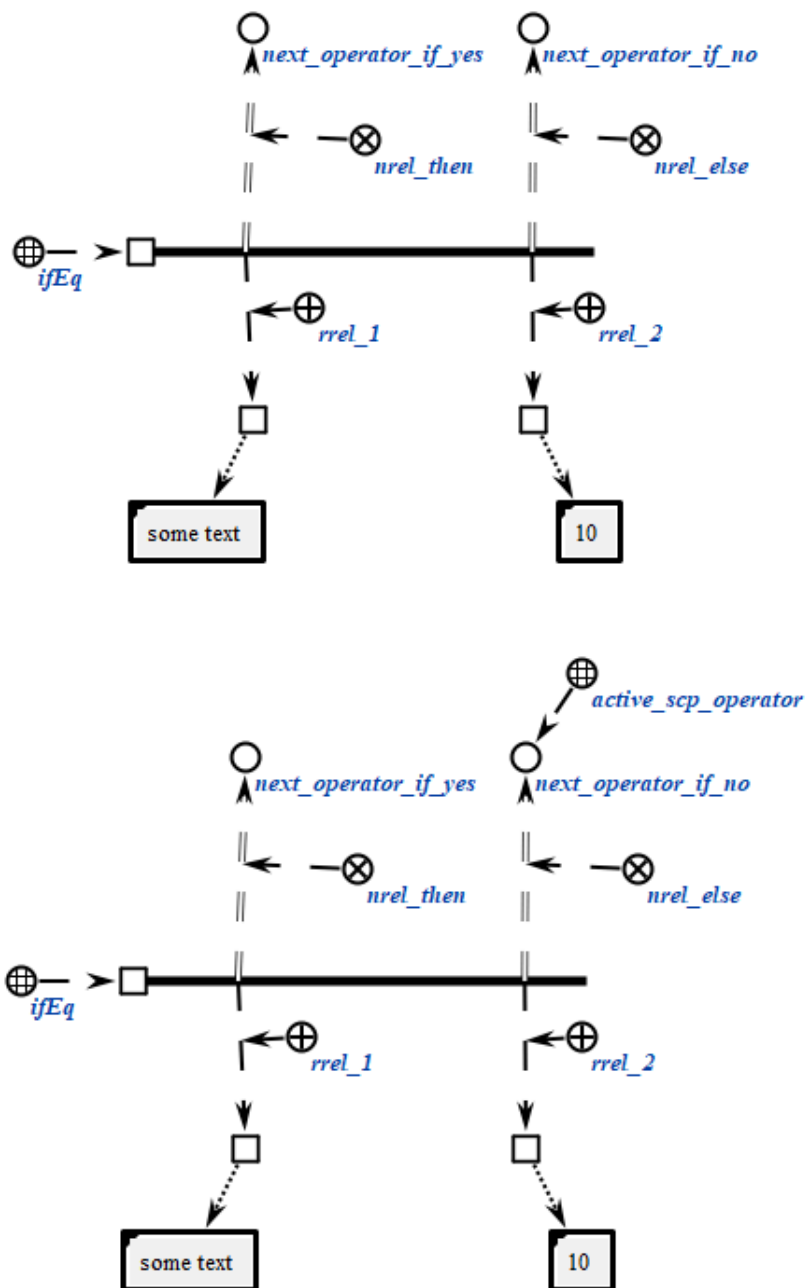
Пример scp-оператора данного класса на SCs-коде представлен ниже:

```

scp_operator_example_ifEq (*
  <-_ ifEq;;
  _-> rrel_1:: _node1;;
  _-> rrel_2:: _node2;;
  _=> nrel_then:: next_operator_if_yes;;
  _=> nrel_else:: next_operator_if_no;;
*);;

```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



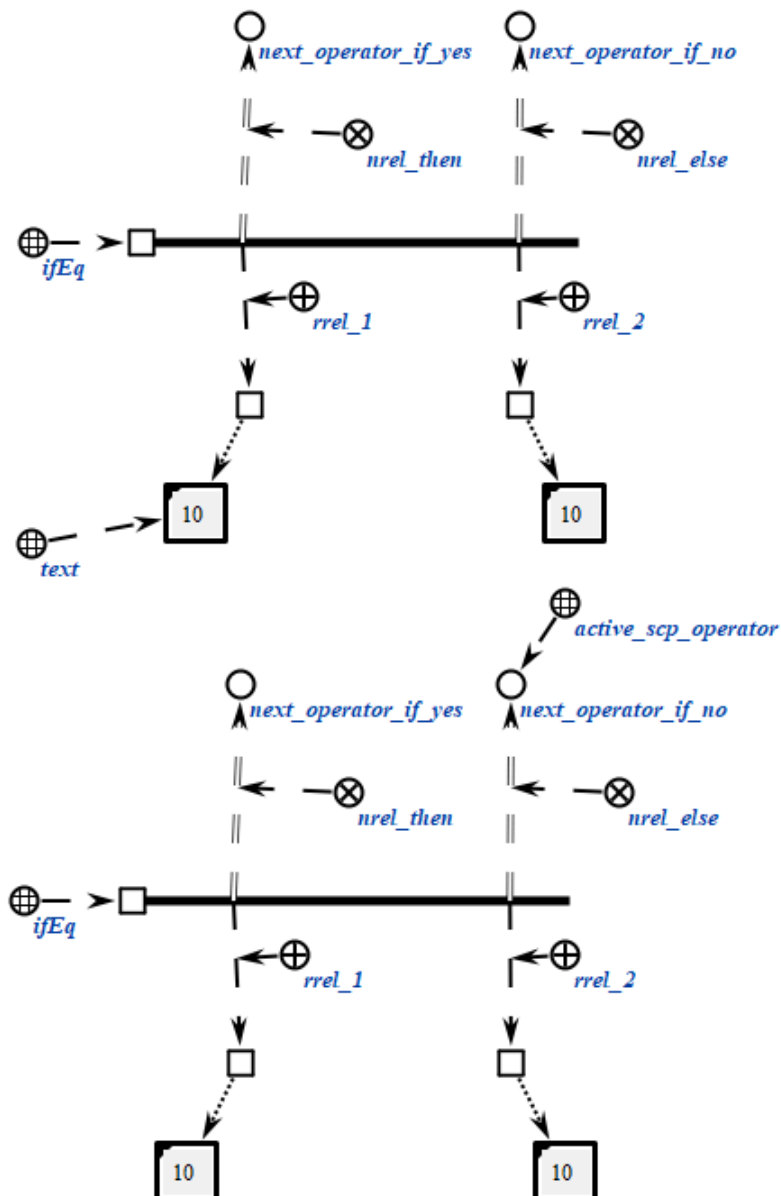
Пример scp-оператора данного класса на SCs-коде представлен ниже:

```

scp_operator_example_ifEq (*
  <-_ ifEq;;
  _-> rrel_1:: _node1;;
  _-> rrel_2:: _node2;;
  _=> nrel_then:: next_operator_if_yes;;
  _=> nrel_else:: next_operator_if_no;;
*);;

```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



6.6.6.1.4.6 scp-оператор сравнения числовых содержимых файлов

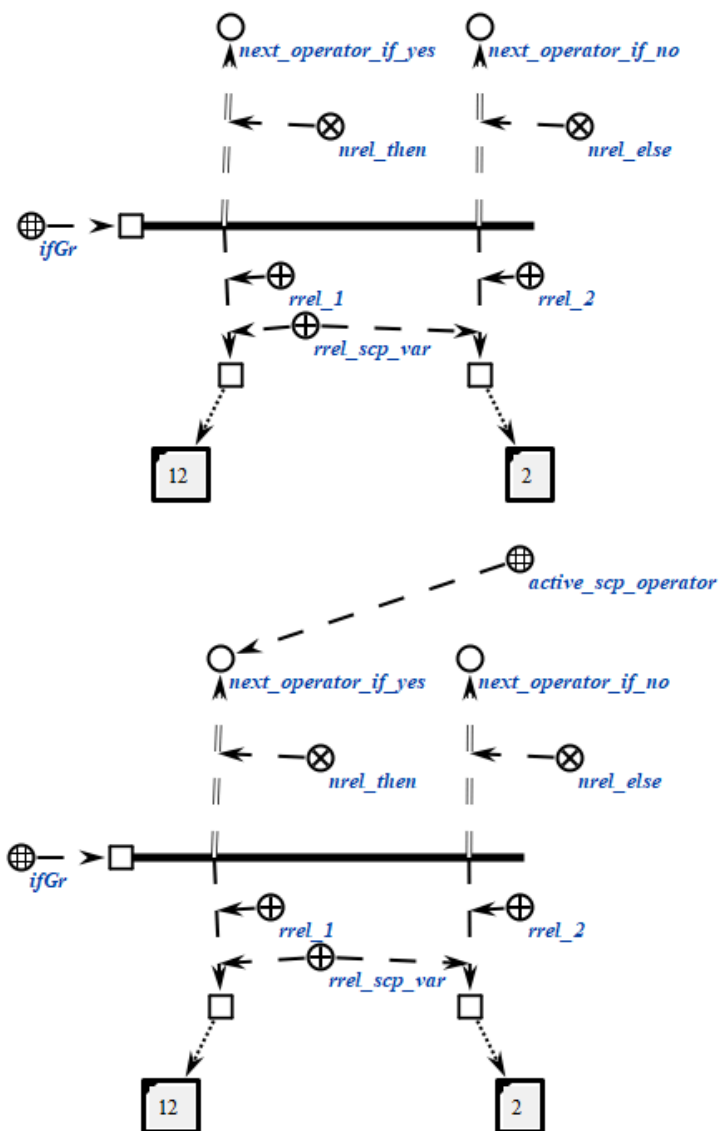
scp-операторы класса *scp-оператор сравнения числовых содержимых файлов* описывают действие сравнения между собой числовых содержимых двух *файлов*, являющихся значениями scp-операндов. Под числовым содержимым понимаются те же виды файлов, что и в случае с *scp-операторами обработки содержимых файлов*. Успешным выполнение считается, значение первого scp-операнда строго больше значения второго scp-операнда с точки зрения теории чисел. Например, число «1e3» больше, чем число «987».

Каждый scp-оператор данного класса содержит два scp-операнда, которые должны быть помечены как *scp-операнд с заданным значением'*. Каждый из scp-операндов может быть как *scp-константой'*, так и *scp-переменной'*.

Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_ifGr (*  
    <-_ ifGr;;  
    _-> rrel_1:: rrel_scp_var:: _node1;;  
    _-> rrel_2:: rrel_scp_var:: _node2;;  
    _=> nrel_then:: next_operator_if_yes;;  
    _=> nrel_else:: next_operator_if_no;;  
*);;
```

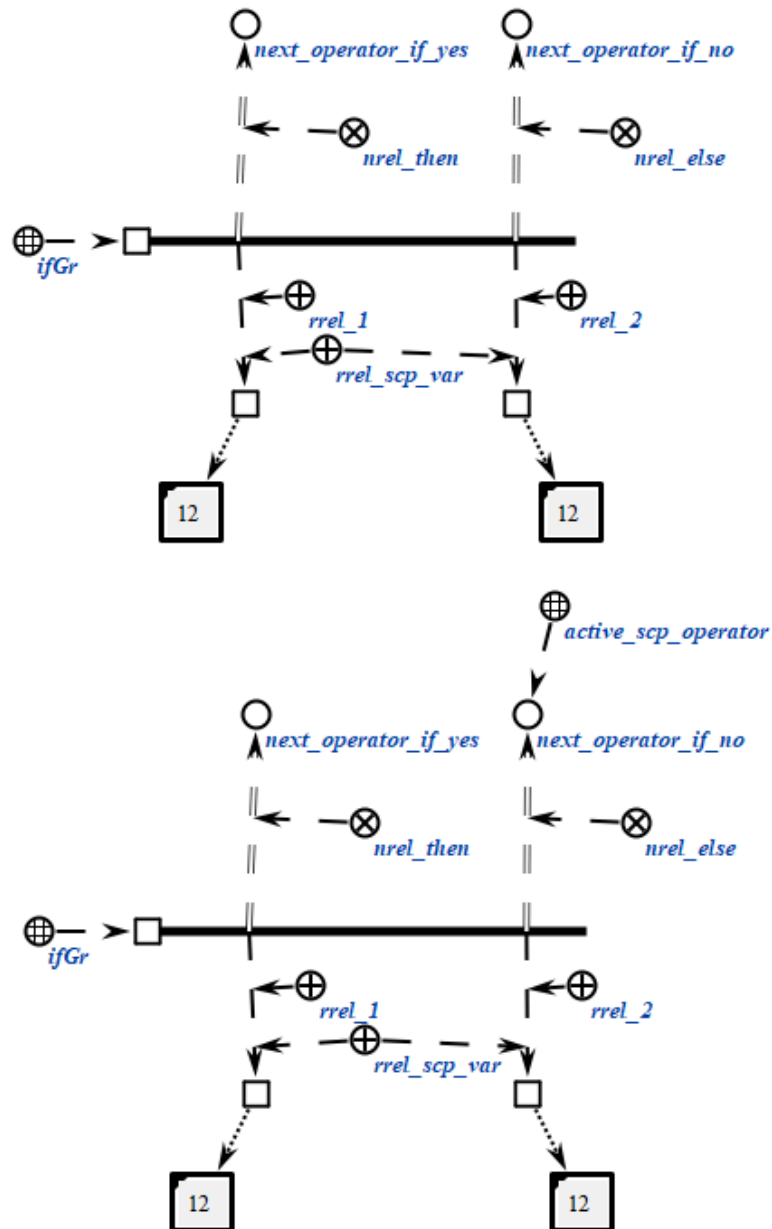
Пример scp-оператора данного класса на SCg-коде представлен ниже:



Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_ifGr (*
  <-_ ifGr;;
  _-> rrel_1:: rrel_scp_var:: _node1;;
  _-> rrel_2:: rrel_scp_var:: _node2;;
  _=> nrel_then:: next_operator_if_yes;;
  _=> nrel_else:: next_operator_if_no;;
*);;
```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



6.6.6.6.1.5 scp-операторы обработки содержимых файлов

scp-операторы класса *scp-оператор* обработки содержимых файлов описывают различные действия, связанные с обработкой содержимых файлов. Большинство scp-операторов данного класса, за исключением *scp-операторов* копирования содержимого файла и *scp-операторов* удаления содержимого файла работают с числовыми содержимыми. Под числовым содержимым подразумевается файл, являющийся идентификатором некоторого действительного числа в десятичной системе счисления. При этом допускаются общепринятые форматы подобной идентификации, например, корректными считаются следующие идентификаторы: «4», «4.555», «6.022e23», «6.63e-34»,

«4e-7». Идентификаторы в других системах счисления либо идентификаторы, использующие какие-либо специальные обозначения, не допускаются.

sqr-операторы класса не предполагают ветвления программы в зависимости от выполнения дополнительных условий, поэтому указание следующих sqr-операторов осуществляется только при помощи отношения *следующий sqr-оператор**, без подмножеств.

Данный класс *sqr-операторов* разбивается на следующие подклассы:

- *sqr-оператор сложения числовых содержимых файлов;*
- *sqr-оператор вычитания числовых содержимых файлов;*
- *sqr-оператор умножения числовых содержимых файлов;*
- *sqr-оператор деления числовых содержимых файлов;*
- *sqr-оператор возведения числового содержимого файлов в степень;*
- *sqr-оператор вычисления логарифма числового содержимого файлов;*
- *sqr-оператор вычисления синуса числового содержимого файлов;*
- *sqr-оператор вычисления косинуса числового содержимого файлов;*
- *sqr-оператор вычисления тангенса числового содержимого файлов;*
- *sqr-оператор вычисления арксинуса числового содержимого файлов;*
- *sqr-оператор вычисления арккосинуса числового содержимого файлов;*
- *sqr-оператор вычисления арктангенса числового содержимого файлов;*
- *sqr-оператор копирования содержимого файлов;*
- *sqr-оператор удаления содержимого файлов;*
- *sqr-оператор перевода в нижний регистр строкового содержимого файла;*
- *sqr-оператор перевода в верхний регистр строкового содержимого файла;*
- *sqr-оператор замены определенной части строкового содержимого файла на содержимое указанного файла;*
- *sqr-оператор проверки совпадения конца строкового содержимого файла со строковым содержимым другого файла;*
- *sqr-оператор проверки совпадения начальной части строкового содержимого файла со строковым содержимым другого файла;*
- *sqr-оператор получения части строкового содержимого файла по индексам;*
- *sqr-оператор поиска строкового содержимого файла в строковом содержимом другого файла;*
- *sqr-оператор вычисления длины строкового содержимого файла;*

- *scp-оператор разбиения строки на подстроки;*
- *scp-оператор лексикографического сравнения строковых содержимых файлов;*
- *scp-оператор проверки равенства строковых содержимых файлов.*

6.6.6.6.1.5.1 scp-оператор сложения числовых содержимых файлов

scp-операторы класса *scp-оператор сложения числовых содержимых файлов* описывают действие сложения между собой числовых содержимых двух *файлов*, являющихся значениями второго и третьего scp-операндов.

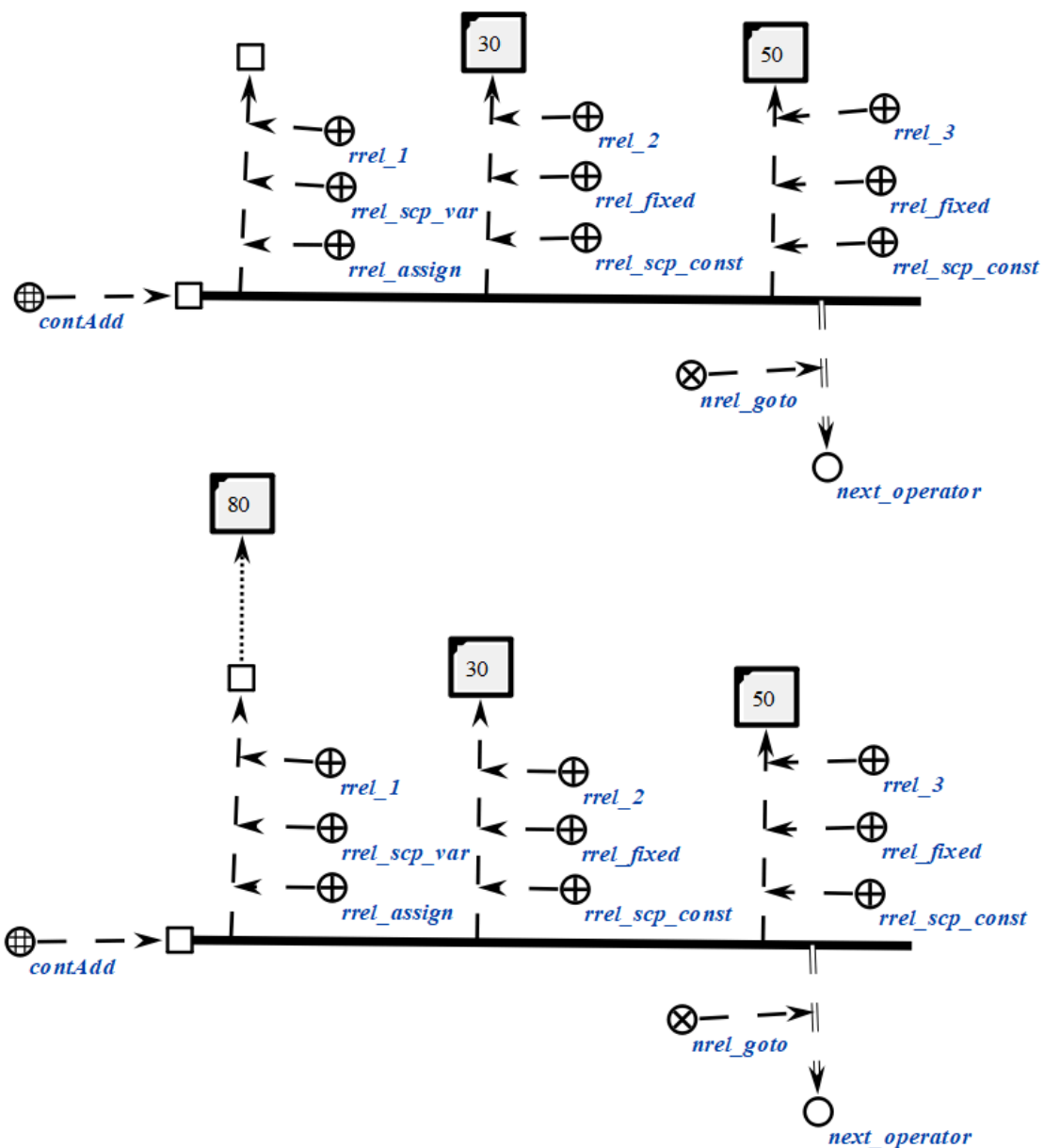
Каждый scp-оператор данного класса содержит три scp-операнда.

Первый scp-операнд может иметь произвольный *тип значения scp-операнда'*. Значением данного scp-операнда является *файл*, идентифицирующий число, являющееся суммой чисел, задаваемых вторым и третьим scp-операндами. В случае, если данный scp-операнд помечен как *scp-операнд со свободным значением'*, то *файл* будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося *файла*.

Второй scp-операнд должен быть помечен как *scp-операнд с заданным значением'*. Значением данного scp-операнда является *файл*, идентифицирующий число, являющееся одним из слагаемых при сложении.

Третий scp-операнд должен быть помечен как *scp-операнд с заданным значением'*. Значением данного scp-операнда является *файл*, идентифицирующая число, являющееся одним из слагаемых при сложении.

```
scp_operator_example_contAdd (*
    <_ contAdd;;
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _e11;;
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [30];;
    _-> rrel_3:: rrel_fixed:: rrel_scp_const:: [50];;
    _=> nrel_goto:: next_operator;;
*);;
```



6.6.6.6.1.5.2 scp-оператор вычитания числовых содержимых файлов

scp-операторы класса *scp-оператор* вычитания числовых содержимых файлов описывают действие вычитания числовых содержимых двух файлов, являющихся значениями второго и третьего scp-операндов.

Каждый scp-оператор данного класса содержит три scp-операнда.

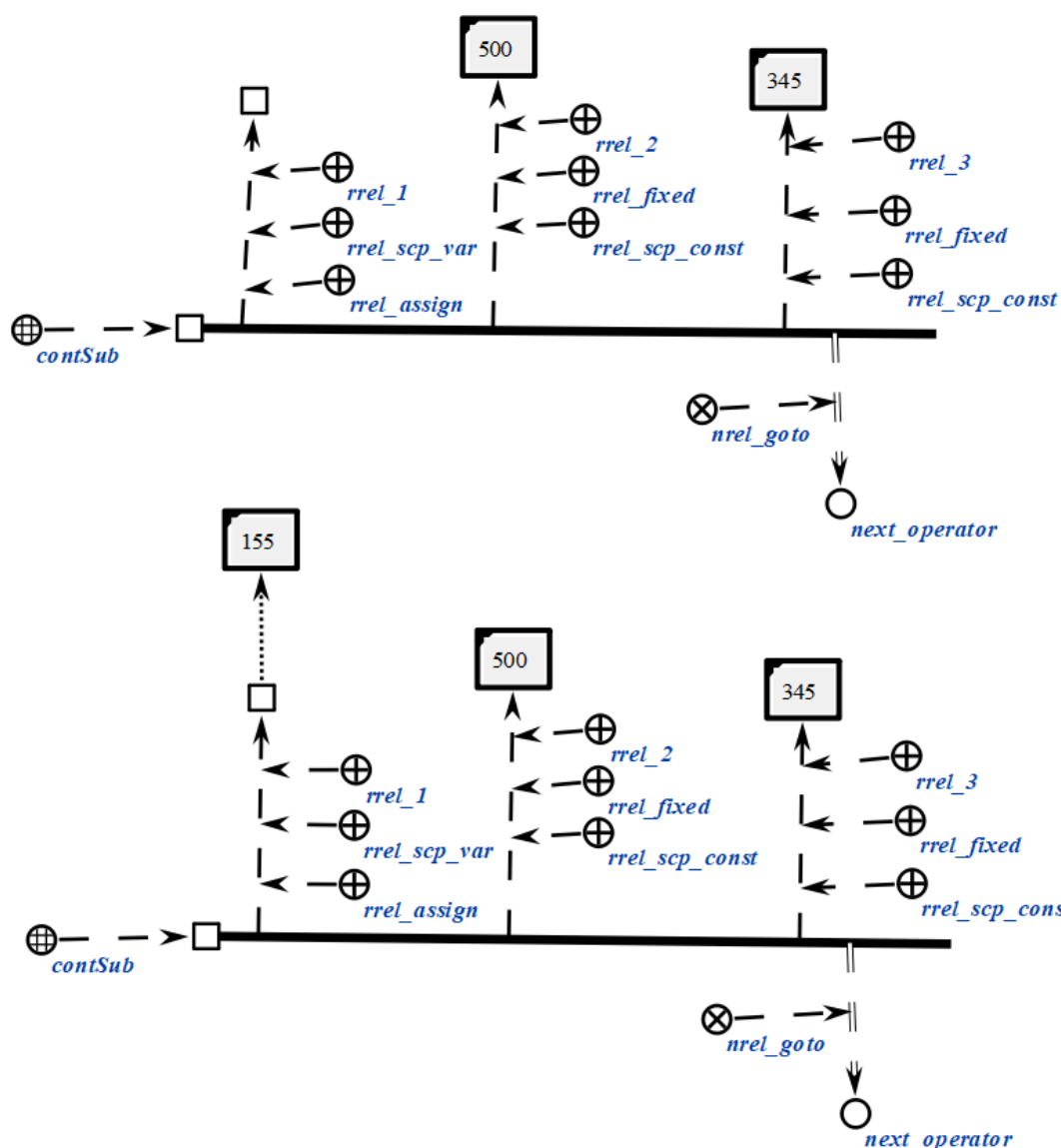
Первый scp-операнд может иметь произвольный *тип значения scp-операнда*'. Значением данного scp-операнда является *файл*, идентифицирующий число, являющееся разностью чисел, задаваемых вторым и третьим scp-операндами. В случае, если данный scp-операнд помечен как

scp-операнд со свободным значением', то *файл* будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося *файла*.

Второй *scp-операнд* должен быть помечен как *scp-операнд с заданным значением'*. Значением данного *scp-операнда* является *файл*, идентифицирующий число, являющееся уменьшаемым при вычитании.

Третий *scp-операнд* должен быть помечен как *scp-операнд с заданным значением'*. Значением данного *scp-операнда* является *файл*, идентифицирующая число, являющееся вычитаемым при сложении.

```
scp_operator_example_contSub (*  
    <=_ contSub;;  
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _e11;;  
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [500];;  
    _-> rrel_3:: rrel_fixed:: rrel_scp_const:: [345];;  
    _=> nrel_goto:: next_operator;;  
*);;
```



6.6.6.6.1.5.3 scp-оператор умножения числовых содержимых файлов

scp-операторы класса *scp-оператор умножения числовых содержимых файлов* описывают действие умножения между собой числовых содержимых двух *файлов*, являющихся значениями второго и третьего scp-операндов.

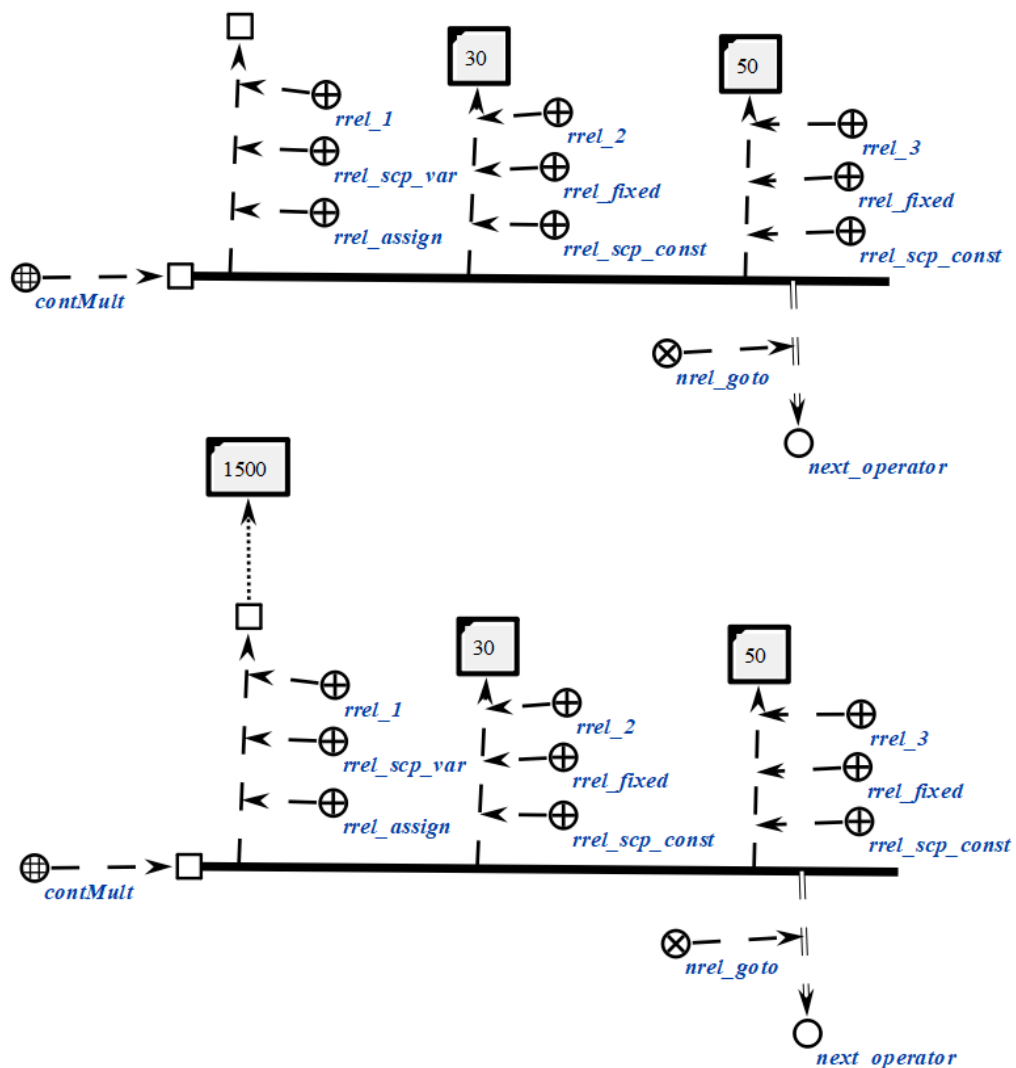
Каждый scp-оператор данного класса содержит три scp-операнда.

Первый scp-операнд может иметь произвольный *тип значения scp-операнда*'. Значением данного scp-операнда является *файл*, идентифицирующий число, являющееся произведением чисел, задаваемых вторым и третьим scp-операндами. В случае, если данный scp-операнд помечен как *scp-операнд со свободным значением*', то *файл* будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося *файла*.

Второй scp-операнд должен быть помечен как *scp-операнд с заданным значением*'. Значением данного scp-операнда является файл, идентифицирующий число, являющееся одним из множителей при произведении.

Третий scp-операнд должен быть помечен как *scp-операнд с заданным значением*'. Значением данного scp-операнда является файл, идентифицирующий число, являющееся одним из множителей при произведении.

```
scp_operator_example_contMult (*
  <-_ contMult;;
  _-> rrel_1:: rrel_assign:: rrel_scp_var:: _ell;;
  _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [30];;
  _-> rrel_3:: rrel_fixed:: rrel_scp_const:: [50];;
  _=> nrel_goto:: next_operator;;
*);;
```



6.6.6.6.1.5.4 scp-оператор деления числовых содержимых файлов

scp-операторы класса *scp-оператор деления числовых содержимых файлов* описывают действие деления числовых содержимых двух *файлов*, являющихся значениями второго и третьего scp-операндов.

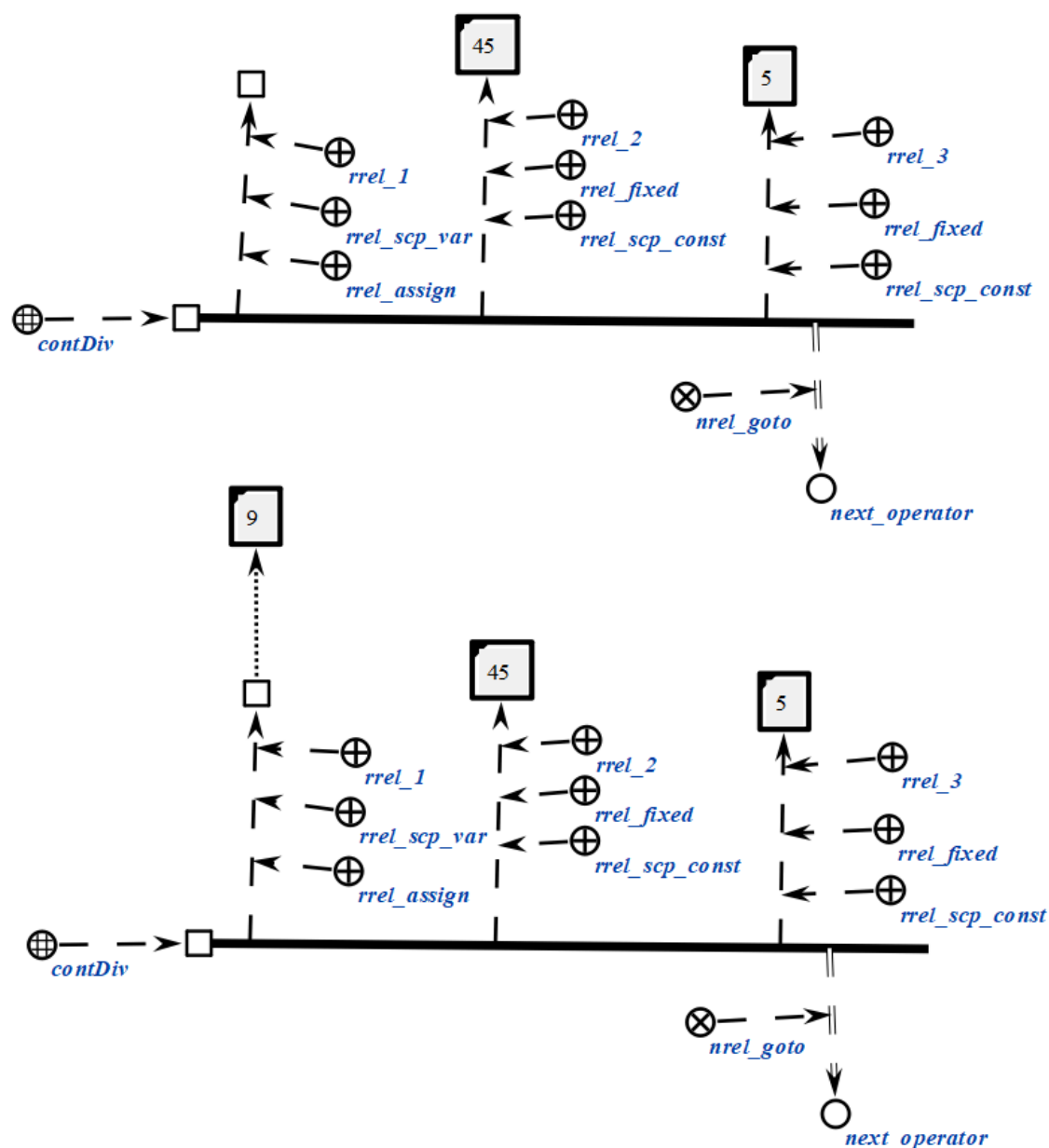
Каждый scp-оператор данного класса содержит три scp-операнда.

Первый scp-операнд может иметь произвольный *тип значения scp-операнда*'. Значением данного scp-операнда является *файл*, идентифицирующий число, являющееся частным чисел, задаваемых вторым и третьим scp-операндами. В случае, если данный scp-операнд помечен как *scp-операнд со свободным значением*', то *файл* будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося *файла*.

Второй scp-операнд должен быть помечен как *scp-операнд с заданным значением*'. Значением данного scp-операнда является *файл*, идентифицирующий число, являющееся делимым при делении.

Третий scp-операнд должен быть помечен как *scp-операнд с заданным значением*'. Значением данного scp-операнда является *файл*, идентифицирующий число, являющееся делителем при делении.

```
scp_operator_example_contDiv (*
    <=_ contDiv;;
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _el1;;
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [45];;
    _-> rrel_3:: rrel_fixed:: rrel_scp_const:: [5];;
    _=> nrel_goto:: next_operator;;
*) ;;
```

6.6.6.6.1.5.5 scp-оператор возведения числового содержимого файла

scp-операторы класса *scp-оператор возведения числового содержимого файла в степень* описывают действие возведения в степень числового содержимого файла, являющегося значением второго scp-операнда.

Каждый scp-оператор данного класса содержит три scp-операнда.

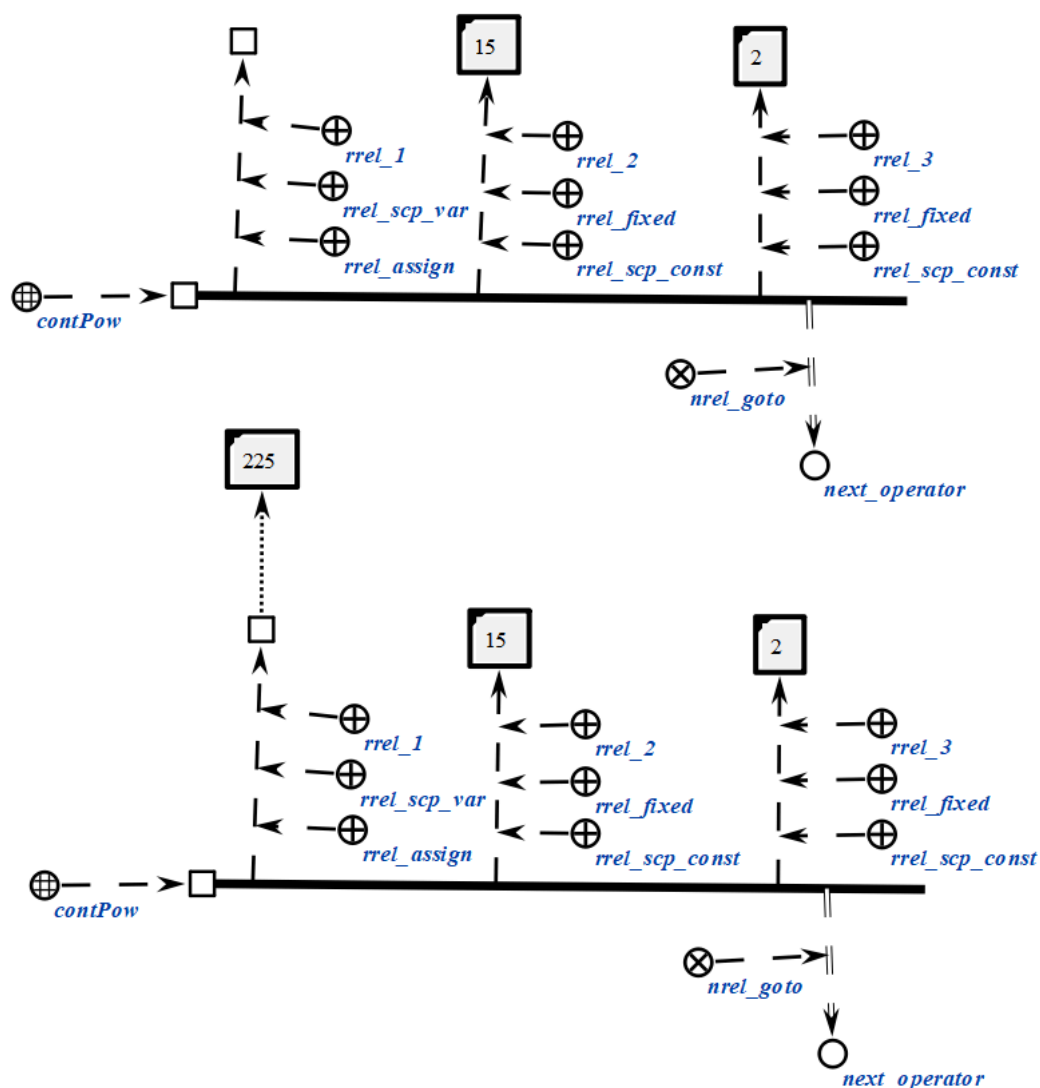
Первый scp-операнд может иметь произвольный *тип значения scp-операнда*'. Значением данного scp-операнда является файл, идентифицирующая число, являющееся результатом возведения числа, задаваемого вторым scp-операндом, в степень, задаваемую третьим scp-операндом. В случае, если данный scp-операнд помечен как *scp-операнд со*

свободным значением', то *файл* будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося *файла*.

Второй *scp-операнд* должен быть помечен как *scp-операнд с заданным значением'*. Значением данного *scp-операнда* является *файл*, идентифицирующий число, являющееся основанием при возведении в степень.

Третий *scp-операнд* должен быть помечен как *scp-операнд с заданным значением'*. Значением данного *scp-операнда* является *файл*, идентифицирующий число, являющееся показателем при возведении в степень.

```
scp_operator_example_contPow (*  
    <_ contPow;;  
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _el1;;  
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [15];;  
    _-> rrel_3:: rrel_fixed:: rrel_scp_const:: [2];;  
    _=> nrel_goto:: next_operator;;  
*);;
```



6.6.6.6.1.5.6 scp-оператор вычисления логарифма числового содержимого файла

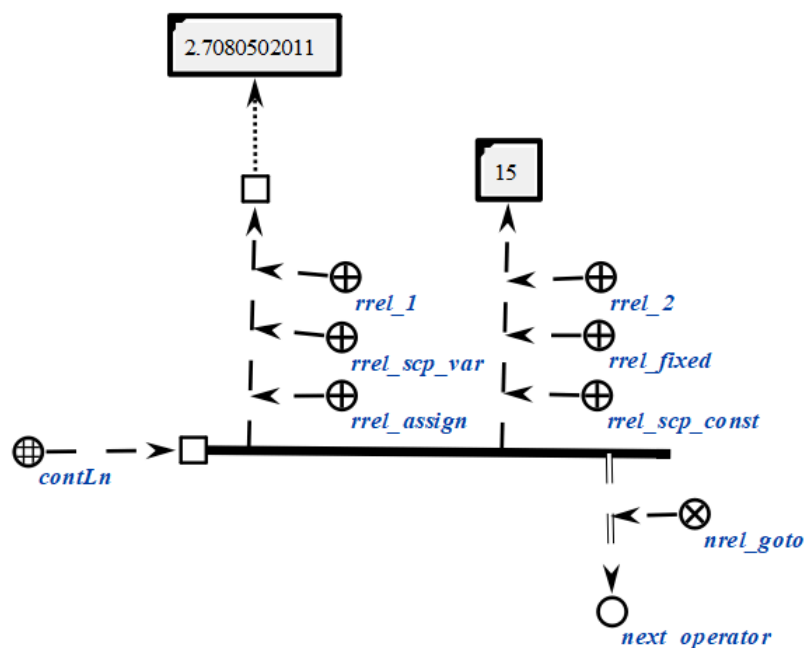
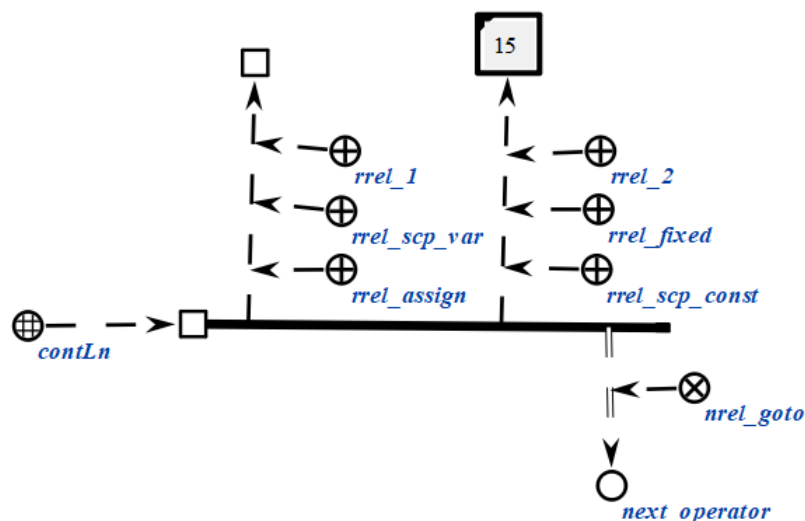
scp-операторы класса *scp-оператор вычисления логарифма числового содержимого файла* описывают действие вычисления натурального логарифма числового содержимого *файла*, являющегося значением второго scp-операнда.

Каждый scp-оператор данного класса содержит два scp-операнда.

Первый scp-операнд может иметь произвольный *тип значения scp-операнда'*. Значением данного scp-операнда является *файл*, идентифицирующий число, являющееся логарифмом числа, задаваемого вторым scp-операндом. В случае, если данный scp-операнд помечен как *scp-операнд со свободным значением'*, то *файл* будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося *файла*.

Второй *scr*-операнд должен быть помечен как *scr-операнд с заданным значением*'. Значением данного *scr*-операнда является *файл*, идентифицирующий число, являющееся аргументом функции логарифма. При этом необходимо следить, чтобы численное содержимое данного *scr*-операнда идентифицировало число, попадающее в область определения функции логарифма с точки зрения классической алгебры.

```
scp_operator_example_contLn (*
    <=_ contLn;;
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _ell;;
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [15];;
    _=> nrel_goto:: next_operator;;
*);;
```



6.6.6.1.5.7 scp-оператор вычисления синуса числового содержимого файла

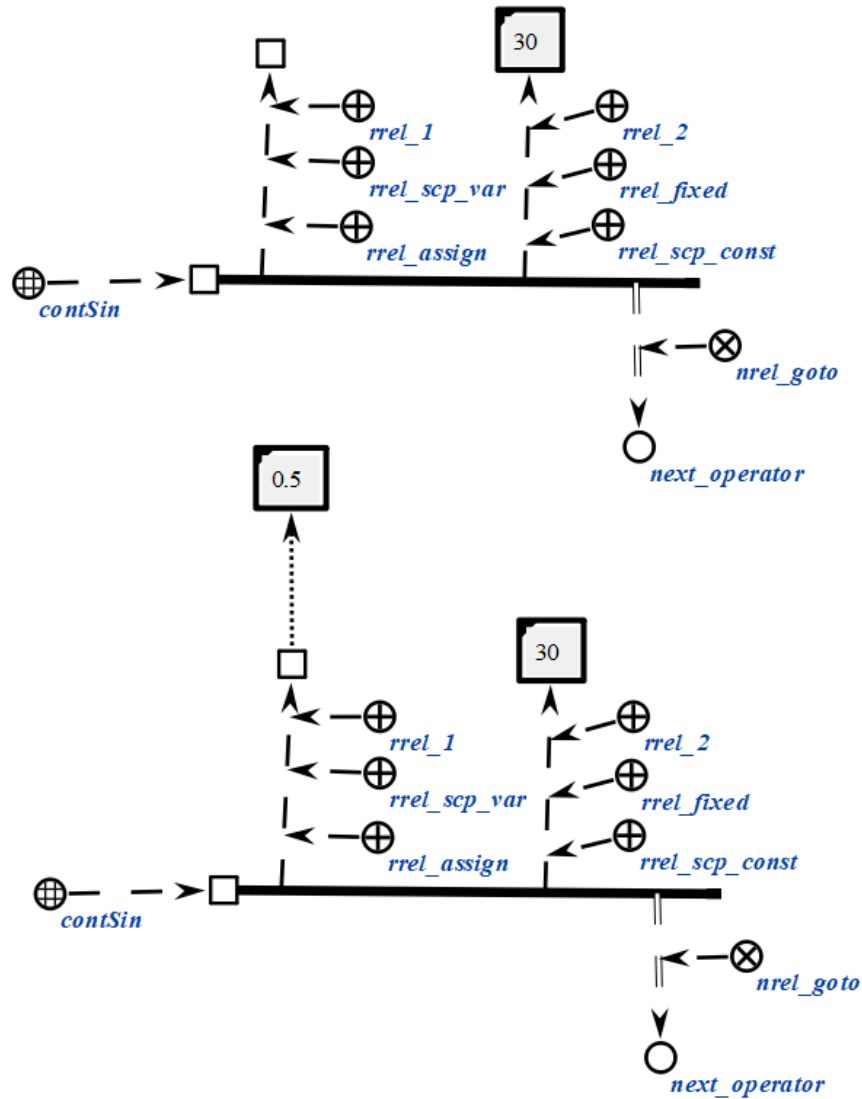
scp-операторы класса *scp-оператор вычисления синуса числового содержимого файла* описывают действие вычисления синуса числового содержимого *файла*, являющегося значением второго scp-операнда.

Каждый scp-оператор данного класса содержит два scp-операнда.

Первый scp-операнд может иметь произвольный *тип значения scp-операнда'*. Значением данного scp-операнда является *файл*, идентифицирующий число, являющееся синусом числа, задаваемого вторым scp-операндом. В случае, если данный scp-операнд помечен как *scp-операнд со свободным значением'*, то *файл* будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося *файла*.

Второй scp-операнд должен быть помечен как *scp-операнд с заданным значением'*. Значением данного scp-операнда является *файл*, идентифицирующий число, являющееся аргументом функции синуса. Предполагается, что указанное число соответствует значению угла в радианах.

```
scp_operator_example_contSin (*
    <_ contSin;;
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _el1;;
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [30];;
    _=> nrel_goto:: next_operator;;
*);;
```



6.6.6.6.1.5.8 scp-оператор вычисления косинуса числового содержимого файла

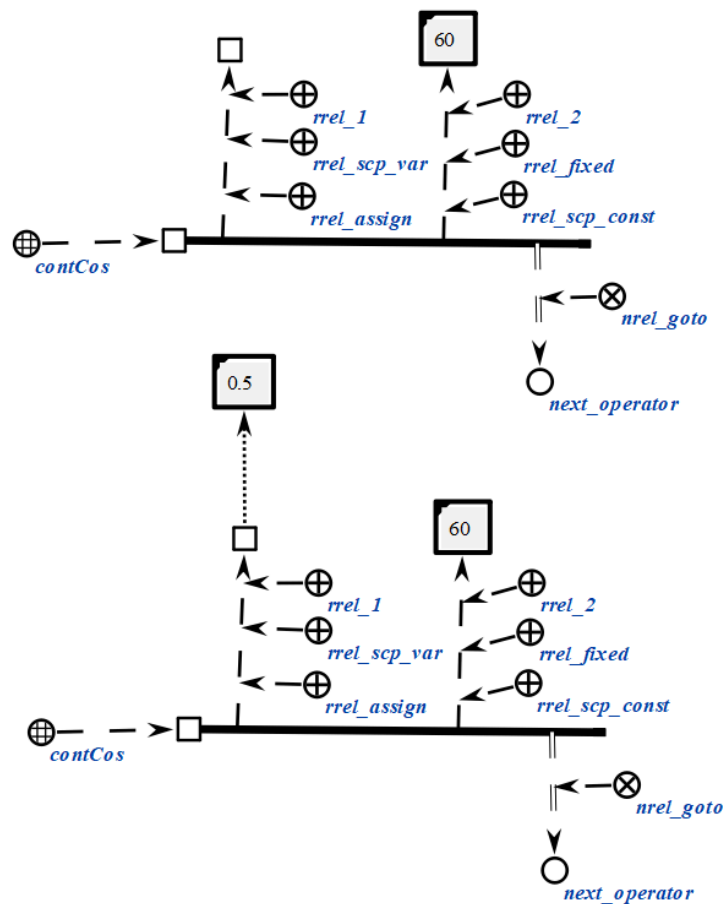
scp-операторы класса *scp-оператор вычисления косинуса числового содержимого файла* описывают действие вычисления косинуса числового содержимого файла, являющегося значением второго scp-операнда.

Каждый scp-оператор данного класса содержит два scp-операнда.

Первый scp-операнд может иметь произвольный тип значения *scp-операнда'*. Значением данного scp-операнда является файл, идентифицирующий число, являющееся косинусом числа, задаваемого вторым scp-операндом. В случае, если данный scp-операнд помечен как *scp-операнд со свободным значением'*, то файл будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося файла.

Второй scp-операнд должен быть помечен как *scp-операнд с заданным значением*'. Значением данного scp-операнда является *файл*, идентифицирующий число, являющееся аргументом функции косинуса. Предполагается, что указанное число соответствует значению угла в радианах.

```
scp_operator_example_contCos (*
    <-_ contCos;;
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _el1;;
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [60];;
    _=> nrel_goto:: next_operator;;
*);;
```



6.6.6.6.1.5.9 scp-оператор вычисления тангенса числового содержимого файла

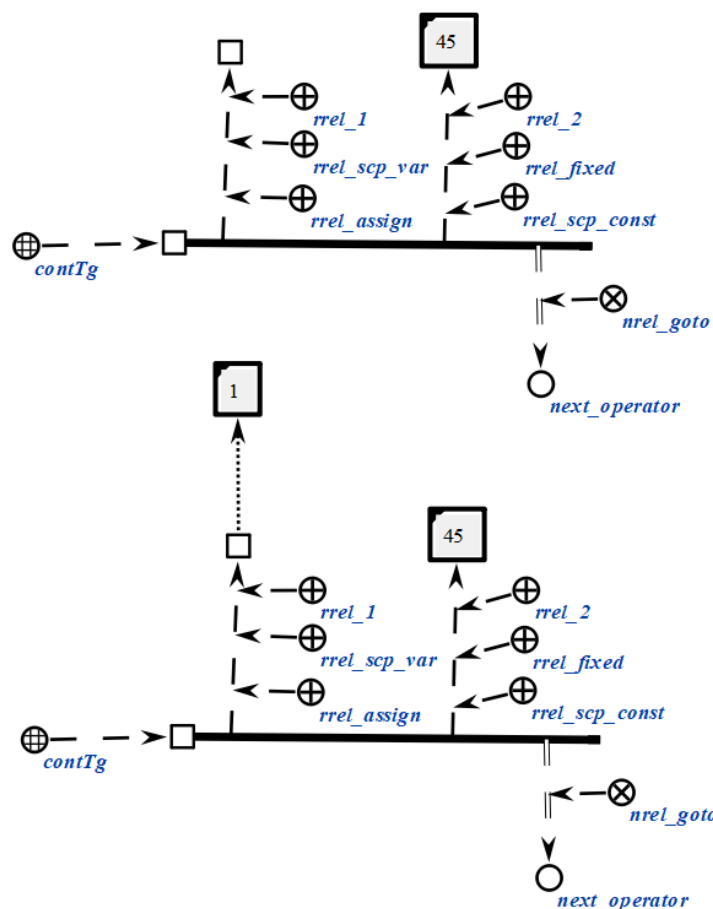
scp-операторы класса *scp-оператор вычисления тангенса числового содержимого файла* описывают действие вычисления тангенса числового содержимого *файла*, являющегося значением второго scp-операнда.

Каждый scp-оператор данного класса содержит два scp-операнда.

Первый scp-операнд может иметь произвольный *тип значения scp-операнда*'. Значением данного scp-операнда является *файл*, идентифицирующий число, являющееся тангенсом числа, задаваемого вторым scp-операндом. В случае, если данный scp-операнд помечен как *scp-операнд со свободным значением*', то *файл* будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося *файла*.

Второй scp-операнд должен быть помечен как *scp-операнд с заданным значением*'. Значением данного scp-операнда является *файл*, идентифицирующий число, являющееся аргументом функции тангенса. Предполагается, что указанное число соответствует значению угла в радианах.

```
scp_operator_example_contTg (*
    <-_ contTg;;
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _ell;;
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [45];;
    _=> nrel_goto:: next_operator;;
*);;
```



6.6.6.6.1.5.10 scp-оператор вычисления арксинуса числового содержимого файла

scp-операторы класса *scp-оператор вычисления арксинуса числового содержимого файла* описывают действие вычисления арксинуса числового содержимого *файла*, являющегося значением второго scp-операнда.

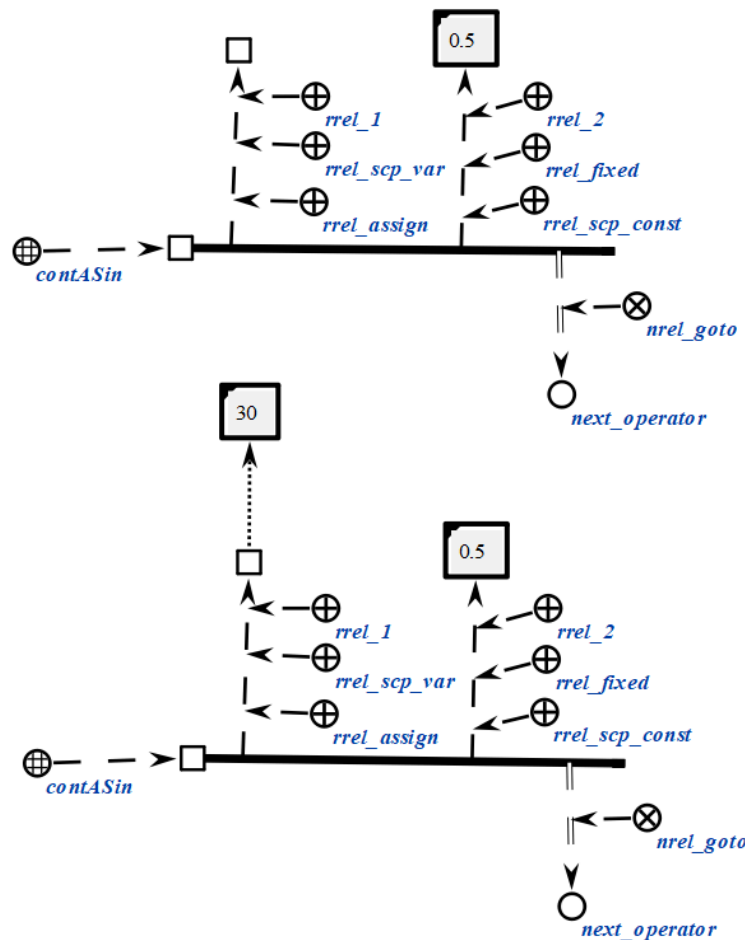
Каждый scp-оператор данного класса содержит два scp-операнда.

Первый scp-операнд может иметь произвольный *тип значения scp-операнда*'. Значением данного scp-операнда является *файл*, идентифицирующий число, являющееся арксинусом числа, задаваемого вторым scp-операндом. В случае, если данный scp-операнд помечен как *scp-операнд со свободным значением*', то *файл* будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося *файла*.

Второй scp-операнд должен быть помечен как *scp-операнд с заданным значением*'. Значением данного scp-операнда является *файл*, идентифицирующий число, являющееся аргументом функции арксинуса. При этом необходимо следить, чтобы численное содержимое данного scp-операнда идентифицировало число, попадающее в область определения функции арксинуса с точки зрения классической алгебры.

Предполагается, что результат вычисления соответствует значению угла в радианах.

```
scp_operator_example_contASin (*  
    <=_ contASin;;  
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _el1;;  
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [0.5];;  
    _=> nrel_goto:: next_operator;;  
*);;
```



6.6.6.6.1.5.11 scp-оператор вычисления арккосинуса числового содержимого файла

scp-операторы класса *scp-оператор вычисления арккосинуса числового содержимого файла* описывают действие вычисления арккосинуса числового содержимого файла, являющегося значением второго scp-операнда.

Каждый scp-оператор данного класса содержит два scp-операнда.

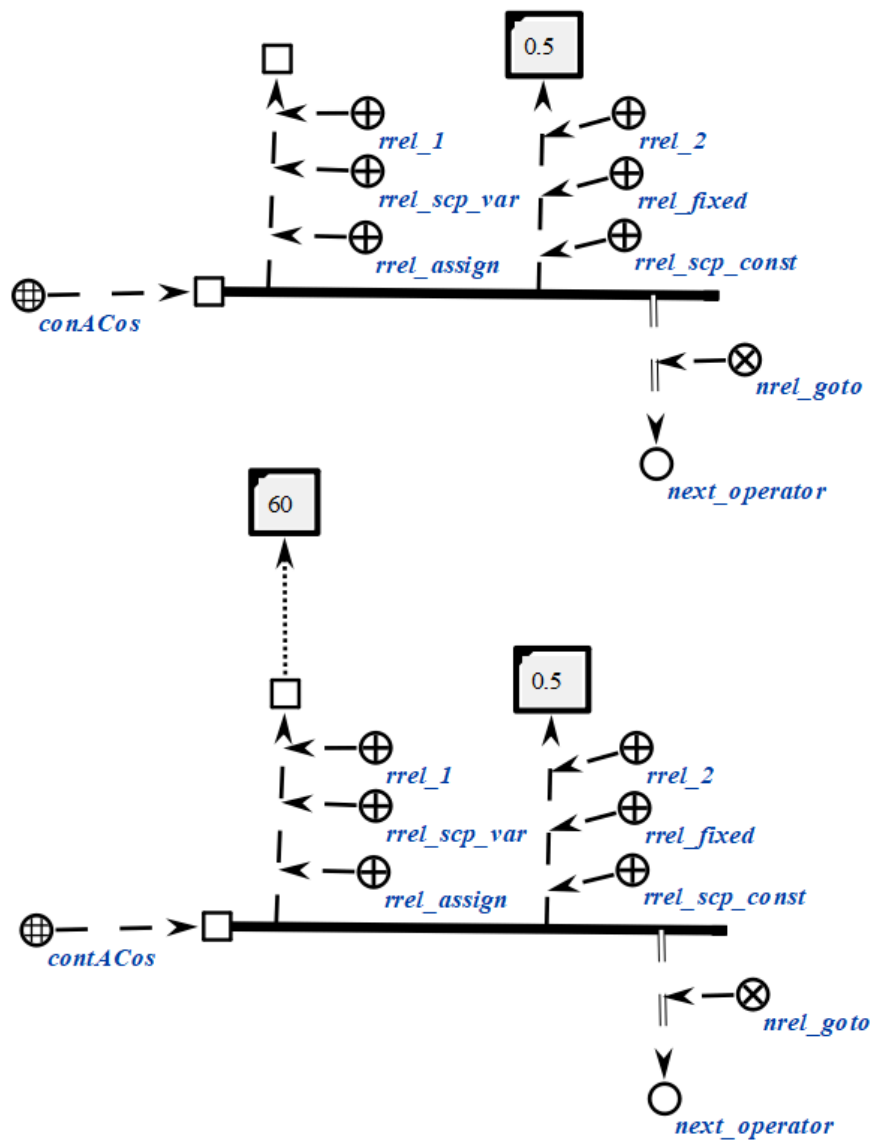
Первый scp-операнд может иметь произвольный тип значения *scp-операнда*'. Значением данного scp-операнда является файл, идентифицирующий число, являющееся арксинусом числа, задаваемого вторым scp-операндом. В случае, если данный scp-операнд помечен как *scp-операнд со свободным значением*', то файл будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося файла.

Второй scp-операнд должен быть помечен как *scp-операнд с заданным значением*'. Значением данного scp-операнда является файл, идентифицирующий число, являющееся аргументом функции арккосинуса. При

этом необходимо следить, чтобы численное содержимое данного scp-операнда идентифицировало число, попадающее в область определения функции арккосинуса с точки зрения классической алгебры.

Предполагается, что результат вычисления соответствует значению угла в радианах.

```
scp_operator_example_contACos (*
  <-_ contACos;;
  _-> rrel_1:: rrel_assign:: rrel_scp_var:: _el1;;
  _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [0.5];;
  _=> nrel_goto:: next_operator;;
*);;
```



6.6.6.6.1.5.12 *scp*-оператор вычисления арктангенса числового содержимого файла

scp-операторы класса *scp-оператор вычисления арктангенса числового содержимого файла* описывают действие вычисления арктангенса числового содержимого *файла*, являющейся значением второго *scp*-операнда.

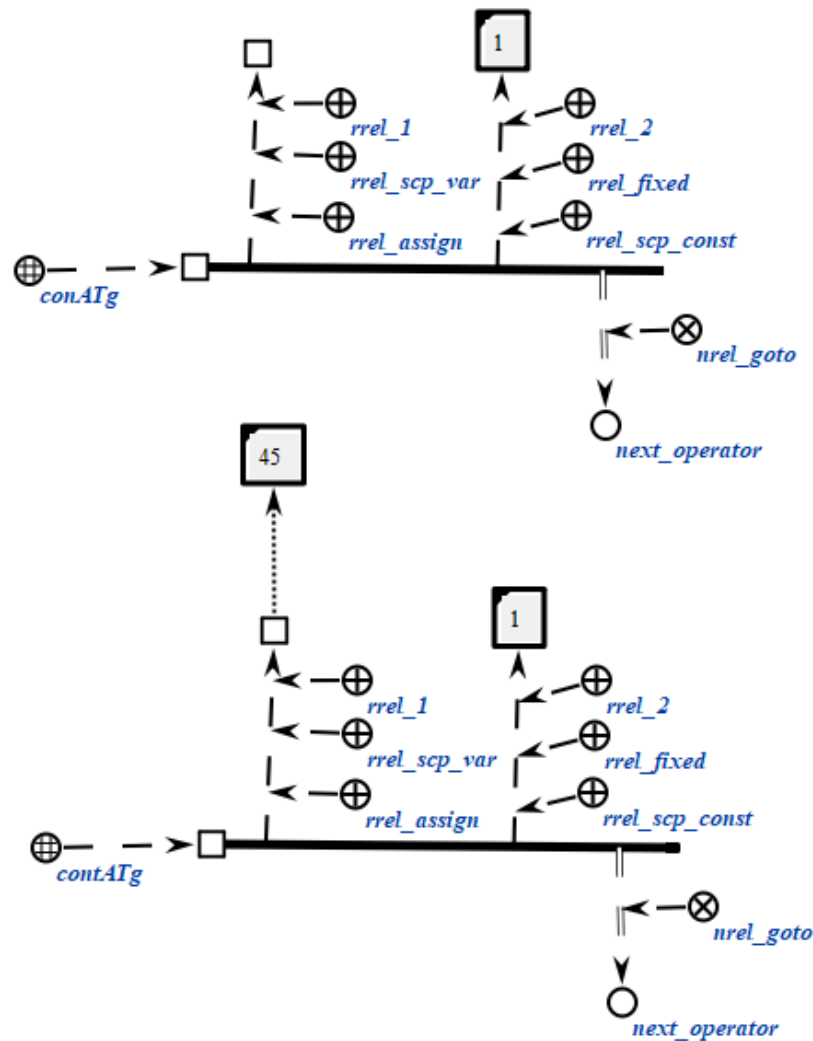
Каждый *scp*-оператор данного класса содержит два *scp*-операнда.

Первый *scp*-операнд может иметь произвольный *тип значения scp-операнда*'. Значением данного *scp*-операнда является *файл*, идентифицирующий число, являющееся арктангенсом числа, задаваемого вторым *scp*-операндом. В случае, если данный *scp*-операнд помечен как *scp-операнд со свободным значением*', то *файл* будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося *файла*.

Второй *scp*-операнд должен быть помечен как *scp-операнд с заданным значением*'. Значением данного *scp*-операнда является *файл*, идентифицирующий число, являющееся аргументом функции арктангенса. При этом необходимо следить, чтобы численное содержимое данного *scp*-операнда идентифицировало число, попадающее в область определения функции арктангенса с точки зрения классической алгебры.

Предполагается, что результат вычисления соответствует значению угла в радианах.

```
scp_operator_example_contATg (*
    <_ contATg;;
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _el1;;
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [1];;
    _=> nrel_goto:: next_operator;;
*);;
```



6.6.6.6.1.5.13 scp-оператор копирования содержимого файла

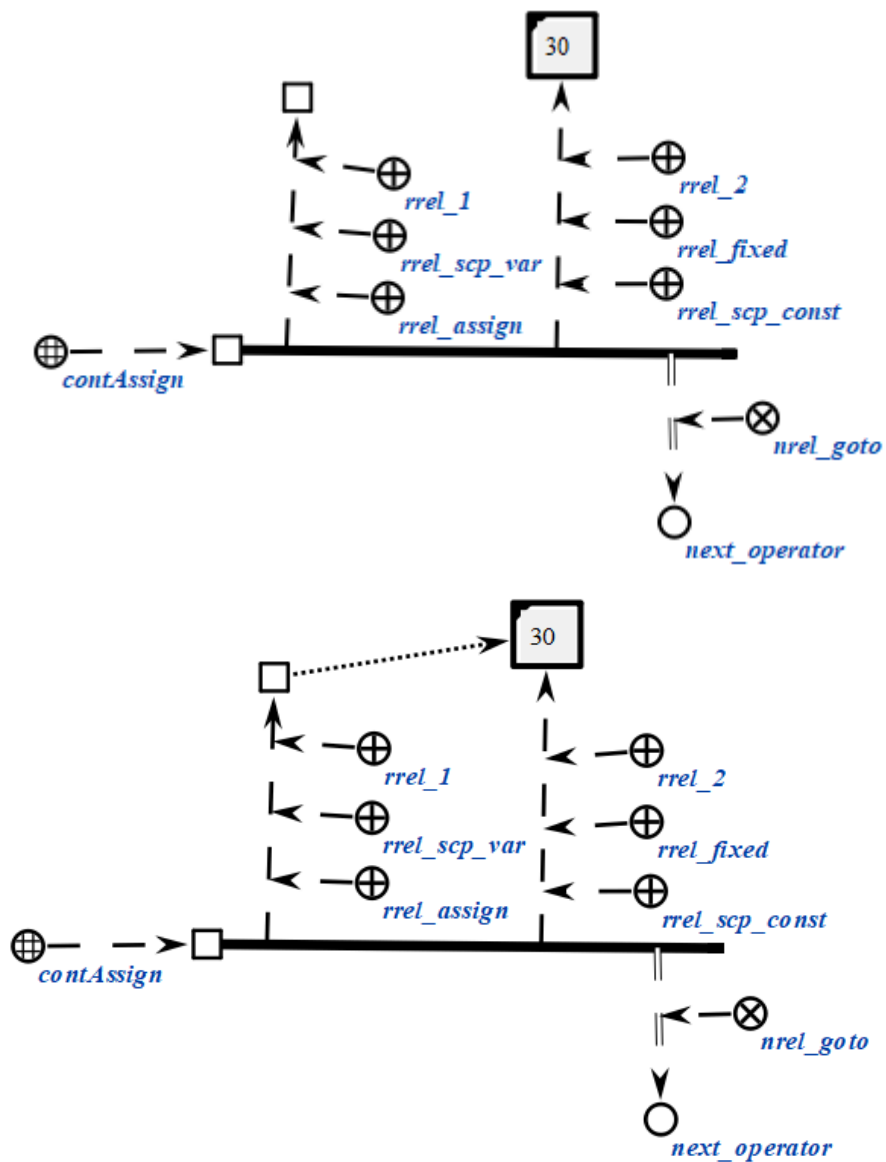
scp-операторы класса *scp-оператор копирования содержимого файла* описывают действие копирования произвольного содержимого одного файла в содержимое другого файла.

Каждый scp-оператор данного класса содержит два scp-операнда.

Первый scp-операнд может иметь произвольный *тип значения scp-операнда'*. Значением данного scp-операнда является *файл*, в который будет записано содержимое в результате копирования. В случае, если данный scp-операнд помечен как *scp-операнд со свободным значением'*, то файл будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося файла.

Второй scp-операнд должен быть помечен как *scp-операнд с заданным значением*'. Значением данного scp-операнда является *файл*, из которого будет взято содержимое для копирования.

```
scp_operator_example_contAssign (*
    <_ contAssign;;
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _el1;;
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [30];;
    _=> nrel_goto:: next_operator;;
*);;
```



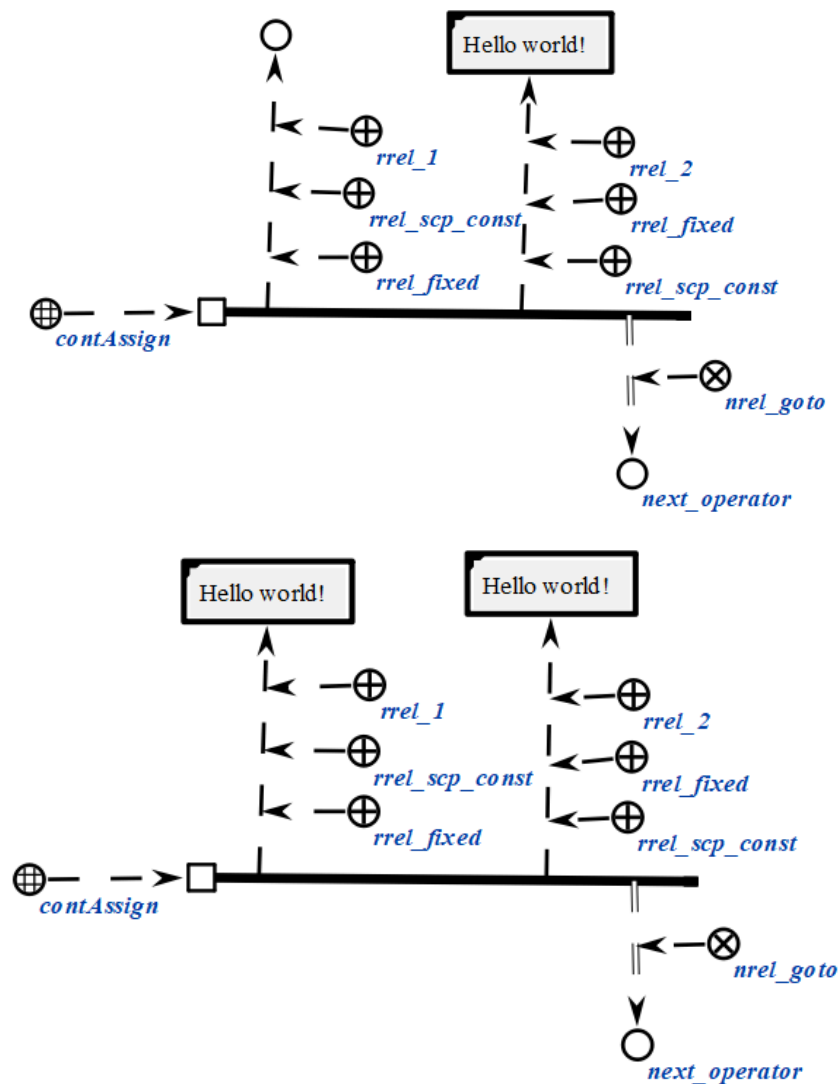
```
scp_operator_example_contAssign (*
    <_ contAssign;;
    _-> rrel_1:: rrel_fixed:: rrel_scp_const:: el1;;
```

```

-> rrel_2:: rrel_fixed:: rrel_scp_const:: [Hello world!];;
_=> nrel_goto:: next_operator;;

*);;

```



6.6.6.6.1.5.14 scp-оператор удаления содержимого файла

scp-операторы класса *scp-оператор* удаления содержимого файла описывает действие удаления произвольного содержимого некоторого файла.

Каждый scp-оператор данного класса содержит один scp-операнд. Данный scp-операнд должен быть помечен как *scp-операнд с заданным значением*. Значением данного scp-операнда является *файл*, содержимое которого будет удалено. После выполнения scp-оператора указанный *файл* будет считаться *файлом*, для которого не сформировано содержимое. При этом сам *файл* не удаляется.

```

scp_operator_example_contErase (*

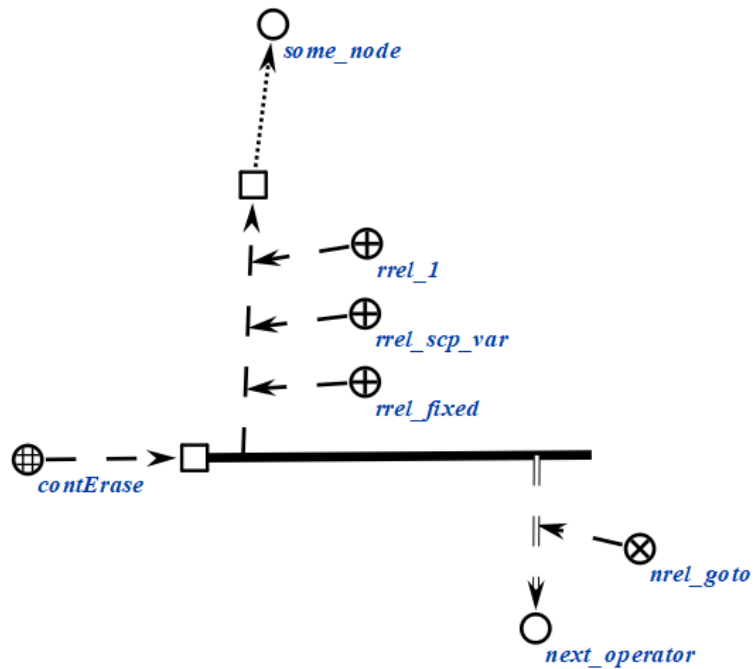
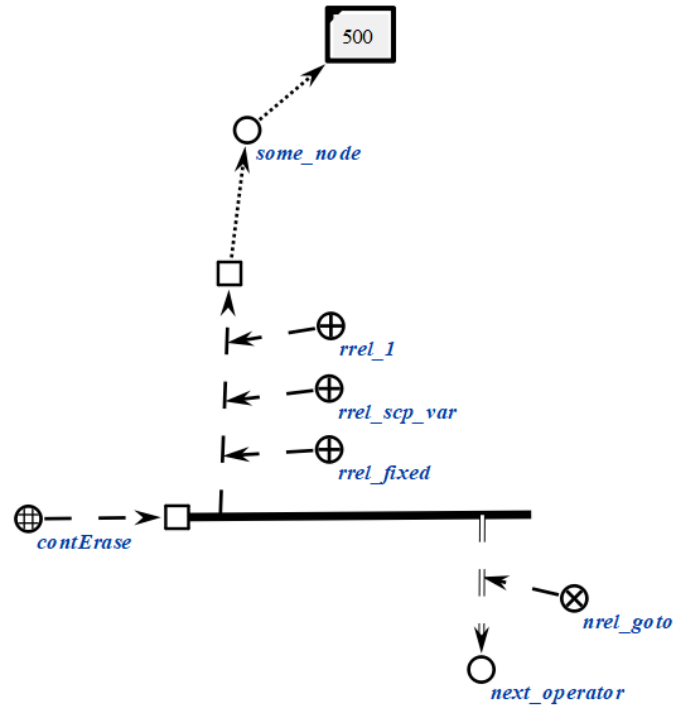
```

```

<-_ contErase;;
_> rrel_1:: rrel_fixed:: rrel_scp_var:: _ell;;
_> nrel_goto:: next_operator;;

*);;

```



```

scp_operator_example_contErase (*
  <-_ contErase;;
  _> rrel_1:: rrel_fixed:: rrel_scp_const:: [Hello world];;

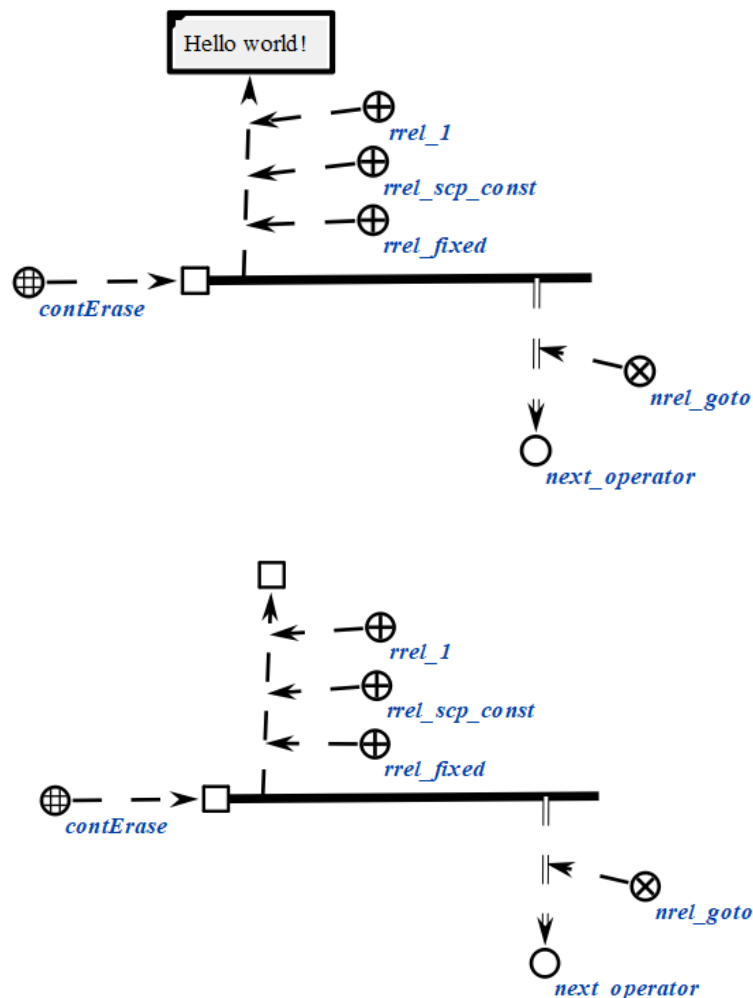
```



```

    _=> nrel_goto:: next_operator;;
*);;

```



6.6.6.6.1.5.15 scp-оператор перевода в нижний регистр строкового содержимого файла

scp-операторы класса *scp-оператор перевода в нижний регистр строкового содержимого файла* описывают действие перевода строкового содержимого файла в нижний регистр, являющегося значением второго scp-операнда.

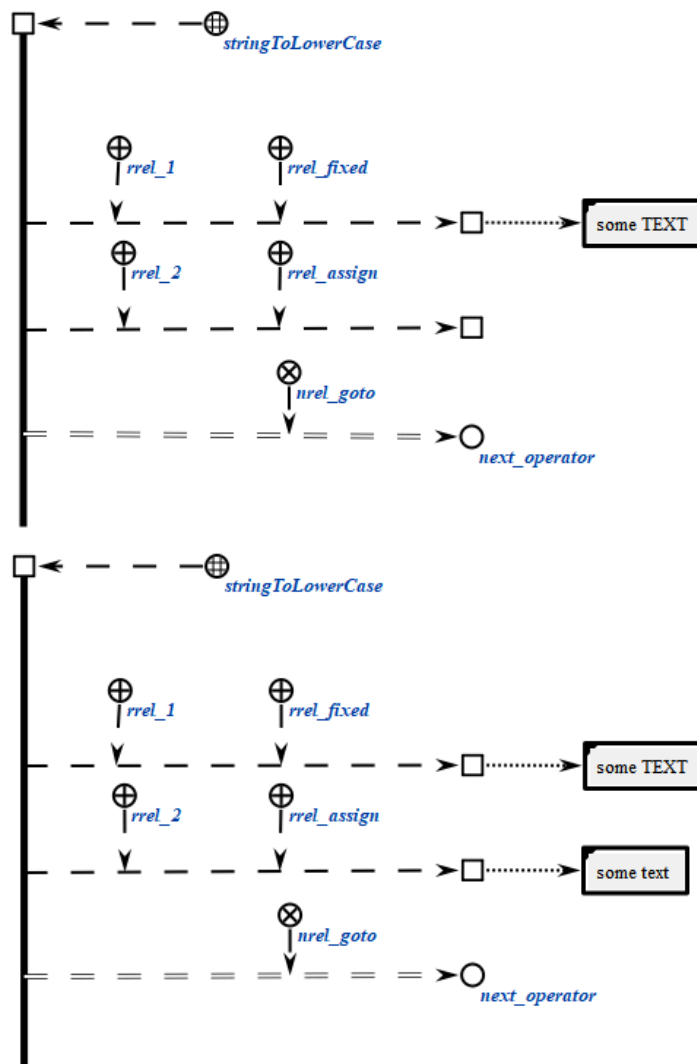
Каждый scp-оператор данного класса содержит два scp-операнда.

Первый scp-операнд может иметь произвольный тип значения scp-операнда'. Значением данного scp-операнда является файл, идентифицирующий строку, являющуюся строкой в верхнем регистре, соответствующей второму scp-операнду. В случае, если данный scp-операнд

помечен как scp-операнд со свободным значением', то файл будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося файла.

Второй scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий строку, являющуюся аргументом функции перевода строки в нижний регистр.

```
scp_operator_example_stringToLowerCase (*
  <-_ stringToLowerCase;;
  _-> rrel_1:: rrel_fixed:: rrel_scp_var:: _el1;;
  _-> rrel_2:: rrel_assign:: rrel_scp_var:: _el2;;
  _=> nrel_goto:: next_operator;;
*);;
```



6.6.6.6.1.5.16 scp-оператор перевода в верхний регистр строкового содержимого файла

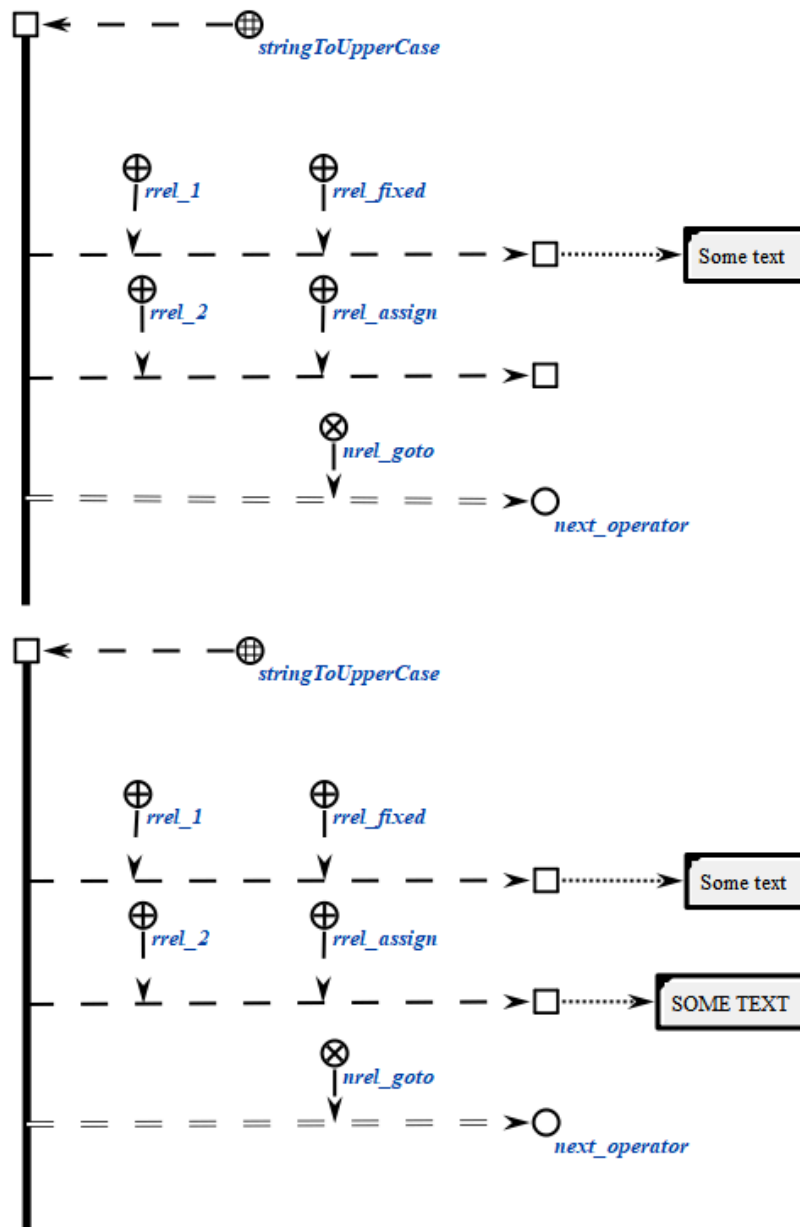
scp-операторы класса *scp-оператор перевода в верхний регистр строкового содержимого файла* описывают действие перевода строкового содержимого в верхний регистр файла, являющегося значением второго scp-операнда.

Каждый scp-оператор данного класса содержит два scp-операнда.

Первый scp-операнд может иметь произвольный тип значения scp-операнда'. Значением данного scp-операнда является файл, идентифицирующий строку, являющуюся строкой в верхнем регистре, соответствующей второму scp-операнду. В случае, если данный scp-операнд помечен как scp-операнд со свободным значением', то файл будет сгенерирована, в противном случае будет изменено содержимое уже имеющегося файла.

Второй scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий строку, являющуюся аргументом функции перевода строки в верхний регистр.

```
scp_operator_example_stringToUpperCase (*  
    <-_ stringToUpperCase;;  
    _-> rrel_1:: rrel_fixed:: rrel_scp_const:: [Some TEXT];;  
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: _el2;;  
    _=> nrel_goto:: next_operator;;  
*);;
```



6.6.6.6.1.5.17 sscr-оператор замены определённой части строкового содержимого файла на содержимое указанного файла

sscr-операторы класса *sscr-оператор замены определённой части строкового содержимого файла на содержимое указанного файла* описывают действие поиска строкового содержимого файла, являющейся значением третьего sscr-операнда, внутри содержимого файла, являющегося значением второго sscr-операнда, и замены найденной подстроки строковым содержимым файла, являющегося значением четвертого sscr-операнда.

Каждый sscr-оператор данного класса содержит четыре sscr-операнда.

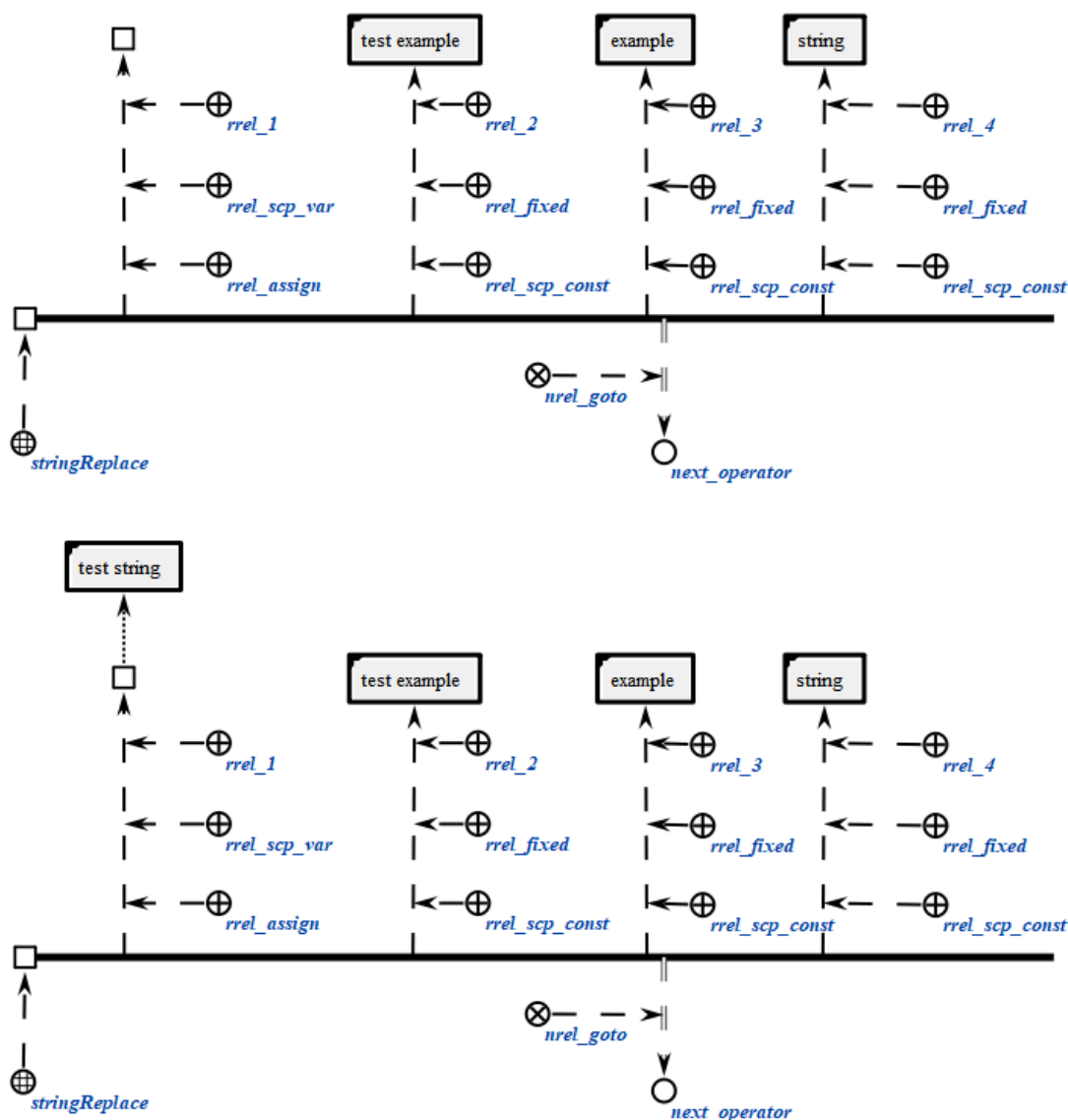
Первый scp-операнд может иметь произвольный тип значения scp-операнда'. Значением данного scp-операнда является файл, идентифицирующий строку, являющейся результатом поиска и замены найденной подстроки указанной строкой, значения которых определены строковым содержимым третьего и четвертого scp-операнда, в строковом содержимом второго scp-операнда. В случае, если данный scp-операнд помечен как scp-операнд со свободным значением', то файл будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося файла.

Второй scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий строку, в которой осуществляется поиск и замены.

Третий scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий строку, вхождения которой в содержимое второго scp-операнда требуется заменить.

Четвертый scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий строку, на которую требуется заменить вхождения содержимого третьего scp-операнда в содержимое второго.

```
scp_operator_example_stringReplace (*  
    <-_ stringReplace;;  
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _ell;;  
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [test example];;  
    _-> rrel_3:: rrel_fixed:: rrel_scp_const:: [example];;  
    _-> rrel_4:: rrel_fixed:: rrel_scp_const:: [string];;  
    _=> nrel_goto:: next_operator;;  
*);;
```



6.6.6.6.1.5.18 scp-оператор проверки совпадения конца строкового содержимого файла со строковым содержимым другого файла

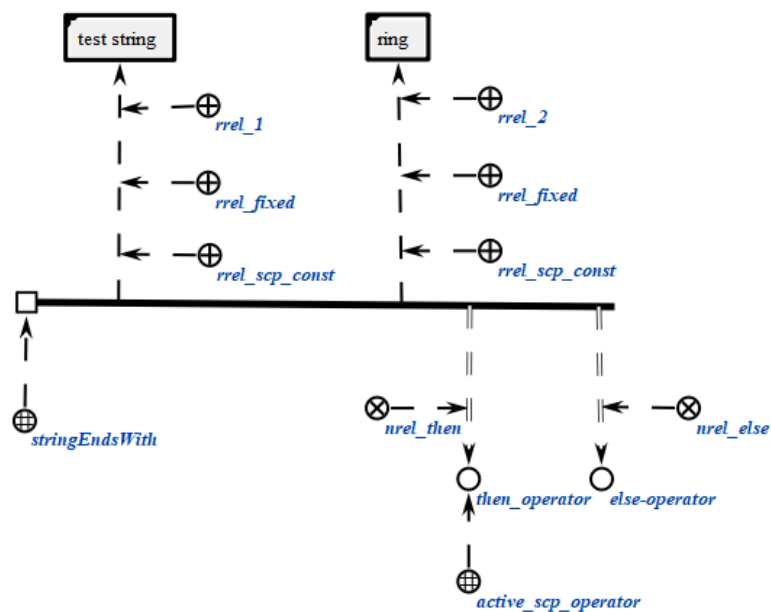
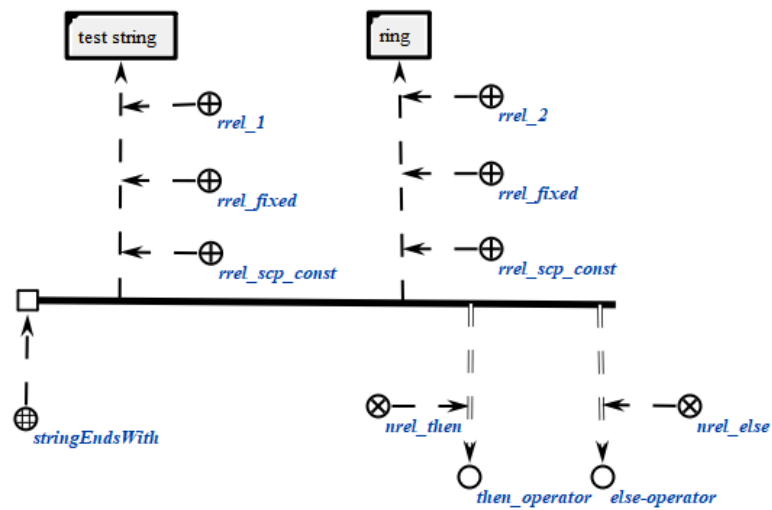
scp-операторы класса *scp-оператор проверки совпадения конца строкового содержимого файла со строковым содержимым другого файла* описывают операции проверки на совпадение конечной части содержимого файла, являющегося значением первого scp-операнда, с содержимым файла, являющегося значением второго scp-операнда.

Каждый scp-оператор данного класса содержит два scp-операнда.

Первый scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий строку, в которой осуществляется поиск.

Второй scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий строку, поиск которой осуществляется в конце содержимого первого scp-операнда.

```
scp_operator_example_stringEndsWith (*
  <-_ stringEndsWith;;
  _-> rrel_1:: rrel_fixed:: rrel_scp_const:: [test string];;
  _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [ring];;
  _=> nrel_then:: then_operator;;
  _=> nrel_else:: else_operator;;
*);;
```



6.6.6.1.5.19 scp-оператор проверки совпадения начала строкового содержимого файла со строковым содержимым другого файла

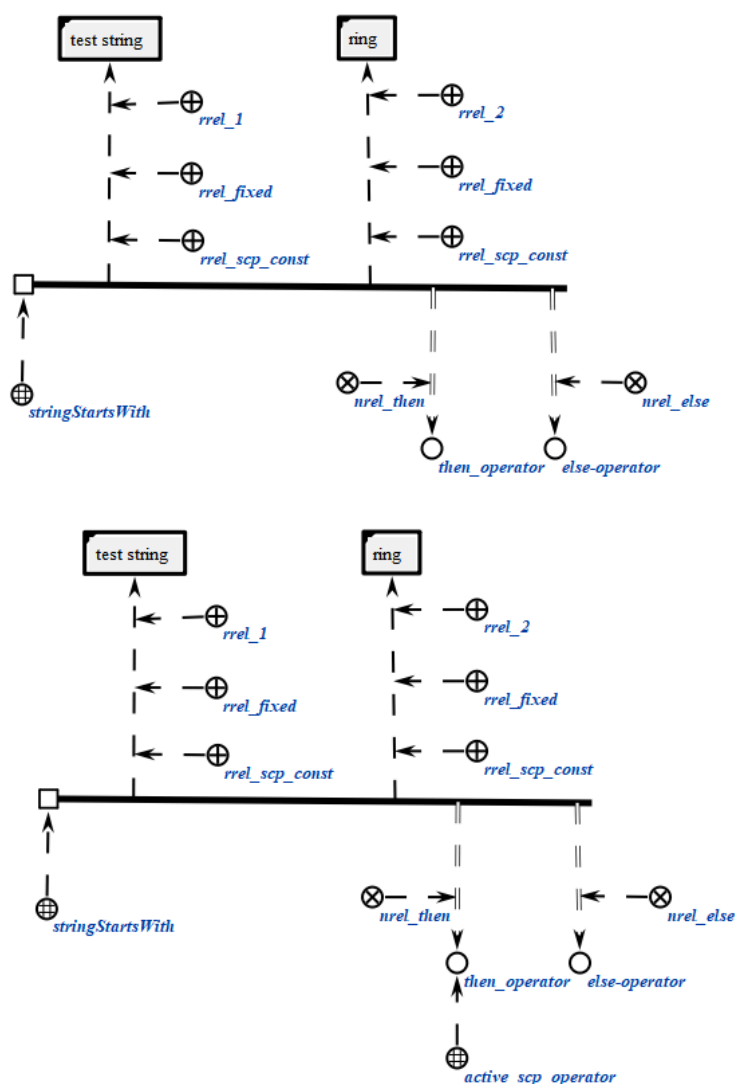
scp-операторы класса *scp-оператор проверки совпадения начальной части строкового содержимого файла со строковым содержимым другого файла* описывают операции проверки на совпадение начальной части содержимого файла, являющегося значением первого scp-операнда, с содержимым файла, являющегося значением второго scp-операнда.

Каждый scp-оператор данного класса содержит два scp-операнда.

Первый scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий строку, в которой осуществляется поиск.

Второй scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий строку, поиск которой осуществляется в начале содержимого первого scp-операнда.

```
scp_operator_example_stringStartsWith (*
    <_ stringStartsWith;;
    _-> rrel_1:: rrel_fixed:: rrel_scp_const:: [test string];;
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [test];;
    _=> nrel_then:: then_operator;;
    _=> nrel_else:: else_operator;;
*);;
```

6.6.6.6.1.5.20. scp-оператор получения части строкового содержимого файла по индексам

scp-операторы класса *scp-оператор получения части строкового содержимого файла по индексам* описывают действие получения части строкового содержимого файла, являющегося значением второго scp-операнда, по двум значениям числового содержимого двух файлов, являющихся значениями третьего и четвертого scp-операнда.

Каждый scp-оператор данного класса содержит четыре scp-операнда.

Первый scp-операнд может иметь произвольный тип значения scp-операнда'. Значением данного scp-операнда является файл, идентифицирующий строку, часть строкового содержимого второго scp-операнда, ограниченную индексами. В случае, если данный scp-операнд

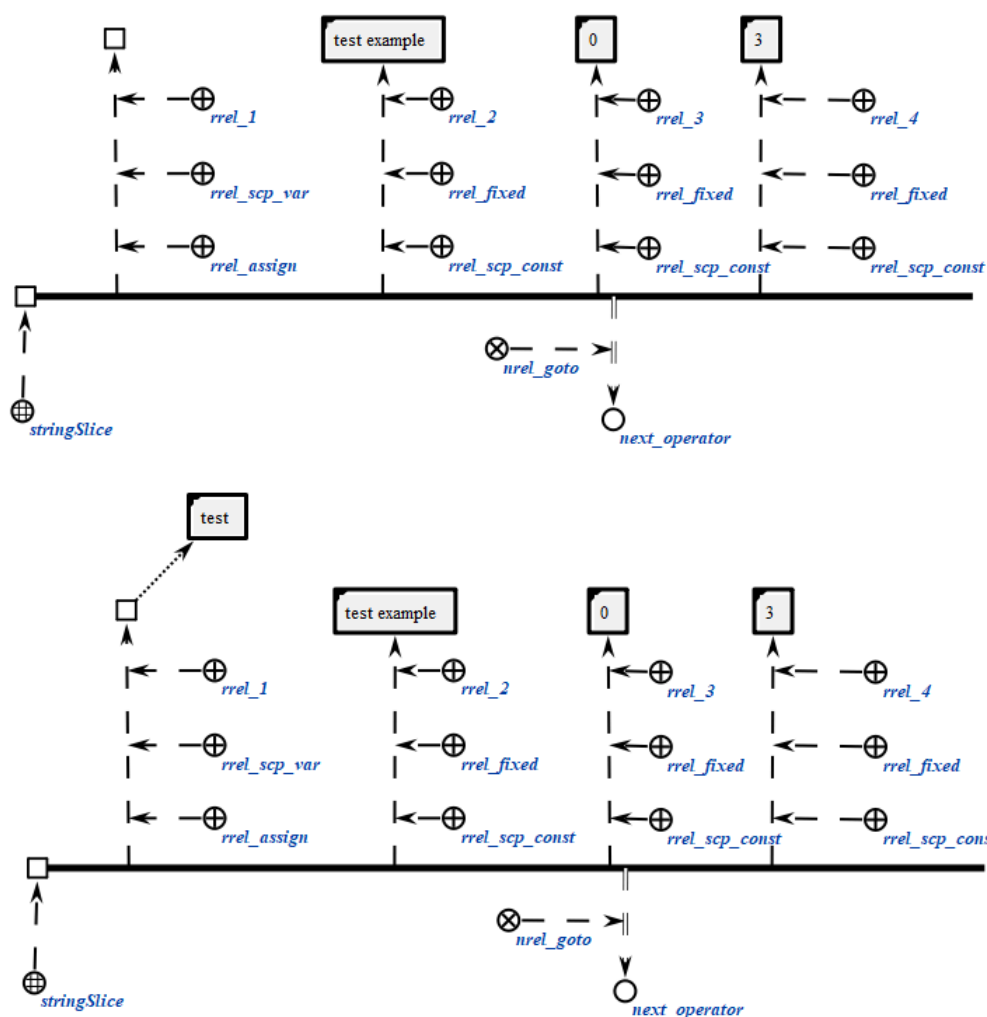
помечен как scp-операнд со свободным значением', то файл, будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося файла.

Второй scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий строку, часть которой требуется получить.

Третий scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий число, являющееся индексом — левой границей запрашиваемой подстроки.

Четвертый scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий число, являющееся индексом — правой границей запрашиваемой подстроки.

```
scp_operator_example_stringSlice (*  
    <_ stringSlice;;  
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _el1;;  
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [test example];;  
    _-> rrel_3:: rrel_fixed:: rrel_scp_const:: [0];;  
    _-> rrel_4:: rrel_fixed:: rrel_scp_const:: [3];;  
    _=> nrel_goto:: next_operator;;  
*);;
```



6.6.6.6.1.5.21 scp-оператор поиска строкового содержимого файла по индексам в строковом содержимом другого файла

scp-операторы класса *scp-оператор поиска строкового содержимого файла в строковом содержимом другого файла* описывают операции поиска содержимого файла, являющегося значением третьего scp-операнда, в содержимом файла, являющегося значением второго scp-операнда.

Каждый scp-оператор данного класса содержит три scp-операнда.

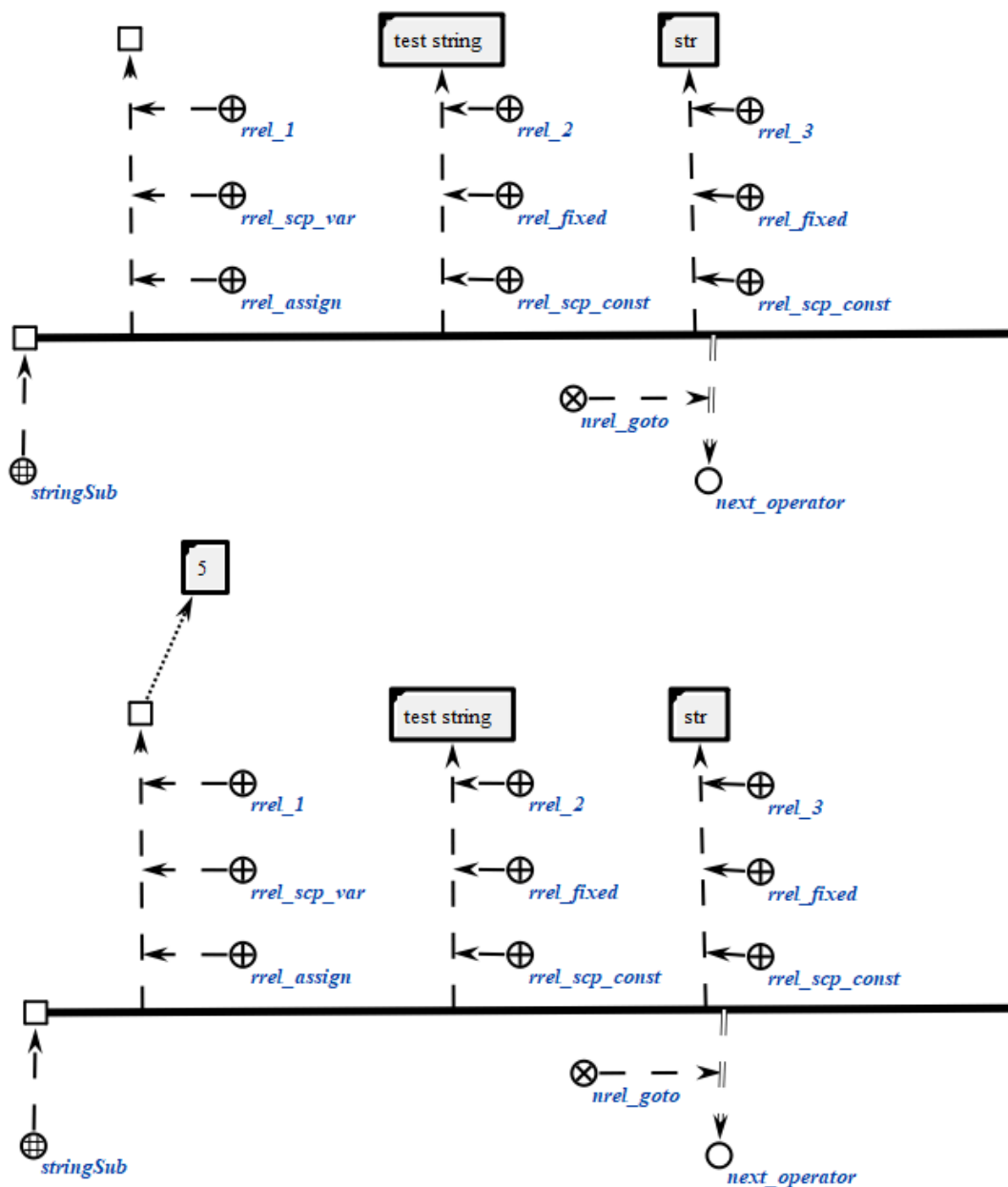
Первый scp-операнд может иметь произвольный тип значения scp-операнда'. Значением данного scp-операнда является файл, идентифицирующий число, являющееся позицией строкового содержимого, задаваемого третьим scp-операндом, в строковом содержимом, задаваемым вторым scp-операндом. В случае, если данный scp-операнд помечен как

scp-операнд со свободным значением', то файл, будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося файла.

Второй scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий строку, в которой осуществляется поиск подстроки.

Третий scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий строку, поиск которой осуществляется в содержимом второго scp-операнда.

```
scp_operator_example_stringSub (*
    <_ stringSub;;
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _el1;;
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [test string];;
    _-> rrel_3:: rrel_fixed:: rrel_scp_const:: [str];;
    _=> nrel_goto:: next_operator;;
*) ;;
```



6.6.6.6.1.5.22 sscr-оператор вычисления длины строкового содержимого файла

sscr-операторы класса *sscr-оператор вычисления длины строкового содержимого файла* описывают действие вычисления длины строкового содержимого файла, являющегося значением второго sscr-операнда.

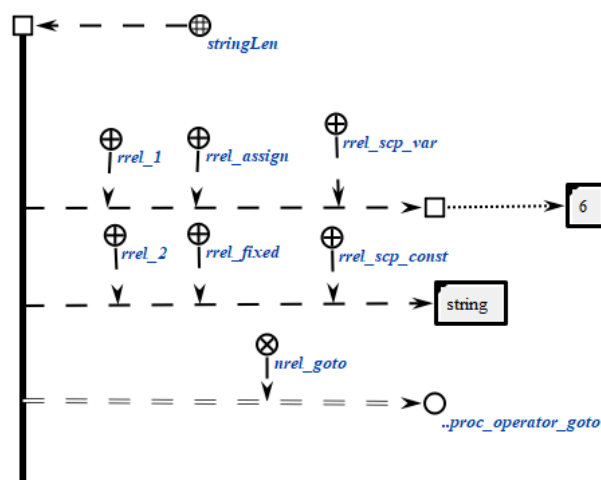
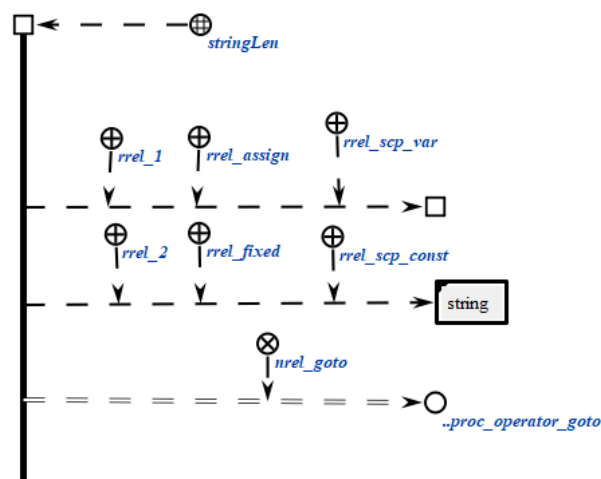
Каждый sscr-оператор данного класса содержит два sscr-операнда.

Первый sscr-операнд может иметь произвольный тип значения sscr-операнда'. Значением данного sscr-операнда является файл,

идентифицирующий число, являющееся длиной строкового содержимого, задаваемого вторым scp-операндом. В случае, если данный scp-операнд помечен как scp-операнд со свободным значением', то файл, будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося файла.

Второй scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий строку, длина которой вычисляется.

```
scp_operator_example_stringLen (*
    <_ stringLen;;
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _ell;;
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [string];;
    _=> nrel_goto:: .proc_operator_goto;;
*);;
```



6.6.6.6.1.5.23 scp-оператор разбиения строки на подстроки

scp-операторы класса *scp-оператор разбиения строки на подстроки* описывают действие разбиения строкового содержимого sc-элемента на подстроки, разделенные разделителем.

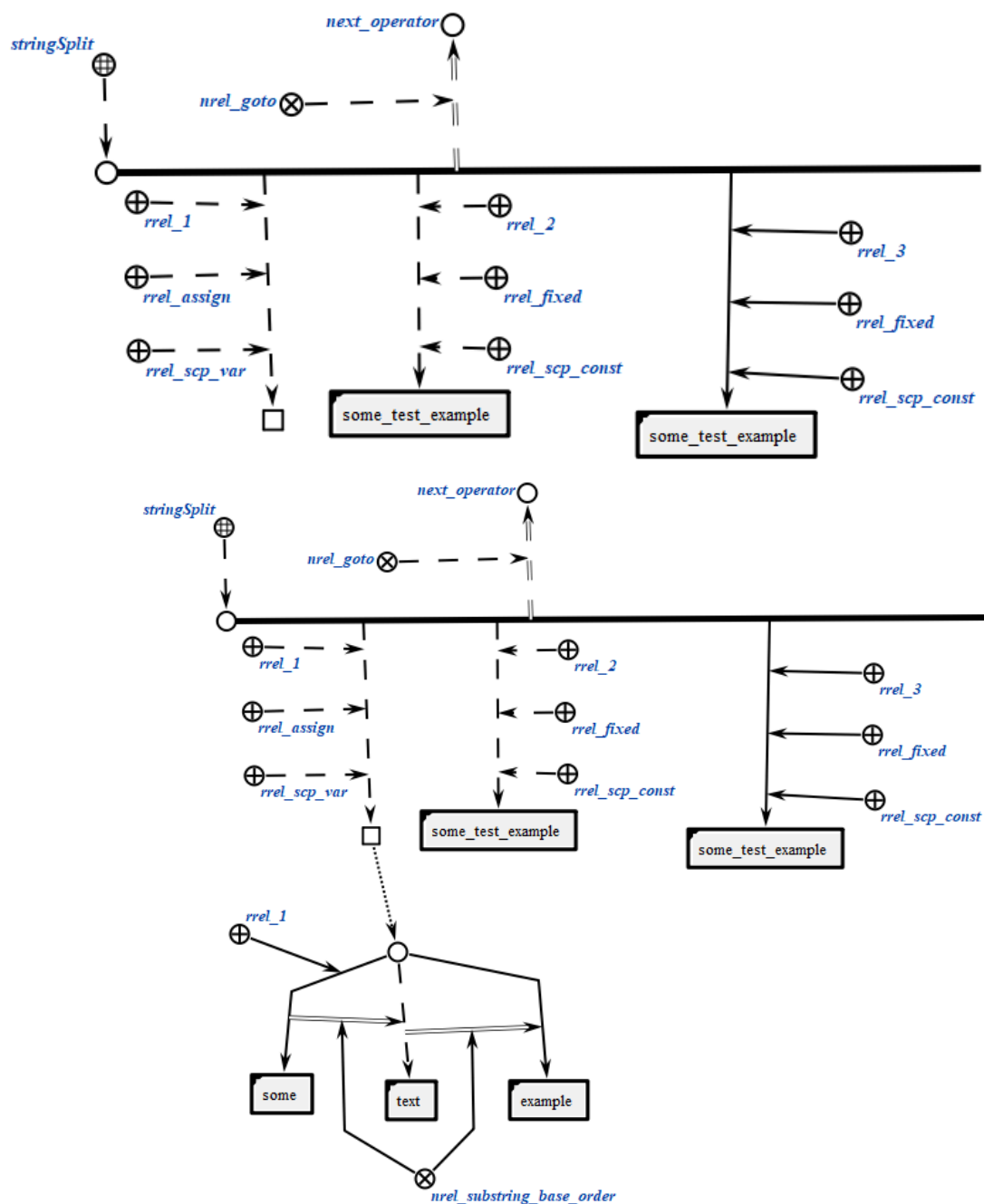
Каждый scp-оператор данного класса содержит три scp-операнда.

Первый scp-операнд может иметь произвольный тип значения scp-операнда'. Значением данного scp-операнда является файл, идентифицирующий множество подстрок строки, задаваемой вторым scp-операндом. В случае, если данный scp-операнд помечен как scp-операнд со свободным значением', то файл, будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося файла.

Второй scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий строку, которая разбивается на подстроки.

Третий scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий строку, которая выступает как разделитель подстрок второго scp-операнда.

```
scp_operator_example_stringSplit (*
    <=_ stringSplit;;
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _ell;;
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [some_test_example];;
    _-> rrel_3:: rrel_fixed:: rrel_scp_const:: [_];;
    _=> nrel_goto:: next_operator;;
*) ;;
```

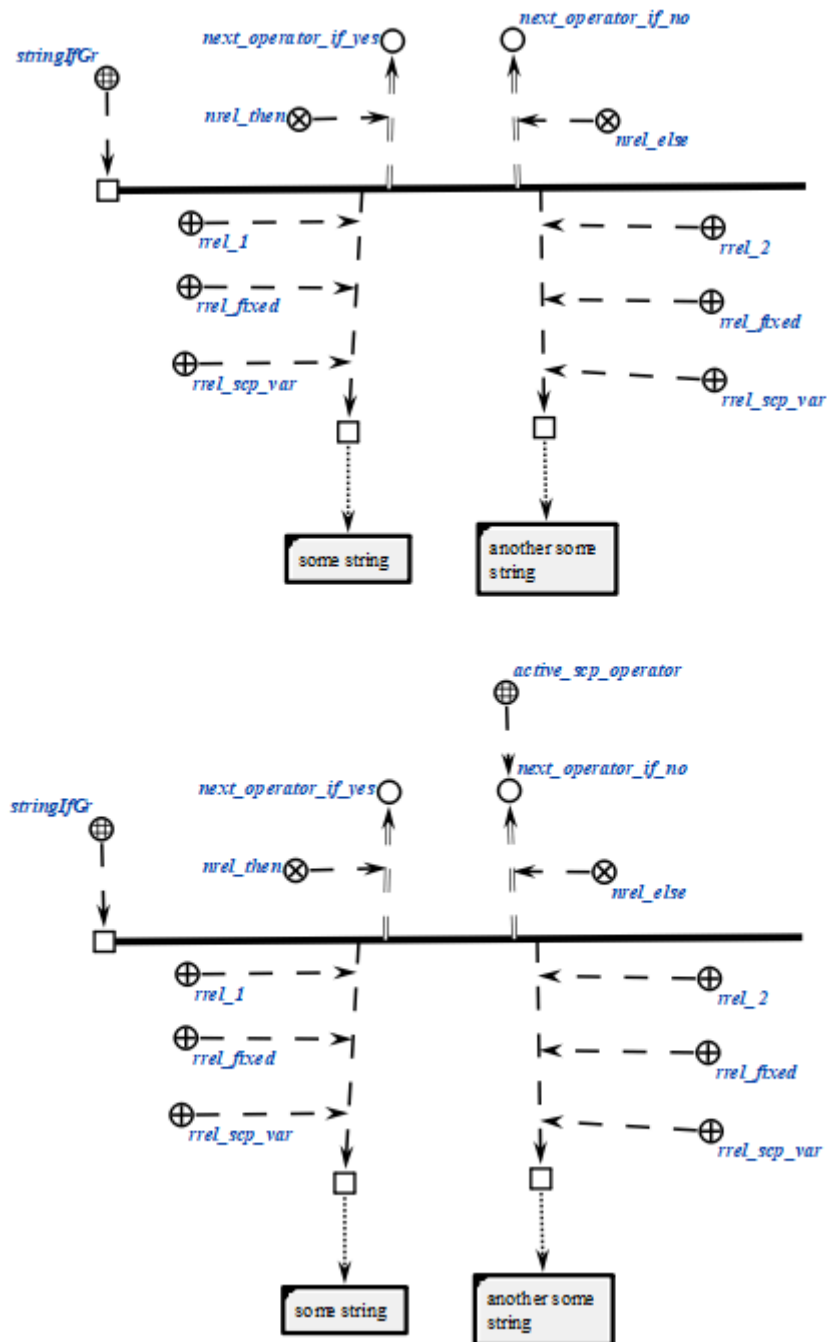


6.6.6.6.1.5.24 *scp*-оператор лексикографического сравнения строковых содержимых файлов

scp-операторы класса *scp-оператор лексикографического сравнения строковых содержимых файлов* описывают действие сравнения между собой строковых содержимых двух файлов, являющихся значениями *scp*-операндов. Успешным выполнением считается, значение первого *scp*-операнда строго больше значения второго *scp*-операнда с точки зрения лексикографического порядка. Каждый *scp*-оператор данного класса содержит два *scp*-операнда,

которые должны быть помечены как scp-операнд с заданным значением'. Каждый из scp-операндов может быть как scp-константой', так и scp-переменной'.

```
scp_operator_example_stringIfGr (*  
    <-_ stringIfGr;;  
    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: _el1;;  
    _-> rrel_2:: rrel_fixed:: rrel_scp_var:: _el1;;  
    _=> nrel_then:: next_operator_if_yes;;  
    _=> nrel_else:: next_operator_if_no;;  
*);;
```

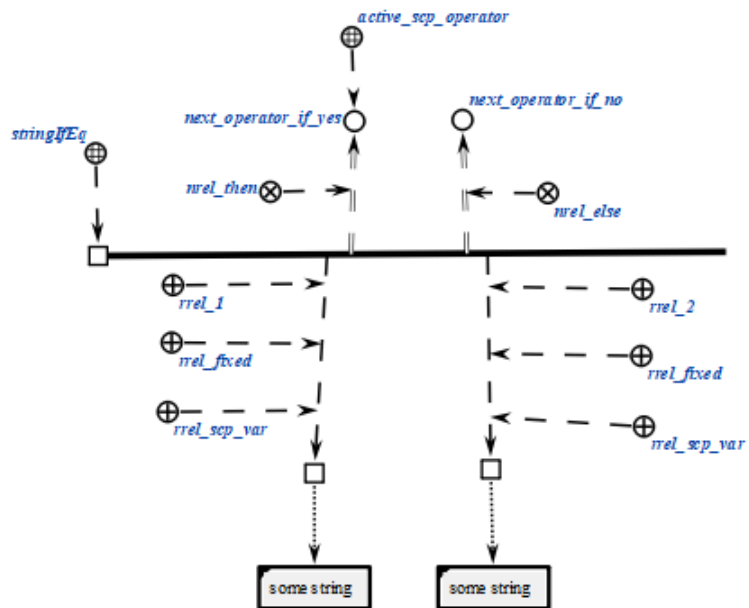
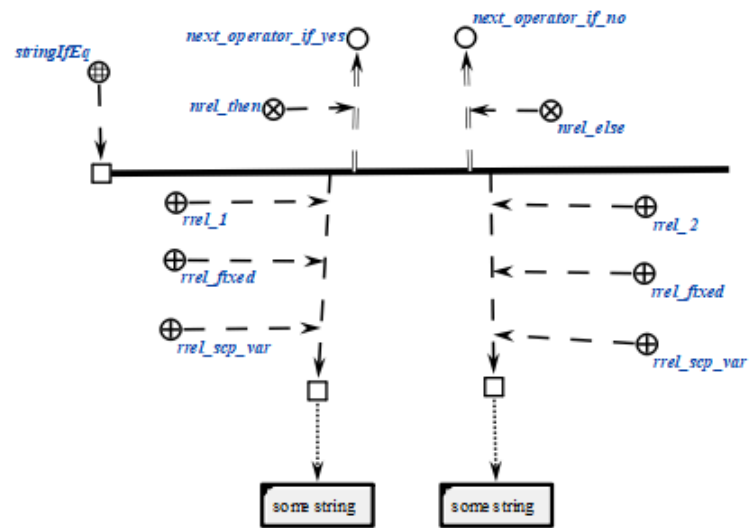


6.6.6.6.1.5.25 scp-оператор проверки равенства строковых содержимых файлов

scp-операторы класса *scp-оператор проверки равенства строковых содержимых файлов* описывают действие проверки того, равны ли между собой строковые содержимые двух файлов, являющихся значениями scp-операндов. Успешным выполнение считается, если строковые содержимые равны. Каждый scp-оператор данного класса содержит два scp-операнда, которые должны быть

помечены как scp-операнд с заданным значением'. Каждый из scp-операндов может быть как scp-константой', так и scp-переменной'.

```
scp_operator_example_stringIfEq (*
  <-_ stringIfEq;;
  _-> rrel_1:: rrel_fixed:: rrel_scp_var:: _ell;;
  _-> rrel_2:: rrel_fixed:: rrel_scp_var:: _ell;;
  _=> nrel_then:: next_operator_if_yes;;
  _=> nrel_else:: next_operator_if_no;;
*);;
```



6.6.6.1.6 scp-операторы управления событиями

scp-операторы класса *scp-оператор управления событиями* описывают действия, связанные с *sc-событиями*.

scp-операторы класса не предполагают ветвления программы в зависимости от выполнения дополнительных условий, поэтому указание следующих scp-операторов осуществляется только при помощи отношения *следующий scp-оператор**, без подмножеств.

В данном классе *scp-операторов* выделяется один scp-оператор:

- *scp-оператор ожидания события*.

6.6.6.1.6.1 scp-оператор ожидания события

scp-операторы класса *scp-оператор ожидания события* описывают действие ожидания возникновения в *sc-памяти* одного из базовых типов *sc-событий*, связанных с каким-либо *sc-элементом*. Если при выполнении *scp-программы* встретился *scp-оператор* данного класса, выполнение *scp-программы* приостанавливается до тех пор, пока в *sc-памяти* не произойдет *sc-событие*, описываемое значениями scp-операндов. При этом учитываются только события, произошедшие после того момента, как при выполнении *scp-программы* встретился *scp-оператор* данного класса, события, произошедшие ранее, не учитываются.

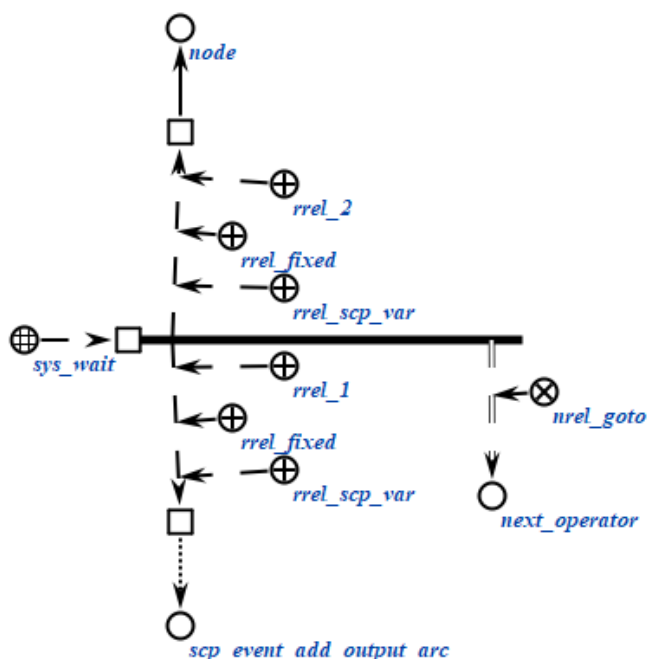
Каждый scp-оператор данного класса содержит два scp-операнда.

Первый scp-операнд должен быть помечен как *scp-операнд с заданным значением'*. Значением данного scp-операнда является один из базовых типов *sc-событий*, т.е. одно из подмножеств множества *sc-событий*.

Второй scp-операнд должен быть помечен как *scp-операнд с заданным значением'*. Значением данного scp-операнда является *sc-элемент*, с которым непосредственно связан указанный тип *sc-события*, т.е. элемент, из которого выходит либо в который входит генерируемая либо удаляемая *sc-дуга*, либо *файл*, содержимое которого было изменено и т.п.

```
scp_operator_example_sys_wait (*
    <-_ sys_wait;;
    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: _el1;;
    _-> rrel_2:: rrel_fixed:: rrel_scp_var:: _el2;;
    _=> nrel_goto:: next_operator;;
```

*) ; ;



6.6.6.6.1.7 scp-операторы управления scp-процессами

scp-операторы класса *scp-оператор управления scp-процессами* описывают действия, связанные с вызовом подпрограмм (т.е. созданием *scp-процессов*), а также управления выполнением *scp-программ*.

scp-операторы класса не предполагают ветвления программы в зависимости от выполнения дополнительных условий, поэтому указание следующих scp-операторов осуществляется только при помощи отношения *следующий scp-оператор**, без подмножеств.

Данный класс *scp-операторов* разбивается на следующие подклассы:

- *scp-оператор асинхронного вызова подпрограммы*;
- *scp-оператор ожидания завершения выполнения scp-программы*;
- *scp-оператор ожидания завершения выполнения множества scp-программ*;
- *scp-оператор завершения выполнения программы*.

6.6.6.6.1.7.1 scp-оператор асинхронного вызова подпрограммы

scp-операторы класса *scp-оператор асинхронного вызова подпрограммы* описывают действие вызова подпрограммы, то есть создания уникального *scp-процесса*, соответствующего указанной *scp-программе*. Вызов является

асинхронным, то есть выполнение дочернего *scp-процесса* начинается сразу же после его создания, параллельно с родительским *scp-процессом*. Каждый *scp-оператор* данного класса содержит три *scp-операнда*.

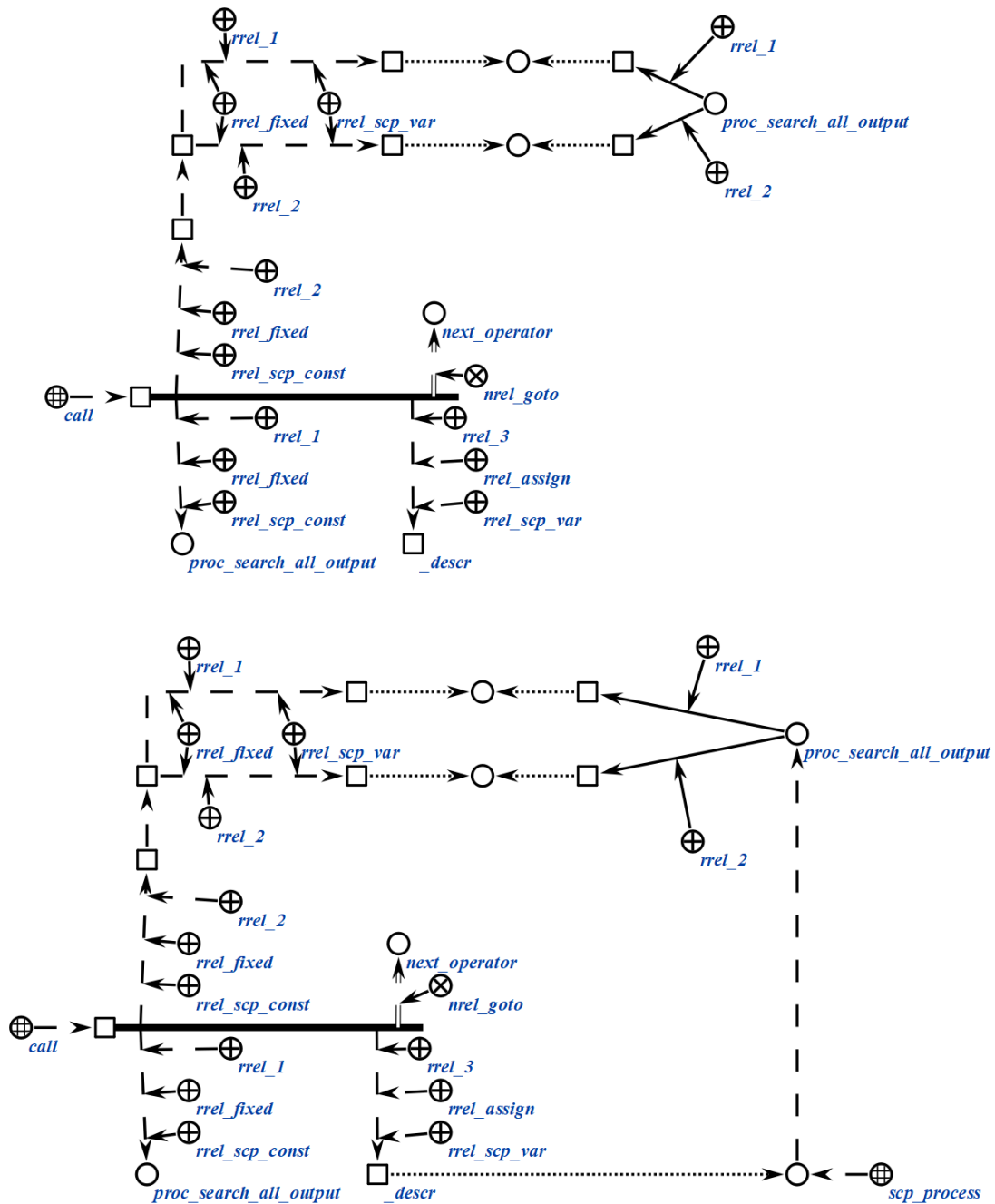
Первый *scp-операнд* должен быть помечен как *scp-операнд с заданным значением'*. Значением данного *scp-операнда* является знак *scp-программы*, вызов которой необходимо осуществить.

Второй *scp-операнд* должен быть помечен как *scp-операнд с заданным значением'*. Значением данного *scp-операнда* является множество *scp-параметров*, передаваемых в вызываемую подпрограмму. Количество элементов данного множества должно совпадать с количеством *scp-параметров* вызываемой *scp-программы*, все элементы упорядочиваются при помощи ролевых отношений 1', 2', 3'. Элемент, помеченный одним из указанных ролевых отношений, соответствует параметру, помеченному тем же ролевым отношением. Элементы, соответствующие *in-параметрам'* должны быть помечены как *scp-операнд с заданным значением'*. Элементы, соответствующие *out-параметрам'*, могут иметь произвольный *тип значения scp-операнда'*.

Третий *scp-операнд* должен быть помечен как *scp-операнд со свободным значением'*. Значением данного *scp-операнда* является знак созданного *scp-процесса*, который далее может быть использован для того, чтобы дождаться завершения созданного *scp-процесса*. Следует отметить, что каждый созданный *scp-процесс* существует в *sc-памяти* до того момента как был выполнен один из *scp-операторов* ожидания завершения *scp-процессов*. В связи с этим при помощи соответствующих *scp-операторов* необходимо обеспечить в конечном итоге ожидание завершения всех созданных во время выполнения *scp-программы scp-процессов* (необязательно одновременно и необязательно в рамках одной *scp-программы*). Нарушение данного правила ведет к засорению *sc-памяти* конструкциями, описывающими созданные *scp-процессы*.

```
scp_operator_example_call (*  
    <-_ call;;  
    _-> rrel_1:: rrel_fixed:: rrel_scp_const:: proc_search_all_outgoing_arcs;;  
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: scp_operator_example_call_params (*  
        _-> rrel_1:: rrel_fixed:: rrel_scp_var:: _param1;;  
        _-> rrel_2:: rrel_fixed:: rrel_scp_var:: _param2;;  
    *);;  
    _-> rrel_3:: rrel_assign:: rrel_scp_var:: _desrc;;  
    _=> nrel_goto:: next_operator;;
```

*);;



6.6.6.1.7.2 scp-оператор ожидания выполнения scp-программы

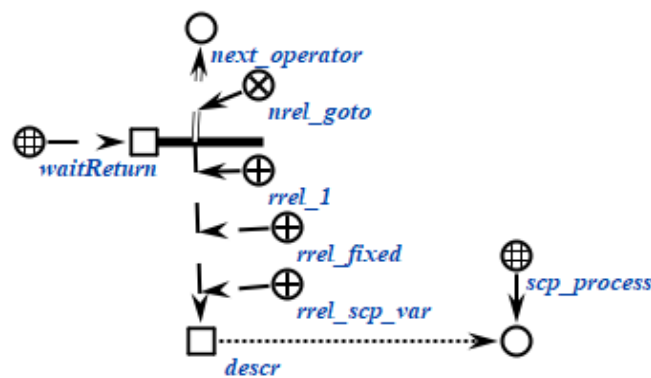
scp-операторы класса *scp-оператор ожидания завершения выполнения scp-программы* описывают действие ожидания завершения выполнения вызванной *scp-программы*, то есть *scp-процесса*, созданного при выполнении *scp-оператора асинхронного вызова подпрограммы*. Если *scp-процесс* завершился до того, как началось выполнение *scp-оператора* данного класса, то

будет выполнен *следующий scp-оператор**, в противном случае *scp-программа* приостановит свое выполнение до того, как ожидаемый *scp-процесс* не завершится. После выполнения *scp-оператора* данного класса конструкция, описывающая заданный *scp-процесс* будет удалена из *sc-памяти*. В связи с этим запрещается использование нескольких *scp-операторов* данного класса для знака одного и того же *scp-процесса*.

Каждый *scp-оператор* данного класса содержит один *scp-операнд*, значением которого является знак *scp-процесса*, завершения которого необходимо дождаться. *scp-операнд* должен быть помечен как *scp-операнд с заданным значением'*.

```
scp_operator_example_waitReturn (*
    <_ waitReturn;;
    -> rrel_1:: rrel_fixed:: rrel_scp_var:: _descr;;

    _=> nrel_goto:: next_operator;;
*);;
```



6.6.6.6.1.7.3 scp-оператор ожидания выполнения множества scp-процессов

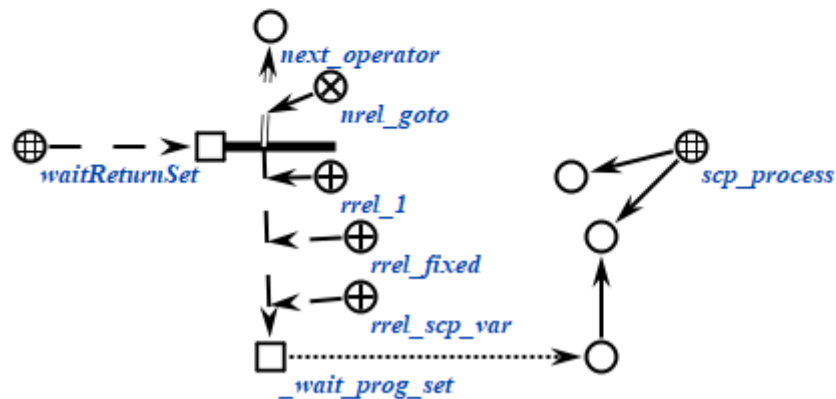
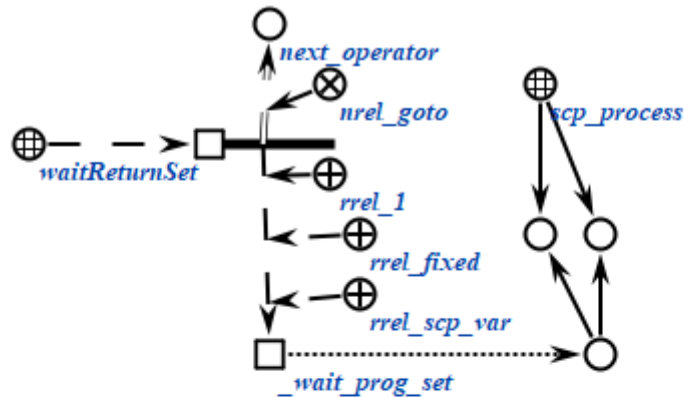
scp-операторы класса *scp-оператор ожидания завершения выполнения множества scp-процессов* описывают действие ожидания завершения выполнения множества вызванных *scp-программ*, то есть *scp-процессов*, созданных при выполнении *scp-операторов асинхронного вызова подпрограммы*.

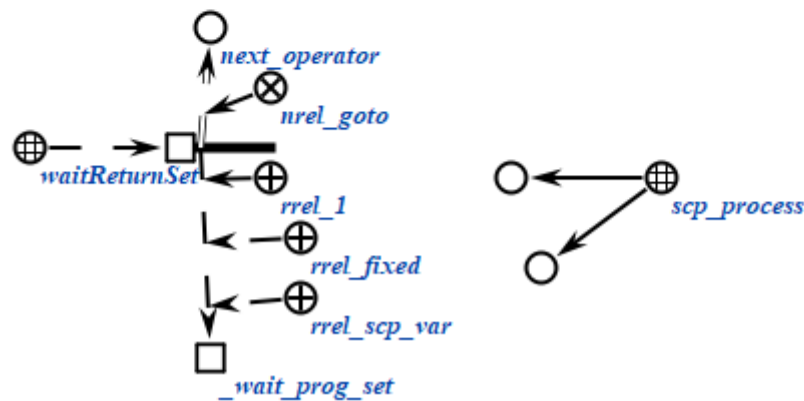
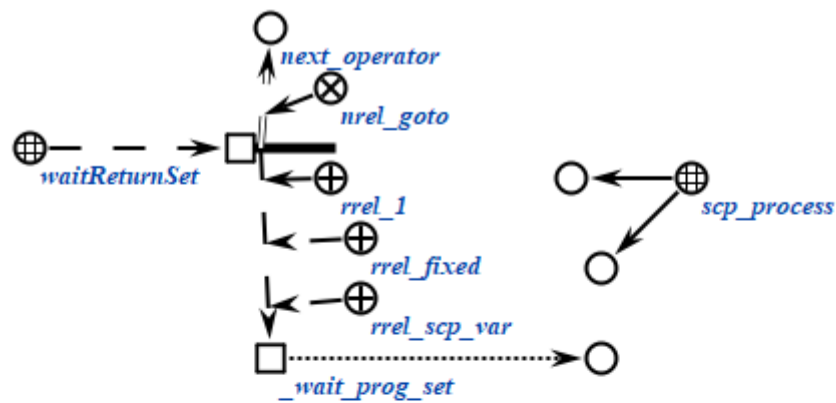
Каждый *scp-оператор* данного класса содержит один *scp-операнд*, значением которого является знак множества *scp-процессов*, завершения

которых необходимо дождаться. *scp*-операнд должен быть помечен как *scp-операнд с заданным значением*'.

scp-процессы удаляются из множества по мере своего завершения, до тех пор, пока множество не станет пустым. После этого уничтожается знак самого множества и начинается выполнение *следующего scp-оператора**.

```
scp_operator_example_waitReturnSet (*
    <_ waitReturnSet;;
    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: _wait_prog_set;;
    _=> nrel_goto:: next_operator;;
*);;
```





6.6.6.6.1.7.4 scp-оператор завершения выполнения scp-программы

scp-операторы класса *scp-оператор завершения выполнения scp-программы* описывают действие завершения выполнения текущей *scp-программы*. scp-операторы данного класса не содержат scp-операндов, и завершают выполнение той *scp-программы*, во множество scp-операторов' которой они входят.

Каждой *scp-программе* могут соответствовать несколько scp-операторов данного класса, каждый из которых может быть достигнут в различных случаях ветвления *scp-программы*, но любой из указанных scp-операторов завершает выполнение данной *scp-программы*. В связи с этим *scp-программа*, в которой scp-оператор данного класса должен быть выполнен параллельно с каким-либо другим *scp-оператором*, может рассматриваться как некорректная.

```
scp_operator_example_return (*
    <_ return;;
*);;
```

6.6.6.6.1.8 scp-операторы управления значениями scp-операндов

scp-операторы класса *scp-оператор управления значениями scp-операндов* описывают действия, связанные со связками отношения *значение**, но не затрагивающими конкретные *sc-элементы*, являющиеся значениями scp-операндов данного класса scp-операторов.

Данный класс *scp-операторов* разбивается на следующие подклассы:

- *scp-оператор присваивания значения scp-переменной*;
- *scp-оператор удаления значения scp-переменной*.

6.6.6.6.1.8.1 scp-оператор присваивания значения scp-переменной

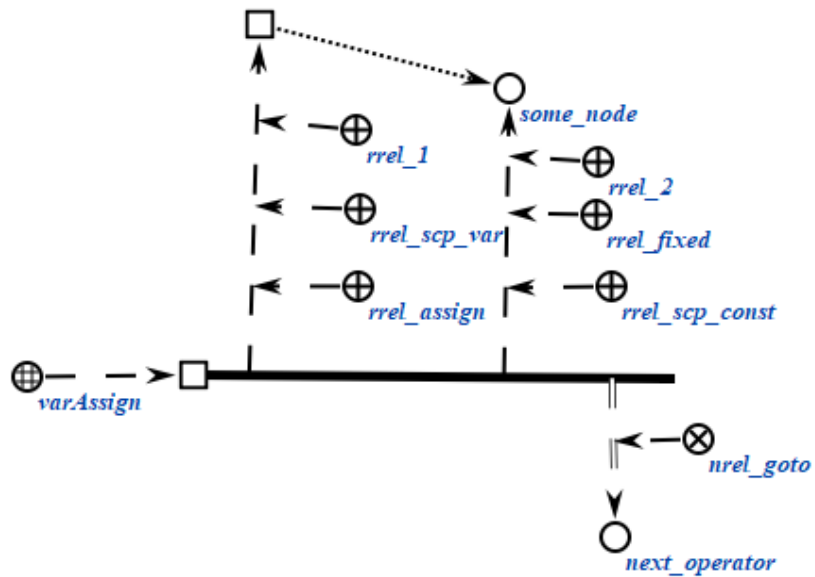
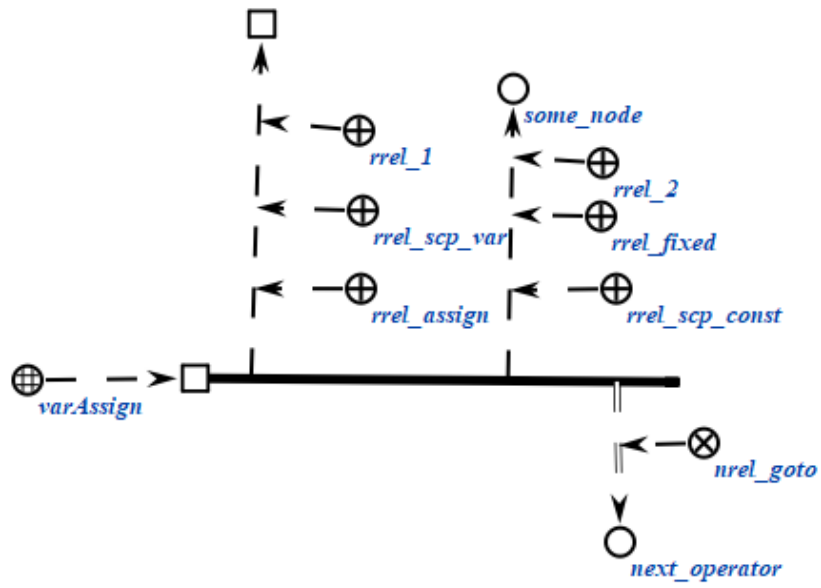
scp-операторы класса *scp-оператор присваивания значения переменной* описывают действие присваивания значения некоторой *scp-переменной*'.

Каждый scp-оператор данного класса содержит два scp-операнда.

Первый scp-операнд может иметь произвольный *тип значения scp-операнда*'. После выполнения scp-оператора значением данного scp-операнда станет значение второго scp-операнда, то есть будет сгенерирована новая *sc-связка* отношения *значение**, соединяющая данный scp-операнд и *sc-элемент*, являющийся значением второго scp-операнда. Данный scp-операнд должен быть помечен как *scp-переменная*'.

Второй scp-операнд должен быть помечен как *scp-операнд с заданным значением*'. При этом данный scp-операнд может быть как *scp-переменной*', так и *scp-константой*'.

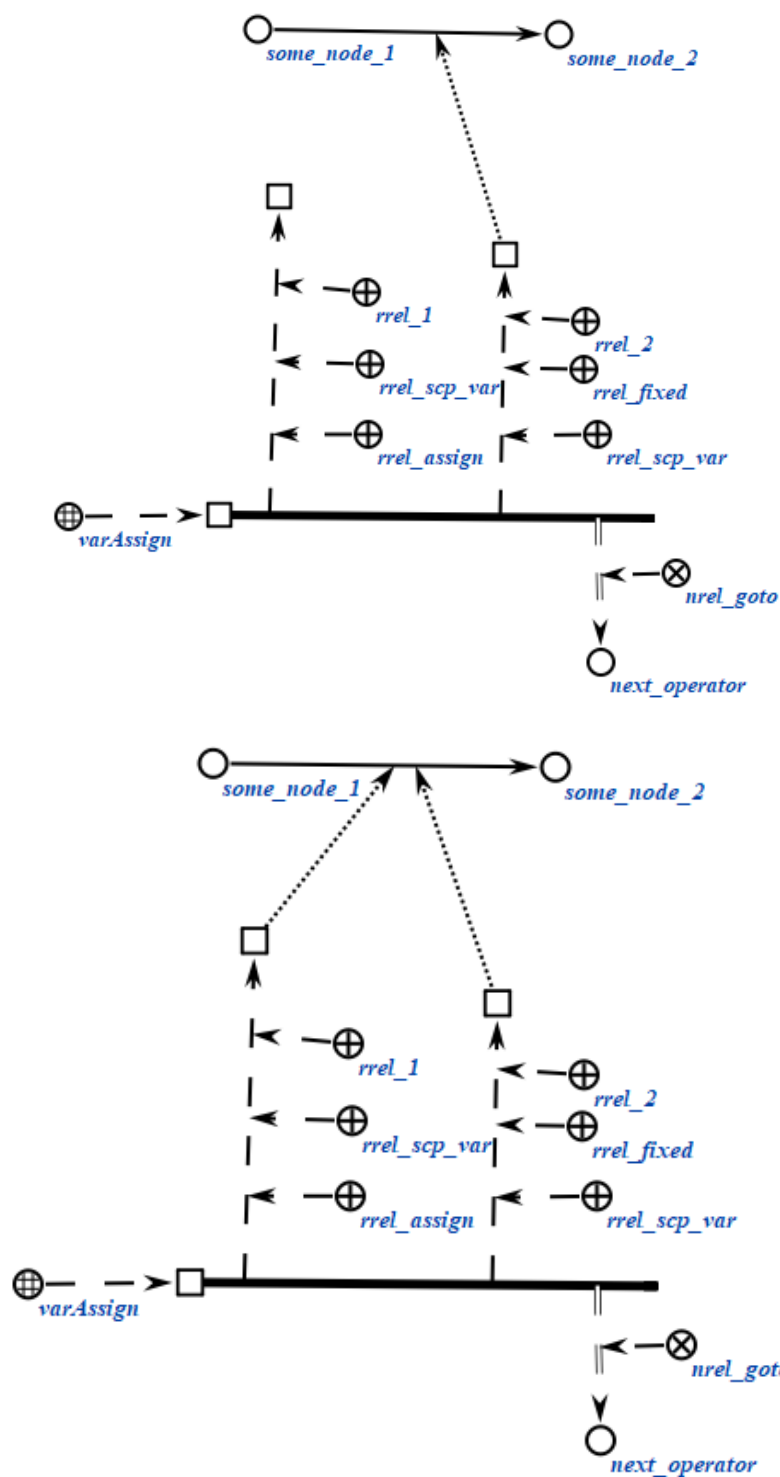
```
scp_operator_example_varAssign (*
    <_ varAssign;;
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _el1;;
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: some_node;;
    _=> nrel_goto:: next_operator;;
*);;
```



```

scp_operator_example_varAssign (*
    <_ varAssign;;
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _ell;;
    _-> rrel_2:: rrel_fixed:: rrel_scp_var:: _some_node;;
    _=> nrel_goto:: next_operator;;
*);;

```

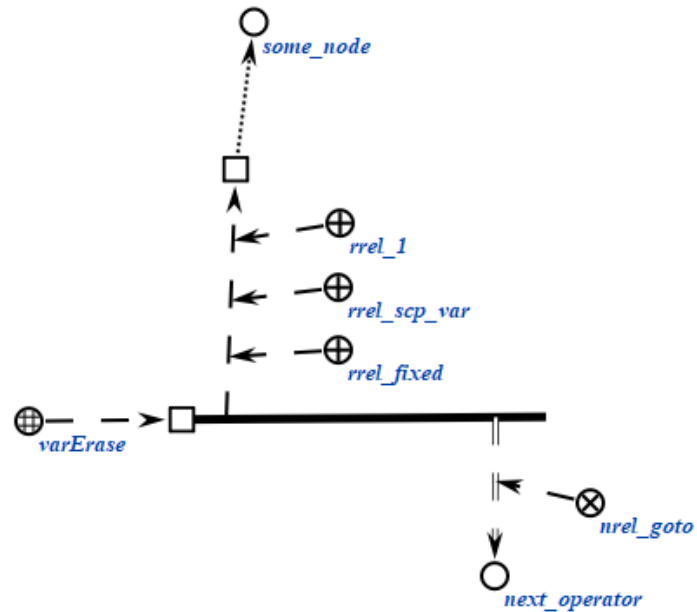


6.6.6.6.1.8.2 ssp-оператор удаления значения ssp-переменной

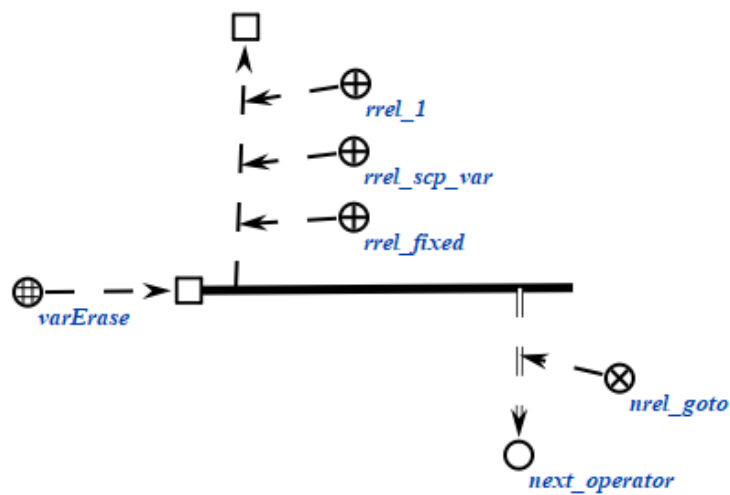
ssp-операторы класса *ssp-оператор* удаления значения *ssp-переменной* описывают действие удаления ss-связки отношения *значение** для некоторой *ssp-переменной*'. Каждый ssp-оператор данного класса содержит один ssp-операнд, который должен быть помечен как *ssp-операнд с заданным значением*' и как *ssp-переменная*'. После выполнения ssp-оператора будет

удалена sc-связка отношения *значение**, связывавшая указанный scp-операнд и его значение.

```
scp_operator_example_varErase (*
  <_ varErase;;
  _-> rrel_1:: rrel_fixed:: rrel_scp_var:: _ell;;
  _=> nrel_goto:: next_operator;;
*);;
```



some_node



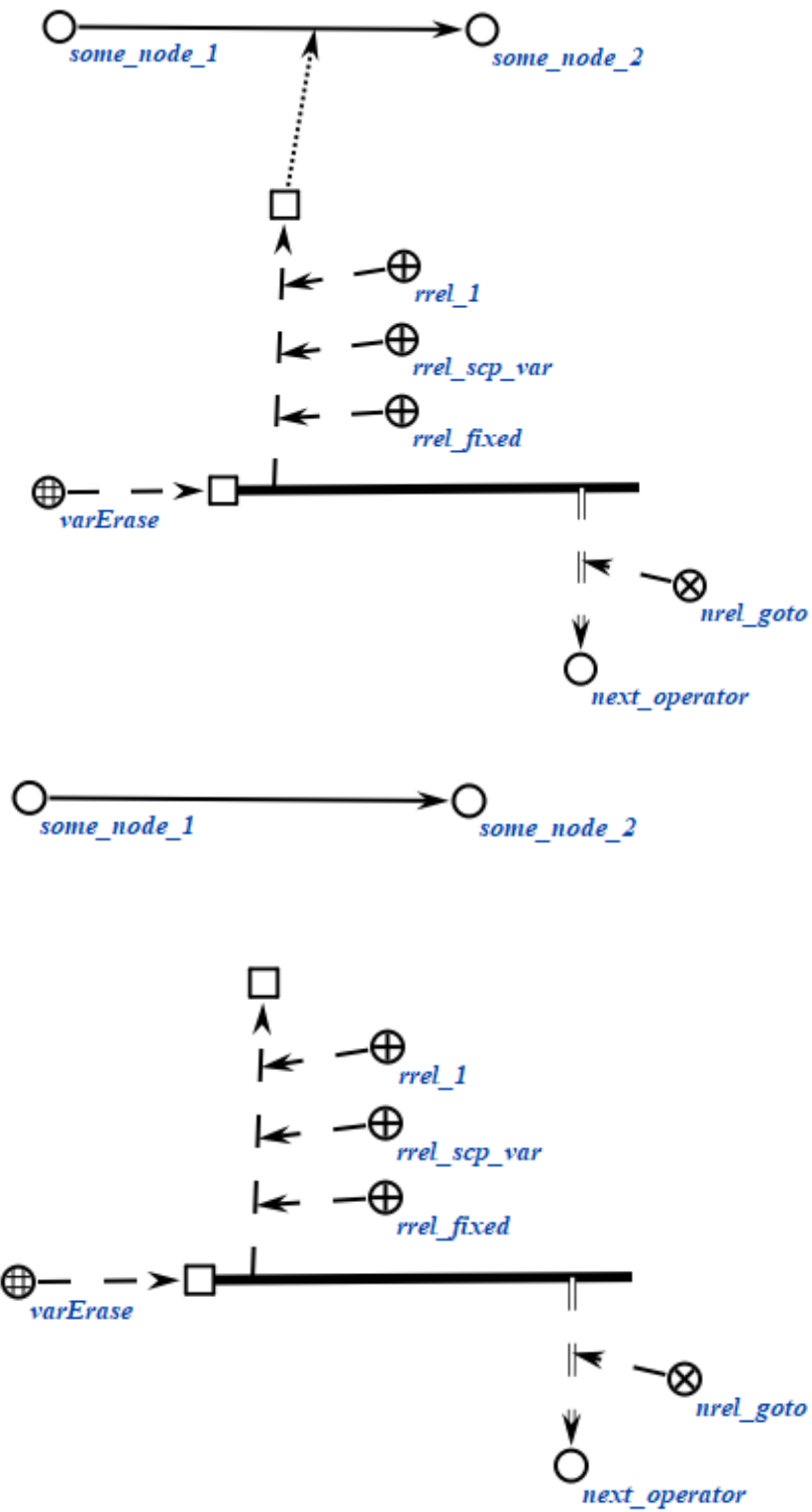
```
scp_operator_example_varErase (*
  <_ varErase;;
```

```

-> rrel_1:: rrel_fixed:: rrel_scp_var:: _ell;;
=> nrel_goto:: next_operator;;

*);;

```



6.6.6.6.2 Системные sc-идентификаторы классов scp-операторов

При программировании на Языке SCP без использования специализированной среды, например, путем редактирования исходных текстов базы знаний на SCs-коде при помощи стандартного текстового редактора, необходимо пользоваться *системными идентификаторами** ключевых понятий языка, список которых приведен ниже.

Русскоязычный основной sc-идентификатор	Системный sc-идентификатор
<i>scp-программа</i>	scp_program
<i>агентная scp-программа</i>	agent_scp_program
<i>активная агентная scp-программа</i>	active_agent_scp_program
<i>in-параметр'</i>	rrel_in
<i>out-параметр'</i>	rrel_out
<i>scp-константа'</i>	rrel_scp_const
<i>scp-переменная'</i>	rrel_scp_var
<i>scp-операнд с заданным значением'</i>	rrel_fixed
<i>scp-операнд со свободным значением'</i>	rrel_assign
<i>константный sc-элемент'</i>	rrel_const
<i>переменный sc-элемент'</i>	rrel_var
<i>sc-узел'</i>	rrel_node
<i>sc-дуга'</i>	rrel_arc
<i>файл'</i>	rrel_node_link
<i>sc-дуга общего вида'</i>	rrel_common_arc
<i>sc-дуга принадлежности'</i>	rrel_membership_arc
<i>позитивная sc-дуга принадлежности'</i>	rrel_pos_arc
<i>негативная sc-дуга принадлежности'</i>	rrel_neg_arc
<i>нечеткая sc-дуга принадлежности'</i>	rrel_fuz_arc
<i>временная sc-дуга принадлежности'</i>	rrel_temp_arc
<i>постоянная sc-дуга принадлежности'</i>	rrel_perm_arc
<i>sc-дуга основного вида'</i>	rrel_const_perm_pos_arc

<i>формируемое множество'</i>	rrel_set
<i>формируемое множество 1', формируемое множество 2'...</i>	rrel_set_1, rrel_set_2...
<i>удаляемый sc-элемент'</i>	rrel_erase
<i>следующий scp-оператор*</i>	nrel_goto
<i>следующий scp-оператор при успешном выполнении*</i>	nrel_then
<i>следующий scp-оператор при безуспешном выполнении*</i>	nrel_else
<i>scp-оператор</i>	scp_operator
<i>1', 2', 3'...</i>	rrel_1, rrel_2, rrel_3 ...
<i>scp-оператор генерации одноэлементной sc-конструкции</i>	genEl
<i>scp-оператор генерации трехэлементной sc-конструкции</i>	genElStr3
<i>scp-оператор генерации пятиэлементной sc-конструкции</i>	genElStr5
<i>scp-оператор генерации sc-конструкции по произвольному образцу</i>	sys_gen
<i>scp-оператор удаления одноэлементной sc-конструкции</i>	eraseEl
<i>scp-оператор удаления трехэлементной sc-конструкции</i>	eraseElStr3
<i>scp-оператор удаления пятиэлементной sc-конструкции</i>	eraseElStr5
<i>scp-оператор удаления множества элементов трехэлементной sc-конструкции</i>	eraseSetStr3
<i>scp-оператор удаления множества элементов пятиэлементной sc-конструкции</i>	eraseSetStr5
<i>scp-оператор поиска трехэлементной sc-конструкции</i>	searchElStr3
<i>scp-оператор поиска пятиэлементной sc-конструкции</i>	searchElStr5

<i>scr-оператор поиска трехэлементной sc-конструкции с формированием sc-множеств</i>	searchSetStr3
<i>scr-оператор поиска пятиэлементной sc-конструкции с формированием sc-множеств</i>	searchSetStr5
<i>scr-оператор поиска sc-конструкции по произвольному образцу</i>	sys_search
<i>scr-оператор проверки типа sc-элемента</i>	ifType
<i>scr-оператор проверки наличия значения у scr-переменной</i>	ifVarAssign
<i>scr-оператор проверки наличия содержимого у файла</i>	ifFormCont
<i>scr-оператор проверки совпадения значений scr-операндов</i>	ifCoin
<i>scr-оператор проверки равенства числовых содержимых файлов</i>	ifEq
<i>scr-оператор сравнения числовых содержимых файлов</i>	ifGr
<i>scr-оператор сложения числовых содержимых файлов</i>	contAdd
<i>scr-оператор вычитания числовых содержимых файлов</i>	contSub
<i>scr-оператор умножения числовых содержимых файлов</i>	contMult
<i>scr-оператор деления числовых содержимых файлов</i>	contDiv
<i>scr-оператор возведения числового содержимого файла в степень</i>	contPow
<i>scr-оператор вычисления логарифма числового содержимого файла</i>	contLn
<i>scr-оператор вычисления синуса числового содержимого файла</i>	contSin

<i>scr-оператор вычисления косинуса числового содержимого файла</i>	contCos
<i>scr-оператор вычисления тангенса числового содержимого файла</i>	contTg
<i>scr-оператор вычисления арксинуса числового содержимого файла</i>	contASin
<i>scr-оператор вычисления арккосинуса числового содержимого файла</i>	contACos
<i>scr-оператор вычисления арктангенса числового содержимого файла</i>	contATg
<i>scr-оператор копирования содержимого файла</i>	contAssign
<i>scr-оператор удаления содержимого файла</i>	contErase
<i>scr-оператор ожидания события</i>	sys_wait
<i>scr-оператор асинхронного вызова подпрограммы</i>	call
<i>scr-оператор ожидания завершения выполнения scr-программы</i>	waitReturn
<i>scr-оператор ожидания завершения выполнения множества scr-программ</i>	waitReturnSet
<i>scr-оператор завершения выполнения scr-программы</i>	return
<i>scr-оператор присваивания значения scr-переменной</i>	varAssign
<i>scr-оператор удаления значения scr-переменной</i>	varErase
<i>scr-оператор вывода содержимого файла</i>	print
<i>scr-оператор вывода содержимого файла с переходом на новую строку</i>	printNI

<i>scr-оператор семантической окрестности файла</i>	<i>распечатки</i>	printEl
<i>scr-оператор выполнения scr-программы</i>	<i>синхронизации</i>	synchronize
<i>scr-оператор перевода в верхний регистр строкового содержимого файла</i>		stringToUpperCase
<i>scr-оператор перевода в нижний регистр строкового содержимого файла</i>		stringToLowerCase
<i>scr-оператор замены определенной части строкового содержимого файла на содержимое указанного файла</i>		stringReplace
<i>scr-оператор проверки совпадения конца строкового содержимого файла со строковым содержимым другого файла</i>		stringEndsWith
<i>scr-оператор проверки совпадения начальной части строкового содержимого файла со строковым содержимым другого файла</i>		stringStartsWith
<i>scr-оператор получения части строкового содержимого файла по индексам</i>		stringSlice
<i>scr-оператор поиска строкового содержимого файла в строковом содержимом другого файла</i>		stringSub
<i>scr-оператор вычисления длины строкового содержимого файла</i>		stringLen
<i>scr-оператор разбиения строки на подстроки</i>		stringSplit
<i>scr-оператор лексикографического сравнения строковых содержимых файлов</i>		stringIfGr

<i>scr-оператор проверки равенства строковых содержимых файлов</i>	stringIfEq
<i>scr-оператор конкатенации строкового содержимого файлов</i>	contConcat

6.7 Принципы визуализации знаний при помощи Технологии OSTIS

6.7.1 Принципы навигации по базе знаний ostis-системы

навигация по базе знаний ostis-системы — это переход от семантической окрестности одной сущности к семантической окрестности другой сущности, представленных в базе знаний в ostis-системы.

С помощью навигации по базе знаний ostis-системы проверяется синтаксическая и семантическая корректность формализованных фрагментов базы знаний.

6.7.1.1 Принципы навигации по базе знаний ostis-системы при помощи SCn-кода

SCn-код является вариантом отображения sc-конструкций в форме, приближенной к естественному языку. SCn-код является основным инструментом *навигации по базе знаний ostis-системы*.

SCn-код (Semantic Code natural) — язык внешнего гипертекстового представления конструкций SC-кода. В отличие от SCs-кода, в SCn-коде используется минимальное количество разделителей между sc.n-предложениями, а для обозначения сущностей вместо соответствующих им системных sc-идентификаторов используются основные sc-идентификаторы. В остальном синтаксис *SCn-кода* совпадает с синтаксисом SCs-кода.

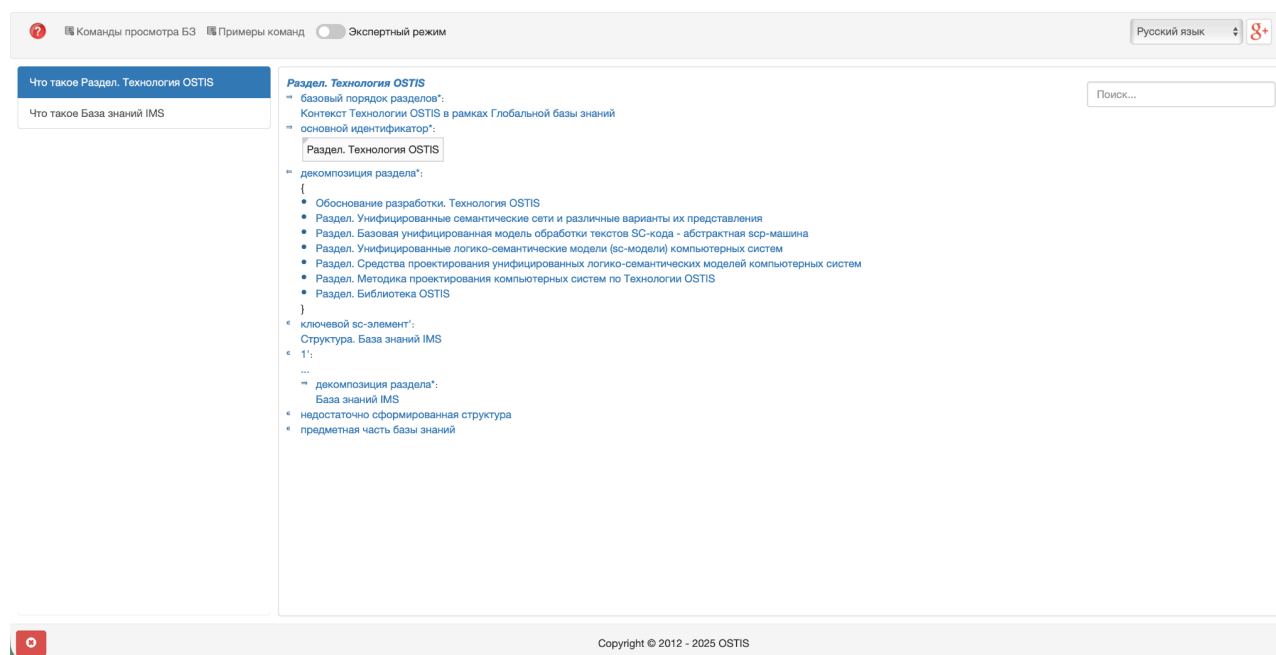
6.7.1.1.1 Навигация по базе знаний ostis-системы на SCn-коде посредством щелчка левой кнопки компьютерной мыши по гиперссылкам sc-идентификаторов сущностей

Для *навигации по базе знаний ostis-системы* на SCn-коде используются гиперссылки.

Каждая сущность базы знаний ostis-системы визуализируется на SCn-коде в виде соответствующего ей основного sc-идентификатора или соответствующего ей системного sc-идентификатора (если основной sc-идентификатор не задан для этой сущности). При этом каждый такой

sc-идентификатор указывается в виде гиперссылки. Щелчок левой кнопкой компьютерной мыши по данной гиперссылке позволяет перейти от семантической окрестности текущей сущности к семантической окрестности другой сущности (по гиперссылке sc-идентификатора которой был осуществлён щелчок компьютерной мышью).

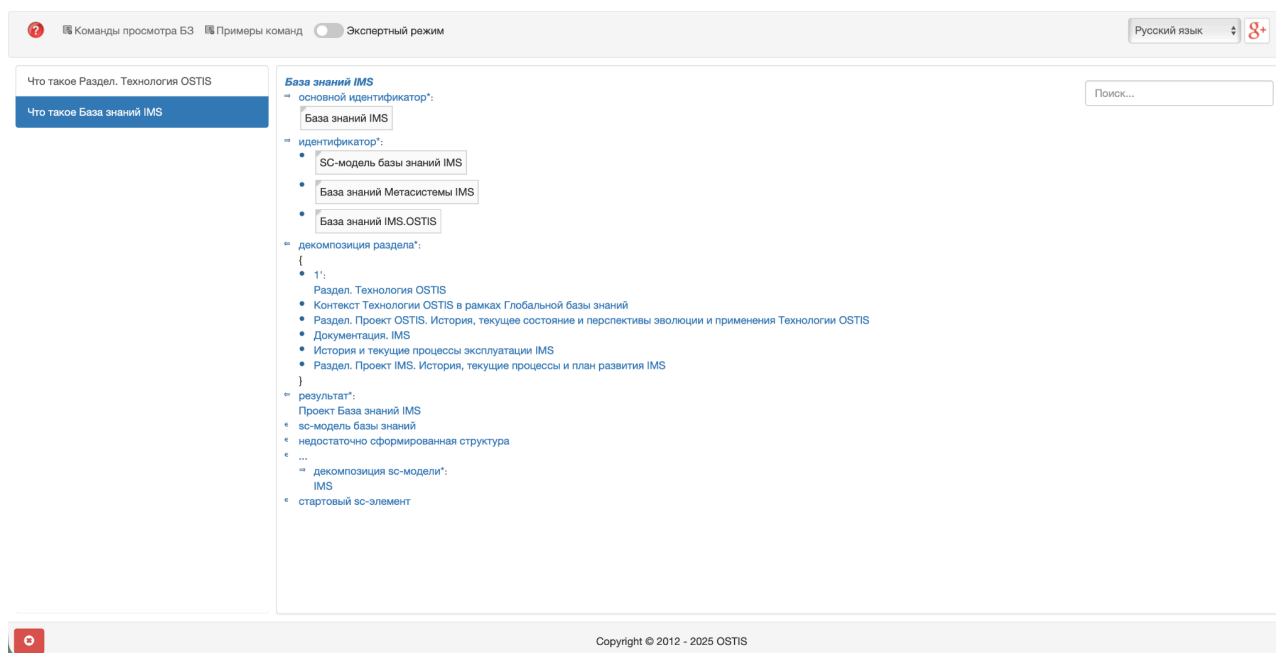
После щелчка левой кнопки компьютерной мыши на sc-идентификатор сущности *Раздел. Технология OSTIS* на странице браузера будет показана семантическая окрестность сущности *Раздел. Технология OSTIS*, представленная на SCn-коде ниже:



6.7.1.1.2 Навигация по базе знаний ostis-системы на SCn-коде посредством щелчка левой кнопки компьютерной мыши по элементам истории навигации по базе знаний ostis-системы

В панели слева — *панели истории навигации по базе знаний ostis-системы* — будет добавлен новый элемент списка *Что такое Раздел. Технология OSTIS*. щелчком левой кнопки компьютерной мыши по *элементам панели истории навигации по базе знаний ostis-системы* можно переходить к семантическим окрестностям сущностям, которые были визуализированы ранее.

С помощью щелчка левой кнопки компьютерной мыши по элементу *Что такое База знаний IMS* можно заново отобразить семантическую окрестность сущности *База знаний IMS*. Результат показан на рисунке ниже:



6.7.1.1.3 Навигация по базе знаний ostis-системы на SCn-коде посредством задания вопросов к сущностям, обозначаемым гиперссылками соответствующих им sc-идентификаторов

Кроме этого переход от семантической окрестности одной сущности к семантической окрестности второй сущности на SCn-коде может быть осуществлён путём задания вопроса ко второй сущности. Для этого необходимо щелчком правой кнопки компьютерной мыши по sc-идентификатору сущности вызвать *контекстное меню*.

Щелчок правой кнопкой компьютерной мыши по sc-идентификатору сущности *Раздел. Технология OSTIS* вызовет *контекстное меню*, показанное на рисунке ниже:

?

Команды просмотра БЗ

Примеры команд

Экспертный режим

Русский язык

8+

Что такое Раздел. Технология OSTIS

Что такое База знаний IMS

База знаний IMS

основной идентификатор*:

База знаний IMS

идентификатор*:

SC-модель базы знаний IMS

База знаний Метасистемы IMS

База знаний IMS.OSTIS

декомпозиция раздела*:

{

1:

Раздел. Технология OSTIS

Контекст Технологии OSTIS в рамках Глобальной базы знаний

Раздел. Проект OSTIS. История, текущее состояние и перспективы эволюции и применения Технологии OSTIS

Документация. IMS

История и текущие процессы эксплуатации IMS

Раздел. Проект IMS. И

}

результат*:

Проект База знаний IMS

SC-модель базы знаний

недостаточно сформирован

...

декомпозиция SC-модели IMS

стартовый SC-элемент

Поиск...

Как выглядит указываемый SC-текст на внешних языках?

Какие варианты декомпозиции соответствуют указываемой сущности?

Какие внешние идентификаторы соответствуют указываемой сущности?

Какие структуры соответствуют заданному шаблону с фиксированной стратегией поиска?

Какие сущности являются общими по отношению к указываемой сущности?

Какие сущности являются частными по отношению к указываемой сущности?

Какие элементы принадлежат указываемому множеству и какие роли они выполняют?

Какие элементы принадлежат указываемому множеству?

Кто является автором указываемой SC-структуры?

Что это такое?

Элементом каких множеств является указываемая сущность и какие роли она там выполняет?

Элементом каких множеств является указываемая сущность?

Copyright © 2012 - 2025 OSTIS

В контекстном меню отображается перечень вопросов, которые можно задать к сущности. Данный перечень вопросов представлен в базе знаний и может быть дополнен.

Совершив щелчок левой кнопкой компьютерной мыши по SC-идентификатору вопроса *Какие сущности являются частными по отношению к указываемой сущности?*, на странице будет показана семантическая окрестность ответа на этот вопрос для указанной сущности:

?

Команды просмотра БЗ

Примеры команд

Экспертный режим

Русский язык

8+

Какие сущности являются частными по отношению к Раздел. Технология OSTIS

Что такое Раздел. Технология OSTIS

Что такое База знаний IMS

Раздел. Технология OSTIS

декомпозиция раздела*:

{

Обоснование разработки. Технология OSTIS

декомпозиция раздела*:

{

Раздел. Недостатки современных технологий разработки компьютерных систем и, в частности, интеллектуальных компьютерных систем

Раздел. Что препятствует устранению недостатков современных технологий разработки компьютерных систем и, в том числе, интеллектуальных компьютерных систем

Раздел. Тип, назначение и состав Технологии OSTIS

Раздел. Принципы, лежащие в основе Технологии OSTIS

Раздел. Актуальность Технологии OSTIS

Раздел. Архитектура компьютерных систем, построенных по Технологии OSTIS

Раздел. Модели компьютерных систем, разрабатываемых по Технологии OSTIS

декомпозиция раздела*:

{

Раздел. Многоагентные компьютерные системы, управляемые знаниями

Раздел. Смысловое представление информации в памяти компьютерных систем

декомпозиция раздела*:

{

Раздел. Принципы смыслового представления информации в памяти компьютерных систем

Раздел. Уточнение понятия семантической сети

Раздел. Семантическая сеть как графовая структура расширенного вида

Раздел. Достоинства смыслового представления информации

}

Раздел. Семантическая модель компьютерной памяти

Раздел. Графовые языки программирования, ориентированные на обработку семантических сетей

Раздел. Семантические модели компьютерных систем, управляемых знаниями – графодинамические модели параллельной асинхронной обработки знаний

Раздел. Достоинства систем, управляемых знаниями, хранимыми в семантической памяти

}

Раздел. Средства, входящие в состав Технологии OSTIS

декомпозиция раздела*:

{

Раздел. Типология средств, входящих в состав Технологии OSTIS

Раздел. Унификация структуры логико-семантических моделей компьютерных систем

Раздел. Унификация структуры логико-семантических моделей компьютерных систем

декомпозиция раздела*:

{

Раздел. Типология средств, входящих в состав Технологии OSTIS

Раздел. Унификация структуры логико-семантических моделей компьютерных систем

Раздел. Унификация структуры логико-семантических моделей компьютерных систем

}

Поиск...

Copyright © 2012 - 2025 OSTIS

Видно, что результатом ответа на вопрос *Какие сущности являются частными по отношению к указываемой сущности?* для сущности *Что такое Раздел. Технология OSTIS* является иерархия декомпозиций данной сущности (раздела) на более частные сущности (разделы).

Каждому вопросу задаётся компонент пользовательского интерфейса (кнопка в контекстном меню), а также агент, который выполняет действие после задания этого вопроса (нажатия на кнопку в контекстном меню).

6.7.1.2 Принципы навигации по базе знаний ostis-системы при помощи SCg-кода

Дополнительным инструментом навигации по базе знаний ostis-системы является SCg-код.

SCg-код (Semantic Code graphical) — язык внешнего графического представления конструкций SC-кода.

6.7.1.2.1 Навигация по базе знаний ostis-системы на SCg-коде посредством двойного щелчка левой кнопки компьютерной мыши по sc.g-элементам

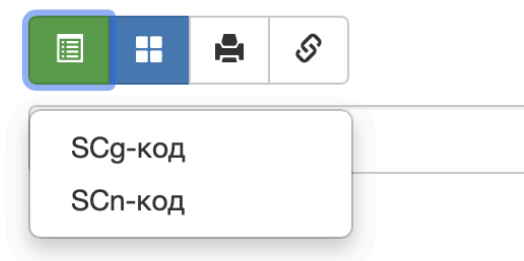
Для активации *режима навигации по базе знаний ostis-системы* на SCg-коде необходимо активировать *экспертный режим* (кнопку в панели сверху). Результат активации *экспертного режима* для самой первой страницы показан ниже:

The screenshot displays the OSTIS expert mode interface. At the top, a navigation bar includes a help icon, a language dropdown set to 'Русский язык', and a 'g+' icon. Below this, a toolbar contains icons for a list, a grid, a document, and a link. The main content area is divided into two panels. The left panel, titled 'Команды просмотра БЗ' and 'Примеры команд', contains three buttons: 'Какие сущности являются частными по отношению к Раздел. Технология OSTIS', 'Что такое Раздел. Технология OSTIS', and 'Что такое База знаний IMS'. The right panel, titled 'База знаний IMS', displays a hierarchical SCg code structure. It starts with 'основной идентификатор:' followed by 'Knowledge base IMS', which branches into 'Английский язык' and 'База знаний IMS'. 'База знаний IMS' further branches into 'Русский язык' and 'системный идентификатор:'. The 'системный идентификатор:' section contains 'knowledge_base_IMS', which branches into 'идентификатор:' and 'SC-модель базы знаний IMS'. 'SC-модель базы знаний IMS' branches into 'Русский язык' and 'База знаний Метасистемы IMS'. 'База знаний Метасистемы IMS' branches into 'Русский язык' and 'База знаний IMS.OSTIS'. 'База знаний IMS.OSTIS' branches into 'Русский язык' and 'декомпозиция раздела:'. The 'декомпозиция раздела:' section contains a list of items: '1:', 'Раздел. Технология OSTIS', 'Контекст Технологии OSTIS в рамках Глобальной базы знаний', 'Раздел. Проект OSTIS. История, текущее состояние и перспективы эволюции и применения Технологии OSTIS', 'Документация. IMS', 'История и текущие процессы эксплуатации IMS', and 'Раздел. Проект IMS. История, текущие процессы и план развития IMS'. The 'результат:' section contains 'Проект База знаний IMS', 'sc-модель базы знаний', 'недостаточно сформированная структура', and '...'. At the bottom of the interface, a footer bar contains a red circle icon and the text 'Copyright © 2012 - 2025 OSTIS'.

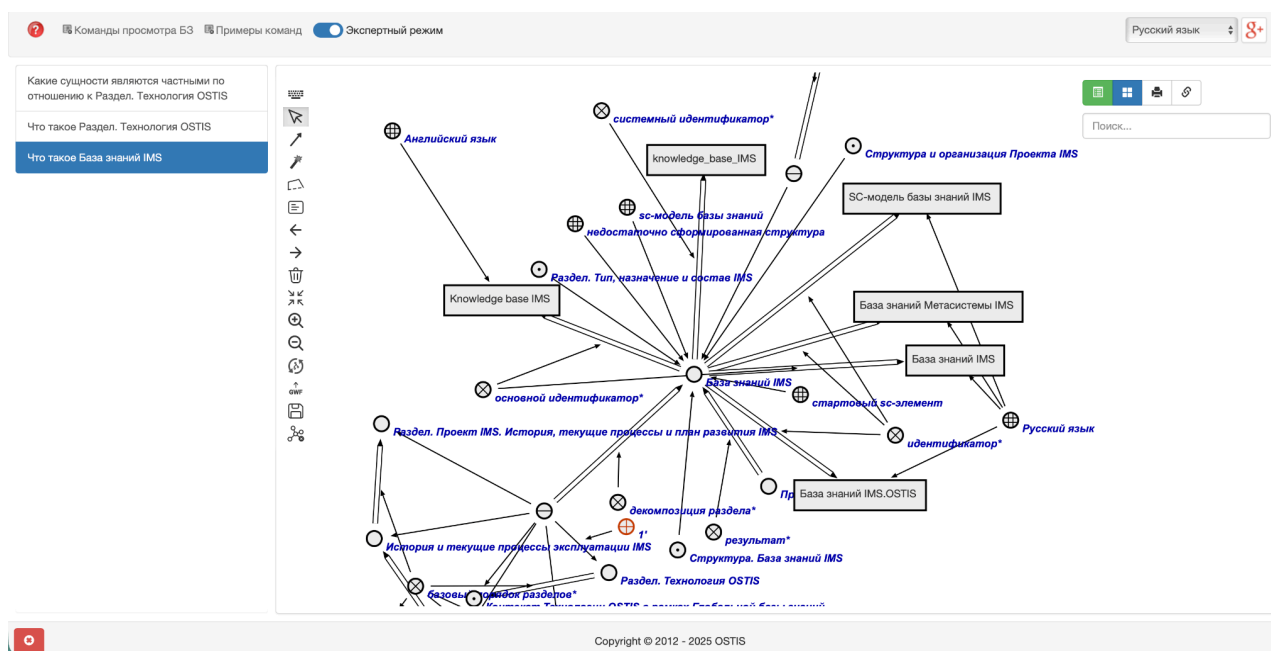
Экспертный режим активирует визуализацию всех sc-идентификаторов сущности в рамках её семантической окрестности (по-умолчанию визуализируются только основные sc-идентификаторы сущностей), а также активирует визуализацию *панели инструментов справа* (4 кнопки).

Первая кнопка в *панели инструментов справа* отвечает за выбор *режима навигации по базе знаний ostis-системы*. Режим *навигации по базе знаний ostis-системы на SCn-коде* является режимом по-умолчанию.

Для выбора *режима навигации по базе знаний ostis-системы на SCg-коде* необходимо совершить щелчок левой кнопкой компьютерной мыши по первой кнопке в *панели инструментов справа*. Результат щелчка показан ниже:



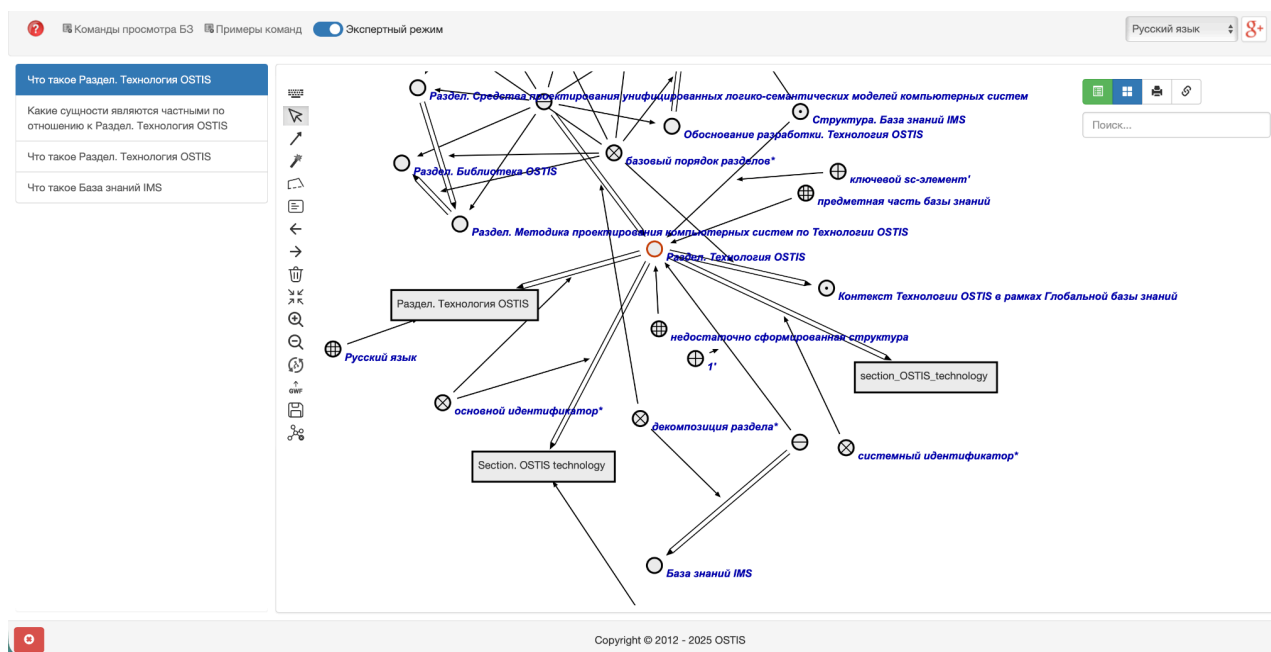
И далее из выпадающего списка щелчком левой кнопки компьютерной мыши выбрать элемент *SCg-код*. Результат щелчка показан ниже:



На главной части страницы — *sc.g-сцене* — показана семантическая окрестность сущности *База знаний IMS*. на SCg-коде.

Двойным щелчком левой кнопки компьютерной мыши по *sc.g-элементу* можно совершить переход от семантической окрестности заданной сущности к семантической окрестности сущности, обозначаемой этим *sc.g-элементом*. При этом результатом щелчка будет являться семантическая окрестность этой сущности, представленная на SCg-коде.

После совершения двойного щелчка левой кнопки компьютерной мыши по *sc.g-элементу*, обозначающему сущность *Раздел. Технология OSTIS*, на странице будет показана семантическая окрестность сущности *Раздел. Технология OSTIS* на SCg-коде. Это показано ниже:



Данный инструмент позволяет не только визуализировать семантические окрестности сущностей на SCg-коде, но и редактировать их. Для этого в левой части *sc.g-сцены* присутствует панель инструментов, с помощью которой можно создавать, изменять и удалять *sc.g-конструкции*. В данной ЛР работа с этими инструментами опускается.

6.7.1.2.2 Навигация по базе знаний ostis-системы на SCg-коде посредством задания вопросов к сущностям, обозначаемым *sc.g-элементами*

Переход от семантической окрестности одной сущности к семантической окрестности второй сущности на SCg-коде, как и на SCn-коде, может быть

осуществлѐн путѐм задания вопроса ко второй сущности. Для этого необходимо щелчком правой кнопки компьютерной мыши по sc-идентификатору сущности вызвать *контекстное меню*, выбрать необходимый вопрос и осуществить щелчок левой кнопкой компьютерной мыши по этому вопросу. Результатом последнего щелчка будет являться семантическая окрестность ответа на вопрос для заданной сущности.

6.7.1.3 Навигация по базе знаний ostis-системы на SCg-коде или SCn-коде посредством поиска сущности по её sc-идентификатору

На каждой странице справа отображается *строка поиска*.



Поиск...

Строка поиска является специализированным инструментом навигации по базе знаний ostis-системы.

Для активации *строки поиска* необходимо совершить щелчок левой кнопкой компьютерной мыши по ней. Далее в строке поиска можно напечатать при помощи клавиатуры префиксную часть sc-идентификатора некоторой сущности.

Предварительно перейдя в режим навигации SCn-коде, активировав строку поиска и напечатав в ней строку “множ” в качестве префиксной части sc-идентификатора сущности *множество*, под строкой поиска появится выпадающий список с sc-идентификаторами сущностей, которые имеют в качестве префикса строку “множ”. Результат представлен ниже:

множ

SC-узел обозначающий
множество всех пользователей
системы

sc-агент интерпретации scr-
оператора ожидания завершения
выполнения **множества** scr-
программ

sc-агент интерпретации scr-
оператора поиска пятиэлементной
конструкции с формированием
множеств

sc-агент интерпретации scr-
оператора поиска трехэлементной
конструкции с формированием
множеств

sc-агент интерпретации scr-
оператора удаления **множества**
элементов пятиэлементной

При помощи данной строки поиска процесс поиска осуществляется только по префиксу sc-идентификатора. При этом для активации поиска необходимо, чтобы префикс имел длину бОльшую чем 3 символа.

Появившийся список можно пролистывать вниз и вверх посредством колеса компьютерной мыши.

Очевидно, что sc-идентификаторов, имеющих в качестве префиксной части строку “множ”, может быть большое количество. Поэтому данную строку можно дополнить до любого более длинного префикса sc-идентификатора той сущности, семантическую окрестность которой необходимо отобразить на странице.

Дополнив заданную строку до строки “множество м”, получается следующий результат:

МНОЖЕСТВО М
МНОЖЕСТВО МНОЖЕСТВ
МНОЖЕСТВО МОЩНОСТИ 1
МНОЖЕСТВО МОЩНОСТИ 2
МНОЖЕСТВО МОЩНОСТИ ТРИ

Количество совпадений по префиксу уменьшилось до минимума. Далее из выпадающего списка можно выбрать *sc*-идентификатор необходимой сущности и совершить щелчок левой кнопкой компьютерной мыши по *sc*-идентификатору этой сущности.

Выбрав *sc*-идентификатор сущности *множество множеств* и совершив щелчок левой кнопкой компьютерной мыши по *sc*-идентификатору этой сущности, на странице будет отображена семантическая окрестность сущности *множество множеств* на SCn-коде:

?

Команды просмотра БЗ

Примеры команд

Экспертный режим

Русский язык

g+

Что такое семейство множеств

Что такое Раздел. Технология OSTIS

Какие сущности являются частными по отношению к Раздел. Технология OSTIS

Что такое Раздел. Технология OSTIS

Что такое База знаний IMS

семейство множеств

→ основной идентификатор:

семейство множеств

→ идентификатор:

множество множеств

→ второй домен:

используемые утверждения*

→ строгое включение:

класс классов

→ ключевой *sc*-элемент:

семейство множеств – это множество, элементами которого являются знаки множеств.

Раздел. Предметная область множеств

немаксимальный класс объектов исследования:

Предметная область множеств

...

разбиение:

множество

...

объединение:

...

Поиск...

Стоит отметить, что форма отображения результата поиска по префиксу *sc*-идентификатора сущности не зависит от изначального выбранного режима навигации по базе знаний *ostis*-системы — в любом случае результат будет отображаться на SCn-коде.

6.8 Принципы решения задач в *ostis*-системах

6.8.1 Агентно-ориентированная архитектура *ostis*-систем

Агентно-ориентированная архитектура в ostis-системах — это модель организации, в которой обработка знаний выполняется не монолитной программой, а совокупностью независимых, взаимодействующих друг с другом программных модулей, называемых *sc*-агентами. Эти агенты оперируют в едином информационном пространстве — семантической памяти (*sc*-памяти).

6.8.2 Понятие агента

Агент — это субъект, способный выполнять действия в *sc*-памяти, принадлежащие некоторому определенному классу логически атомарных действий.

В отличие от классических многоагентных систем, где агенты обмениваются сообщениями напрямую, в *ostis*-системах коммуникация происходит опосредованно, через общую *sc*-память. Один агент может создать или изменить данные, на которые отреагирует другой агент. Это обеспечивает слабую связанность компонентов системы.

Добавление новых или удаление существующих агентов не требует изменения других агентов, что упрощает расширение и поддержку системы.

6.8.2.1 Понятие платформенно-независимого агента и понятие платформенно-зависимого агента

Агенты делятся на два основных класса:

- *платформенно-независимые агенты*, или *sc*-агенты — это агенты, которые реализуются на Языке SCP и интерпретируются специальным компонентом — *scp*-машиной.
- *платформенно-зависимые агенты* — это агенты, которые реализуются на языках низкого уровня, таких как C++, с использованием API *sc*-машины.

Возможность создания платформенно-независимых агентов позволяет создавать гибкие, масштабируемые и расширяемые интеллектуальные системы, где знания (декларативные) и методы их обработки (процедурные) представлены в одной и той же форме.

6.8.3 Понятие события в sc-памяти

Логическая атомарность выполняемых агентом действий предполагает, что каждый sc-агент реагирует на соответствующий ему класс событий, происходящих в sc-памяти, и осуществляет определенное преобразование sc-текста, находящегося в семантической окрестности обрабатываемого события.

событие в sc-памяти — это изменение состояния некоторой sc-структуры (sc-текста), вызванное добавлением, удалением или модификацией sc-элементов и sc-связей между ними.

элементарное событие в sc-памяти — это событие в sc-памяти, в результате выполнения которого изменяется состояние только одного sc-элемента.

Элементарное событие в sc-памяти классифицируется на:

- *событие добавления sc-коннектора, инцидентного заданному sc-элементу;*
- *событие добавления sc-дуги, выходящей из заданного sc-элемента;*
- *событие добавления sc-дуги, входящей в заданный sc-элемент;*
- *событие добавления sc-ребра, инцидентного заданному sc-элементу;*
- *событие удаления sc-коннектора, инцидентного заданному sc-элементу;*
- *событие удаления sc-дуги, выходящей из заданного sc-элемента;*
- *событие удаления sc-дуги, входящей в заданный sc-элемент;*
- *событие удаления sc-ребра, инцидентного заданному sc-элементу;*
- *событие удаления sc-элемента;*
- *событие изменения содержимого файла.*

6.8.4 Понятие абстрактного sc-агента

Поскольку предполагается, что копии одного и того же агента или функционально эквивалентные агенты могут работать в разных ostis-системах, будучи при этом физически разными агентами, то целесообразно рассматривать свойства и классификацию не агентов, а классов функционально эквивалентных агентов, которые будем называть абстрактными sc-агентами.

абстрактный sc-агент — это класс функционально эквивалентных sc-агентов, разные экземпляры (то есть представители) которого могут быть реализованы по-разному.

6.8.4.1 Понятие неатомарного абстрактного sc-агента и понятие атомарного абстрактного sc-агента

С точки зрения реализации можно выделить два класса абстрактных sc-агентов:

- *неатомарный абстрактный sc-агент* — это абстрактный sc-агент, декомпозируемый на коллектив более простых абстрактных sc-агентов, каждый из которых, в свою очередь, может быть как атомарным абстрактным sc-агентом, так и неатомарным абстрактным sc-агентом;
- *атомарный абстрактный sc-агент* — это абстрактный sc-агент, для которого уточняется конкретная, соответствующая данному агенту программа.

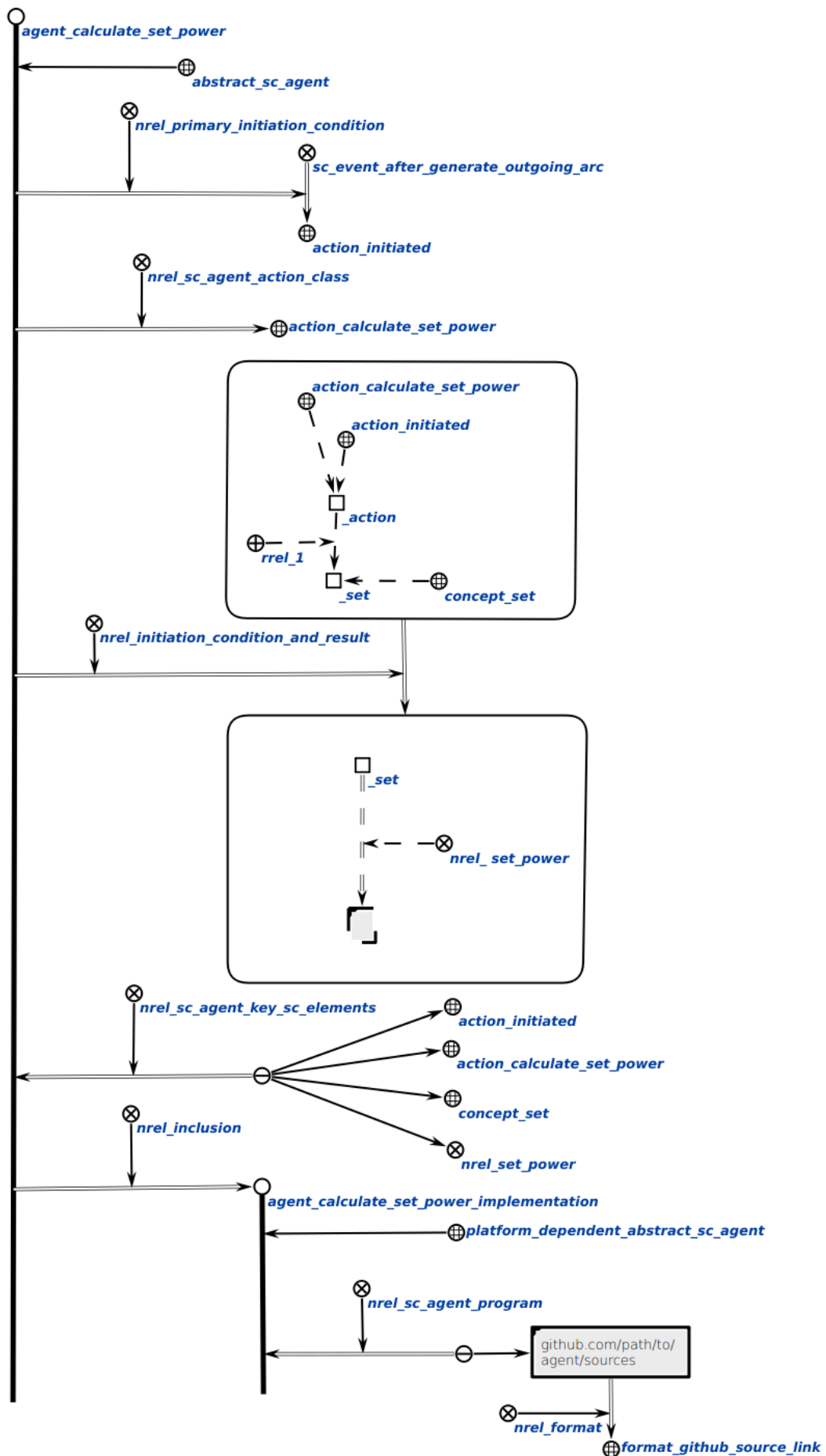
Каждый абстрактный sc-агент имеет соответствующую ему спецификацию.

6.8.5 Понятие спецификации абстрактного sc-агента

спецификация абстрактного sc-агента — это sc-знание, которое содержит:

- формальное описание *первичного условия инициирования данного sc-агента* (то есть такой ситуации в sc-памяти, которая побуждает sc-агента перейти в активное состояние и начать проверку наличия своего полного условия инициирования);
- указание *класса действий*, выполняемых этим sc-агентом;
- формальное описание *полного условия инициирования данного sc-агента* (то есть тех ситуаций в sc-памяти, которые иницируют деятельность данного sc-агента);
- указание *ключевых sc-элементов* этого sc-агента (то есть тех sc-элементов, хранимых в sc-памяти, которые для данного sc-агента являются «точками опоры»);
- строгое, полное, однозначно понимаемое описание деятельности данного sc-агента, оформленное при помощи каких-либо понятных, общепринятых средств, не требующих специального изучения, например на естественном языке;
- описание результатов выполнения данного sc-агента.

На рисунке ниже представлен пример спецификации абстрактного sc-агента подсчёта мощности множества на SCg-коде:



Ниже представлен пример спецификации абстрактного sc-агента подсчёта мощности множества на SCs-коде:

```
// Абстрактный sc-агент
agent_calculate_set_power
<- abstract_sc_agent;
=> nrel_primary_initiation_condition:
  // Класс sc-события и элемент прослушивания (подписки)
  (sc_event_after_generate_outgoing_arc => action_initiated);
=> nrel_sc_agent_action_class:
  // Класс действий, выполняемых агентом
  action_calculate_set_power;
=> nrel_initiation_condition_and_result:
  (..agent_calculate_set_power_initiation_condition
   => ..agent_calculate_set_power_result_condition);
<= nrel_sc_agent_key_sc_elements:
// Множество ключевых sc-элементов, используемых этим агентом
{
  action_initiated;
  action_calculate_set_power;
  concept_set;
  nrel_set_power
};
=> nrel_inclusion:
  // Экземпляр абстрактного sc-агента; конкретная реализация агента на C++
  agent_calculate_set_power_implementation
  (*
    <- platform_dependent_abstract_sc_agent;;
    // Набор ссылок с путями к исходным кодам программ агента
    <= nrel_sc_agent_program:
      {
        [github.com/path/to/agent/sources]
        (*
          => nrel_format: format_github_source_link;;
        *)
      };
  *);;

// Полное условие инициирования агента
..agent_calculate_set_power_initiation_condition
= [*
  action_calculate_set_power _-> .._action;;
  action_initiated _-> .._action;;
  .._action _-> rrel_1:: .._set;;
  concept_set _-> .._set;;
```

```

*];;
// Агент проверяет по этому sc-шаблону, что инициированное действие является
экземпляром класса action_calculate_set_power и что у него есть аргумент.
// Полное условие результата агента
..agent_calculate_set_power_result_condition
= [*
  .._set _=> nrel_set_power:: _[];;
*];;
// Агент проверяет по этому sc-шаблону, что результат действия содержит
sc-конструкцию, созданную после выполнения действия.

```

6.8.5.1 Понятие первичного условия инициирования sc-агента

*первичное условие инициирования sc-агента** — это ориентированное квазибинарное отношение, первыми элементами sc-пар которого являются sc-узлы, обозначающие абстрактные sc-агенты, а вторыми элементами sc-пар которого являются ориентированные sc-пары, не являющиеся sc-парами принадлежности, первыми элементами которых являются классы событий в sc-памяти, а вторыми элементами которых являются sc-элементы, являющиеся ключевыми sc-элементами событий в sc-памяти, возникновение которых побуждает заданных sc-агентов перейти в активное состояние и начать проверку наличия своих полных условий инициирования.

6.8.5.2 Понятие класса действия sc-агента

*класс действий sc-агента** — это ориентированное бинарное отношение, первыми элементами sc-пар которого являются sc-узлы, обозначающие абстрактные sc-агенты, а вторыми элементами sc-пар которого являются sc-узлы, обозначающие классы действий, выполняемых этими sc-агентами.

6.8.5.3 Понятие условия инициирования и результата sc-агента

*условие инициирования и результат sc-агента** — это ориентированное квазибинарное отношение, первыми элементами sc-пар которого являются sc-узлы, обозначающие абстрактные sc-агенты, а вторыми элементами sc-пар которого являются ориентированные sc-пары, не являющиеся sc-парами принадлежности, первыми элементами которых являются условия инициирования заданных абстрактных sc-агентов, а вторыми элементами которых являются условия результата выполнения заданных абстрактных sc-агентов.

6.8.5.4 Понятие ключевых sc-элементов sc-агента

*ключевые sc-элементы sc-агента** — это ориентированное квазибинарное отношение, вторыми элементами sc-пар которого являются sc-узлы, обозначающие абстрактные sc-агенты, а первыми элементами sc-пар которого являются неориентированные sc-связки sc-элементов, хранимых в sc-памяти, которые для данных sc-агентов являются «точками опоры», то есть которые используются в процессе выполнения действий этими sc-агентами.

6.8.5.5 Понятие программы sc-агента

*программа sc-агента** — это ориентированное бинарное отношение, вторыми элементами sc-пар которого являются sc-узлы, обозначающие реализации абстрактных sc-агентов, а первыми элементами sc-пар которого являются неориентированные sc-связки, элементами которых являются sc-структуры, обозначающие scr-программы, и/или файлы с исходным кодом программ этих sc-агентов.

6.8.5.6 Понятие агентной scr-программы

агентная scr-программа — это scr-программа, которая представляет собой реализацию некоторого агента обработки знаний, имеющая ровно два in-параметра', значением первого из которых является класс событий, на которые реагирует указанный sc-агент, а значением второго из которых является sc-элемент, с которым непосредственно связано событие, в результате возникновения которого был инициирован соответствующий sc-агент (то есть, например, созданная либо удаляемая sc-дуга).

6.8.6 Понятие машины обработки знаний ostis-системы

машина обработки знаний ostis-системы — это совокупность интерпретаторов всех навыков, составляющих некоторый решатель задач заданной ostis-системы.

С учетом многоагентного подхода к обработке информации, используемого в рамках *Технологии OSTIS*, машина обработки знаний представляет собой sc-агент (чаще всего — неатомарный sc-агент), в состав которого входят более простые sc-агенты, обеспечивающие интерпретацию соответствующего множества методов.

Таким образом, машина обработки знаний в общем случае представляет собой иерархическую систему sc-агентов.

6.9 Принципы разработки ostis-систем

6.9.1 Проект примера ostis-системы

В качестве базового проекта разработки рекомендуется использовать проект примера ostis-системы, который [доступен в открытом репозитории на GitHub](#). Данный проект представляет собой исходный код примера ostis-системы, на основе которого можно получить новую ostis-систему посредством изменения, замещения существующих компонентов и добавления новых компонентов.

6.9.1.1 Компоненты проекта примера ostis-системы

Для установки, конфигурации и запуска проекта примера ostis-системы необходимо следовать инструкциям, указанным в [README-файле проекта на GitHub](#).

Ключевыми компонентами данного репозитория являются:

- директория `knowledge-base/` — содержит исходники базы знаний ostis-системы;
- директория `problem-solver/` — содержит исходники решателя задач ostis-системы;
- директория `interface/` — содержит исходники пользовательского интерфейса ostis-системы.

Вообще говоря, любая *ostis-система* — это интеллектуальная система, состоящая из базы знаний, решателя задач и пользовательского интерфейса.

6.9.2 Принципы разработки базы знаний ostis-системы

6.9.2.1 Понятие фрагмента базы знаний

фрагмент базы знаний — это логически выделяемое sc-знание от других sc-знаний в базе знаний, которое может быть независимо формализовано, протестировано и интегрировано в общую структуру базы знаний.

Формализацию фрагментов базы знаний следует рассматривать как методологический приём для формализации базы знаний обеспечивающий поэтапное и систематическое построение полной базы знаний ostis-системы.

Такой подход позволяет уменьшить сложность процесса разработки за счёт декомпозиции предметной области на относительно независимые, но взаимосвязанные компоненты.

В качестве примера рассмотрим процесс формализации фрагмента предметной области, описывающей университеты — Предметной области университетов. Для формализации рекомендуется использовать SCs-код. При выборе SCg-кода для формализации каждый sc.g-узел следует сопровождать терминами, соответствующими системным sc-идентификаторам, а не основным. Это обусловлено тем, что операция склеивания (идентификации) одноимённых sc.g-элементов на этапе *сборки базы знаний* осуществляется именно на основании системных sc-идентификаторов.

6.9.2.2 Понятие исходного файла базы знаний

исходный файл базы знаний — это файл с расширением .gwf или .scs, содержащий некоторый фрагмент базы знаний на SCg-коде или SCs-коде.

исходный файл базы знаний с расширением .gwf — это исходный файл, содержащий некоторый фрагмент базы знаний на SCg-коде.

исходный файл базы знаний с расширением .scs — это исходный файл, содержащий некоторый фрагмент базы знаний на SCs-коде.

6.9.2.3 Принципы работы с инструментами разработки исходных текстов баз знаний

Для разработки текстов на SCg-коде следует использовать:

- **Десктопный редактор текстов на SCg-коде:**
 - [Редактор KBE для Windows](#);
 - [Редактор KBE для Linux](#).

Для разработки текстов на SCs-коде следует использовать обычный текстовый редактор, встроенный в IDE или нет.

6.9.2.3.1 Принципы работы с редактором KBE

6.9.2.3.1.1 Создание sc.g-узла заданного типа

Для создания sc.g-узла заданного типа необходимо включить режим выделения sc.g-элемента, то есть выбрать инструмент выделения sc.g-элемента

на панели инструментов. Инструмент выделения sc.g-элемента показан на рисунке ниже:



Для активации инструмента выделения sc.g-элемента достаточно совершить щелчок левой кнопкой компьютерной мыши по изображению этого инструмента или нажать на клавишу с цифрой 1.

Далее с помощью двойного щелчка левой кнопкой компьютерной мыши по свободной зоне холста редактора можно создать константный sc.g-узел. После создания sc.g-узел приобретает жёлтый цвет. Результат показан на рисунке ниже:



С помощью щелчка левой кнопкой компьютерной мыши по свободной зоне холста редактора можно убрать выделение заданного sc.g-узла. После этого sc.g-узел приобретает чёрный цвет:



Чтобы снова выделить заданный sc.g-узел необходимо совершить щелчок левой кнопкой компьютерной мыши по нему. После выделения sc.g-узел также окрашивается в жёлтый цвет.



Наведя курсор компьютерной мыши на созданный sc.g-узел, зажав левую кнопку компьютерной мыши и переместив курсор компьютерной мыши, можно переместить созданный sc.g-узел в любую точку свободной зоны холста редактора. На рисунках ниже показаны расположения до и после перемещения выделенного sc.g-узла соответственно:



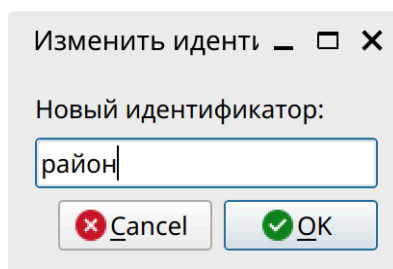
Также для созданного sc.g-узла можно уточнить его тип. Для этого необходимо выделить sc.g-узел, затем на клавиатуре нажать на клавишу с английской буквой T (от англ. Type). После этого появится диалоговое окно для выбора типа sc.g-узла. Фрагмент данного диалогового окна показан на рисунке ниже:

Константы (C)	Переменные (V)
☐ node/const/perm/general	☐ node/var/perm/general
☒ node/const/perm/group	☒ node/var/perm/group
☒ node/const/perm/relation	☒ node/var/perm/relation
☒ node/const/perm/role	☒ node/var/perm/role
☒ node/const/perm/struct	☒ node/var/perm/struct
☒ node/const/perm/super_group	☒ node/var/perm/super_group
☒ node/const/perm/terminal	☒ node/var/perm/terminal
☒ node/const/perm/tuple	☒ node/var/perm/tuple

Совершив щелчок левой кнопкой компьютерной мыши по необходимому типу, можно установить новый тип для заданного sc.g-узла. После выбора типа появившееся диалоговое окно исчезает, а заданный sc.g-узел меняет тип:



Также для созданного sc.g-узла можно задать основной sc-идентификатор. Для этого необходимо выделить заданный sc.g-узел и нажать на клавишу с английской буквой I (от англ. Identifier). После этого появится диалоговое окно, в котором можно ввести sc-идентификатор:



В появившемся окне в строке ввода следует ввести строку и нажать на кнопку “OK” (или клавишу Enter). Результат задания sc-идентификатора показан на рисунке ниже:



район

Если основной sc-идентификатор sc.g-узла длинный, то следует воспользоваться любым текстовым редактором и разбить его на несколько строк. На рисунке ниже показан пример sc.g-узла с длинным основным sc-идентификатором:

Предметная область backend разработки на языке программирования Python



На рисунке ниже показан пример sc.g-узла с длинным основным sc-идентификатором, представленным в виде двух строк, разделённых символом переноса строки:

*Предметная область backend
разработки на языке программирования Python* 

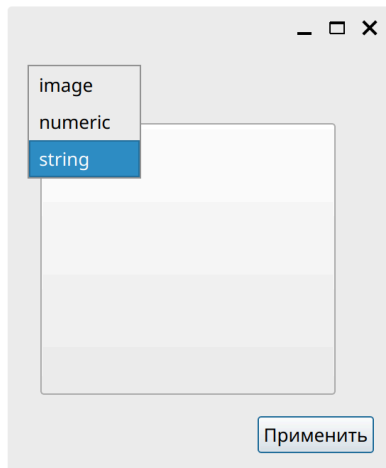
Также можно поменять расположение основного sc-идентификатора sc.g-узла. Для этого необходимо выделить заданный sc.g-узел, навести курсор компьютерной мыши на sc-идентификатор этого sc.g-узла, зажать левую кнопку компьютерной мыши и переместить курсор компьютерной мыши в необходимое место по отношению к sc.g-узлу. На рисунке ниже показан sc.g-узел с основным sc-идентификатором, имеющим стандартное расположение относительно этого sc.g-узла:


район

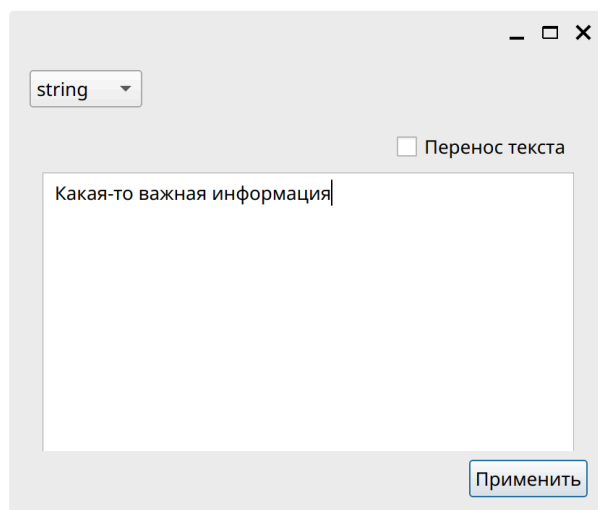
На рисунке ниже показан sc.g-узел с основным sc-идентификатором, для которого было изменено расположение относительно этого sc.g-узла:

район 

Любой sc.g-узел можно сделать файлом ostis-системы. Для этого необходимо выделить этот sc.g-узел и нажать на клавишу с английской буквой С (от англ. Content). После появится диалоговое окно для выбора типа содержимого файла ostis-системы и самого содержимого.



В появившемся окне в выпадающем списке можно выбрать тип “string”, ввести необходимую текстовую информацию в поле ввода и нажать на кнопку “Применить” (или клавишу Enter):

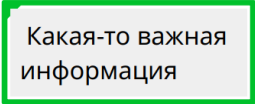


Можно выполнить аналогичные действия, если файл ostis-системы должен хранить изображение или число.

После задания содержимого возле этого sc.g-узла появится синий кружок, что будет означать, что содержимое этого sc.g-узла “спрятано”, то есть не показывается на холсте редактора:



Чтобы включить показ содержимого sc.g-узла необходимо выделить этот sc.g-узел и нажать на клавишу с английской буквой H (от англ. Hide). Результат показан на рисунке ниже:



Какая-то важная информация

6.9.2.3.1.2 Создание sc.g-коннектора заданного типа

Для создания sc.g-коннектора заданного типа между двумя заданными sc.g-элементами необходимо включить режим создания sc.g-коннектора, то есть выбрать инструмент создания sc.g-коннектора. Инструмент создания sc.g-коннектора показан на рисунке ниже:

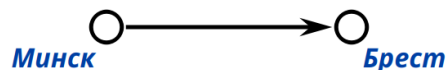


Для активации инструмента создания sc.g-коннектора достаточно совершить щелчок левой кнопкой компьютерной мыши по изображению этого инструмента или нажать на клавишу с цифрой 2.

Далее, совершив щелчок левой кнопкой компьютерной мыши по выделенному sc.g-элементу и переместив курсор компьютерной мыши в сторону от выделенного sc.g-элемента, на холсте редактора появится штриховая линия, являющаяся создаваемым sc.g-коннектором:



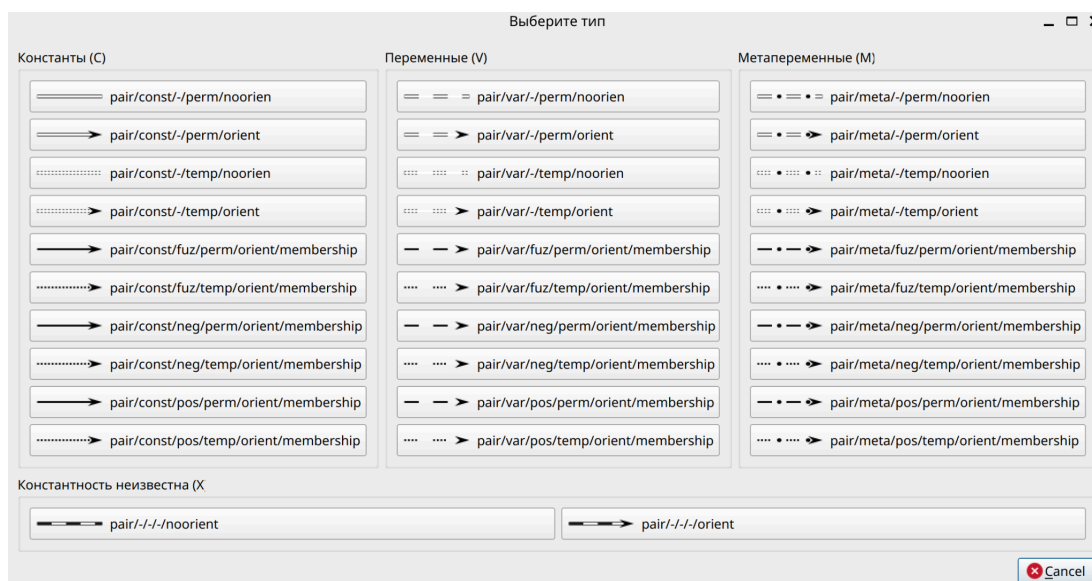
После переместив курсор компьютерной мыши на второй заданный sc.g-элемент и совершив щелчок левой кнопкой компьютерной мыши по нему, на холсте редактора появится сплошная стрелка, являющаяся созданным sc.g-коннектором. На рисунке ниже показан пример результата создания sc.g-коннектора между двумя заданными sc.g-элементами:



Для изменения типа созданного sc.g-коннектора следует включить режим выделения sc.g-элемента, то есть выбрать инструмент выделения sc.g-элемента на панели инструментов. На рисунке ниже показан результат выделения созданного sc.g-коннектора:



Выделив созданный sc.g-коннектор и нажав на клавишу с английской буквой T, можно вызвать диалоговое окно для выбора типа sc.g-коннектора. На рисунке ниже показан фрагмент диалогового окна для выбора типа sc.g-коннектора:



Совершив щелчок левой кнопкой компьютерной мыши по необходимому типу, можно установить новый тип для заданного sc.g-коннектора. После выбора типа появившееся диалоговое окно исчезает, а заданный sc.g-коннектор меняет тип:

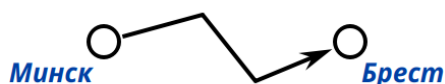


При необходимости (в частности, чтобы избежать наложение sc.g-коннекторов) можно сделать sc.g-коннекторы произвольной формы.

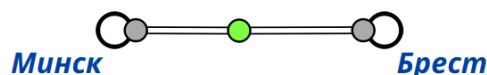
Чтобы создать такой sc.g-коннектор, необходимо перед перемещением курсора компьютерной мыши к sc.g-элементу, в который создаваемый sc.g-коннектор должен входить, совершить щелчок левой кнопкой компьютерной мыши по свободной зоне холста редактора. Процесс создания такого sc.g-коннектора показан на рисунке ниже:



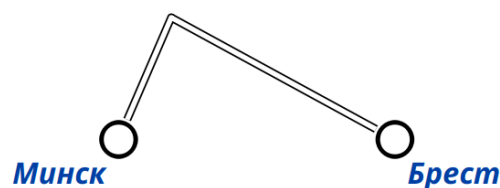
В результате будет создан sc.g-коннектор с изгибами:



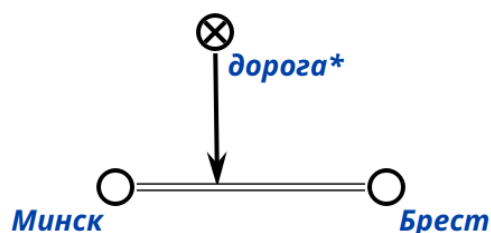
Кроме этого форму sc.g-коннектора можно изменить и после его создания. Для этого достаточно выделить sc.g-коннектор и затем сделать двойной щелчок левой кнопкой компьютерной мыши в то место sc.g-коннектора, где должен быть изгиб. На месте двойного щелчка левой кнопкой компьютерной мыши появиться маркер:



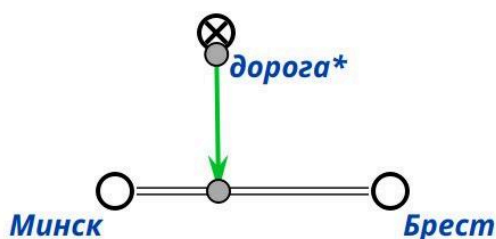
Переместив курсор компьютерной мыши на этот маркер, зажав левую кнопку компьютерной мыши и переместив курсор в сторону, перпендикулярную прямой, проходящей через заданный sc.g-коннектор, можно получить изгиб для этого sc.g-коннектора:



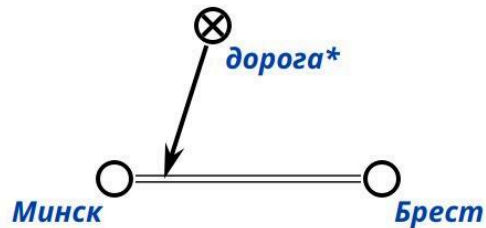
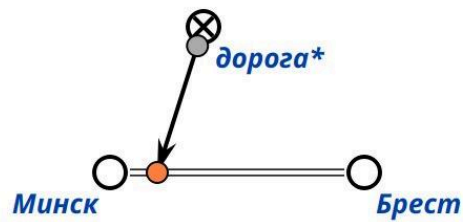
Стоит отметить, что sc.g-коннекторы могут быть заданы не только между sc.g-элементами, которые являются sc.g-узлами, но и между sc.g-коннекторами:



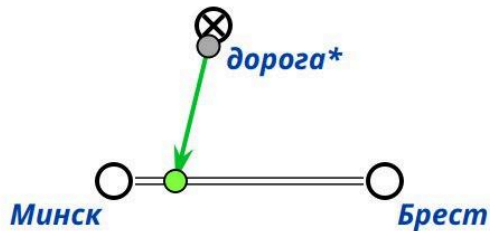
Чтобы изменить положение заданного sc.g-коннектора в первую очередь необходимо, используя соответствующий инструмент, выделить этот sc.g-коннектор. В результате на концах заданного sc.g-коннектора появятся специальные маркеры, показанные на рисунке ниже:



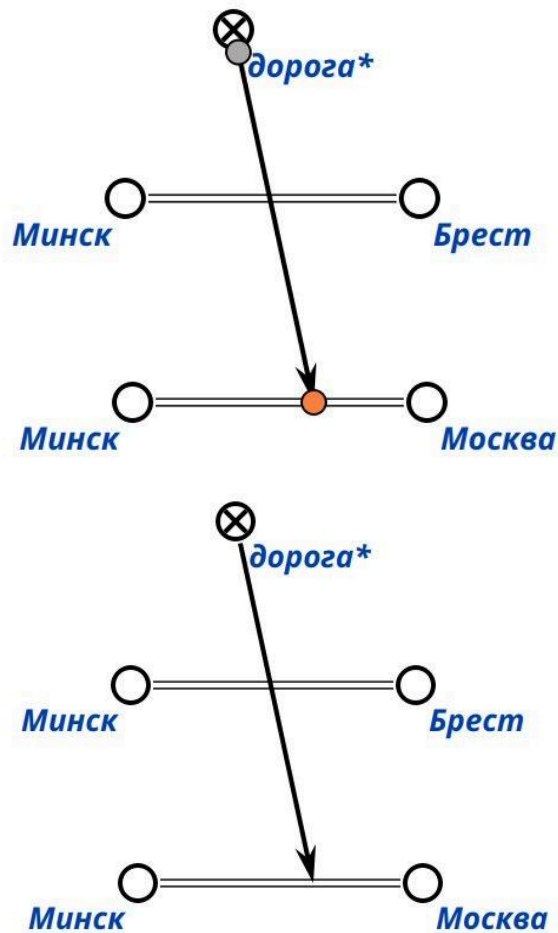
Далее, наведя курсор компьютерной мыши по маркеру возле стрелки заданного sc.g-коннектора, зажав левую кнопку компьютерной мыши, переместив курсор в место, куда должен вести заданный sc.g-коннектор, отпустив левую кнопку компьютерной мыши, переместив курсор компьютерной мыши на свободную зону холста редактора и совершив щелчок левой кнопкой компьютерной мыши, можно изменить положение заданного sc.g-коннектора.



Чтобы изменить инцидентные sc.g-элементы для заданного sc.g-коннектора необходимо выделить заданный sc.g-коннектор. В результате появятся маркеры концов заданного sc.g-коннектора.



После, переместив курсор компьютерной мыши на один из маркеров, который следует переместить к новому инцидентному sc.g-элементу, зажав левую кнопку компьютерной мыши, переместив курсор компьютерной мыши к новому инцидентному sc.g-элементу, отпустив левую кнопку компьютерной мыши, переместив курсор компьютерной мыши на свободную зону холста редактора и совершив щелчок левой кнопкой компьютерной мыши, можно изменить инцидентный sc.g-элемент для заданного sc.g-коннектора.



6.9.2.3.1.3 Создание sc.g-шины

При формализации фрагментов баз знаний на SCg-коде частой ситуацией является ситуация, когда из одного и того же sc.g-элемента выходит и/или когда в один и тот же sc.g-элемент входит большое количество sc.g-коннекторов. Некоторые из этих sc.g-коннекторов могут накладываться друг на друга, что может влиять на читаемость того, что представлено на SCg-коде.

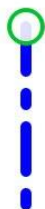
В таких случаях для улучшения читаемости sc.g-текста следует использовать sc.g-шину — синтаксически выделяемый sc.g-элемент, который не является графическим обозначением какого-либо sc-элемента и используется для увеличения области взаимодействия с тем sc.g-элементом, из которого данная sc.g-шина выходит. Можно сказать, что sc.g-шина является одним из элементов синтаксического сахара, используемого для формализации текста на SCg-коде.

Для создания sc.g-шины необходимо включить режим создания scg-шины, то есть выбрать инструмент создания sc.g-шины на панели инструментов. Инструмент создания sc.g-шины показан на рисунке ниже:

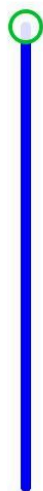


Для активации инструмента создания sc.g-шины достаточно совершить щелчок левой кнопкой компьютерной мыши по изображению этого инструмента или нажать на клавишу с цифрой 3.

Далее необходимо совершить двойной щелчок левой кнопкой компьютерной мыши по заданному sc.g-элементу и переместить курсор компьютерной мыши в свободную зону холста редактора. В результате на холсте будет отображаться штриховая линия, являющаяся создаваемой sc.g-шиной:



После, совершив щелчок левой кнопкой компьютерной мыши по свободной зоне холста редактора, можно закрепить часть sc.g-шины на холсте редактора:



Не перемещая курсор компьютерной мыши и совершив ещё один щелчок левой кнопкой компьютерной мыши по той же свободной зоне холста редактора, можно закончить процесс создания sc.g-шины:



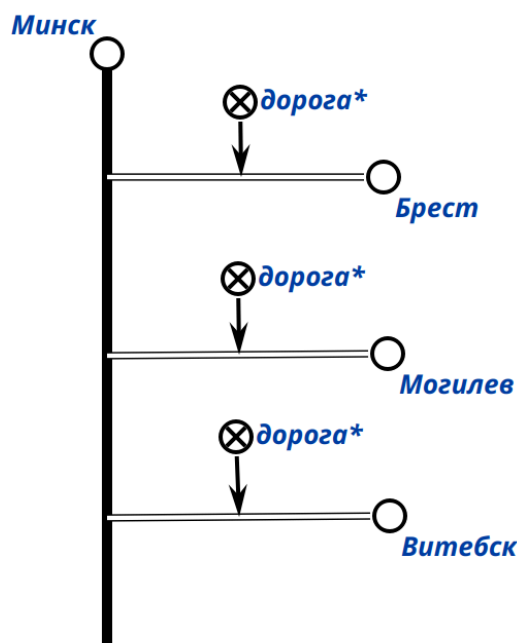
Чтобы изменить размер sc.g-шины необходимо выбрать инструмент выделения на панели инструментов и выделить щелчком левой кнопки компьютерной мыши эту sc.g-шину. В результате на концах sc.g-шины появятся специальные маркеры:



Наведя курсор на маркер свободного конца sc.g-шины, зажав левую кнопку компьютерной мыши и переместив курсор в сторону занятого конца sc.g-шины, можно получить ту же sc.g-шину, но уменьшенного размера:



С sc.g-шиной можно взаимодействовать таким же образом, как и с другими sc.g-элементами:



Стоит отметить, что текущая версия редактора КВЕ не поддерживает создание sc.g-шины для sc.g-коннекторов, а также не поддерживает создания более одной sc.g-шины для одного и того же sc.g-узла.

6.9.2.3.1.4 Создание sc.g-контура

Одним из частных видов формализации фрагментов баз знаний является формализация sc-структур.

По определению, sc-структура является sc-элементом, из которого выходит достаточно большое количество sc-дуг основного вида. Тем самым, процесс формализации и чтения sc-структур на SCg-коде может быть затруднён.

В таких случаях для упрощения формализации и улучшения читаемости sc-структур следует представлять эти sc-структуры на SCg-коде в виде

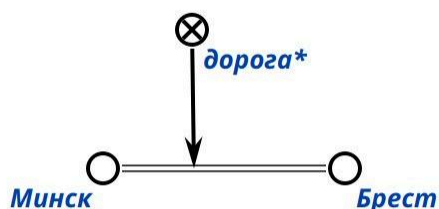
sc.g-контуров — синтаксически выделяемых sc.g-элементов, которые являются графическими обозначениями соответствующих им заданных sc-структур и используются для скрывания sc-дуг основного вида, выходящих из этих sc-структур. Как и sc.g-шина, sc.g-контур является одним из элементов синтаксического сахара, используемого для формализации текста на SCg-коде.

Для создания sc.g-контура необходимо включить режим создания sc.g-контура, то есть выбрать инструмент создания sc.g-контура на панели инструментов. Инструмент создания sc.g-контура показан на рисунке ниже:

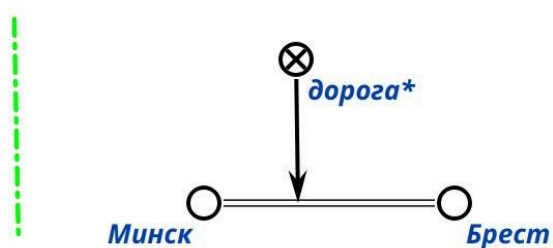


Для активации инструмента создания sc.g-контура достаточно совершить щелчок левой кнопкой компьютерной мыши по изображению этого инструмента или нажать на клавишу с цифрой 4.

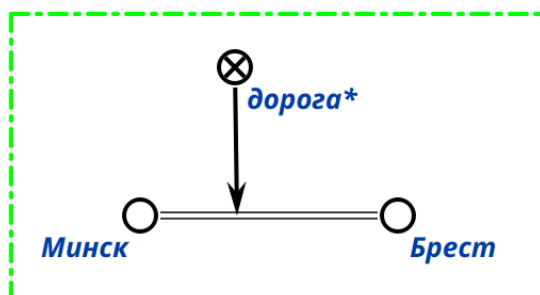
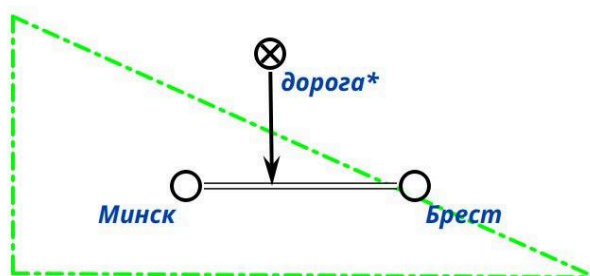
Далее необходимо определить и переместить в одну и ту же свободную зону холста редактора sc.g-элементы, денотаты которых будут принадлежать sc-структуре, обозначаемой создаваемым sc.g-контуром:

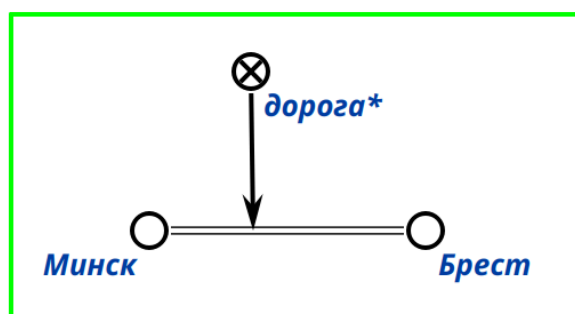


Далее, совершив щелчок левой кнопкой компьютерной мыши по свободной зоне холста редактора можно зафиксировать начальную точку sc.g-контура:

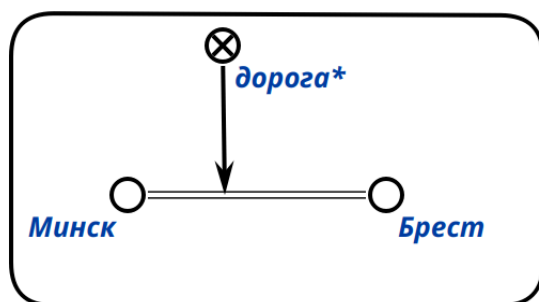


Повторив данную процедуру несколько раз, тем самым нарисовав, например, прямоугольник, вернувшись к начальной точке sc.g-контура и совершив щелчок левой кнопкой компьютерной мыши по ней, можно получить sc.g-контур:

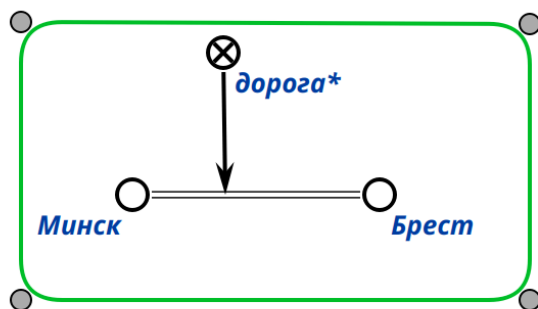


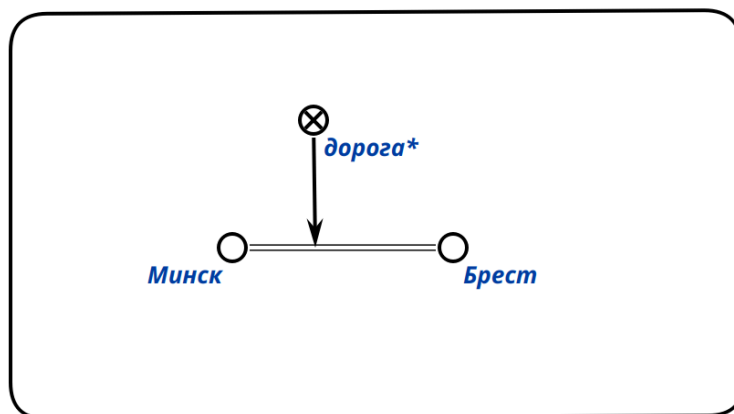


Чтобы выровнять sc.g-контур, можно использовать клавишу Shift. Стоит отметить, что sc.g-контур считается созданным, если он был замкнут, то есть последняя созданная точка sc.g-контура совпадает с начальной точкой этого sc.g-контура. Это подтверждается тем, что линия sc.g-контура в результате становится сплошной.

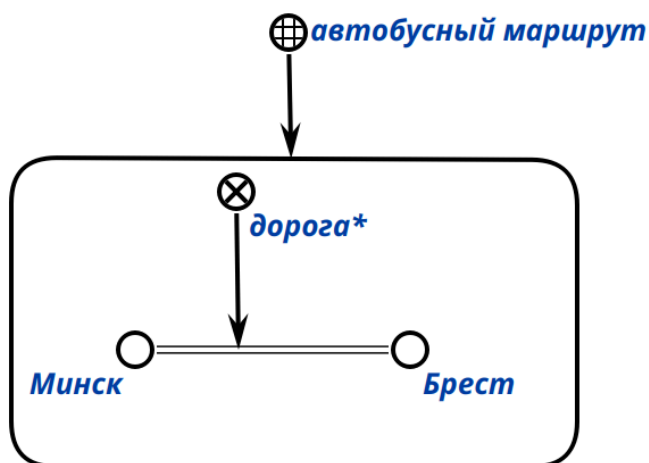


Чтобы изменить форму sc.g-контура необходимо выделить его при помощи инструмента выделения sc.g-элемента. В результате появятся специальные маркеры, перемещение по холсту редактора которых позволяет изменить форму и размер заданного sc.g-контура.





С sc.g-контуром можно взаимодействовать таким же образом, как и с другими sc.g-элементами:



6.9.2.3.1.5 Выравнивание sc.g-конструкций

С помощью редактора КВЕ можно выравнивать sc.g-конструкции по горизонтали и вертикали. Чтобы выровнять sc.g-конструкцию необходимо включить режим выравнивания sc.g-конструкции, то есть выбрать инструмент выравнивания sc.g-конструкции на панели инструментов. Инструмент выравнивания sc.g-конструкции показан на рисунке ниже:

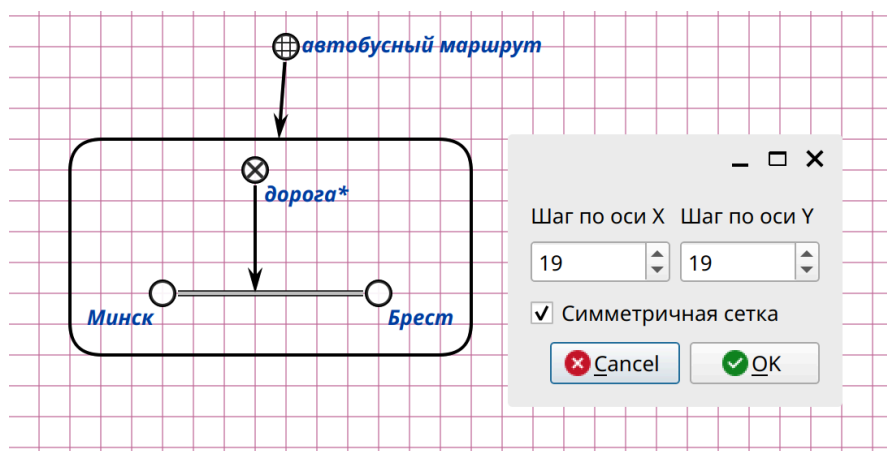


Для активации инструмента выравнивания sc.g-конструкции достаточно совершить щелчок левой кнопкой компьютерной мыши по изображению этого инструмента.

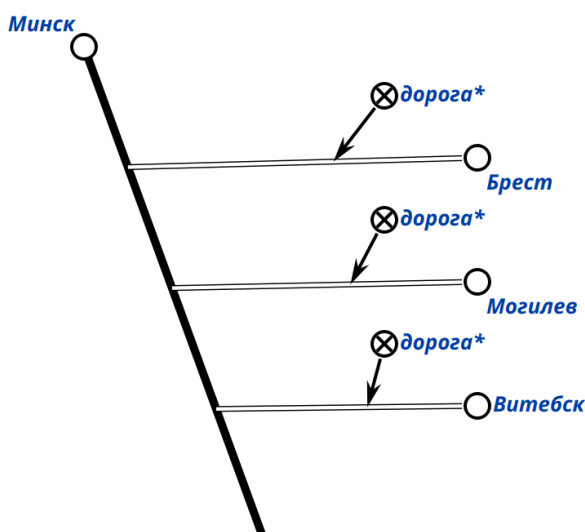
Инструмент выравнивания sc.g-конструкции объединяет несколько инструментов выравнивания sc.g-конструкции. Данные инструменты показаны на рисунке ниже:

	Выравнивание по сетке	5
	Выравнивание связки с sc.g-шиной	6
	Вертикальное выравнивание	7
	Горизонтальное выравнивание	8

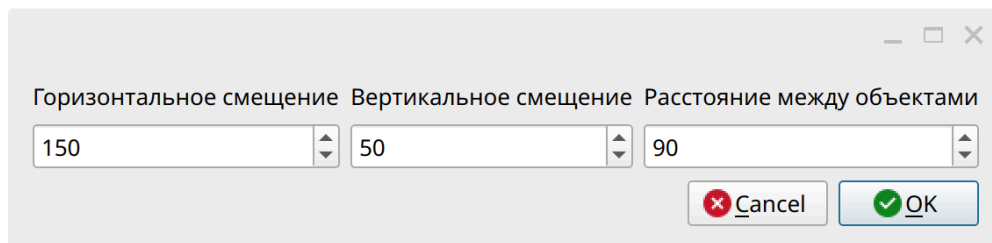
Для использования инструмента выравнивания sc.g-конструкции по сетке необходимо нажать на клавишу с цифрой 5.



Для использования инструмента выравнивания sc.g-конструкции с sc.g-шиной необходимо нажать на клавишу с цифрой 6.



После нажатия на инструмент выравнивания sc.g-конструкции с sc.g-шиной появится диалоговое окно, в котором можно указать необходимые параметры выравнивания, например, расстояния между связками:

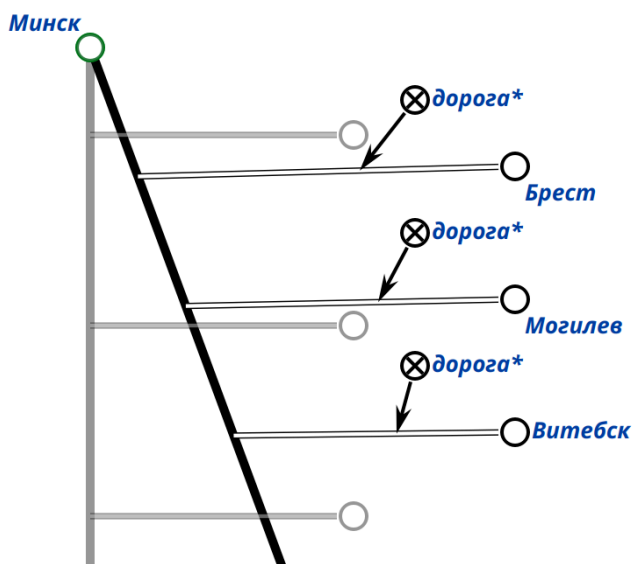


Горизонтальное смещение Вертикальное смещение Расстояние между объектами

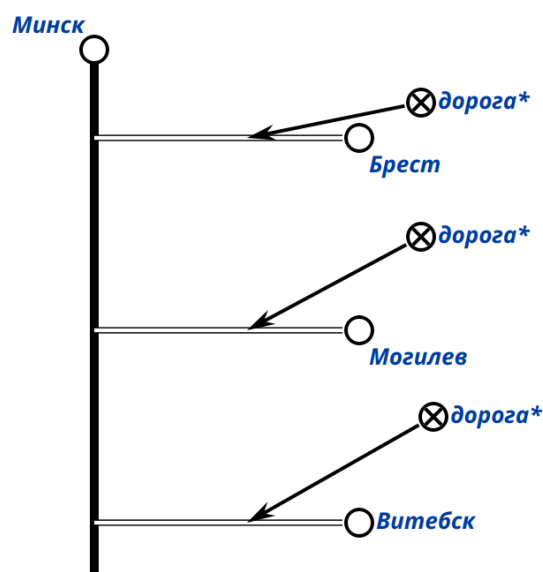
150	50	90
-----	----	----

Cancel OK

На рисунке ниже показана sc.g-конструкция до выравнивания:

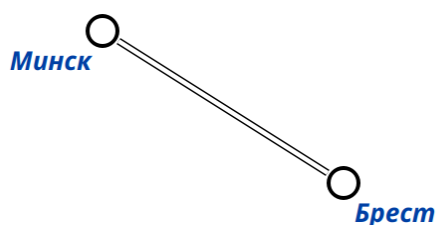


На рисунке ниже показана та же sc.g-конструкция после выравнивания:

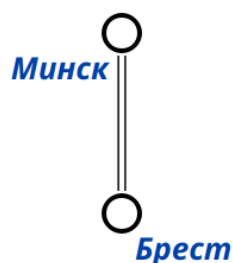


Для использования инструмента выравнивания sc.g-конструкции по вертикали необходимо нажать на клавишу с цифрой 7.

На рисунке ниже показана sc.g-конструкция до выравнивания по вертикали:

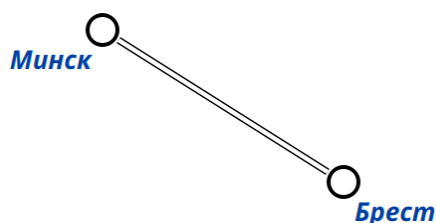


На рисунке ниже показана sc.g-конструкция после выравнивания по вертикали:



Для использования инструмента выравнивания sc.g-конструкции по горизонтали необходимо нажать на клавишу с цифрой 8.

На рисунке ниже показана sc.g-конструкция до выравнивания по горизонтали:



На рисунке ниже показана sc.g-конструкция после выравнивания по горизонтали:



6.9.2.3.1.6 Выделение множества sc.g-элементов

С помощью редактора КВЕ можно выделять несколько sc.g-элементов сразу. Чтобы выделить несколько sc.g-элементов необходимо включить режим выделения множества sc.g-элементов, то есть выбрать инструмент выделения множества sc.g-элементов на панели инструментов. Инструмент выделения множества sc.g-элементов показан на рисунке ниже:

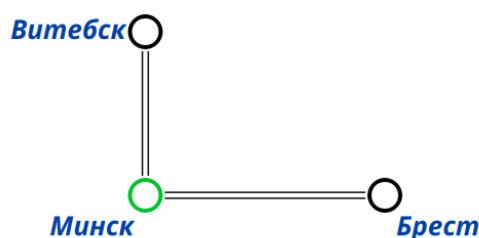


Для активации инструмента выделения множества sc.g-элементов достаточно совершить щелчок левой кнопкой компьютерной мыши по изображению этого инструмента.

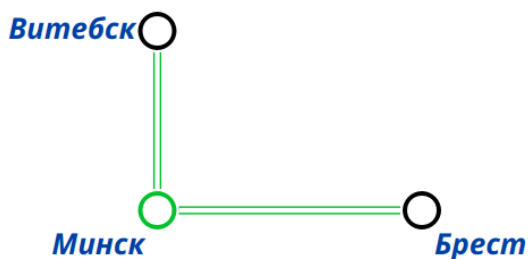
Инструмент выравнивания выделения множества sc.g-элементов объединяет несколько инструментов выделения множества sc.g-элементов. Данные инструменты показаны на рисунке ниже:

-  Выделить входящие/выходящие sc.g-пары (дуги)
-  Выделить подграф

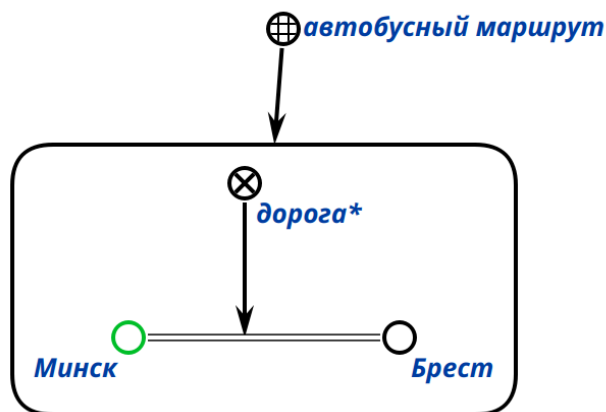
С помощью инструмента выделения входящих/выходящих sc.g-дуг можно выделить все инцидентные sc.g-коннекторы для заданного выделенного sc.g-элемента. Для этого необходимо выделить заданный sc.g-элемент и активировать инструмент выделения входящих/выходящих sc.g-дуг:



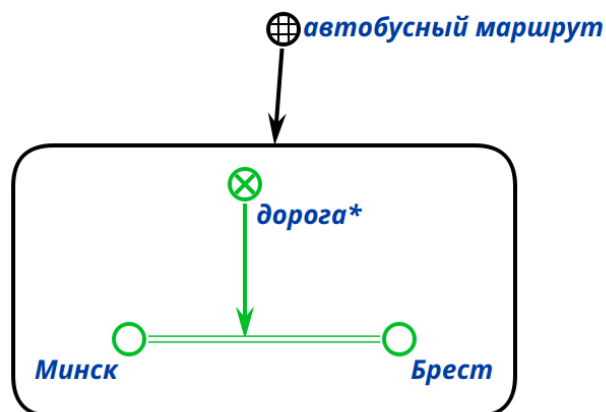
Результат использования данного инструмента показан на рисунке ниже:



С помощью инструмента выделения подграфа можно выделить компоненту связности sc.g-графа, представленного на холсте редактора, элементом которой является заданный выделенный sc.g-элемент. Для этого необходимо выделить заданный sc.g-элемент и активировать инструмент выделения подграфа:



Результат использования данного инструмента показан на рисунке ниже:



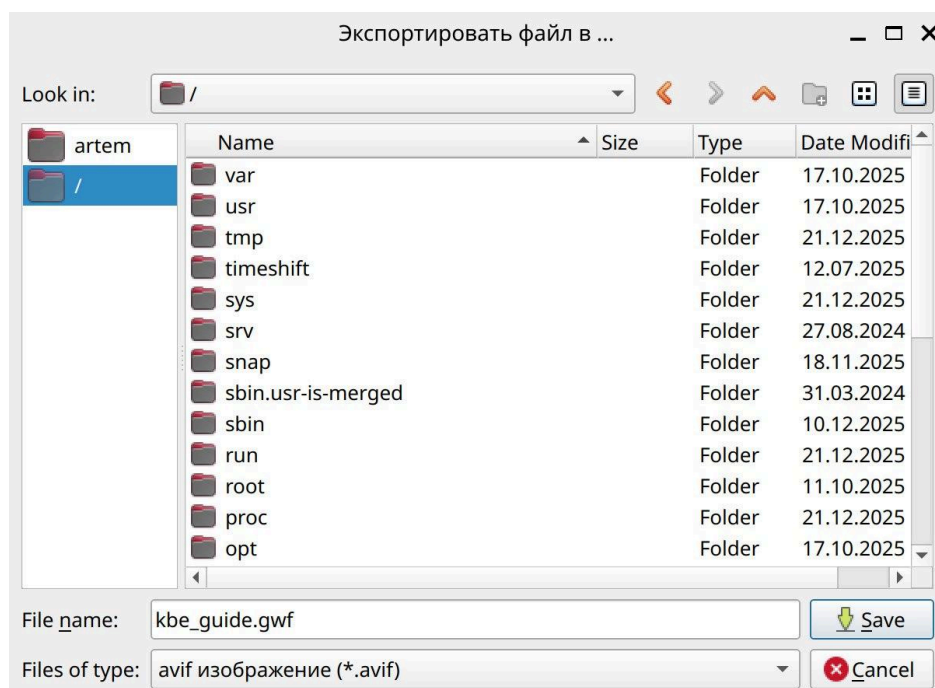
6.9.2.3.1.7 Экспорт изображения sc.g-графа

Редактор КВЕ предоставляет возможность экспортировать изображение sc.g-графа, представленного на холсте. Чтобы сделать экспорт изображения sc.g-графа необходимо включить режим экспорта изображения sc.g-графа, то есть выбрать инструмент экспорта изображения sc.g-графа на панели инструментов. Инструмент экспорта изображения sc.g-графа показан на рисунке ниже:



Для активации инструмента экспорта изображения sc.g-графа достаточно совершить щелчок левой кнопкой компьютерной мыши по изображению этого инструмента.

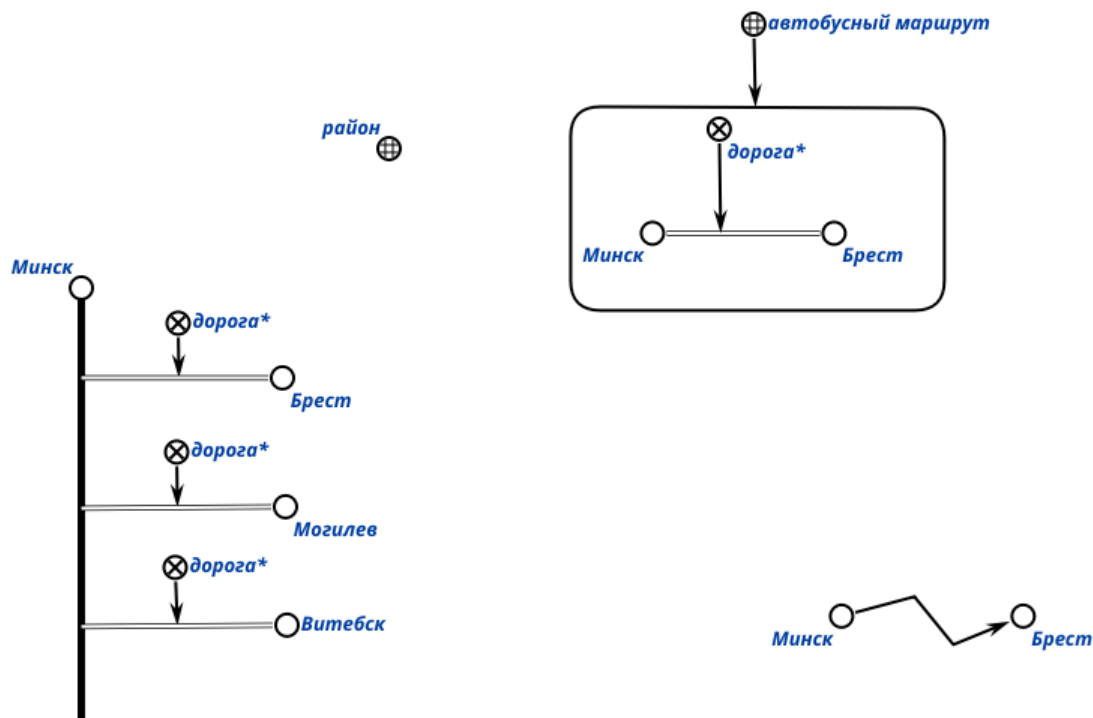
После активации инструмента экспорта изображения sc.g-графа появится диалоговое окно, в котором следует указать путь к файлу и формат сохраняемого изображения.



На рисунке ниже показан фрагмент возможных форматов изображений для экспорта.

avif изображение (*.avif)
bmp изображение (*.bmp)
bw изображение (*.bw)
cur изображение (*.cur)
eps изображение (*.eps)
epsf изображение (*.epsf)
epsi изображение (*.epsi)
heic изображение (*.heic)
heif изображение (*.heif)
icns изображение (*.icns)
ico изображение (*.ico)
jpeg изображение (*.jpeg)
jpg изображение (*.jpg)
jxl изображение (*.jxl)
pbm изображение (*.pbm)
psx изображение (*.psx)
pgm изображение (*.pgm)
pic изображение (*.pic)
png изображение (*.png)
ppm изображение (*.ppm)
qoi изображение (*.qoi)
rgb изображение (*.rgb)
rgba изображение (*.rgba)
sgi изображение (*.sgi)

На рисунке ниже показан пример изображения, которое может получено в результате использования данного инструмента.



6.9.2.3.1.8 Регулирование масштаба холста

Редактор КВЕ предоставляет возможность регулирования масштаба холста. Для этого можно использовать инструменты увеличения и уменьшения или комбинаций клавиш “Shift” и “+” и/или “Shift” и “-” соответственно.



6.9.2.4 Пример формализации фрагментов базы знаний в примере ostis-системы

В данном случае необходимо дополнить директорию knowledge-base/, то есть добавить исходники базы знаний ostis-системы. Структура директорий и исходников с формализацией фрагментов *Предметной области университетов* может выглядеть следующим образом:

```
knowledge-base/
├─ examples/
│   └─ subject_domain_of_universities
│       └─ concepts
│           └─ classes
```

```

|   |   |   |   └─ concept_group.scs
|   |   |   |   └─ concept_student.scs
|   |   |   |   └─ concept_university.scs
|   |   |   └─ relations
|   |   |   └─ nrel_fio.scs
|   |   |   └─ nrel_average_mark.scs
|   |   |   └─ rrel_group.scs
|   |   |   └─ rrel_student.scs
|   |   └─ entities
|   |   └─ bsuir.scs
|   |   └─ bsuir_students.scs
|   |   └─ templates
|   |   └─ template_for_searching_bsuir_students.scs

```

Содержимое всех исходных файлов фрагментов заданной предметной области представлено в виде таблицы ниже:

<pre>// concept_group.scs concept_group <- sc_node_class; => nrel_main_idtf: [группа студентов] (* <- lang_ru;; *); <= nrel_inclusion: concept_set;;</pre>	<pre>// concept_student.scs concept_student <- sc_node_class; => nrel_main_idtf: [студент] (* <- lang_ru;; *); <= nrel_inclusion: concept_human;;</pre>	<pre>// concept_university.scs concept_university <- sc_node_class; => nrel_main_idtf: [университет] (* <- lang_ru;; *);;</pre>
<pre>// nrel_fio.scs nrel_fio <- sc_node_non_role_relation; => nrel_main_idtf:</pre>	<pre>// nrel_average_mark.scs nrel_average_mark <- sc_node_non_role_relation; => nrel_main_idtf:</pre>	<pre>// rrel_group.scs rrel_group <- sc_node_role_relation; => nrel_main_idtf:</pre>

<pre> [ФИО*] (*) <- lang_ru;; *); => nrel_first_domain: concept_human; => nrel_second_domain: lang_ru;; </pre>	<pre> [средняя оценка*] (*) <- lang_ru;; *); => nrel_first_domain: concept_student; => nrel_second_domain: concept_number;; </pre>	<pre> [группа'] (*) <- lang_ru;; *); => nrel_first_domain: concept_university; => nrel_second_domain: concept_group;; </pre>
<pre> // rrel_student.scs rrel_student <- sc_node_role_relation; => nrel_main_idtf: [студент'] (*) <- lang_ru;; *); => nrel_first_domain: concept_group; => nrel_second_domain: concept_student;; </pre>	<pre> // bsuir.scs BSUIR <- concept_university; => nrel_main_idtf: [ВГУИР] (*) <- lang_ru;; *); -> rrel_group: group1; group2;; group1 => nrel_main_idtf: [Группа 1] (*) <- lang_ru;; *); <- concept_group; -> rrel_student: Petrov; Ivanov;; group2 </pre>	<pre> // bsuir_students.scs Petrov <- concept_student; => nrel_fio: nrel_main_idtf: [Петров П.П.] (*) <- lang_ru;; *); => nrel_average_mark: [9.2];; Ivanov <- concept_student; => nrel_fio: nrel_main_idtf: [Иванов И.И.] (*) <- lang_ru;; *); => nrel_average_mark: [5.1];; Olegov <- concept_student; => nrel_fio: nrel_main_idtf: </pre>

	<pre> => nrel_main_idtf: [Группа 2] (* <- lang_ru;; *); <- concept_group; -> rrel_student: Olegov; Sidorov;; </pre>	<pre> [Олегов О.О.] (* <- lang_ru;; *); => nrel_average_mark: [8.1];; Sidorov <- concept_student; => nrel_fio: nrel_main_idtf: [Сидоров С.С.] (* <- lang_ru;; *); => nrel_average_mark: [7.1];; </pre>
--	--	--

Для заданных фрагментов предметной области можно формализовать на SCs-коде следующий sc-шаблон с фиксированной стратегией поиска:

```
// template_for_searching_bsuir_students.scs
```

```

@search_students_template = {

  rrel_init_template: {

    rrel_template: [*

      @input_param1 = (.._university <-_ concept_university);;

      rrel_group _-> (.._university _-> .._group);;

      @output_param1 = (.._group <-_ concept_group);;

    *];

    rrel_template_input_params: {

      @input_param1

    };

    rrel_template_output_params: {

```

```

        @output_param1

    }

} (* <- nrel_search_set_template;; *);

rrel_next_template: {

    rrel_init_template: {

        rrel_template: [*

            @input_param1 = (.._group <-_ concept_group);;

            rrel_student _-> (.._group _-> .._student);;

            @output_param1 = (.._student <-_ concept_student);;

        *];

        rrel_template_input_params: {

            @input_param1

        };

        rrel_template_output_params: {

            @output_param1

        }

    }

} (* <- nrel_search_set_template;; *);

rrel_next_template: {

    rrel_init_template: {

        rrel_template: [*

            @input_param1 = (.._student <-_ concept_student);;

            nrel_fio _-> (.._student _=> .._fio);;

            @output_param1 = (.._fio <-_ lang_ru);;

        *];

        rrel_template_input_params: {

            @input_param1

        };

    }

}

```

```

        nrel_template_output_params: {

            @output_param1

        }

    } (* <- nrel_search_template;; *)

    } (* <- nrel_fixed_search_strategy_template;; *)

} (* <- nrel_fixed_search_strategy_template;; *)

};;

@search_students_template

<- nrel_fixed_search_strategy_template;

=> nrel_main_idtf:

    [Шаблон поиска студентов университета]

    (*

        <- lang_ru;;

    *);;

```

Представленный на SCs-коде sc-шаблон с фиксированной стратегией поиска полностью соответствует указанному ранее sc-шаблону с фиксированной стратегией поиска на SCg-коде. Для формализации sc-шаблонов также рекомендуется использовать SCs-код.

Данные исходные файлы находятся в указанных директориях в проекте примера ostis-системы. Поэтому после установки примера ostis-системы они будут автоматически доступны для редактирования и дополнения.

6.9.2.5 Принципы тестирования базы знаний ostis-системы

Процесс тестирования формализованных фрагментов предметной области направлен на проверку соответствия их синтаксиса и семантики правилам формализации фрагментов базы знаний. При этом тестирование следует рассматривать как обязательный этап цикла разработки базы знаний, обеспечивающий согласованность новых компонентов с уже существующими элементами.

6.9.2.5.1 Сборка и пересборка базы знаний ostis-системы

Для проверки синтаксической корректности формализованного фрагмента базы знаний требуется выполнить процесс *сборки всей базы знаний* с этим новым фрагментом.

сборка (пересборка) базы знаний ostis-системы — это трансляция множества заданных исходных файлов базы знаний во множество бинарных файлов базы знаний, которые представляют собой форму кодирования внутреннего представления информации в базе знаний ostis-системы и служат основой для ее эффективного хранения, доступа и последующей обработки в этой ostis-системе.

6.9.2.5.1.1 Понятие бинарного файла базы знаний

бинарный файл базы знаний — это бинарный файл, кодирующий информацию на SC-коде.

6.9.2.5.1.2 Понятие sc-сборщика базы знаний ostis-системы

Для *сборки (пересборки) базы знаний ostis-системы* следует использовать *sc-сборщик базы знаний ostis-системы*.

sc-сборщик базы знаний ostis-системы — это программное средство для сборки базы знаний ostis-системы, являющееся компонентом *Программной платформы ostis-систем*.

6.9.2.5.1.3 Понятие Программной платформы ostis-систем

Программная платформа ostis-систем — это программная платформа, реализующая интерпретацию информации в ostis-системах, обеспечивающая их компонентное проектирование, платформенную независимость, а также семантическую совместимость в рамках Экосистемы OSTIS.

Программная платформа ostis-систем состоит из:

- *программной реализации sc-памяти* — общей семантической памяти для хранения и обработки текстов на SC-коде;
- *программного интерфейса, обеспечивающего доступ к sc-памяти* — набора методов для создания, изменения, удаления и поиска sc-конструкций в sc-памяти;

- интерпретаторов различных подязыков SC-кода (*Языка SCP, Языка SCL* и других), обеспечивающих универсальность и платформенную независимость ostis-систем;
- а также средств разработки ostis-систем (*sc-сборщика базы знаний ostis-систем* и других).

6.9.2.5.1.4 Принципы использования sc-сборщика базы знаний ostis-системы

sc-сборщик базы знаний ostis-системы компилируется отдельным независимым исполняемым бинарным файлом. В проекте примера ostis-системы его можно использовать следующим образом:

```
cd ostis-example-app/

./install/sc-machine/bin/sc-builder -i repo.path -o kb.bin --clear
```

или

```
./scripts/start.sh build_kb
```

Опция `-i` (`--input`) позволяет указать либо путь к папке с исходными файлами базы знаний, либо путь к файлу конфигурации базы знаний, задающему пути тех исходных файлов базы знаний, которые следует собирать, и пути тех исходных файлов базы знаний, которые не следует собирать.

Файл конфигурации базы знаний в проекте примера ostis-системы — `repo.path` — содержит следующую информацию:

```
# components

knowledge-base/

!knowledge-base/ims.ostis.kb/ui/menu/ui_main_menu.scs

# specifications

problem-solver/cxx/example-module/specifications/agent_of_isomorphic_search

problem-solver/cxx/example-module/specifications/agent_of_subdividing_search
```

Данную информацию можно проинтерпретировать как “собрать все исходные файлы из директории `knowledge-base/`, за исключением исходного файла `knowledge-base/ims.ostis.kb/ui/menu/ui_main_menu.scs` и собрать исходные файлы `problem-solver/cxx/example-module/specifications/agent_of_isomorphic_search` и `problem-solver/cxx/example-module/specifications/agent_of_subdividing_search`”.

В файле конфигурации базы знаний можно указывать как относительные пути, так и абсолютные пути к файлам.

Опция `-o (--output)` позволяет указать путь к папке, в которую будут компилироваться бинарные файлы базы знаний, то есть в которой в результате сборки исходных файлов базы знаний появятся бинарные файлы базы знаний.

Флаг `--clear` позволяет запустить `sc`-сборщик базы знаний `ostis`-системы в режиме полной переинициализации, при котором все ранее существующие бинарные файлы в директории `kb.bin` удаляются и заново формируются на основе заданных исходных файлов базы знаний.

Если выполнить сборку базы знаний `ostis`-систем без указания флага `--clear`, тогда существующие бинарные файлы в директории `kb.bin` будут дополнены указанными исходными файлами базы знаний.

После запуска `sc`-сборщика следует дождаться его завершения. Если в процессе сборки исходных файлов базы знаний появится ошибка с текстом красного цвета, то это значит, что какой-то исходный файл базы знаний составлен некорректно и имеет грамматические и синтаксические ошибки. Пример ошибки показан ниже:

```
[1372/1397]: knowledge-base/ims.ostis.kb/ui/ui_nrel_user_used_language.scs - ok
[1373/1397]: knowledge-base/ims.ostis.kb/to_check/nrel_email.scs - ok
[1374/1397]: knowledge-base/ims.ostis.kb/ims/ostis_tech/unificated_models/section_unificated_models_kb/section_str
structure_of_knowledge_base_model/section_structure_of_knowledge_base_model.scs - ok
[1375/1397]: knowledge-base/ims.ostis.kb/ims/ostis_tech/lib_ostis/section_library_of_typical_subsystems_of_compute
r_systems/section_library_advisory_educational_subsystems.scs - ok
[1376/1397]: knowledge-base/ims.ostis.kb/ims/ostis_tech/unificated_models/section_unificated_models_interface/mode
l_of_user_interface/demonstrational_dialog.scs - ok
[1377/1397]: knowledge-base/ims.ostis.kb/ims/ostis_tech/lib_ostis/sectn_lib_reusable_comp_kpm/reusable_sc_agents/l
ib_scp_agents/search/find_key_concepts/sc_agent_of_finding_key_concepts.scs - ok
[1378/1397]: knowledge-base/ims.ostis.kb/ims/ostis_tech/lib_ostis/sectn_lib_reusable_comp_kpm/programs_for_sc_text
_processing/scp_program/find_arcs_between_concepts/find_arcs_between_concepts.scs - ok
[1379/1397]: knowledge-base/ims.ostis.kb/ims/ostis_tech/lib_ostis/sectn_lib_reusable_comp_kpm/reusable_sc_agents/l
ib_scp_agents/verification/agent_of_checking_domains_of_relation/lib_component_agent_of_checking_domains_of_relati
on.scs - ok
[1380/1397]: knowledge-base/examples/subject_domain_of_universities/templates/template_for_searching_bsuir_student
s.scs - line 54:0 mismatched input '<EOF>' expecting '{17<<<'}'
failed
Parse error at line 54,0: mismatched input '<EOF>' expecting '{17<<<'}'
Clean state...
[17:20:02][Info]: Shutdown ScKeynodes
[17:20:02][Info]: [sc-memory] Shutdown
[17:20:02][Info]: [sc-memory] Shutdown extensions
[17:20:02][Info]: [sc-memory] Extensions shutdown
[17:20:02][Info]: [sc-memory] Shutdown sc-helper
[17:20:02][Info]: [sc-fs-memory] Save sc-memory segments
[17:20:02][Info]: Loaded segments count: 6
[17:20:02][Info]: Sc-segments size: 25165872
[17:20:02][Info]: Last not engaged segment num: 0
[17:20:02][Info]: Last released segment num: 6
[17:20:02][Info]: [sc-fs-memory] Sc-memory segments saved
[17:20:02][Info]: [sc-fs-memory] Save sc-fs-memory dictionaries
[17:20:02][Info]: [sc-fs-memory] Dictionary `term - offsets` written
[17:20:02][Info]: [sc-fs-memory] Dictionary `string offsets - link hashes` written
[17:20:02][Info]: Last string offset: 1132823
[17:20:02][Info]: [sc-fs-memory] All sc-fs-memory dictionaries saved
[17:20:02][Info]: [sc-fs-memory] Shutdown
[17:20:02][Info]: [sc-fs-memory] Successfully shutdown
[17:20:02][Info]: [sc-memory] Shutdown context manager
[17:20:02][Info]: [sc-memory] Shutdown
```

Сборка базы знаний прервалась на 1380м исходном файле из 1397ми исходных файлов.

Если исходные файлы базы знаний составлены корректно, тогда sc-сборщик базы знаний ostis-системы завершит трансляцию исходных файлов успешно (без ошибок), а это значит, что полученные бинарные файлы базы знаний можно использовать для запуска ostis-системы.

Пример успешно собранной базы знаний демонстрируется на картинке ниже:

```
concept_of_reusable_component_interface.scs - ok
[1387/1397]: knowledge-base/ims.ostis.kb/ims/ostis_tech/lib_ostis/sectn_lib_of_reusable_comp_ui/ui_menu/
kb_editing/ui_menu_file_for_changing_main_identifier/lib_component_ui_menu_file_for_changing_main_identi
fier.scs - ok
[1388/1397]: knowledge-base/ims.ostis.kb/ims/ostis_tech/lib_ostis/sectn_lib_of_reusable_comp_ui/ui_menu/
na_activity_automatisation/ontology_forming/ui_menu_na_ontology_forming_content.scs - ok
[1389/1397]: knowledge-base/examples/subject_domain_of_universities/concepts/relations/rrel_fio.scs - ok
[1390/1397]: knowledge-base/examples/subject_domain_of_universities/concepts/relations/rrel_student.scs
- ok
[1391/1397]: knowledge-base/ims.ostis.kb/ui/model/ui_descr_oper_semantic_control.scs - ok
[1392/1397]: knowledge-base/ims.ostis.kb/ims/ostis_tech/lib_ostis/sectn_lib_of_reusable_comp_ui/ui_menu/
search/authorized_users/without_context/find_key_concepts/ui_menu_file_for_finding_key_concepts_content.
scs - ok
[1393/1397]: knowledge-base/ims.ostis.kb/ims/ostis_tech/lib_ostis/sectn_lib_reusable_comp_kpm/reusable_s
c_agents/lib_scp_agents/activity_automatisation/expert/assign_form_verif_prod/agent_assign_form_verif_pr
od_content.scs - ok
[1394/1397]: knowledge-base/ims.ostis.kb/ims/ostis_tech/lib_ostis/sectn_lib_of_reusable_comp_ui/ui_menu/
search/unauthorized_users/with_context/find_all_output_const_pos_arc_with_rel_in_the_agreed_part_of_kb/l
ib_component_ui_menu_view_all_output_const_pos_arc_with_rel_in_the_agreed_part_of_kb.scs - ok
[1395/1397]: knowledge-base/ims.ostis.kb/ims/ostis_tech/lib_ostis/sectn_lib_reusable_comp_kpm/reusable_s
c_agents/lib_scp_agents/Methods_evaluation/Kb_volume_metrics/agent_of_calculation_number_of_concepts_and
_relations/agent_of_calculation_number_of_concepts_and_relations_content.scs - ok
[1396/1397]: knowledge-base/examples/subject_domain_of_universities/concepts/relations/rrel_group.scs -
ok
[1397/1397]: knowledge-base/ims.ostis.kb/ims/ostis_tech/lib_ostis/sectn_lib_reusable_comp_kpm/reusable_s
c_agents/lib_scp_agents/verification/scp_program_component/section_library_of_reusable_scp_programs_for_
verification_agents.scs - ok
Clean state...
Statistics
Nodes: 25781(6.621%)
Connectors: 351351(90.233%)
Links: 12250(3.14601%)
Total: 389382
[17:24:56][Info]: Shutdown ScKeynodes
[17:24:56][Info]: [sc-memory] Shutdown
[17:24:56][Info]: [sc-memory] Shutdown extensions
[17:24:56][Info]: [sc-memory] Extensions shutdown
[17:24:56][Info]: [sc-memory] Shutdown sc-helper
[17:24:56][Info]: [sc-fs-memory] Save sc-memory segments
[17:24:56][Info]: Loaded segments count: 6
[17:24:56][Info]: Sc-segments size: 25165872
[17:24:56][Info]: Last not engaged segment num: 0
[17:24:56][Info]: Last released segment num: 6
[17:24:56][Info]: [sc-fs-memory] Sc-memory segments saved
[17:24:56][Info]: [sc-fs-memory] Save sc-fs-memory dictionaries
[17:24:56][Info]: [sc-fs-memory] Dictionary `term - offsets` written
[17:24:56][Info]: [sc-fs-memory] Dictionary `string offsets - link hashes` written
[17:24:56][Info]: Last string offset: 1136938
[17:24:56][Info]: [sc-fs-memory] All sc-fs-memory dictionaries saved
[17:24:56][Info]: [sc-fs-memory] Shutdown
[17:24:56][Info]: [sc-fs-memory] Successfully shutdown
[17:24:56][Info]: [sc-memory] Shutdown context manager
[17:24:56][Info]: [sc-memory] Shutdown
```

6.9.2.5.2 Запуск ostis-системы

После успешной сборки базы знаний ostis-системы можно запустить ostis-систему.

запуск ostis-системы — это запуск Программной платформы ostis-систем вместе с компонентами заданной ostis-системы: базой знаний, решателем задач и пользовательским интерфейсом этой ostis-системы.

Для запуска примера *ostis*-системы необходимо запустить в разных терминалах:

- *sc-машину*:

```
cd ostis-example-app  
./scripts/start.sh machine
```

- *sc-веб*:

```
cd ostis-example-app  
./scripts/start.sh web
```

6.9.2.5.2.1 Понятие *sc-машины*

sc-машина — это программное средство, представляющее собой программную реализацию *sc*-памяти и программных интерфейсов для работы с ней.

При запуске *sc-машины* могут появиться ошибки с текстом в жёлтом или красном цветах. Это значит, что скорее всего после такого запуска *sc-машины* *ostis*-система будет работать неправильно. Следовательно, перед началом работы с *ostis*-системой необходимо устранить ошибки, возникающие при запуске *sc-машины*.

6.9.2.5.2.2 Понятие *sc-веба*

sc-веб — это реализация веб-ориентированного интерпретатора пользовательского интерфейса *ostis*-системы, взаимодействующего с *sc-машиной*.

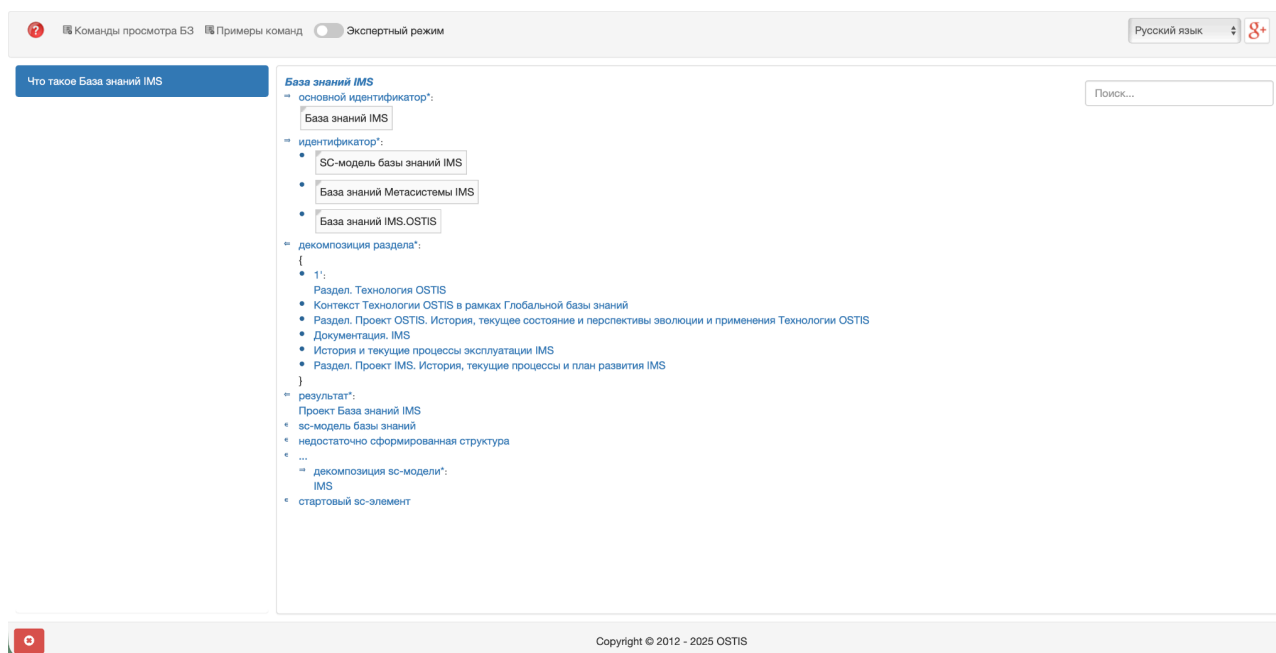
6.9.2.5.2.3 Понятие *localhost:8000*

sc-машина и *sc-веб* являются серверными приложениями. Это значит, что после запуска они должны “висеть” в терминалах, то есть эти терминалы нельзя закрывать в процессе работы с *ostis*-системой.

Чтобы остановить работу приложений в терминалах, необходимо в каждом терминале применить комбинацию клавиш **Ctrl+C** (**Control+C**).

После успешного запуска обоих приложений следует перейти в браузер и указать в адресной строке `localhost:8000`.

После загрузки появится следующая страница:



На главной части данной страницы показана семантическая окрестность сущности *База знаний IMS*, представленная на SCn-коде. С помощью данного интерфейса можно осуществлять навигацию по базе знаний ostis-системы.

6.9.2.5.3 Примеры навигации по формализованным фрагментам базы знаний в примере ostis-системы

Семантическая окрестность сущности *БГУИР* на SCn-коде представлена на рисунке ниже:

БГУИР

⇒ основной идентификатор*:

БГУИР

€ Русский язык

⇒ системный идентификатор*:

BSUIR

⇒ группа':

- Группа 2
- Группа 1

— ⇒ ...

€ университет

€ университет

Семантическая окрестность сущности *Группа 1* на SCn-коде представлена на рисунке ниже:

Группа 1

⇒ основной идентификатор*:

Группа 1

€ Русский язык

⇒ системный идентификатор*:

group1

⇒ студент':

- Ivanov
- Petrov

€ группа':

БГУИР

€ группа студентов

Семантическая окрестность сущности *Группа 2* на SCn-коде представлена на рисунке ниже:

Группа 2

⇒ основной идентификатор*:

Группа 2

€ Русский язык

⇒ системный идентификатор*:

group2

⇒ студент':

- Sidorov
- Olegov

€ группа':

БГУИР

€ группа студентов

Семантические окрестности других сущностей, в том числе понятий, представленных в формализации указанных выше фрагментов базы знаний, могут быть получены соответствующим образом.

6.9.2.5.4 Типичные ошибки, выявляемые в процессе навигации по фрагментам базы знаний ostis-системы

В процессе навигации по фрагментам базы знаний можно находить ошибки в формализации этих фрагментов. Это могут быть как синтаксические ошибки, так и семантические ошибки.

Типичными синтаксическими ошибками, выявляемыми в процессе навигации по фрагментам базы знаний, являются:

- опечатка в sc-идентификаторе сущности;
- некорректно указанное направление sc-коннектора;
- использование одинаковых sc-идентификаторов для разных сущностей;
- и другие.

Типичными семантическими ошибками, выявляемыми в процессе навигации по фрагментам базы знаний, являются:

- некорректно указанный семантический тип sc-элемента;
- неправильно указанные связи между сущностями (понятиями);
- противоречия;
- и другие.

6.9.2.5.5 Принципы тестирования sc-шаблонов с фиксированной стратегией поиска

Для тестирования семантической корректности sc-шаблонов с фиксированной стратегией поиска используется агент интерпретации sc-шаблонов с фиксированной стратегией поиска.

Данный агент осуществляет интерпретацию заданного sc-шаблона с фиксированной стратегией поиска и производит поиск sc-конструкций, изоморфных данному sc-шаблону.

Для упрощения вызова этого агента и получения его результата в примере ostis-системы добавлен компонент пользовательского интерфейса, реализующий вопрос *Какие структуры соответствуют заданному шаблону с фиксированной стратегией поиска?* С помощью данного вопроса можно производить поиск

sc-конструкций, изоморфных заданному sc-шаблону, дополнительно указывая аргументы для этого sc-шаблона.

Чтобы найти sc-конструкции, изоморфные заданному sc-шаблону с фиксированной стратегией поиска, необходимо:

- 1) Найти sc-шаблон с фиксированной стратегией поиска.
- 2) Найти все sc-коннекторы, которые необходимо указать как аргументы для заданного sc-шаблона (аргументы sc-шаблона).
- 3) При помощи панели истории навигации по базе знаний ostis-системы перейти к семантической окрестности ранее найденного sc-шаблона; совершить щелчок правой кнопкой компьютерной мыши по sc-идентификатору этого sc-шаблона и нажать в появившемся контекстном меню на кнопку “закрепить” — в результате sc-идентификатор заданного sc-шаблона будет закреплён на панели снизу.
- 4) Перейти в режим навигации по базе знаний ostis-системы на SCg-коде и при помощи панели инструментов слева создать sc.g-узел, обозначающий sc-множество; провести от него sc.g-дуги основного вида ко всем аргументам sc-шаблона; совершить щелчок правой кнопкой компьютерной мыши по созданному sc.g-узлу и нажать в появившемся контекстном меню на кнопку “закрепить” — в результате sc-адрес созданного sc.g-узла будет закреплён на панели снизу.
- 5) В панели сверху щелчком левой кнопки компьютерной мыши нажать на меню *Примеры команд*; в появившемся списке щелчком левой кнопки компьютерной мыши нажать на элемент, обозначающий вопрос *Какие структуры соответствуют заданному шаблону с фиксированной стратегией поиска?*.

В результате на странице появятся все sc-конструкции, изоморфные заданному sc-шаблону с фиксированной стратегией поиска.

Ниже рассматривается пример поиска по sc-шаблону с фиксированной стратегией поиска.

В качестве примера можно использовать *Шаблон поиска студентов университета*, который был формализован ранее. Данный sc-шаблон можно найти при помощи строки поиска, указав sc-идентификатор этого sc-шаблона. Процесс использования *строки поиска* показан на рисунке ниже:

Шаблон
Шаблон поиска студентов университета

Результатом поиска заданного sc-шаблона будет являться семантическая окрестность единичного радиуса этого sc-шаблона, представленная на SCn-коде:

Шаблон поиска студентов университета

⇒ основной идентификатор*:

Шаблон поиска студентов университета

⇒ следующий sc-шаблон':

...

⇒ начальный sc-шаблон':

...

€ sc-шаблон с фиксированной стратегией поиска*

Для найденного sc-шаблона в качестве аргумента можно указать sc-дугу основного вида между sc-классом *университет* и сущностью *БГУИР*. Чтобы найти эту sc-дугу, необходимо найти либо sc-класс *университет*, либо сущность *БГУИР*. Для этого можно использовать *строку поиска*. Процесс использования *строки поиска* показан на рисунке ниже:

БГУИР
БГУИР Проект развития ostis-системы автоматизации управления кафедрой интеллектуальных информационных технологий БГУИР Проект, направленный на развитие Кафедры интеллектуальных информационных технологий БГУИР , осуществляющей подготовку молодых специалистов по Специальности «искусственный интеллект» Семейство прикладных проектов ostis-систем, не связанных с информационной поддержкой и автоматизацией деятельности

Результатом поиска заданной сущности будет являться семантическая окрестность единичного радиуса этой сущности, представленная на SCn-коде:

БГУИР

⇒ **основной идентификатор***:

БГУИР

€ **Русский язык**

⇒ **системный идентификатор***:

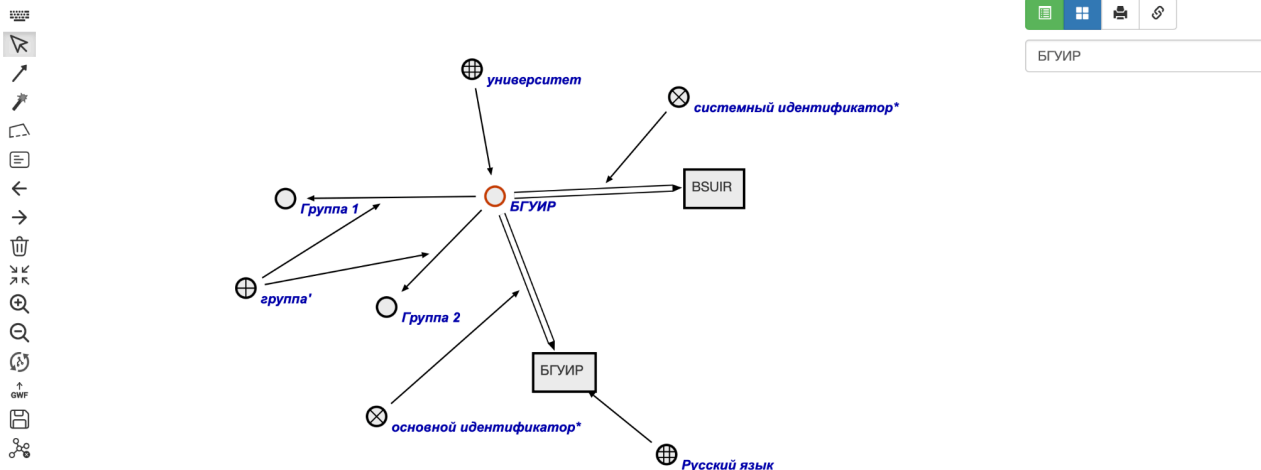
BSUIR

⇒ **группа'**:

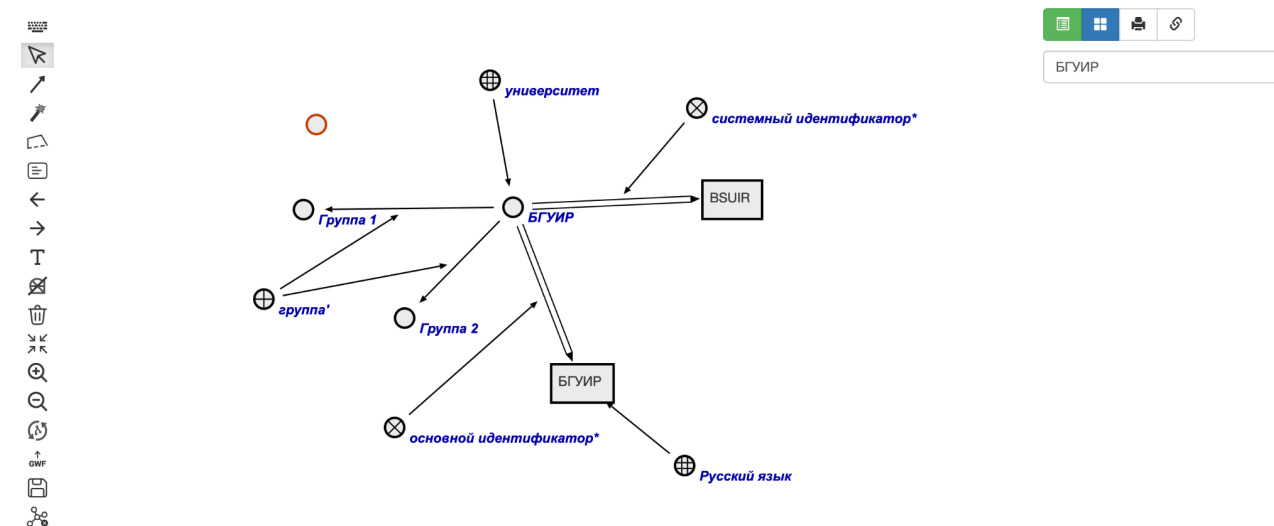
- **Группа 2**
- **Группа 1**

€ **университет**

После перехода в *режим навигации по базе знаний ostis-системы на SCg-коде* данная семантическая окрестность будет выглядеть следующим образом:

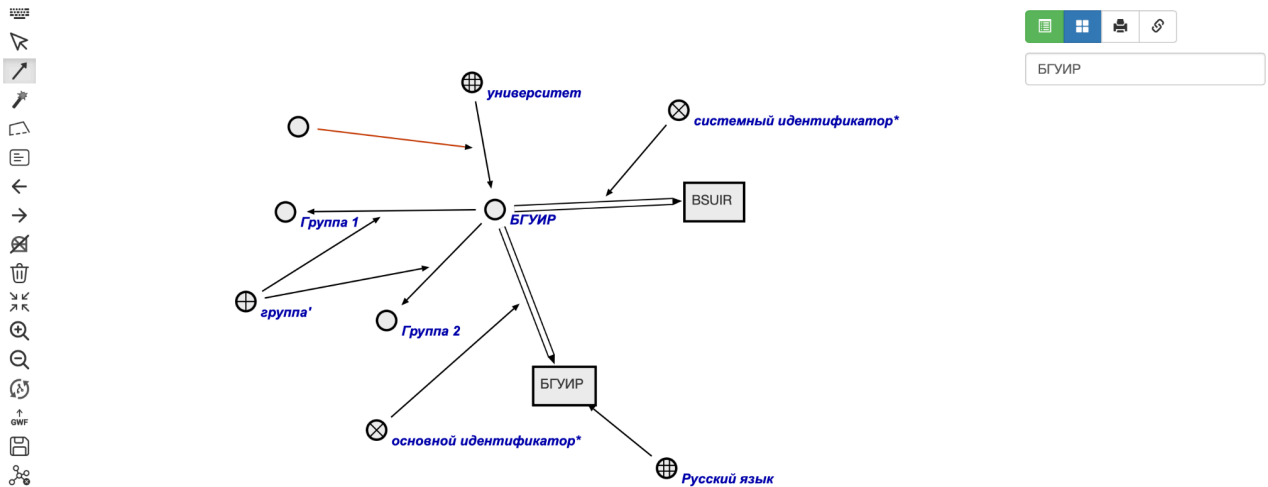
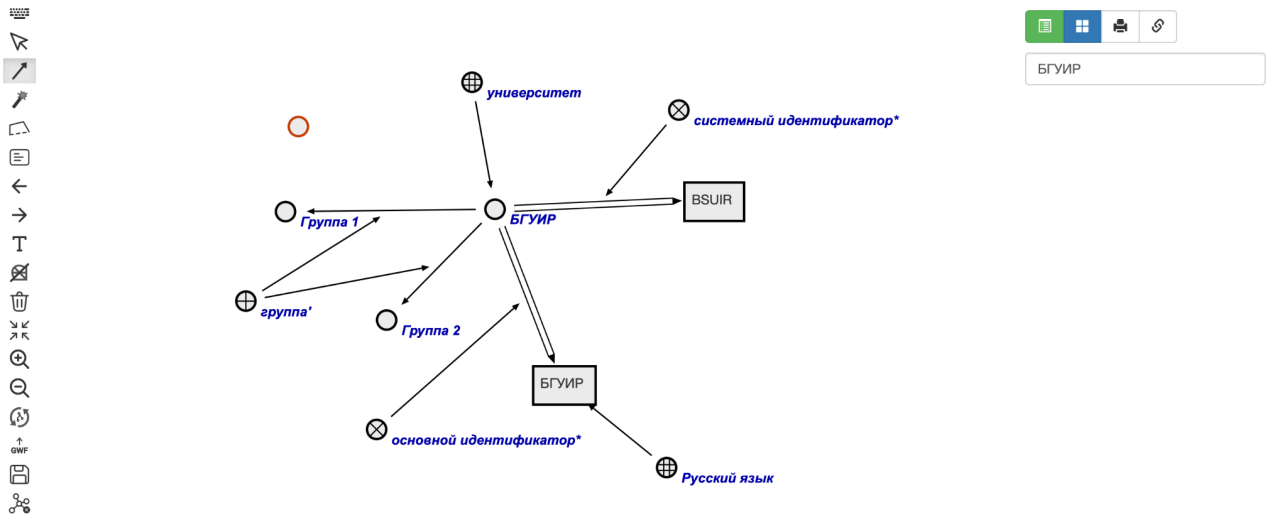


Двойным щелчком левой кнопки компьютерной мыши на данной sc.g-сцене можно создать sc.g-узел, обозначающий новое sc-множество. Результат показан на рисунке ниже:



Выбрав в панели инструментов слева инструмент создания sc.g-коннектора, совершив щелчок левой кнопкой компьютерной мыши по sc.g-узлу, обозначающему новое sc-множество, проведя появившуюся sc.g-дугу основного вида к sc.g-дуге основного вида между sc.g-узлом, обозначающим

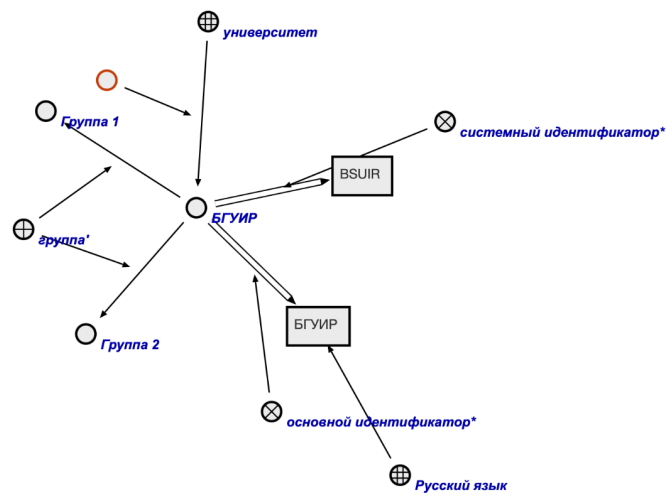
sc-класс *университет*, и sc.g-узлом, обозначающим сущность *БГУИР*, на sc.g-сцене появится новая sc.g-дуга.



Для того, чтобы записать изменения в базу знаний *ostis-системы*, необходимо выбрать в панели инструментов слева инструмент *синхронизации с базой знаний* (4ая кнопка с конца). После нажатия на эту кнопку, граф, представленный на sc.g-сцене, изменит своё местоположение, а новые sc.g-элементы перекрасятся в другой цвет.



БГУИР



Далее при помощи *панели истории навигации по базе знаний ostis-системы* можно осуществить переход к семантической окрестности *Шаблона поиска студентов университета*.

Что такое БГУИР

Что такое Шаблон поиска студентов университета

Что такое База знаний IMS

Что такое БГУИР

Что такое Шаблон поиска студентов университета

Что такое База знаний IMS

Шаблон поиска студентов университета

⇒ **основной идентификатор***:

Шаблон поиска студентов университета

€ **Русский язык**

⇒ **следующий sc-шаблон'**:

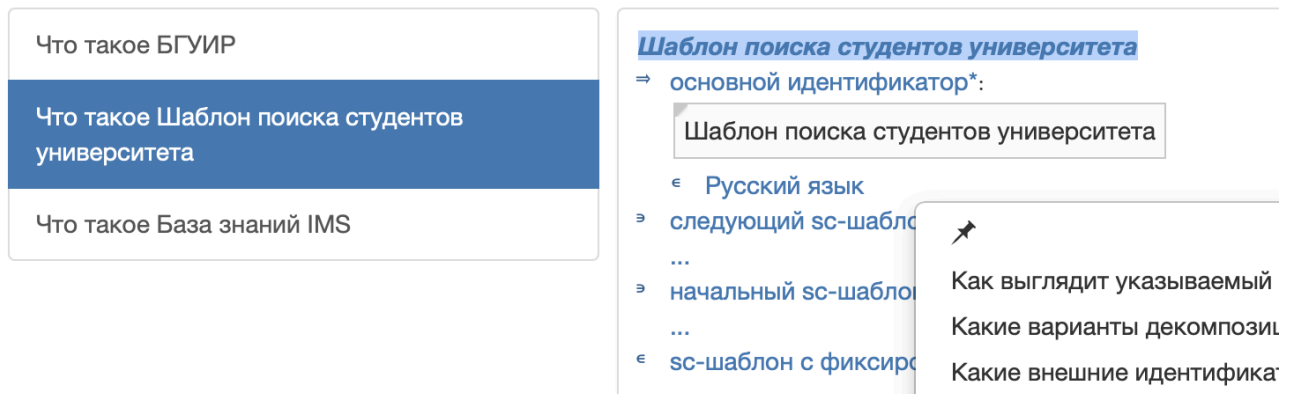
...

⇒ **начальный sc-шаблон'**:

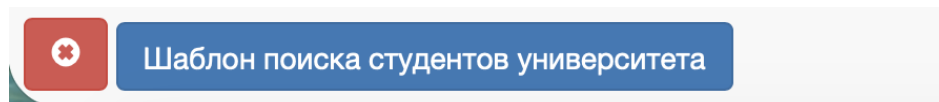
...

€ **sc-шаблон с фиксированной стратегией поиска***

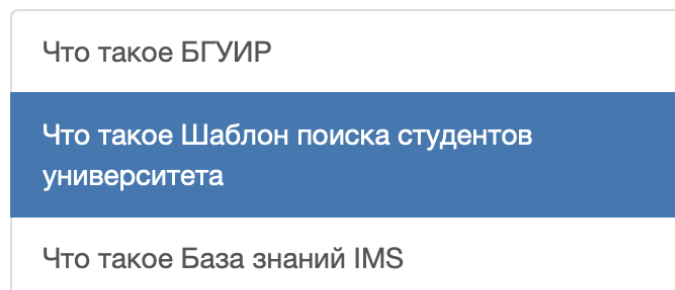
Далее для данной сущности необходимо вызвать *контекстное меню* и нажать на *кнопку закрепления sc-элемента на панели снизу*:



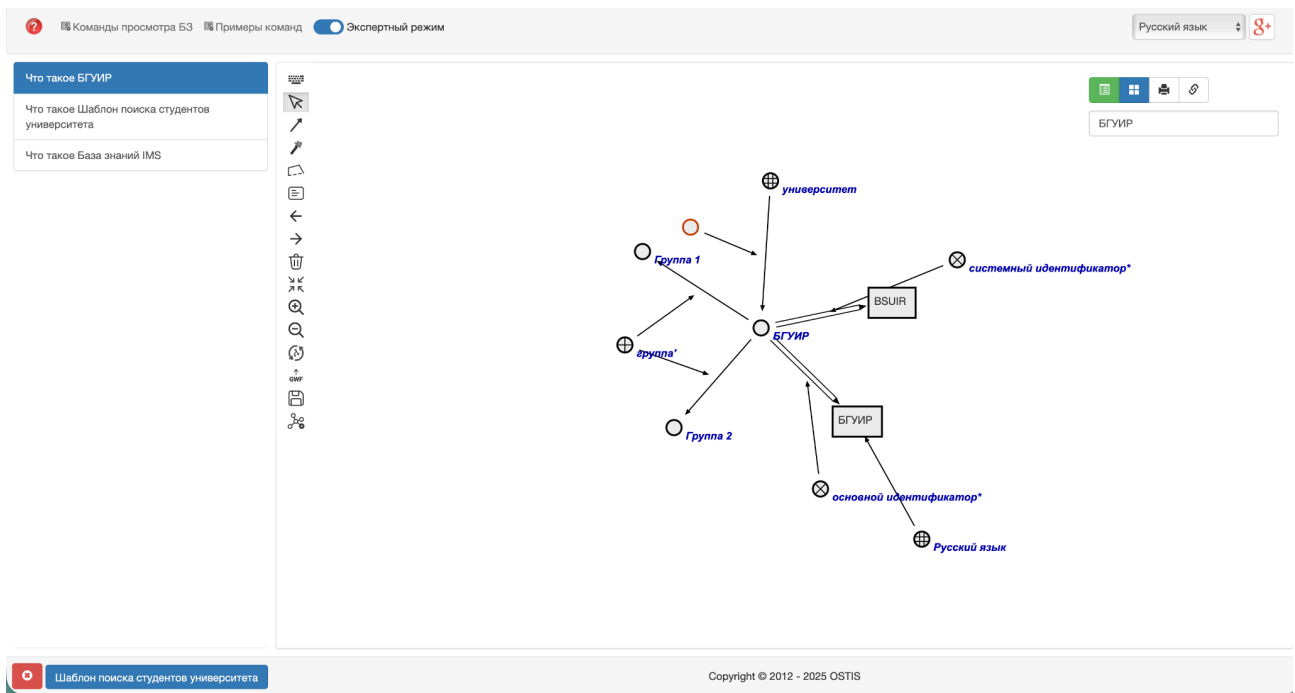
В результате на *панели снизу* будет следующее:



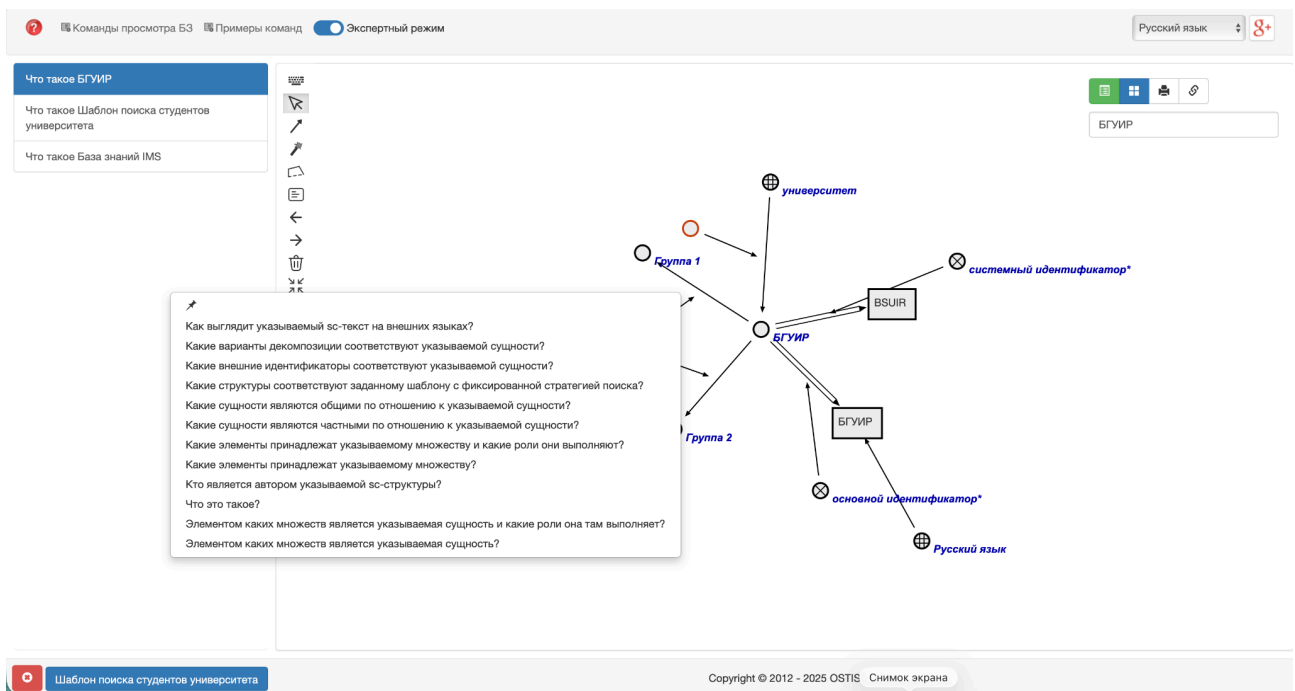
После при помощи *панели истории навигации по базе знаний ostis-системы* необходимо осуществить переход к семантической окрестности сущности *БГУИР*:



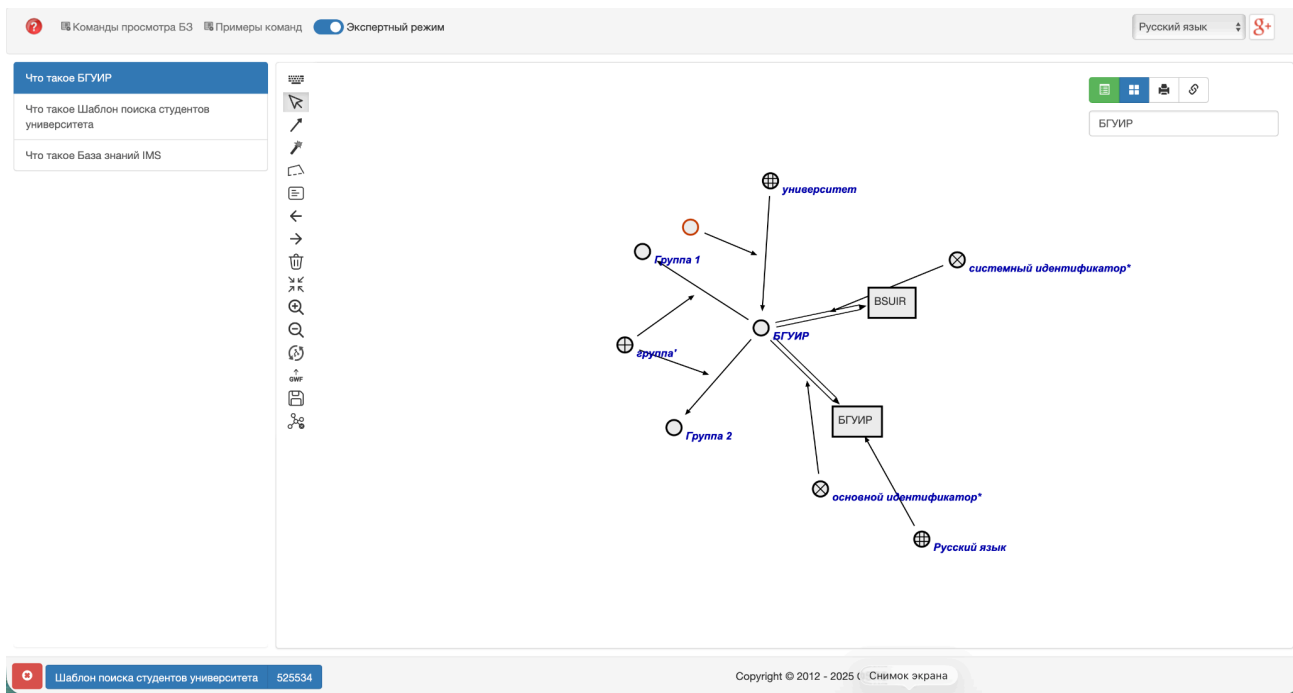
В результате перехода на *панели снизу* закреплённый ранее sc-элемент останется на месте:



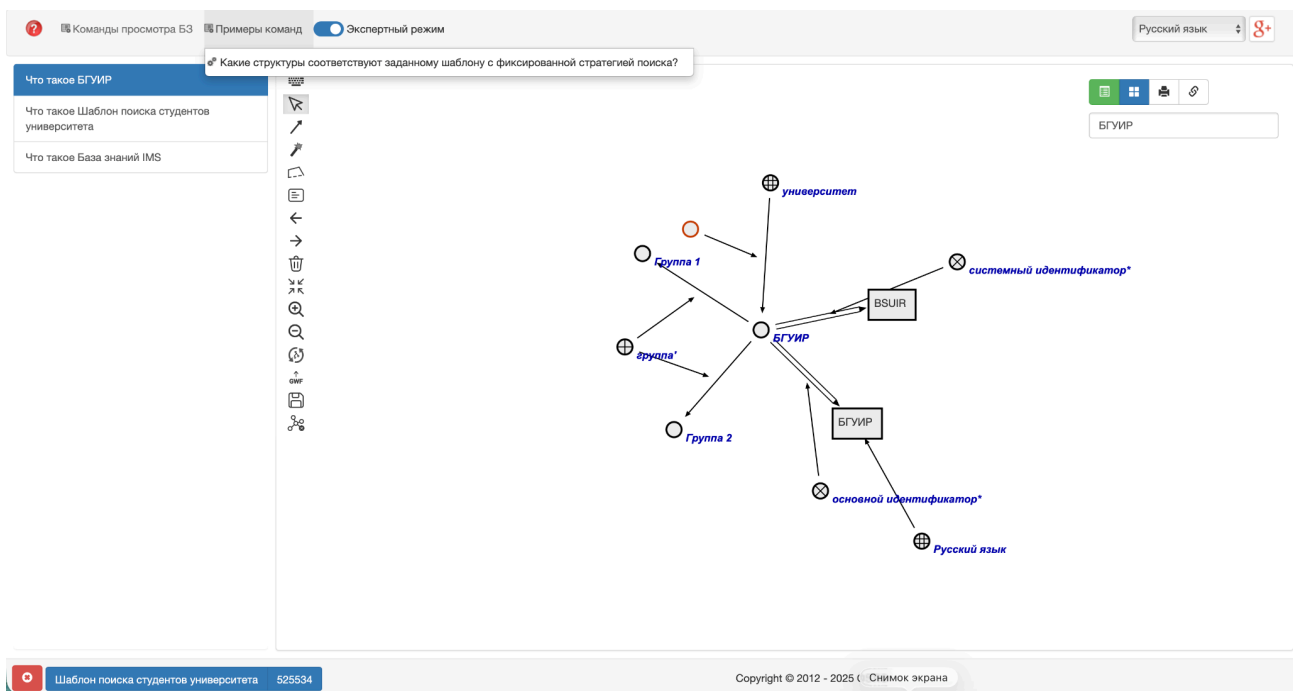
Далее для созданного ранее sc.g-узла необходимо вызвать *контекстное меню* и нажать на *кнопку закрепления sc-элемента на панели снизу*:



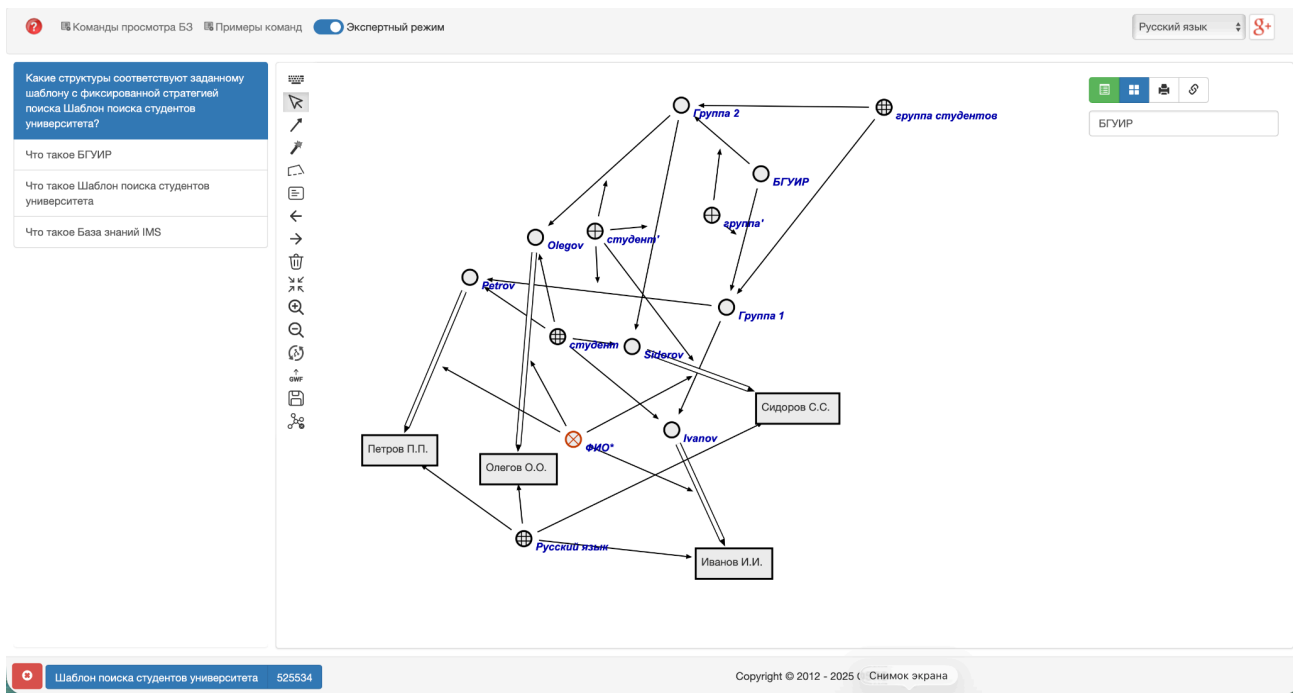
В результате на *панели снизу* будет следующее:



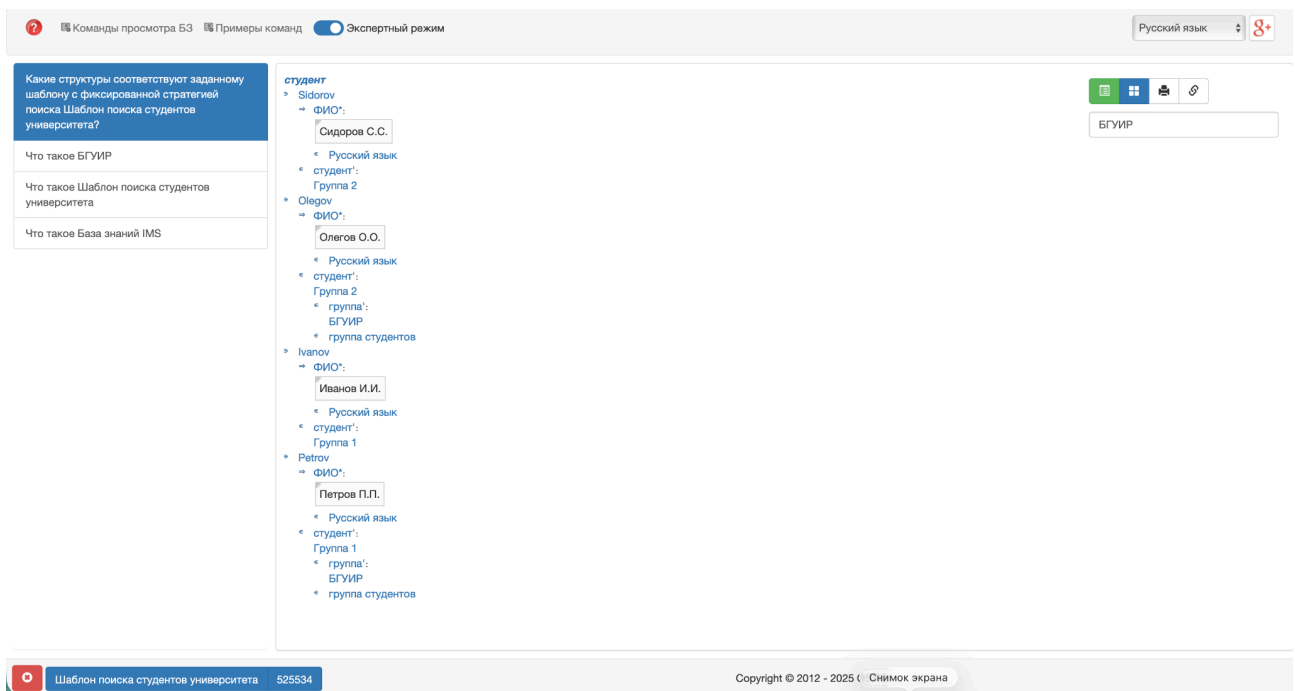
В конце необходимо на панели сверху из меню *Примеры команд* выбрать и активировать вопрос *Какие структуры соответствуют заданному шаблону с фиксированной стратегией поиска?*.



После поиска ответа на этот вопрос на *sc.g*-сцене появятся *sc.g*-конструкции, изоморфные заданному *sc*-шаблону с фиксированной стратегией поиска:



После перехода в режим навигации по базе знаний ostis-системы на SCn-коде sc-конструкции будут выглядеть следующим образом:



Поиск по другим sc-шаблонам с фиксированной стратегией поиска можно осуществлять аналогичным образом.

6.9.3 Принципы разработки решателя задач ostis-системы

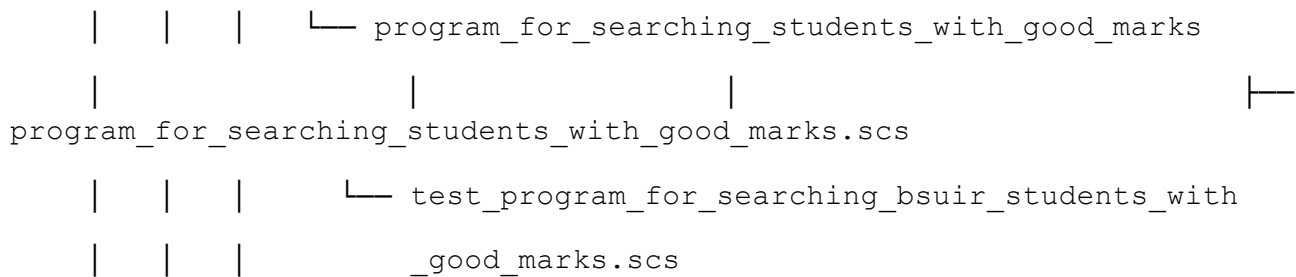
6.9.3.1 Принципы разработки scr-программ для заданной предметной области

Разработка scr-программы в рамках заданной предметной области должна происходить после формализации самой предметной области либо её отдельных фрагментов.

Под *разработкой scr-программы* понимается процесс создания исходного файла и формализация в рамках созданного файла текста этой scr-программы на SCg-коде или SCs-коде.

В качестве примера разработаем scr-программу в рамках предметной области. Структура директорий и исходников с формализацией фрагментов *Предметной области университетов* выглядит следующим образом:

```
knowledge-base/  
├─ examples/  
│   ├─ subject_domain_of_universities  
│   │   ├─ concepts  
│   │   │   ├─ classes  
│   │   │   │   ├─ concept_group.scs  
│   │   │   │   ├─ concept_student.scs  
│   │   │   │   └─ concept_university.scs  
│   │   │   └─ relations  
│   │   │       ├─ nrel_fio.scs  
│   │   │       ├─ nrel_average_mark.scs  
│   │   │       ├─ rrel_group.scs  
│   │   │       └─ rrel_student.scs  
│   │   └─ entities  
│   │       ├─ bsuir.scs  
│   │       └─ bsuir_students.scs  
│   └─ programs
```



6.9.3.1.1 Пример scp-программы для заданной предметной области

Для заданных фрагментов предметной области можно формализовать на SCs-коде следующую scp-программу:

```

// program_for_searching_students_with_good_marks.scs
// =====
// ПРОГРАММА: Программа поиска студентов с хорошими оценками
// =====
// НАЗНАЧЕНИЕ:
// Эта scp-программа ищет всех студентов в указанном университете,
// у которых средняя оценка >= 8.0, и собирает их в результирующее множество.
//
// ВХОДНЫЕ ПАРАМЕТРЫ:
// .._university - университет, в рамках которого ищутся студенты с хорошими оценками.
//
// ВЫХОДНЫЕ ПАРАМЕТРЫ:
// .._result_students - результирующая структура, содержащая всех найденных студентов со
//                      средней оценкой >= 8.0.
//
// АЛГОРИТМ:
// 1 Создать пустое результирующее множество студентов.
// 2 Проверить, что входной аргумент - это экземпляр класса университетов.
// 3 Найти все группы, принадлежащие указанному университету.
// 4 Для каждой группы пройти через всех студентов.
// 5 Для каждого студента получить его среднюю оценку.
// 6 Если средняя оценка >= 8.0, добавить студента в результирующее множество.
// 7 Очистить временные множества и вернуть результат.
//
// ИСПОЛЬЗУЕМЫЕ КЛЮЧЕВЫЕ SC-ЭЛЕМЕНТЫ:
// - concept_university: класс университетов
// - nrel_group: связывает университет с его группами
// - rrel_student: связывает группы со студентами
// - concept_student: класс студентов
// - nrel_average_mark: содержит среднюю оценку студента
//
// =====

program_for_searching_students_with_good_marks
=> nrel_main_idtf:
  [Программа поиска студентов заданного университета, которые имеют хорошие оценки]
  (*
    <- lang_ru;;
  *);
  [Program for searching students of specified university with good marks]
  (*
    <- lang_en;;
  *);
<- scp_program;
-> rrel_key_sc_element:
  .._process;;
  
```

```

program_for_searching_students_with_good_marks = [*
  ...process
  <-_ scp_process;

  // ОБЪЯВЛЕНИЕ ВХОДНЫХ И ВЫХОДНЫХ ПАРАМЕТРОВ:
  _-> rrel_1:: rrel_in:: ..._university; // университет, в рамках которого ищутся студенты с хорошими оценками.
  _-> rrel_2:: rrel_out:: ..._result_students; // результирующая структура студентов с хорошими оценками.

  <=_ nrel_decomposition_of_action:: ...
  (*
    // ШАГ 1: ИНИЦИАЛИЗАЦИЯ РЕЗУЛЬТИРУЮЩЕЙ СТРУКТУРЫ
    // Создать пустую структуру для добавления студентов.
    _-> rrel_1:: ..._generate_result_set_of_students_with_good_marks
    (*
      <-_ genElStr3;;

      _-> rrel_1:: rrel_assign:: rrel_scp_var:: rrel_const:: rrel_node:: rrel_structure:: ..._result_students;;
      _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: ..._arc_from_result_students_to_class;;
      _-> rrel_3:: rrel_fixed:: rrel_scp_const:: concept_student;;

      // Перейти к проверке входного аргумента.
      _=> nrel_goto:: ..._check_university;;
    *);

    // ШАГ 2: ПРОВЕРКА ВХОДНОГО АРГУМЕНТА
    // Проверить, что входной аргумент действительно является экземпляром класса университетов.
    _-> ..._check_university
    (*
      <-_ searchElStr3;;

      _-> rrel_1:: rrel_fixed:: rrel_scp_const:: concept_university;;
      _-> rrel_2:: rrel_assign:: rrel_const_perm_pos_arc:: rrel_scp_var:: ..._arc_from_class_to_university;;
      _-> rrel_3:: rrel_fixed:: rrel_scp_const:: ..._university;;

      // Если входной аргумент является экземпляром класса университетов,
      // то перейти к печати в терминал, что он является университетом.
      _=> nrel_then:: ..._print_that_specified_argument_is_university;;
      // Если входной аргумент не является экземпляром класса университетов,
      // то перейти к печати в терминал, что он не является университетом.
      _=> nrel_else:: ..._print_that_specified_argument_is_not_university;;
    *);

    // Напечатать в терминал, что входной аргумент является университетом.
    _-> ..._print_that_specified_argument_is_university
    (*
      <-_ printNl;;
      _-> rrel_1:: rrel_fixed:: rrel_scp_const:: [Specified argument is university];

      // Перейти к поиску всех групп в университете.
      _=> nrel_goto:: ..._search_university_groups;
    *);

    // ШАГ 3: ПОИСК ВСЕХ ГРУПП В УНИВЕРСИТЕТЕ
    // Найти все группы, принадлежащие университету, сформировать множество из них и записать в ..._found_groups.
    _-> ..._search_university_groups
    (*
      <-_ searchSetStr5;;

      _-> rrel_1:: rrel_fixed:: rrel_scp_const:: ..._university;;
      _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: ..._arc_from_university_to_group;;
      _-> rrel_3:: rrel_assign:: rrel_scp_var:: rrel_const:: rrel_node:: ..._group;;
      _-> rrel_4:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: ..._arc_to_arc_from_university_to_group;;
    *)
  ]

```

```

    _-> rrel_5:: rrel_fixed:: rrel_scp_const:: rrel_group;;

    _-> rrel_set_3:: rrel_assign:: rrel_scp_var:: .._found_groups;;

    // Если группы найдены, то перейти к выбору группы.
    _=> nrel_then:: ..._get_next_group;;
    // Если группы не найдены, то перейти к печати в терминал, что все группы были пройдены.
    _=> nrel_else:: ..._print_that_all_groups_were_iterated;;

*);;

// ШАГ 4: ВЫБРАТЬ ГРУППУ ИЗ МНОЖЕСТВА НАЙДЕННЫХ ГРУПП
// Найти любую группу во множестве найденных групп, являющегося значением переменной .._found_groups.
_-> ..._get_next_group
(*)
    <-_ searchElStr3;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: ..._found_groups;;
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: ..._arc_from_group_set_to_group;;
    _-> rrel_3:: rrel_assign:: rrel_scp_var:: ..._group;;

    // Если группы еще остались, то перейти к удалению выбранной группы из множества.
    _=> nrel_then:: ..._pop_group_from_set;;
    // Если все группы были пройдены, то перейти к печати в терминал, что все группы были пройдены.
    _=> nrel_else:: ..._print_that_all_groups_were_iterated;;

*);;

// ШАГ 5: УДАЛИТЬ ВЫБРАННУЮ ГРУППУ ИЗ МНОЖЕСТВА НАЙДЕННЫХ ГРУПП
// Удалить sc-дугу между множеством в .._found_groups и выбранной группой.
_-> ..._pop_group_from_set
(*)
    <-_ eraseEl;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: rrel_erase:: ..._arc_from_group_set_to_group;;

    // Перейти к печати в терминал, что группа успешно извлечена из множества.
    _=> nrel_goto:: ..._print_that_university_group_was_found;;

*);;

// Напечатать в терминал, что группа успешно извлечена из множества.
_-> ..._print_that_university_group_was_found
(*)
    <-_ printNl;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_const:: [Group was found in specified university];;

    // Перейти к поиску всех студентов в выбранной группе.
    _=> nrel_goto:: ..._search_group_students;;

*);;

// ШАГ 6: ПОИСК ВСЕХ СТУДЕНТОВ В ТЕКУЩЕЙ ГРУППЕ
// Найти всех студентов, обучающихся в текущей группе, сформировать множество из них и записать в
.._found_students.
_-> ..._search_group_students
(*)
    <-_ searchSetStr5;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: ..._group;;
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: ..._arc_from_group_to_student;;
    _-> rrel_3:: rrel_assign:: rrel_scp_var:: rrel_const:: rrel_node:: ..._student;;
    _-> rrel_4:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: ..._arc_to_arc_from_group_to_student;;
    _-> rrel_5:: rrel_fixed:: rrel_scp_const:: rrel_student;;

    _-> rrel_set_3:: rrel_assign:: rrel_scp_var:: .._found_students;;

```

```

        // Если студенты найдены, то перейти к выбору студента.
        _=> nrel_then:: ...get_next_student;;
        // Если студенты не найдены, то перейти к выбору следующей группы.
        _=> nrel_else:: ...get_next_group;;

*);;

// ШАГ 7: ВЫБРАТЬ СТУДЕНТА ИЗ МНОЖЕСТВА НАЙДЕННЫХ СТУДЕНТОВ
// Найти любого студента во множестве найденных студентов, являющегося значением переменной
...found_students.
_-> ...get_next_student
(*
    <-_ searchElStr3;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: ...found_students;;
    _-> rrel_2:: rrel_assign:: rrel_const_perm_pos_arc:: rrel_scp_var:: ...arc_from_student_set_to_student;;
    _-> rrel_3:: rrel_assign:: rrel_scp_var:: ...student;;

    // Если студенты еще остались, то перейти к удалению выбранного студента из множества.
    _=> nrel_then:: ...pop_student_from_set;;
    // Если все студенты были пройдены, то перейти к печати в терминал, что все студенты были пройдены.
    _=> nrel_else:: ...print_that_all_students_were_iterated;;

*);;

// ШАГ 8: УДАЛИТЬ ВЫБРАННОГО СТУДЕНТА ИЗ МНОЖЕСТВА НАЙДЕННЫХ СТУДЕНТОВ
// Удалить sc-дугу между множеством в ...found_students и выбранным студентом.
_-> ...pop_student_from_set
(*
    <-_ eraseEl;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: rrel_erase:: ...arc_from_student_set_to_student;;

    // Перейти к печати в терминал, что студент успешно извлечен из множества.
    _=> nrel_goto:: ...print_that_group_student_was_found;;

*);;

// Напечатать в терминал, что студент успешно извлечен из множества.
_-> ...print_that_group_student_was_found
(*
    <-_ printNl;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_const:: [Student was found in specified group];;

    // Перейти к поиску средней оценки студента.
    _=> nrel_goto:: ...search_student_average_mark;;

*);;

// ШАГ 9: ПОЛУЧИТЬ СРЕДНЮЮ ОЦЕНКУ СТУДЕНТА
// Найти связку отношения nrel_average_mark для текущего студента.
_-> ...search_student_average_mark
(*
    <-_ searchElStr5;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: ...student;;
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const:: rrel_common_arc:: ...arc_from_student_to_mark;;
    _-> rrel_3:: rrel_assign:: rrel_scp_var:: rrel_const:: rrel_node:: rrel_node_link:: ...average_mark;;
    _-> rrel_4:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: ...arc_to_arc_from_student_to_mark;;
    _-> rrel_5:: rrel_fixed:: rrel_scp_const:: nrel_average_mark;;

    // Если средняя оценка найдена, то перейти к печати в терминал, что средняя оценка найдена.
    _=> nrel_then:: ...print_that_average_mark_is_found;;
    // Если средняя оценка не найдена, то перейти к печати в терминал, что средняя оценка не найдена.
    _=> nrel_else:: ...print_that_average_mark_is_not_found;;

```

```

*);

// Напечатать в терминал, что средняя оценка найдена.
_> ...print_that_average_mark_is_found
(*
    <_ printNL;;

    _> rrel_1:: rrel_fixed:: rrel_scp_const:: [Average mark is found for specified student];

    // Перейти к проверке порога средней оценки.
    _=> nrel_goto:: ...check_student_average_mark;;
*);

// ШАГ 6: ПРОВЕРКА ПОРОГА СРЕДНЕЙ ОЦЕНКИ
// Проверить, что средняя оценка студента >= 8.0 (порог для "хороших оценок").
_> ...check_student_average_mark
(*
    <_ ifGr;;

    _> rrel_1:: rrel_fixed:: rrel_scp_var:: ...average_mark;;
    _> rrel_2:: rrel_fixed:: rrel_scp_const:: [8.0];

    // Перейти к проверке принадлежности студента к классу concept_student.
    _=> nrel_then:: ...check_student_class;;
    // Перейти к выбору следующего студента.
    _=> nrel_else:: ...get_next_student;;
*);

// ШАГ 11: ПРОВЕРКА ПРИНАДЛЕЖНОСТИ К КЛАССУ СТУДЕНТОВ
// Проверить, что студент принадлежит классу concept_student.
// Если студент принадлежит классу concept_student, то добавить sc-дугу базового вида между ними
// в результирующее множество.
_> ...check_student_class
(*
    <_ searchSetStr3;;

    _> rrel_1:: rrel_fixed:: rrel_scp_const:: concept_student;;
    _> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: ...arc_from_class_to_student;;
    _> rrel_3:: rrel_fixed:: rrel_scp_var:: ...student;;

    _> rrel_set_2:: rrel_fixed:: rrel_scp_var:: ...result_students;;

    // Перейти к добавлению студента в результирующее множество.
    _=> nrel_goto:: ...add_student_to_result_students_set;;
*);

// ШАГ 12: ДОБАВИТЬ СТУДЕНТА В РЕЗУЛЬТИРУЮЩЕЕ МНОЖЕСТВО
// Создать sc-дугу базового вида от структуры в ...result_students к текущему студенту.
_> ...add_student_to_result_students_set
(*
    <_ genElStr3;;

    _> rrel_1:: rrel_fixed:: rrel_scp_var:: ...result_students;;
    _> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: ...arc_from_result_students_set_to_student;;
    _> rrel_3:: rrel_fixed:: rrel_scp_var:: ...student;;

    // Перейти к печати в терминал, что студент успешно добавлен в результат.
    _=> nrel_goto:: ...print_that_student_was_added_to_result_students_set;;
*);

// Напечатать в терминал, что студент успешно добавлен в результат.
_> ...print_that_student_was_added_to_result_students_set
(*

```

```

        <-_ printNl;;

        _-> rrel_1:: rrel_fixed:: rrel_scp_const:: [Student was added to result students set];;

        // Перейти к выбору следующего студента.
        _=> nrel_goto:: ...get_next_student;;

    *);;

    // Напечатать в терминал, что средняя оценка не найдена.
    _-> ...print_that_average_mark_is_not_found
    (*
        <-_ printNl;;

        _-> rrel_1:: rrel_fixed:: rrel_scp_const:: [Average mark is not found for specified student];;

        // Перейти к выбору следующего студента.
        _=> nrel_goto:: ...get_next_student;;

    *);;

    // Напечатать в терминал, что все студенты в группе были обработаны.
    _-> ...print_that_all_students_were_iterated
    (*
        <-_ printNl;;

        _-> rrel_1:: rrel_fixed:: rrel_scp_const:: [All students were iterated in specified group];;

        // Перейти к удалению временного множества студентов.
        _=> nrel_goto:: ...erase_found_students_set;;

    *);;

    // Удалить временное множество студентов.
    _-> ...erase_found_students_set
    (*
        <-_ eraseEl;;

        _-> rrel_1:: rrel_fixed:: rrel_scp_var:: rrel_erase:: ...found_students;;

        // Перейти к выбору следующей группы.
        _=> nrel_goto:: ...get_next_group;;

    *);;

    // Напечатать в терминал, что все группы в университете были обработаны.
    _-> ...print_that_all_groups_were_iterated
    (*
        <-_ printNl;;

        _-> rrel_1:: rrel_fixed:: rrel_scp_const:: [All groups were iterated in specified university];;

        // Перейти к удалению временного множества групп.
        _=> nrel_goto:: ...erase_found_groups_set;;

    *);;

    // Удалить временное множество групп.
    _-> ...erase_found_groups_set
    (*
        <-_ eraseEl;;

        _-> rrel_1:: rrel_fixed:: rrel_scp_var:: rrel_erase:: ...found_groups;;

        // Перейти к завершению программы.
        _=> nrel_goto:: ...return_result;;

    *);;

```



```

// Напечатать в терминал, что указанный аргумент не является университетом.
_> ...print_that_specified_argument_is_not_university
(*
    <-_ printNl;;

    _> rrel_1:: rrel_fixed:: rrel_scp_const:: [Specified argument is not university];;

    // Перейти к завершению программы.
    _=> nrel_goto:: ...return_result;;

*);;

// Завершить программу.
_> ...return_result
(*
    <-_ return;;

*);;
*);;
*];;

```

Представленная программа реализует поиск студентов заданного университета, которые имеют хорошие оценки — среднюю оценку выше 8.0. Для проверки работоспособности разработанной программы следует разработать тестовые программы.

6.9.3.1.2 Пример тестовой scp-программы для scp-программы заданной предметной области

Пример тестовой программы для вышерассмотренной программы представлен ниже:

```

// test_program_for_searching_bsuir_students_with_good_marks.scs
// =====
// ТЕСТОВАЯ ПРОГРАММА
// =====
// НАЗНАЧЕНИЕ:
// Тестовая программа для проверки программы поиска студентов БГУИРа, которые имеют хорошие оценки.
//
// =====

test_program_for_searching_bsuir_students_with_good_marks
=> nrel_main_idtf:
[Тестовая программа поиска студентов БГУИРа, которые имеют хорошие оценки]
(*
    <- lang_ru;;
*);
[Test program for searching BSUIR students with good marks]
(*
    <- lang_en;;
*);
<- scp_program;
-> rrel_key_sc_element:
    ...process;;

test_program_for_searching_bsuir_students_with_good_marks = [*
    ...process
    <-_ scp_process;

```

```

<=_ nrel_decomposition_of_action:: ...
(*
  _-> rrel_1:: ...call_program_for_searching_bsuir_students_with_good_marks
  (*
    <=_ call;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_const:: program_for_searching_students_with_good_marks;;
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: ...params
    (*
      _-> rrel_1:: rrel_fixed:: rrel_scp_const:: BSUIR;;
      _-> rrel_2:: rrel_fixed:: rrel_scp_var:: ...result_students;;

    *);;
    _-> rrel_3:: rrel_assign:: rrel_scp_var:: ...descriptor;;

    _=> nrel_goto:: ...wait_for_result_of_program_for_searching_bsuir_students_with_good_marks;;
  *);;
  _-> ...wait_for_result_of_program_for_searching_bsuir_students_with_good_marks
  (*
    <=_ waitReturn;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: ...descriptor;;

    _=> nrel_goto:: ...print_result_students;;
  *);;
  _-> ...print_result_students
  (*
    <=_ printEl;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: ...result_students;;

    _=> nrel_goto:: ...return;;
  *);;
  _-> ...return
  (*
    <=_ return;;
  *);;
*);;
*);;
*);;

```

Представленные исходные файлы scp-программ находятся в указанных директориях проекта примера ostis-системы. Поэтому после установки примера ostis-системы они будут автоматически доступны для редактирования и дополнения. В качестве языка разработки scp-программы рекомендуется использовать SCs-код.

6.9.3.2.3 Принципы тестирования scp-программ для заданной предметной области

Процесс тестирования scp-программы заданной предметной области направлен на проверку соответствия их синтаксиса и семантики правилам разработки scp-программ.

Для проверки синтаксической корректности scp-программы требуется выполнить процесс *сборки всей базы знаний* с этой новой scp-программой.

Для проверки семантической корректности scr-программы требуется выполнить запуск тестовой scr-программы для заданной scr-программы.

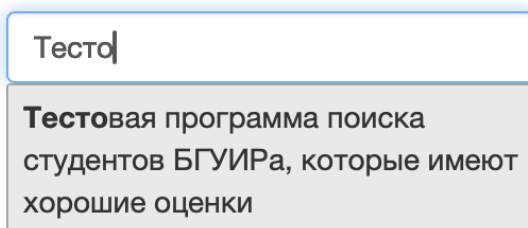
Для упрощения запуска тестовых scr-программ и получения их результата в примере ostis-системы добавлен компонент пользовательского интерфейса, реализующий команду *Запустить scr-программу*.

Чтобы запустить тестовую scr-программу следует:

- 1) Найти тестовую scr-программу.
- 2) Совершить щелчок правой кнопкой компьютерной мыши по найденной тестовой scr-программе и нажать в появившемся контекстном меню на кнопку “закрепить” — в результате sc-адрес найденного sc-узла будет закреплён на панели снизу.
- 3) В панели сверху щелчком левой кнопки компьютерной мыши нажать на меню *Примеры команд*; в появившемся списке щелчком левой кнопки компьютерной мыши нажать на элемент, обозначающий команду *Запустить scr-программу*.

Ниже рассматривается пример запуска тестовой scr-программы.

В качестве примера можно использовать *Тестовую программу поиска студентов БГУИРа, которые имеют хорошие оценки*, которая была разработана ранее. Данную scr-программу можно найти при помощи строки поиска, указав sc-идентификатор этой scr-программы. Процесс использования *строки поиска* показан на рисунке ниже:



Результатом поиска заданной scr-программы будет являться семантическая окрестность этой scr-программы, представленная на SCn-коде:

Тестовая программа поиска студентов БГУИРа, которые имеют хорошие оценки

⇒ основной идентификатор*:

Поиск...

Тестовая программа поиска студентов БГУИРа, которые имеют хорошие оценки

⇒ ключевой sc-элемент*:

...

€ #scp-программа

=

... ⇒ #следующий оператор*:

...

... ⇒ #следующий оператор*:

...

€ _ scp-оператор завершения выполнения программы

€ _ scp-оператор ожидания завершения выполнения scp-программы

€ _ ...

... ⇒ scp-операнд со свободным значением*::scp-переменная*::3*:

...

€ _ 1*::scp-операнд с заданным значением*::scp-переменная*::1*:

...

... ⇒ scp-операнд с заданным значением*::scp-константа*::2*:

...

... ⇒ scp-операнд с заданным значением*::scp-переменная*::2*:

...

... ⇒ scp-операнд с заданным значением*::scp-константа*::1*:

БГУИР

... ⇒ scp-операнд с заданным значением*::scp-константа*::1*:

Программа поиска студентов заданного университета, которые имеют хорошие оценки

€ _ 1*:

...

... ⇒ декомпозиция действия*:

...

€ _ scp-процесс

...

... ⇒ scp-оператор асинхронного вызова подпрограммы



Далее для данной scp-программы необходимо вызвать *контекстное меню* и нажать на *кнопку закрепления sc-элемента на панели снизу*:

Что такое Тестовая программа поиска студентов БГУИРа, которые имеют хорошие оценки

Что такое База знаний IMS

Тестовая программа поиска студентов БГУИРа, которые имеют хорошие оценки

⇒ основной идентификатор*:

Тестовая программа поиска студентов БГУИРа, которые имеют хорошие оценки

⇒ ключевой sc-элемент*:

...

€ #scp-программа

=

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

... ⇒ #следующий

- ✱
- Запустить scp-программу
- Как выглядит указываемый sc-текст на внешних языках?
- Какие варианты декомпозиции соответствуют указываемой сущности?
- Какие внешние идентификаторы соответствуют указываемой сущности?
- Какие структуры соответствуют заданному шаблону с фиксированной стратегией поиска?
- Какие сущности являются общими по отношению к указываемой сущности?
- Какие сущности являются частными по отношению к указываемой сущности?
- Какие элементы принадлежат указываемому множеству и какие роли они выполняют?
- Какие элементы принадлежат указываемому множеству?
- Кто является автором указываемой sc-структуры?
- Найти студентов заданного университета, которые имеют хорошие оценки
- Что это такое?
- Элементом каких множеств является указываемая сущность и какие роли она там выполняет?
- Элементом каких множеств является указываемая сущность?

В результате на *панели снизу* будет следующее:



Тестовая программа поиска студентов БГУИРа, которые имеют хорошие оценки

В конце необходимо на панели сверху из меню *Примеры команд* выбрать и активировать команду *Запустить scp-программу*.

?

Команды просмотра БЗ

Примеры команд

Экспертный режим

Что такое Тестовая программа поиска студентов БГУИРа, которые имеют хорошие оценки

Что такое База знаний IMS

Какие структуры соответствуют заданному шаблону с фиксированной стратегией поиска?

Запустить scr-программу

Найти студентов заданного университета, которые имеют хорошие оценки

Тестовая программа поиска студентов БГУИРа, которые имеют хорошие оценки

ключевой sc-элемент':

...

€ #scr-программа

=

_ => #следующий оператор*::

...

_ => #следующий оператор*::

...

€ _ scr-оператор завершения выполнения программы

€ _ scr-оператор ожидания завершения выполнения scr-программы

€ _ ...

_ => scr-операнд со свободным значением':scr-переменная':3'::

...

€ _ 1'::scr-операнд с заданным значением':scr-переменная':

...

_ => scr-операнд с заданным значением':scr-константа':2'::

...

_ => scr-операнд с заданным значением':scr-переменная':2'::

...

_ => scr-операнд с заданным значением':scr-константа':1'::

БГУИР

_ => scr-операнд с заданным значением':scr-константа':1'::

Программа поиска студентов заданного университета, которые имеют хорошие

€ _ 1'::

...

_ => декомпозиция действия*::

...

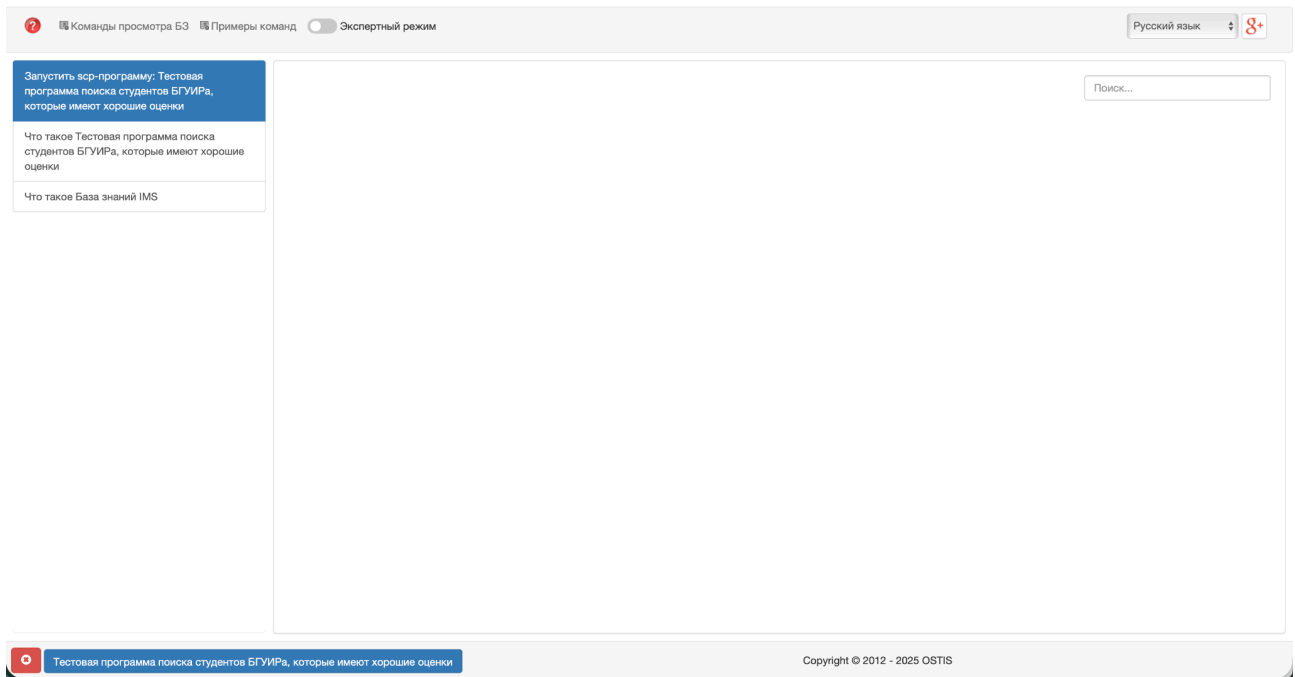
€ _ scr-процесс

_ => ...

€ _ scr-оператор асинхронного вызова подпрограммы

Тестовая программа поиска студентов БГУИРа, которые имеют хорошие оценки

После активации команды главный экран будет выглядеть следующим образом:



Текущая версия пользовательского интерфейса примера ostis-системы не позволяет отображать результаты выполнения scr-программ. Для получения результата необходимо найти результат и отобразить его на экране. Также можно посмотреть логи интерпретации тестовой scr-программы в терминале запущенной sc-машины.

Для заданной тестовой scr-программы логи будут выглядеть следующим образом:

```
call
====Called scr-program: program_for_searching_students_with_good_marks
waitReturn
genElStr3
searchElStr3
println
Specified argument is university
searchSetStr5
searchElStr3
eraseEl
println
Group was found in specified university
searchSetStr5
searchElStr3
eraseEl
println
Student was found in specified group
searchElStr5
println
Average mark is found for specified student
ifGr
```

```
SCPOperatorIfGr execute(): start
Input is a double
double: 9.2
Input is a double
double: 8.0
Input is a double
double: 9.2
Input is a double
double: 8.0
Link is Double
searchSetStr3
genElStr3
println
Student was added to result students set
searchElStr3
eraseEl
println
Student was found in specified group
searchElStr5
println
Average mark is found for specified student
ifGr
SCPOperatorIfGr execute(): start
Input is a double
double: 5.1
Input is a double
double: 8.0
Input is a double
double: 5.1
Input is a double
double: 8.0
Link is Double
searchElStr3
println
All students were iterated in specified group
eraseEl
searchElStr3
eraseEl
println
Group was found in specified university
searchSetStr5
searchElStr3
eraseEl
println
Student was found in specified group
searchElStr5
println
Average mark is found for specified student
ifGr
SCPOperatorIfGr execute(): start
Input is a double
double: 8.1
Input is a double
```

```

double: 8.0
Input is a double
double: 8.1
Input is a double
double: 8.0
Link is Double
searchSetStr3
genElStr3
printNl
Student was added to result students set
searchElStr3
eraseEl
printNl
Student was found in specified group
searchElStr5
printNl
Average mark is found for specified student
ifGr
SCPOperatorIfGr execute(): start
Input is a double
double: 7.1
Input is a double
double: 8.0
Input is a double
double: 7.1
Input is a double
double: 8.0
Link is Double
searchElStr3
printNl
All students were iterated in specified group
eraseEl
searchElStr3
printNl
All groups were iterated in specified university
eraseEl
return
printEl
10|4
Input arcs:
    10|5 <- 8|1510
Total input arcs: 1
Output arcs:
    10|534 -> Olegov
    10|514 -> (concept_student->Olegov)
    11|34 -> Petrov
    11|14 -> (concept_student->Petrov)
    10|6 -> concept_student
Total output arcs: 5
return

```

Видно, что в результате были найдены студенты Олегов и Петров.

Запуск других scr-программ можно осуществлять аналогичным образом.

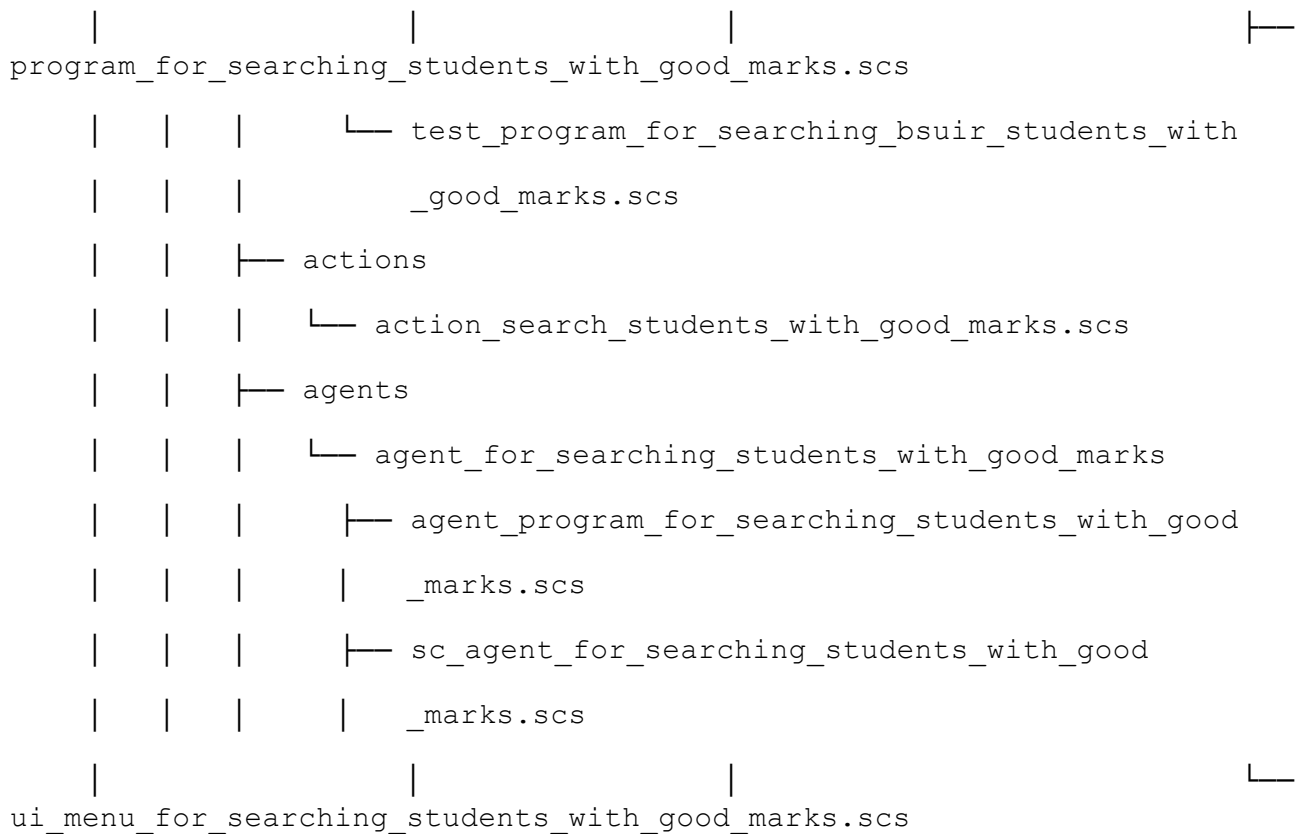
Проблему отображения результата выполнения scr-программы можно решить, если оформить эту scr-программу как sc-агент.

3 Принципы разработки sc-агентов в ostis-системах

Разработка sc-агента в рамках заданной предметной области должна происходить после формализации самой предметной области либо её отдельных фрагментов, а также после разработки scr-программ, необходимых для работы sc-агента.

В качестве примера разработаем sc-агент в рамках предметной области. Структура директорий и исходников с формализацией фрагментов *Предметной области университетов* выглядит следующим образом:

```
knowledge-base/
├─ examples/
|   └─ subject_domain_of_universities
|       └─ concepts
|           └─ classes
|               └─ concept_group.scs
|               └─ concept_student.scs
|               └─ concept_university.scs
|           └─ relations
|               └─ nrel_fio.scs
|               └─ nrel_average_mark.scs
|               └─ rrel_group.scs
|               └─ rrel_student.scs
|       └─ entities
|           └─ bsuir.scs
|           └─ bsuir_students.scs
|       └─ programs
|           └─ program_for_searching_students_with_good_marks
```



Под *разработкой sc-агента* понимается процесс разработки:

- scr-программ, необходимых для работы данного sc-агента;
- его агентной scr-программы;
- спецификации соответствующего ему абстрактного sc-агента;
- и компонента пользовательского интерфейса для вызова данного sc-агента.

В качестве примера дополним *Программу поиска студентов заданного университета, которые имеют хорошие оценки до sc-агента поиска студентов заданного университета, которые имеют хорошие оценки.*

6.9.3.2 Принципы разработки sc-агентов

6.9.3.2.1 Пример агентной scr-программы

Любую scr-программу можно представить в виде sc-агента. Для этого необходимо разработать для неё агентную scr-программу, которая будет вызывать заданную scr-программу.

В качестве примера создадим агентную scr-программу для вызова scr-программы *program_for_searching_students_with_good_marks*.

Ниже представлен пример агентной scp-программы для scp-программы *program_for_searching_students_with_good_marks*:

```
agent_program_for_searching_students_with_good_marks
=> nrel_main_idtf:
  [Агентная программа поиска студентов заданного университета, которые имеют хорошие оценки]
  (*
    <- lang_ru;;
  *);
  [Agent program for searching students of specified university with good marks]
  (*
    <- lang_en;;
  *);
  <- scp_program;
  <- agent_scp_program;
  -> rrel_key_sc_element:
    .._process;;

agent_program_for_searching_students_with_good_marks = [*
  .._process
  <- _ scp_process;

  _-> rrel_1:: rrel_in:: .._event;
  _-> rrel_2:: rrel_in:: ..incoming_arc;

  <=_ nrel_decomposition_of_action:: ...
  (*
    _-> rrel_1:: .._get_action
    (*
      <- _ searchElStr3;;

      _-> rrel_1:: rrel_assign:: rrel_scp_var:: ..subscription_element;;
      _-> rrel_2:: rrel_fixed:: rrel_scp_const:: ..incoming_arc;;
      _-> rrel_3:: rrel_assign:: rrel_scp_var:: ..action;;

      _=> nrel_goto:: .._get_action_argument;;
    *);
    _-> .._get_action_argument
    (*
      <- _ searchElStr5;;

      _-> rrel_1:: rrel_fixed:: rrel_scp_var:: ..action;;
      _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: ..arc1;;
      _-> rrel_3:: rrel_assign:: rrel_scp_var:: rrel_const:: rrel_node:: ..university;;
      _-> rrel_4:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: ..arc2;;
      _-> rrel_5:: rrel_fixed:: rrel_scp_const:: rrel_1;;

      _=> nrel_then:: ..call_program_for_searching_students_with_good_marks;;
      _=> nrel_else:: ..finish_action_unsuccessfully;;
    *);
    _-> ..call_program_for_searching_students_with_good_marks
    (*
      <- _ call;;

      _-> rrel_1:: rrel_fixed:: rrel_scp_const:: program_for_searching_students_with_good_marks;;
      _-> rrel_2:: rrel_fixed:: rrel_scp_const:: ..params
    (*
      _-> rrel_1:: rrel_fixed:: rrel_scp_var:: ..university;;
      _-> rrel_2:: rrel_fixed:: rrel_scp_var:: ..result_students;;
```

```

*);
-> rrel_3:: rrel_assign:: rrel_scp_var:: ...descriptor;;

_=> nrel_goto:: ...wait_for_result;;

*);
-> ...wait_for_result
(*
    <-_ waitReturn;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: ...descriptor;;

    _=> nrel_goto:: ...print_result_students;;

*);
-> ...print_result_students
(*
    <-_ printEl;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: ...result_students;;

    _=> nrel_goto:: ...set_result;;

*);
-> ...set_result
(*
    <-_ genElStr5;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: ...action;;
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const:: rrel_common_arc:: ...arc1;;
    _-> rrel_3:: rrel_fixed:: rrel_scp_var:: ...result_students;;
    _-> rrel_4:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: ...arc2;;
    _-> rrel_5:: rrel_fixed:: rrel_scp_const:: nrel_result;;

    _=> nrel_goto:: ...finish_action_successfully;;

*);
-> ...finish_action_successfully
(*
    <-_ genElStr3;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_const:: action_finished_successfully;;
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: ...arc1;;
    _-> rrel_3:: rrel_fixed:: rrel_scp_var:: ...action;;

    _=> nrel_goto:: ...finish_action;;

*);
-> ...finish_action
(*
    <-_ genElStr3;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_const:: action_finished;;
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: ...arc1;;
    _-> rrel_3:: rrel_fixed:: rrel_scp_var:: ...action;;

    _=> nrel_goto:: ...return;;

*);
-> ...finish_action_unsuccessfully
(*
    <-_ genElStr3;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_const:: action_finished_unsuccessfully;;
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: ...arc1;;
    _-> rrel_3:: rrel_fixed:: rrel_scp_var:: ...action;;

    _=> nrel_goto:: ...finish_action;;

*);

```

```

_> ...return
(*
    <-_ return;;
*);
*);
*];

```

6.9.3.2.2 Пример спецификации абстрактного sc-агента

Спецификация абстрактного sc-агента необходимо для явного указания:

- класса события в sc-памяти, на который должен реагировать разрабатываемый sc-агента,
- sc-элемента, для которого должно происходить событие, чтобы агент среагировал на него;
- класса действий, которые агент будет выполнять;
- полное условие и инициирования агента

Ниже представлен пример спецификации sc-агента поиска студентов заданного университета, которые имеют хорошие оценки:

```

sc_agent_for_searching_students_with_good_marks
=> nrel_main_idtf:
    [sc-агент поиска студентов заданного университета, которые имеют хорошие оценки]
    (*
        <- lang_ru;;
    *);
    [sc-agent for searching students of specified university with good marks]
    (*
        <- lang_en;;
    *);
<- abstract_sc_agent;
// Агент активируется при возникновении события создания выходящей дуги из класса action_initiated.
=> nrel_primary_initiation_condition:
    (sc_event_after_generate_outgoing_arc => action_initiated);
=> nrel_initiation_condition_and_result:
    (..sc_agent_for_searching_students_with_good_marks_condition
        => ..sc_agent_for_searching_students_with_good_marks_result);
=> nrel_sc_agent_action_class:
    action_search_students_with_good_marks;
<= nrel_sc_agent_key_sc_elements:
{
    action_initiated;
    action_search_students_with_good_marks
};
=> nrel_inclusion: ...
    (*
        // Агент реализован на Языке SCP.
        <- platform_independent_abstract_sc_agent;;
        // Перечень программ sc-агента, включая агентную scp-программу.
        <= nrel_sc_agent_program:
        {
            agent_program_for_searching_students_with_good_marks;
            program_for_searching_students_with_good_marks
        };
    *)

```

```

-> scp_agent_for_searching_students_with_good_marks
(*
    <- active_sc_agent;;
*);,

// Агент начинает выполнять действие, если оно является
// экземпляром класса action_search_students_with_good_marks
// и имеет в качестве первого аргумента университет.
..sc_agent_for_searching_students_with_good_marks_condition
= [*
    action_search_students_with_good_marks _-> .._action;;
    action_initiated _-> .._action;;
    .._action _-> rrel_1:: .._university;;
*];,

// Агент завершает выполнять действие успешно, если он нашел студентов с хорошими оценками.
..sc_agent_for_searching_students_with_good_marks_result
= [*
    concept_student _-> .._student;;
*];,

```

Формализация спецификации для платформенно-независимых агентов является обязательным шагом их разработки.

6.9.3.2.3 Пример компонента пользовательского интерфейса для вызова *sc*-агента

Для любого агента можно разработать компонент пользовательского интерфейса, с помощью которого можно осуществлять вызов этого агента.

Ниже представлен компонент пользовательского интерфейса для вызова *sc*-агента *поиска студентов заданного университета*, которые имеют хорошие оценки:

```

ui_menu_for_searching_students_with_good_marks
<- ui_user_command_class_atom;
<- ui_user_command_class_view_kb;
=> nrel_main_idtf:
    [Найти студентов заданного университета, которые имеют хорошие оценки]
    (*
        <- lang_ru;;
    *);
    [Search students of specified university with good marks]
    (*
        <- lang_en;;
    *);
=> ui_nrel_command_template:
    [*
        action_search_students_with_good_marks _-> .._action
        (*
            _-> rrel_1:: ui_arg_1;;

```

```

        *);;
        .._action <- _action;;
    *];
=> ui_nrel_command_lang_template:
    [Найти студентов $ui_arg_1, которые имеют хорошие оценки]
    (*
        <- lang_ru;;
    *);
    [Search students of $ui_arg_1 with good marks]
    (*
        <- lang_en;;
    *);;

```

6.9.3.2.4 Добавление компонента пользовательского интерфейса в меню пользовательского интерфейса

После создания компонента пользовательского интерфейса для вызова sc-агента необходимо добавить его в меню.

Созданный компонент можно добавить в меню *Примеры команд*. Исходный текст меню *Примеры команд* находится в *knowledge-base/ui_menu/ui_menu_na_example_commands.scs*.

После обновления этот исходный файл должен выглядеть следующим образом:

```

ui_menu_na_example_commands
<- ui_user_command_class_noatom;
=> nrel_main_idtf:
    [Примеры команд]
    (*
        <- lang_ru;;
    *);
    [Command examples]
    (*
        <- lang_en;;
    *);
<= nrel_ui_commands_decomposition:
{
    ui_menu_fixed_search_strategy_template_search;
    ui_menu_run_scp_program;
    ui_menu_for_searching_students_with_good_marks
};;

```

6.9.3.2.4 Принципы тестирования sc-агентов в ostis-системах

Процесс тестирования sc-агента для заданной предметной области направлен на проверку соответствия их синтаксиса и семантики правилам разработки sc-агентов.

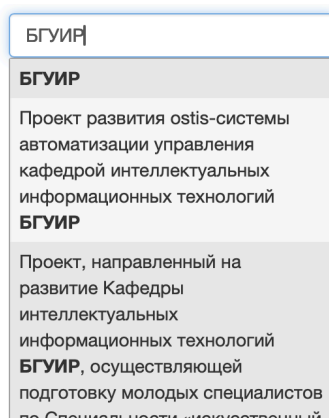
Для проверки синтаксической корректности sc-агента требуется выполнить процесс *сборки всей базы знаний* с этим новым sc-агентом.

Для проверки семантической корректности sc-агента требуется выполнить запуск sc-агента.

Чтобы запустить *sc-агент поиска студентов заданного университета* следует:

- 1) Найти университет БГУИР.
- 2) Совершить щелчок правой кнопкой компьютерной мыши по найденному университету и нажать в появившемся контекстном меню на кнопку “закрепить” — в результате sc-адрес найденного sc-узла будет закреплён на панели снизу.
- 3) В панели сверху щелчком левой кнопки компьютерной мыши нажать на меню *Примеры команд*; в появившемся списке щелчком левой кнопки компьютерной мыши нажать на элемент, обозначающий команду *Найти студентов заданного университета, которые имеют хорошие оценки*.

Университет БГУИР можно найти по его основному sc-идентификатору, используя *строку поиска*. Процесс использования *строки поиска* показан на рисунке ниже:



Результатом поиска заданной сущности будет являться семантическая окрестность этой сущности, представленная на SCn-коде:

БГУИР

⇒ основной идентификатор*:

БГУИР

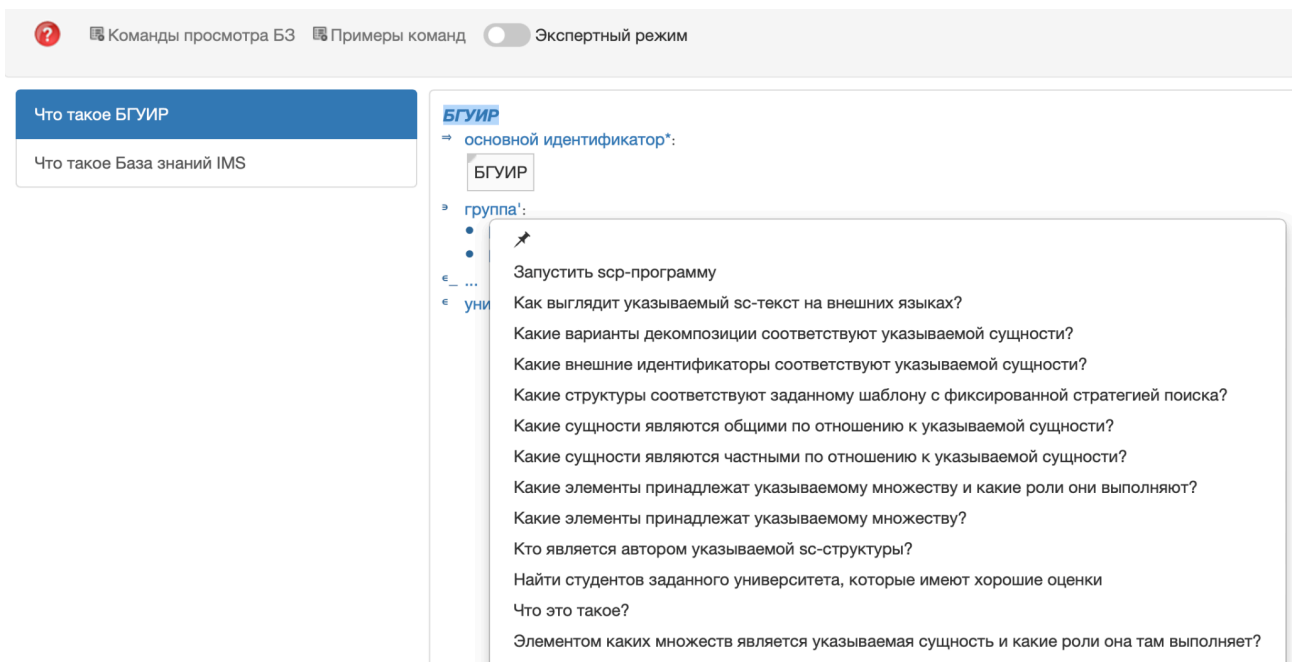
⇒ группа':

- Группа 2
- Группа 1

€ _ ...

€ университет

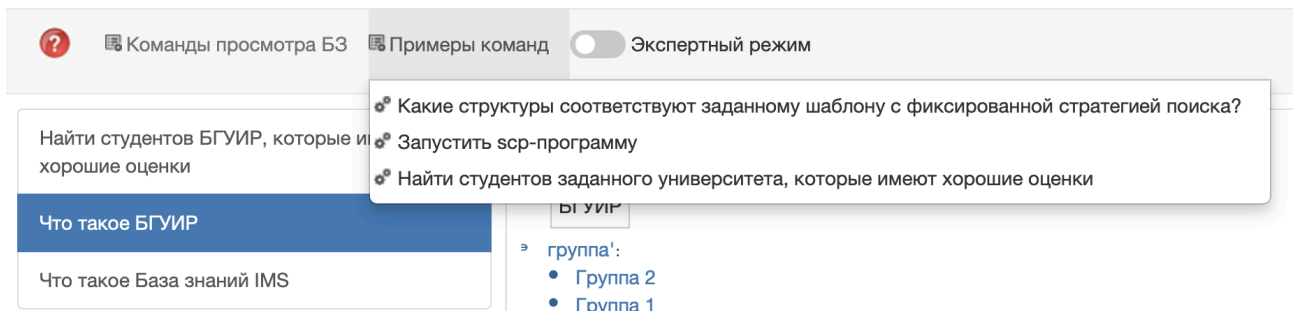
Далее для найденной сущности необходимо вызвать *контекстное меню* и нажать на *кнопку закрепления sc-элемента на панели снизу*:



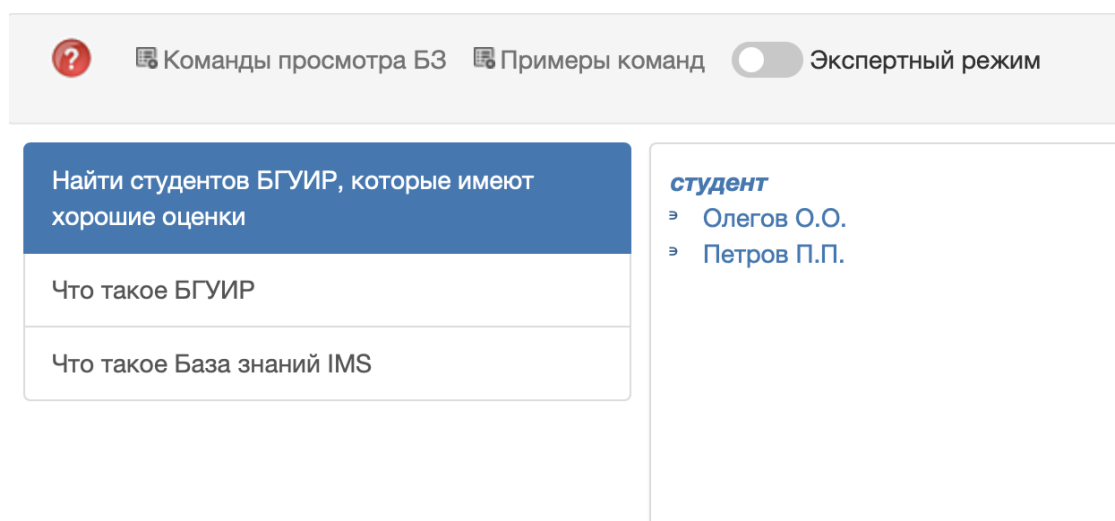
В результате на *панели снизу* будет следующее:



В конце необходимо на панели сверху из меню *Примеры команд* выбрать и активировать команду *Найти студентов заданного университета, которые имеют хорошие оценки*.



После активации команды главный экран будет выглядеть следующим образом:



Запуск других sc-агентов можно осуществлять аналогичным образом.

ЗАКЛЮЧЕНИЕ

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ